TogetherWeGo: complete source code

FBLA Mobile Application Development 2026-27, “Together We Go: Group Trip Planner”. Expo SDK 57 / React Native / TypeScript. 115 files. Photos and fonts are listed in ATTRIBUTIONS.md and are not printed.

1. .claude/settings.json
2. .vscode/extensions.json
3. .vscode/settings.json
4. AGENTS.md
5. ATTRIBUTIONS.md
6. README.md
7. app.json
8. eas.json
9. eslint.config.js
10. package.json
11. tsconfig.json
12. src/app/(tabs)/_layout.tsx
13. src/app/(tabs)/budget.tsx
14. src/app/(tabs)/chat.tsx
15. src/app/(tabs)/explore.tsx
16. src/app/(tabs)/itinerary.tsx
17. src/app/(tabs)/profile.tsx
18. src/app/_layout.tsx
19. src/app/about.tsx
20. src/app/airport.tsx
21. src/app/bucket-list.tsx
22. src/app/crew.tsx
23. src/app/currency.tsx
24. src/app/destination/[id].tsx
25. src/app/destinations.tsx
26. src/app/documents.tsx
27. src/app/emergency.tsx
28. src/app/flights.tsx
29. src/app/forgot-password.tsx
30. src/app/help.tsx
31. src/app/how-it-works.tsx
32. src/app/index.tsx
33. src/app/join.tsx
34. src/app/map.tsx
35. src/app/nearby.tsx
36. src/app/new-trip.tsx
37. src/app/notifications.tsx
38. src/app/onboarding.tsx
39. src/app/packing.tsx
40. src/app/panorama.tsx
41. src/app/personalize.tsx
42. src/app/photos.tsx
43. src/app/phrasebook.tsx
44. src/app/quiz.tsx
45. src/app/recap.tsx
46. src/app/settings.tsx
47. src/app/sign-in.tsx
48. src/app/sign-up.tsx
49. src/app/survey.tsx

50. src/app/trips.tsx
51. src/app/weather.tsx
52. src/components/CurrencyConverter.tsx
53. src/components/CurrencyPickerSheet.tsx
54. src/components/GuidedTour.tsx
55. src/components/Logo.tsx
56. src/components/ReminderService.tsx
57. src/components/ShareSheet.tsx
58. src/components/SkylineHero.tsx
59. src/components/TabBar.tsx
60. src/components/explore.tsx
61. src/components/map/OfflineRadar.tsx
62. src/components/map/TripMap.tsx
63. src/components/map/TripMap.web.tsx
64. src/components/map/types.ts
65. src/components/panorama/Scene.tsx
66. src/components/panorama/shapes.ts
67. src/components/pickers.tsx
68. src/components/trip/ActivityCard.tsx
69. src/components/trip/AddActivitySheet.tsx
70. src/components/trip/MemberPickerSheet.tsx
71. src/components/trip/NoTrip.tsx
72. src/components/trip/TripMenuSheet.tsx
73. src/components/trip/meta.ts
74. src/components/ui/Button.tsx
75. src/components/ui/Donut.tsx
76. src/components/ui/Input.tsx
77. src/components/ui/Press.tsx
78. src/components/ui/Screen.tsx
79. src/components/ui/Segmented.tsx
80. src/components/ui/Sheet.tsx
81. src/components/ui/Slider.tsx
82. src/components/ui/T.tsx
83. src/components/ui/bits.tsx
84. src/components/ui/feedback.tsx
85. src/components/ui/motion.tsx
86. src/data/airports.ts
87. src/data/destinations.ts
88. src/data/images.ts
89. src/data/panoramas.ts
90. src/data/personal.ts
91. src/data/places.ts
92. src/data/reference.ts
93. src/hooks/useTrip.ts
94. src/i18n/index.ts
95. src/i18n/strings.ts
96. src/services/assistant.ts
97. src/services/currency.ts
98. src/services/files.ts
99. src/services/location.ts
100. src/services/notifications.ts
101. src/services/print.ts
102. src/services/secrets.ts
103. src/services/share.ts
104. src/services/weather.ts

105. src/store/seed.ts
106. src/store/types.ts
107. src/store/useAppStore.ts
108. src/theme/index.ts
109. src/utils/format.ts
110. src/utils/geo.ts
111. src/utils/time.ts
112. src/utils/validation.ts
113. docs/DEMO_SCRIPT.md
114. docs/PLANNING.md
115. docs/RATING_SHEET.md

.claude/settings.json

```
{
  "enabledPlugins": {
    "expo@claude-plugins-official": true
  }
}
```

.vscode/extensions.json

```
{ "recommendations": ["expo.vscode-expo-tools"] }
```

.vscode/settings.json

```
{
  "editor.codeActionsOnSave": {
    "source.fixAll": "explicit",
    "source.organizeImports": "explicit",
    "source.sortMembers": "explicit"
  }
}
```

AGENTS.md

````
This is an Expo/React Native mobile application. Prioritize mobile-first patterns, performance, and cross-platform compatibility.

## Expo has changed — do not trust your training data

Expo ships breaking changes every SDK release. APIs you remember are likely renamed, moved, or removed. Before writing any code that tou
ches an Expo, EAS, or React Native API:

1. Read the major version of the `expo` package in `package.json`.
2. Fetch the matching versioned docs: `https://docs.expo.dev/versions/v<major>.0.0/`
3. For anything else, fetch https://docs.expo.dev/llms.txt — an index of all Expo docs with corrections to common LLM misconceptions. Fo
llow its links to the specific page you need; never answer from memory.

## Commands

Use `bunx` instead of `npx` if the project uses bun (`bun.lock` present).

```bash
npx expo install <package>  # ALWAYS use instead of npm/yarn/pnpm/bun add — resolves SDK-compatible versions
npx expo start              # start the dev server
npx expo lint               # lint
npx tsc --noEmit            # typecheck
npx expo-doctor             # diagnose dependency and config issues
npx expo install --fix      # fix incompatible package versions
```

Run lint and typecheck before declaring any task done.

## Navigation & Routing

- Use **Expo Router** for all navigation. Routes live in `src/app/` — every file there is a screen, `_layout.tsx` files define navigator
s. Keep non-route code (components, hooks, utils) outside `src/app/`.
- Import `Link`, `router`, and `useLocalSearchParams` from `expo-router`.
- Docs: https://docs.expo.dev/router/introduction.md

## Building with EAS

Use EAS to build, sign, and submit the app in the cloud (`eas build`, `eas submit`) and to ship over-the-air updates (`eas update`) — no
local Xcode or Android Studio required. Run EAS CLI as `bunx eas-cli <command>` in Bun projects, or `npx eas-cli@latest <command>` other
wise; substitute that for bare `eas` in docs examples.
Docs: https://docs.expo.dev/eas/index.md

## Rules

- If `ios/` and `android/` directories do not exist, they are generated (Continuous Native Generation). Never create or edit them by han
d — configure native behavior in `app.json` and config plugins.
- Expo Go only includes its bundled native modules. After adding a library with native code, the app needs a development build: `npx exp
o run:ios|android` locally, or `eas build --profile development`.
- Prefer recommended Expo modules over third-party libraries, and check your available skills before adding dependencies. Docs: https://
docs.expo.dev/versions/latest/index.md
````

ATTRIBUTIONS.md

```
# Attributions, sources & licences

TogetherWeGo was written for the FBLA Mobile Application Development event (2026–27). This file documents every third-party library, dat
a source, font, icon set and image used, as required by the rating sheet’s *Documentation and copyright compliance* item.

## Starter template
The project was scaffolded with `npx create-expo-app` (default template, MIT licence — see `LICENSE`, © 650 Industries / Expo). The temp
late’s example screens were removed; all screens, components, data and services in `src/` are original work for this project.

## Libraries (all open source)
| Package | Version | Licence |
|---|---|---|
| `expo` (SDK 57) and Expo modules: `expo-router`, `expo-notifications`, `expo-location`, `expo-image`, `expo-image-picker`, `expo-docum
ent-picker`, `expo-file-system`, `expo-print`, `expo-sharing`, `expo-speech`, `expo-crypto`, `expo-secure-store`, `expo-clipboard`, `exp
o-haptics`, `expo-linear-gradient`, `expo-linking`, `expo-localization`, `expo-asset`, `expo-font`, `expo-splash-screen`, `expo-status-b
ar`, `expo-system-ui`, `expo-constants` | 57.x | MIT |
| `react`, `react-dom` | 19.2.3 | MIT |
| `react-native` | 0.86.3 | MIT |
| `react-native-web` | 0.21.3 | MIT |
| `react-native-maps` | 1.27.2 | MIT |
| `react-native-svg` (spending donut chart) | 15.x | MIT |
| `react-native-safe-area-context`, `react-native-screens`, `react-native-gesture-handler`, `react-native-reanimated`, `react-native-wor
klets` | — | MIT |
| `@react-native-async-storage/async-storage` | 2.2.0 | MIT |
| `zustand` | 5.0.15 | MIT |
| `immer` | 11.1.21 | MIT |
| `@expo/vector-icons` (Ionicons, Material Community Icons) | 15.1.1 | MIT |
| `@expo-google-fonts/manrope` (Manrope by Mikhail Sharanda), `@expo-google-fonts/anton` (Anton by Vernon Adams) | 0.4.x | MIT (package)
/ SIL Open Font License 1.1 (fonts) |
| `@anthropic-ai/sdk` (optional Claude mode) | 0.131.0 | MIT |

## APIs & data
| Data | Source | Terms |
|---|---|---|
| Weather forecasts | [Open-Meteo](https://open-meteo.com) | Free, no key, CC BY 4.0 attribution |
| Exchange rates | [ExchangeRate-API open endpoint](https://www.exchangerate-api.com/docs/free) (`open.er-api.com`) | Free with attribut
ion; cached for 6 h |
| Maps (phone) | Apple Maps (iOS) / Google Maps (Android) via react-native-maps | Platform map terms |
| Maps (web preview) | [OpenStreetMap](https://www.openstreetmap.org/copyright) embed | © OpenStreetMap contributors, ODbL |
| Airport coordinates & codes | Public IATA codes and coordinates (compiled by hand, 100+ airports) | Factual data |
| Emergency numbers | Official tourism / government pages, e.g. [JNTO emergency numbers](https://www.jnto.go.jp/safety-tips/eng/emergenc
y-numbers.html) | Factual data |
| Destination & place facts | Summarised in our own words from public tourism information | Original text |
| AI chatbot (optional) | [Anthropic Claude API](https://docs.claude.com) — only when a user adds their own API key | Anthropic usage po
licies |

The built-in offline Trip Assistant is original rule-based code (`src/services/assistant.ts`) that reads the trip’s own data.

## Photography
All photos are from [Unsplash](https://unsplash.com) and used under the [Unsplash License](https://unsplash.com/license) (free for comme
rcial and non-commercial use, no permission needed). They are bundled in `assets/images/photos/` (resized/compressed) so the app looks c
omplete offline. Source URLs:

| File | Source |
|---|---|
| `amalfi.jpg` | https://images.unsplash.com/photo-1612698093158-e07ac200d44e |
| `tokyo.jpg` | https://images.unsplash.com/photo-1536098561742-ca998e48cbcc |
| `kyoto.jpg` | https://images.unsplash.com/photo-1493976040374-85c8e12f0c0e |
| `paris.jpg` | https://images.unsplash.com/photo-1502602898657-3e91760cbb34 |
| `bali.jpg` | https://images.unsplash.com/photo-1537996194471-e657df975ab4 |
| `bangkok.jpg` | https://images.unsplash.com/photo-1563492065599-3520f775eeed |
| `singapore.jpg` | https://images.unsplash.com/photo-1525625293386-3f8f99389edd |
| `cape-town.jpg` | https://images.unsplash.com/photo-1580060839134-75a5edca2e99 |
| `new-york.jpg` | https://images.unsplash.com/photo-1534430480872-3498386e7856 |
| `santorini.jpg` | https://images.unsplash.com/photo-1613395877344-13d4a8e0d49e |
| `rome.jpg` | https://images.unsplash.com/photo-1552832230-c0197dd311b5 |
| `london.jpg` | https://images.unsplash.com/photo-1513635269975-59663e0ac1ad |
| `iceland.jpg` | https://images.unsplash.com/photo-1504893524553-b855bce32c67 |
| `swiss-alps.jpg` | https://images.unsplash.com/photo-1530122037265-a5f1f91d3b99 |
| `maldives.jpg` | https://images.unsplash.com/photo-1514282401047-d79a71a590e8 |
| `machu-picchu.jpg` | https://images.unsplash.com/photo-1526392060635-9d6019884377 |
| `ha-long-bay.jpg` | https://images.unsplash.com/photo-1528127269322-539801943592 |
| `dubai.jpg` | https://images.unsplash.com/photo-1512453979798-5ea266f8880c |
| `barcelona.jpg` | https://images.unsplash.com/photo-1583422409516-2895a77efded |
| `sydney.jpg` | https://images.unsplash.com/photo-1506973035872-a4ec16b8e8d9 |
| `seoul.jpg` | https://images.unsplash.com/photo-1517154421773-0529f29ea451 |
| `istanbul.jpg` | https://images.unsplash.com/photo-1541432901042-2d8bd64b4a9b |
| `amsterdam.jpg` | https://images.unsplash.com/photo-1534351590666-13e3e96b5017 |
| `prague.jpg` | https://images.unsplash.com/photo-1519677100203-a0e668c92439 |
| `lisbon.jpg` | https://images.unsplash.com/photo-1585208798174-6cedd86e019a |
| `banff.jpg` | https://images.unsplash.com/photo-1561134643-668f9057cce4 |
| `marrakech.jpg` | https://images.unsplash.com/photo-1597212618440-806262de4f6b |
| `rio.jpg` | https://images.unsplash.com/photo-1483729558449-99ef09a8c325 |
| `mexico-city.jpg` | https://images.unsplash.com/photo-1512813195386-6cf811ad3542 |
```

`grand-canyon.jpg`	https://images.unsplash.com/photo-1615551043360-33de8b5f410c
`cinque-terre.jpg`	https://images.unsplash.com/photo-1499678329028-101435549a4e
`venice.jpg`	https://images.unsplash.com/photo-1514890547357-a9ee288728e0
`mount-fuji.jpg`	https://images.unsplash.com/photo-1528164344705-47542687000d
`onboard-1.jpg`	https://images.unsplash.com/photo-1501555088652-021faa106b9b
`onboard-2.jpg`	https://images.unsplash.com/photo-1469854523086-cc02fe5d8800
`onboard-3.jpg`	https://images.unsplash.com/photo-1507525428034-b723cf961d3e
`lagoon.jpg`	https://images.unsplash.com/photo-1506929562872-bb421503ef21
`shibuya.jpg`	https://images.unsplash.com/photo-1542051841857-5f90071e7989
`akihabara.jpg`	https://images.unsplash.com/photo-1540959733332-eab4deabeeaf
`fushimi-inari.jpg`	https://images.unsplash.com/photo-1478436127897-769e1b3f0f36
`omoide-yokocho.jpg`	https://images.unsplash.com/photo-1554797589-7241bb691973
`lantern-alley.jpg`	https://images.unsplash.com/photo-1528360983277-13d401cdc186
`fuji-blossom.jpg`	https://images.unsplash.com/photo-1490806843957-31f4c9a91c65
`tokyo-skyline.jpg`	https://images.unsplash.com/photo-1513407030348-c983a97b98d8
`kyoto-skyline.jpg`	https://images.unsplash.com/photo-1545569341-9eb8b30979d9
`sushi.jpg`	https://images.unsplash.com/photo-1579871494447-9811cf80d66c
`ramen.jpg`	https://images.unsplash.com/photo-1569718212165-3a8278d5f624
`planning.jpg`	https://images.unsplash.com/photo-1488646953014-85cb44e25828
`avatar-maya.jpg`	https://images.unsplash.com/photo-1494790108377-be9c29b29330
`avatar-diego.jpg`	https://images.unsplash.com/photo-1507003211169-0a1dd7228f2d
`avatar-priya.jpg`	https://images.unsplash.com/photo-1534528741775-53994a69daeb
`avatar-sam.jpg`	https://images.unsplash.com/photo-1500648767791-00dcc994a43e
`avatar-ana.jpg`	https://images.unsplash.com/photo-1438761681033-6461ffad8d80
`avatar-noah.jpg`	https://images.unsplash.com/photo-1506794778202-cad84cf45f1d

Portrait photos are used only as avatars for the fictional demo group (Maya, Diego, Priya, Sam, Ana, Noah). All demo people, trips, chats and bookings are fictional sample data.

App icon & logo
Original design for this project: forest-green tile, the Material Community Icons “airplane-takeoff” glyph (Apache 2.0 / MIT via `@expo/vector-icons`) and three dots representing the travel group. Rendered to PNG for the app icon, Android adaptive icon and splash screen.

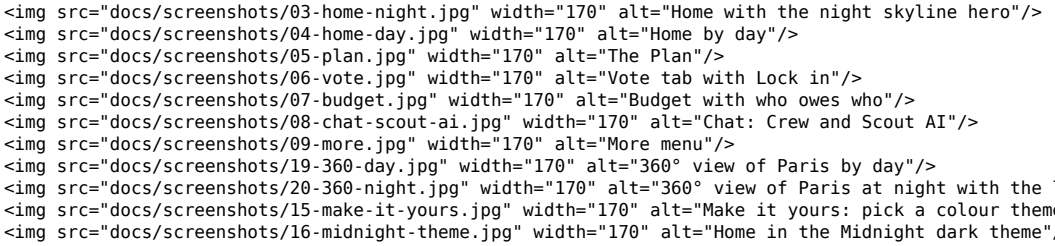
Design reference
The visual style (cream background, forest-green accents, pill navigation, card layouts) follows the reference prototype video provided by the team.

Developer tooling
- **Ponytail** skills (`.claude/skills/ponytail*`, from [DietrichGebert/ponytail](https://github.com/DietrichGebert/ponytail) @ 9cc65d0, MIT licence) — coding-style instructions for the Claude Code assistant used while building the app. Not part of the app itself.
- The 360° city scenes (`src/components/panorama`, `src/data/panoramas.ts`) are original code-drawn illustrations of public landmarks. Landmark facts were written from public sources (e.g. official tower and building heights).

TogetherWeGo ✈️ — Group Trip Planner

FBLA Mobile Application Development 2026–27 · Topic: “Together We Go: Group Trip Planner”

TogetherWeGo helps a group of friends or family plan, budget and enjoy a shared trip — from the first idea, through the trip itself, to the final recap. It runs natively on **iPhone and Android** (Expo / React Native, TypeScript) and keeps working **offline**, which matters at a conference venue with unreliable wifi.



---

1. Run it on your phone (5 minutes)

You need: a laptop with [Node.js 20+](https://nodejs.org), and the free **Expo Go** app on your phone ([iOS](https://apps.apple.com/app/expo-go/id982107779) · [Android](https://play.google.com/store/apps/details?id=host.exp.exponent)). Make sure Expo Go is up to date — this project uses **Expo SDK 57**.

```bash
cd together-we-go
npm install
npx expo start            # same wifi for phone + laptop
# or, on school / conference wifi that blocks devices from seeing each other:
npx expo start --tunnel
```

Scan the QR code (iPhone: Camera app · Android: Expo Go → *Scan QR code*).

Demo login (also shown on the sign-in screen — tap the box to fill it in):

Email	Password
`maya.chen@togetherwego.app`	`TravelTogether2026!`

The sample trip is generated **relative to today**, so “Amalfi Squad” (4 friends, 6 days in Positano, Italy) always starts in 3 days; a second 10-day “Tokyo & Kyoto Explorer” trip, a Bali plan and a finished Barcelona trip show multi-trip management. *Settings → Demo timeline* rebuilds it as **upcoming**, **in progress (Day 2)** or **just finished** to show every stage.

For the competition: run with zero internet
Expo Go needs the laptop’s dev server. For a fully standalone app, build an installable copy with EAS (free account):

```bash
npx eas-cli@latest build -p android --profile preview   # gives you an .apk to install
```

(iOS standalone builds need an Apple Developer account; Expo Go over a phone hotspot also works without venue wifi.)
Once installed, everything except live weather, live exchange rates, maps tiles and the optional Claude AI works offline — and those fall back to cached/estimated data automatically.

Other scripts: `npm run web` (browser preview), `npx tsc --noEmit` (type-check), `npx expo lint`.

---

2. What it does — mapped to the brief

Brief area	Feature in TogetherWeGo	Where
Collaboration	Trips with auto-generated invite codes (`AML-2026`), join by code or deep link	Home code pill · More → Crew & roles · *Join a trip*
	Roles: Organizer, Treasurer, Navigator, Food Lead, Photographer… with descriptions	More → Crew & roles
	Shared checklist with assignees	More → Crew & roles
	Voting: 👍/👎 on activities (majority auto-confirms) + multi-option polls	Plan → Vote tab (progress bar + **Lock in**) · More → Crew & roles (polls)
Communication	Group trip chat with typing indicators	Chat tab
	Trip Assistant (AI chatbot) — answers from live trip data: budget, who owes who, weather, food nearby, next activity, flights, time zones, currency, phrases, emergencies. Works offline; optional Claude mode with your own key	Chat tab (✨ button or quick chips)
Scheduling	Day-by-day itinerary/calendar with timeline, status pills, assignees	Plan tab
	Members upload flight tickets (photo/PDF); everyone’s arrivals & departures in one list	Flights
	Time zones: times stored as exact instants; toggle trip time ↔ your phone’s time	More → Flights & arrivals
	Trip map with itinerary pins, places and airports + **offline radar map** (no tiles needed)	Trip map
	Live reminders: tap 🔔 on an activity → real phone notification before it starts (10/30/60/120 min) + live countdown on Home	Activity card · Settings

Budgeting	Group budget cap, spent so far, per-person, % used	Budget tab
	Spending breakdown by category (Wanderlog-style bar)	Budget tab
	Currency converter (live rates, cached offline) + **home ↔ local currency view**	Budget tab · Currency
	Expenses in any currency, split between chosen members, **settle-up with fewest payments**	Budget tab
	Daily spending planner + “budget finds” within budget & radius	Budget tab
	Printable planner (print or save as PDF)	Plan → printer button · More → Printable Planner
Organization	Offline document vault (PDFs, photos, notes)	More → Offline Docs
	Weather (7-day forecast, best outdoor day, packing tips)	Weather
	Choose your own radius → popular places nearby	Home · More → Explore Nearby
	Photo dump with likes, captions, per-day filter, share to socials	Photo dump
	Activity filters (adventure, sightseeing, food…) **and** time filters (under 1 h, 1–2 h, 2–4 h, half day, full day)	Plan · Nearby
Other	Interactive travel quiz with timer, streak bonus and group leaderboard	Travel quiz
	6 languages: English, Español, Français, 日本語, हिन्दी, 中文	Settings → Language
	Bucket list with inspiration	Bucket list
	Tracks location → nearest airport, distance, drive time, directions	Nearest airport
	Recommendation survey → ranked destinations with photography	For you
	Offline map	Trip map → Offline
Trip management	Multiple trips, switch / create / edit / leave / delete, planning → active → completed	My trips

Extras to stand out
- **360° city views** — drag (or use the arrows / auto-rotate) to spin all the way around 10 popular cities: Positano, Tokyo, Paris, New York, London, Dubai, Sydney, Singapore, Rio and Rome. Three parallax skyline layers, famous landmarks (tap a landmark chip to turn to it and read a fact), a compass readout and a **day / night toggle**: at night the windows light up, the landmarks glow, cars stream along the street and stars and a moon come out. Open it from Home (360° chip), a destination page or More → 360° City Views.
- **Skyline hero that follows the clock** — the Home photo switches between a sunny day look (sun glare, drifting clouds) and a blue-hour night look (stars, moon) based on the local time in the trip’s city, with a sun/moon chip to preview the other half of the day.
- **Lock in voting** — Plan → Vote shows each proposal’s vote count, a progress bar towards a majority and a **Lock in** button (organizer only) that confirms it and turns its reminder on.
- **Scout AI tab** — the group chat stays human; the Trip Assistant has its own tab with quick questions (budget, food nearby, weather, who owes who…).
- **Today card** — once the trip starts, Home shows “Today · Day 2 of 6” with a tick-off list.
- **Who’s bringing what** — the packing list assigns every item to a person, with a “Split the rest fairly” button.
- **Receipt photos** on expenses, and a shareable **story card** for the trip recap.
- **First-run tour** and a **How it works** page (architecture, security, validation and a rubric map) inside the app.
- **Make it yours** — a 7-step animated personalization survey: 6 colour themes (Forest, Ocean, Sunset, Lavender, Sakura, **Midnight dark mode**), card shape (Rounded / Bubbly / Crisp), travel interests with photos, pace & budget style, which Explore sections to show, nickname + emoji avatar, language and home currency — with a live preview and a confetti finish. New accounts get it right after sign-up; it's always in *Settings*, the 🎨 button on Explore and the menu.
- **“Picked for you”** — Home ranks destinations against your interests and budget style and shows a match %.
- **Motion everywhere** — animated home hero (a plane flies a dotted route past drifting clouds and a pulsing sun), staggered card entrances, a tab highlight that glides between tabs, Ken Burns zoom on onboarding photos, floating logo, pulsing live-reminder bell. One *Animations* switch turns it all off for people who prefer less motion.
- **Trip recap** — stats (days, km between stops, votes, photos), highlights, final balances, rating, shareable summary.
- **Emergency & safety** — local police/ambulance numbers (tap to call), share live location, hospital & embassy finder.
- **Phrasebook** with **text-to-speech** pronunciation (Japanese, French, Spanish, Italian, Korean, Thai, Portuguese).
- **Smart packing list** — suggestions from the destination and the forecast (e.g. “IC card (Suica)”, umbrella when rain is likely).
- **Demo timeline** switch to show the whole trip lifecycle live.
- **Social media integration** — direct share into WhatsApp, Instagram, X, Facebook, Messenger, Telegram, Messages, Email + the native share sheet; photos share as images.
- Accessibility: screen-reader labels on every control, adjustable text size, haptics toggle, high-contrast palette.

Judges: [`docs/RATING_SHEET.md`](docs/RATING_SHEET.md) lists every line of the FBLA rating sheet with where it is shown. The 7-minute script is in [`docs/DEMO_SCRIPT.md`](docs/DEMO_SCRIPT.md).

---

3. How it’s built

```
src/
  app/            Screens (Expo Router — every file is a route)
    (tabs)/       Home · Plan · Budget · Chat · More
    *.tsx         Feature screens (flights, map, weather, photos, …)
  components/     Reusable UI kit (ui/), trip widgets (trip/), map (native + web)
  store/          Zustand store: data model, actions, selectors, demo seed
  services/       Side effects: weather, currency, notifications, assistant, files, print, share, location
  data/           Offline datasets: 32 destinations, curated places, 100+ airports, phrases, quiz…
  i18n/           Translations (6 languages) + useT() hook
  utils/          Validation, time zones, geo math, formatting
```

- **Architecture:** layered MVVM-style. Screens (views) read state through selector hooks (view-model) and call store actions; services wrap every external API/device capability so screens never talk to the network directly. One-way data flow.
- **State & storage:** Zustand + Immer, persisted to AsyncStorage → the whole app works offline and survives restarts.
- **Navigation:** Expo Router with `Stack.Protected` guards (signed-out users can only see onboarding/auth).
- **Platform-specific code:** `TripMap.tsx` (react-native-maps) vs `TripMap.web.tsx` (OpenStreetMap embed).
- **Theming:** `src/theme` holds 6 palettes and 3 corner styles. The saved choice is read synchronously before any screen's styles are created, so every `StyleSheet` picks up the user's palette; applying a new look restarts the UI once (`DevSettings.reload` on phones, a page reload on web).
- **Animations:** `src/components/ui/motion.tsx` (FadeIn, Float, Pulse, looping values) on React Native's `Animated` API with the native driver on iOS/Android.
- **React Compiler** enabled; the code passes the compiler’s lint rules (`npx expo lint` → 0 problems) and `tsc --noEmit` with `strict: true`.

Data handling & security
- Passwords are **salted and SHA-256 hashed** (expo-crypto) — never stored in plain text.
- “Remember me” off → the session is not persisted.
- The optional Anthropic API key lives in the **iOS Keychain / Android Keystore** (expo-secure-store), never in the app database.
- Documents are copied into the app’s private storage so they stay available offline.
- Network calls have timeouts and fall back to cached or bundled data.

Input validation (syntactic + semantic)
Email format, password strength meter, matching passwords, names (letters only), money amounts (`25` / `25.50`, positive, sane max), trip dates (end after start, ≤ 60 days, not in the past), invite-code format `ABC-1234`, flight numbers (`NH 175`), IATA codes checked against the airport database, arrival-after-departure across time zones, seat format, overlapping-activity warning, duplicate poll options, file-size limits.

---

4. Libraries, sources & licences
See **ATTRIBUTIONS.md** for every library, API, font, icon set and photo source. Planning documents: **docs/PLANNING.md**. Presentation script: **docs/DEMO_SCRIPT.md**.

5. Honest limitations
- There is **no cloud server**: trips are stored on each device. Invite codes join trips that exist on that device plus a built-in demo directory (try `SEO-5521`). A production version would sync through a backend such as Firebase or Supabase — the store/service split means only the service layer would change.
- The 360° city views are illustrated scenes drawn in code (SVG), not photographs, so they work offline and need no image licences. The Home hero’s day/night effect is a bundled photo with code-drawn light, stars and clouds (no video file), so it works offline and adds no download size.
- “Continue with Google” is a simulated account chooser; a store build would use Google OAuth (expo-auth-session).
- Claude AI mode requires the user’s own Anthropic API key; without it the built-in offline assistant answers.
- Standalone Android builds need a Google Maps API key for the native map (Expo Go already includes one); the offline radar map needs nothing.

app.json

```
{
  "expo": {
    "name": "TogetherWeGo",
    "slug": "together-we-go",
    "description": "Plan, budget and enjoy group trips together.",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/images/icon.png",
    "scheme": "togetherwego",
    "userInterfaceStyle": "light",
    "backgroundColor": "#F1F2F4",
    "ios": {
      "bundleIdentifier": "com.togetherwego.app",
      "supportsTablet": true
    },
    "android": {
      "package": "com.togetherwego.app",
      "adaptiveIcon": {
        "backgroundColor": "#1F4D25",
        "foregroundImage": "./assets/images/android-icon-foreground.png",
        "backgroundImage": "./assets/images/android-icon-background.png",
        "monochromeImage": "./assets/images/android-icon-monochrome.png"
      },
      "predictiveBackGestureEnabled": false
    },
    "web": {
      "output": "single",
      "favicon": "./assets/images/favicon.png",
      "name": "TogetherWeGo",
      "themeColor": "#0B0B0D",
      "backgroundColor": "#F1F2F4"
    },
    "plugins": [
      "expo-router",
      [
        "expo-splash-screen",
        {
          "backgroundColor": "#F1F2F4",
          "image": "./assets/images/splash-icon.png",
          "imageWidth": 160
        }
      ],
      [
        "expo-location",
        {
          "locationWhenInUsePermission": "TogetherWeGo uses your location to show places nearby, the closest airport and to share your l
ocation in an emergency."
        }
      ],
      [
        "expo-image-picker",
        {
          "photosPermission": "TogetherWeGo lets you add trip photos and flight tickets from your library.",
          "cameraPermission": "TogetherWeGo lets you snap photos for the group photo dump."
        }
      ],
      [
        "expo-notifications",
        {
          "color": "#0B0B0D"
        }
      ],
      "expo-sharing",
      "expo-localization",
      "expo-secure-store",
      "expo-asset",
      "expo-font"
    ],
    "experiments": {
      "typedRoutes": true,
      "reactCompiler": true
    }
  }
}
```

eas.json

```
{
  "cli": {
    "version": ">= 16.0.0",
    "appVersionSource": "remote"
  },
  "build": {
    "preview": {
      "distribution": "internal",
      "android": {
        "buildType": "apk"
      }
    },
    "production": {
      "autoIncrement": true
    }
  },
  "submit": {
    "production": {}
  }
}
```

eslint.config.js

```
// https://docs.expo.dev/guides/using-eslint/
const { defineConfig } = require('eslint/config');
const expoConfig = require("eslint-config-expo/flat");

module.exports = defineConfig([
  expoConfig,
  {
    ignores: ["dist/*"],
  }
]);
```

package.json

```
{
  "name": "together-we-go",
  "main": "expo-router/entry",
  "version": "1.0.0",
  "dependencies": {
    "@anthropic-ai/sdk": "^0.131.0",
    "@expo-google-fonts/anton": "^0.4.2",
    "@expo-google-fonts/manrope": "^0.4.2",
    "@expo/vector-icons": "^15.0.2",
    "@react-native-async-storage/async-storage": "2.2.0",
    "expo": "~57.0.29",
    "expo-asset": "~57.0.18",
    "expo-clipboard": "~57.0.2",
    "expo-constants": "~57.0.20",
    "expo-crypto": "~57.0.4",
    "expo-document-picker": "~57.0.3",
    "expo-file-system": "~57.0.7",
    "expo-font": "~57.0.4",
    "expo-haptics": "~57.0.3",
    "expo-image": "~57.0.5",
    "expo-image-picker": "~57.0.20",
    "expo-linear-gradient": "~57.0.2",
    "expo-linking": "~57.0.13",
    "expo-localization": "~57.0.2",
    "expo-location": "~57.0.21",
    "expo-notifications": "~57.0.23",
    "expo-print": "~57.0.2",
    "expo-router": "~57.0.27",
    "expo-secure-store": "~57.0.4",
    "expo-sharing": "~57.0.23",
    "expo-speech": "~57.0.3",
    "expo-splash-screen": "~57.0.10",
    "expo-status-bar": "~57.0.1",
    "expo-system-ui": "~57.0.4",
    "immer": "^11.1.21",
    "react": "19.2.3",
    "react-dom": "19.2.3",
    "react-native": "0.86.3",
    "react-native-gesture-handler": "~2.32.0",
    "react-native-maps": "1.27.2",
    "react-native-reanimated": "4.5.1",
    "react-native-safe-area-context": "~5.7.0",
    "react-native-screens": "~4.26.0",
    "react-native-svg": "15.15.4",
    "react-native-web": "~0.21.0",
    "react-native-worklets": "0.10.1",
    "zustand": "^5.0.15"
  },
  "devDependencies": {
    "@types/react": "~19.2.2",
    "eslint": "^9.0.0",
    "eslint-config-expo": "~57.0.2",
    "typescript": "~6.0.3"
  },
  "scripts": {
    "start": "expo start",
    "reset-project": "node ./scripts/reset-project.js",
    "android": "expo start --android",
    "ios": "expo start --ios",
    "web": "expo start --web",
    "lint": "expo lint"
  },
  "private": true
}
```

tsconfig.json

```
{
  "extends": "expo/tsconfig.base",
  "compilerOptions": {
    "strict": true,
    "paths": {
      "@/*": [
        "./src/*"
      ],
      "@/assets/*": [
        "./assets/*"
      ]
    }
  },
  "include": [
    "**/*.ts",
    "**/*.tsx",
    ".expo/types/**/*.ts",
    "expo-env.d.ts"
  ]
}
```

src/app/(tabs)/_layout.tsx

```
import { Tabs } from 'expo-router/js-tabs';

import { TabBar } from '@/components/TabBar';
import { colors } from '@/theme';

export default function TabsLayout() {
  return (
    <Tabs
      initialRouteName="explore"
      tabBar={(props) => <TabBar {...props} />}
      screenOptions={{ headerShown: false, sceneStyle: { backgroundColor: colors.bg } }}>
      <Tabs.Screen name="explore" />
      <Tabs.Screen name="itinerary" />
      <Tabs.Screen name="budget" />
      <Tabs.Screen name="chat" />
      <Tabs.Screen name="profile" />
    </Tabs>
  );
}
```

src/app/(tabs)/budget.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { router } from 'expo-router';
import { useState } from 'react';
import { Platform, ScrollView, StyleSheet, TextInput, View } from 'react-native';

import { CurrencyConverter } from '@/components/CurrencyConverter';
import { NoTrip } from '@/components/trip/NoTrip';
import { expenseCategories, placeCategoryIcon } from '@/components/trip/meta';
import { Button } from '@/components/ui/Button';
import { Avatar, AvatarStack, Card, Chip, Pill } from '@/components/ui/bits';
import { Donut } from '@/components/ui/Donut';
import { confirmAction, toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { Press, tap } from '@/components/ui/Press';
import { Screen } from '@/components/ui/Screen';
import { Segmented } from '@/components/ui/Segmented';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { placesFor } from '@/data/places';
import { useRatesLoader } from '@/hooks/useTrip';
import { useT } from '@/i18n';
import { convert, useRates } from '@/services/currency';
import { deleteLocalFile, pickImage } from '@/services/files';
import { balances, memberById, settlePlan, totalSpent, useActiveTrip, useAppStore } from '@/store/useAppStore';
import type { Expense, ExpenseCategory } from '@/store/types';
import { categoryColors, colors, fonts, radius } from '@/theme';
import { money, priceTier } from '@/utils/format';
import { distanceKm, kmLabel } from '@/utils/geo';
import { daysBetween, todayInTz } from '@/utils/time';
import { firstError, maxLen, parseAmount, required, validateAmount } from '@/utils/validation';

export default function Budget() {
  const trip = useActiveTrip();
  if (!trip) return <NoTrip />;
  return <BudgetInner key={trip.id} />;
}

function BudgetInner() {
  const t = useT();
  useRatesLoader();
  const trip = useActiveTrip()!;
  const rates = useRates();
  const home = useAppStore((s) => s.settings.homeCurrency);
  const userId = useAppStore((s) => s.session?.userId ?? '');
  const addExpense = useAppStore((s) => s.addExpense);
  const deleteExpense = useAppStore((s) => s.deleteExpense);
  const updateTrip = useAppStore((s) => s.updateTrip);
  const settleUp = useAppStore((s) => s.settleUp);
  const dest = destinationById(trip.destinationId);
  const local = dest?.currency ?? 'EUR';

  const [view, setView] = useState<'home' | 'local'>('home');
  const [conv, setConv] = useState<[string, string]>([home, local === home ? 'EUR' : local]);
  const [capText, setCapText] = useState(String(trip.budget));
  const [capErr, setCapErr] = useState<string | null>(null);
  const [detail, setDetail] = useState<Expense | null>(null);
  const [receiptBig, setReceiptBig] = useState(false);
  const [settleOpen, setSettleOpen] = useState(false);
  const [addOpen, setAddOpen] = useState(false);
  const [daily, setDaily] = useState(String(dest?.dailyCost ?? 120));
  const [today] = useState(() => todayInTz(dest?.tz ?? 'UTC'));

  // add-expense form
  const [title, setTitle] = useState('');
  const [amount, setAmount] = useState('');
  const [entryCur, setEntryCur] = useState<'USD' | string>('USD');
  const [cat, setCat] = useState<ExpenseCategory>('Food');
  const [paidBy, setPaidBy] = useState(userId);
  const [split, setSplit] = useState<string[]>(trip.members.map((m) => m.id));
  const [receipt, setReceipt] = useState<string | null>(null);
  const [attaching, setAttaching] = useState(false);
  const [errors, setErrors] = useState<Record<string, string | null>>({});

  const viewCode = view === 'home' ? home : local;
  const fmt = (usd: number) => money(convert(usd, 'USD', viewCode, rates), viewCode);

  const spent = totalSpent(trip);
  const per = spent / Math.max(1, trip.members.length);
  const catTotals: Record<string, number> = {};
  trip.expenses.filter((e) => !e.settlement).forEach((e) => (catTotals[e.category] = (catTotals[e.category] ?? 0) + e.amount));
  const byCat = Object.entries(catTotals).sort((a, b) => b[1] - a[1]);
  const bal = balances(trip);
```

```
const plan = settlePlan(trip);
const tripLen = daysBetween(trip.startDate, trip.endDate) + 1;
const daysLeft = Math.max(1, Math.min(tripLen, daysBetween(today, trip.endDate) + 1));
const dailyN = parseAmount(daily) ?? 0;
const plannedTotal = dailyN * tripLen * trip.members.length;
const remaining = trip.budget - spent;
const activityAllowance = dailyN * 0.3;
const finds = dest
  ? [trip.destinationId, ...trip.extraDestinationIds]
      .flatMap((id) => placesFor(id))
      .filter((p) => p.cost <= activityAllowance)
      .map((p) => ({ p, km: distanceKm(dest, p) }))
      .filter((x) => x.km <= 8)
      .sort((a, b) => b.p.rating - a.p.rating)
      .slice(0, 3)
  : [];

const saveCap = () => {
  const e = validateAmount(capText, { min: 1, max: 1_000_000, label: 'a budget' });
  setCapErr(e);
  if (e) return;
  const v = parseAmount(capText)!;
  updateTrip(trip.id, { budget: v });
  toast(`Budget cap set to ${money(v)}`);
};

const attach = async (camera: boolean) => {
  setAttaching(true);
  try {
    const f = await pickImage(camera);
    if (f) {
      deleteLocalFile(receipt ?? undefined);
      setReceipt(f.uri);
    }
  } catch (e) {
    toast(e instanceof Error ? e.message : 'Couldn’t open photos', { tone: 'warn' });
  } finally {
    setAttaching(false);
  }
};

const removeReceipt = () => {
  deleteLocalFile(receipt ?? undefined);
  setReceipt(null);
};

const submit = () => {
  const e = {
    title: firstError(title, required('what it was for'), maxLen(40, 'Description')),
    amount: validateAmount(amount, { max: 100000 }),
    split: split.length ? null : 'Pick at least one person to split with',
  };
  setErrors(e);
  if (Object.values(e).some(Boolean)) return tap('warning');
  const n = parseAmount(amount)!;
  const usd = entryCur === 'USD' ? n : convert(n, entryCur, 'USD', rates);
  addExpense(trip.id, {
    title: title.trim(),
    amount: Math.round(usd * 100) / 100,
    category: cat,
    paidBy,
    splitBetween: split,
    date: todayInTz(dest?.tz ?? 'UTC'),
    original: entryCur === 'USD' ? undefined : { amount: n, currency: entryCur },
    receipt: receipt ?? undefined,
  });
  tap('success');
  toast(`Added ${money(usd)} · ${money(usd / split.length)} each`);
  setTitle('');
  setAmount('');
  setReceipt(null);
  setAddOpen(false);
  if (spent + usd > trip.budget && spent <= trip.budget) toast('Heads up: this puts the group over budget', { tone: 'warn' });
};

const openDetail = (e: Expense) => {
  setReceiptBig(false);
  setDetail(e);
};

const firstName = (id: string, name: string) => (id === userId ? 'You' : name.split(' ')[0]);

return (
  <Screen bottomInset={30}>
    <View style={styles.head}>
      <View style={{ flex: 1 }}>
```

```
          <T variant="kicker" color={colors.textSecondary} numberOfLines={1}>
            {trip.name}
          </T>
          <T variant="display" accessibilityRole="header">
            {t('budget.title')}
          </T>
        </View>
        <Press onPress={() => setAddOpen(true)} style={styles.plus} accessibilityLabel="Add expense" testID="add-expense-open">
          <Ionicons name="add" size={26} color={colors.onPrimary} />
        </Press>
      </View>

      <Segmented
        options={[
          { key: 'home', label: `Home · ${home}` },
          { key: 'local', label: `Local · ${local}` },
        ]}
        value={view}
        onChange={setView}
      />

      <Card style={[styles.card, { marginTop: 14 }]}>
        <View style={styles.donutRow}>
          <Donut
            size={132}
            parts={byCat.map(([c, v]) => ({ value: v, color: categoryColors[c] ?? colors.textMuted }))}
            centerLabel={remaining < 0 ? 'OVER' : 'LEFT'}
            centerValue={fmt(Math.abs(remaining))}
          />
          <View style={{ flex: 1, gap: 8 }}>
            <T variant="kicker" weight="bold" color={colors.textSecondary} numberOfLines={1} adjustsFontSizeToFit style={styles.tinyKick
er}>
              {`Spent ${fmt(spent)} / ${fmt(trip.budget)}`}
            </T>
            {byCat.map(([c, v]) => (
              <View key={c} style={styles.legendRow}>
                <View style={[styles.dot, { backgroundColor: categoryColors[c] ?? colors.textMuted }]} />
                <T variant="small" weight="medium" style={{ flex: 1 }} numberOfLines={1}>
                  {c}
                </T>
                <T variant="small" weight="bold">
                  {fmt(v)}
                </T>
              </View>
            ))}
            {!byCat.length ? (
              <T variant="small" color={colors.textMuted}>
                No expenses yet. Tap + to add your first one.
              </T>
            ) : null}
          </View>
        </View>
      </Card>

      <View style={styles.tiles}>
        <Card style={[styles.card, styles.tile]}>
          <T variant="kicker" weight="bold" color={colors.textSecondary} style={styles.tinyKicker}>
            Per person
          </T>
          <T variant="h2" weight="extrabold" numberOfLines={1} adjustsFontSizeToFit>
            {fmt(per)}
          </T>
        </Card>
        <Card style={[styles.card, styles.tile]}>
          <T variant="kicker" weight="bold" color={colors.textSecondary} style={styles.tinyKicker}>
            Daily allowance
          </T>
          <T variant="h2" weight="extrabold" numberOfLines={1} adjustsFontSizeToFit>
            {fmt(Math.max(0, remaining) / daysLeft)}
          </T>
        </Card>
      </View>

      <View style={styles.sectionRow}>
        <T variant="headline" accessibilityRole="header" style={{ flexShrink: 1 }}>
          Who owes who
        </T>
        <Button label={t('budget.settle')} size="sm" full={false} variant="soft" icon="swap-horizontal" onPress={() => setSettleOpen(tru
e)} />
      </View>
      <Card padded={false} style={styles.card}>
        {trip.members.map((m, i) => {
          const v = bal[m.id] ?? 0;
          return (
            <View key={m.id} style={[styles.balRow, i > 0 && styles.balDivider]}>
              <Avatar name={m.name} src={m.avatar} size={34} />
```

```
              <View style={styles.nameWrap}>
                <T variant="bodySm" weight="semibold" numberOfLines={1} style={{ flexShrink: 1 }}>
                  {firstName(m.id, m.name)}
                </T>
                {m.role === 'Treasurer' ? (
                  <View style={styles.badge}>
                    <T variant="micro" weight="bold" color={colors.onAccent}>
                      Treasurer
                    </T>
                  </View>
                ) : null}
              </View>
              <T variant="small" weight="bold" color={Math.abs(v) < 0.5 ? colors.textMuted : v > 0 ? colors.positive : colors.red}>
                {Math.abs(v) < 0.5 ? 'settled' : v > 0 ? `gets back ${fmt(v)}` : `owes ${fmt(-v)}`}
              </T>
            </View>
          );
        })}
      </Card>

      <T variant="headline" accessibilityRole="header" style={styles.section}>
        Converter
      </T>
      <Card style={styles.card}>
        <CurrencyConverter from={conv[0]} to={conv[1]} onChange={(a, b) => setConv([a, b])} />
      </Card>

      <T variant="headline" accessibilityRole="header" style={styles.section}>
        {t('budget.planner')}
      </T>
      <Card style={[styles.card, { gap: 12 }]}>
        <T variant="small" color={colors.textSecondary}>
          How much does each person want to spend per day in {dest?.city}? (Typical: {money(dest?.dailyCost ?? 0)})
        </T>
        <View style={styles.dailyRow}>
          <T variant="h3">$</T>
          <TextInput value={daily} onChangeText={(v) => setDaily(v.replace(/[^0-9.]/g, ''))} keyboardType="decimal-pad" style={styles.da
ilyInput} accessibilityLabel="Daily spend per person in USD" maxLength={6} />
          <T variant="small" color={colors.textSecondary}>
            / person / day
          </T>
        </View>
        {dailyN > 0 ? (
          <>
            <View style={styles.splitRow}>
              {[
                ['Food', 0.35],
                ['Activities', 0.3],
                ['Transport', 0.15],
                ['Shopping', 0.12],
                ['Other', 0.08],
              ].map(([k, f]) => (
                <View key={k as string} style={styles.splitCell}>
                  <View style={[styles.dot, { backgroundColor: categoryColors[k as string] }]} />
                  <T variant="micro" color={colors.textSecondary}>
                    {k as string}
                  </T>
                  <T variant="small" weight="bold">
                    {money(dailyN * (f as number))}
                  </T>
                </View>
              ))}
            </View>
            <View style={[styles.fit, { backgroundColor: plannedTotal <= remaining ? colors.primarySofter : colors.orangeSoft }]}>
              <Ionicons name={plannedTotal <= remaining ? 'checkmark-circle' : 'alert-circle'} size={16} color={plannedTotal <= remainin
g ? colors.positive : colors.orange} />
              <T variant="caption" style={{ flex: 1 }}>
                {tripLen} days × {trip.members.length} people = {money(plannedTotal)} on the ground ·{' '}
                {plannedTotal <= remaining ? `fits the ${money(remaining)} left` : `${money(plannedTotal - remaining)} more than what’s
left`}
              </T>
            </View>
            <T variant="small" weight="bold" style={{ marginTop: 2 }}>
              Budget finds near {dest?.city} (≤ {money(activityAllowance)} each)
            </T>
            {finds.map(({ p, km }) => (
              <Press key={p.id} onPress={() => router.push({ pathname: '/nearby', params: { focus: p.id, maxCost: String(Math.round(acti
vityAllowance)) } })} style={styles.findRow}>
                <View style={styles.findIcon}>
                  <Ionicons name={placeCategoryIcon[p.category]} size={15} color={colors.text} />
                </View>
                <T variant="small" weight="medium" style={{ flex: 1 }} numberOfLines={1}>
                  {p.name}
                </T>
                <T variant="caption" color={colors.textSecondary}>
                  {priceTier(p.cost)} · {kmLabel(km)}
```

```
              </T>
            </Press>
          ))}
        </>
      ) : (
        <T variant="caption" color={colors.red}>
          Enter a daily amount greater than 0
        </T>
      )}
    </Card>

    <View style={[styles.sectionRow, { marginTop: 26 }]}>
      <T variant="headline" accessibilityRole="header" style={{ flexShrink: 1 }}>
        {t('budget.expenses')}
      </T>
      <View style={styles.capBox}>
        <T variant="caption" weight="semibold" color={colors.textSecondary}>
          {t('budget.cap')}
        </T>
        <TextInput
          value={capText}
          onChangeText={(v) => setCapText(v.replace(/[^0-9.]/g, ''))}
          onBlur={saveCap}
          onSubmitEditing={saveCap}
          keyboardType="decimal-pad"
          style={[styles.capInput, capErr && { borderColor: colors.red }]}
          accessibilityLabel="Group budget cap in USD"
          maxLength={8}
        />
      </View>
    </View>
    {capErr ? (
      <T variant="caption" color={colors.red} style={{ marginBottom: 8, marginTop: -4 }}>
        {capErr}
      </T>
    ) : null}
    <View style={{ gap: 10 }}>
      {trip.expenses.map((e) => (
        <Card key={e.id} padded={false} onPress={() => openDetail(e)} style={styles.expense} accessibilityLabel={`${e.title}, ${fmt(e.
amount)}`}>
          <View style={[styles.expenseDot, { backgroundColor: e.settlement ? colors.textMuted : categoryColors[e.category] ?? colors.t
extMuted }]} />
          <View style={{ flex: 1 }}>
            <View style={styles.titleRow}>
              <T variant="bodySm" weight="bold" numberOfLines={1} style={{ flexShrink: 1 }}>
                {e.title}
              </T>
              {e.receipt ? <Ionicons name="attach" size={15} color={colors.textMuted} accessibilityLabel="Has receipt" /> : null}
            </View>
            <T variant="caption" color={colors.textSecondary} numberOfLines={1}>
              {e.settlement ? 'Settlement' : e.category} · {t('budget.paidBy', { name: memberById(trip, e.paidBy)?.name.split(' ')[0]
?? '—' })}
              {e.original ? ` · ${money(e.original.amount, e.original.currency)}` : ''}
            </T>
          </View>
          <T variant="small" weight="extrabold">
            {fmt(e.amount)}
          </T>
        </Card>
      ))}
    </View>

    {/* Add expense */}
    <Sheet
      visible={addOpen}
      onClose={() => setAddOpen(false)}
      title={t('budget.addExpense')}
      footer={<Button label={t('budget.add')} icon="add" onPress={submit} testID="add-expense" />}>
      <View style={{ gap: 12 }}>
        <Input placeholder={t('budget.whatFor')} value={title} onChangeText={setTitle} error={errors.title} maxLength={45} testID="exp
ense-title" />
        <View style={{ flexDirection: 'row', gap: 8 }}>
          <Input placeholder="100" value={amount} onChangeText={setAmount} keyboardType="decimal-pad" error={errors.amount} containerS
tyle={{ flex: 1 }} icon="cash-outline" testID="expense-amount" />
          <View style={styles.curToggle}>
            {['USD', local].filter((c, i, a) => a.indexOf(c) === i).map((c) => (
              <Press key={c} onPress={() => setEntryCur(c)} style={[styles.curBtn, entryCur === c && styles.curOn]} accessibilityLabel
={`Enter amount in ${c}`}>
                <T variant="caption" weight="semibold" color={entryCur === c ? colors.onPrimary : colors.textSecondary}>
                  {c}
                </T>
              </Press>
            ))}
          </View>
        </View>
        {entryCur !== 'USD' && parseAmount(amount) ? (
```

```
            <T variant="caption" color={colors.textSecondary}>
              ≈ {money(convert(parseAmount(amount)!, entryCur, 'USD', rates), 'USD', { decimals: true })} — saved in USD so the group ca
n compare
            </T>
          ) : null}
          <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 6 }}>
            {expenseCategories.map((c) => (
              <Chip key={c.key} small label={c.key} icon={c.icon} active={cat === c.key} onPress={() => setCat(c.key)} />
            ))}
          </ScrollView>
          <T variant="caption" weight="semibold" color={colors.textSecondary}>
            Paid by
          </T>
          <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 6 }}>
            {trip.members.map((m) => (
              <Chip key={m.id} small label={m.name.split(' ')[0]} image={m.avatar} active={paidBy === m.id} onPress={() => setPaidBy(m.i
d)} />
            ))}
          </ScrollView>
          <View style={styles.rowBetween}>
            <T variant="caption" weight="semibold" color={colors.textSecondary}>
              Split between ({split.length})
            </T>
            <Press onPress={() => setSplit(split.length === trip.members.length ? [userId] : trip.members.map((m) => m.id))} hitSlop={8}
>
              <T variant="caption" weight="bold">
                {split.length === trip.members.length ? 'Just me' : 'Everyone'}
              </T>
            </Press>
          </View>
          <View style={{ flexDirection: 'row', flexWrap: 'wrap', gap: 6 }}>
            {trip.members.map((m) => (
              <Chip
                key={m.id}
                small
                label={m.name.split(' ')[0]}
                icon={split.includes(m.id) ? 'checkmark' : undefined}
                active={split.includes(m.id)}
                onPress={() => setSplit((s) => (s.includes(m.id) ? s.filter((x) => x !== m.id) : [...s, m.id]))}
              />
            ))}
          </View>
          {errors.split ? (
            <T variant="caption" color={colors.red}>
              {errors.split}
            </T>
          ) : null}
          <View style={styles.receiptRow}>
            <Button label="Attach receipt" icon="camera-outline" variant="soft" size="md" loading={attaching} onPress={() => attach(fals
e)} style={{ flex: 1 }} />
            {Platform.OS !== 'web' ? <Button label="Take photo" icon="scan-outline" variant="soft" size="md" disabled={attaching} onPres
s={() => attach(true)} style={{ flex: 1 }} /> : null}
          </View>
          {receipt ? (
            <View style={styles.receiptRow}>
              <View>
                <Image source={{ uri: receipt }} style={styles.thumb} contentFit="cover" accessibilityLabel="Receipt photo" />
                <Press onPress={removeReceipt} style={styles.thumbX} hitSlop={8} accessibilityLabel="Remove receipt">
                  <Ionicons name="close" size={13} color={colors.onPrimary} />
                </Press>
              </View>
              <T variant="small" color={colors.textSecondary} style={{ flex: 1 }}>
                Receipt attached. It is saved with the expense.
              </T>
            </View>
          ) : null}
        </View>
      </Sheet>

      {/* Expense detail */}
      <Sheet visible={!!detail} onClose={() => setDetail(null)} title={detail?.title} subtitle={detail ? `${detail.category} · ${detail.
date}` : undefined}>
        {detail ? (
          <>
            <View style={styles.rowBetween}>
              <T variant="display" style={{ fontSize: 36, lineHeight: 40 }}>
                {fmt(detail.amount)}
              </T>
              {detail.original ? <Pill label={`Paid as ${money(detail.original.amount, detail.original.currency)}`} tone="blue" /> : nul
l}
            </View>
            <View style={styles.rowBetween}>
              <T variant="small" color={colors.textSecondary}>
                Paid by {memberById(trip, detail.paidBy)?.name}
              </T>
              <T variant="small" weight="bold">
```

```
              {fmt(detail.amount / Math.max(1, detail.splitBetween.length))} each
            </T>
          </View>
          <AvatarStack people={detail.splitBetween.map((id) => memberById(trip, id)).filter((m): m is NonNullable<typeof m> => !!m)} m
ax={6} size={30} />
          {detail.receipt ? (
            <View style={{ gap: 8 }}>
              <T variant="kicker" color={colors.textSecondary}>
                Receipt
              </T>
              <Press onPress={() => setReceiptBig((b) => !b)} scaleTo={0.99} accessibilityLabel={receiptBig ? 'Shrink receipt' : 'Enla
rge receipt'} style={{ alignSelf: receiptBig ? 'stretch' : 'flex-start' }}>
                <Image source={{ uri: detail.receipt }} style={receiptBig ? styles.receiptBig : styles.receiptSmall} contentFit={recei
ptBig ? 'contain' : 'cover'} accessibilityLabel="Receipt photo" />
              </Press>
            </View>
          ) : null}
          <Button
            label="Delete expense"
            variant="danger"
            icon="trash-outline"
            onPress={() => {
              const d = detail;
              setDetail(null);
              confirmAction('Delete expense?', `${d.title} (${money(d.amount)}) will be removed.`, () => {
                deleteExpense(trip.id, d.id);
                deleteLocalFile(d.receipt);
                toast('Expense deleted');
              }, { destructive: true, confirmLabel: 'Delete' });
            }}
          />
        </>
      ) : null}
    </Sheet>

    {/* Settle up */}
    <Sheet visible={settleOpen} onClose={() => setSettleOpen(false)} title="Settle up" subtitle="Fewest payments to make everyone eve
n">
      {plan.length ? (
        plan.map((p) => {
          const from = memberById(trip, p.from);
          const to = memberById(trip, p.to);
          return (
            <View key={`${p.from}-${p.to}`} style={styles.settleRow}>
              <Avatar name={from?.name ?? '?'} src={from?.avatar} size={30} />
              <Ionicons name="arrow-forward" size={14} color={colors.textMuted} />
              <Avatar name={to?.name ?? '?'} src={to?.avatar} size={30} />
              <View style={{ flex: 1 }}>
                <T variant="small" weight="semibold">
                  {from?.name.split(' ')[0]} pays {to?.name.split(' ')[0]}
                </T>
                <T variant="caption" color={colors.textSecondary}>
                  {money(p.amount, 'USD', { decimals: true })}
                </T>
              </View>
              <Button
                label="Mark paid"
                size="sm"
                full={false}
                onPress={() => {
                  settleUp(trip.id, p.from, p.to, p.amount);
                  tap('success');
                  toast('Payment recorded ✅');
                }}
              />
            </View>
          );
        })
      ) : (
        <View style={{ alignItems: 'center', gap: 8, paddingVertical: 20 }}>
          <Ionicons name="checkmark-circle" size={40} color={colors.positive} />
          <T variant="title">Everyone is square! 🎉</T>
        </View>
      )}
    </Sheet>
  </Screen>
);
}

const styles = StyleSheet.create({
  head: { flexDirection: 'row', alignItems: 'flex-end', gap: 12, marginTop: 12, marginBottom: 14 },
  plus: { width: 44, height: 44, borderRadius: 22, backgroundColor: colors.primary, alignItems: 'center', justifyContent: 'center', marg
inBottom: 4 },
  card: { borderRadius: radius.xl },
  donutRow: { flexDirection: 'row', alignItems: 'center', gap: 18 },
  legendRow: { flexDirection: 'row', alignItems: 'center', gap: 8 },
```

```
  dot: { width: 9, height: 9, borderRadius: 5 },
  tinyKicker: { fontSize: 10.5, letterSpacing: 0.6 },
  tiles: { flexDirection: 'row', gap: 12, marginTop: 12 },
  tile: { flex: 1, gap: 4, paddingHorizontal: 16, paddingVertical: 14 },
  sectionRow: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', gap: 8, marginTop: 24, marginBottom: 12 },
  section: { marginTop: 24, marginBottom: 12 },
  rowBetween: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', gap: 8 },
  balRow: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingHorizontal: 16, paddingVertical: 13 },
  balDivider: { borderTopWidth: 1, borderTopColor: colors.border },
  nameWrap: { flex: 1, flexDirection: 'row', alignItems: 'center', gap: 8 },
  badge: { backgroundColor: colors.accent, borderRadius: 999, paddingHorizontal: 8, paddingVertical: 2 },
  dailyRow: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  dailyInput: { width: 96, height: 46, borderRadius: radius.md, backgroundColor: colors.bgAlt, paddingHorizontal: 14, fontFamily: fonts.
semibold, fontSize: 17, color: colors.text, ...({ outlineStyle: 'none' } as object) },
  splitRow: { flexDirection: 'row', justifyContent: 'space-between' },
  splitCell: { alignItems: 'center', gap: 2 },
  fit: { flexDirection: 'row', gap: 8, alignItems: 'center', padding: 12, borderRadius: radius.md },
  findRow: { flexDirection: 'row', alignItems: 'center', gap: 10, paddingVertical: 4 },
  findIcon: { width: 30, height: 30, borderRadius: 15, backgroundColor: colors.bgAlt, alignItems: 'center', justifyContent: 'center' },
  capBox: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  capInput: { width: 80, height: 34, borderRadius: 999, borderWidth: 1, borderColor: colors.border, backgroundColor: colors.card, textAl
ign: 'center', fontFamily: fonts.bold, fontSize: 13, color: colors.text, ...({ outlineStyle: 'none' } as object) },
  expense: { flexDirection: 'row', alignItems: 'center', gap: 12, borderRadius: radius.lg, paddingHorizontal: 16, paddingVertical: 14 },
  expenseDot: { width: 11, height: 11, borderRadius: 6 },
  titleRow: { flexDirection: 'row', alignItems: 'center', gap: 5 },
  curToggle: { flexDirection: 'row', height: 50, borderRadius: radius.md, padding: 4, gap: 4, backgroundColor: colors.bgAlt },
  curBtn: { paddingHorizontal: 12, borderRadius: radius.sm, justifyContent: 'center' },
  curOn: { backgroundColor: colors.primary },
  receiptRow: { flexDirection: 'row', alignItems: 'center', gap: 10 },
  thumb: { width: 64, height: 64, borderRadius: radius.sm, backgroundColor: colors.bgAlt },
  thumbX: { position: 'absolute', top: -6, right: -6, width: 20, height: 20, borderRadius: 10, backgroundColor: colors.primary, alignIte
ms: 'center', justifyContent: 'center', borderWidth: 1.5, borderColor: colors.white },
  receiptSmall: { width: 96, height: 96, borderRadius: radius.md, backgroundColor: colors.bgAlt },
  receiptBig: { width: '100%', height: 380, borderRadius: radius.md, backgroundColor: colors.bgAlt },
  settleRow: { flexDirection: 'row', alignItems: 'center', gap: 8, padding: 12, borderRadius: radius.md, backgroundColor: colors.bgAlt
},
});
```

src/app/(tabs)/chat.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router, useFocusEffect } from 'expo-router';
import { useCallback, useEffect, useRef, useState } from 'react';
import { FlatList, KeyboardAvoidingView, Platform, ScrollView, StyleSheet, TextInput, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { NoTrip } from '@/components/trip/NoTrip';
import { Card, Chip, IconButton } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Press, tap } from '@/components/ui/Press';
import { Segmented } from '@/components/ui/Segmented';
import { T } from '@/components/ui/T';
import { useT } from '@/i18n';
import { answerLocally, askClaude, type AssistantContext } from '@/services/assistant';
import { getApiKey } from '@/services/secrets';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import type { ChatMessage, Trip } from '@/store/types';
import { MAX_WIDTH, colors, fonts, shadow } from '@/theme';

const QUICK = ['chat.q.budget', 'chat.q.food', 'chat.q.weather', 'chat.q.next'];
const AVATAR_COLORS = ['#E8533F', '#1E73E8', '#2FB863', '#E0A100', '#7C5CD6'];
const TABS = [
  { key: 'crew' as const, label: 'Crew' },
  { key: 'ai' as const, label: 'Scout AI' },
];

const inAiThread = (m: ChatMessage) => m.channel === 'ai' || m.ai === true;

/** Friendly canned replies so the demo group feels alive (offline). */
function friendReply(text: string): string {
  const q = text.toLowerCase();
  if (/food|eat|dinner|lunch|ramen|sushi|hungry/.test(q)) return 'I’m always down for food 😋 count me in!';
  if (/\?$/.test(q.trim())) return 'Good question — let’s ask the assistant or put it to a vote 🗳️';
  if (/late|delay|wait/.test(q)) return 'No stress, we’ll save you a seat 🙌';
  if (/photo|pic|camera/.test(q)) return 'Drop them in the photo dump! 📸';
  if (/money|pay|owe|\$/.test(q)) return 'I’ll log it in Budget so we can settle up later 💸';
  const pool = ['Love it 🔥', 'Works for me 👍', 'Yesss can’t wait!! ✈️', 'Adding it to my calendar 📅', 'Haha same 😂', 'Let’s goooo
🙌'];
  return pool[Math.floor(Math.random() * pool.length)];
}

/** Picks a random group member to reply (70% of the time) and a natural-feeling delay. */
function planFriendReply<M>(others: M[]): { friend: M; delay: number } | null {
  if (!others.length || Math.random() >= 0.7) return null;
  return { friend: others[Math.floor(Math.random() * others.length)], delay: 2600 + Math.random() * 1200 };
}

export default function Chat() {
  const trip = useActiveTrip();
  if (!trip) return <NoTrip />;
  return <ChatInner key={trip.id} />;
}

function ChatInner() {
  const t = useT();
  const insets = useSafeAreaInsets();
  const trip = useActiveTrip()!;
  const userId = useAppStore((s) => s.session?.userId ?? '');
  const send = useAppStore((s) => s.sendMessage);
  const aiMode = useAppStore((s) => s.settings.aiMode);
  const markKindRead = useAppStore((s) => s.markKindRead);
  const [text, setText] = useState('');
  const [tab, setTab] = useState<'crew' | 'ai'>('crew');
  const [typing, setTyping] = useState<string | null>(null);
  const list = useRef<FlatList<ChatMessage>>(null);
  const timers = useRef<ReturnType<typeof setTimeout>[]>([]);

  useFocusEffect(
    useCallback(() => {
      markKindRead('chat');
    }, [markKindRead]),
  );
  useEffect(() => () => timers.current.forEach(clearTimeout), []);

  const later = (fn: () => void, ms: number) => timers.current.push(setTimeout(fn, ms));

  const askAssistant = async (question: string, history: ChatMessage[]) => {
    setTyping('assistant');
    const s = useAppStore.getState();
    const live = s.trips.find((x) => x.id === trip.id) as Trip;
    const ctx: AssistantContext = { trip: live, userId, weather: s.weather[live.destinationId], rates: s.rates, homeCurrency: s.setting
s.homeCurrency };
    let reply = '';
    let source: 'local' | 'claude' = 'local';
```

```
    if (aiMode === 'claude') {
      const key = await getApiKey();
      if (key) {
        try {
          reply = await askClaude(key, question, history, ctx);
          source = 'claude';
        } catch {
          toast('Claude is unavailable — answered offline instead', { tone: 'warn', icon: 'cloud-offline-outline' });
        }
      }
    }
    if (!reply) {
      await new Promise((r) => setTimeout(r, 650));
      reply = answerLocally(question, ctx);
    }
    setTyping(null);
    send(trip.id, { authorId: 'assistant', ai: true, aiSource: source, channel: 'ai', text: reply });
    tap();
  };

  const submit = (raw?: string) => {
    const msg = (raw ?? text).trim();
    if (!msg) return;
    if (msg.length > 1000) return toast('Messages can be up to 1000 characters', { tone: 'warn' });
    setText('');
    if (tab === 'ai') {
      send(trip.id, { authorId: userId, text: msg, channel: 'ai' });
      askAssistant(msg, trip.messages.filter(inAiThread));
      return;
    }
    send(trip.id, { authorId: userId, text: msg });
    const plan = planFriendReply(trip.members.filter((m) => m.id !== userId && !m.invited));
    if (plan) {
      later(() => setTyping(plan.friend.id), 900);
      later(() => {
        setTyping(null);
        send(trip.id, { authorId: plan.friend.id, text: friendReply(msg) });
      }, plan.delay);
    }
  };

  const isAi = tab === 'ai';
  const data = trip.messages.filter((m) => inAiThread(m) === isAi);
  const author = (id: string) => trip.members.find((m) => m.id === id);
  const typingFor = typing === 'assistant' ? (isAi ? 'Scout AI' : null) : isAi ? null : typing ? author(typing)?.name.split(' ')[0] : nu
ll;

  const renderItem = ({ item, index }: { item: ChatMessage; index: number }) => {
    if (item.kind === 'system') {
      return (
        <View style={styles.system}>
          <T variant="caption" color={colors.textMuted} center>
            {item.text}
          </T>
        </View>
      );
    }
    const mine = item.authorId === userId;
    const prev = data[index - 1];
    const grouped = !!prev && prev.kind !== 'system' && prev.authorId === item.authorId && item.ts - prev.ts < 5 * 60000;
    const m = author(item.authorId);
    const memberIndex = trip.members.findIndex((x) => x.id === item.authorId);
    const avatarBg = memberIndex >= 0 ? AVATAR_COLORS[memberIndex % AVATAR_COLORS.length] : colors.textMuted;
    const name = item.ai ? 'Scout AI' : m?.name.split(' ')[0] ?? 'Former member';
    return (
      <View style={[styles.msgRow, mine && { flexDirection: 'row-reverse' }, grouped && { marginTop: 2 }]}>
        {!mine ? (
          <View style={{ width: 30 }}>
            {!grouped ? (
              item.ai ? (
                <View style={[styles.avatar, { backgroundColor: colors.black }]}>
                  <Ionicons name="sparkles" size={14} color={colors.white} />
                </View>
              ) : (
                <View style={[styles.avatar, { backgroundColor: avatarBg }]}>
                  <T variant="micro" weight="bold" color={colors.white} style={{ fontSize: 12 }}>
                    {(m?.name.trim()[0] ?? '?').toUpperCase()}
                  </T>
                </View>
              )
            ) : null}
          </View>
        ) : null}
        <View style={{ maxWidth: '78%', alignItems: mine ? 'flex-end' : 'flex-start' }}>
          {!grouped && !mine ? (
            <View style={styles.nameRow}>
```

```
              <T variant="micro" weight="semibold" color={colors.textSecondary}>
                {name}
              </T>
              {item.ai && item.aiSource === 'claude' ? (
                <T variant="micro" color={colors.purple}>
                  · Claude
                </T>
              ) : null}
            </View>
          ) : null}
          <View style={[styles.bubble, mine ? styles.mine : item.ai ? styles.ai : styles.theirs]}>
            <T variant="bodySm" color={mine ? colors.white : item.ai ? colors.onAccent : colors.text} selectable>
              {item.text}
            </T>
          </View>
        </View>
      </View>
    );
  };

  return (
    <View style={styles.root}>
      <KeyboardAvoidingView behavior={Platform.OS === 'ios' ? 'padding' : undefined} style={styles.column}>
        <View style={[styles.header, { paddingTop: insets.top + 12 }]}>
          <View style={styles.titleRow}>
            <T variant="display" numberOfLines={1} style={{ flex: 1 }}>
              {trip.name}
            </T>
            <IconButton icon="images-outline" label="Photo dump" onPress={() => router.push('/photos')} />
          </View>
          <Segmented options={TABS} value={tab} onChange={setTab} />
          {isAi ? (
            <Press onPress={() => router.push('/settings')} style={styles.modePill} accessibilityLabel="Assistant settings">
              <Ionicons name={aiMode === 'claude' ? 'sparkles' : 'cloud-offline-outline'} size={12} color={colors.text} />
              <T variant="micro" weight="semibold" color={colors.text}>
                {aiMode === 'claude' ? 'Answers by Claude · change in Settings' : 'Answers offline from your trip data · change in Setti
ngs'}
              </T>
            </Press>
          ) : null}
        </View>

        <FlatList
          key={tab}
          ref={list}
          data={data}
          keyExtractor={(m) => m.id}
          renderItem={renderItem}
          contentContainerStyle={{ paddingHorizontal: 16, paddingVertical: 12, flexGrow: 1 }}
          onContentSizeChange={() => list.current?.scrollToEnd({ animated: true })}
          onLayout={() => list.current?.scrollToEnd({ animated: false })}
          keyboardShouldPersistTaps="handled"
          ListEmptyComponent={
            isAi ? (
              <Card style={styles.intro}>
                <View style={[styles.avatar, styles.introIcon]}>
                  <Ionicons name="sparkles" size={20} color={colors.white} />
                </View>
                <T variant="headline">Scout AI</T>
                <T variant="small" color={colors.textSecondary} center>
                  Scout AI knows your trip — budget, weather, food nearby, who owes who, flights… Ask away or tap a question below.
                </T>
              </Card>
            ) : (
              <View style={{ alignItems: 'center', padding: 30, gap: 6 }}>
                <Ionicons name="chatbubbles-outline" size={34} color={colors.textMuted} />
                <T variant="title">Say hi to your crew 👋</T>
                <T variant="small" color={colors.textSecondary} center>
                  Messages here are shared with everyone on the trip. Switch to Scout AI to ask the assistant.
                </T>
              </View>
            )
          }
          ListFooterComponent={
            typingFor ? (
              <View style={styles.typing}>
                <T variant="caption" color={colors.textMuted}>
                  {t('chat.typing', { name: typingFor })}
                </T>
              </View>
            ) : null
          }
        />

        {isAi ? (
          <ScrollView horizontal showsHorizontalScrollIndicator={false} keyboardShouldPersistTaps="handled" contentContainerStyle={{ ga
```

```
p: 6, paddingHorizontal: 16, paddingBottom: 8 }} style={{ flexGrow: 0 }}>
            {QUICK.map((k) => (
              <Chip key={k} small label={t(k)} onPress={() => submit(t(k))} />
            ))}
            <Chip small label="Who owes who?" onPress={() => submit('Who owes who?')} />
            <Chip small label="Convert 50 USD" onPress={() => submit('Convert 50 USD')} />
          </ScrollView>
        ) : null}

        <View style={styles.inputBar}>
          <TextInput
            value={text}
            onChangeText={setText}
            placeholder={isAi ? 'Ask Scout anything…' : 'Message the crew…'}
            placeholderTextColor={colors.textMuted}
            style={styles.input}
            multiline
            maxLength={1000}
            accessibilityLabel="Message"
            onSubmitEditing={() => submit()}
            submitBehavior="submit"
            returnKeyType="send"
            testID="chat-input"
          />
          <Press onPress={() => submit()} style={[styles.send, !text.trim() && { opacity: 0.4 }]} disabled={!text.trim()} accessibilityL
abel="Send message" testID="chat-send">
            <Ionicons name="arrow-up" size={20} color={colors.white} />
          </Press>
        </View>
      </KeyboardAvoidingView>
    </View>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.bg, alignItems: 'center' },
  column: { flex: 1, width: '100%', maxWidth: MAX_WIDTH },
  header: { gap: 12, paddingHorizontal: 20, paddingBottom: 12, backgroundColor: colors.card, borderBottomWidth: 1, borderBottomColor: co
lors.border },
  titleRow: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  modePill: { alignSelf: 'flex-start', flexDirection: 'row', alignItems: 'center', gap: 4, paddingHorizontal: 10, paddingVertical: 6, bo
rderRadius: 999, backgroundColor: colors.bgAlt },
  system: { alignSelf: 'center', backgroundColor: colors.bgAlt, paddingHorizontal: 12, paddingVertical: 5, borderRadius: 999, marginVert
ical: 8 },
  msgRow: { flexDirection: 'row', alignItems: 'flex-end', gap: 8, marginTop: 12 },
  nameRow: { flexDirection: 'row', gap: 3, marginBottom: 3, marginHorizontal: 6 },
  avatar: { width: 30, height: 30, borderRadius: 15, alignItems: 'center', justifyContent: 'center' },
  bubble: { paddingHorizontal: 16, paddingVertical: 11, borderRadius: 20 },
  mine: { backgroundColor: colors.blue },
  theirs: { backgroundColor: colors.card, ...shadow },
  ai: { backgroundColor: colors.accentSoft, ...shadow },
  intro: { alignItems: 'center', gap: 8, paddingVertical: 24, marginTop: 8 },
  introIcon: { width: 48, height: 48, borderRadius: 24, backgroundColor: colors.black },
  typing: { paddingHorizontal: 44, paddingTop: 10 },
  inputBar: { flexDirection: 'row', alignItems: 'flex-end', gap: 8, paddingHorizontal: 16, paddingTop: 6, paddingBottom: 10 },
  input: {
    flex: 1,
    minHeight: 44,
    maxHeight: 120,
    borderRadius: 22,
    backgroundColor: colors.card,
    borderWidth: 1,
    borderColor: colors.border,
    paddingHorizontal: 18,
    paddingTop: 12,
    paddingBottom: 12,
    fontFamily: fonts.regular,
    fontSize: 15,
    color: colors.text,
    ...({ outlineStyle: 'none' } as object),
  },
  send: { width: 44, height: 44, borderRadius: 22, backgroundColor: colors.black, alignItems: 'center', justifyContent: 'center' },
});
```

src/app/(tabs)/explore.tsx

```
import { Ionicons } from '@expo/vector-icons';
import * as Clipboard from 'expo-clipboard';
import { Image } from 'expo-image';
import { LinearGradient } from 'expo-linear-gradient';
import { router } from 'expo-router';
import { useState } from 'react';
import { ScrollView, StyleSheet, View, useWindowDimensions } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { PlaceCard } from '@/components/explore';
import { GuidedTour } from '@/components/GuidedTour';
import { SkylineHero } from '@/components/SkylineHero';
import { placeCategoryIcon, typeEmoji } from '@/components/trip/meta';
import { TripMenuSheet } from '@/components/trip/TripMenuSheet';
import { AvatarStack, Card, IconButton, ProgressBar, SectionHeader, type IconName } from '@/components/ui/bits';
import { Button } from '@/components/ui/Button';
import { toast } from '@/components/ui/feedback';
import { FadeIn, Pulse } from '@/components/ui/motion';
import { Press, tap } from '@/components/ui/Press';
import { Slider } from '@/components/ui/Slider';
import { T } from '@/components/ui/T';
import { destinationById, destinations, type Destination } from '@/data/destinations';
import { imageSource } from '@/data/images';
import { INTERESTS } from '@/data/personal';
import { placesFor } from '@/data/places';
import { useNow, useWeather } from '@/hooks/useTrip';
import { useT } from '@/i18n';
import { nextUpcoming } from '@/services/notifications';
import { weatherInfo } from '@/services/weather';
import type { Personal } from '@/store/types';
import { totalSpent, tripStatus, useActiveTrip, useAppStore, useCurrentUser } from '@/store/useAppStore';
import { MAX_WIDTH, colors, radius, shadow } from '@/theme';
import { countdownLabel, lengthTag, money } from '@/utils/format';
import { distanceKm, kmLabel } from '@/utils/geo';
import { daysBetween, dayLabel, deviceTimeZone, greetingFor, partsInTz, time12, todayInTz, weekdayShort } from '@/utils/time';

/** Big hero headline that follows the trip's stage. */
function headlineFor(status: 'planning' | 'active' | 'completed' | null, daysToGo: number) {
  if (status === 'active') return 'Living it.';
  if (status === 'completed') return 'That was epic.';
  if (status === 'planning') return daysToGo <= 1 ? 'Tomorrow.' : `${daysToGo} days to go.`;
  return 'Where to next?';
}

export default function Home() {
  const t = useT();
  const insets = useSafeAreaInsets();
  const { width } = useWindowDimensions();
  const user = useCurrentUser();
  const trip = useActiveTrip();
  const unread = useAppStore((s) => s.notifications.filter((n) => !n.read).length);
  const personal = useAppStore((s) => s.personal);
  const setToured = useAppStore((s) => s.setToured);
  const updateActivity = useAppStore((s) => s.updateActivity);
  const now = useNow(30000);
  const [radiusKm, setRadiusKm] = useState(5);
  const [menu, setMenu] = useState(false);

  const dest = destinationById(trip?.destinationId ?? 'positano');
  const tz = dest?.tz ?? 'UTC';
  const weather = useWeather(dest?.id);
  const status = trip ? tripStatus(trip) : null;
  const next = nextUpcoming(trip);
  const show = personal.sections;
  const colW = Math.min(width, MAX_WIDTH) - 40;
  const today = todayInTz(tz);
  const tripLen = trip ? daysBetween(trip.startDate, trip.endDate) + 1 : 0;
  const daysToGo = trip ? Math.max(0, daysBetween(today, trip.startDate)) : 0;
  const dayNo = trip ? Math.min(tripLen, Math.max(1, daysBetween(trip.startDate, today) + 1)) : 0;
  const votingCount = trip ? trip.activities.filter((a) => a.status === 'voting').length : 0;
  const spent = trip ? totalSpent(trip) : 0;
  const firstName = personal.nickname || (user?.name.split(' ')[0] ?? 'traveller');
  const todays = trip && status === 'active' ? trip.activities.filter((a) => a.date === today && a.status !== 'voting').sort((a, b) =>
a.time.localeCompare(b.time)) : [];
  const picks = pickedForYou(personal);
  const homeTz = deviceTimeZone();
  const todayWx = weather?.daily[0];

  const nearby =
    dest && trip
      ? [trip.destinationId, ...trip.extraDestinationIds]
          .flatMap((id) => placesFor(id))
          .map((p) => ({ p, km: distanceKm(dest, p) }))
          .filter((x) => x.km <= radiusKm)
```

```
          .sort((a, b) => a.km - b.km)
      : [];

  const copyCode = async () => {
    if (!trip) return;
    await Clipboard.setStringAsync(trip.inviteCode).catch(() => {});
    tap('success');
    toast(`Invite code ${trip.inviteCode} copied`, { icon: 'copy-outline' });
  };

  const quick: { icon: IconName; label: string; go: () => void }[] = [
    { icon: 'checkbox-outline', label: `Vote (${votingCount})`, go: () => router.navigate({ pathname: '/(tabs)/itinerary', params: { ta
b: 'vote' } }) },
    { icon: 'airplane-outline', label: 'Flights', go: () => router.push('/flights') },
    { icon: 'folder-open-outline', label: 'Docs', go: () => router.push('/documents') },
    { icon: 'camera-outline', label: 'Photos', go: () => router.push('/photos') },
  ];

  return (
    <View style={styles.root}>
      <ScrollView showsVerticalScrollIndicator={false} contentContainerStyle={{ paddingBottom: 40 }}>
        <View style={styles.column}>
          <SkylineHero image={dest?.image ?? 'amalfi'} tz={tz} height={400 + insets.top}>
            <View style={[styles.heroTop, { top: insets.top + 10 }]}>
              <Press onPress={() => (trip ? setMenu(true) : router.push('/trips'))} style={styles.tripPill} accessibilityLabel={trip ? `
${trip.name}. Trip options and switching` : 'Choose a trip'}>
                <T variant="small" weight="bold" color={colors.white} numberOfLines={1} style={{ maxWidth: 190 }}>
                  {trip?.name ?? 'No trip yet'}
                </T>
                <Ionicons name="chevron-down" size={14} color={colors.white} />
              </Press>
              <View style={{ flex: 1 }} />
              <Press onPress={() => router.push({ pathname: '/panorama', params: { city: dest?.id } })} style={styles.pano} accessibilit
yLabel="Open the 360 degree city view">
                <Ionicons name="globe-outline" size={15} color={colors.white} />
                <T variant="micro" weight="extrabold" color={colors.white}>
                  360°
                </T>
              </Press>
              <IconButton icon="notifications-outline" label={`Notifications, ${unread} unread`} badge={unread} onPress={() => router.pu
sh('/notifications')} bg="rgba(255,255,255,0.92)" color="#111111" bordered={false} />
            </View>

            <View style={styles.heroBottom}>
              <View style={styles.place}>
                <Ionicons name="location-sharp" size={13} color="rgba(255,255,255,0.9)" />
                <T variant="kicker" color="rgba(255,255,255,0.92)">
                  {dest ? `${dest.city}, ${dest.country}` : 'Somewhere great'}
                </T>
              </View>
              <FadeIn delay={120} distance={26}>
                <T variant="display" color={colors.white} style={{ fontSize: 60, lineHeight: 64 }} accessibilityRole="header">
                  {headlineFor(status, daysToGo)}
                </T>
              </FadeIn>
              <View style={styles.heroRow}>
                {trip ? <AvatarStack people={trip.members} max={4} size={32} /> : <View />}
                {trip ? (
                  <Press onPress={copyCode} style={styles.code} accessibilityLabel={`Invite code ${trip.inviteCode}. Tap to copy`}>
                    <T variant="small" weight="extrabold" color={colors.onAccent}>
                      Code {trip.inviteCode}
                    </T>
                    <Ionicons name="copy-outline" size={14} color={colors.onAccent} />
                  </Press>
                ) : null}
              </View>
            </View>
          </SkylineHero>

          <View style={styles.sheet}>
            <T variant="small" color={colors.textSecondary} style={{ marginBottom: 14 }}>
              {t(greetingFor(new Date(now).getHours()))}, {firstName} {personal.emoji}
            </T>

            {!trip ? (
              <Card style={{ gap: 12, marginBottom: 14 }}>
                <T variant="headline">Plan your first group trip</T>
                <T variant="small" color={colors.textSecondary}>
                  Create a trip, invite friends with a code and start planning together.
                </T>
                <View style={{ flexDirection: 'row', gap: 8 }}>
                  <Button label="Create trip" size="md" icon="add" onPress={() => router.push('/new-trip')} style={{ flex: 1 }} />
                  <Button label="Join" size="md" variant="secondary" icon="people-outline" onPress={() => router.push('/join')} style={{
flex: 1 }} />
                </View>
              </Card>
```

```
            ) : null}

            {show.glance ? (
              <FadeIn delay={80}>
                <View style={styles.tiles}>
                  <Press onPress={() => router.push('/weather')} style={styles.tile} accessibilityLabel={`Weather in ${dest?.city}`}>
                    <T variant="kicker" color={colors.textSecondary}>
                      Weather
                    </T>
                    <View style={styles.tileRow}>
                      <Ionicons name={weather ? weatherInfo(weather.current.code).icon : 'partly-sunny'} size={34} color="#F5A524" />
                      <T variant="h1" weight="extrabold" style={{ fontSize: 36, lineHeight: 40 }}>
                        {weather ? `${weather.current.temp}°` : '—'}
                      </T>
                    </View>
                    <T variant="micro" color={colors.textSecondary}>
                      {todayWx ? `H ${Math.round(todayWx.max)}° · L ${Math.round(todayWx.min)}°` : 'Loading forecast…'}
                    </T>
                  </Press>
                  <View style={styles.tile} accessible accessibilityLabel={`Local time in ${dest?.city}`}>
                    <T variant="kicker" color={colors.textSecondary}>
                      Local time
                    </T>
                    <T variant="h1" weight="extrabold" style={{ marginTop: 6 }}>
                      {time12(partsInTz(now, tz).hhmm)}
                    </T>
                    <T variant="micro" color={colors.textSecondary} style={{ marginTop: 4 }}>
                      Home {time12(partsInTz(now, homeTz).hhmm)}
                    </T>
                  </View>
                </View>
              </FadeIn>
            ) : null}

            {trip && show.budget ? (
              <FadeIn delay={140}>
                <Press onPress={() => router.navigate('/(tabs)/budget')} style={styles.budget} accessibilityLabel="Group budget. Open bu
dget">
                  <View style={styles.rowBetween}>
                    <T variant="kicker" color={colors.textSecondary}>
                      Group budget
                    </T>
                    <Ionicons name="chevron-forward" size={16} color={colors.textMuted} />
                  </View>
                  <View style={{ flexDirection: 'row', alignItems: 'baseline', gap: 6, marginTop: 4 }}>
                    <T variant="h1" weight="extrabold">
                      {money(spent)}
                    </T>
                    <T variant="small" color={colors.textSecondary}>
                      of {money(trip.budget)}
                    </T>
                  </View>
                  <View style={{ marginTop: 12 }}>
                    <ProgressBar value={spent / Math.max(1, trip.budget)} color={spent > trip.budget ? colors.red : colors.primary} heig
ht={9} />
                  </View>
                </Press>
              </FadeIn>
            ) : null}

            {trip && show.upnext ? (
              <FadeIn delay={200}>
                <SectionHeader title="Up next" action="See all" onAction={() => router.navigate('/(tabs)/itinerary')} style={{ marginTo
p: 24 }} />
                {status === 'active' ? (
                  <Card style={{ marginBottom: 10, gap: 10 }}>
                    <View style={styles.rowBetween}>
                      <T variant="title" weight="bold">
                        Today · Day {dayNo} of {tripLen}
                      </T>
                      <T variant="caption" color={colors.textSecondary}>
                        {todays.filter((a) => a.done).length}/{todays.length} done
                      </T>
                    </View>
                    {todays.length ? (
                      todays.map((a) => (
                        <Press key={a.id} onPress={() => updateActivity(trip.id, a.id, { done: !a.done })} style={styles.todayRow} acces
sibilityRole="checkbox" accessibilityState={{ checked: !!a.done }} accessibilityLabel={`${a.title} at ${a.time}`}>
                          <Ionicons name={a.done ? 'checkmark-circle' : 'ellipse-outline'} size={22} color={a.done ? colors.positive : c
olors.borderStrong} />
                          <T variant="caption" weight="bold" color={colors.textSecondary} style={{ width: 40 }}>
                            {a.time}
                          </T>
                          <T variant="small" weight="semibold" numberOfLines={1} style={[{ flex: 1 }, a.done && { textDecorationLine: 'l
ine-through', color: colors.textMuted }]}>
                            {a.title}
```

```
                      </T>
                    </Press>
                  ))
                ) : (
                  <T variant="small" color={colors.textSecondary}>
                    A free day. Enjoy it!
                  </T>
                )}
              </Card>
            ) : null}
            {next ? (
              <Press onPress={() => router.navigate('/(tabs)/itinerary')} style={styles.next} accessibilityLabel={`Next: ${next.a.ti
tle}`}>
                <View style={styles.nextBadge}>
                  <T style={{ fontSize: 22, lineHeight: 28 }}>{typeEmoji[next.a.type]}</T>
                </View>
                <View style={{ flex: 1, gap: 2 }}>
                  <T variant="small" weight="bold" numberOfLines={1}>
                    {next.a.title}
                  </T>
                  <T variant="caption" color={colors.textSecondary} numberOfLines={1}>
                    {weekdayShort(next.a.date)}, {dayLabel(next.a.date)} · {time12(next.a.time)} · {lengthTag(next.a.durationMin)}
                  </T>
                </View>
                {next.a.reminder ? (
                  <Pulse>
                    <Ionicons name="notifications" size={16} color={colors.primary} />
                  </Pulse>
                ) : null}
                <View style={styles.countdown}>
                  <T variant="micro" weight="bold" color={colors.onPrimary}>
                    {next.startUtc - now > 0 ? `in ${countdownLabel(next.startUtc - now)}` : 'Now'}
                  </T>
                </View>
              </Press>
            ) : (
              <Card>
                <T variant="small" color={colors.textSecondary}>
                  Nothing scheduled yet.
                </T>
              </Card>
            )}
          </FadeIn>
        ) : null}

        {trip && show.quick ? (
          <FadeIn delay={260}>
            <View style={styles.quickRow}>
              {quick.map((q) => (
                <Press key={q.label} onPress={q.go} style={styles.quick} accessibilityLabel={q.label}>
                  <Ionicons name={q.icon} size={22} color={colors.text} />
                  <T variant="micro" weight="bold" numberOfLines={1}>
                    {q.label}
                  </T>
                </Press>
              ))}
            </View>
          </FadeIn>
        ) : null}

        {show.forYou ? (
          personal.done && picks.length ? (
            <FadeIn delay={300}>
              <SectionHeader title="Picked for you" action="Edit" onAction={() => router.push('/personalize')} style={{ marginTop: 2
6 }} />
              <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 12, paddingHorizontal: 20
}} style={{ marginHorizontal: -20 }}>
                {picks.map(({ d, match, why }) => (
                  <Press key={d.id} onPress={() => router.push({ pathname: '/destination/[id]', params: { id: d.id } })} style={styl
es.pick} accessibilityLabel={`${d.city}, ${match}% match`}>
                    <Image source={imageSource(d.image)} style={StyleSheet.absoluteFill} contentFit="cover" transition={200} />
                    <LinearGradient colors={['rgba(0,0,0,0)', 'rgba(0,0,0,0.75)']} locations={[0.35, 1]} style={StyleSheet.absoluteF
ill} />
                    <View style={styles.match}>
                      <T variant="micro" weight="bold" color={colors.onAccent}>
                        {match}% match
                      </T>
                    </View>
                    <View style={{ position: 'absolute', left: 12, right: 12, bottom: 12 }}>
                      <T variant="title" weight="bold" color={colors.white} numberOfLines={1}>
                        {d.city}
                      </T>
                      <T variant="micro" color="rgba(255,255,255,0.9)" numberOfLines={1}>
                        {why}
                      </T>
                    </View>
```

```
                    </Press>
                  ))}
                </ScrollView>
              </FadeIn>
            ) : !personal.done ? (
              <Press onPress={() => router.push('/personalize')} style={styles.cta} accessibilityLabel="Make TogetherWeGo yours — take
the 60-second survey">
                <LinearGradient colors={[colors.accent, '#F59E0B']} start={{ x: 0, y: 0 }} end={{ x: 1, y: 1 }} style={StyleSheet.abso
luteFill} />
                <View style={styles.ctaEmoji}>
                  <T style={{ fontSize: 24, lineHeight: 30 }}>🎨</T>
                </View>
                <View style={{ flex: 1 }}>
                  <T variant="title" weight="extrabold" color={colors.onAccent}>
                    Make it yours
                  </T>
                  <T variant="caption" color={colors.onAccent}>
                    60-sec survey · colours, interests & your home screen
                  </T>
                </View>
                <Ionicons name="arrow-forward-circle" size={28} color={colors.onAccent} />
              </Press>
            ) : null}
          ) : null}

          {trip && dest && show.nearby ? (
            <FadeIn delay={340}>
              <SectionHeader title="Nearby" action="Filters" onAction={() => router.push('/nearby')} style={{ marginTop: 26 }} />
              <Card>
                <View style={styles.rowBetween}>
                  <T variant="small" color={colors.textSecondary}>
                    Search radius
                  </T>
                  <T variant="small" weight="extrabold">
                    {radiusKm} km
                  </T>
                </View>
                <Slider value={radiusKm} min={1} max={20} onChange={setRadiusKm} label="Search radius in kilometres" formatValue={(v)
=> `${v} km`} />
                <T variant="caption" color={colors.textMuted} style={{ marginBottom: 6 }}>
                  {nearby.length} places within {radiusKm} km of central {dest.city}
                </T>
                {nearby.slice(0, 4).map(({ p, km }) => (
                  <Press key={p.id} onPress={() => router.push({ pathname: '/nearby', params: { focus: p.id } })} style={styles.nearRo
w} scaleTo={0.99}>
                    <View style={styles.nearIcon}>
                      <Ionicons name={placeCategoryIcon[p.category]} size={15} color={colors.text} />
                    </View>
                    <View style={{ flex: 1 }}>
                      <T variant="small" weight="bold" numberOfLines={1}>
                        {p.name}
                      </T>
                      <T variant="caption" color={colors.textSecondary} numberOfLines={1}>
                        {p.category[0].toUpperCase() + p.category.slice(1)} · {lengthTag(p.durationMin)} · {p.cost ? money(p.cost) :
'Free'}
                      </T>
                    </View>
                    <T variant="caption" weight="semibold" color={colors.textSecondary}>
                      {kmLabel(km)}
                    </T>
                  </Press>
                ))}
              </Card>
            </FadeIn>
          ) : null}

          {show.discover ? (
            <FadeIn delay={380}>
              <SectionHeader title="Discover" action="See all" onAction={() => router.push('/destinations')} style={{ marginTop: 26 }}
/>
              <View style={styles.grid}>
                {destinations.filter((d) => d.id !== dest?.id).slice(0, 4).map((d) => (
                  <PlaceCard key={d.id} d={d} style={{ width: (colW - 12) / 2 }} />
                ))}
              </View>
            </FadeIn>
          ) : null}

          <Press onPress={() => router.push('/personalize')} style={styles.customize} accessibilityLabel="Customize this page">
            <Ionicons name="color-palette-outline" size={16} color={colors.text} />
            <T variant="small" weight="bold">
              Customize this page
            </T>
          </Press>
        </View>
      </View>
```

```
      </ScrollView>

      {trip ? <TripMenuSheet visible={menu} onClose={() => setMenu(false)} trip={trip} /> : null}
      <GuidedTour visible={!personal.toured && !!trip} onClose={() => setToured(true)} />
    </View>
  );
}

/** Ranks destinations against the “Make it yours” answers (interests + budget style). */
function pickedForYou(p: Personal): { d: Destination; match: number; why: string }[] {
  if (!p.interests.length) return [];
  return destinations
    .map((d) => {
      const hits = INTERESTS.filter((i) => p.interests.includes(i.key) && d.tags.includes(i.key));
      const budgetFit = p.budgetStyle === 'saver' ? (d.priceLevel <= 2 ? 1 : 0) : p.budgetStyle === 'treat' ? (d.priceLevel >= 3 ? 1 :
0) : d.priceLevel <= 3 ? 1 : 0;
      const ratio = hits.length / Math.min(3, p.interests.length);
      const match = Math.min(99, Math.round(52 + 36 * Math.min(1, ratio) + 7 * budgetFit + (d.rating - 4.5) * 8));
      return { d, match, hits, why: `${hits.map((h) => h.emoji).join(' ')} · ${d.country}` };
    })
    .filter((x) => x.hits.length > 0)
    .sort((a, b) => b.match - a.match || b.d.rating - a.d.rating)
    .slice(0, 8);
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.bg, alignItems: 'center' },
  column: { width: '100%', maxWidth: MAX_WIDTH },
  heroTop: { position: 'absolute', left: 16, right: 16, flexDirection: 'row', alignItems: 'center', gap: 10 },
  pano: { flexDirection: 'row', alignItems: 'center', gap: 5, height: 40, paddingHorizontal: 12, borderRadius: 999, backgroundColor: 'rg
ba(20,20,24,0.55)' },
  tripPill: { flexDirection: 'row', alignItems: 'center', gap: 6, height: 40, paddingHorizontal: 14, borderRadius: 999, backgroundColor:
'rgba(20,20,24,0.55)' },
  heroBottom: { position: 'absolute', left: 20, right: 20, bottom: 44 },
  place: { flexDirection: 'row', alignItems: 'center', gap: 4, marginBottom: 2 },
  heroRow: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', marginTop: 12 },
  code: { flexDirection: 'row', alignItems: 'center', gap: 8, height: 38, paddingHorizontal: 14, borderRadius: 999, backgroundColor: col
ors.accent },
  sheet: { marginTop: -28, borderTopLeftRadius: 28, borderTopRightRadius: 28, backgroundColor: colors.bg, paddingHorizontal: 20, padding
Top: 22 },
  tiles: { flexDirection: 'row', gap: 12 },
  tile: { flex: 1, backgroundColor: colors.card, borderRadius: radius.xl, padding: 16, minHeight: 112, ...shadow },
  tileRow: { flexDirection: 'row', alignItems: 'center', gap: 8, marginTop: 6 },
  budget: { marginTop: 12, backgroundColor: colors.card, borderRadius: radius.xl, padding: 16, ...shadow },
  rowBetween: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', gap: 8 },
  todayRow: { flexDirection: 'row', alignItems: 'center', gap: 10, paddingVertical: 2 },
  next: { flexDirection: 'row', alignItems: 'center', gap: 12, backgroundColor: colors.card, borderRadius: radius.xl, padding: 14, ...sh
adow },
  nextBadge: { width: 46, height: 46, borderRadius: 23, backgroundColor: colors.bgAlt, alignItems: 'center', justifyContent: 'center' },
  countdown: { backgroundColor: colors.primary, paddingHorizontal: 10, paddingVertical: 6, borderRadius: 999 },
  quickRow: { flexDirection: 'row', gap: 10, marginTop: 14 },
  quick: { flex: 1, height: 84, backgroundColor: colors.card, borderRadius: radius.xl, alignItems: 'center', justifyContent: 'center', g
ap: 8, ...shadow },
  pick: { width: 156, height: 196, borderRadius: radius.xl, overflow: 'hidden', ...shadow },
  match: { position: 'absolute', top: 10, left: 10, backgroundColor: colors.accent, paddingHorizontal: 8, paddingVertical: 4, borderRadi
us: 999 },
  cta: { marginTop: 22, flexDirection: 'row', alignItems: 'center', gap: 12, padding: 14, borderRadius: radius.xl, overflow: 'hidden',
...shadow },
  ctaEmoji: { width: 44, height: 44, borderRadius: 22, backgroundColor: 'rgba(255,255,255,0.4)', alignItems: 'center', justifyContent:
'center' },
  nearRow: { flexDirection: 'row', alignItems: 'center', gap: 10, paddingVertical: 9, borderTopWidth: 1, borderTopColor: colors.border
},
  nearIcon: { width: 32, height: 32, borderRadius: 16, backgroundColor: colors.bgAlt, alignItems: 'center', justifyContent: 'center' },
  grid: { flexDirection: 'row', flexWrap: 'wrap', gap: 12 },
  customize: { flexDirection: 'row', alignItems: 'center', justifyContent: 'center', gap: 6, marginTop: 26, paddingVertical: 13, borderR
adius: 999, borderWidth: 1, borderStyle: 'dashed', borderColor: colors.borderStrong },
});
```

src/app/(tabs)/itinerary.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useLocalSearchParams } from 'expo-router';
import { useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';

import { ActivityCard } from '@/components/trip/ActivityCard';
import { AddActivitySheet } from '@/components/trip/AddActivitySheet';
import { MemberPickerSheet } from '@/components/trip/MemberPickerSheet';
import { activityFilters, typeEmoji } from '@/components/trip/meta';
import { NoTrip } from '@/components/trip/NoTrip';
import { Chip, EmptyState, IconButton } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Press } from '@/components/ui/Press';
import { Screen } from '@/components/ui/Screen';
import { Segmented } from '@/components/ui/Segmented';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { useT } from '@/i18n';
import { printPlanner } from '@/services/print';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import type { Activity, ActivityType } from '@/store/types';
import { colors } from '@/theme';
import { LENGTH_FILTERS, matchesLength, type LengthFilter } from '@/utils/format';
import { dayLabel, todayInTz, tripDays, tzAbbrev, utcOffsetLabel } from '@/utils/time';

export default function Plan() {
  const trip = useActiveTrip();
  if (!trip) return <NoTrip />;
  return <PlanInner key={trip.id} />;
}

function PlanInner() {
  const t = useT();
  const trip = useActiveTrip()!;
  const params = useLocalSearchParams<{ tab?: string }>();
  const toggleAssign = useAppStore((s) => s.toggleAssign);
  const dest = destinationById(trip.destinationId);
  const tz = dest?.tz ?? 'UTC';
  const days = tripDays(trip.startDate, trip.endDate);
  const today = todayInTz(tz);

  const [tab, setTab] = useState<'itinerary' | 'vote'>(params.tab === 'vote' ? 'vote' : 'itinerary');
  const [seenTab, setSeenTab] = useState(params.tab);
  // Home’s “Vote” tile navigates here with ?tab=vote while this screen is already mounted.
  if (params.tab !== seenTab) {
    setSeenTab(params.tab);
    if (params.tab) setTab(params.tab === 'vote' ? 'vote' : 'itinerary');
  }
  const [day, setDay] = useState<'all' | string>('all');
  const [cat, setCat] = useState<'all' | ActivityType>('all');
  const [len, setLen] = useState<LengthFilter>('any');
  const [addOpen, setAddOpen] = useState(false);
  const [editing, setEditing] = useState<Activity | null>(null);
  const [assigning, setAssigning] = useState<Activity | null>(null);

  const selectedDay = day === 'all' || days.includes(day) ? day : 'all';
  const matches = (a: Activity) => (selectedDay === 'all' || a.date === selectedDay) && (cat === 'all' || a.type === cat) && matchesLeng
th(a.durationMin, len);
  const byTime = (a: Activity, b: Activity) => (a.date + a.time).localeCompare(b.date + b.time);
  const voting = trip.activities.filter((a) => a.status === 'voting');
  const shown = trip.activities.filter((a) => (tab === 'vote' ? a.status === 'voting' : a.status !== 'voting') && matches(a)).sort(byTim
e);
  const assigningLive = assigning ? trip.activities.find((a) => a.id === assigning.id) ?? null : null;

  const print = async () => {
    try {
      await printPlanner(trip);
    } catch {
      toast('Allow pop-ups to print the planner', { tone: 'warn' });
    }
  };

  return (
    <Screen
      bottomInset={30}
      header={
        <View style={styles.header}>
          <View style={{ flex: 1 }}>
            <T variant="kicker" color={colors.textSecondary} numberOfLines={1}>
              {dest ? `${dest.city}, ${dest.country}` : trip.name}
            </T>
            <T variant="display" accessibilityRole="header">
              The plan
            </T>
```

```
          </View>
          <IconButton icon="print-outline" label="Print planner" onPress={print} size={44} />
          <IconButton
            icon="add"
            label="Add activity"
            onPress={() => {
              setEditing(null);
              setAddOpen(true);
            }}
            size={44}
            bg={colors.primary}
            color={colors.onPrimary}
            bordered={false}
          />
        </View>
      }>
      <Segmented
        value={tab}
        onChange={setTab}
        options={[
          { key: 'itinerary', label: 'Itinerary' },
          { key: 'vote', label: `Vote (${voting.length})` },
        ]}
      />

      <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={styles.chips} style={styles.chipScroll}>
        <Chip label="All days" active={selectedDay === 'all'} onPress={() => setDay('all')} />
        {days.map((d, i) => (
          <Chip key={d} label={`Day ${i + 1} · ${dayLabel(d)}`} active={selectedDay === d} onPress={() => setDay(d)} />
        ))}
      </ScrollView>
      <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={styles.chips} style={styles.chipScroll}>
        {activityFilters.map((f) => (
          <Chip key={f.key} small label={f.key === 'all' ? t(f.labelKey) : `${typeEmoji[f.key]} ${t(f.labelKey)}`} active={cat === f.ke
y} onPress={() => setCat(f.key)} />
        ))}
      </ScrollView>
      <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={styles.chips} style={styles.chipScroll}>
        {LENGTH_FILTERS.map((f) => (
          <Chip key={f.key} small icon={f.key === 'any' ? undefined : 'timer-outline'} label={t(f.labelKey)} active={len === f.key} onPr
ess={() => setLen(f.key)} />
        ))}
      </ScrollView>

      <View style={styles.tzRow}>
        <Ionicons name="globe-outline" size={13} color={colors.textMuted} />
        <T variant="caption" color={colors.textMuted}>
          Times are in {dest?.city ?? 'local'} time ({tzAbbrev(tz)}, {utcOffsetLabel(tz)}){days.includes(today) ? ' · today is on the tr
ip' : ''}
        </T>
      </View>

      <View style={{ marginTop: 10 }}>
        {shown.length ? (
          shown.map((a) => (
            <ActivityCard
              key={a.id}
              trip={trip}
              activity={a}
              mode={tab === 'vote' ? 'vote' : 'plan'}
              onEdit={(x) => {
                setEditing(x);
                setAddOpen(true);
              }}
              onAssign={setAssigning}
            />
          ))
        ) : (
          <EmptyState
            icon={tab === 'vote' ? 'thumbs-up-outline' : 'calendar-outline'}
            title={tab === 'vote' ? 'Nothing to vote on' : (day !== 'all' || cat !== 'all' || len !== 'any') && trip.activities.length ?
t('itin.noMatch') : t('itin.noActivities')}
            body={tab === 'vote' ? 'Add an activity and mark it “voting” — the crew decides together.' : (day !== 'all' || cat !== 'all'
|| len !== 'any') && trip.activities.length ? 'Try a different day, category or length.' : t('itin.noActivitiesSub')}
          />
        )}
      </View>

      <Press
        onPress={() => {
          setEditing(null);
          setAddOpen(true);
        }}
        style={styles.addRow}
        accessibilityLabel={t('itin.addActivity')}
        testID="add-activity">
```

```
        <Ionicons name="add-circle" size={20} color={colors.primary} />
        <T variant="small" weight="bold" color={colors.primary}>
          {t('itin.addActivity')}
        </T>
      </Press>

      <AddActivitySheet visible={addOpen} onClose={() => setAddOpen(false)} trip={trip} date={selectedDay === 'all' ? days[0] : selected
Day} editing={editing} />
      <MemberPickerSheet visible={!!assigning} onClose={() => setAssigning(null)} members={trip.members} selected={assigningLive?.assign
ed ?? []} onToggle={(id) => assigning && toggleAssign(trip.id, assigning.id, id)} />
    </Screen>
  );
}

const styles = StyleSheet.create({
  header: { flexDirection: 'row', alignItems: 'flex-end', gap: 10, paddingHorizontal: 20, paddingTop: 8, paddingBottom: 14 },
  chipScroll: { marginHorizontal: -20, marginTop: 12, flexGrow: 0 },
  chips: { gap: 8, paddingHorizontal: 20 },
  tzRow: { flexDirection: 'row', alignItems: 'center', gap: 6, marginTop: 14 },
  addRow: { flexDirection: 'row', alignItems: 'center', justifyContent: 'center', gap: 6, paddingVertical: 12, marginTop: 4 },
});
```

src/app/(tabs)/profile.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router } from 'expo-router';
import { StyleSheet, View } from 'react-native';

import { Avatar, Card, type IconName } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Press } from '@/components/ui/Press';
import { Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { useT } from '@/i18n';
import { printPlanner } from '@/services/print';
import { useActiveTrip, useCurrentUser } from '@/store/useAppStore';
import { colors, radius } from '@/theme';

type MenuRow = { icon: IconName; title: string; sub?: string; href?: string; onPress?: () => void };

export default function More() {
  const t = useT();
  const trip = useActiveTrip();
  const user = useCurrentUser();

  const printTrip = async () => {
    if (!trip) return toast('Create or join a trip to print its planner', { tone: 'info' });
    try {
      await printPlanner(trip);
    } catch {
      toast('Allow pop-ups to print the planner', { tone: 'warn' });
    }
  };

  const groups: { title: string; rows: MenuRow[] }[] = [
    {
      title: 'Trip',
      rows: [
        { icon: 'people-outline', title: 'Crew & roles', sub: 'Invite code, roles, votes & checklist', href: '/crew' },
        { icon: 'airplane-outline', title: 'Flights & arrivals', sub: 'Everyone’s arrival & departure times', href: '/flights' },
        { icon: 'swap-horizontal-outline', title: 'My trips', sub: 'Switch, create or join a trip', href: '/trips' },
        { icon: 'trophy-outline', title: 'Trip recap', sub: 'Stats, highlights & share card', href: '/recap' },
      ],
    },
    {
      title: 'Get around',
      rows: [
        { icon: 'map-outline', title: 'Trip Map', sub: 'Pins + nearest airport', href: '/map' },
        { icon: 'globe-outline', title: '360° City Views', sub: 'Day & night skylines with the lights on', href: '/panorama' },
        { icon: 'compass-outline', title: 'Explore Nearby', sub: 'Food & sights by radius', href: '/nearby' },
        { icon: 'partly-sunny-outline', title: 'Weather', sub: '7-day forecast', href: '/weather' },
        { icon: 'cash-outline', title: 'Currency', sub: 'Live converter', href: '/currency' },
        { icon: 'locate-outline', title: 'Nearest airport', sub: 'Based on your location', href: '/airport' },
      ],
    },
    {
      title: 'Keep it safe',
      rows: [
        { icon: 'folder-open-outline', title: 'Offline Docs', sub: 'Passports, bookings — no wifi needed', href: '/documents' },
        { icon: 'camera-outline', title: 'Photo Dump', sub: 'Everyone’s shots in one place', href: '/photos' },
        { icon: 'print-outline', title: 'Printable Planner', sub: 'Paper itinerary', onPress: printTrip },
        { icon: 'medkit-outline', title: 'Emergency', sub: 'Local numbers & safety', href: '/emergency' },
        { icon: 'briefcase-outline', title: 'Packing list', sub: 'Smart suggestions', href: '/packing' },
      ],
    },
    {
      title: 'Fun',
      rows: [
        { icon: 'star-outline', title: 'Bucket List', sub: 'Must-dos for the trip', href: '/bucket-list' },
        { icon: 'game-controller-outline', title: 'Trip Quiz', sub: 'Test the crew', href: '/quiz' },
        { icon: 'sparkles-outline', title: 'Discover', sub: 'Survey → your dream destination', href: '/survey' },
        { icon: 'language-outline', title: 'Phrasebook', sub: 'Speak the basics', href: '/phrasebook' },
        { icon: 'color-palette-outline', title: 'Make it yours', sub: 'Themes, interests & home screen', href: '/personalize' },
        { icon: 'settings-outline', title: 'Language & Settings', sub: '6 languages', href: '/settings' },
      ],
    },
    {
      title: 'Help',
      rows: [
        { icon: 'bulb-outline', title: 'How it works', sub: 'A tour of the app', href: '/how-it-works' },
        { icon: 'help-circle-outline', title: 'Help & FAQ', href: '/help' },
        { icon: 'information-circle-outline', title: 'About', href: '/about' },
      ],
    },
  ];

  return (
    <Screen bottomInset={30}>
```

```
      <View style={styles.head}>
        {trip ? (
          <T variant="kicker" color={colors.textSecondary} numberOfLines={1}>
            {trip.name}
          </T>
        ) : null}
        <T variant="display" accessibilityRole="header">
          {t('nav.profile')}
        </T>
      </View>

      <Card padded={false} style={styles.card}>
        <Press onPress={() => router.push('/settings')} style={styles.profile} scaleTo={0.99} accessibilityLabel="Open your profile and
settings">
          <Avatar name={user?.name ?? 'You'} src={user?.avatar} size={52} />
          <View style={{ flex: 1 }}>
            <T variant="title" weight="bold" numberOfLines={1}>
              {user?.name ?? 'You'}
            </T>
            <T variant="caption" color={colors.textSecondary} numberOfLines={1}>
              {[user?.email, user?.homeCity].filter(Boolean).join(' · ')}
            </T>
          </View>
          <Ionicons name="chevron-forward" size={16} color={colors.textMuted} />
        </Press>
      </Card>

      {groups.map((g) => (
        <View key={g.title}>
          <T variant="kicker" color={colors.textSecondary} style={styles.kicker}>
            {g.title}
          </T>
          <Card padded={false} style={styles.card}>
            {g.rows.map((r, i) => (
              <Press
                key={r.title}
                onPress={() => (r.onPress ? r.onPress() : router.push(r.href as never))}
                style={[styles.row, i > 0 && styles.line]}
                scaleTo={0.99}
                accessibilityLabel={r.sub ? `${r.title}. ${r.sub}` : r.title}>
                <View style={styles.icon}>
                  <Ionicons name={r.icon} size={19} color={colors.text} />
                </View>
                <View style={{ flex: 1 }}>
                  <T variant="title" weight="bold">
                    {r.title}
                  </T>
                  {r.sub ? (
                    <T variant="caption" color={colors.textSecondary} numberOfLines={1}>
                      {r.sub}
                    </T>
                  ) : null}
                </View>
                <Ionicons name="chevron-forward" size={16} color={colors.textMuted} />
              </Press>
            ))}
          </Card>
        </View>
      ))}
    </Screen>
  );
}

const styles = StyleSheet.create({
  head: { marginTop: 12, marginBottom: 14 },
  card: { borderRadius: radius.xl },
  profile: { flexDirection: 'row', alignItems: 'center', gap: 14, paddingHorizontal: 16, paddingVertical: 14 },
  kicker: { marginTop: 24, marginBottom: 8, marginLeft: 6 },
  row: { flexDirection: 'row', alignItems: 'center', gap: 14, paddingHorizontal: 16, paddingVertical: 13 },
  line: { borderTopWidth: 1, borderTopColor: colors.border },
  icon: { width: 40, height: 40, borderRadius: 20, backgroundColor: colors.bgAlt, alignItems: 'center', justifyContent: 'center' },
});
```

src/app/_layout.tsx

```
import { Anton_400Regular } from '@expo-google-fonts/anton';
import { Manrope_400Regular, Manrope_500Medium, Manrope_600SemiBold, Manrope_700Bold, Manrope_800ExtraBold, useFonts } from '@expo-googl
e-fonts/manrope';
import { Stack } from 'expo-router';
import * as SplashScreen from 'expo-splash-screen';
import { StatusBar } from 'expo-status-bar';
import { useEffect } from 'react';
import { SafeAreaProvider } from 'react-native-safe-area-context';

import { ReminderService } from '@/components/ReminderService';
import { DialogHost, ToastHost, toast } from '@/components/ui/feedback';
import { initNotifications } from '@/services/notifications';
import { useAppStore } from '@/store/useAppStore';
import { colors, isDark } from '@/theme';

SplashScreen.preventAutoHideAsync().catch(() => {});

export default function RootLayout() {
  const [fontsLoaded, fontError] = useFonts({ Anton_400Regular, Manrope_400Regular, Manrope_500Medium, Manrope_600SemiBold, Manrope_700B
old, Manrope_800ExtraBold });
  const hydrated = useAppStore((s) => s.hydrated);
  const seeded = useAppStore((s) => s.seeded);
  const signedIn = useAppStore((s) => !!s.session);
  const ensureSeed = useAppStore((s) => s.ensureSeed);

  useEffect(() => {
    initNotifications();
  }, []);

  useEffect(() => {
    if (hydrated) ensureSeed();
  }, [hydrated, ensureSeed]);

  const ready = (fontsLoaded || !!fontError) && hydrated && seeded;
  useEffect(() => {
    if (!ready) return;
    SplashScreen.hideAsync().catch(() => {});
    // Back from the reload that applies a new theme: say so, then forget the one-off session.
    const { restyled, setRestyled } = useAppStore.getState();
    if (restyled) {
      setRestyled(false);
      setTimeout(() => toast('Your new look is live ✨', { icon: 'sparkles' }), 600);
    }
  }, [ready]);

  if (!ready) return null;

  return (
    <SafeAreaProvider>
      <StatusBar style={isDark ? 'light' : 'dark'} />
      <Stack screenOptions={{ headerShown: false, contentStyle: { backgroundColor: colors.bg }, animation: 'slide_from_right' }}>
        <Stack.Screen name="index" />
        <Stack.Protected guard={!signedIn}>
          <Stack.Screen name="onboarding" options={{ animation: 'fade' }} />
          <Stack.Screen name="sign-in" options={{ animation: 'fade' }} />
          <Stack.Screen name="sign-up" />
          <Stack.Screen name="forgot-password" />
        </Stack.Protected>
        <Stack.Protected guard={signedIn}>
          <Stack.Screen name="(tabs)" options={{ animation: 'fade' }} />
          <Stack.Screen name="destination/[id]" />
          <Stack.Screen name="destinations" />
          <Stack.Screen name="nearby" />
          <Stack.Screen name="trips" />
          <Stack.Screen name="new-trip" options={{ presentation: 'modal', animation: 'slide_from_bottom' }} />
          <Stack.Screen name="join" options={{ presentation: 'modal', animation: 'slide_from_bottom' }} />
          <Stack.Screen name="flights" />
          <Stack.Screen name="crew" />
          <Stack.Screen name="how-it-works" />
          <Stack.Screen name="panorama" options={{ animation: 'fade' }} />
          <Stack.Screen name="map" />
          <Stack.Screen name="weather" />
          <Stack.Screen name="photos" />
          <Stack.Screen name="bucket-list" />
          <Stack.Screen name="quiz" />
          <Stack.Screen name="survey" />
          <Stack.Screen name="documents" />
          <Stack.Screen name="packing" />
          <Stack.Screen name="currency" />
          <Stack.Screen name="notifications" />
          <Stack.Screen name="settings" />
          <Stack.Screen name="emergency" />
          <Stack.Screen name="phrasebook" />
          <Stack.Screen name="recap" />
```

```
          <Stack.Screen name="airport" />
          <Stack.Screen name="help" />
          <Stack.Screen name="about" />
          <Stack.Screen name="personalize" options={{ presentation: 'modal', animation: 'slide_from_bottom' }} />
        </Stack.Protected>
      </Stack>
      {signedIn ? <ReminderService /> : null}
      <ToastHost />
      <DialogHost />
    </SafeAreaProvider>
  );
}
```

src/app/about.tsx

```
import * as Linking from 'expo-linking';
import { StyleSheet, View } from 'react-native';

import { LogoMark } from '@/components/Logo';
import { Card } from '@/components/ui/bits';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { colors } from '@/theme';

const SOURCES: [string, string, string][] = [
  ['Weather', 'Open-Meteo forecast API (CC BY 4.0)', 'https://open-meteo.com'],
  ['Exchange rates', 'ExchangeRate-API open access endpoint', 'https://www.exchangerate-api.com/docs/free'],
  ['Maps', 'Apple Maps / Google Maps via react-native-maps; OpenStreetMap on web (ODbL)', 'https://www.openstreetmap.org/copyright'],
  ['Photography', 'Unsplash — free to use under the Unsplash License (see ATTRIBUTIONS.md)', 'https://unsplash.com/license'],
  ['Fonts', 'Manrope (Mikhail Sharanda) and Anton (Vernon Adams) — SIL Open Font License', 'https://fonts.google.com'],
  ['Icons', 'Ionicons & Material Community Icons (MIT) via @expo/vector-icons', 'https://icons.expo.fyi'],
  ['Emergency numbers', 'Official tourism / government sources per country (e.g. JNTO for Japan)', 'https://www.jnto.go.jp/safety-tips/e
ng/emergency-numbers.html'],
  ['AI (optional)', 'Anthropic Claude API — only when the user adds their own key', 'https://docs.claude.com'],
];

const STACK = [
  'Expo SDK 57 + React Native 0.86 (iOS & Android, TypeScript)',
  'Expo Router — file-based navigation with protected routes',
  'Zustand + Immer — state; AsyncStorage — offline persistence',
  'expo-crypto (SHA-256 password hashing), expo-secure-store (API key)',
  'expo-notifications, expo-location, expo-image-picker, expo-document-picker',
  'expo-print, expo-sharing, expo-speech, expo-file-system, react-native-maps',
];

export default function About() {
  return (
    <Screen header={<Header title="About" subtitle="Credits, sources & how it’s built" />}>
      <View style={styles.brand}>
        <LogoMark size={64} />
        <T variant="h2">TogetherWeGo</T>
        <T variant="small" color={colors.textSecondary} center>
          Together We Go: Group Trip Planner · FBLA Mobile Application Development 2026–27 · v1.0
        </T>
      </View>
      <Card style={{ gap: 10 }}>
        <T variant="title">Architecture</T>
        <T variant="small" color={colors.textSecondary}>
          Layered MVVM-style design: screens (views) in src/app, reusable components in src/components, a single store (model + view-mod
el selectors) in src/store, and side-effect services (weather, currency, notifications, AI, files) in src/services. Data flows one way:
screens call store actions → state updates → UI re-renders.
        </T>
        {STACK.map((s) => (
          <T key={s} variant="caption">
            • {s}
          </T>
        ))}
      </Card>
      <Card style={{ gap: 12, marginTop: 14 }}>
        <T variant="title">Data sources & licences</T>
        {SOURCES.map(([k, v, url]) => (
          <View key={k}>
            <T variant="small" weight="semibold">
              {k}
            </T>
            <T variant="caption" color={colors.textSecondary}>
              {v}
            </T>
            <T variant="caption" color={colors.primary} onPress={() => Linking.openURL(url)} accessibilityRole="link">
              {url}
            </T>
          </View>
        ))}
      </Card>
      <T variant="caption" color={colors.textMuted} center style={{ marginTop: 14 }}>
        Demo people, trips and chats are fictional sample data. Destination facts are summarised from public tourism information.
      </T>
    </Screen>
  );
}

const styles = StyleSheet.create({
  brand: { alignItems: 'center', gap: 8, marginBottom: 18 },
});
```

src/app/airport.tsx

```
import { Ionicons } from '@expo/vector-icons';
import * as Linking from 'expo-linking';
import { useState } from 'react';
import { ActivityIndicator, StyleSheet, View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { Card, Chip, Pill } from '@/components/ui/bits';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { getLocation, type LocationResult } from '@/services/location';
import { useNow } from '@/hooks/useTrip';
import { useActiveTrip } from '@/store/useAppStore';
import { colors, radius } from '@/theme';
import { bearingDeg, compassPoint, driveMinutes, kmLabel, nearestAirports } from '@/utils/geo';
import { partsInTz, time12, tzAbbrev } from '@/utils/time';

/** Tracks your location and finds the closest airports (works offline — the airport list is bundled). */
export default function AirportScreen() {
  const trip = useActiveTrip();
  const dest = destinationById(trip?.destinationId ?? 'tokyo')!;
  const [from, setFrom] = useState<'trip' | 'me'>('trip');
  const [loc, setLoc] = useState<LocationResult | null>(null);
  const [loading, setLoading] = useState(false);
  const now = useNow(60000);

  const useMine = async () => {
    setFrom('me');
    if (loc) return;
    setLoading(true);
    setLoc(await getLocation(dest));
    setLoading(false);
  };

  const origin = from === 'me' && loc?.source === 'device' ? loc : { lat: dest.lat, lng: dest.lng };
  const list = nearestAirports(origin, 6);

  return (
    <Screen header={<Header title="Nearest airport" subtitle={from === 'me' ? 'From your current location' : `From central ${dest.city}
`} />}>
      <View style={{ flexDirection: 'row', gap: 8, marginBottom: 14 }}>
        <Chip icon="business-outline" label={`${dest.city} centre`} active={from === 'trip'} onPress={() => setFrom('trip')} />
        <Chip icon="locate-outline" label="My location" active={from === 'me'} onPress={useMine} />
      </View>
      {loading ? (
        <View style={{ padding: 30, alignItems: 'center', gap: 8 }}>
          <ActivityIndicator color={colors.primary} />
          <T variant="small" color={colors.textSecondary}>
            Finding you…
          </T>
        </View>
      ) : null}
      {from === 'me' && loc?.source === 'fallback' ? (
        <Card style={{ marginBottom: 12, backgroundColor: colors.orangeSoft, borderColor: colors.orangeSoft }}>
          <T variant="small">Location permission is off, so we’re using central {dest.city}. Enable location in your phone settings to u
se GPS.</T>
        </Card>
      ) : null}
      <View style={{ gap: 10 }}>
        {list.map((a, i) => {
          const local = time12(partsInTz(now, a.tz).hhmm);
          return (
            <Card key={a.iata} style={[{ gap: 8 }, i === 0 && { borderColor: colors.primary, borderWidth: 2 }]}>
              <View style={styles.row}>
                <View style={[styles.code, i === 0 && { backgroundColor: colors.primary }]}>
                  <T variant="title" weight="bold" color={i === 0 ? colors.white : colors.primary}>
                    {a.iata}
                  </T>
                </View>
                <View style={{ flex: 1 }}>
                  <T variant="small" weight="semibold">
                    {a.name}
                  </T>
                  <T variant="caption" color={colors.textSecondary}>
                    {a.city}, {a.country}
                  </T>
                </View>
                {i === 0 ? <Pill label="Closest" /> : null}
              </View>
              <View style={[styles.row, { gap: 14 }]}>
                <View style={styles.stat}>
                  <Ionicons name="navigate-outline" size={13} color={colors.textSecondary} />
                  <T variant="caption">
                    {kmLabel(a.km)} {compassPoint(bearingDeg(origin, a))}
```

```
                </T>
              </View>
              <View style={styles.stat}>
                <Ionicons name="car-outline" size={13} color={colors.textSecondary} />
                <T variant="caption">~{driveMinutes(a.km)} min</T>
              </View>
              <View style={styles.stat}>
                <Ionicons name="time-outline" size={13} color={colors.textSecondary} />
                <T variant="caption">
                  {local} {tzAbbrev(a.tz)}
                </T>
              </View>
            </View>
            <Button
              label="Directions"
              icon="navigate"
              size="sm"
              variant={i === 0 ? 'primary' : 'soft'}
              onPress={() => Linking.openURL(`https://www.google.com/maps/dir/?api=1&destination=${a.lat},${a.lng}`)}
            />
          </Card>
        );
      })}
    </View>
  </Screen>
 );
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', alignItems: 'center', gap: 12 },
  code: { width: 56, height: 44, borderRadius: radius.sm, backgroundColor: colors.primarySoft, alignItems: 'center', justifyContent: 'ce
nter' },
  stat: { flexDirection: 'row', alignItems: 'center', gap: 4 },
});
```

src/app/bucket-list.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { router } from 'expo-router';
import { useState } from 'react';
import { ScrollView, StyleSheet, TextInput, View } from 'react-native';

import { Card, EmptyState, ProgressBar } from '@/components/ui/bits';
import { confirmAction, toast } from '@/components/ui/feedback';
import { Press, tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { destinationById, destinations } from '@/data/destinations';
import { imageSource } from '@/data/images';
import { useAppStore } from '@/store/useAppStore';
import { colors, fonts, radius } from '@/theme';

export default function BucketList() {
  const bucket = useAppStore((s) => s.bucket);
  const favorites = useAppStore((s) => s.favorites);
  const add = useAppStore((s) => s.addBucket);
  const toggle = useAppStore((s) => s.toggleBucket);
  const del = useAppStore((s) => s.deleteBucket);
  const [text, setText] = useState('');
  const [err, setErr] = useState<string | null>(null);
  const done = bucket.filter((b) => b.done).length;
  const ideas = destinations.filter((d) => !bucket.some((b) => b.destinationId === d.id)).sort((a, b) => Number(favorites.includes(b.i
d)) - Number(favorites.includes(a.id)) || b.rating - a.rating).slice(0, 8);

  const submit = () => {
    const t = text.trim();
    if (t.length < 3) return setErr('Write a few words about your dream');
    if (t.length > 80) return setErr('Keep it under 80 characters');
    add(t);
    setText('');
    setErr(null);
    tap('success');
    toast('Added to your bucket list ✨');
  };

  return (
    <Screen header={<Header title="Bucket list" subtitle={`${done} of ${bucket.length} dreams ticked off`} />}>
      <Card style={{ gap: 8 }}>
        <ProgressBar value={bucket.length ? done / bucket.length : 0} height={8} />
        <View style={{ flexDirection: 'row', gap: 8, marginTop: 6 }}>
          <TextInput value={text} onChangeText={(v) => { setText(v); setErr(null); }} placeholder="Someday I want to…" placeholderTextCo
lor={colors.textMuted} style={[styles.input, err && { borderColor: colors.red }]} onSubmitEditing={submit} returnKeyType="done" accessib
ilityLabel="New bucket list item" maxLength={90} />
          <Press onPress={submit} style={styles.addBtn} accessibilityLabel="Add to bucket list">
            <Ionicons name="add" size={22} color={colors.white} />
          </Press>
        </View>
        {err ? (
          <T variant="caption" color={colors.red}>
            {err}
          </T>
        ) : null}
      </Card>

      <T variant="h3" style={{ marginTop: 18, marginBottom: 10 }}>
        Inspiration
      </T>
      <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 10, paddingHorizontal: 20 }} style={{
marginHorizontal: -20 }}>
        {ideas.map((d) => (
          <Press
            key={d.id}
            onPress={() => {
              add(`Visit ${d.city}, ${d.country}`, d.id);
              toast(`${d.city} added ✨`);
            }}
            style={styles.idea}
            accessibilityLabel={`Add ${d.city} to bucket list`}>
            <Image source={imageSource(d.image)} style={StyleSheet.absoluteFill} contentFit="cover" />
            <View style={styles.ideaShade} />
            <Ionicons name="add-circle" size={22} color={colors.white} style={{ position: 'absolute', top: 8, right: 8 }} />
            <T variant="small" weight="bold" color={colors.white} style={{ position: 'absolute', left: 10, bottom: 10 }}>
              {d.city}
            </T>
          </Press>
        ))}
      </ScrollView>

      <T variant="h3" style={{ marginTop: 18, marginBottom: 10 }}>
        My list
```

```
      </T>
      {bucket.length ? (
        <View style={{ gap: 8 }}>
          {bucket.map((b) => {
            const d = b.destinationId ? destinationById(b.destinationId) : undefined;
            return (
              <View key={b.id} style={[styles.item, b.done && { opacity: 0.7 }]}>
                <Press onPress={() => toggle(b.id)} accessibilityRole="checkbox" accessibilityState={{ checked: b.done }} accessibilityL
abel={b.text} hitSlop={6}>
                  <Ionicons name={b.done ? 'checkmark-circle' : 'ellipse-outline'} size={24} color={b.done ? colors.primary : colors.bor
derStrong} />
                </Press>
                {d ? <Image source={imageSource(d.image)} style={styles.thumb} contentFit="cover" /> : null}
                <Press style={{ flex: 1 }} onPress={() => (d ? router.push({ pathname: '/destination/[id]', params: { id: d.id } }) : to
ggle(b.id))}>
                  <T variant="small" weight="semibold" style={b.done && { textDecorationLine: 'line-through' }}>
                    {b.text}
                  </T>
                  {d ? (
                    <T variant="caption" color={colors.textSecondary}>
                      {d.country} · best {d.bestMonths.split(',')[0]}
                    </T>
                  ) : null}
                </Press>
                <Press onPress={() => confirmAction('Remove from bucket list?', b.text, () => del(b.id), { confirmLabel: 'Remove', destr
uctive: true })} hitSlop={8} accessibilityLabel="Remove">
                  <Ionicons name="close" size={18} color={colors.textMuted} />
                </Press>
              </View>
            );
          })}
        </View>
      ) : (
        <EmptyState icon="flag-outline" title="Dream big" body="Add places and experiences you want to do with your crew one day." />
      )}
    </Screen>
  );
}

const styles = StyleSheet.create({
  input: { flex: 1, height: 46, borderRadius: radius.md, borderWidth: 1, borderColor: colors.border, backgroundColor: colors.cardMuted,
paddingHorizontal: 14, fontFamily: fonts.regular, fontSize: 15, color: colors.text },
  addBtn: { width: 46, height: 46, borderRadius: 23, backgroundColor: colors.primary, alignItems: 'center', justifyContent: 'center' },
  idea: { width: 120, height: 150, borderRadius: radius.lg, overflow: 'hidden' },
  ideaShade: { ...StyleSheet.absoluteFill, backgroundColor: 'rgba(0,0,0,0.25)' },
  item: { flexDirection: 'row', alignItems: 'center', gap: 10, padding: 12, borderRadius: radius.md, backgroundColor: colors.card, borde
rWidth: 1, borderColor: colors.border },
  thumb: { width: 44, height: 44, borderRadius: 10 },
});
```

src/app/crew.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router } from 'expo-router';
import { useState } from 'react';
import { StyleSheet, TextInput, View } from 'react-native';

import { ShareSheet } from '@/components/ShareSheet';
import { MemberPickerSheet } from '@/components/trip/MemberPickerSheet';
import { NoTrip } from '@/components/trip/NoTrip';
import { Button } from '@/components/ui/Button';
import { Avatar, Card, Pill, ProgressBar, SectionHeader } from '@/components/ui/bits';
import { confirmAction, toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { Press, tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { ROLE_OPTIONS, roleDescriptions, type Role } from '@/data/reference';
import { useT } from '@/i18n';
import { copy, inviteMessage } from '@/services/share';
import { tripStatus, useActiveTrip, useAppStore, useMyTrips } from '@/store/useAppStore';
import type { Member } from '@/store/types';
import { colors, fonts, radius } from '@/theme';
import { rangeLabel } from '@/utils/time';
import { firstError, maxLen, required, validateEmail, validateName } from '@/utils/validation';

export default function Crew() {
  const trip = useActiveTrip();
  if (!trip) return <NoTrip />;
  return <Hub key={trip.id} />;
}

function Hub() {
  const t = useT();
  const trip = useActiveTrip()!;
  const myTrips = useMyTrips();
  const userId = useAppStore((s) => s.session?.userId ?? '');
  // Actions are stable references, so read them without subscribing to the whole store.
  const s = useAppStore.getState();
  const owner = trip.ownerId === userId;
  const dest = destinationById(trip.destinationId);

  const [shareOpen, setShareOpen] = useState(false);
  const [roleFor, setRoleFor] = useState<Member | null>(null);
  const [addOpen, setAddOpen] = useState(false);
  const [pollOpen, setPollOpen] = useState(false);
  const [task, setTask] = useState('');
  const [taskErr, setTaskErr] = useState<string | null>(null);
  const [assignTaskId, setAssignTaskId] = useState<string | null>(null);

  const done = trip.tasks.filter((k) => k.done).length;
  const status = tripStatus(trip);

  const addTask = () => {
    const e = firstError(task, required('a task'), maxLen(60, 'Task'));
    setTaskErr(e);
    if (e) return tap('warning');
    s.addTask(trip.id, task);
    setTask('');
    tap('success');
  };

  return (
    <Screen header={<Header title="Crew & roles" subtitle="Invite code, roles, votes and checklist" backLabel="More" />} bottomInset={3
0}>
      {/* Trip switcher */}
      <Card padded={false}>
        <Press onPress={() => router.push('/trips')} style={styles.tripRow} accessibilityLabel="Switch trip">
          <View style={styles.tripIcon}>
            <Ionicons name="airplane" size={18} color={colors.white} />
          </View>
          <View style={{ flex: 1 }}>
            <T variant="title" numberOfLines={1}>
              {trip.name}
            </T>
            <T variant="caption" color={colors.textSecondary} numberOfLines={1}>
              {dest?.city} · {rangeLabel(trip.startDate, trip.endDate)}
            </T>
          </View>
          <Pill label={status === 'planning' ? 'Planning' : status === 'active' ? 'In progress' : 'Completed'} tone={status === 'active'
? 'green' : status === 'planning' ? 'blue' : 'gray'} />
          <T variant="caption" weight="semibold" color={colors.textSecondary}>
            {myTrips.length > 1 ? `+${myTrips.length - 1}` : ''}
          </T>
```

```
          <Ionicons name="chevron-forward" size={16} color={colors.textMuted} />
        </Press>
      </Card>

      {/* Invite */}
      <Card style={styles.invite}>
        <T variant="kicker" color={colors.textSecondary}>
          {t('hub.invite')}
        </T>
        <View style={styles.codeRow}>
          <T variant="display" numberOfLines={1} adjustsFontSizeToFit style={styles.code} accessibilityLabel={`Invite code ${trip.invite
Code}`}>
            {trip.inviteCode}
          </T>
          <Press onPress={() => copy(trip.inviteCode, `Invite code ${trip.inviteCode} copied`)} style={styles.copyPill} accessibilityLab
el={`Copy invite code ${trip.inviteCode}`}>
            <Ionicons name="copy-outline" size={15} color={colors.onAccent} />
            <T variant="small" weight="bold" color={colors.onAccent}>
              Copy
            </T>
          </Press>
        </View>
        <T variant="caption" color={colors.textSecondary}>
          Friends enter this in “Join a trip”
        </T>
        <View style={styles.row}>
          <Button label={t('hub.share')} icon="share-social-outline" size="md" onPress={() => setShareOpen(true)} style={{ flex: 1 }} />
          {owner ? (
            <Button
              label="New code"
              icon="refresh-outline"
              variant="secondary"
              size="md"
              onPress={() =>
                confirmAction('New invite code?', 'The old code will stop working. Members already on the trip stay.', () => {
                  s.regenerateInvite(trip.id);
                  toast('New invite code created');
                })
              }
              style={{ flex: 1 }}
            />
          ) : null}
        </View>
      </Card>

      {/* Members */}
      <SectionHeader title={t('hub.members')} action="+ Add" onAction={() => setAddOpen(true)} style={{ marginTop: 26 }} />
      <Card padded={false}>
        {trip.members.map((m, i) => (
          <Press key={m.id} onPress={() => setRoleFor(m)} style={[styles.member, i > 0 && styles.line]} scaleTo={0.99} accessibilityLabe
l={`${m.name}, ${m.role}. Tap to change role`}>
            <Avatar name={m.name} src={m.avatar} size={40} />
            <View style={{ flex: 1 }}>
              <T variant="title" weight="bold" numberOfLines={1}>
                {m.name}
                {m.id === userId ? ' (you)' : ''}
                {m.id === trip.ownerId ? ' 👑' : ''}
              </T>
              <T variant="caption" color={colors.textSecondary} numberOfLines={1}>
                {m.invited ? 'Invited · hasn’t joined yet' : m.homeCity ? `From ${m.homeCity}` : 'Member'}
              </T>
            </View>
            <View style={styles.rolePill}>
              <T variant="micro" weight="bold" color={colors.onAccent}>
                {m.role}
              </T>
            </View>
          </Press>
        ))}
      </Card>

      {/* Polls */}
      <SectionHeader title={t('hub.vote')} action="+ New poll" onAction={() => setPollOpen(true)} style={{ marginTop: 26 }} />
      <View style={{ gap: 10 }}>
        {trip.polls.length ? (
          trip.polls.map((p) => {
            const total = p.options.reduce((n, o) => n + o.votes.length, 0);
            const voted = p.options.find((o) => o.votes.includes(userId));
            return (
              <Card key={p.id} style={{ gap: 10 }}>
                <View style={styles.rowBetween}>
                  <T variant="title" weight="bold" style={{ flex: 1 }}>
                    {p.question}
                  </T>
                  {p.closed ? <Pill label="Closed" tone="gray" /> : <Pill label={`${total} vote${total === 1 ? '' : 's'}`} tone="yellow"
/>}
```

```
              </View>
              {p.options.map((o) => {
                const pctv = total ? Math.round((o.votes.length / total) * 100) : 0;
                const mine = o.votes.includes(userId);
                return (
                  <Press
                    key={o.id}
                    disabled={p.closed}
                    onPress={() => {
                      s.votePoll(trip.id, p.id, o.id);
                      tap();
                    }}
                    style={[styles.option, mine && styles.optionOn]}
                    accessibilityLabel={`${o.text}, ${pctv} percent${mine ? ', your vote' : ''}`}>
                    <View style={styles.rowBetween}>
                      <T variant="caption" weight={mine ? 'semibold' : 'regular'} style={{ flex: 1 }}>
                        {mine ? '✓ ' : ''}
                        {o.text}
                      </T>
                      <T variant="caption" color={colors.textSecondary}>
                        {pctv}%
                      </T>
                    </View>
                    <ProgressBar value={pctv / 100} height={5} />
                  </Press>
                );
              })}
              <View style={styles.rowBetween}>
                <T variant="caption" color={colors.textMuted} style={{ flex: 1 }}>
                  {voted ? 'Tap again to remove your vote' : p.closed ? 'Voting closed' : 'Tap an option to vote'}
                </T>
                {p.createdBy === userId || owner ? (
                  <View style={{ flexDirection: 'row', gap: 12 }}>
                    <Press onPress={() => s.closePoll(trip.id, p.id)} hitSlop={6}>
                      <T variant="caption" weight="semibold" color={colors.text}>
                        {p.closed ? 'Reopen' : 'Close'}
                      </T>
                    </Press>
                    <Press
                      onPress={() => confirmAction('Delete poll?', p.question, () => s.deletePoll(trip.id, p.id), { destructive: true,
confirmLabel: 'Delete' })}
                      hitSlop={6}>
                      <T variant="caption" weight="semibold" color={colors.red}>
                        Delete
                      </T>
                    </Press>
                  </View>
                ) : null}
              </View>
            </Card>
          );
        })
      ) : (
        <Card>
          <T variant="small" color={colors.textSecondary}>
            No polls yet — start one to settle a decision fast.
          </T>
        </Card>
      )}
    </View>

    {/* Checklist */}
    <SectionHeader title={t('hub.checklist')} action={t('hub.done', { a: done, b: trip.tasks.length })} style={{ marginTop: 26 }} />
    <Card padded={false} style={{ paddingHorizontal: 16, paddingVertical: 4 }}>
      {trip.tasks.map((k) => {
        const who = trip.members.find((m) => m.id === k.assignee);
        return (
          <View key={k.id} style={styles.task}>
            <Press onPress={() => s.toggleTask(trip.id, k.id)} accessibilityRole="checkbox" accessibilityState={{ checked: k.done }} a
ccessibilityLabel={k.text} hitSlop={6}>
              <Ionicons name={k.done ? 'checkmark-circle' : 'ellipse-outline'} size={22} color={k.done ? colors.primary : colors.borde
rStrong} />
            </Press>
            <Press onPress={() => s.toggleTask(trip.id, k.id)} style={{ flex: 1 }} haptic={false}>
              <T variant="small" color={k.done ? colors.textMuted : colors.text} style={k.done && { textDecorationLine: 'line-through'
}}>
                {k.text}
              </T>
            </Press>
            <Press onPress={() => setAssignTaskId(k.id)} hitSlop={6} accessibilityLabel="Assign task">
              <T variant="caption" color={colors.textMuted}>
                {who ? who.name.split(' ')[0] : 'Everyone'}
              </T>
            </Press>
            <Press onPress={() => s.deleteTask(trip.id, k.id)} hitSlop={6} accessibilityLabel={`Delete task ${k.text}`}>
              <Ionicons name="close" size={16} color={colors.textMuted} />
```

```
            </Press>
          </View>
        );
      })}
      <View style={styles.addTask}>
        <TextInput
          value={task}
          onChangeText={(v) => {
            setTask(v);
            if (taskErr) setTaskErr(null);
          }}
          placeholder={t('hub.addTask')}
          placeholderTextColor={colors.textMuted}
          style={[styles.taskInput, taskErr && { borderColor: colors.red }]}
          onSubmitEditing={addTask}
          returnKeyType="done"
          maxLength={70}
          accessibilityLabel="New task"
        />
        <Press onPress={addTask} style={styles.addBtn} accessibilityLabel="Add task">
          <Ionicons name="add" size={20} color={colors.white} />
        </Press>
      </View>
      {taskErr ? (
        <T variant="caption" color={colors.red} style={{ marginBottom: 8 }}>
          {taskErr}
        </T>
      ) : null}
    </Card>

    {/* Documents */}
    <SectionHeader title={t('hub.documents')} action="Open vault" onAction={() => router.push('/documents')} style={{ marginTop: 26, m
arginBottom: 2 }} />
    <T variant="caption" color={colors.textSecondary} style={{ marginBottom: 10 }}>
      {t('hub.documentsSub')}
    </T>
    {trip.docs.length ? (
      <Card padded={false}>
        {trip.docs.slice(0, 4).map((d, i) => (
          <Press key={d.id} onPress={() => router.push({ pathname: '/documents', params: { open: d.id } })} style={[styles.doc, i > 0
&& styles.line]} scaleTo={0.99}>
            <View style={styles.docIcon}>
              <Ionicons name={d.kind === 'image' ? 'image-outline' : d.kind === 'note' ? 'document-text-outline' : 'document-outline'}
size={18} color={colors.text} />
            </View>
            <View style={{ flex: 1 }}>
              <T variant="small" weight="bold" numberOfLines={1}>
                {d.name}
              </T>
              <T variant="caption" color={colors.textMuted}>
                {d.kind === 'note' ? 'Note' : d.kind.toUpperCase()}
              </T>
            </View>
            {d.offline ? <Pill label={t('hub.offline')} icon="cloud-done-outline" tone="gray" /> : null}
          </Press>
        ))}
      </Card>
    ) : null}

    <ShareSheet visible={shareOpen} onClose={() => setShareOpen(false)} title="Invite your group" message={inviteMessage(trip.name, tr
ip.inviteCode)} preview={inviteMessage(trip.name, trip.inviteCode)} />

    <RoleSheet
      member={roleFor}
      canEdit={owner || roleFor?.id === userId}
      canRemove={owner && roleFor?.id !== trip.ownerId}
      onClose={() => setRoleFor(null)}
      onPick={(role) => {
        if (roleFor) {
          s.setRole(trip.id, roleFor.id, role);
          toast(`${roleFor.name.split(' ')[0]} is now ${role}`);
        }
        setRoleFor(null);
      }}
      onRemove={() => {
        const m = roleFor!;
        setRoleFor(null);
        confirmAction(`Remove ${m.name}?`, 'They'll lose access to this trip.', () => {
          s.removeMember(trip.id, m.id);
          toast(`${m.name.split(' ')[0]} removed`);
        }, { destructive: true, confirmLabel: 'Remove' });
      }}
    />
    <AddMemberSheet visible={addOpen} onClose={() => setAddOpen(false)} onAdd={(n, e) => s.addMember(trip.id, n, e)} onShare={() => {
      setAddOpen(false);
      setTimeout(() => setShareOpen(true), 250);
```

```
      }} />
      <NewPollSheet visible={pollOpen} onClose={() => setPollOpen(false)} onCreate={(q, o) => s.createPoll(trip.id, q, o)} />
      <MemberPickerSheet
        visible={!!assignTaskId}
        single
        title="Assign task"
        onClose={() => setAssignTaskId(null)}
        members={trip.members}
        selected={trip.tasks.find((k) => k.id === assignTaskId)?.assignee ? [trip.tasks.find((k) => k.id === assignTaskId)!.assignee!] :
[]}
        onToggle={(id) => assignTaskId && s.assignTask(trip.id, assignTaskId, id)}
      />
    </Screen>
  );
}


function RoleSheet({ member, canEdit, canRemove, onClose, onPick, onRemove }: { member: Member | null; canEdit: boolean; canRemove: bool
ean; onClose: () => void; onPick: (r: Role) => void; onRemove: () => void }) {
  return (
    <Sheet visible={!!member} onClose={onClose} title={member?.name} subtitle={canEdit ? 'Choose a role for this trip' : 'Only the organ
izer can change other people’s roles'}>
      {ROLE_OPTIONS.map((r) => (
        <Press key={r} disabled={!canEdit} onPress={() => onPick(r)} style={[styles.roleRow, member?.role === r && styles.optionOn, !can
Edit && { opacity: 0.6 }]} accessibilityState={{ selected: member?.role === r }}>
          <View style={{ flex: 1 }}>
            <T variant="small" weight="semibold">
              {r}
            </T>
            <T variant="caption" color={colors.textSecondary}>
              {roleDescriptions[r]}
            </T>
          </View>
          {member?.role === r ? <Ionicons name="checkmark-circle" size={20} color={colors.primary} /> : null}
        </Press>
      ))}
      {canRemove ? <Button label="Remove from trip" variant="danger" icon="person-remove-outline" onPress={onRemove} /> : null}
    </Sheet>
  );
}

function AddMemberSheet({ visible, onClose, onAdd, onShare }: { visible: boolean; onClose: () => void; onAdd: (name: string, email?: str
ing) => void; onShare: () => void }) {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [errors, setErrors] = useState<{ name?: string | null; email?: string | null }>({});
  return (
    <Sheet visible={visible} onClose={onClose} title="Add a member" subtitle="Add someone by name, or share the invite code so they can
join themselves">
      <Button label="Share invite code instead" variant="soft" icon="share-social-outline" onPress={onShare} />
      <Input label="Name" value={name} onChangeText={setName} error={errors.name} icon="person-outline" autoCapitalize="words" />
      <Input label="Email (optional)" value={email} onChangeText={setEmail} error={errors.email} icon="mail-outline" autoCapitalize="non
e" keyboardType="email-address" />
      <Button
        label="Add member"
        icon="person-add-outline"
        onPress={() => {
          const e = { name: validateName(name), email: email.trim() ? validateEmail(email) : null };
          setErrors(e);
          if (e.name || e.email) return tap('warning');
          onAdd(name, email);
          toast(`${name.trim().split(' ')[0]} added to the trip`);
          setName('');
          setEmail('');
          onClose();
        }}
      />
    </Sheet>
  );
}

function NewPollSheet({ visible, onClose, onCreate }: { visible: boolean; onClose: () => void; onCreate: (q: string, opts: string[]) =>
void }) {
  const [q, setQ] = useState('');
  const [opts, setOpts] = useState(['', '']);
  const [err, setErr] = useState<string | null>(null);
  return (
    <Sheet visible={visible} onClose={onClose} title="New poll" subtitle="Let the group decide">
      <Input label="Question" value={q} onChangeText={setQ} placeholder="e.g. Where should we eat on Day 3?" maxLength={80} />
      {opts.map((o, i) => (
        <View key={i} style={{ flexDirection: 'row', gap: 8, alignItems: 'center' }}>
          <TextInput
            value={o}
            onChangeText={(v) => setOpts((arr) => arr.map((x, j) => (j === i ? v : x)))}
            placeholder={`Option ${i + 1}`}
            placeholderTextColor={colors.textMuted}
```

```
            style={styles.optInput}
            maxLength={50}
            accessibilityLabel={`Option ${i + 1}`}
          />
          {opts.length > 2 ? (
            <Press onPress={() => setOpts((arr) => arr.filter((_, j) => j !== i))} accessibilityLabel={`Remove option ${i + 1}`}>
              <Ionicons name="close-circle" size={22} color={colors.textMuted} />
            </Press>
          ) : null}
        </View>
      ))}
      {opts.length < 5 ? <Button label="Add option" variant="ghost" size="sm" icon="add" onPress={() => setOpts((a) => [...a, ''])} /> :
null}
      {err ? (
        <T variant="caption" color={colors.red}>
          {err}
        </T>
      ) : null}
      <Button
        label="Start poll"
        icon="stats-chart-outline"
        onPress={() => {
          const clean = opts.map((o) => o.trim()).filter(Boolean);
          if (!q.trim()) return setErr('Write a question for the poll');
          if (clean.length < 2) return setErr('Add at least two options');
          if (new Set(clean.map((c) => c.toLowerCase())).size !== clean.length) return setErr('Options must be different');
          onCreate(q, clean);
          setQ('');
          setOpts(['', '']);
          setErr(null);
          toast('Poll started — the group has been notified');
          onClose();
        }}
      />
    </Sheet>
  );
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', alignItems: 'center', gap: 10 },
  rowBetween: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', gap: 8 },
  line: { borderTopWidth: 1, borderTopColor: colors.border },
  tripRow: { flexDirection: 'row', alignItems: 'center', gap: 10, padding: 14 },
  tripIcon: { width: 40, height: 40, borderRadius: 20, backgroundColor: colors.primary, alignItems: 'center', justifyContent: 'center'
},
  invite: { marginTop: 14, gap: 6, borderRadius: radius.xl, padding: 18 },
  codeRow: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', gap: 12 },
  code: { flexShrink: 1, fontSize: 44, lineHeight: 52, letterSpacing: 3 },
  copyPill: { flexDirection: 'row', alignItems: 'center', gap: 6, backgroundColor: colors.accent, paddingHorizontal: 16, height: 38, bor
derRadius: 999 },
  member: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingHorizontal: 16, paddingVertical: 12 },
  rolePill: { paddingHorizontal: 10, paddingVertical: 4, borderRadius: 999, backgroundColor: colors.accent },
  option: { gap: 6, padding: 10, borderRadius: radius.sm, borderWidth: 1, borderColor: colors.border },
  optionOn: { borderColor: colors.primaryLine, backgroundColor: colors.primarySofter },
  task: { flexDirection: 'row', alignItems: 'center', gap: 10, paddingVertical: 11, borderBottomWidth: 1, borderBottomColor: colors.bord
er },
  addTask: { flexDirection: 'row', alignItems: 'center', gap: 8, paddingVertical: 10 },
  taskInput: { flex: 1, height: 42, borderRadius: radius.sm, borderWidth: 1, borderColor: colors.border, backgroundColor: colors.cardMut
ed, paddingHorizontal: 12, fontFamily: fonts.regular, color: colors.text },
  addBtn: { width: 40, height: 40, borderRadius: 20, backgroundColor: colors.primary, alignItems: 'center', justifyContent: 'center' },
  doc: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingHorizontal: 16, paddingVertical: 12 },
  docIcon: { width: 36, height: 36, borderRadius: 18, backgroundColor: colors.bgAlt, alignItems: 'center', justifyContent: 'center' },
  roleRow: { flexDirection: 'row', alignItems: 'center', gap: 10, padding: 12, borderRadius: radius.md, borderWidth: 1, borderColor: col
ors.border },
  optInput: { flex: 1, height: 46, borderRadius: radius.md, borderWidth: 1, borderColor: colors.border, backgroundColor: colors.cardMute
d, paddingHorizontal: 12, fontFamily: fonts.regular, color: colors.text },
});
```

src/app/currency.tsx

```
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { CurrencyConverter } from '@/components/CurrencyConverter';
import { Card, Pill } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { useRatesLoader } from '@/hooks/useTrip';
import { convert, fetchRates, useRates } from '@/services/currency';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import { colors } from '@/theme';
import { money, timeAgo } from '@/utils/format';

const QUICK = [1, 5, 10, 20, 50, 100, 500, 1000];

export default function Currency() {
  useRatesLoader();
  const trip = useActiveTrip();
  const home = useAppStore((s) => s.settings.homeCurrency);
  const rates = useRates();
  const local = destinationById(trip?.destinationId ?? 'tokyo')?.currency ?? 'JPY';
  const [pair, setPair] = useState<[string, string]>([home, local === home ? 'EUR' : local]);
  const [refreshing, setRefreshing] = useState(false);

  return (
    <Screen
      header={<Header title="Currency converter" subtitle={rates.live ? `Live rates · updated ${timeAgo(rates.fetchedAt)}` : 'Offline re
ference rates'} />}
      onRefresh={async () => {
        setRefreshing(true);
        const r = await fetchRates(true);
        setRefreshing(false);
        toast(r.live ? 'Rates updated' : 'Offline — using saved rates', { tone: r.live ? 'ok' : 'warn' });
      }}
      refreshing={refreshing}>
      <Card>
        <CurrencyConverter from={pair[0]} to={pair[1]} onChange={(a, b) => setPair([a, b])} />
      </Card>

      <View style={styles.head}>
        <T variant="h3">Quick reference</T>
        <Pill label={`${pair[0]} → ${pair[1]}`} />
      </View>
      <Card padded={false}>
        {QUICK.map((n, i) => (
          <View key={n} style={[styles.row, i % 2 === 1 && { backgroundColor: colors.cardMuted }]}>
            <T variant="small" weight="semibold">
              {money(n, pair[0])}
            </T>
            <T variant="small">{money(convert(n, pair[0], pair[1], rates), pair[1], { decimals: true })}</T>
          </View>
        ))}
      </Card>
      <T variant="caption" color={colors.textMuted} style={{ marginTop: 12 }}>
        Rates from open.er-api.com, cached on your phone so the converter keeps working offline. Your home currency ({home}) can be chan
ged in Settings.
      </T>
    </Screen>
  );
}

const styles = StyleSheet.create({
  head: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', marginTop: 20, marginBottom: 10 },
  row: { flexDirection: 'row', justifyContent: 'space-between', paddingHorizontal: 16, paddingVertical: 11 },
});
```

src/app/destination/[id].tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { LinearGradient } from 'expo-linear-gradient';
import { router, useLocalSearchParams } from 'expo-router';
import { useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { AddToTripSheet, FavStar } from '@/components/explore';
import { placeCategoryIcon } from '@/components/trip/meta';
import { Button } from '@/components/ui/Button';
import { panoById } from '@/data/panoramas';
import { Card, EmptyState, IconButton, Pill, type IconName } from '@/components/ui/bits';
import { Press } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { imageSource } from '@/data/images';
import { placesFor } from '@/data/places';
import { currencyByCode } from '@/data/reference';
import { useNow, useWeather } from '@/hooks/useTrip';
import { shareText } from '@/services/share';
import { weatherInfo } from '@/services/weather';
import { MAX_WIDTH, colors, radius } from '@/theme';
import { durationLabel, money, priceTier } from '@/utils/format';
import { distanceKm, kmLabel, nearestAirports } from '@/utils/geo';
import { hoursAhead, deviceTimeZone, partsInTz, time12, utcOffsetLabel } from '@/utils/time';

export default function DestinationScreen() {
  const { id } = useLocalSearchParams<{ id: string }>();
  const d = destinationById(String(id));
  const insets = useSafeAreaInsets();
  const weather = useWeather(d?.id);
  const [adding, setAdding] = useState<string | null>(null);
  const now = useNow(60000);
  if (!d) return <EmptyState icon="alert-circle-outline" title="Destination not found" />;

  const places = placesFor(d.id)
    .map((p) => ({ p, km: distanceKm(d, p) }))
    .sort((a, b) => b.p.rating - a.p.rating)
    .slice(0, 6);
  const air = nearestAirports(d, 1)[0];
  const diff = hoursAhead(d.tz, deviceTimeZone());
  const cur = currencyByCode(d.currency);

  const facts: [IconName, string, string][] = [
    ['calendar-outline', 'Best time', d.bestMonths],
    ['wallet-outline', 'Daily cost', `~${money(d.dailyCost)} pp`],
    ['cash-outline', 'Currency', `${d.currency}${cur ? ` (${cur.symbol})` : ''}`],
    ['language-outline', 'Language', d.language],
    ['time-outline', 'Local time', `${time12(partsInTz(now, d.tz).hhmm)} · ${utcOffsetLabel(d.tz)}`],
    ['airplane-outline', 'Airport', `${air.iata} · ${kmLabel(air.km)}`],
  ];

  return (
    <View style={styles.root}>
      <ScrollView contentContainerStyle={{ paddingBottom: insets.bottom + 100 }} showsVerticalScrollIndicator={false}>
        <View style={styles.hero}>
          <Image source={imageSource(d.image)} style={StyleSheet.absoluteFill} contentFit="cover" transition={250} />
          <LinearGradient colors={['rgba(0,0,0,0.35)', 'rgba(0,0,0,0)', 'rgba(0,0,0,0.55)']} locations={[0, 0.4, 1]} style={StyleSheet.a
bsoluteFill} />
          <View style={[styles.heroTop, { top: insets.top + 8 }]}>
            <IconButton icon="chevron-back" label="Back" onPress={() => (router.canGoBack() ? router.back() : router.replace('/(tabs)/ex
plore'))} bg="rgba(255,255,255,0.9)" bordered={false} />
            <View style={{ flexDirection: 'row', gap: 8 }}>
              <IconButton icon="share-social-outline" label={`Share ${d.city}`} onPress={() => shareText(`${d.city}, ${d.country} 🌍
${d.blurb} — found on TogetherWeGo`)} bg="rgba(255,255,255,0.9)" bordered={false} />
              <FavStar id={d.id} size={40} />
            </View>
          </View>
          <View style={styles.heroText}>
            <T variant="display" color={colors.white}>
              {d.city}
            </T>
            <View style={styles.row}>
              <Ionicons name="location-sharp" size={13} color="rgba(255,255,255,0.9)" />
              <T variant="small" color="rgba(255,255,255,0.9)">
                {d.country}
              </T>
              <Ionicons name="star" size={13} color={colors.star} style={{ marginLeft: 8 }} />
              <T variant="small" weight="bold" color={colors.white}>
                {d.rating} <T variant="caption" color="rgba(255,255,255,0.8)">({d.reviews.toLocaleString()} reviews)</T>
              </T>
            </View>
          </View>
```

```
        </View>

        <View style={styles.body}>
          <View style={styles.tags}>
            {d.tags.map((t) => (
              <Pill key={t} label={t === 'best' ? 'Top pick' : t[0].toUpperCase() + t.slice(1)} tone={t === 'best' ? 'yellow' : 'green'}
/>
            ))}
            <Pill label={'$'.repeat(d.priceLevel)} tone="gray" />
          </View>
          <T variant="body" color={colors.textSecondary}>
            {d.blurb}
          </T>

          {weather ? (
            <Press onPress={() => router.push({ pathname: '/weather', params: { id: d.id } })} style={styles.weather}>
              <Ionicons name={weatherInfo(weather.current.code).icon} size={26} color="#E8A23A" />
              <View style={{ flex: 1 }}>
                <T variant="title">
                  {weather.current.temp}°C · {weatherInfo(weather.current.code).label}
                </T>
                <T variant="caption" color={colors.textSecondary}>
                  Next 3 days: {weather.daily.slice(1, 4).map((x) => `${x.max}°`).join(' · ')} {weather.live ? '' : '(estimate)'}
                </T>
              </View>
              <Ionicons name="chevron-forward" size={16} color={colors.textMuted} />
            </Press>
          ) : null}

          <View style={styles.facts}>
            {facts.map(([icon, label, value]) => (
              <View key={label} style={styles.fact}>
                <Ionicons name={icon} size={16} color={colors.primary} />
                <T variant="micro" color={colors.textMuted}>
                  {label}
                </T>
                <T variant="caption" weight="semibold" numberOfLines={2}>
                  {value}
                </T>
              </View>
            ))}
          </View>
          <T variant="caption" color={colors.textMuted}>
            {diff === 0 ? 'Same time zone as you.' : `${Math.abs(diff)}h ${diff > 0 ? 'ahead of' : 'behind'} your time zone.`}
          </T>

          <T variant="h3" style={{ marginTop: 6 }}>
            Top things to do
          </T>
          {places.map(({ p, km }) => (
            <Card key={p.id} style={styles.placeRow} onPress={() => setAdding(d.id)}>
              <View style={styles.placeIcon}>
                <Ionicons name={placeCategoryIcon[p.category]} size={17} color={colors.primary} />
              </View>
              <View style={{ flex: 1 }}>
                <T variant="small" weight="semibold">
                  {p.name}
                </T>
                <T variant="caption" color={colors.textSecondary} numberOfLines={2}>
                  {p.blurb}
                </T>
                <T variant="caption" color={colors.textMuted} style={{ marginTop: 2 }}>
                  ★ {p.rating} · {durationLabel(p.durationMin)} · {priceTier(p.cost)} · {kmLabel(km)} from centre
                </T>
              </View>
            </Card>
          ))}
        </View>
      </ScrollView>
      <View style={[styles.footer, { paddingBottom: insets.bottom + 12 }]}>
        {panoById(d.id) ? <Button label="360° view" icon="globe-outline" variant="secondary" full={false} onPress={() => router.push({ p
athname: '/panorama', params: { city: d.id } })} /> : null}
        <Button label="Add to Trip" icon="add-circle-outline" onPress={() => setAdding(d.id)} style={{ flex: 1 }} />
      </View>
      <AddToTripSheet destinationId={adding} onClose={() => setAdding(null)} />
    </View>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.bg },
  hero: { height: 360, width: '100%', maxWidth: MAX_WIDTH, alignSelf: 'center' },
  heroTop: { position: 'absolute', left: 16, right: 16, flexDirection: 'row', justifyContent: 'space-between' },
  heroText: { position: 'absolute', left: 20, right: 20, bottom: 36, gap: 4 },
  row: { flexDirection: 'row', alignItems: 'center', gap: 4 },
  body: { marginTop: -22, backgroundColor: colors.bg, borderTopLeftRadius: 26, borderTopRightRadius: 26, padding: 20, gap: 14, width: '1
```

```
00%', maxWidth: MAX_WIDTH, alignSelf: 'center' },
  tags: { flexDirection: 'row', flexWrap: 'wrap', gap: 6 },
  weather: { flexDirection: 'row', alignItems: 'center', gap: 12, backgroundColor: colors.yellowSoft, padding: 14, borderRadius: radius.
lg },
  facts: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
  fact: { width: '31.5%', backgroundColor: colors.card, borderRadius: radius.md, padding: 10, gap: 3, borderWidth: 1, borderColor: color
s.border },
  placeRow: { flexDirection: 'row', gap: 12, padding: 14 },
  placeIcon: { width: 36, height: 36, borderRadius: 10, backgroundColor: colors.primarySofter, alignItems: 'center', justifyContent: 'ce
nter' },
  footer: { position: 'absolute', left: 0, right: 0, bottom: 0, paddingHorizontal: 20, paddingTop: 12, backgroundColor: 'rgba(244,244,23
8,0.96)', borderTopWidth: 1, borderTopColor: colors.border, flexDirection: 'row', alignItems: 'center', alignSelf: 'center', maxWidth: M
AX_WIDTH, width: '100%' },
});
```

src/app/destinations.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useLocalSearchParams } from 'expo-router';
import { useMemo, useState } from 'react';
import { ScrollView, StyleSheet, TextInput, View, useWindowDimensions } from 'react-native';

import { PlaceCard } from '@/components/explore';
import { Chip, EmptyState } from '@/components/ui/bits';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { destinations, exploreCategories, type DestinationTag } from '@/data/destinations';
import { useT } from '@/i18n';
import { useAppStore } from '@/store/useAppStore';
import { MAX_WIDTH, colors, fonts, radius } from '@/theme';

type Sort = 'rating' | 'cheap' | 'name';

export default function Destinations() {
  const t = useT();
  const params = useLocalSearchParams<{ q?: string; cat?: string }>();
  const { width } = useWindowDimensions();
  const favorites = useAppStore((s) => s.favorites);
  const [q, setQ] = useState(params.q ?? '');
  const [cat, setCat] = useState<'all' | DestinationTag | 'fav'>((params.cat as DestinationTag) ?? 'all');
  const [sort, setSort] = useState<Sort>('rating');
  const colW = Math.min(width, MAX_WIDTH) - 40;

  const list = useMemo(() => {
    const s = q.trim().toLowerCase();
    return destinations
      .filter((d) => (cat === 'all' ? true : cat === 'fav' ? favorites.includes(d.id) : d.tags.includes(cat)))
      .filter((d) => !s || `${d.city} ${d.country} ${d.highlights.join(' ')}`.toLowerCase().includes(s))
      .sort((a, b) => (sort === 'rating' ? b.rating - a.rating : sort === 'cheap' ? a.dailyCost - b.dailyCost : a.city.localeCompare(b.c
ity)));
  }, [q, cat, sort, favorites]);

  return (
    <Screen header={<Header title="All destinations" subtitle={`${list.length} places to explore`} />}>
      <View style={styles.search}>
        <Ionicons name="search" size={18} color={colors.textMuted} />
        <TextInput value={q} onChangeText={setQ} placeholder={t('explore.search')} placeholderTextColor={colors.textMuted} style={style
s.input} accessibilityLabel="Search destinations" />
      </View>
      <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 8, paddingHorizontal: 20 }} style={{ m
arginHorizontal: -20, marginTop: 12 }}>
        <Chip label="★ Favourites" active={cat === 'fav'} onPress={() => setCat('fav')} />
        {exploreCategories.map((c) => (
          <Chip key={c.key} label={t(c.labelKey)} image={c.image} active={cat === c.key} onPress={() => setCat(c.key)} />
        ))}
      </ScrollView>
      <View style={styles.sortRow}>
        <T variant="caption" color={colors.textSecondary}>
          Sort by
        </T>
        {(
          [
            ['rating', 'Top rated'],
            ['cheap', 'Budget-friendly'],
            ['name', 'A–Z'],
          ] as [Sort, string][]
        ).map(([k, l]) => (
          <Chip key={k} small label={l} active={sort === k} onPress={() => setSort(k)} />
        ))}
      </View>
      {list.length ? (
        <View style={styles.grid}>
          {list.map((d) => (
            <View key={d.id} style={{ width: (colW - 12) / 2, gap: 4 }}>
              <PlaceCard d={d} height={170} />
              <T variant="caption" color={colors.textSecondary}>
                ~${d.dailyCost}/day · {d.bestMonths.split(',')[0]}
              </T>
            </View>
          ))}
        </View>
      ) : (
        <EmptyState icon="search-outline" title="No matches" body={cat === 'fav' ? 'Tap the ☆ on any place to save it here.' : 'Try anot
her search or category.'} />
      )}
    </Screen>
  );
}

const styles = StyleSheet.create({
  search: { flexDirection: 'row', alignItems: 'center', gap: 10, backgroundColor: colors.card, borderRadius: radius.md, borderWidth: 1,
```

```
borderColor: colors.border, paddingHorizontal: 14, height: 48 },
  input: { flex: 1, fontFamily: fonts.regular, fontSize: 15, color: colors.text, height: '100%', ...({ outlineStyle: 'none' } as object)
},
  sortRow: { flexDirection: 'row', alignItems: 'center', gap: 6, marginVertical: 14 },
  grid: { flexDirection: 'row', flexWrap: 'wrap', gap: 12 },
});
```

src/app/documents.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import * as Linking from 'expo-linking';
import * as Sharing from 'expo-sharing';
import { useLocalSearchParams } from 'expo-router';
import { useState } from 'react';
import { Platform, StyleSheet, View } from 'react-native';

import { NoTrip } from '@/components/trip/NoTrip';
import { Button } from '@/components/ui/Button';
import { Card, EmptyState, Pill } from '@/components/ui/bits';
import { confirmAction, toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { Press, tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import { FileTooLargeError, deleteLocalFile, pickDocument, pickImage, sizeLabel } from '@/services/files';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import type { TripDoc } from '@/store/types';
import { colors, radius } from '@/theme';
import { timeAgo } from '@/utils/format';
import { firstError, maxLen, required } from '@/utils/validation';

/** Offline document vault: tickets, bookings, passport copies and notes. */
export default function Documents() {
  const params = useLocalSearchParams<{ open?: string }>();
  const trip = useActiveTrip();
  const userId = useAppStore((s) => s.session?.userId ?? '');
  const addDoc = useAppStore((s) => s.addDoc);
  const deleteDoc = useAppStore((s) => s.deleteDoc);
  const [open, setOpen] = useState<TripDoc | null>(() => (params.open ? trip?.docs.find((d) => d.id === params.open) ?? null : null));
  const [noteOpen, setNoteOpen] = useState(false);
  const [name, setName] = useState('');
  const [note, setNote] = useState('');
  const [errors, setErrors] = useState<Record<string, string | null>>({});

  if (!trip) return <NoTrip />;

  const upload = async (kind: 'file' | 'photo') => {
    try {
      const f = kind === 'photo' ? await pickImage() : await pickDocument();
      if (!f) return;
      addDoc(trip.id, { name: f.name.replace(/\.[^.]+$/, ''), kind: f.kind, uri: f.uri, size: f.size, offline: true });
      tap('success');
      toast('Saved to the vault — available offline');
    } catch (e) {
      toast(e instanceof FileTooLargeError ? e.message : 'Couldn’t save that file', { tone: 'warn' });
    }
  };

  const openFile = async (d: TripDoc) => {
    if (!d.uri) return;
    try {
      if (Platform.OS !== 'web' && (await Sharing.isAvailableAsync())) await Sharing.shareAsync(d.uri, { dialogTitle: d.name });
      else await Linking.openURL(d.uri);
    } catch {
      toast('No app available to open this file', { tone: 'warn' });
    }
  };

  const used = trip.docs.reduce((n, d) => n + (d.size ?? (d.note?.length ?? 0)), 0);

  return (
    <Screen header={<Header title="Documents" subtitle={`${trip.docs.length} items · all available offline`} />}>
      <Card style={styles.banner}>
        <Ionicons name="cloud-done-outline" size={22} color={colors.primary} />
        <View style={{ flex: 1 }}>
          <T variant="small" weight="semibold">
            Works without wifi or data
          </T>
          <T variant="caption" color={colors.textSecondary}>
            Files are copied into TogetherWeGo’s private storage on this phone · {sizeLabel(used) || '0 KB'} used
          </T>
        </View>
      </Card>
      <View style={{ flexDirection: 'row', gap: 8, marginTop: 12 }}>
        <Button label="File" icon="document-attach-outline" size="md" onPress={() => upload('file')} style={{ flex: 1 }} />
        <Button label="Photo" icon="image-outline" size="md" variant="secondary" onPress={() => upload('photo')} style={{ flex: 1 }} />
        <Button label="Note" icon="create-outline" size="md" variant="secondary" onPress={() => setNoteOpen(true)} style={{ flex: 1 }} /
>
      </View>

      <View style={{ gap: 8, marginTop: 16 }}>
```

```
        {trip.docs.length ? (
          trip.docs.map((d) => (
            <Press key={d.id} onPress={() => setOpen(d)} style={styles.row} scaleTo={0.99} accessibilityLabel={`Open ${d.name}`}>
              <View style={styles.icon}>
                <Ionicons name={d.kind === 'image' ? 'image-outline' : d.kind === 'note' ? 'document-text-outline' : d.kind === 'pdf' ?
'document-outline' : 'attach-outline'} size={20} color={colors.primary} />
              </View>
              <View style={{ flex: 1 }}>
                <T variant="small" weight="semibold" numberOfLines={1}>
                  {d.name}
                </T>
                <T variant="caption" color={colors.textMuted}>
                  {d.kind === 'note' ? 'Note' : d.kind.toUpperCase()} {d.size ? `· ${sizeLabel(d.size)}` : ''} · {timeAgo(d.addedAt)} ·
{trip.members.find((m) => m.id === d.addedBy)?.name.split(' ')[0]}
                </T>
              </View>
              <Pill label="Offline" icon="cloud-done-outline" />
            </Press>
          ))
        ) : (
          <EmptyState icon="folder-open-outline" title="No documents yet" body="Add tickets, bookings and passport copies so they're alw
ays on hand." />
        )}
      </View>

      <Sheet visible={!!open} onClose={() => setOpen(null)} title={open?.name} subtitle={open ? `${open.kind === 'note' ? 'Note' : open.
kind.toUpperCase()} · added ${timeAgo(open.addedAt)}` : undefined}>
        {open?.kind === 'note' ? (
          <View style={styles.note}>
            <T variant="bodySm" selectable>
              {open.note}
            </T>
          </View>
        ) : null}
        {open?.kind === 'image' && open.uri ? <Image source={{ uri: open.uri }} style={{ width: '100%', height: 320, borderRadius: 14 }}
contentFit="contain" /> : null}
        {open && open.kind !== 'note' && open.uri ? <Button label="Open / share file" icon="open-outline" onPress={() => openFile(open)}
/> : null}
        {open && (open.addedBy === userId || trip.ownerId === userId) ? (
          <Button
            label="Delete"
            variant="danger"
            icon="trash-outline"
            onPress={() => {
              const d = open;
              setOpen(null);
              confirmAction('Delete document?', `"${d.name}" will be removed from the vault.`, () => {
                deleteLocalFile(d.uri);
                deleteDoc(trip.id, d.id);
                toast('Document deleted');
              }, { destructive: true, confirmLabel: 'Delete' });
            }}
          />
        ) : null}
      </Sheet>

      <Sheet
        visible={noteOpen}
        onClose={() => setNoteOpen(false)}
        title="New note"
        subtitle="Addresses, booking refs, allergy cards…"
        footer={
          <Button
            label="Save note"
            icon="checkmark"
            onPress={() => {
              const e = { name: firstError(name, required('a title'), maxLen(40, 'Title')), note: firstError(note, required('some tex
t'), maxLen(2000, 'Note')) };
              setErrors(e);
              if (e.name || e.note) return tap('warning');
              addDoc(trip.id, { name: name.trim(), kind: 'note', note: note.trim(), offline: true });
              setName('');
              setNote('');
              setNoteOpen(false);
              toast('Note saved offline');
            }}
          />
        }>
        <Input label="Title" value={name} onChangeText={setName} error={errors.name} placeholder="e.g. Hotel address in Japanese" />
        <Input label="Note" value={note} onChangeText={setNote} error={errors.note} multiline placeholder="Write anything you'll need of
fline…" style={{ minHeight: 140 }} />
      </Sheet>
    </Screen>
  );
}
```

```
const styles = StyleSheet.create({
  banner: { flexDirection: 'row', alignItems: 'center', gap: 12, backgroundColor: colors.primarySofter, borderColor: colors.primaryLine
},
  row: { flexDirection: 'row', alignItems: 'center', gap: 12, backgroundColor: colors.card, padding: 12, borderRadius: radius.md, border
Width: 1, borderColor: colors.border },
  icon: { width: 40, height: 40, borderRadius: 12, backgroundColor: colors.primarySofter, alignItems: 'center', justifyContent: 'center'
},
  note: { backgroundColor: colors.cardMuted, padding: 14, borderRadius: radius.md, borderWidth: 1, borderColor: colors.border },
});
```

src/app/emergency.tsx

```
import { Ionicons } from '@expo/vector-icons';
import * as Linking from 'expo-linking';
import { router } from 'expo-router';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { ShareSheet } from '@/components/ShareSheet';
import { Button } from '@/components/ui/Button';
import { Card, ListRow } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Press } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { emergencyByCountry } from '@/data/reference';
import { getLocation } from '@/services/location';
import { useActiveTrip } from '@/store/useAppStore';
import { colors, radius } from '@/theme';

/** Safety screen — everything here works offline except the map searches. */
export default function Emergency() {
  const trip = useActiveTrip();
  const dest = destinationById(trip?.destinationId ?? 'tokyo')!;
  const info = emergencyByCountry[dest.countryCode] ?? emergencyByCountry.DEFAULT;
  const [shareMsg, setShareMsg] = useState<string | null>(null);

  const call = (n: string) => Linking.openURL(`tel:${n}`).catch(() => toast(`Dial ${n} from your phone`, { tone: 'warn' }));

  const shareLocation = async () => {
    const loc = await getLocation(dest);
    if (loc.source !== 'device') toast('GPS unavailable — sharing the trip centre instead', { tone: 'warn' });
    setShareMsg(`🚨 I need help. My location: https://maps.google.com/?q=${loc.lat.toFixed(5)},${loc.lng.toFixed(5)} — sent from TogetherWeGo (${trip?.name ?? 'trip'})`);
  };

  return (
    <Screen header={<Header title="Emergency & safety" subtitle={`${dest.country} · works offline`} />}>
      <View style={styles.grid}>
        {[
          ['Police', info.police, 'shield-outline', '#2B5FB0'],
          ['Ambulance', info.ambulance, 'medkit-outline', colors.red],
          ['Fire', info.fire, 'flame-outline', colors.orange],
        ].map(([label, num, icon, color]) => (
          <Press key={label} onPress={() => call(num)} style={[styles.callBox, { borderColor: `${color}55` }]} accessibilityLabel={`Call ${label}, ${num}`}>
            <Ionicons name={icon as 'shield-outline'} size={24} color={color} />
            <T variant="h1" color={color}>
              {num}
            </T>
            <T variant="caption" weight="semibold" color={colors.textSecondary}>
              {label}
            </T>
          </Press>
        ))}
      </View>

      <Button label="Share my live location with the group" icon="location" variant="danger" onPress={shareLocation} style={{ marginTop: 14 }} />

      <Card style={{ marginTop: 14, gap: 8 }}>
        <T variant="title">Good to know in {dest.country}</T>
        {info.tips.map((tip) => (
          <View key={tip} style={{ flexDirection: 'row', gap: 8 }}>
            <Ionicons name="information-circle-outline" size={16} color={colors.primary} style={{ marginTop: 2 }} />
            <T variant="small" style={{ flex: 1 }}>
              {tip}
            </T>
          </View>
        ))}
      </Card>

      <Card style={{ marginTop: 14 }}>
        <ListRow icon="medical-outline" label="Find the nearest hospital" sub="Opens maps around you" tint={colors.red} onPress={() => Linking.openURL('https://www.google.com/maps/search/hospital+near+me')} />
        <ListRow icon="flag-outline" label="Find my embassy" sub={`Embassies and consulates in ${dest.city}`} onPress={() => Linking.openURL(`https://www.google.com/maps/search/embassy+${encodeURIComponent(dest.city)}`)} />
        <ListRow icon="document-lock-outline" label="Passport copies & insurance" sub="Your offline document vault" onPress={() => router.push('/documents')} />
        <ListRow icon="language-outline" label="Emergency phrases" sub={`“Help!” and more in ${dest.language}`} onPress={() => router.push('/phrasebook')} />
        <ListRow icon="airplane-outline" label="Nearest airport" onPress={() => router.push('/airport')} />
      </Card>

      <ShareSheet visible={!!shareMsg} onClose={() => setShareMsg(null)} title="Send your location" message={shareMsg ?? ''} preview={sh
```

```
areMsg ?? ''} />
    </Screen>
  );
}

const styles = StyleSheet.create({
  grid: { flexDirection: 'row', gap: 10 },
  callBox: { flex: 1, alignItems: 'center', gap: 2, paddingVertical: 16, borderRadius: radius.lg, backgroundColor: colors.card, borderWi
dth: 1.5 },
});
```

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import * as Linking from 'expo-linking';
import { useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';

import { DatePickerSheet, TimePickerSheet } from '@/components/pickers';
import { NoTrip } from '@/components/trip/NoTrip';
import { Button } from '@/components/ui/Button';
import { Avatar, Card, Chip, ToggleRow } from '@/components/ui/bits';
import { confirmAction, toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { Press, tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import { airportByIata } from '@/data/airports';
import { destinationById } from '@/data/destinations';
import { FileTooLargeError, pickDocument, pickImage } from '@/services/files';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import type { Flight, Member } from '@/store/types';
import { colors, radius } from '@/theme';
import { durationLabel } from '@/utils/format';
import { distanceKm, kmLabel } from '@/utils/geo';
import { addDays, dayLabel, deviceTimeZone, formatInTz, partsInTz, time12, tzAbbrev, utcOffsetLabel, zonedToUtc } from '@/utils/time';
import { maxLen, validateFlightNumber, validateIata } from '@/utils/validation';

const AVATAR_COLORS = ['#F2553D', '#1E88E5', '#2FBF5B', '#E5A100', '#8E5CD6', '#D94F86'];

type Person = { id: string; name: string; member?: Member; flights: Flight[] };

export default function Flights() {
  const trip = useActiveTrip();
  const userId = useAppStore((s) => s.session?.userId ?? '');
  const showTripTime = useAppStore((s) => s.settings.showTripTime);
  const updateSettings = useAppStore((s) => s.updateSettings);
  const deleteFlight = useAppStore((s) => s.deleteFlight);
  const [adding, setAdding] = useState(false);
  if (!trip) return <NoTrip />;
  const dest = destinationById(trip.destinationId)!;
  const tz = showTripTime ? dest.tz : deviceTimeZone();
  const sorted = trip.flights.slice().sort((a, b) => (a.direction === 'arrival' ? a.arriveUtc : a.departUtc) - (b.direction === 'arriva
l' ? b.arriveUtc : b.departUtc));
  const arrivals = sorted.filter((f) => f.direction === 'arrival');
  const firstLand = arrivals[0];
  const lastLand = arrivals[arrivals.length - 1];

  const everyone: Person[] = trip.members.map((m) => ({ id: m.id, name: m.id === userId ? 'You' : m.name, member: m, flights: sorted.fil
ter((f) => f.memberId === m.id) }));
  const former = sorted.filter((f) => !trip.members.some((m) => m.id === f.memberId));
  if (former.length) everyone.push({ id: 'former', name: 'Former member', flights: former });
  const people = [...everyone.filter((p) => p.flights.length), ...everyone.filter((p) => !p.flights.length)];

  const when = (utc: number) => {
    const p = partsInTz(utc, tz);
    return `${time12(p.hhmm)} local · ${dayLabel(p.ymd)}`;
  };

  const detail = (f: Flight) => {
    const from = airportByIata(f.from);
    const to = airportByIata(f.to);
    const canDelete = f.memberId === userId || trip.ownerId === userId;
    const extra = [f.seat ? `Seat ${f.seat}` : null, f.confirmation ? `Conf. ${f.confirmation}` : null, from && to ? `Airport time ${tim
e12(partsInTz(f.departUtc, from.tz).hhmm)} → ${time12(partsInTz(f.arriveUtc, to.tz).hhmm)}` : null]
      .filter(Boolean)
      .join(' · ');
    return (
      <View key={f.id} style={styles.detail}>
        <Ionicons name={f.direction === 'arrival' ? 'airplane' : 'airplane-outline'} size={14} color={colors.textMuted} />
        <View style={{ flex: 1 }}>
          <T variant="caption" weight="semibold">
            {f.direction === 'arrival' ? 'Arrival' : 'Departure'} · {f.flightNo} · {f.from} → {f.to}
            <T variant="caption" color={colors.textSecondary}>
              {` · ${durationLabel(Math.round((f.arriveUtc - f.departUtc) / 60000))}`}
            </T>
          </T>
          {extra ? (
            <T variant="caption" color={colors.textMuted}>
              {extra}
            </T>
          ) : null}
        </View>
        {f.ticketUri ? (
          <Press onPress={() => Linking.openURL(f.ticketUri!).catch(() => toast('Ticket saved offline in Documents'))} style={styles.tic
```

```
ket} accessibilityLabel={`Open ticket ${f.ticketName ?? ''}`}>
            {/\.(png|jpe?g|webp|heic)$/i.test(f.ticketName ?? '') || f.ticketUri.startsWith('data:image') ? (
              <Image source={{ uri: f.ticketUri }} style={{ width: 22, height: 22, borderRadius: 6 }} />
            ) : (
              <Ionicons name="document-attach-outline" size={15} color={colors.text} />
            )}
            <T variant="caption" weight="semibold" numberOfLines={1} style={{ maxWidth: 80 }}>
              {f.ticketName ?? 'Ticket'}
            </T>
          </Press>
        ) : null}
        {canDelete ? (
          <Press
            onPress={() => confirmAction('Remove flight?', `${f.flightNo} will be removed from the trip.`, () => deleteFlight(trip.id,
f.id), { destructive: true, confirmLabel: 'Remove' })}
            hitSlop={8}
            accessibilityLabel={`Remove flight ${f.flightNo}`}>
            <Ionicons name="trash-outline" size={17} color={colors.textMuted} />
          </Press>
        ) : null}
      </View>
    );
  };

  const personCard = (p: Person) => {
    const arr = p.flights.filter((f) => f.direction === 'arrival');
    const dep = p.flights.filter((f) => f.direction === 'departure');
    const labels = [...new Set([arr[0], dep[0]].filter((f): f is Flight => !!f).map((f) => `${f.airline} ${f.flightNo}`))];
    const bg = AVATAR_COLORS[(p.name.charCodeAt(0) + p.name.length) % AVATAR_COLORS.length];
    return (
      <Card key={p.id} style={styles.person}>
        <View style={styles.row}>
          {p.member?.avatar ? (
            <Avatar name={p.name} src={p.member.avatar} size={34} />
          ) : (
            <View style={[styles.dot, { backgroundColor: bg }]} accessibilityLabel={p.name}>
              <T variant="small" weight="bold" color={colors.white}>
                {p.name.trim().charAt(0).toUpperCase()}
              </T>
            </View>
          )}
          <T variant="title" weight="bold" numberOfLines={1} style={{ flex: 1 }}>
            {p.name}
          </T>
          <View style={{ alignItems: 'flex-end', gap: 4 }}>
            {labels.map((l) => (
              <View key={l} style={styles.flightPill}>
                <T variant="micro" weight="bold" numberOfLines={1}>
                  {l}
                </T>
              </View>
            ))}
          </View>
        </View>
        {p.flights.length ? (
          <>
            {Array.from({ length: Math.max(arr.length, dep.length) }, (_, i) => (
              <View key={i} style={styles.leg}>
                <View style={{ flex: 1 }}>
                  <T variant="kicker" color={colors.textMuted} style={styles.legLabel}>
                    Arrives
                  </T>
                  <T variant="small" weight="bold" color={arr[i] ? colors.text : colors.textMuted}>
                    {arr[i] ? when(arr[i].arriveUtc) : '—'}
                  </T>
                </View>
                <Ionicons name="airplane" size={16} color={colors.textMuted} />
                <View style={{ flex: 1, alignItems: 'flex-end' }}>
                  <T variant="kicker" color={colors.textMuted} style={styles.legLabel}>
                    Departs
                  </T>
                  <T variant="small" weight="bold" color={dep[i] ? colors.text : colors.textMuted}>
                    {dep[i] ? when(dep[i].departUtc) : '—'}
                  </T>
                </View>
              </View>
            ))}
            <View style={{ gap: 2 }}>{[...arr, ...dep].map(detail)}</View>
          </>
        ) : (
          <T variant="small" color={colors.textMuted}>
            No flight added yet
          </T>
        )}
      </Card>
    );
```

```
  };

  return (
    <Screen
      header={
        <Header
          title="Flights"
          subtitle={`Times shown in ${tz}`}
          right={<Button label="Mine" icon="cloud-upload-outline" size="sm" full={false} onPress={() => setAdding(true)} accessibilityHi
nt="Add your flight or upload a ticket" testID="add-flight" />}
        />
      }>
      <Press onPress={() => updateSettings({ showTripTime: !showTripTime })} style={styles.tz} accessibilityLabel={`Showing ${showTripTi
me ? `${dest.city} time` : 'your time'}. Tap to switch time zone`}>
        <Ionicons name="globe-outline" size={15} color={colors.blue} />
        <T variant="caption" weight="semibold" color={colors.blue}>
          {showTripTime ? `${dest.city} time` : 'Your time'} · {tzAbbrev(tz)} ({utcOffsetLabel(tz)}) · tap to switch
        </T>
        <Ionicons name="swap-horizontal" size={15} color={colors.blue} />
      </Press>

      <View style={{ gap: 12 }}>{people.map(personCard)}</View>

      {firstLand && lastLand ? (
        <Card style={styles.summary}>
          <Ionicons name="people-outline" size={20} color={colors.text} />
          <T variant="caption" style={{ flex: 1 }}>
            First arrival {formatInTz(firstLand.arriveUtc, tz)}, last {formatInTz(lastLand.arriveUtc, tz)} — the group is complete{' '}
            {durationLabel(Math.round((lastLand.arriveUtc - firstLand.arriveUtc) / 60000))} after the first landing.
          </T>
        </Card>
      ) : null}

      <AddFlightSheet visible={adding} onClose={() => setAdding(false)} />
    </Screen>
  );
}

function AddFlightSheet({ visible, onClose }: { visible: boolean; onClose: () => void }) {
  const trip = useActiveTrip()!;
  const dest = destinationById(trip.destinationId)!;
  const userId = useAppStore((s) => s.session?.userId ?? '');
  const addFlight = useAppStore((s) => s.addFlight);
  const addDoc = useAppStore((s) => s.addDoc);
  const [memberId, setMemberId] = useState(userId);
  const [direction, setDirection] = useState<'arrival' | 'departure'>('arrival');
  const [airline, setAirline] = useState('');
  const [flightNo, setFlightNo] = useState('');
  const [from, setFrom] = useState('');
  const [to, setTo] = useState(dest.airport);
  const [depDate, setDepDate] = useState(addDays(trip.startDate, -1));
  const [depTime, setDepTime] = useState('11:00');
  const [arrDate, setArrDate] = useState(trip.startDate);
  const [arrTime, setArrTime] = useState('14:30');
  const [seat, setSeat] = useState('');
  const [conf, setConf] = useState('');
  const [ticket, setTicket] = useState<{ uri: string; name: string; size?: number } | null>(null);
  const [saveDoc, setSaveDoc] = useState(true);
  const [picker, setPicker] = useState<null | 'depDate' | 'depTime' | 'arrDate' | 'arrTime'>(null);
  const [errors, setErrors] = useState<Record<string, string | null>>({});

  const setDir = (d: 'arrival' | 'departure') => {
    setDirection(d);
    if (d === 'arrival') {
      setTo(to || 'HND');
      setDepDate(addDays(trip.startDate, -1));
      setArrDate(trip.startDate);
    } else {
      setFrom(from || 'HND');
      setDepDate(trip.endDate);
      setArrDate(trip.endDate);
    }
  };

  const attach = async (kind: 'photo' | 'file') => {
    try {
      const f = kind === 'photo' ? await pickImage() : await pickDocument();
      if (f) {
        setTicket(f);
        toast('Ticket attached');
      }
    } catch (e) {
      toast(e instanceof FileTooLargeError ? e.message : 'Couldn’t attach that file', { tone: 'warn' });
    }
  };
```

```
  const submit = () => {
    const A = airportByIata(from);
    const B = airportByIata(to);
    const e: Record<string, string | null> = {
      airline: airline.trim() ? maxLen(30, 'Airline')(airline) : 'Enter the airline',
      flightNo: validateFlightNumber(flightNo),
      from: validateIata(from, 'departure'),
      to: validateIata(to, 'arrival') ?? (from.trim().toUpperCase() === to.trim().toUpperCase() ? 'Departure and arrival airports must d
iffer' : null),
      seat: seat && !/^\d{1,3}[A-K]$/i.test(seat.trim()) ? 'Seats look like 32A' : null,
    };
    let departUtc = 0;
    let arriveUtc = 0;
    if (A && B && !e.from && !e.to) {
      departUtc = zonedToUtc(depDate, depTime, A.tz);
      arriveUtc = zonedToUtc(arrDate, arrTime, B.tz);
      const hrs = (arriveUtc - departUtc) / 3600000;
      if (hrs <= 0) e.arr = 'Arrival must be after departure (times are local to each airport)';
      else if (hrs > 22) e.arr = `That’s a ${Math.round(hrs)}h flight — double-check the dates`;
    }
    setErrors(e);
    if (Object.values(e).some(Boolean)) return tap('warning');
    const name = trip.members.find((m) => m.id === memberId)?.name.split(' ')[0] ?? '';
    addFlight(trip.id, {
      memberId,
      airline: airline.trim(),
      flightNo: flightNo.trim().toUpperCase().replace(/^([A-Z0-9]{2})\s?/, '$1 '),
      from: from.trim().toUpperCase(),
      to: to.trim().toUpperCase(),
      departUtc,
      arriveUtc,
      direction,
      seat: seat.trim().toUpperCase() || undefined,
      confirmation: conf.trim().toUpperCase() || undefined,
      ticketUri: ticket?.uri,
      ticketName: ticket?.name,
    });
    if (ticket && saveDoc) {
      addDoc(trip.id, { name: `${name}’s ticket · ${flightNo.toUpperCase()}`, kind: /\.pdf$/i.test(ticket.name) ? 'pdf' : 'image', uri:
ticket.uri, size: ticket.size, offline: true });
    }
    const near = B ? distanceKm(B, dest) : 0;
    tap('success');
    toast(direction === 'arrival' && near > 150 ? `Flight saved — note ${to.toUpperCase()} is ${kmLabel(near)} from ${dest.city}` : 'Fli
ght saved ✈️ The group can see it now');
    onClose();
  };

  const A = airportByIata(from);
  const B = airportByIata(to);
  return (
    <Sheet
      visible={visible}
      onClose={onClose}
      title="Add a flight"
      subtitle="Times are local to each airport — we convert them for everyone"
      footer={<Button label="Save flight" icon="airplane" onPress={submit} />}>
      <View style={{ flexDirection: 'row', gap: 8 }}>
        <Chip label="Arriving" icon="airplane" active={direction === 'arrival'} onPress={() => setDir('arrival')} />
        <Chip label="Departing" icon="airplane-outline" active={direction === 'departure'} onPress={() => setDir('departure')} />
      </View>
      <View style={{ gap: 6 }}>
        <T variant="small" weight="semibold">
          Traveller
        </T>
        <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 6 }}>
          {trip.members.map((m) => (
            <Chip key={m.id} small label={m.name.split(' ')[0]} image={m.avatar} active={memberId === m.id} onPress={() => setMemberId
(m.id)} />
          ))}
        </ScrollView>
      </View>
      <View style={{ flexDirection: 'row', gap: 10 }}>
        <Input label="Airline" placeholder="ANA" value={airline} onChangeText={setAirline} error={errors.airline} containerStyle={{ fle
x: 1 }} />
        <Input label="Flight no." placeholder="NH 175" value={flightNo} onChangeText={setFlightNo} autoCapitalize="characters" error={er
rors.flightNo} containerStyle={{ flex: 1 }} />
      </View>
      <View style={{ flexDirection: 'row', gap: 10 }}>
        <Input label="From" placeholder="JFK" value={from} onChangeText={(v) => setFrom(v.toUpperCase().slice(0, 3))} autoCapitalize="ch
aracters" error={errors.from} hint={A ? A.city : undefined} containerStyle={{ flex: 1 }} />
        <Input label="To" placeholder={dest.airport} value={to} onChangeText={(v) => setTo(v.toUpperCase().slice(0, 3))} autoCapitalize
="characters" error={errors.to} hint={B ? B.city : undefined} containerStyle={{ flex: 1 }} />
      </View>
      <T variant="small" weight="semibold">
        Departs {A ? `(${A.city} time)` : ''}
```

```
      </T>
      <View style={{ flexDirection: 'row', gap: 10 }}>
        <Input value={`${dayLabel(depDate)}, ${depDate.slice(0, 4)}`} icon="calendar-outline" onPressField={() => setPicker('depDate')}
containerStyle={{ flex: 1.3 }} />
        <Input value={time12(depTime)} icon="time-outline" onPressField={() => setPicker('depTime')} containerStyle={{ flex: 1 }} />
      </View>
      <T variant="small" weight="semibold">
        Arrives {B ? `(${B.city} time)` : ''}
      </T>
      <View style={{ flexDirection: 'row', gap: 10 }}>
        <Input value={`${dayLabel(arrDate)}, ${arrDate.slice(0, 4)}`} icon="calendar-outline" onPressField={() => setPicker('arrDate')}
containerStyle={{ flex: 1.3 }} />
        <Input value={time12(arrTime)} icon="time-outline" onPressField={() => setPicker('arrTime')} containerStyle={{ flex: 1 }} />
      </View>
      {errors.arr ? (
        <T variant="caption" color={colors.red}>
          {errors.arr}
        </T>
      ) : null}
      <View style={{ flexDirection: 'row', gap: 10 }}>
        <Input label="Seat (optional)" placeholder="32A" value={seat} onChangeText={setSeat} autoCapitalize="characters" error={errors.s
eat} containerStyle={{ flex: 1 }} />
        <Input label="Confirmation (optional)" placeholder="ABC123" value={conf} onChangeText={setConf} autoCapitalize="characters" maxL
ength={10} containerStyle={{ flex: 1 }} />
      </View>
      <View style={{ gap: 8 }}>
        <T variant="small" weight="semibold">
          Ticket / boarding pass
        </T>
        {ticket ? (
          <View style={styles.attached}>
            <Ionicons name="document-attach-outline" size={18} color={colors.primary} />
            <T variant="caption" style={{ flex: 1 }} numberOfLines={1}>
              {ticket.name}
            </T>
            <Press onPress={() => setTicket(null)} accessibilityLabel="Remove attachment">
              <Ionicons name="close-circle" size={18} color={colors.textMuted} />
            </Press>
          </View>
        ) : (
          <View style={{ flexDirection: 'row', gap: 8 }}>
            <Button label="Photo" icon="image-outline" variant="secondary" size="md" onPress={() => attach('photo')} style={{ flex: 1 }}
/>
            <Button label="PDF / file" icon="document-outline" variant="secondary" size="md" onPress={() => attach('file')} style={{ fle
x: 1 }} />
          </View>
        )}
        {ticket ? <ToggleRow icon="cloud-done-outline" label="Also save to offline documents" value={saveDoc} onChange={setSaveDoc} /> :
null}
      </View>

      <DatePickerSheet visible={picker === 'depDate'} value={depDate} onClose={() => setPicker(null)} onChange={setDepDate} title="Depar
ture date" />
      <DatePickerSheet visible={picker === 'arrDate'} value={arrDate} onClose={() => setPicker(null)} onChange={setArrDate} title="Arriv
al date" />
      <TimePickerSheet visible={picker === 'depTime'} value={depTime} onClose={() => setPicker(null)} onChange={setDepTime} />
      <TimePickerSheet visible={picker === 'arrTime'} value={arrTime} onClose={() => setPicker(null)} onChange={setArrTime} />
    </Sheet>
  );
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', alignItems: 'center', gap: 10 },
  tz: { flexDirection: 'row', alignItems: 'center', alignSelf: 'flex-start', gap: 6, marginBottom: 14, paddingVertical: 4 },
  person: { gap: 12, borderRadius: radius.xl, paddingHorizontal: 16, paddingVertical: 14 },
  dot: { width: 34, height: 34, borderRadius: 17, alignItems: 'center', justifyContent: 'center' },
  flightPill: { backgroundColor: colors.bgAlt, borderRadius: radius.pill, paddingHorizontal: 10, paddingVertical: 4, maxWidth: 170 },
  leg: { flexDirection: 'row', alignItems: 'center', gap: 10 },
  legLabel: { fontSize: 9.5, lineHeight: 12, letterSpacing: 1 },
  detail: { flexDirection: 'row', alignItems: 'center', gap: 8, paddingTop: 10, marginTop: 2, borderTopWidth: 1, borderTopColor: colors.
border },
  ticket: { flexDirection: 'row', alignItems: 'center', gap: 6, paddingHorizontal: 8, paddingVertical: 5, borderRadius: radius.pill, bac
kgroundColor: colors.bgAlt },
  summary: { marginTop: 14, flexDirection: 'row', gap: 10, alignItems: 'center', borderRadius: radius.xl },
  attached: { flexDirection: 'row', alignItems: 'center', gap: 8, padding: 12, borderRadius: radius.md, backgroundColor: colors.primaryS
ofter },
});
```

src/app/forgot-password.tsx

```
import { router } from 'expo-router';
import { useState } from 'react';
import { View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { Card } from '@/components/ui/bits';
import { showDialog, toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { useAppStore } from '@/store/useAppStore';
import { colors } from '@/theme';
import { validateEmail, validatePassword } from '@/utils/validation';

/** Two-step reset: request a 6-digit code, then set a new password. Works fully offline. */
export default function ForgotPassword() {
  const accountExists = useAppStore((s) => s.accountExists);
  const resetPassword = useAppStore((s) => s.resetPassword);
  const [step, setStep] = useState<1 | 2>(1);
  const [email, setEmail] = useState('');
  const [code, setCode] = useState('');
  const [sent, setSent] = useState('');
  const [pw, setPw] = useState('');
  const [errors, setErrors] = useState<Record<string, string | null>>({});
  const [loading, setLoading] = useState(false);

  const sendCode = () => {
    const e = validateEmail(email);
    if (e) return setErrors({ email: e });
    if (!accountExists(email)) return setErrors({ email: 'We couldn’t find an account with that email.' });
    const c = String(Math.floor(100000 + Math.random() * 900000));
    setSent(c);
    setErrors({});
    setStep(2);
    showDialog('Check your inbox', `We sent a 6-digit code to ${email.trim()}.\n\nDemo build: your code is ${c}`, [{ label: 'OK' }], 'ma
il-unread-outline');
  };

  const reset = async () => {
    const e = {
      code: !/^\d{6}$/.test(code) ? 'Enter the 6-digit code' : code !== sent ? 'That code doesn’t match' : null,
      pw: validatePassword(pw),
    };
    setErrors(e);
    if (e.code || e.pw) return tap('warning');
    setLoading(true);
    const r = await resetPassword(email, pw);
    setLoading(false);
    if (!r.ok) return setErrors({ code: r.error });
    tap('success');
    toast('Password updated — sign in with your new password');
    router.back();
  };

  return (
    <Screen header={<Header title="Reset password" subtitle={step === 1 ? 'Step 1 of 2 · verify email' : 'Step 2 of 2 · new password'} /
>}>
      <Card>
        <T variant="bodySm" color={colors.textSecondary} style={{ marginBottom: 14 }}>
          {step === 1 ? 'Enter the email you signed up with and we’ll send you a reset code.' : `Enter the code sent to ${email.trim()}
and choose a new password.`}
        </T>
        {step === 1 ? (
          <View style={{ gap: 14 }}>
            <Input label="Email address" icon="mail-outline" value={email} onChangeText={setEmail} keyboardType="email-address" autoCapi
talize="none" error={errors.email} />
            <Button label="Send reset code" onPress={sendCode} icon="paper-plane" />
          </View>
        ) : (
          <View style={{ gap: 14 }}>
            <Input label="6-digit code" icon="key-outline" value={code} onChangeText={(v) => setCode(v.replace(/\D/g, '').slice(0, 6))}
keyboardType="number-pad" error={errors.code} />
            <Input label="New password" icon="lock-closed-outline" value={pw} onChangeText={setPw} secureToggle error={errors.pw} hint
="8+ characters with a letter and a number" />
            <Button label="Update password" onPress={reset} loading={loading} />
            <Button label="Resend code" variant="ghost" size="md" onPress={sendCode} />
          </View>
        )}
      </Card>
    </Screen>
  );
}
```

src/app/help.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router, type Href } from 'expo-router';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { Card, type IconName } from '@/components/ui/bits';
import { Press } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { colors, radius, shadow } from '@/theme';

const JOURNEY: { icon: IconName; title: string; body: string; href: Href }[] = [
  { icon: 'add-circle-outline', title: '1. Create or join a trip', body: 'Pick a destination, dates and a group budget — or enter a friend’s invite code.', href: '/trips' },
  { icon: 'share-social-outline', title: '2. Invite your group', body: 'More → Crew & roles → Share sends it straight to WhatsApp, Instagram, Messages and more.', href: '/crew' },
  { icon: 'people-outline', title: '3. Give everyone a role', body: 'Treasurer, Navigator, Food Lead… tap a member to assign who owns what.', href: '/crew' },
  { icon: 'map-outline', title: '4. Build the plan', body: 'In the Plan tab, add activities per day, filter by type or length, and vote with “I’m in”. Majority confirms.', href: '/(tabs)/itinerary' },
  { icon: 'airplane-outline', title: '5. Add flights & tickets', body: 'Everyone’s arrivals show in one list, converted to the trip’s time zone automatically.', href: '/flights' },
  { icon: 'wallet-outline', title: '6. Track the budget', body: 'In the Budget tab, log expenses in any currency, see the breakdown and settle up with the fewest payments.', href: '/(tabs)/budget' },
  { icon: 'chatbubbles-outline', title: '7. Chat & ask the assistant', body: 'In the Chat tab, message the group, or tap ✨ to ask the Trip Assistant about budget, weather, food or what’s next.', href: '/(tabs)/chat' },
  { icon: 'notifications-outline', title: '8. Get live reminders', body: 'Tap the bell on any activity — everyone gets an alert before it starts.', href: '/settings' },
  { icon: 'trophy-outline', title: '9. Wrap it up', body: 'Mark the trip complete for a shareable recap, final balances and the photo dump.', href: '/recap' },
];

const FAQ = [
  ['Does it work without internet?', 'Yes. Trips, chats, documents, the phrasebook, emergency numbers, the offline map and the Trip Assistant all work offline. Weather and currency rates refresh when you’re back online and are cached meanwhile.'],
  ['Where is my data stored?', 'On your phone, in TogetherWeGo’s private storage. Passwords are salted and hashed; an optional AI key goes in the secure keychain.'],
  ['How do time zones work?', 'Itinerary times are in the destination’s local time. Flights are stored as exact moments, so they display correctly in trip time or your own time — toggle it in Flights.'],
  ['How does voting work?', 'Activities marked “Voting” become “Confirmed” once more than half the group taps “I’m in”. Polls in More let the group pick between options.'],
  ['Can I change the language?', 'Settings → Language: English, Español, Français, 日本語, हिन्दी and 中文.'],
  ['How do I try the demo?', 'Sign in with the demo account on the sign-in screen. Settings → Demo timeline switches the sample trip between upcoming, in progress and completed.'],
];

export default function Help() {
  const [open, setOpen] = useState<number | null>(0);
  return (
    <Screen header={<Header title="How to use TogetherWeGo" subtitle="Your group trip, from first idea to final recap" />}>
      <Card onPress={() => router.push('/how-it-works')} style={styles.how} accessibilityLabel="How it works">
        <View style={styles.howIcon}>
          <Ionicons name="construct-outline" size={20} color={colors.onAccent} />
        </View>
        <View style={{ flex: 1 }}>
          <T variant="title" weight="bold">
            How it works
          </T>
          <T variant="caption" color={colors.textSecondary}>
            How the app is built, how your data is kept safe, and where to find every feature
          </T>
        </View>
        <Ionicons name="chevron-forward" size={16} color={colors.textMuted} />
      </Card>
      <T variant="headline" style={{ marginTop: 22, marginBottom: 10 }} accessibilityRole="header">
        The journey
      </T>
      <View style={{ gap: 8 }}>
        {JOURNEY.map((s) => (
          <Card key={s.title} onPress={() => router.push(s.href)} style={styles.step} accessibilityLabel={s.title}>
            <View style={styles.icon}>
              <Ionicons name={s.icon} size={18} color={colors.onAccent} />
            </View>
            <View style={{ flex: 1 }}>
              <T variant="small" weight="semibold">
                {s.title}
              </T>
              <T variant="caption" color={colors.textSecondary}>
                {s.body}
              </T>
            </View>
            <Ionicons name="chevron-forward" size={16} color={colors.textMuted} />
          </Card>
        ))}
```

```
      </View>
      <T variant="headline" style={{ marginTop: 22, marginBottom: 10 }} accessibilityRole="header">
        FAQ
      </T>
      <View style={{ gap: 8 }}>
        {FAQ.map(([q, a], i) => (
          <Press key={q} onPress={() => setOpen(open === i ? null : i)} style={styles.faq} accessibilityState={{ expanded: open === i }}
>
            <View style={{ flexDirection: 'row', alignItems: 'center', gap: 8 }}>
              <T variant="small" weight="semibold" style={{ flex: 1 }}>
                {q}
              </T>
              <Ionicons name={open === i ? 'chevron-up' : 'chevron-down'} size={16} color={colors.textMuted} />
            </View>
            {open === i ? (
              <T variant="small" color={colors.textSecondary} style={{ marginTop: 6 }}>
                {a}
              </T>
            ) : null}
          </Press>
        ))}
      </View>
    </Screen>
  );
}

const styles = StyleSheet.create({
  how: { flexDirection: 'row', alignItems: 'center', gap: 12, padding: 14, borderRadius: radius.xl },
  howIcon: { width: 42, height: 42, borderRadius: 21, backgroundColor: colors.accent, alignItems: 'center', justifyContent: 'center' },
  step: { flexDirection: 'row', alignItems: 'center', gap: 12, padding: 14, borderRadius: radius.xl },
  icon: { width: 38, height: 38, borderRadius: 19, backgroundColor: colors.accent, alignItems: 'center', justifyContent: 'center' },
  faq: { padding: 14, borderRadius: radius.xl, backgroundColor: colors.card, borderWidth: 1, borderColor: colors.border, ...shadow },
});
```

src/app/how-it-works.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router, type Href } from 'expo-router';
import type { ReactNode } from 'react';
import { StyleSheet, View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { Card, Divider, type IconName } from '@/components/ui/bits';
import { Press } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { useAppStore } from '@/store/useAppStore';
import { colors, radius } from '@/theme';

const LAYERS: { icon: IconName; title: string; where: string; body: string }[] = [
  { icon: 'phone-portrait-outline', title: 'Screens', where: 'src/app · src/components', body: 'Show the data and turn taps into action
s.' },
  { icon: 'funnel-outline', title: 'Selector hooks', where: 'useActiveTrip · balances · settlePlan', body: 'Work out exactly what each s
creen needs.' },
  { icon: 'server-outline', title: 'Zustand + Immer store', where: 'src/store', body: 'One source of truth. Saved to AsyncStorage on eve
ry change.' },
  { icon: 'cloud-outline', title: 'Services', where: 'src/services', body: 'Weather, currency, notifications, AI, files and sharing.' },
];

const SECURITY: [IconName, string, string][] = [
  ['key-outline', 'Passwords are never stored', 'Each one is salted and hashed with SHA-256 (expo-crypto) before it is saved.'],
  ['lock-closed-outline', 'API key in the keychain', 'The optional AI key lives in the iOS Keychain or Android Keystore (expo-secure-sto
re), not in the app database.'],
  ['cloud-offline-outline', 'Offline-first', 'Trips, chats, documents and maps are saved on the phone. Weather and exchange rates are ca
ched for when you lose signal.'],
  ['shield-checkmark-outline', 'Your data stays with you', 'Trips and chats are stored on the device. Only weather, exchange-rate and op
tional AI requests go online.'],
];

const VALIDATION: [string, string, string][] = [
  ['Email', 'Must look like name@mail.com', 'That email doesn’t look right'],
  ['Password', '8+ characters with a letter and a number', 'Use at least 8 characters'],
  ['Expense amount', 'Numbers only, above 0 and at most 1,000,000', 'Use numbers only, e.g. 25 or 25.50'],
  ['Trip dates', 'End on or after the start, up to 60 days', 'End date must be on or after the start date'],
  ['Invite code', 'Three letters, a dash, four digits', 'Codes look like ABC-1234'],
  ['Flight number', 'Airline code plus 1 to 4 digits', 'Flight numbers look like NH 175'],
  ['Packing item', '50 characters or fewer, no duplicates', 'That’s already on the list'],
];

const RUBRIC: { icon: IconName; area: string; where: string; href: Href }[] = [
  { icon: 'people-outline', area: 'Collaboration', where: 'More: members, roles, polls and checklist. Plan: voting. Packing: who’s bring
ing what.', href: '/crew' },
  { icon: 'chatbubbles-outline', area: 'Communication', where: 'Chat tab: group chat and the Scout AI assistant.', href: '/(tabs)/chat'
},
  { icon: 'calendar-outline', area: 'Scheduling', where: 'Plan tab: day-by-day itinerary, reminders, flights and time zones.', href: '/
(tabs)/itinerary' },
  { icon: 'wallet-outline', area: 'Budgeting', where: 'Budget tab: expenses, split, settle up and currency converter.', href: '/(tabs)/b
udget' },
  { icon: 'folder-open-outline', area: 'Organization', where: 'More: offline docs, packing list, weather, photo dump and printable plann
er.', href: '/documents' },
  { icon: 'sparkles-outline', area: 'Other features', where: 'Themes and languages, trip quiz, bucket list, phrasebook and the recap sto
ry card.', href: '/personalize' },
  { icon: 'airplane-outline', area: 'Trip management', where: 'Home: switch trips. Create or join with an invite code. Recap when the tr
ip ends.', href: '/trips' },
];

const LIBRARIES = [
  'Expo SDK 57',
  'React Native 0.86',
  'TypeScript',
  'Expo Router',
  'Zustand',
  'Immer',
  'AsyncStorage',
  'expo-crypto',
  'expo-secure-store',
  'expo-notifications',
  'expo-location',
  'expo-image-picker',
  'expo-document-picker',
  'expo-file-system',
  'expo-print',
  'expo-sharing',
  'expo-speech',
  'expo-haptics',
  'react-native-maps',
  'react-native-svg',
  'Reanimated',
  'Anthropic SDK (optional)',
];
```

```
function Section({ icon, title, children }: { icon: IconName; title: string; children: ReactNode }) {
  return (
    <Card style={styles.card}>
      <View style={styles.sectionHead}>
        <View style={styles.sectionIcon}>
          <Ionicons name={icon} size={17} color={colors.onAccent} />
        </View>
        <T variant="headline" accessibilityRole="header" style={{ flex: 1 }}>
          {title}
        </T>
      </View>
      {children}
    </Card>
  );
}

export default function HowItWorks() {
  const setToured = useAppStore((s) => s.setToured);
  return (
    <Screen header={<Header title="How it works" subtitle="How TogetherWeGo is built and where to find everything" backLabel="More" />}>
      <View style={{ gap: 14 }}>
        <Section icon="layers-outline" title="Architecture">
          <T variant="small" color={colors.textSecondary}>
            A layered, MVVM-style design. Data flows one way: a screen calls a store action, the state updates, and the screen redraws.
          </T>
          <View>
            {LAYERS.map((l, i) => (
              <View key={l.title}>
                <View style={styles.layer}>
                  <View style={styles.layerIcon}>
                    <Ionicons name={l.icon} size={17} color={colors.primary} />
                  </View>
                  <View style={{ flex: 1 }}>
                    <T variant="small" weight="bold">
                      {l.title}
                    </T>
                    <T variant="caption" weight="semibold" color={colors.textMuted}>
                      {l.where}
                    </T>
                    <T variant="caption" color={colors.textSecondary}>
                      {l.body}
                    </T>
                  </View>
                </View>
                {i < LAYERS.length - 1 ? <Ionicons name="arrow-down" size={14} color={colors.borderStrong} style={{ alignSelf: 'center',
marginVertical: 3 }} /> : null}
              </View>
            ))}
          </View>
        </Section>

        <Section icon="shield-checkmark-outline" title="Data & security">
          {SECURITY.map(([icon, title, body]) => (
            <View key={title} style={styles.row}>
              <Ionicons name={icon} size={20} color={colors.primary} style={{ marginTop: 1 }} />
              <View style={{ flex: 1 }}>
                <T variant="small" weight="bold">
                  {title}
                </T>
                <T variant="caption" color={colors.textSecondary}>
                  {body}
                </T>
              </View>
            </View>
          ))}
        </Section>

        <Section icon="checkmark-done-outline" title="Input validation">
          <T variant="small" color={colors.textSecondary}>
            Every form checks the format and whether the value makes sense, then shows a plain message next to the field.
          </T>
          {VALIDATION.map(([field, rule, message], i) => (
            <View key={field}>
              {i ? <Divider style={{ marginBottom: 10 }} /> : null}
              <T variant="small" weight="bold">
                {field}
              </T>
              <T variant="caption" color={colors.textSecondary}>
                {rule}
              </T>
              <View style={styles.msg}>
                <Ionicons name="alert-circle" size={13} color={colors.red} />
                <T variant="caption" color={colors.red} style={{ flex: 1 }}>
                  {message}
                </T>
```

```
              </View>
            </View>
          ))}
        </Section>

        <Section icon="ribbon-outline" title="Rubric map">
          <T variant="small" color={colors.textSecondary}>
            Each competition area and where to find it. Tap a row to open it.
          </T>
          {RUBRIC.map((r, i) => (
            <View key={r.area}>
              {i ? <Divider style={{ marginBottom: 10 }} /> : null}
              <Press onPress={() => router.push(r.href)} style={styles.row} accessibilityLabel={`${r.area}. ${r.where}`}>
                <Ionicons name={r.icon} size={20} color={colors.primary} style={{ marginTop: 1 }} />
                <View style={{ flex: 1 }}>
                  <T variant="small" weight="bold">
                    {r.area}
                  </T>
                  <T variant="caption" color={colors.textSecondary}>
                    {r.where}
                  </T>
                </View>
                <Ionicons name="chevron-forward" size={16} color={colors.textMuted} style={{ marginTop: 2 }} />
              </Press>
            </View>
          ))}
        </Section>

        <Section icon="cube-outline" title="Libraries used">
          <View style={styles.tags}>
            {LIBRARIES.map((l) => (
              <View key={l} style={styles.tag}>
                <T variant="caption" weight="semibold">
                  {l}
                </T>
              </View>
            ))}
          </View>
        </Section>
      </View>

      <Button
        label="Replay the app tour"
        icon="play-circle-outline"
        style={{ marginTop: 18 }}
        onPress={() => {
          setToured(false);
          router.navigate('/(tabs)/explore');
        }}
      />
    </Screen>
  );
}

const styles = StyleSheet.create({
  card: { borderRadius: radius.xl, gap: 12 },
  sectionHead: { flexDirection: 'row', alignItems: 'center', gap: 10 },
  sectionIcon: { width: 32, height: 32, borderRadius: 16, backgroundColor: colors.accent, alignItems: 'center', justifyContent: 'center'
},
  layer: { flexDirection: 'row', alignItems: 'center', gap: 12, padding: 12, borderRadius: radius.md, backgroundColor: colors.cardMuted,
borderWidth: 1, borderColor: colors.border },
  layerIcon: { width: 36, height: 36, borderRadius: 18, backgroundColor: colors.card, alignItems: 'center', justifyContent: 'center', bo
rderWidth: 1, borderColor: colors.border },
  row: { flexDirection: 'row', alignItems: 'flex-start', gap: 12 },
  msg: { flexDirection: 'row', alignItems: 'center', gap: 5, marginTop: 5, paddingHorizontal: 10, paddingVertical: 5, borderRadius: radi
us.sm, backgroundColor: colors.redSoft, alignSelf: 'flex-start' },
  tags: { flexDirection: 'row', flexWrap: 'wrap', gap: 6 },
  tag: { paddingHorizontal: 11, paddingVertical: 6, borderRadius: radius.pill, backgroundColor: colors.bgAlt },
});
```

src/app/index.tsx

```
import { Redirect } from 'expo-router';

import { useAppStore } from '@/store/useAppStore';

/** Entry point: routes to onboarding, sign-in or the app depending on state. */
export default function Index() {
  const signedIn = useAppStore((s) => !!s.session);
  const hasOnboarded = useAppStore((s) => s.hasOnboarded);
  if (signedIn) return <Redirect href="/(tabs)/explore" />;
  return <Redirect href={hasOnboarded ? '/sign-in' : '/onboarding'} />;
}
```

src/app/join.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router, useLocalSearchParams } from 'expo-router';
import { useState } from 'react';
import { StyleSheet, TextInput, View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { Card } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Press, tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { useAppStore } from '@/store/useAppStore';
import { colors, fonts, radius } from '@/theme';
import { normalizeInvite, validateInvite } from '@/utils/validation';

/** Join a trip with an invite code (also opened by togetherwego://join?code=XXX-1234 links). */
export default function Join() {
  const params = useLocalSearchParams<{ code?: string }>();
  const joinTrip = useAppStore((s) => s.joinTrip);
  const [code, setCode] = useState(params.code ? normalizeInvite(String(params.code)) : '');
  const [error, setError] = useState<string | null>(null);

  const submit = () => {
    const e = validateInvite(code);
    if (e) {
      setError(e);
      return tap('warning');
    }
    const r = joinTrip(code);
    if (!r.ok) {
      setError(r.error);
      return tap('warning');
    }
    tap('success');
    toast('You’re in! 🎉 Say hi in the trip chat');
    router.dismissAll?.();
    router.replace('/(tabs)/chat');
  };

  return (
    <Screen header={<Header title="Join a trip" subtitle="Enter the code your friend shared" />}>
      <View style={styles.hero}>
        <View style={styles.heroIcon}>
          <Ionicons name="people" size={30} color={colors.primary} />
        </View>
        <T variant="h2" center>
          Got an invite code?
        </T>
        <T variant="small" color={colors.textSecondary} center>
          Codes look like <T variant="small" weight="bold">AML-2026</T>. Ask the organizer to share it from More → Crew & roles.
        </T>
      </View>
      <TextInput
        value={code}
        onChangeText={(v) => {
          setCode(normalizeInvite(v));
          setError(null);
        }}
        placeholder="ABC-1234"
        placeholderTextColor={colors.borderStrong}
        autoCapitalize="characters"
        autoCorrect={false}
        maxLength={8}
        style={[styles.code, error && { borderColor: colors.red }]}
        accessibilityLabel="Invite code"
        onSubmitEditing={submit}
        returnKeyType="go"
        testID="invite-code"
      />
      {error ? (
        <T variant="small" color={colors.red} center style={{ marginTop: 8 }} accessibilityLiveRegion="assertive">
          {error}
        </T>
      ) : null}
      <Button label="Join trip" icon="enter-outline" onPress={submit} style={{ marginTop: 18 }} />
      <Card style={{ marginTop: 18, gap: 8 }}>
        <T variant="small" weight="semibold">
          Try it with a demo trip
        </T>
        <T variant="caption" color={colors.textSecondary}>
          “Seoul Food Crawl” is waiting for new members. Tap to fill its code:
        </T>
        <Press onPress={() => setCode('SEO-5521')} style={styles.demo} accessibilityLabel="Use demo code SEO-5521">
          <T variant="title" weight="bold" color={colors.primary} style={{ letterSpacing: 2 }}>
```

```
            SEO-5521
          </T>
          <Ionicons name="arrow-forward-circle" size={22} color={colors.primary} />
        </Press>
      </Card>
    </Screen>
  );
}

const styles = StyleSheet.create({
  hero: { alignItems: 'center', gap: 8, marginVertical: 18 },
  heroIcon: { width: 70, height: 70, borderRadius: 35, backgroundColor: colors.primarySoft, alignItems: 'center', justifyContent: 'cente
r', marginBottom: 6 },
  code: { height: 72, borderRadius: radius.lg, borderWidth: 2, borderColor: colors.primaryLine, backgroundColor: colors.card, textAlign:
'center', fontFamily: fonts.bold, fontSize: 30, letterSpacing: 6, color: colors.text },
  demo: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', padding: 12, borderRadius: radius.md, backgroundC
olor: colors.primarySofter },
});
```

src/app/map.tsx

```
import { Ionicons } from '@expo/vector-icons';
import * as Linking from 'expo-linking';
import { router } from 'expo-router';
import { useMemo, useState } from 'react';
import { Platform, ScrollView, StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { OfflineRadar } from '@/components/map/OfflineRadar';
import { TripMap } from '@/components/map/TripMap';
import type { MapPoint } from '@/components/map/types';
import { typeMeta } from '@/components/trip/meta';
import { NoTrip } from '@/components/trip/NoTrip';
import { Chip, IconButton } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Press } from '@/components/ui/Press';
import { Slider } from '@/components/ui/Slider';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { placesFor } from '@/data/places';
import { getLocation } from '@/services/location';
import { useActiveTrip } from '@/store/useAppStore';
import { MAX_WIDTH, colors, radius, shadowStrong } from '@/theme';
import { durationLabel } from '@/utils/format';
import { distanceKm, kmLabel, nearestAirports } from '@/utils/geo';
import { dayLabel, time12, tripDays } from '@/utils/time';

export default function MapScreen() {
  const trip = useActiveTrip();
  const insets = useSafeAreaInsets();
  const [mode, setMode] = useState<'live' | 'offline'>('live');
  const [day, setDay] = useState<string>('all');
  const [layers, setLayers] = useState({ plan: true, places: true, airports: true });
  const [radiusKm, setRadiusKm] = useState(10);
  const [selected, setSelected] = useState<string | null>(null);
  const [me, setMe] = useState<{ lat: number; lng: number } | null>(null);
  const dest = destinationById(trip?.destinationId ?? 'tokyo')!;
  const [center, setCenter] = useState({ lat: dest.lat, lng: dest.lng });

  const points = useMemo<MapPoint[]>(() => {
    if (!trip) return [];
    const out: MapPoint[] = [];
    if (layers.plan)
      trip.activities
        .filter((a) => a.lat && a.lng && (day === 'all' || a.date === day))
        .forEach((a) => out.push({ id: a.id, lat: a.lat!, lng: a.lng!, title: a.title, subtitle: `${dayLabel(a.date)} · ${time12(a.tim
e)} · ${durationLabel(a.durationMin)}`, color: typeMeta(a.type).color, kind: 'activity' }));
    if (layers.places) {
      const planned = new Set(trip.activities.map((a) => a.placeId));
      [trip.destinationId, ...trip.extraDestinationIds]
        .flatMap((id) => placesFor(id))
        .filter((p) => !planned.has(p.id))
        .forEach((p) => out.push({ id: p.id, lat: p.lat, lng: p.lng, title: p.name, subtitle: `${p.category} · ★ ${p.rating}`, color: '#
9AA390', kind: 'place' }));
    }
    if (layers.airports) nearestAirports(center, 3).forEach((a) => out.push({ id: `air-${a.iata}`, lat: a.lat, lng: a.lng, title: `${a.i
ata} · ${a.name}`, subtitle: `${kmLabel(a.km)} away`, color: colors.blue, kind: 'airport' }));
    if (me) out.push({ id: 'me', lat: me.lat, lng: me.lng, title: 'You are here', subtitle: '', color: colors.red, kind: 'me' });
    return out;
  }, [trip, layers, day, center, me]);

  if (!trip) return <NoTrip />;
  const sel = points.find((p) => p.id === selected);
  const days = tripDays(trip.startDate, trip.endDate);

  const locate = async () => {
    const loc = await getLocation(dest);
    if (loc.source !== 'device') return toast('Location unavailable — showing the trip centre', { tone: 'warn' });
    setMe(loc);
    if (distanceKm(loc, dest) < 80) setCenter(loc);
    else toast(`You're ${kmLabel(distanceKm(loc, dest))} from ${dest.city}`, { icon: 'navigate-outline' });
  };

  return (
    <View style={styles.root}>
      <View style={styles.mapWrap}>
        {mode === 'live' ? (
          <TripMap center={center} points={points} radiusKm={radiusKm} selectedId={selected} onSelect={setSelected} showsUser={!!me} />
        ) : (
          <OfflineRadar center={center} points={points} radiusKm={radiusKm} selectedId={selected} onSelect={setSelected} />
        )}
      </View>

      <View style={[styles.top, { paddingTop: insets.top + 8 }]}>
        <View style={styles.topRow}>
```

```
          <IconButton icon="chevron-back" label="Back" onPress={() => (router.canGoBack() ? router.back() : router.replace('/(tabs)/expl
ore'))} bordered={false} />
          <View style={styles.seg}>
            {(['live', 'offline'] as const).map((m) => (
              <Press key={m} onPress={() => setMode(m)} style={[styles.segBtn, mode === m && styles.segOn]} accessibilityState={{ select
ed: mode === m }}>
                <Ionicons name={m === 'live' ? 'map' : 'cloud-offline-outline'} size={13} color={mode === m ? colors.white : colors.text
Secondary} />
                <T variant="caption" weight="semibold" color={mode === m ? colors.white : colors.textSecondary}>
                  {m === 'live' ? 'Map' : 'Offline'}
                </T>
              </Press>
            ))}
          </View>
          <IconButton icon="locate-outline" label="Show my location" onPress={locate} bordered={false} />
        </View>
        <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 6, paddingHorizontal: 16 }}>
          <Chip small label="All days" active={day === 'all'} onPress={() => setDay('all')} />
          {days.map((d, i) => (
            <Chip key={d} small label={`Day ${i + 1}`} active={day === d} onPress={() => setDay(d)} />
          ))}
        </ScrollView>
        <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 6, paddingHorizontal: 16 }}>
          <Chip small icon="calendar-outline" label="Itinerary" active={layers.plan} onPress={() => setLayers((l) => ({ ...l, plan: !l.p
lan }))} />
          <Chip small icon="compass-outline" label="Places" active={layers.places} onPress={() => setLayers((l) => ({ ...l, places: !l.p
laces }))} />
          <Chip small icon="airplane-outline" label="Airports" active={layers.airports} onPress={() => setLayers((l) => ({ ...l, airport
s: !l.airports }))} />
        </ScrollView>
      </View>

      <View style={[styles.bottom, { paddingBottom: insets.bottom + 12 }]}>
        {sel ? (
          <View style={styles.selCard}>
            <View style={[styles.selDot, { backgroundColor: sel.color }]} />
            <View style={{ flex: 1 }}>
              <T variant="small" weight="semibold" numberOfLines={1}>
                {sel.title}
              </T>
              <T variant="caption" color={colors.textSecondary} numberOfLines={1}>
                {sel.subtitle} · {kmLabel(distanceKm(center, sel))} from centre
              </T>
            </View>
            <Press onPress={() => Linking.openURL(`https://www.google.com/maps/dir/?api=1&destination=${sel.lat},${sel.lng}`)} style={st
yles.go} accessibilityLabel="Get directions">
              <Ionicons name="navigate" size={16} color={colors.white} />
            </Press>
          </View>
        ) : (
          <T variant="caption" color={colors.textSecondary}>
            {points.length} pins · tap one for details{mode === 'offline' ? ' · works with no internet' : Platform.OS === 'web' ? ' · we
b preview uses OpenStreetMap' : ''}
          </T>
        )}
        <View style={styles.radiusRow}>
          <T variant="caption" weight="semibold">
            Radius {radiusKm} km
          </T>
          <View style={{ flex: 1 }}>
            <Slider value={radiusKm} min={2} max={40} onChange={setRadiusKm} label="Map radius" formatValue={(v) => `${v} km`} />
          </View>
        </View>
        {mode === 'live' && Platform.OS === 'web' && points.length ? (
          <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 6 }}>
            {points.filter((p) => p.kind !== 'place').slice(0, 12).map((p) => (
              <Chip key={p.id} small label={p.title.slice(0, 22)} active={p.id === selected} onPress={() => setSelected(p.id)} />
            ))}
          </ScrollView>
        ) : null}
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.bg },
  mapWrap: { ...StyleSheet.absoluteFill },
  top: { position: 'absolute', left: 0, right: 0, top: 0, gap: 8, alignItems: 'center' },
  topRow: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', width: '100%', maxWidth: MAX_WIDTH, paddingHori
zontal: 16 },
  seg: { flexDirection: 'row', backgroundColor: colors.card, borderRadius: 999, padding: 3, ...shadowStrong },
  segBtn: { flexDirection: 'row', alignItems: 'center', gap: 4, paddingHorizontal: 12, paddingVertical: 7, borderRadius: 999 },
  segOn: { backgroundColor: colors.primary },
  bottom: { position: 'absolute', left: 12, right: 12, bottom: 0, backgroundColor: colors.card, borderTopLeftRadius: radius.xl, borderTo
pRightRadius: radius.xl, padding: 16, gap: 8, alignSelf: 'center', maxWidth: MAX_WIDTH, ...shadowStrong },
```

```
  selCard: { flexDirection: 'row', alignItems: 'center', gap: 10 },
  selDot: { width: 12, height: 12, borderRadius: 6 },
  go: { width: 38, height: 38, borderRadius: 19, backgroundColor: colors.primary, alignItems: 'center', justifyContent: 'center' },
  radiusRow: { flexDirection: 'row', alignItems: 'center', gap: 12 },
});
```

src/app/nearby.tsx

```
import { Ionicons } from '@expo/vector-icons';
import * as Linking from 'expo-linking';
import { useLocalSearchParams } from 'expo-router';
import { useMemo, useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';

import { AddActivitySheet, type ActivityDraft } from '@/components/trip/AddActivitySheet';
import { placeCategoryIcon, placeToActivityType } from '@/components/trip/meta';
import { Button } from '@/components/ui/Button';
import { Card, Chip, EmptyState, Pill } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Press } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { Slider } from '@/components/ui/Slider';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { placesFor, type PlaceCategory } from '@/data/places';
import { useT } from '@/i18n';
import { getLocation } from '@/services/location';
import { useActiveTrip } from '@/store/useAppStore';
import { colors, radius } from '@/theme';
import { LENGTH_FILTERS, durationLabel, lengthBucketLabel, matchesLength, money, priceTier, type LengthFilter } from '@/utils/format';
import { bearingDeg, compassPoint, distanceKm, kmLabel, type LatLng } from '@/utils/geo';
import { todayInTz, tripDays } from '@/utils/time';

const CATS: ('all' | PlaceCategory)[] = ['all', 'food', 'sightseeing', 'adventure', 'culture', 'nightlife', 'shopping', 'relax'];
type Sort = 'distance' | 'rating' | 'price';

export default function Nearby() {
  const t = useT();
  const params = useLocalSearchParams<{ focus?: string; maxCost?: string }>();
  const trip = useActiveTrip();
  const dest = destinationById(trip?.destinationId ?? 'tokyo')!;
  const [center, setCenter] = useState<LatLng & { label: string }>({ lat: dest.lat, lng: dest.lng, label: `Central ${dest.city}` });
  const [radiusKm, setRadiusKm] = useState(params.focus ? 25 : 6);
  const [cat, setCat] = useState<'all' | PlaceCategory>('all');
  const [len, setLen] = useState<LengthFilter>('any');
  const [maxCost, setMaxCost] = useState(params.maxCost ? Math.min(100, Number(params.maxCost)) : 100);
  const [sort, setSort] = useState<Sort>(params.focus ? 'distance' : 'rating');
  const [locating, setLocating] = useState(false);
  const [draft, setDraft] = useState<ActivityDraft | null>(null);

  const all = useMemo(() => (trip ? [trip.destinationId, ...trip.extraDestinationIds] : ['tokyo']).flatMap((id) => placesFor(id)), [tri
p]);
  const results = useMemo(
    () =>
      all
        .map((p) => ({ p, km: distanceKm(center, p) }))
        .filter(({ p, km }) => km <= radiusKm && (cat === 'all' || p.category === cat) && matchesLength(p.durationMin, len) && (maxCost
>= 100 || p.cost <= maxCost))
        .sort((a, b) => (sort === 'distance' ? a.km - b.km : sort === 'rating' ? b.p.rating - a.p.rating : a.p.cost - b.p.cost)),
    [all, center, radiusKm, cat, len, maxCost, sort],
  );
  const planned = new Set(trip?.activities.map((a) => a.placeId).filter(Boolean));

  const useMyLocation = async () => {
    setLocating(true);
    const loc = await getLocation(dest);
    setLocating(false);
    if (loc.source === 'device') {
      const far = distanceKm(loc, dest) > 60;
      setCenter({ lat: loc.lat, lng: loc.lng, label: 'My location' });
      if (far) {
        setRadiusKm(25);
        toast(`You're ${kmLabel(distanceKm(loc, dest))} from ${dest.city} — results will appear once you're there`, { tone: 'info', ico
n: 'navigate-outline' });
      } else toast('Using your current location');
    } else toast('Location unavailable — using the trip centre', { tone: 'warn' });
  };

  const today = trip ? todayInTz(dest.tz) : '';
  const defaultDay = trip ? (tripDays(trip.startDate, trip.endDate).includes(today) ? today : trip.startDate) : '';

  return (
    <Screen header={<Header title="Nearby" subtitle={`${results.length} places · ${center.label}`} />}>
      <Card style={{ gap: 10 }}>
        <View style={styles.between}>
          <T variant="title">Search radius</T>
          <T variant="title" color={colors.primary}>
            {radiusKm} km
          </T>
        </View>
        <Slider value={radiusKm} min={1} max={25} onChange={setRadiusKm} label="Search radius" formatValue={(v) => `${v} km`} />
        <View style={{ flexDirection: 'row', gap: 8 }}>
```

```
          <Chip small icon="business-outline" label={`${dest.city} centre`} active={center.label !== 'My location'} onPress={() => setCe
nter({ lat: dest.lat, lng: dest.lng, label: `Central ${dest.city}` })} />
          <Chip small icon="locate-outline" label={locating ? 'Locating…' : 'My location'} active={center.label === 'My location'} onPre
ss={useMyLocation} />
        </View>
        <View style={styles.between}>
          <T variant="small" weight="semibold">
            Max price per person
          </T>
          <T variant="small" weight="semibold" color={colors.primary}>
            {maxCost >= 100 ? 'Any' : maxCost === 0 ? 'Free only' : money(maxCost)}
          </T>
        </View>
        <Slider value={maxCost} min={0} max={100} step={5} onChange={setMaxCost} label="Maximum price per person" formatValue={(v) => (v
>= 100 ? 'Any' : `$${v}`)} />
      </Card>

      <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 6, paddingHorizontal: 20 }} style={{ m
arginHorizontal: -20, marginTop: 14 }}>
        {CATS.map((c) => (
          <Chip key={c} small icon={c === 'all' ? undefined : placeCategoryIcon[c]} label={t(`cat.${c === 'all' ? 'all' : c}`)} active=
{cat === c} onPress={() => setCat(c)} />
        ))}
      </ScrollView>
      <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 6, paddingHorizontal: 20 }} style={{ m
arginHorizontal: -20, marginTop: 8 }}>
        {LENGTH_FILTERS.map((f) => (
          <Chip key={f.key} small tone="soft" label={t(f.labelKey)} active={len === f.key} onPress={() => setLen(f.key)} />
        ))}
      </ScrollView>
      <View style={styles.sortRow}>
        <T variant="caption" color={colors.textSecondary}>
          Sort
        </T>
        {(['distance', 'rating', 'price'] as Sort[]).map((s) => (
          <Chip key={s} small label={s === 'distance' ? 'Closest' : s === 'rating' ? 'Top rated' : 'Cheapest'} active={sort === s} onPre
ss={() => setSort(s)} />
        ))}
      </View>

      {results.length ? (
        <View style={{ gap: 10 }}>
          {results.map(({ p, km }) => {
            const focus = params.focus === p.id;
            return (
              <Card key={p.id} style={[{ gap: 8 }, focus && { borderColor: colors.primary, borderWidth: 2 }]}>
                <View style={{ flexDirection: 'row', gap: 12 }}>
                  <View style={styles.icon}>
                    <Ionicons name={placeCategoryIcon[p.category]} size={18} color={colors.primary} />
                  </View>
                  <View style={{ flex: 1 }}>
                    <T variant="bodySm" weight="semibold">
                      {p.name}
                    </T>
                    <T variant="caption" color={colors.textSecondary}>
                      {p.area} · ★ {p.rating}
                    </T>
                  </View>
                  <View style={{ alignItems: 'flex-end' }}>
                    <T variant="small" weight="bold">
                      {kmLabel(km)}
                    </T>
                    <T variant="micro" color={colors.textMuted}>
                      {compassPoint(bearingDeg(center, p))}
                    </T>
                  </View>
                </View>
                <T variant="caption" color={colors.textSecondary}>
                  {p.blurb}
                </T>
                <View style={{ flexDirection: 'row', gap: 6, flexWrap: 'wrap' }}>
                  <Pill label={t(`cat.${p.category}`)} />
                  <Pill label={`${lengthBucketLabel(p.durationMin)} · ${durationLabel(p.durationMin)}`} tone="blue" icon="time-outline"
/>
                  <Pill label={p.cost ? `${priceTier(p.cost)} · ~${money(p.cost)}` : 'Free'} tone="yellow" />
                  {planned.has(p.id) ? <Pill label="On itinerary" tone="purple" icon="checkmark" /> : null}
                </View>
                <View style={{ flexDirection: 'row', gap: 8 }}>
                  {trip ? (
                    <Button
                      label="Add to plan"
                      size="sm"
                      icon="add"
                      style={{ flex: 1 }}
                      onPress={() =>
                        setDraft({ title: p.name, location: `${p.area}, ${destinationById(p.destinationId)?.city ?? ''}`, type: placeToA
```

```
ctivityType[p.category], durationMin: p.durationMin, costPerPerson: p.cost, lat: p.lat, lng: p.lng, placeId: p.id, notes: p.blurb })
                    }
                  />
                ) : null}
                <Press
                  onPress={() => Linking.openURL(`https://www.google.com/maps/search/?api=1&query=${p.lat},${p.lng}`)}
                  style={styles.mapBtn}
                  accessibilityLabel={`Directions to ${p.name}`}>
                  <Ionicons name="navigate-outline" size={16} color={colors.primary} />
                  <T variant="small" weight="semibold" color={colors.primary}>
                    Directions
                  </T>
                </Press>
              </View>
            </Card>
          );
        })}
      </View>
    ) : (
      <EmptyState icon="compass-outline" title="Nothing matches yet" body="Widen the radius, raise the max price or pick another categ
ory." />
    )}

    {trip ? <AddActivitySheet visible={!!draft} onClose={() => setDraft(null)} trip={trip} date={defaultDay} draft={draft} /> : null}
  </Screen>
 );
}

const styles = StyleSheet.create({
 between: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center' },
 sortRow: { flexDirection: 'row', alignItems: 'center', gap: 6, marginVertical: 14 },
 icon: { width: 40, height: 40, borderRadius: 12, backgroundColor: colors.primarySofter, alignItems: 'center', justifyContent: 'center'
},
 mapBtn: { flexDirection: 'row', alignItems: 'center', gap: 6, paddingHorizontal: 14, height: 36, borderRadius: radius.md, borderWidth:
1, borderColor: colors.primaryLine },
});
```

src/app/new-trip.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { router, useLocalSearchParams } from 'expo-router';
import { useMemo, useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';

import { DatePickerSheet } from '@/components/pickers';
import { Button } from '@/components/ui/Button';
import { Card } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { Press, tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { destinationById, destinations } from '@/data/destinations';
import { imageSource } from '@/data/images';
import { useAppStore } from '@/store/useAppStore';
import { colors, radius } from '@/theme';
import { money } from '@/utils/format';
import { addDays, dayLabel, daysBetween, todayInTz } from '@/utils/time';
import { firstError, maxLen, parseAmount, required, validateAmount, validateTripDates } from '@/utils/validation';

/** Create a trip (or edit one with ?edit=<tripId>). */
export default function NewTrip() {
  const params = useLocalSearchParams<{ edit?: string; destination?: string }>();
  const editing = useAppStore((s) => (params.edit ? s.trips.find((t) => t.id === params.edit) : undefined));
  const createTrip = useAppStore((s) => s.createTrip);
  const updateTrip = useAppStore((s) => s.updateTrip);
  const today = todayInTz('UTC');

  const initialDest = editing?.destinationId ?? params.destination ?? 'tokyo';
  const [destId, setDestId] = useState(initialDest);
  const [name, setName] = useState(editing?.name ?? '');
  const [start, setStart] = useState(editing?.startDate ?? addDays(today, 30));
  const [end, setEnd] = useState(editing?.endDate ?? addDays(today, 36));
  const [budget, setBudget] = useState(editing ? String(editing.budget) : '');
  const [q, setQ] = useState('');
  const [picker, setPicker] = useState<'start' | 'end' | null>(null);
  const [errors, setErrors] = useState<Record<string, string | null | undefined>>({});

  const dest = destinationById(destId)!;
  const days = Math.max(1, daysBetween(start, end) + 1);
  const suggested = Math.round(dest.dailyCost * days * 4 + 900 * 4);
  const filtered = useMemo(() => destinations.filter((d) => !q || `${d.city} ${d.country}`.toLowerCase().includes(q.toLowerCase())),
[q]);

  const save = () => {
    const dateErr = validateTripDates(start, end);
    const e = {
      name: firstError(name, required('a trip name'), maxLen(40, 'Trip name')),
      budget: validateAmount(budget, { min: 50, max: 1_000_000, label: 'a group budget' }),
      start: dateErr.start ?? (!editing && daysBetween(today, start) < -1 ? 'Start date can’t be in the past' : null),
      end: dateErr.end,
    };
    setErrors(e);
    if (Object.values(e).some(Boolean)) return tap('warning');
    const b = parseAmount(budget)!;
    if (editing) {
      updateTrip(editing.id, { name: name.trim(), startDate: start, endDate: end, budget: b, destinationId: destId, cover: destId !== ed
iting.destinationId ? dest.image : editing.cover });
      toast('Trip updated');
      router.back();
    } else {
      createTrip({ name, destinationId: destId, startDate: start, endDate: end, budget: b });
      tap('success');
      toast('Trip created 🎉 — share the invite code with your group');
      router.dismissAll?.();
      router.replace('/crew');
    }
  };

  return (
    <Screen header={<Header title={editing ? 'Edit trip' : 'New trip'} subtitle={editing ? editing.name : 'Where is the group headed?'}
/>}>
      <View style={{ gap: 16 }}>
        <View style={{ gap: 8 }}>
          <T variant="small" weight="semibold">
            Destination
          </T>
          <Input placeholder="Search destinations…" icon="search" value={q} onChangeText={setQ} />
          <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 10, paddingHorizontal: 20 }} style
={{ marginHorizontal: -20 }}>
            {filtered.map((d) => {
              const on = d.id === destId;
```

```
              return (
                <Press key={d.id} onPress={() => setDestId(d.id)} style={[styles.dest, on && styles.destOn]} accessibilityState={{ selec
ted: on }} accessibilityLabel={`${d.city}, ${d.country}`}>
                  <Image source={imageSource(d.image)} style={styles.destImg} contentFit="cover" />
                  {on ? (
                    <View style={styles.check}>
                      <Ionicons name="checkmark" size={14} color={colors.white} />
                    </View>
                  ) : null}
                  <T variant="caption" weight="semibold" numberOfLines={1} style={{ marginTop: 6 }}>
                    {d.city}
                  </T>
                  <T variant="micro" color={colors.textMuted} numberOfLines={1}>
                    {d.country}
                  </T>
                </Press>
              );
            })}
          </ScrollView>
        </View>

        <Input label="Trip name" placeholder={`e.g. ${dest.city} with the crew`} value={name} onChangeText={setName} error={errors.name}
maxLength={45} icon="create-outline" />

        <View style={{ flexDirection: 'row', gap: 10 }}>
          <Input label="Start date" icon="calendar-outline" value={dayLabel(start) + `, ${start.slice(0, 4)}`} onPressField={() => setPi
cker('start')} error={errors.start} containerStyle={{ flex: 1 }} />
          <Input label="End date" icon="calendar-outline" value={dayLabel(end) + `, ${end.slice(0, 4)}`} onPressField={() => setPicker
('end')} error={errors.end} containerStyle={{ flex: 1 }} />
        </View>
        <T variant="caption" color={colors.textSecondary}>
          {days} day{days === 1 ? '' : 's'} · local time in {dest.city} ({dest.tz.split('/').pop()?.replace('_', ' ')})
        </T>

        <Input label="Group budget (USD)" placeholder={String(suggested)} value={budget} onChangeText={setBudget} keyboardType="decimal-
pad" error={errors.budget} icon="wallet-outline" hint={`Tip: ~${money(dest.dailyCost)}/person/day in ${dest.city}. For 4 people incl. fl
ights ≈ ${money(suggested)}.`} />
        {!budget ? <Button label={`Use suggested ${money(suggested)}`} variant="soft" size="sm" onPress={() => setBudget(String(suggeste
d))} /> : null}

        {!editing ? (
          <Card style={{ flexDirection: 'row', gap: 10, alignItems: 'center' }}>
            <Ionicons name="key-outline" size={18} color={colors.primary} />
            <T variant="caption" color={colors.textSecondary} style={{ flex: 1 }}>
              We'll generate an invite code like <T variant="caption" weight="bold">{dest.city.slice(0, 3).toUpperCase()}-1234</T> so fr
iends can join instantly.
            </T>
          </Card>
        ) : null}

        <Button label={editing ? 'Save changes' : 'Create trip'} icon={editing ? 'checkmark' : 'airplane'} onPress={save} />
      </View>

      <DatePickerSheet visible={picker === 'start'} value={start} title="Start date" onClose={() => setPicker(null)} onChange={(v) => {
        setStart(v);
        if (daysBetween(v, end) < 0) setEnd(addDays(v, 5));
      }} />
      <DatePickerSheet visible={picker === 'end'} value={end} min={start} max={addDays(start, 60)} title="End date" onClose={() => setPi
cker(null)} onChange={setEnd} />
    </Screen>
  );
}

const styles = StyleSheet.create({
  dest: { width: 104, padding: 6, borderRadius: radius.md, borderWidth: 2, borderColor: 'transparent' },
  destOn: { borderColor: colors.primary, backgroundColor: colors.primarySofter },
  destImg: { width: '100%', height: 76, borderRadius: radius.sm },
  check: { position: 'absolute', top: 10, right: 10, width: 22, height: 22, borderRadius: 11, backgroundColor: colors.primary, alignItem
s: 'center', justifyContent: 'center' },
});
```

src/app/notifications.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router, type Href } from 'expo-router';
import { useEffect } from 'react';
import { StyleSheet, View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { EmptyState, type IconName } from '@/components/ui/bits';
import { Press } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { sendTestReminder } from '@/services/notifications';
import { useAppStore } from '@/store/useAppStore';
import type { AppNotification } from '@/store/types';
import { colors, radius } from '@/theme';
import { timeAgo } from '@/utils/format';
import { toast } from '@/components/ui/feedback';

const ICONS: Record<AppNotification['kind'], { icon: IconName; color: string }> = {
  reminder: { icon: 'alarm-outline', color: colors.orange },
  chat: { icon: 'chatbubble-ellipses-outline', color: colors.blue },
  vote: { icon: 'stats-chart-outline', color: colors.purple },
  expense: { icon: 'wallet-outline', color: colors.primary },
  system: { icon: 'information-circle-outline', color: colors.textSecondary },
  member: { icon: 'person-add-outline', color: colors.teal },
};

export default function Notifications() {
  const list = useAppStore((s) => s.notifications);
  const markAllRead = useAppStore((s) => s.markAllRead);
  const clear = useAppStore((s) => s.clearNotifications);
  const setActive = useAppStore((s) => s.setActiveTrip);

  // Mark everything read when leaving the screen.
  useEffect(() => () => markAllRead(), [markAllRead]);

  return (
    <Screen
      header={
        <Header
          title="Notifications"
          subtitle={`${list.filter((n) => !n.read).length} unread`}
          right={
            list.length ? (
              <Press onPress={clear} hitSlop={8} accessibilityLabel="Clear all notifications">
                <T variant="small" weight="semibold" color={colors.textSecondary}>
                  Clear
                </T>
              </Press>
            ) : null
          }
        />
      }>
      <Button
        label="Send a test reminder (5 sec)"
        icon="alarm-outline"
        variant="soft"
        onPress={() => {
          sendTestReminder(5);
          toast('Test reminder scheduled — lock your phone to see it arrive', { icon: 'alarm-outline' });
        }}
      />
      <View style={{ gap: 8, marginTop: 14 }}>
        {list.length ? (
          list.map((n) => {
            const meta = ICONS[n.kind];
            return (
              <Press
                key={n.id}
                onPress={() => {
                  if (n.tripId) setActive(n.tripId);
                  if (n.route) router.push((n.route === '/(tabs)/profile' ? '/crew' : n.route) as Href);
                }}
                style={[styles.item, !n.read && styles.unread]}
                accessibilityLabel={`${n.title}. ${n.body}`}>
                <View style={[styles.icon, { backgroundColor: `${meta.color}1A` }]}>
                  <Ionicons name={meta.icon} size={18} color={meta.color} />
                </View>
                <View style={{ flex: 1 }}>
                  <T variant="small" weight="semibold">
                    {n.title}
                  </T>
                  <T variant="caption" color={colors.textSecondary}>
                    {n.body}
                  </T>
```

```
                <T variant="micro" color={colors.textMuted} style={{ marginTop: 3 }}>
                  {timeAgo(n.ts)}
                </T>
              </View>
              {!n.read ? <View style={styles.dot} /> : null}
            </Press>
          );
        })
      ) : (
        <EmptyState icon="notifications-off-outline" title="You’re all caught up" body="Reminders, votes and chat mentions will show u
p here." />
      )}
    </View>
  </Screen>
 );
}

const styles = StyleSheet.create({
  item: { flexDirection: 'row', gap: 12, padding: 14, borderRadius: radius.md, backgroundColor: colors.card, borderWidth: 1, borderColo
r: colors.border },
  unread: { backgroundColor: colors.primarySofter, borderColor: colors.primaryLine },
  icon: { width: 38, height: 38, borderRadius: 19, alignItems: 'center', justifyContent: 'center' },
  dot: { width: 9, height: 9, borderRadius: 5, backgroundColor: colors.primary, marginTop: 4 },
});
```

```
import { Image } from 'expo-image';
import { LinearGradient } from 'expo-linear-gradient';
import { router } from 'expo-router';
import { useEffect, useRef, useState } from 'react';
import { Animated, FlatList, StyleSheet, View, useWindowDimensions, type NativeScrollEvent, type NativeSyntheticEvent } from 'react-nati
ve';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { LogoMark } from '@/components/Logo';
import { Button } from '@/components/ui/Button';
import { FadeIn, Float, nativeDriver, useMotion } from '@/components/ui/motion';
import { Press } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { photos } from '@/data/images';
import { useT } from '@/i18n';
import { useAppStore } from '@/store/useAppStore';
import { MAX_WIDTH, colors } from '@/theme';

const slides = [
  { key: '1', image: photos['onboard-1'], title: 'Your Journey\nBegins Here', body: 'Welcome to a seamless way to explore the world with
the people you love.' },
  { key: '2', image: photos['onboard-2'], title: 'Plan Together,\nTravel Better', body: 'Coordinate itineraries, budgets, and memories —
all in one place.' },
  { key: '3', image: photos['onboard-3'], title: 'Every Trip,\nUnforgettable', body: 'From bucket lists to group chats — TogetherWeGo ha
s you covered.' },
];

export default function Onboarding() {
  const t = useT();
  const { width: winW, height } = useWindowDimensions();
  const width = Math.min(winW, MAX_WIDTH);
  const insets = useSafeAreaInsets();
  const [index, setIndex] = useState(0);
  const list = useRef<FlatList>(null);
  const complete = useAppStore((s) => s.completeOnboarding);
  const motion = useMotion();
  const [zoom] = useState(() => new Animated.Value(0));

  // Slow "Ken Burns" zoom on the current photo, restarted on every slide.
  useEffect(() => {
    zoom.setValue(0);
    if (!motion) return;
    const anim = Animated.timing(zoom, { toValue: 1, duration: 9000, useNativeDriver: nativeDriver });
    anim.start();
    return () => anim.stop();
  }, [index, motion, zoom]);
  const scale = zoom.interpolate({ inputRange: [0, 1], outputRange: [1, 1.14] });

  const finish = () => {
    complete();
    router.replace('/sign-in');
  };
  const next = () => {
    if (index >= slides.length - 1) return finish();
    list.current?.scrollToIndex({ index: index + 1, animated: true });
    setIndex(index + 1);
  };
  const onScroll = (e: NativeSyntheticEvent<NativeScrollEvent>) => {
    const i = Math.round(e.nativeEvent.contentOffset.x / width);
    if (i !== index) setIndex(i);
  };

  return (
    <View style={styles.root}>
      <View style={{ width, height, alignSelf: 'center' }}>
        <FlatList
          ref={list}
          data={slides}
          horizontal
          pagingEnabled
          bounces={false}
          showsHorizontalScrollIndicator={false}
          onMomentumScrollEnd={onScroll}
          onScroll={onScroll}
          scrollEventThrottle={32}
          getItemLayout={(_, i) => ({ length: width, offset: width * i, index: i })}
          keyExtractor={(s) => s.key}
          renderItem={({ item, index: i }) => (
            <View style={{ width, height, overflow: 'hidden' }}>
              <Animated.View style={[StyleSheet.absoluteFill, i === index ? { transform: [{ scale }] } : null]}>
                <Image source={item.image} style={StyleSheet.absoluteFill} contentFit="cover" transition={300} accessibilityIgnoresInver
tColors />
              </Animated.View>
              <LinearGradient colors={['rgba(0,0,0,0.05)', 'rgba(0,0,0,0)', 'rgba(8,16,6,0.78)']} locations={[0, 0.45, 1]} style={StyleS
```

```
heet.absoluteFill} />
            </View>
          )}
        />
        <View style={[styles.top, { top: insets.top + 8 }]}>
          <FadeIn from="down">
            <View style={styles.brand}>
              <Float amount={5} duration={2000}>
                <LogoMark size={26} />
              </Float>
              <T variant="small" weight="bold" color={colors.white}>
                TogetherWeGo
              </T>
            </View>
          </FadeIn>
          {index < slides.length - 1 ? (
            <Press onPress={finish} accessibilityLabel={t('common.skip')} hitSlop={10} style={styles.skip}>
              <T variant="small" weight="semibold" color={colors.white}>
                {t('common.skip')}
              </T>
            </Press>
          ) : null}
        </View>
        <View style={[styles.bottom, { paddingBottom: insets.bottom + 24 }]}>
          <FadeIn key={`k${index}`} delay={80} distance={26}>
            <View style={styles.kicker}>
              <T variant="micro" weight="bold" color="#1A1C17">
                {['✈️  PLAN', '🤝  TOGETHER', '📸  REMEMBER'][index]}
              </T>
            </View>
            <T variant="display" color={colors.white} style={{ fontSize: 34, lineHeight: 40 }} accessibilityRole="header">
              {slides[index].title}
            </T>
          </FadeIn>
          <FadeIn key={`b${index}`} delay={220} distance={20}>
            <T variant="body" color="rgba(255,255,255,0.85)" style={{ marginTop: 10, maxWidth: 320 }}>
              {slides[index].body}
            </T>
          </FadeIn>
          <View style={styles.dots} accessibilityLabel={`Page ${index + 1} of ${slides.length}`}>
            {slides.map((s, i) => (
              <View key={s.key} style={[styles.dot, i === index && styles.dotActive]} />
            ))}
          </View>
          <Button label={index === slides.length - 1 ? t('common.getStarted') : t('common.next')} onPress={next} iconRight="arrow-forwar
d" style={{ borderRadius: 999 }} />
        </View>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: '#0F1A0D' },
  top: { position: 'absolute', left: 20, right: 20, flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center' },
  brand: { flexDirection: 'row', alignItems: 'center', gap: 8, backgroundColor: 'rgba(255,255,255,0.18)', paddingLeft: 5, paddingRight:
12, paddingVertical: 5, borderRadius: 999 },
  kicker: { alignSelf: 'flex-start', backgroundColor: 'rgba(255,255,255,0.92)', paddingHorizontal: 10, paddingVertical: 5, borderRadius:
999, marginBottom: 12 },
  skip: { backgroundColor: 'rgba(0,0,0,0.25)', paddingHorizontal: 14, paddingVertical: 7, borderRadius: 999 },
  bottom: { position: 'absolute', left: 24, right: 24, bottom: 0 },
  dots: { flexDirection: 'row', gap: 6, marginTop: 22, marginBottom: 22 },
  dot: { width: 7, height: 7, borderRadius: 4, backgroundColor: 'rgba(255,255,255,0.45)' },
  dotActive: { width: 24, backgroundColor: colors.accent },
});
```

src/app/packing.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { ScrollView, StyleSheet, TextInput, View } from 'react-native';

import { MemberPickerSheet } from '@/components/trip/MemberPickerSheet';
import { NoTrip } from '@/components/trip/NoTrip';
import { Button } from '@/components/ui/Button';
import { Avatar, Card, Chip, ProgressBar } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Press, tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { Segmented } from '@/components/ui/Segmented';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { packingBase } from '@/data/reference';
import { useWeather } from '@/hooks/useTrip';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import { colors, fonts, radius } from '@/theme';

type Filter = 'all' | 'mine' | 'open';

const COUNTRY_EXTRAS: Record<string, string[]> = {
  JP: ['IC transit card (Suica / ICOCA)', 'Type A plug adapter', 'Coin purse — lots of cash & coins', 'Small towel (many restrooms lack
dryers)', 'Slip-on shoes for temples & ryokan'],
  FR: ['Type E plug adapter', 'Reusable shopping bag'],
  IT: ['Type L/F plug adapter', 'Scarf to cover shoulders in churches'],
  TH: ['Mosquito repellent', 'Light clothes covering knees for temples'],
  ID: ['Reef-safe sunscreen', 'Mosquito repellent'],
};

export default function Packing() {
  const trip = useActiveTrip();
  const me = useAppStore((s) => s.session?.userId);
  const toggle = useAppStore((s) => s.togglePacked);
  const assign = useAppStore((s) => s.assignPacking);
  const addItems = useAppStore((s) => s.addPackingItems);
  const del = useAppStore((s) => s.deletePackingItem);
  const weather = useWeather(trip?.destinationId);
  const [text, setText] = useState('');
  const [cat, setCat] = useState('Extras');
  const [err, setErr] = useState<string | null>(null);
  const [filter, setFilter] = useState<Filter>('all');
  const [pickId, setPickId] = useState<string | null>(null);
  if (!trip) return <NoTrip />;

  const dest = destinationById(trip.destinationId);
  const total = trip.packing.length;
  const packed = trip.packing.filter((p) => p.packed).length;
  const bringer = (broughtBy?: string) => trip.members.find((m) => m.id === broughtBy);
  const needsSomeone = trip.packing.filter((p) => !p.packed && !bringer(p.broughtBy));
  const shown = trip.packing.filter((p) => (filter === 'mine' ? p.broughtBy === me : filter === 'open' ? !p.packed && !bringer(p.brought
By) : true));
  const groups = Array.from(new Set(shown.map((p) => p.category)));
  const have = new Set(trip.packing.map((p) => p.text.toLowerCase()));
  const picking = trip.packing.find((p) => p.id === pickId);
  const suggestions = [
    ...(COUNTRY_EXTRAS[dest?.countryCode ?? ''] ?? []),
    ...(weather?.daily.some((d) => d.rain >= 50) ? ['Compact umbrella'] : []),
    ...(weather && Math.max(...weather.daily.map((d) => d.max)) >= 27 ? ['Sunglasses & hat'] : []),
    ...(weather && Math.min(...weather.daily.map((d) => d.min)) <= 10 ? ['Warm jacket'] : []),
  ].filter((s) => !have.has(s.toLowerCase()));

  const add = () => {
    const t = text.trim();
    if (!t) return setErr('Type an item to add');
    if (t.length > 50) return setErr('Keep items under 50 characters');
    if (have.has(t.toLowerCase())) return setErr('That’s already on the list');
    addItems(trip.id, [{ text: t, category: cat }]);
    setText('');
    setErr(null);
    tap('success');
  };

  const splitFairly = () => {
    if (!needsSomeone.length || !trip.members.length) return;
    const load: Record<string, number> = {};
    trip.members.forEach((m) => (load[m.id] = trip.packing.filter((p) => p.broughtBy === m.id).length));
    needsSomeone.forEach((p) => {
      const next = trip.members.reduce((a, b) => (load[b.id] < load[a.id] ? b : a));
      assign(trip.id, p.id, next.id);
      load[next.id] += 1;
    });
    tap('success');
    toast(`Assigned ${needsSomeone.length} item${needsSomeone.length === 1 ? '' : 's'}`);
```

```
  };

  return (
    <Screen header={<Header title="Packing list" subtitle={`Shared with ${trip.members.length} travellers`} />}>
      <Card style={[styles.card, styles.stack]}>
        <View style={styles.between}>
          <T variant="headline">Shared checklist</T>
          <View style={styles.pct}>
            <T variant="small" weight="bold" color={colors.onAccent}>
              {total ? Math.round((packed / total) * 100) : 0}%
            </T>
          </View>
        </View>
        <T variant="bodySm" weight="semibold">
          {packed} of {total} packed · {needsSomeone.length} still need someone
        </T>
        <ProgressBar value={total ? packed / total : 0} height={10} />
        {total ? (
          <View style={styles.wrap}>
            {trip.members.map((m) => (
              <View key={m.id} style={styles.load}>
                <Avatar name={m.name} src={m.avatar} size={22} />
                <T variant="caption" weight="semibold">
                  {m.name.split(' ')[0]} · {trip.packing.filter((p) => p.broughtBy === m.id).length}
                </T>
              </View>
            ))}
          </View>
        ) : null}
        <Button label="Split the rest fairly" icon="shuffle-outline" size="md" disabled={!needsSomeone.length} onPress={splitFairly} />
      </Card>

      <View style={{ marginTop: 14 }}>
        <Segmented
          value={filter}
          onChange={setFilter}
          options={[
            { key: 'all', label: 'All' },
            { key: 'mine', label: 'Mine' },
            { key: 'open', label: 'Unassigned' },
          ]}
        />
      </View>

      {suggestions.length ? (
        <Card style={[styles.card, styles.stack, { marginTop: 14, backgroundColor: colors.accentSoft, borderColor: colors.accent }]}>
          <View style={styles.between}>
            <T variant="small" weight="bold" color={colors.onAccent}>
              Smart suggestions for {dest?.city}
            </T>
            <Press
              onPress={() => {
                addItems(trip.id, suggestions.map((s) => ({ text: s, category: 'Suggested' })));
                toast(`Added ${suggestions.length} items`);
              }}
              hitSlop={8}
              style={styles.addAll}>
              <T variant="caption" weight="bold" color={colors.onPrimary}>
                Add all
              </T>
            </Press>
          </View>
          <View style={styles.wrap}>
            {suggestions.map((s) => (
              <Chip key={s} small icon="add" label={s} onPress={() => addItems(trip.id, [{ text: s, category: 'Suggested' }])} />
            ))}
          </View>
        </Card>
      ) : null}

      <Card style={[styles.card, { marginTop: 14, gap: 8 }]}>
        <View style={{ flexDirection: 'row', gap: 8 }}>
          <TextInput value={text} onChangeText={(v) => { setText(v); setErr(null); }} placeholder="Add an item…" placeholderTextColor={c
olors.textMuted} style={[styles.input, err && { borderColor: colors.red }]} onSubmitEditing={add} returnKeyType="done" accessibilityLabe
l="New packing item" />
          <Press onPress={add} style={styles.addBtn} accessibilityLabel="Add item">
            <Ionicons name="add" size={22} color={colors.onPrimary} />
          </Press>
        </View>
        {err ? (
          <T variant="caption" color={colors.red}>
            {err}
          </T>
        ) : null}
        <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 6 }}>
          {[...packingBase.map((g) => g.category), 'Weather', 'Suggested'].map((c) => (
```

```
            <Chip key={c} small label={c} active={cat === c} onPress={() => setCat(c)} />
          ))}
        </ScrollView>
      </Card>

      {groups.map((g) => (
        <View key={g} style={{ marginTop: 20 }}>
          <T variant="headline" style={{ marginBottom: 10 }} accessibilityRole="header">
            {g}
          </T>
          <Card padded={false} style={[styles.card, { paddingHorizontal: 14 }]}>
            {shown
              .filter((p) => p.category === g)
              .map((p, i, arr) => {
                const who = bringer(p.broughtBy);
                return (
                  <View key={p.id} style={[styles.item, i === arr.length - 1 && { borderBottomWidth: 0 }]}>
                    <Press onPress={() => toggle(trip.id, p.id)} style={styles.itemMain} accessibilityRole="checkbox" accessibilityState
={{ checked: p.packed }} accessibilityLabel={p.text}>
                      <Ionicons name={p.packed ? 'checkbox' : 'square-outline'} size={22} color={p.packed ? colors.primary : colors.bord
erStrong} />
                      <T variant="small" color={p.packed ? colors.textMuted : colors.text} style={[{ flex: 1 }, p.packed && { textDecora
tionLine: 'line-through' }]}>
                        {p.text}
                      </T>
                    </Press>
                    <Press onPress={() => setPickId(p.id)} hitSlop={6} style={[styles.who, !who && styles.whoEmpty]} accessibilityLabel=
{who ? `${p.text} is brought by ${who.name}. Change` : `Assign ${p.text}`}>
                      {who ? (
                        <>
                          <Avatar name={who.name} src={who.avatar} size={22} />
                          <T variant="caption" weight="semibold" numberOfLines={1} style={{ maxWidth: 56 }}>
                            {who.id === me ? 'Me' : who.name.split(' ')[0]}
                          </T>
                        </>
                      ) : (
                        <T variant="caption" weight="semibold" color={colors.textSecondary}>
                          Anyone
                        </T>
                      )}
                    </Press>
                    <Press onPress={() => del(trip.id, p.id)} hitSlop={8} accessibilityLabel={`Remove ${p.text}`}>
                      <Ionicons name="close" size={16} color={colors.textMuted} />
                    </Press>
                  </View>
                );
              })}
          </Card>
        </View>
      ))}
      {total && !shown.length ? (
        <Card style={[styles.card, { marginTop: 20, alignItems: 'center' }]}>
          <T variant="small" color={colors.textSecondary} center>
            {filter === 'mine' ? 'Nothing is assigned to you yet. Tap “Anyone” on an item to pick it up.' : 'Every item has someone brin
ging it. Nice teamwork!'}
          </T>
        </Card>
      ) : null}
      {!total ? (
        <Button
          label="Start with the essentials"
          icon="sparkles-outline"
          variant="secondary"
          style={{ marginTop: 16 }}
          onPress={() => addItems(trip.id, packingBase.flatMap((g) => g.items.map((text) => ({ text, category: g.category }))))}
        />
      ) : null}

      <MemberPickerSheet
        visible={!!picking}
        onClose={() => setPickId(null)}
        members={trip.members}
        selected={picking?.broughtBy ? [picking.broughtBy] : []}
        onToggle={(id) => picking && assign(trip.id, picking.id, picking.broughtBy === id ? undefined : id)}
        title={picking ? `Who’s bringing ${picking.text}?` : 'Who’s bringing it?'}
        single
      />
    </Screen>
  );
}

const styles = StyleSheet.create({
  card: { borderRadius: radius.xl },
  stack: { gap: 12 },
  between: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', gap: 8 },
  wrap: { flexDirection: 'row', flexWrap: 'wrap', gap: 6 },
```

```
  pct: { backgroundColor: colors.accent, borderRadius: radius.pill, paddingHorizontal: 12, paddingVertical: 4 },
  load: { flexDirection: 'row', alignItems: 'center', gap: 6, paddingRight: 10, paddingLeft: 3, height: 28, borderRadius: radius.pill, b
ackgroundColor: colors.bgAlt },
  addAll: { backgroundColor: colors.primary, borderRadius: radius.pill, paddingHorizontal: 12, height: 26, alignItems: 'center', justify
Content: 'center' },
  input: { flex: 1, height: 46, borderRadius: radius.pill, borderWidth: 1, borderColor: colors.border, backgroundColor: colors.cardMute
d, paddingHorizontal: 18, fontFamily: fonts.regular, fontSize: 15, color: colors.text },
  addBtn: { width: 46, height: 46, borderRadius: 23, backgroundColor: colors.primary, alignItems: 'center', justifyContent: 'center' },
  item: { flexDirection: 'row', alignItems: 'center', gap: 10, borderBottomWidth: 1, borderBottomColor: colors.border },
  itemMain: { flex: 1, flexDirection: 'row', alignItems: 'center', gap: 10, paddingVertical: 12 },
  who: { flexDirection: 'row', alignItems: 'center', gap: 5, height: 30, paddingLeft: 4, paddingRight: 10, borderRadius: radius.pill, ba
ckgroundColor: colors.bgAlt },
  whoEmpty: { paddingLeft: 10, backgroundColor: 'transparent', borderWidth: 1, borderStyle: 'dashed', borderColor: colors.borderStrong
},
});
```

src/app/panorama.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { LinearGradient } from 'expo-linear-gradient';
import { router, useLocalSearchParams } from 'expo-router';
import { useEffect, useRef, useState } from 'react';
import { Animated, Easing, ScrollView, StyleSheet, View, useWindowDimensions } from 'react-native';
import { StatusBar } from 'expo-status-bar';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { headingOf, K, layout, PanoramaScene, posForHeading } from '@/components/panorama/Scene';
import { timeOfDayAt } from '@/components/SkylineHero';
import { Press, tap } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { useNow } from '@/hooks/useTrip';
import { PANORAMAS, panoById } from '@/data/panoramas';
import { MAX_WIDTH, colors } from '@/theme';

const WIND = ['N', 'NNE', 'NE', 'ENE', 'E', 'ESE', 'SE', 'SSE', 'S', 'SSW', 'SW', 'WSW', 'W', 'WNW', 'NW', 'NNW'];
const wind = (deg: number) => WIND[Math.round(deg / 22.5) % 16];
/** Smallest signed angle from a to b, in degrees. */
const angleDiff = (a: number, b: number) => ((((b - a) % 360) + 540) % 360) - 180;

export default function PanoramaScreen() {
  const { city, focus } = useLocalSearchParams<{ city?: string; focus?: string }>();
  const { width, height } = useWindowDimensions();
  const insets = useSafeAreaInsets();
  const S = Math.min(width, MAX_WIDTH);
  const H = height;
  const now = useNow(60000);

  const [cityId, setCityId] = useState(panoById(city)?.id ?? PANORAMAS[0].id);
  const pano = panoById(cityId) ?? PANORAMAS[0];
  const startLm = pano.landmarks.find((l) => l.id === focus) ?? pano.landmarks[0];
  const [forced, setForced] = useState<'day' | 'night' | null>(null);
  const [auto, setAuto] = useState(true);
  const [heading, setHeading] = useState(startLm.heading);
  const mode = forced ?? timeOfDayAt(pano.tz, now);

  const [pos] = useState(() => new Animated.Value(posForHeading(startLm.heading, S)));
  const [night] = useState(() => new Animated.Value(mode === 'night' ? 1 : 0));
  const posRef = useRef(posForHeading(startLm.heading, S));
  const velRef = useRef(0);
  const dragRef = useRef(false);
  const busyRef = useRef(false);
  const idleRef = useRef(1000);
  const autoRef = useRef(true);
  const gestureRef = useRef({ x: 0, pos: 0, last: 0 });

  // Keep the mirrors of state that the animation loop and gesture handlers read.
  useEffect(() => {
    autoRef.current = auto;
  }, [auto]);
  useEffect(() => {
    const id = pos.addListener(({ value }) => {
      posRef.current = value;
    });
    return () => pos.removeListener(id);
  }, [pos]);

  // Day ↔ night cross-fade.
  useEffect(() => {
    const anim = Animated.timing(night, { toValue: mode === 'night' ? 1 : 0, duration: 1200, easing: Easing.inOut(Easing.cubic), useNati
veDriver: false });
    anim.start();
    return () => anim.stop();
  }, [mode, night]);

  // One ~30 fps loop: coasting after a flick, gentle auto-rotation when idle, and the compass readout.
  useEffect(() => {
    let tick = 0;
    const id = setInterval(() => {
      tick++;
      if (!dragRef.current && !busyRef.current) {
        velRef.current *= 0.94;
        idleRef.current++;
        const idle = idleRef.current > 105;
        const step = velRef.current + (autoRef.current && idle && Math.abs(velRef.current) < 0.2 ? 0.45 : 0);
        if (Math.abs(step) > 0.01) pos.setValue(posRef.current + step);
      }
      if (tick % 4 === 0) setHeading(Math.round(headingOf(posRef.current, S)) % 360);
    }, 33);
    return () => clearInterval(id);
  }, [pos, S]);

  const glide = (delta: number) => {
```

```
    busyRef.current = true;
    idleRef.current = 0;
    velRef.current = 0;
    Animated.timing(pos, { toValue: posRef.current + delta, duration: 800, easing: Easing.out(Easing.cubic), useNativeDriver: false }).s
tart(() => {
      busyRef.current = false;
    });
  };
  const nudge = (dir: 1 | -1) => {
    tap();
    glide((dir * S * 0.45) / K.mid);
  };
  const lookAt = (deg: number) => {
    tap();
    const period = layout(S).far;
    const mid = layout(S).mid;
    const want = (angleDiff(headingOf(posRef.current, S), deg) / 360) * mid;
    glide((want / K.mid) % period);
  };
  const pick = (id: string) => {
    tap();
    const p = panoById(id);
    if (!p) return;
    setCityId(p.id);
    setForced(null);
    const h = p.landmarks[0].heading;
    setHeading(h);
    const target = posForHeading(h, S);
    posRef.current = target;
    velRef.current = 0;
    pos.setValue(target);
  };

  const near = pano.landmarks.find((l) => Math.abs(angleDiff(heading, l.heading)) < 13);
  const top = insets.top + 10;

  return (
    <View style={styles.root}>
      <StatusBar style="light" />
      <View
        style={{ width: S, height: H, overflow: 'hidden', backgroundColor: '#05070F' }}
        onStartShouldSetResponder={() => true}
        onMoveShouldSetResponder={() => true}
        onResponderGrant={(e) => {
          dragRef.current = true;
          busyRef.current = false;
          idleRef.current = 0;
          velRef.current = 0;
          gestureRef.current = { x: e.nativeEvent.pageX, pos: posRef.current, last: e.nativeEvent.pageX };
        }}
        onResponderMove={(e) => {
          const g = gestureRef.current;
          const x = e.nativeEvent.pageX;
          idleRef.current = 0;
          velRef.current = -(x - g.last) / K.mid;
          g.last = x;
          pos.setValue(g.pos - (x - g.x) / K.mid);
        }}
        onResponderRelease={() => {
          dragRef.current = false;
        }}
        onResponderTerminate={() => {
          dragRef.current = false;
        }}>
        <PanoramaScene key={pano.id} pano={pano} night={night} pos={pos} S={S} H={H} />
        <LinearGradient pointerEvents="none" colors={['rgba(0,0,0,0.55)', 'rgba(0,0,0,0)']} style={[styles.fade, { top: 0, height: 200
}]} />
        <LinearGradient pointerEvents="none" colors={['rgba(0,0,0,0)', 'rgba(0,0,0,0.7)']} style={[styles.fade, { bottom: 0, height: 330
}]} />

        {/* Top bar */}
        <View style={[styles.top, { top }]}>
          <Press onPress={() => (router.canGoBack() ? router.back() : router.replace('/(tabs)/explore'))} style={styles.round} accessibi
lityLabel="Go back">
            <Ionicons name="chevron-back" size={20} color={colors.white} />
          </Press>
          <View style={{ flex: 1 }} />
          <Press onPress={() => setAuto((a) => !a)} style={styles.chip} accessibilityRole="switch" accessibilityState={{ checked: auto
}} accessibilityLabel="Auto rotate">
            <Ionicons name={auto ? 'sync' : 'pause'} size={14} color={colors.white} />
            <T variant="micro" weight="bold" color={colors.white}>
              {auto ? 'Auto' : 'Paused'}
            </T>
          </Press>
          <Press
            onPress={() => {
```

```
            tap();
            setForced(mode === 'night' ? 'day' : 'night');
          }}
          onLongPress={() => setForced(null)}
          style={[styles.chip, { backgroundColor: mode === 'night' ? '#1B2250' : 'rgba(255,255,255,0.28)' }]}
          accessibilityLabel={mode === 'night' ? 'Night view. Switch to day' : 'Day view. Switch to night'}
          accessibilityHint="Long press to follow local time">
          <Ionicons name={mode === 'night' ? 'moon' : 'sunny'} size={15} color={mode === 'night' ? '#FFE9A8' : '#FFFFFF'} />
          <T variant="micro" weight="bold" color={colors.white}>
            {mode === 'night' ? 'Night · lights on' : 'Day'}
            {forced ? '' : ' · live'}
          </T>
        </Press>
      </View>

      {/* Title + compass */}
      <View style={[styles.titleBlock, { top: top + 54 }]} pointerEvents="none">
        <T variant="kicker" color="rgba(255,255,255,0.85)">
          {pano.emoji} {pano.country}
        </T>
        <T variant="display" color={colors.white} style={styles.titleText} accessibilityRole="header">
          {pano.name}
        </T>
        <View style={styles.compass}>
          <Ionicons name="compass-outline" size={13} color={colors.white} />
          <T variant="micro" weight="bold" color={colors.white}>
            360° · {wind(heading)} {heading}°
          </T>
        </View>
      </View>

      {/* Arrows */}
      <Press onPress={() => nudge(-1)} style={[styles.arrow, { left: 10 }]} accessibilityLabel="Look left">
        <Ionicons name="chevron-back" size={22} color={colors.white} />
      </Press>
      <Press onPress={() => nudge(1)} style={[styles.arrow, { right: 10 }]} accessibilityLabel="Look right">
        <Ionicons name="chevron-forward" size={22} color={colors.white} />
      </Press>

      {/* Bottom: callout, landmark jumps, city picker */}
      <View style={[styles.bottom, { paddingBottom: insets.bottom + 12 }]}>
        <View style={styles.callout} accessibilityLiveRegion="polite">
          {near ? (
            <>
              <T variant="title" weight="extrabold" color={colors.white}>
                {near.name}
              </T>
              <T variant="small" color="rgba(255,255,255,0.9)">
                {near.fact}
              </T>
            </>
          ) : (
            <View style={{ flexDirection: 'row', alignItems: 'center', gap: 8 }}>
              <Ionicons name="hand-left-outline" size={18} color={colors.white} />
              <T variant="small" weight="semibold" color={colors.white} style={{ flex: 1 }}>
                Drag to look around — it wraps all the way back to where you started.
              </T>
            </View>
          )}
        </View>
        <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={styles.row} style={{ flexGrow: 0 }}>
          {pano.landmarks.map((l) => (
            <Press key={l.id} onPress={() => lookAt(l.heading)} style={[styles.lm, near?.id === l.id && { backgroundColor: colors.acce
nt }]} accessibilityLabel={`Look at ${l.name}`}>
              <Ionicons name="locate" size={13} color={near?.id === l.id ? colors.onAccent : colors.white} />
              <T variant="micro" weight="bold" color={near?.id === l.id ? colors.onAccent : colors.white}>
                {l.name}
              </T>
            </Press>
          ))}
        </ScrollView>
        <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={styles.row} style={{ flexGrow: 0 }}>
          {PANORAMAS.map((p) => {
            const on = p.id === pano.id;
            return (
              <Press
                key={p.id}
                onPress={() => pick(p.id)}
                style={[styles.city, on && { backgroundColor: colors.white }]}
                accessibilityRole="button"
                accessibilityState={{ selected: on }}
                accessibilityLabel={`${p.name} 360 degree view`}>
                <T style={{ fontSize: 16, lineHeight: 20 }}>{p.emoji}</T>
                <T variant="small" weight="bold" color={on ? '#111111' : colors.white}>
                  {p.name}
                </T>
```

```
              </Press>
            );
          })}
        </ScrollView>
      </View>
    </View>
  </View>
  );
}

const glass = 'rgba(20,22,32,0.5)';
const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: '#000', alignItems: 'center', overflow: 'hidden' },
  fade: { position: 'absolute', left: 0, right: 0 },
  top: { position: 'absolute', left: 14, right: 14, flexDirection: 'row', alignItems: 'center', gap: 8 },
  round: { width: 42, height: 42, borderRadius: 21, backgroundColor: glass, alignItems: 'center', justifyContent: 'center' },
  chip: { flexDirection: 'row', alignItems: 'center', gap: 6, height: 38, paddingHorizontal: 12, borderRadius: 999, backgroundColor: gla
ss },
  titleBlock: { position: 'absolute', left: 20, right: 20, gap: 2 },
  titleText: { fontSize: 52, lineHeight: 56, textShadowColor: 'rgba(0,0,0,0.5)', textShadowRadius: 10 },
  compass: { flexDirection: 'row', alignItems: 'center', gap: 5, alignSelf: 'flex-start', marginTop: 6, paddingHorizontal: 10, paddingVe
rtical: 5, borderRadius: 999, backgroundColor: glass },
  arrow: { position: 'absolute', top: '46%', width: 40, height: 40, borderRadius: 20, backgroundColor: glass, alignItems: 'center', just
ifyContent: 'center' },
  bottom: { position: 'absolute', left: 0, right: 0, bottom: 0, gap: 10 },
  callout: { marginHorizontal: 16, padding: 14, borderRadius: 20, backgroundColor: glass, gap: 4, minHeight: 58, justifyContent: 'cente
r' },
  row: { gap: 8, paddingHorizontal: 16 },
  lm: { flexDirection: 'row', alignItems: 'center', gap: 5, height: 34, paddingHorizontal: 12, borderRadius: 999, backgroundColor: glass
},
  city: { flexDirection: 'row', alignItems: 'center', gap: 6, height: 42, paddingHorizontal: 14, borderRadius: 999, backgroundColor: gla
ss },
});
```

src/app/personalize.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { LinearGradient } from 'expo-linear-gradient';
import { router } from 'expo-router';
import { useEffect, useState } from 'react';
import { Animated, ScrollView, StyleSheet, Switch, TextInput, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { type IconName } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { FadeIn, Float, nativeDriver } from '@/components/ui/motion';
import { Press } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { currencies } from '@/data/reference';
import { photos } from '@/data/images';
import { INTERESTS } from '@/data/personal';
import { LANGUAGES, type LangCode } from '@/i18n';
import { useAppStore, useCurrentUser } from '@/store/useAppStore';
import type { HomeSection, Personal } from '@/store/types';
import { MAX_WIDTH, appearance, applyAppearance, baseColors, cornerStyles, fonts, radius, shadow, themes, type CornerStyle, type ThemeKe
y } from '@/theme';

const EMOJIS = ['✈️', '🌴', '🏔️', '🍜', '📸', '🎒', '🌸', '🏄', '🚂', '🗺️', '🌙', '🦋'];

const SECTIONS: { key: HomeSection; label: string; sub: string; icon: IconName }[] = [
  { key: 'glance', label: 'Weather & local time', sub: 'Two quick tiles under the hero', icon: 'partly-sunny-outline' },
  { key: 'budget', label: 'Group budget', sub: 'Spent so far with a progress bar', icon: 'wallet-outline' },
  { key: 'upnext', label: 'Up next & today', sub: 'Countdown and today’s checklist', icon: 'alarm-outline' },
  { key: 'quick', label: 'Quick tiles', sub: 'Vote, flights, docs and photos', icon: 'grid-outline' },
  { key: 'forYou', label: 'Picked for you', sub: 'Destinations matching your interests', icon: 'sparkles-outline' },
  { key: 'nearby', label: 'Nearby radius', sub: 'Places around your trip', icon: 'navigate-outline' },
  { key: 'discover', label: 'Discover destinations', sub: 'Photo cards to dream about', icon: 'images-outline' },
];

const PACE: {
  key: Personal['travelStyle'];
  label: string;
  emoji: string;
  sub: string;
}[] = [
  {
    key: 'chill',
    label: 'Chill',
    emoji: '🧘',
    sub: '2–3 things a day, long lunches',
  },
  {
    key: 'balanced',
    label: 'Balanced',
    emoji: '⚖️',
    sub: 'A plan, with room to wander',
  },
  {
    key: 'packed',
    label: 'Packed',
    emoji: '⚡',
    sub: 'See everything, sleep later',
  },
];
const BUDGET: {
  key: Personal['budgetStyle'];
  label: string;
  emoji: string;
  sub: string;
}[] = [
  { key: 'saver', label: 'Saver', emoji: '🪙', sub: 'Street food & hostels' },
  { key: 'mid', label: 'Mid-range', emoji: '💳', sub: 'Comfy, not crazy' },
  {
    key: 'treat',
    label: 'Treat yo self',
    emoji: '💎',
    sub: 'Splurge on the good stuff',
  },
];

const STEPS = ['Hello', 'Vibe', 'Interests', 'Style', 'Home', 'You', 'Done'] as const;

/** Colours of a palette (merged over Forest) — used for the live previews. */
function paletteOf(k: ThemeKey) {
  return { ...baseColors, ...themes[k].palette };
}

export default function Personalize() {
  const insets = useSafeAreaInsets();
```

```
const user = useCurrentUser();
const saved = useAppStore((s) => s.personal);
const settings = useAppStore((s) => s.settings);
const updatePersonal = useAppStore((s) => s.updatePersonal);
const updateSettings = useAppStore((s) => s.updateSettings);
const setRestyled = useAppStore((s) => s.setRestyled);

const [step, setStep] = useState(0);
const [theme, setTheme] = useState<ThemeKey>(appearance.theme);
const [corners, setCorners] = useState<CornerStyle>(appearance.corners);
const [p, setP] = useState<Personal>(() => ({
  ...saved,
  sections: { ...saved.sections },
  interests: [...saved.interests],
}));
const [lang, setLang] = useState<LangCode>(settings.language);
const [currency, setCurrency] = useState(settings.homeCurrency);
const [applying, setApplying] = useState(false);
const [progress] = useState(() => new Animated.Value(0));

useEffect(() => {
  Animated.spring(progress, {
    toValue: step / (STEPS.length - 1),
    useNativeDriver: false,
    friction: 9,
  }).start();
}, [progress, step]);

const set = (patch: Partial<Personal>) => setP((cur) => ({ ...cur, ...patch }));
const toggleInterest = (k: string) =>
  set({
    interests: p.interests.includes(k) ? p.interests.filter((x) => x !== k) : [...p.interests, k],
  });
const pal = paletteOf(theme);
const firstName = user?.name.split(' ')[0] ?? 'traveller';
const shownName = p.nickname.trim() || firstName;
const last = step === STEPS.length - 1;

const next = () => {
  if (step === 2 && p.interests.length === 0) {
    toast('Pick at least one interest so we can tailor Explore', {
      tone: 'warn',
      icon: 'alert-circle-outline',
    });
    return;
  }
  if (step === 5 && p.nickname.trim().length > 20) {
    toast('Keep your nickname under 20 characters', {
      tone: 'warn',
      icon: 'alert-circle-outline',
    });
    return;
  }
  setStep((s) => Math.min(STEPS.length - 1, s + 1));
};
const back = () => (step === 0 ? router.back() : setStep((s) => s - 1));

const apply = () => {
  setApplying(true);
  updatePersonal({ ...p, nickname: p.nickname.trim(), done: true });
  updateSettings({ language: lang, homeCurrency: currency });
  const restyle = theme !== appearance.theme || corners !== appearance.corners;
  if (!restyle) {
    toast('All set — the app is now yours ✨', { icon: 'sparkles' });
    router.replace('/(tabs)/explore');
    return;
  }
  setRestyled(true);
  // Give the store a moment to save before the UI restarts with the new look.
  setTimeout(() => {
    if (!applyAppearance({ theme, corners })) {
      setApplying(false);
      toast('Saved! Close and reopen the app to see your new look.', {
        icon: 'refresh',
      });
      router.replace('/(tabs)/explore');
    }
  }, 450);
};

return (
  <View style={[styles.root, { backgroundColor: pal.bg, paddingTop: insets.top }]}>
    <View style={styles.column}>
      {/* Top bar */}
      <View style={styles.top}>
        <Press onPress={back} accessibilityLabel={step === 0 ? 'Close' : 'Back'} style={[styles.round, { backgroundColor: pal.card, bo
```

```
rderColor: pal.border }]}>
            <Ionicons name={step === 0 ? 'close' : 'chevron-back'} size={20} color={pal.text} />
          </Press>
          <View style={{ flex: 1 }}>
            <View style={[styles.track, { backgroundColor: pal.border }]}>
              <Animated.View
                style={[
                  styles.fill,
                  {
                    backgroundColor: pal.primary,
                    width: progress.interpolate({
                      inputRange: [0, 1],
                      outputRange: ['4%', '100%'],
                    }),
                  },
                ]}
              />
            </View>
            <T variant="micro" color={pal.textSecondary} style={{ marginTop: 5 }}>
              Step {step + 1} of {STEPS.length} · {STEPS[step]}
            </T>
          </View>
          {!last ? (
            <Press onPress={() => setStep(STEPS.length - 1)} hitSlop={8} accessibilityLabel="Skip to the end">
              <T variant="small" weight="semibold" color={pal.textSecondary}>
                Skip
              </T>
            </Press>
          ) : (
            <View style={{ width: 30 }} />
          )}
        </View>

        <ScrollView contentContainerStyle={{ padding: 20, paddingBottom: 30 }} keyboardShouldPersistTaps="handled" showsVerticalScrollIn
dicator={false}>
          <FadeIn key={step} from="left" distance={28} duration={380}>
            {step === 0 ? (
              <View style={{ alignItems: 'center', paddingTop: 10 }}>
                <View style={styles.bubbles}>
                  {['🌍', '✈️', '🎒', '🏝️', '🍜', '📸'].map((e, i) => (
                    <Float key={e} amount={10} duration={1800 + i * 260} delay={i * 120} style={[styles.bubble, bubblePos[i], { backgrou
ndColor: pal.card, borderColor: pal.border }]}>
                      <T style={{ fontSize: 26, lineHeight: 32 }}>{e}</T>
                    </Float>
                  ))}
                  <LinearGradient colors={[pal.heroA, pal.heroB, pal.heroC]} start={{ x: 0, y: 0 }} end={{ x: 1, y: 1 }} style={styles.h
eroCircle}>
                    <T style={{ fontSize: 54, lineHeight: 64 }}>{p.emoji}</T>
                  </LinearGradient>
                </View>
                <T variant="kicker" color={pal.primary} style={{ marginTop: 18 }}>
                  60-second survey
                </T>
                <T variant="display" center color={pal.text} style={{ marginTop: 6 }}>
                  Let’s make it{'\n'}yours, {firstName}
                </T>
                <T variant="body" center color={pal.textSecondary} style={{ marginTop: 10, maxWidth: 320 }}>
                  Pick a colour vibe, your travel interests and what shows up on your home screen. You can change any of it later in Set
tings.
                </T>
                <View
                  style={{
                    flexDirection: 'row',
                    flexWrap: 'wrap',
                    gap: 8,
                    justifyContent: 'center',
                    marginTop: 18,
                  }}>
                  {['🎨 6 colour themes', '❤️ Your interests', '🏠 Your home screen', '😎 Your avatar'].map((x) => (
                    <View
                      key={x}
                      style={[
                        styles.tag,
                        {
                          backgroundColor: pal.primarySofter,
                          borderColor: pal.primaryLine,
                        },
                      ]}>
                      <T variant="small" weight="medium" color={pal.primary}>
                        {x}
                      </T>
                    </View>
                  ))}
                </View>
              </View>
            ) : null}
```

```
            {step === 1 ? (
              <View>
                <StepTitle pal={pal} kicker="Pick your vibe" title="Which colours feel like you?" sub="The whole app re-paints itself —
buttons, cards, the animated home banner, everything." />
                <View style={styles.grid}>
                  {(Object.keys(themes) as ThemeKey[]).map((k, i) => {
                    const tp = paletteOf(k);
                    const on = theme === k;
                    return (
                      <FadeIn key={k} delay={i * 60} from="scale" style={{ width: '48%' }}>
                        <Press
                          onPress={() => setTheme(k)}
                          accessibilityRole="radio"
                          accessibilityState={{ checked: on }}
                          accessibilityLabel={`${themes[k].name} theme`}
                          style={[
                            styles.themeCard,
                            {
                              backgroundColor: tp.bg,
                              borderColor: on ? pal.primary : pal.border,
                              borderWidth: on ? 2.5 : 1,
                            },
                          ]}>
                          <LinearGradient colors={[tp.heroA, tp.heroB, tp.heroC]} start={{ x: 0, y: 0 }} end={{ x: 1, y: 1 }} style={sty
les.themeHero}>
                            <T style={{ fontSize: 22 }}>{themes[k].emoji}</T>
                            <View style={[styles.themeSun, { backgroundColor: tp.accent }]} />
                          </LinearGradient>
                          <View style={{ padding: 10, gap: 6 }}>
                            <View
                              style={{
                                flexDirection: 'row',
                                alignItems: 'center',
                                justifyContent: 'space-between',
                              }}>
                              <T variant="title" weight="bold" color={tp.text}>
                                {themes[k].name}
                              </T>
                              {on ? <Ionicons name="checkmark-circle" size={20} color={pal.primary} /> : null}
                            </View>
                            <T variant="caption" color={tp.textSecondary} numberOfLines={1}>
                              {themes[k].vibe}
                            </T>
                            <View style={{ flexDirection: 'row', gap: 5 }}>
                              {[tp.primary, tp.primarySoft, tp.accent, tp.card].map((c, j) => (
                                <View
                                  key={j}
                                  style={[
                                    styles.swatch,
                                    {
                                      backgroundColor: c,
                                      borderColor: tp.border,
                                    },
                                  ]}
                                />
                              ))}
                            </View>
                          </View>
                        </Press>
                      </FadeIn>
                    );
                  })}
                </View>
                <MiniPreview pal={pal} corners={corners} name={shownName} emoji={p.emoji} />
              </View>
            ) : null}

            {step === 2 ? (
              <View>
                <StepTitle
                  pal={pal}
                  kicker="Your interests"
                  title="What gets you excited to travel?"
                  sub={`Pick as many as you like — “Picked for you” on Explore uses these. ${p.interests.length ? `${p.interests.length}
selected` : ''}`}
                />
                <View style={styles.grid}>
                  {INTERESTS.map((it, i) => {
                    const on = p.interests.includes(it.key);
                    return (
                      <FadeIn key={it.key} delay={i * 50} from="scale" style={{ width: '48%' }}>
                        <Press
                          onPress={() => toggleInterest(it.key)}
                          accessibilityRole="checkbox"
                          accessibilityState={{ checked: on }}
```

```
                        accessibilityLabel={it.label}
                        style={[styles.interest, { borderColor: on ? pal.primary : 'transparent' }]}>
                        <Image source={photos[it.image]} style={StyleSheet.absoluteFill} contentFit="cover" />
                        <LinearGradient colors={on ? [`${pal.primary}55`, `${pal.primary}EE`] : ['rgba(0,0,0,0)', 'rgba(0,0,0,0.72)']}
style={StyleSheet.absoluteFill} />
                        <View
                          style={[
                            styles.check,
                            {
                              backgroundColor: on ? pal.accent : 'rgba(255,255,255,0.35)',
                            },
                          ]}>
                          {on ? <Ionicons name="checkmark" size={14} color="#1A1C17" /> : null}
                        </View>
                        <T style={{ fontSize: 24 }}>{it.emoji}</T>
                        <T variant="small" weight="bold" color="#FFFFFF">
                          {it.label}
                        </T>
                      </Press>
                    </FadeIn>
                  );
                })}
              </View>
            </View>
          ) : null}

          {step === 3 ? (
            <View>
              <StepTitle pal={pal} kicker="Travel style" title="How do you like to travel?" sub="Helps the Trip Assistant and recommen
dations match your pace and wallet." />
              <T variant="title" color={pal.text} style={{ marginBottom: 8 }}>
                Pace
              </T>
              <View style={{ gap: 8 }}>
                {PACE.map((o) => (
                  <OptionRow key={o.key} pal={pal} on={p.travelStyle === o.key} emoji={o.emoji} label={o.label} sub={o.sub} onPress=
{() => set({ travelStyle: o.key })} />
                ))}
              </View>
              <T variant="title" color={pal.text} style={{ marginTop: 18, marginBottom: 8 }}>
                Budget
              </T>
              <View style={{ gap: 8 }}>
                {BUDGET.map((o) => (
                  <OptionRow key={o.key} pal={pal} on={p.budgetStyle === o.key} emoji={o.emoji} label={o.label} sub={o.sub} onPress=
{() => set({ budgetStyle: o.key })} />
                ))}
              </View>
            </View>
          ) : null}

          {step === 4 ? (
            <View>
              <StepTitle pal={pal} kicker="Your home screen" title="Build your Explore page" sub="Switch sections on or off, pick a ca
rd shape and decide how lively it should feel." />
              <View style={[styles.card, { backgroundColor: pal.card, borderColor: pal.border }]}>
                {SECTIONS.map((s, i) => (
                  <View
                    key={s.key}
                    style={[
                      styles.secRow,
                      i > 0 && {
                        borderTopWidth: 1,
                        borderTopColor: pal.border,
                      },
                    ]}>
                    <View style={[styles.secIcon, { backgroundColor: pal.primarySofter }]}>
                      <Ionicons name={s.icon} size={17} color={pal.primary} />
                    </View>
                    <View style={{ flex: 1 }}>
                      <T variant="small" weight="semibold" color={pal.text}>
                        {s.label}
                      </T>
                      <T variant="caption" color={pal.textSecondary}>
                        {s.sub}
                      </T>
                    </View>
                    <Switch
                      value={p.sections[s.key]}
                      onValueChange={(v) => set({ sections: { ...p.sections, [s.key]: v } })}
                      trackColor={{
                        true: pal.primary,
                        false: pal.borderStrong,
                      }}
                      thumbColor="#FFFFFF"
                      accessibilityLabel={`Show ${s.label}`}
```

```
                    />
                  </View>
                ))}
              </View>

              <T variant="title" color={pal.text} style={{ marginTop: 18, marginBottom: 8 }}>
                Card shape
              </T>
              <View style={{ flexDirection: 'row', gap: 8 }}>
                {(Object.keys(cornerStyles) as CornerStyle[]).map((c) => {
                  const on = corners === c;
                  return (
                    <Press
                      key={c}
                      onPress={() => setCorners(c)}
                      accessibilityRole="radio"
                      accessibilityState={{ checked: on }}
                      accessibilityLabel={`${cornerStyles[c].name} corners`}
                      style={[
                        styles.cornerOpt,
                        {
                          backgroundColor: on ? pal.primarySoft : pal.card,
                          borderColor: on ? pal.primary : pal.border,
                        },
                      ]}>
                      <View
                        style={[
                          styles.cornerDemo,
                          {
                            borderRadius: Math.round(14 * cornerStyles[c].scale),
                            backgroundColor: pal.primary,
                          },
                        ]}
                      />
                      <T variant="small" weight="semibold" color={on ? pal.primary : pal.text}>
                        {cornerStyles[c].name}
                      </T>
                    </Press>
                  );
                })}
              </View>

              <View
                style={[
                  styles.card,
                  styles.secRow,
                  {
                    backgroundColor: pal.card,
                    borderColor: pal.border,
                    marginTop: 14,
                  },
                ]}>
                <View style={[styles.secIcon, { backgroundColor: pal.accentSoft }]}>
                  <Ionicons name="sparkles" size={17} color={pal.accent} />
                </View>
                <View style={{ flex: 1 }}>
                  <T variant="small" weight="semibold" color={pal.text}>
                    Animations
                  </T>
                  <T variant="caption" color={pal.textSecondary}>
                    Flying plane, floating cards, smooth entrances
                  </T>
                </View>
                <Switch value={p.motion} onValueChange={(v) => set({ motion: v })} trackColor={{ true: pal.primary, false: pal.borderS
trong }} thumbColor="#FFFFFF" accessibilityLabel="Animations" />
              </View>
            </View>
          ) : null}

          {step === 5 ? (
            <View>
              <StepTitle pal={pal} kicker="About you" title="How should we greet you?" sub="Choose a nickname and a travel avatar. The
y show up on your home screen." />
              <View style={[styles.input, { backgroundColor: pal.card, borderColor: pal.border }]}>
                <Ionicons name="happy-outline" size={18} color={pal.textMuted} />
                <TextInput
                  value={p.nickname}
                  onChangeText={(v) => set({ nickname: v })}
                  placeholder={`Nickname (default: ${firstName})`}
                  placeholderTextColor={pal.textMuted}
                  maxLength={24}
                  style={[styles.inputText, { color: pal.text }]}
                  accessibilityLabel="Nickname"
                />
              </View>
              <T variant="title" color={pal.text} style={{ marginTop: 18, marginBottom: 8 }}>
```

```
                  Travel avatar
                </T>
                <View style={{ flexDirection: 'row', flexWrap: 'wrap', gap: 8 }}>
                  {EMOJIS.map((e) => {
                    const on = p.emoji === e;
                    return (
                      <Press
                        key={e}
                        onPress={() => set({ emoji: e })}
                        accessibilityRole="radio"
                        accessibilityState={{ checked: on }}
                        accessibilityLabel={`Avatar ${e}`}
                        scaleTo={0.85}
                        style={[
                          styles.emoji,
                          {
                            backgroundColor: on ? pal.primarySoft : pal.card,
                            borderColor: on ? pal.primary : pal.border,
                          },
                        ]}>
                        <T style={{ fontSize: 24, lineHeight: 30 }}>{e}</T>
                      </Press>
                    );
                  })}
                </View>
                <T variant="title" color={pal.text} style={{ marginTop: 18, marginBottom: 8 }}>
                  Language
                </T>
                <View style={{ flexDirection: 'row', flexWrap: 'wrap', gap: 8 }}>
                  {LANGUAGES.map((l) => (
                    <SmallChip key={l.code} pal={pal} on={lang === l.code} label={`${l.flag} ${l.native}`} onPress={() => setLang(l.cod
e)} />
                  ))}
                </View>
                <T variant="title" color={pal.text} style={{ marginTop: 18, marginBottom: 8 }}>
                  Home currency
                </T>
                <View style={{ flexDirection: 'row', flexWrap: 'wrap', gap: 8 }}>
                  {currencies.slice(0, 10).map((c) => (
                    <SmallChip key={c.code} pal={pal} on={currency === c.code} label={`${c.symbol} ${c.code}`} onPress={() => setCurrenc
y(c.code)} />
                  ))}
                </View>
              </View>
            ) : null}

            {step === 6 ? (
              <View style={{ alignItems: 'center' }}>
                <Confetti pal={pal} />
                <LinearGradient colors={[pal.heroA, pal.heroB, pal.heroC]} start={{ x: 0, y: 0 }} end={{ x: 1, y: 1 }} style={styles.her
oCircle}>
                  <T style={{ fontSize: 54, lineHeight: 64 }}>{p.emoji}</T>
                </LinearGradient>
                <T variant="display" center color={pal.text} style={{ marginTop: 14 }}>
                  Looking good, {shownName}!
                </T>
                <T variant="body" center color={pal.textSecondary} style={{ marginTop: 6 }}>
                  Here’s your TogetherWeGo:
                </T>
                <View
                  style={[
                    styles.card,
                    {
                      backgroundColor: pal.card,
                      borderColor: pal.border,
                      alignSelf: 'stretch',
                      marginTop: 16,
                    },
                  ]}>
                  <Summary pal={pal} icon="color-palette-outline" label="Theme" value={`${themes[theme].emoji} ${themes[theme].name} ·
${cornerStyles[corners].name} corners`} />
                  <Summary
                    pal={pal}
                    icon="heart-outline"
                    label="Interests"
                    value={
                      p.interests.length
                        ? INTERESTS.filter((i) => p.interests.includes(i.key))
                            .map((i) => i.emoji)
                            .join(' ')
                        : 'Everything'
                    }
                  />
                  <Summary pal={pal} icon="walk-outline" label="Style" value={`${PACE.find((x) => x.key === p.travelStyle)?.label} pace
· ${BUDGET.find((x) => x.key === p.budgetStyle)?.label}`} />
                  <Summary pal={pal} icon="home-outline" label="Home sections" value={`${Object.values(p.sections).filter(Boolean).lengt
```

```
h} of ${SECTIONS.length} on`} />
                <Summary pal={pal} icon="globe-outline" label="Language & currency" value={`${LANGUAGES.find((l) => l.code === lang)?.
native} · ${currency}`} />
              </View>
              <MiniPreview pal={pal} corners={corners} name={shownName} emoji={p.emoji} />
            </View>
          ) : null}
        </FadeIn>
      </ScrollView>

      <View
        style={[
          styles.footer,
          {
            paddingBottom: insets.bottom + 14,
            backgroundColor: pal.bg,
            borderTopColor: pal.border,
          },
        ]}>
        <Press
          onPress={last ? apply : next}
          disabled={applying}
          accessibilityRole="button"
          accessibilityLabel={last ? 'Apply my style' : step === 0 ? 'Let’s go' : 'Continue'}
          style={[styles.cta, { opacity: applying ? 0.6 : 1 }]}>
          <LinearGradient colors={[pal.primary, pal.heroB]} start={{ x: 0, y: 0 }} end={{ x: 1, y: 0 }} style={[StyleSheet.absoluteFil
l, { borderRadius: 999 }]} />
          <T variant="title" weight="bold" color="#FFFFFF">
            {applying ? 'Applying your look…' : last ? 'Apply my style ✨' : step === 0 ? 'Let’s go' : 'Continue'}
          </T>
          {!last ? <Ionicons name="arrow-forward" size={18} color="#FFFFFF" /> : null}
        </Press>
      </View>
    </View>
  </View>
  );
}

type Pal = ReturnType<typeof paletteOf>;

const bubblePos = [
  { left: 0, top: 30 },
  { right: 4, top: 10 },
  { left: 22, bottom: 0 },
  { right: 0, bottom: 20 },
  { left: 92, top: -6 },
  { right: 86, bottom: -12 },
] as const;

function StepTitle({ pal, kicker, title, sub }: { pal: Pal; kicker: string; title: string; sub: string }) {
  return (
    <View style={{ marginBottom: 16 }}>
      <T variant="kicker" color={pal.primary}>
        {kicker}
      </T>
      <T variant="h1" color={pal.text} style={{ marginTop: 4 }} accessibilityRole="header">
        {title}
      </T>
      <T variant="bodySm" color={pal.textSecondary} style={{ marginTop: 6 }}>
        {sub}
      </T>
    </View>
  );
}

function OptionRow({ pal, on, emoji, label, sub, onPress }: { pal: Pal; on: boolean; emoji: string; label: string; sub: string; onPress:
() => void }) {
  return (
    <Press
      onPress={onPress}
      accessibilityRole="radio"
      accessibilityState={{ checked: on }}
      accessibilityLabel={label}
      style={[
        styles.option,
        {
          backgroundColor: on ? pal.primarySofter : pal.card,
          borderColor: on ? pal.primary : pal.border,
        },
      ]}>
      <T style={{ fontSize: 26, lineHeight: 32 }}>{emoji}</T>
      <View style={{ flex: 1 }}>
        <T variant="title" color={pal.text}>
          {label}
        </T>
        <T variant="caption" color={pal.textSecondary}>
```

```
          {sub}
        </T>
      </View>
      <Ionicons name={on ? 'radio-button-on' : 'radio-button-off'} size={22} color={on ? pal.primary : pal.textMuted} />
    </Press>
  );
}

function SmallChip({ pal, on, label, onPress }: { pal: Pal; on: boolean; label: string; onPress: () => void }) {
  return (
    <Press
      onPress={onPress}
      accessibilityRole="radio"
      accessibilityState={{ checked: on }}
      accessibilityLabel={label}
      style={[
        styles.smallChip,
        {
          backgroundColor: on ? pal.primary : pal.card,
          borderColor: on ? pal.primary : pal.border,
        },
      ]}>
      <T variant="small" weight="semibold" color={on ? '#FFFFFF' : pal.text}>
        {label}
      </T>
    </Press>
  );
}

function Summary({ pal, icon, label, value }: { pal: Pal; icon: IconName; label: string; value: string }) {
  return (
    <View style={styles.secRow}>
      <View style={[styles.secIcon, { backgroundColor: pal.primarySofter }]}>
        <Ionicons name={icon} size={16} color={pal.primary} />
      </View>
      <T variant="small" color={pal.textSecondary} style={{ width: 120 }}>
        {label}
      </T>
      <T variant="small" weight="semibold" color={pal.text} style={{ flex: 1, textAlign: 'right' }}>
        {value}
      </T>
    </View>
  );
}

/** A tiny phone-style preview of the Explore page in the chosen palette + corners. */
function MiniPreview({ pal, corners, name, emoji }: { pal: Pal; corners: CornerStyle; name: string; emoji: string }) {
  const r = (n: number) => Math.round(n * cornerStyles[corners].scale);
  return (
    <View style={[styles.preview, { backgroundColor: pal.bg, borderColor: pal.borderStrong }]} accessibilityLabel="Live preview">
      <T variant="kicker" color={pal.textMuted} style={{ marginBottom: 8 }}>
        Live preview
      </T>
      <LinearGradient colors={[pal.heroA, pal.heroB, pal.heroC]} start={{ x: 0, y: 0 }} end={{ x: 1, y: 1 }} style={{ borderRadius: r(1
8), padding: 12, height: 86 }}>
        <T variant="caption" color="rgba(255,255,255,0.85)">
          Hi {name} {emoji}
        </T>
        <T variant="h3" color="#FFFFFF">
          Where to next?
        </T>
        <View style={[styles.previewSun, { backgroundColor: pal.accent }]} />
      </LinearGradient>
      <View style={{ flexDirection: 'row', gap: 8, marginTop: 8 }}>
        <View
          style={{
            flex: 1,
            height: 44,
            borderRadius: r(14),
            backgroundColor: pal.primary,
          }}
        />
        <View
          style={{
            flex: 1,
            height: 44,
            borderRadius: r(14),
            backgroundColor: pal.card,
            borderWidth: 1,
            borderColor: pal.border,
          }}
        />
      </View>
      <View style={{ flexDirection: 'row', gap: 6, marginTop: 8 }}>
        {[0, 1, 2].map((i) => (
          <View
```

```
            key={i}
            style={{
              paddingHorizontal: 12,
              height: 22,
              borderRadius: 11,
              backgroundColor: i === 0 ? pal.primary : pal.primarySoft,
            }}
          />
        ))}
      </View>
    </View>
  );
}

const CONFETTI = ['#F2B33D', '#E8833A', '#3D7DD8', '#7C5CD6', '#2A9D8F', '#D94F86', '#8CC97A', '#F5C451', '#5CC8D6', '#FF8A5B', '#B8326
8', '#2FAE66'];

/** One-shot confetti burst for the final step. */
function Confetti({ pal }: { pal: Pal }) {
  const [v] = useState(() => new Animated.Value(0));
  const motion = useAppStore((s) => s.personal.motion);
  useEffect(() => {
    if (!motion) return;
    Animated.timing(v, {
      toValue: 1,
      duration: 1600,
      useNativeDriver: nativeDriver,
    }).start();
  }, [v, motion]);
  if (!motion) return null;
  return (
    <View pointerEvents="none" style={styles.confetti}>
      {CONFETTI.concat(CONFETTI).map((c, i) => {
        const angle = (i / 24) * Math.PI * 2;
        const dist = 90 + (i % 4) * 30;
        return (
          <Animated.View
            key={i}
            style={[
              styles.piece,
              {
                backgroundColor: i % 5 === 0 ? pal.accent : c,
                borderRadius: i % 3 === 0 ? 5 : 2,
                opacity: v.interpolate({
                  inputRange: [0, 0.1, 0.8, 1],
                  outputRange: [0, 1, 1, 0],
                }),
                transform: [
                  {
                    translateX: v.interpolate({
                      inputRange: [0, 1],
                      outputRange: [0, Math.cos(angle) * dist],
                    }),
                  },
                  {
                    translateY: v.interpolate({
                      inputRange: [0, 0.6, 1],
                      outputRange: [0, Math.sin(angle) * dist - 20, Math.sin(angle) * dist + 40],
                    }),
                  },
                  {
                    rotate: v.interpolate({
                      inputRange: [0, 1],
                      outputRange: ['0deg', `${(i % 2 ? 1 : -1) * 540}deg`],
                    }),
                  },
                ],
              },
            ]}
          />
        );
      })}
    </View>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, alignItems: 'center' },
  column: { flex: 1, width: '100%', maxWidth: MAX_WIDTH },
  top: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 12,
    paddingHorizontal: 20,
    paddingTop: 8,
    paddingBottom: 4,
```

```
},
round: {
  width: 40,
  height: 40,
  borderRadius: 20,
  borderWidth: 1,
  alignItems: 'center',
  justifyContent: 'center',
},
track: { height: 8, borderRadius: 4, overflow: 'hidden' },
fill: { height: 8, borderRadius: 4 },
bubbles: {
  width: 260,
  height: 190,
  alignItems: 'center',
  justifyContent: 'center',
},
bubble: {
  position: 'absolute',
  width: 54,
  height: 54,
  borderRadius: 27,
  borderWidth: 1,
  alignItems: 'center',
  justifyContent: 'center',
  ...shadow,
},
heroCircle: {
  width: 116,
  height: 116,
  borderRadius: 58,
  alignItems: 'center',
  justifyContent: 'center',
  ...shadow,
},
tag: {
  paddingHorizontal: 12,
  paddingVertical: 7,
  borderRadius: 999,
  borderWidth: 1,
},
grid: {
  flexDirection: 'row',
  flexWrap: 'wrap',
  justifyContent: 'space-between',
  rowGap: 12,
},
themeCard: { borderRadius: radius.lg, overflow: 'hidden' },
themeHero: { height: 64, padding: 10, justifyContent: 'flex-end' },
themeSun: {
  position: 'absolute',
  right: 12,
  top: 10,
  width: 22,
  height: 22,
  borderRadius: 11,
},
swatch: { width: 18, height: 18, borderRadius: 9, borderWidth: 1 },
interest: {
  height: 128,
  borderRadius: radius.lg,
  overflow: 'hidden',
  padding: 12,
  justifyContent: 'flex-end',
  borderWidth: 3,
},
check: {
  position: 'absolute',
  top: 10,
  right: 10,
  width: 24,
  height: 24,
  borderRadius: 12,
  alignItems: 'center',
  justifyContent: 'center',
},
option: {
  flexDirection: 'row',
  alignItems: 'center',
  gap: 12,
  padding: 14,
  borderRadius: radius.lg,
  borderWidth: 1.5,
},
card: {
  borderRadius: radius.lg,
```

```
  borderWidth: 1,
  paddingHorizontal: 14,
  paddingVertical: 4,
},
secRow: {
  flexDirection: 'row',
  alignItems: 'center',
  gap: 10,
  paddingVertical: 10,
},
secIcon: {
  width: 34,
  height: 34,
  borderRadius: 11,
  alignItems: 'center',
  justifyContent: 'center',
},
cornerOpt: {
  flex: 1,
  alignItems: 'center',
  gap: 8,
  paddingVertical: 14,
  borderRadius: radius.md,
  borderWidth: 1.5,
},
cornerDemo: { width: 44, height: 30 },
input: {
  flexDirection: 'row',
  alignItems: 'center',
  gap: 10,
  borderWidth: 1,
  borderRadius: radius.md,
  paddingHorizontal: 14,
  height: 52,
},
inputText: {
  flex: 1,
  fontFamily: fonts.regular,
  fontSize: 15,
  height: '100%',
  ...({ outlineStyle: 'none' } as object),
},
emoji: {
  width: 52,
  height: 52,
  borderRadius: 16,
  borderWidth: 1.5,
  alignItems: 'center',
  justifyContent: 'center',
},
smallChip: {
  paddingHorizontal: 13,
  paddingVertical: 8,
  borderRadius: 999,
  borderWidth: 1,
},
preview: {
  alignSelf: 'stretch',
  marginTop: 18,
  borderWidth: 1,
  borderRadius: 22,
  padding: 12,
},
previewSun: {
  position: 'absolute',
  right: 14,
  top: 12,
  width: 26,
  height: 26,
  borderRadius: 13,
},
footer: { paddingHorizontal: 20, paddingTop: 12, borderTopWidth: 1 },
cta: {
  height: 56,
  borderRadius: 999,
  flexDirection: 'row',
  alignItems: 'center',
  justifyContent: 'center',
  gap: 8,
  overflow: 'hidden',
  ...shadow,
},
confetti: {
  position: 'absolute',
  top: 58,
  left: 0,
```

```
    right: 0,
    alignItems: 'center',
    zIndex: 2,
  },
  piece: { position: 'absolute', width: 9, height: 14 },
});
```

src/app/photos.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { useMemo, useState } from 'react';
import { Modal, Platform, ScrollView, StyleSheet, View, useWindowDimensions } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { ShareSheet } from '@/components/ShareSheet';
import { NoTrip } from '@/components/trip/NoTrip';
import { Button } from '@/components/ui/Button';
import { Avatar, Chip, EmptyState } from '@/components/ui/bits';
import { confirmAction, toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { Press, tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import { imageSource } from '@/data/images';
import { destinationById } from '@/data/destinations';
import { FileTooLargeError, deleteLocalFile, pickImage } from '@/services/files';
import { sharePhoto } from '@/services/share';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import type { Photo } from '@/store/types';
import { MAX_WIDTH, colors } from '@/theme';
import { timeAgo } from '@/utils/format';
import { dayLabel, todayInTz, tripDays } from '@/utils/time';
import { maxLen } from '@/utils/validation';

type Filter = 'all' | 'mine' | 'liked' | string;

export default function Photos() {
  const trip = useActiveTrip();
  const { width } = useWindowDimensions();
  const insets = useSafeAreaInsets();
  const userId = useAppStore((s) => s.session?.userId ?? '');
  const addPhoto = useAppStore((s) => s.addPhoto);
  const like = useAppStore((s) => s.toggleLikePhoto);
  const remove = useAppStore((s) => s.deletePhoto);
  const [filter, setFilter] = useState<Filter>('all');
  const [viewing, setViewing] = useState<Photo | null>(null);
  const [pending, setPending] = useState<string | null>(null);
  const [caption, setCaption] = useState('');
  const [captionErr, setCaptionErr] = useState<string | null>(null);
  const [shareFor, setShareFor] = useState<Photo | null>(null);

  const list = useMemo(() => {
    if (!trip) return [];
    const p = trip.photos.slice();
    if (filter === 'mine') return p.filter((x) => x.by === userId);
    if (filter === 'liked') return p.sort((a, b) => b.likes.length - a.likes.length);
    if (filter !== 'all') return p.filter((x) => x.day === filter);
    return p;
  }, [trip, filter, userId]);

  if (!trip) return <NoTrip />;
  const colW = Math.min(width, MAX_WIDTH) - 40;
  const tile = (colW - 8) / 3;
  const live = viewing ? trip.photos.find((p) => p.id === viewing.id) ?? null : null;
  const days = tripDays(trip.startDate, trip.endDate);

  const add = async (camera: boolean) => {
    try {
      const f = await pickImage(camera);
      if (f) {
        setPending(f.uri);
        setCaption('');
      }
    } catch (e) {
      toast(e instanceof FileTooLargeError ? e.message : e instanceof Error ? e.message : 'Couldn’t open photos', { tone: 'warn' });
    }
  };

  const today = todayInTz(destinationById(trip.destinationId)?.tz ?? 'UTC');

  return (
    <Screen header={<Header title="Photo dump" subtitle={`${trip.photos.length} photos · ${trip.name}`} />}>
      <View style={{ flexDirection: 'row', gap: 8 }}>
        <Button label="Add photos" icon="images-outline" size="md" onPress={() => add(false)} style={{ flex: 1 }} />
        {Platform.OS !== 'web' ? <Button label="Camera" icon="camera-outline" size="md" variant="secondary" onPress={() => add(true)} st
yle={{ flex: 1 }} /> : null}
      </View>
      <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 6, paddingHorizontal: 20 }} style={{ m
arginHorizontal: -20, marginVertical: 14 }}>
        <Chip small label="All" active={filter === 'all'} onPress={() => setFilter('all')} />
        <Chip small label="❤️ Most liked" active={filter === 'liked'} onPress={() => setFilter('liked')} />
```

```
        <Chip small label="Mine" active={filter === 'mine'} onPress={() => setFilter('mine')} />
        {days.map((d, i) => (
          <Chip key={d} small label={`Day ${i + 1}`} active={filter === d} onPress={() => setFilter(d)} />
        ))}
      </ScrollView>
      {list.length ? (
        <View style={styles.grid}>
          {list.map((p) => (
            <Press key={p.id} onPress={() => setViewing(p)} style={{ width: tile, height: tile }} scaleTo={0.96} accessibilityLabel={p.c
aption || 'Photo'}>
              <Image source={imageSource(p.uri)} style={styles.img} contentFit="cover" transition={150} />
              {p.likes.length ? (
                <View style={styles.likes}>
                  <Ionicons name="heart" size={10} color={colors.white} />
                  <T variant="micro" color={colors.white}>
                    {p.likes.length}
                  </T>
                </View>
              ) : null}
            </Press>
          ))}
        </View>
      ) : (
        <EmptyState icon="images-outline" title="No photos here yet" body="Add your best shots — everyone in the trip can like and share
them." />
      )}

      <Sheet
        visible={!!pending}
        onClose={() => setPending(null)}
        title="Add a caption"
        footer={
          <Button
            label="Post to photo dump"
            icon="cloud-upload-outline"
            onPress={() => {
              const e = maxLen(80, 'Caption')(caption);
              setCaptionErr(e);
              if (e || !pending) return;
              addPhoto(trip.id, { uri: pending, caption: caption.trim(), day: days.includes(today) ? today : trip.startDate });
              setPending(null);
              tap('success');
              toast('Photo added 📸');
            }}
          />
        }>
        {pending ? <Image source={{ uri: pending }} style={{ width: '100%', height: 220, borderRadius: 16 }} contentFit="cover" /> : nul
l}
        <Input placeholder="Say something about it…" value={caption} onChangeText={setCaption} error={captionErr} maxLength={90} />
      </Sheet>

      <Modal visible={!!live} transparent animationType="fade" onRequestClose={() => setViewing(null)}>
        {live ? (
          <View style={[styles.viewer, { paddingTop: insets.top + 8, paddingBottom: insets.bottom + 16 }]}>
            <View style={styles.viewerTop}>
              <Press onPress={() => setViewing(null)} style={styles.round} accessibilityLabel="Close photo">
                <Ionicons name="close" size={22} color={colors.white} />
              </Press>
              <View style={{ flexDirection: 'row', gap: 10 }}>
                <Press onPress={() => setShareFor(live)} style={styles.round} accessibilityLabel="Share photo to social media">
                  <Ionicons name="share-social-outline" size={20} color={colors.white} />
                </Press>
                {live.by === userId ? (
                  <Press
                    onPress={() =>
                      confirmAction('Delete photo?', 'It will be removed from the photo dump for everyone.', () => {
                        deleteLocalFile(live.uri);
                        remove(trip.id, live.id);
                        setViewing(null);
                      }, { destructive: true, confirmLabel: 'Delete' })
                    }
                    style={styles.round}
                    accessibilityLabel="Delete photo">
                    <Ionicons name="trash-outline" size={20} color={colors.white} />
                  </Press>
                ) : null}
              </View>
            </View>
            <Image source={imageSource(live.uri)} style={{ flex: 1, width: '100%' }} contentFit="contain" />
            <View style={styles.viewerBottom}>
              <View style={{ flexDirection: 'row', alignItems: 'center', gap: 10 }}>
                {(() => {
                  const m = trip.members.find((x) => x.id === live.by);
                  return <Avatar name={m?.name ?? '?'} src={m?.avatar} size={32} />;
                })()}
                <View style={{ flex: 1 }}>
```

```
                  <T variant="small" weight="semibold" color={colors.white}>
                    {live.caption || 'Untitled'}
                  </T>
                  <T variant="caption" color="rgba(255,255,255,0.7)">
                    {trip.members.find((x) => x.id === live.by)?.name.split(' ')[0]} · {live.day ? dayLabel(live.day) : ''} · {timeAgo(l
ive.ts)}
                  </T>
                </View>
                <Press onPress={() => like(trip.id, live.id)} style={styles.likeBtn} accessibilityLabel={live.likes.includes(userId) ?
'Unlike' : 'Like'}>
                  <Ionicons name={live.likes.includes(userId) ? 'heart' : 'heart-outline'} size={22} color={live.likes.includes(userId)
? '#FF6B81' : colors.white} />
                  <T variant="small" weight="semibold" color={colors.white}>
                    {live.likes.length}
                  </T>
                </Press>
              </View>
              <Button label="Save & share" variant="light" size="md" icon="download-outline" onPress={() => sharePhoto(live.uri, live.ca
ption)} />
            </View>
          </View>
        ) : null}
        <ShareSheet
          visible={!!shareFor}
          onClose={() => setShareFor(null)}
          title="Share this memory"
          message={`${shareFor?.caption ?? ''} 📸 from our trip “${trip.name}” — planned with TogetherWeGo`}
        />
      </Modal>
    </Screen>
  );
}

const styles = StyleSheet.create({
  grid: { flexDirection: 'row', flexWrap: 'wrap', gap: 4 },
  img: { width: '100%', height: '100%', borderRadius: 8 },
  likes: { position: 'absolute', bottom: 5, left: 5, flexDirection: 'row', alignItems: 'center', gap: 2, backgroundColor: 'rgba(0,0,0,0.
45)', paddingHorizontal: 5, paddingVertical: 2, borderRadius: 999 },
  viewer: { flex: 1, backgroundColor: '#0B0F0A' },
  viewerTop: { flexDirection: 'row', justifyContent: 'space-between', paddingHorizontal: 16, marginBottom: 8 },
  round: { width: 42, height: 42, borderRadius: 21, backgroundColor: 'rgba(255,255,255,0.14)', alignItems: 'center', justifyContent: 'ce
nter' },
  viewerBottom: { paddingHorizontal: 16, paddingTop: 12, gap: 12 },
  likeBtn: { flexDirection: 'row', alignItems: 'center', gap: 4, paddingHorizontal: 12, paddingVertical: 8, borderRadius: 999, backgroun
dColor: 'rgba(255,255,255,0.14)' },
});
```

src/app/phrasebook.tsx

```
import { Ionicons } from '@expo/vector-icons';
import * as Speech from 'expo-speech';
import { useState } from 'react';
import { ScrollView, StyleSheet, TextInput, View } from 'react-native';

import { Card, Chip, EmptyState } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Press } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { phrasebook } from '@/data/reference';
import { copy } from '@/services/share';
import { useActiveTrip } from '@/store/useAppStore';
import { colors, fonts, radius } from '@/theme';

/** Offline phrasebook with native text-to-speech pronunciation. */
export default function Phrasebook() {
  const trip = useActiveTrip();
  const dest = destinationById(trip?.destinationId ?? 'tokyo');
  const initial = dest && phrasebook[dest.language] ? dest.language : 'Japanese';
  const [lang, setLang] = useState(initial);
  const [q, setQ] = useState('');
  const [speaking, setSpeaking] = useState<string | null>(null);
  const book = phrasebook[lang];
  const list = book.phrases.filter((p) => !q || `${p.en} ${p.local} ${p.roman ?? ''}`.toLowerCase().includes(q.toLowerCase()));

  const say = (text: string) => {
    Speech.stop();
    setSpeaking(text);
    Speech.speak(text, {
      language: book.speechCode,
      rate: 0.85,
      onDone: () => setSpeaking(null),
      onStopped: () => setSpeaking(null),
      onError: () => {
        setSpeaking(null);
        toast(`Install the ${book.lang} voice in your phone settings to hear this`, { tone: 'warn' });
      },
    });
  };

  return (
    <Screen header={<Header title="Phrasebook" subtitle={`${book.phrases.length} ${book.lang} phrases · tap 🔊 to hear them`} />}>
      <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 6, paddingHorizontal: 20 }} style={{ m
arginHorizontal: -20 }}>
        {Object.keys(phrasebook).map((l) => (
          <Chip key={l} label={l} active={l === lang} onPress={() => setLang(l)} />
        ))}
      </ScrollView>
      <View style={styles.search}>
        <Ionicons name="search" size={17} color={colors.textMuted} />
        <TextInput value={q} onChangeText={setQ} placeholder="Search phrases…" placeholderTextColor={colors.textMuted} style={styles.inp
ut} accessibilityLabel="Search phrases" />
      </View>
      {list.length ? (
        <View style={{ gap: 10 }}>
          {list.map((p) => (
            <Card key={p.en} style={styles.row}>
              <View style={{ flex: 1, gap: 2 }}>
                <T variant="caption" color={colors.textSecondary}>
                  {p.en}
                </T>
                <T variant="h3" selectable>
                  {p.local}
                </T>
                {p.roman ? (
                  <T variant="small" color={colors.primary} style={{ fontStyle: 'italic' }}>
                    {p.roman}
                  </T>
                ) : null}
              </View>
              <Press onPress={() => copy(p.local, 'Phrase copied')} style={styles.iconBtn} accessibilityLabel={`Copy ${p.en}`}>
                <Ionicons name="copy-outline" size={17} color={colors.primary} />
              </Press>
              <Press onPress={() => say(p.local)} style={[styles.iconBtn, speaking === p.local && { backgroundColor: colors.primary }]}
accessibilityLabel={`Hear ${p.en} in ${book.lang}`}>
                <Ionicons name="volume-high-outline" size={18} color={speaking === p.local ? colors.white : colors.primary} />
              </Press>
            </Card>
          ))}
        </View>
      ) : (
        <EmptyState icon="language-outline" title="No phrases match" />
```

```
      )}
    </Screen>
  );
}

const styles = StyleSheet.create({
  search: { flexDirection: 'row', alignItems: 'center', gap: 8, height: 46, marginVertical: 14, paddingHorizontal: 14, borderRadius: rad
ius.md, borderWidth: 1, borderColor: colors.border, backgroundColor: colors.card },
  input: { flex: 1, fontFamily: fonts.regular, fontSize: 15, color: colors.text, height: '100%', ...({ outlineStyle: 'none' } as object)
},
  row: { flexDirection: 'row', alignItems: 'center', gap: 10 },
  iconBtn: { width: 40, height: 40, borderRadius: 20, backgroundColor: colors.primarySofter, alignItems: 'center', justifyContent: 'cent
er' },
});
```

src/app/quiz.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { useEffect, useMemo, useRef, useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { ShareSheet } from '@/components/ShareSheet';
import { Button } from '@/components/ui/Button';
import { Avatar, Card, ProgressBar } from '@/components/ui/bits';
import { Press, tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { photos } from '@/data/images';
import { quizQuestions, type QuizQuestion } from '@/data/reference';
import { useActiveTrip, useAppStore, useCurrentUser } from '@/store/useAppStore';
import { colors, radius } from '@/theme';

const ROUND = 8;
const SECONDS = 20;

function shuffle<T>(a: T[]): T[] {
  const x = a.slice();
  for (let i = x.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [x[i], x[j]] = [x[j], x[i]];
  }
  return x;
}

export default function Quiz() {
  const best = useAppStore((s) => s.quizBest);
  const setBest = useAppStore((s) => s.setQuizBest);
  const trip = useActiveTrip();
  const user = useCurrentUser();
  const [phase, setPhase] = useState<'intro' | 'play' | 'done'>('intro');
  const [qs, setQs] = useState<QuizQuestion[]>([]);
  const [i, setI] = useState(0);
  const [picked, setPicked] = useState<number | null>(null);
  const [score, setScore] = useState(0);
  const [streak, setStreak] = useState(0);
  const [left, setLeft] = useState(SECONDS);
  const [share, setShare] = useState(false);
  const timer = useRef<ReturnType<typeof setInterval> | null>(null);

  const start = () => {
    setQs(shuffle(quizQuestions).slice(0, ROUND));
    setI(0);
    setScore(0);
    setStreak(0);
    setPicked(null);
    setLeft(SECONDS);
    setPhase('play');
  };

  // Running out of time counts as answer -1 (derived, so no extra state sync needed).
  const answer = picked ?? (phase === 'play' && left <= 0 ? -1 : null);
  const ticking = phase === 'play' && answer === null;

  useEffect(() => {
    if (!ticking) return;
    timer.current = setInterval(() => setLeft((l) => Math.max(0, l - 1)), 1000);
    return () => {
      if (timer.current) clearInterval(timer.current);
    };
  }, [ticking, i]);

  const choose = (k: number) => {
    if (answer !== null) return;
    setPicked(k);
    const q = qs[i];
    if (k === q.answer) {
      const bonus = Math.max(0, Math.round(left / 4));
      setScore((s) => s + 10 + bonus + streak * 2);
      setStreak((s) => s + 1);
      tap('success');
    } else {
      setStreak(0);
      tap('warning');
    }
  };

  const next = () => {
    if (answer === -1) setStreak(0);
    if (i + 1 >= qs.length) {
      setPhase('done');
```

```
      setBest(score);
      return;
    }
    setI(i + 1);
    setPicked(null);
    setLeft(SECONDS);
  };

  // Friendly leaderboard: group members get stable pseudo-scores, you get your real best.
  const board = useMemo(() => {
    const others = (trip?.members ?? []).filter((m) => m.id !== user?.id).map((m) => ({ name: m.name, avatar: m.avatar, score: 60 + ((m.
name.charCodeAt(0) * 7 + m.name.length * 13) % 90) }));
    return [...others, { name: user?.name ?? 'You', avatar: user?.avatar, score: Math.max(best, phase === 'done' ? score : 0), me: true
}].sort((a, b) => b.score - a.score);
  }, [trip, user, best, score, phase]);

  if (phase === 'intro' || phase === 'done') {
    const finished = phase === 'done';
    return (
      <Screen header={<Header title="Travel quiz" subtitle="Test your globe-trotting knowledge" />}>
        <View style={styles.hero}>
          <Image source={photos['fuji-blossom']} style={StyleSheet.absoluteFill} contentFit="cover" />
          <View style={styles.heroShade} />
          <T variant="kicker" color="rgba(255,255,255,0.85)">
            {finished ? 'Round complete' : `${ROUND} questions · ${SECONDS}s each`}
          </T>
          <T variant="display" color={colors.white}>
            {finished ? `${score} pts` : 'Trip Trivia'}
          </T>
          <T variant="small" color="rgba(255,255,255,0.9)">
            {finished ? (score >= 100 ? 'Legendary navigator! 🧭' : score >= 60 ? 'Seasoned traveller ✈️' : 'Every expert was once a tou
rist 🌱') : 'Faster answers and streaks earn bonus points.'}
          </T>
        </View>
        <Button label={finished ? 'Play again' : 'Start quiz'} icon="play" onPress={start} style={{ marginTop: 16 }} />
        {finished ? <Button label="Challenge the group" icon="share-social-outline" variant="secondary" onPress={() => setShare(true)} s
tyle={{ marginTop: 10 }} /> : null}
        <Card style={{ marginTop: 16, gap: 10 }}>
          <View style={{ flexDirection: 'row', justifyContent: 'space-between' }}>
            <T variant="title">Group leaderboard</T>
            <T variant="caption" color={colors.textSecondary}>
              Your best: {best}
            </T>
          </View>
          {board.map((b, k) => (
            <View key={b.name} style={[styles.boardRow, 'me' in b && styles.me]}>
              <T variant="small" weight="bold" style={{ width: 22 }}>
                {k === 0 ? '🥇' : k === 1 ? '🥈' : k === 2 ? '🥉' : k + 1}
              </T>
              <Avatar name={b.name} src={b.avatar} size={28} />
              <T variant="small" style={{ flex: 1 }}>
                {b.name.split(' ')[0]}
                {'me' in b ? ' (you)' : ''}
              </T>
              <T variant="small" weight="semibold">
                {b.score}
              </T>
            </View>
          ))}
        </Card>
        <ShareSheet visible={share} onClose={() => setShare(false)} title="Share your score" message={`I scored ${score} points on the T
ogetherWeGo travel quiz 🧭 Think you can beat me?`} />
      </Screen>
    );
  }

  const q = qs[i];
  return (
    <Screen header={<Header title={`Question ${i + 1} of ${qs.length}`} subtitle={`${score} pts${streak > 1 ? ` · 🔥 ${streak} streak` :
''}`} />}>
      <ProgressBar value={(i + (answer !== null ? 1 : 0)) / qs.length} height={6} />
      <View style={styles.timerRow}>
        <Ionicons name="timer-outline" size={16} color={left <= 5 ? colors.red : colors.textSecondary} />
        <View style={{ flex: 1 }}>
          <ProgressBar value={left / SECONDS} color={left <= 5 ? colors.red : colors.orange} height={5} />
        </View>
        <T variant="small" weight="bold" color={left <= 5 ? colors.red : colors.text} style={{ width: 28, textAlign: 'right' }}>
          {Math.max(0, left)}s
        </T>
      </View>
      <T variant="h2" style={{ marginVertical: 16 }} accessibilityRole="header">
        {q.q}
      </T>
      <View style={{ gap: 10 }}>
        {q.options.map((o, k) => {
          const isRight = answer !== null && k === q.answer;
```

```
          const isWrong = answer === k && k !== q.answer;
          return (
            <Press
              key={o}
              onPress={() => choose(k)}
              disabled={answer !== null}
              style={[styles.option, isRight && styles.right, isWrong && styles.wrong]}
              accessibilityLabel={o}
              accessibilityState={{ selected: answer === k }}>
              <View style={[styles.letter, isRight && { backgroundColor: colors.primary }, isWrong && { backgroundColor: colors.red }]}>
                <T variant="small" weight="bold" color={isRight || isWrong ? colors.white : colors.primary}>
                  {'ABCD'[k]}
                </T>
              </View>
              <T variant="bodySm" weight="medium" style={{ flex: 1 }}>
                {o}
              </T>
              {isRight ? <Ionicons name="checkmark-circle" size={20} color={colors.primary} /> : isWrong ? <Ionicons name="close-circle"
size={20} color={colors.red} /> : null}
            </Press>
          );
        })}
      </View>
      {answer !== null ? (
        <Card style={{ marginTop: 16, gap: 10, backgroundColor: answer === q.answer ? colors.primarySofter : colors.orangeSoft }}>
          <T variant="small" weight="semibold">
            {answer === q.answer ? 'Correct! 🎉' : answer === -1 ? 'Time’s up ⏰' : 'Not quite'}
          </T>
          <T variant="small" color={colors.textSecondary}>
            {q.fact}
          </T>
          <Button label={i + 1 >= qs.length ? 'See results' : 'Next question'} iconRight="arrow-forward" onPress={next} />
        </Card>
      ) : null}
    </Screen>
  );
}

const styles = StyleSheet.create({
  hero: { height: 220, borderRadius: radius.xl, overflow: 'hidden', padding: 20, justifyContent: 'flex-end', gap: 4 },
  heroShade: { ...StyleSheet.absoluteFill, backgroundColor: 'rgba(10,25,10,0.45)' },
  boardRow: { flexDirection: 'row', alignItems: 'center', gap: 10, padding: 8, borderRadius: radius.sm },
  me: { backgroundColor: colors.primarySofter },
  timerRow: { flexDirection: 'row', alignItems: 'center', gap: 8, marginTop: 14 },
  option: { flexDirection: 'row', alignItems: 'center', gap: 12, padding: 14, borderRadius: radius.md, backgroundColor: colors.card, bor
derWidth: 1.5, borderColor: colors.border },
  right: { borderColor: colors.primary, backgroundColor: colors.primarySofter },
  wrong: { borderColor: colors.red, backgroundColor: colors.redSoft },
  letter: { width: 30, height: 30, borderRadius: 15, backgroundColor: colors.primarySoft, alignItems: 'center', justifyContent: 'center'
},
});
```

src/app/recap.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { LinearGradient } from 'expo-linear-gradient';
import { router } from 'expo-router';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { ShareSheet } from '@/components/ShareSheet';
import { NoTrip } from '@/components/trip/NoTrip';
import { Button } from '@/components/ui/Button';
import { Avatar, Card, Pill, type IconName } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Press } from '@/components/ui/Press';
import { FadeIn } from '@/components/ui/motion';
import { Header, Screen } from '@/components/ui/Screen';
import { T, type TProps } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { imageSource } from '@/data/images';
import { settlePlan, totalSpent, tripStatus, useActiveTrip } from '@/store/useAppStore';
import { colors, radius, shadow } from '@/theme';
import { money } from '@/utils/format';
import { distanceKm, kmLabel } from '@/utils/geo';
import { daysBetween, rangeLabel } from '@/utils/time';

/** End-of-trip summary: the "completion" stage of the planning → travel → memories journey. */
export default function Recap() {
  const trip = useActiveTrip();
  const [share, setShare] = useState(false);
  const [rating, setRating] = useState(0);
  if (!trip) return <NoTrip />;

  const dest = destinationById(trip.destinationId)!;
  const status = tripStatus(trip);
  const days = daysBetween(trip.startDate, trip.endDate) + 1;
  const spent = totalSpent(trip);
  const stops = trip.activities.filter((a) => a.lat && a.lng).sort((a, b) => (a.date + a.time).localeCompare(b.date + b.time));
  const km = stops.reduce((sum, a, i) => (i ? sum + distanceKm(stops[i - 1] as { lat: number; lng: number }, a as { lat: number; lng: nu
mber }) : 0), 0);
  const votes = trip.activities.reduce((n, a) => n + Object.keys(a.votes).length, 0) + trip.polls.reduce((n, p) => n + p.options.reduce
((m, o) => m + o.votes.length, 0), 0);
  const topPhoto = trip.photos.slice().sort((a, b) => b.likes.length - a.likes.length)[0];
  const topActivity = trip.activities.filter((a) => a.type !== 'transport' && a.type !== 'stay').sort((a, b) => Object.values(b.votes).f
ilter((v) => v === 1).length - Object.values(a.votes).filter((v) => v === 1).length)[0];
  const byCat: Record<string, number> = {};
  trip.expenses.filter((e) => !e.settlement).forEach((e) => (byCat[e.category] = (byCat[e.category] ?? 0) + e.amount));
  const topCat = Object.entries(byCat).sort((a, b) => b[1] - a[1])[0];
  const photographer = trip.members
    .map((m) => ({ m, n: trip.photos.filter((p) => p.by === m.id).length }))
    .sort((a, b) => b.n - a.n)[0];
  const owed = settlePlan(trip);

  const perPerson = spent / Math.max(1, trip.members.length);
  const stats: [IconName, string, string][] = [
    ['calendar-outline', `${days}`, 'days'],
    ['flag-outline', `${trip.activities.length}`, 'activities'],
    ['navigate-outline', kmLabel(km), 'between stops'],
    ['wallet-outline', money(spent), 'spent together'],
    ['images-outline', `${trip.photos.length}`, 'photos'],
    ['thumbs-up-outline', `${votes}`, 'votes cast'],
  ];
  const storyStats: [string, string][] = [
    [`${days}`, 'days'],
    [`${trip.activities.length}`, 'activities'],
    [`${trip.photos.length}`, 'photos'],
    [money(perPerson), 'spent each'],
  ];

  const summary = `Our trip “${trip.name}” 🌏 ${days} days in ${dest.city} · ${trip.activities.length} activities · ${trip.photos.lengt
h} photos · ${kmLabel(km)} explored · ${money(perPerson)} each. Planned together on TogetherWeGo ✈️`;

  return (
    <Screen header={<Header title="Trip recap" subtitle={status === 'completed' ? 'Your memories, wrapped up' : 'A preview — the full re
cap unlocks when the trip ends'} />}>
      <View style={{ alignSelf: 'center', marginBottom: 14 }}>
        <Pill label={status === 'completed' ? 'Trip complete 🎉' : status === 'active' ? 'In progress' : 'Coming up'} tone={status ===
'completed' ? 'yellow' : 'green'} />
      </View>

      <FadeIn from="scale" style={{ alignItems: 'center' }}>
        <View style={styles.story} accessible accessibilityLabel={`Story card. ${summary}`}>
          <Image source={imageSource(trip.cover)} style={StyleSheet.absoluteFill} contentFit="cover" />
          <LinearGradient colors={['rgba(0,0,0,0.35)', 'rgba(0,0,0,0.05)', 'rgba(0,0,0,0.92)']} locations={[0, 0.3, 0.82]} style={StyleS
heet.absoluteFill} />
          <View style={styles.storyTag}>
```

```
            <Fixed variant="micro" weight="bold" color={colors.onAccent} style={{ letterSpacing: 1.2 }}>
              TRIP RECAP
            </Fixed>
          </View>
          <View style={styles.storyBottom}>
            <Fixed variant="display" color={colors.white} numberOfLines={3} style={{ fontSize: 40, lineHeight: 42 }}>
              {trip.name}
            </Fixed>
            <Fixed variant="small" weight="semibold" color={colors.white} style={{ marginTop: 8 }}>
              {dest.city}, {dest.country}
            </Fixed>
            <Fixed variant="caption" color="rgba(255,255,255,0.75)">
              {rangeLabel(trip.startDate, trip.endDate)}
            </Fixed>
            <View style={styles.storyStats}>
              {storyStats.map(([value, label]) => (
                <View key={label} style={styles.storyStat}>
                  <Fixed variant="headline" color={colors.accent} numberOfLines={1} style={{ fontSize: 24, lineHeight: 28 }}>
                    {value}
                  </Fixed>
                  <Fixed variant="micro" color="rgba(255,255,255,0.8)">
                    {label}
                  </Fixed>
                </View>
              ))}
            </View>
            <View style={styles.storyBrand}>
              <View style={styles.storyLogo}>
                <Ionicons name="airplane" size={11} color={colors.onAccent} />
              </View>
              <Fixed variant="small" weight="extrabold" color={colors.white} style={{ letterSpacing: 0.3 }}>
                TogetherWeGo
              </Fixed>
            </View>
          </View>
        </View>
      </FadeIn>
      <Button label="Share story card" icon="share-social-outline" onPress={() => setShare(true)} style={{ marginTop: 14 }} />

      <View style={styles.grid}>
        {stats.map(([icon, value, label]) => (
          <View key={label} style={styles.stat}>
            <View style={styles.statIcon}>
              <Ionicons name={icon} size={15} color={colors.onAccent} />
            </View>
            <T variant="h3">{value}</T>
            <T variant="caption" color={colors.textSecondary}>
              {label}
            </T>
          </View>
        ))}
      </View>

      <Card style={[styles.card, { marginTop: 14, gap: 12 }]}>
        <T variant="headline" accessibilityRole="header">
          Highlights
        </T>
        {topActivity ? <Highlight icon="trophy-outline" title="Group favourite" body={`${topActivity.title} — ${Object.values(topActivit
y.votes).filter((v) => v === 1).length}/${trip.members.length} said “I’m in”`} /> : null}
        {topPhoto ? <Highlight icon="heart-outline" title="Most-liked photo" body={`“${topPhoto.caption || 'Untitled'}” · ${topPhoto.lik
es.length} likes`} /> : null}
        {photographer && photographer.n ? <Highlight icon="camera-outline" title="Chief photographer" body={`${photographer.m.name.split
(' ')[0]} with ${photographer.n} photos`} /> : null}
        {topCat ? <Highlight icon="pie-chart-outline" title="Biggest spend" body={`${topCat[0]} · ${money(topCat[1])} (${Math.round((top
Cat[1] / Math.max(1, spent)) * 100)}%)`} /> : null}
        <Highlight icon="people-outline" title="The crew" body={trip.members.map((m) => `${m.name.split(' ')[0]} (${m.role})`).join(',
')} />
      </Card>

      <Card style={[styles.card, { marginTop: 14, gap: 10 }]}>
        <View style={{ flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center' }}>
          <T variant="headline" accessibilityRole="header">
            Final balances
          </T>
          <Press onPress={() => router.navigate('/(tabs)/budget')} hitSlop={8}>
            <T variant="small" weight="semibold" color={colors.blue}>
              Settle up
            </T>
          </Press>
        </View>
        {owed.length ? (
          owed.slice(0, 4).map((p) => {
            const a = trip.members.find((m) => m.id === p.from);
            const b = trip.members.find((m) => m.id === p.to);
            return (
              <View key={`${p.from}${p.to}`} style={{ flexDirection: 'row', alignItems: 'center', gap: 8 }}>
```

```
                <Avatar name={a?.name ?? '?'} src={a?.avatar} size={24} />
                <T variant="small" style={{ flex: 1 }}>
                  {a?.name.split(' ')[0]} → {b?.name.split(' ')[0]}
                </T>
                <T variant="small" weight="semibold">
                  {money(p.amount)}
                </T>
              </View>
            );
          })
        ) : (
          <T variant="small" color={colors.textSecondary}>
            Everyone is square ✅
          </T>
        )}
      </Card>

      <Card style={[styles.card, { marginTop: 14, alignItems: 'center', gap: 8 }]}>
        <T variant="headline" accessibilityRole="header">
          How was the trip?
        </T>
        <View style={{ flexDirection: 'row', gap: 6 }}>
          {[1, 2, 3, 4, 5].map((n) => (
            <Press
              key={n}
              onPress={() => {
                setRating(n);
                toast(n >= 4 ? 'Saved — sounds like a great trip! 🥳' : 'Thanks for the feedback');
              }}
              accessibilityLabel={`${n} star${n > 1 ? 's' : ''}`}>
              <Ionicons name={n <= rating ? 'star' : 'star-outline'} size={32} color={colors.star} />
            </Press>
          ))}
        </View>
      </Card>

      <Button label="Open the photo dump" icon="images-outline" variant="secondary" onPress={() => router.push('/photos')} style={{ marg
inTop: 16 }} />
      <ShareSheet visible={share} onClose={() => setShare(false)} title="Share your trip recap" message={summary} preview={summary} />
    </Screen>
  );
}

/** Text for the story card: a fixed-size graphic, so it ignores the text-size setting. */
function Fixed(props: TProps) {
  return <T allowFontScaling={false} {...props} />;
}

function Highlight({ icon, title, body }: { icon: IconName; title: string; body: string }) {
  return (
    <View style={{ flexDirection: 'row', gap: 10 }}>
      <View style={styles.hlIcon}>
        <Ionicons name={icon} size={16} color={colors.onAccent} />
      </View>
      <View style={{ flex: 1 }}>
        <T variant="caption" color={colors.textSecondary}>
          {title}
        </T>
        <T variant="small" weight="semibold">
          {body}
        </T>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  card: { borderRadius: radius.xl },
  story: { width: 270, height: 480, borderRadius: 28, overflow: 'hidden', backgroundColor: '#111111', justifyContent: 'space-between', p
adding: 20 },
  storyTag: { alignSelf: 'flex-start', backgroundColor: colors.accent, borderRadius: radius.pill, paddingHorizontal: 10, paddingVertica
l: 4 },
  storyBottom: { gap: 0 },
  storyStats: { flexDirection: 'row', flexWrap: 'wrap', rowGap: 10, columnGap: 8, marginTop: 16 },
  storyStat: { width: '47%', borderRadius: 14, backgroundColor: 'rgba(255,255,255,0.14)', paddingHorizontal: 12, paddingVertical: 8 },
  storyBrand: { flexDirection: 'row', alignItems: 'center', gap: 8, marginTop: 16 },
  storyLogo: { width: 22, height: 22, borderRadius: 11, backgroundColor: colors.accent, alignItems: 'center', justifyContent: 'center'
},
  grid: { flexDirection: 'row', flexWrap: 'wrap', gap: 8, marginTop: 14 },
  stat: { width: '31.8%', backgroundColor: colors.card, borderRadius: radius.lg, padding: 12, gap: 2, borderWidth: 1, borderColor: color
s.border, alignItems: 'flex-start', ...shadow },
  statIcon: { width: 28, height: 28, borderRadius: 14, backgroundColor: colors.accent, alignItems: 'center', justifyContent: 'center', m
arginBottom: 4 },
  hlIcon: { width: 32, height: 32, borderRadius: 16, backgroundColor: colors.accent, alignItems: 'center', justifyContent: 'center' },
});
```

src/app/settings.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { LinearGradient } from 'expo-linear-gradient';
import { router } from 'expo-router';
import { useEffect, useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';

import { CurrencyPickerSheet } from '@/components/CurrencyPickerSheet';
import { Button } from '@/components/ui/Button';
import { Avatar, Card, Chip, ListRow, ToggleRow } from '@/components/ui/bits';
import { confirmAction, toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { Press, tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { currencyByCode } from '@/data/reference';
import { LANGUAGES, useT } from '@/i18n';
import { CLAUDE_MODEL } from '@/services/assistant';
import { ensurePermission, sendTestReminder } from '@/services/notifications';
import { getApiKey, looksLikeApiKey, setApiKey } from '@/services/secrets';
import { pickImage } from '@/services/files';
import { shareText } from '@/services/share';
import { useActiveTrip, useAppStore, useCurrentUser } from '@/store/useAppStore';
import type { TextScale } from '@/store/types';
import { appearance, colors, cornerStyles, radius, themes } from '@/theme';
import { maxLen, validateName } from '@/utils/validation';

const AVATARS = ['avatar-maya', 'avatar-diego', 'avatar-priya', 'avatar-sam', 'avatar-ana', 'avatar-noah'];

export default function Settings() {
  const t = useT();
  const user = useCurrentUser();
  const trip = useActiveTrip();
  const settings = useAppStore((s) => s.settings);
  const update = useAppStore((s) => s.updateSettings);
  const updateProfile = useAppStore((s) => s.updateProfile);
  const reseed = useAppStore((s) => s.reseedDemo);
  const timeline = useAppStore((s) => s.demoTimeline);
  const signOut = useAppStore((s) => s.signOut);
  const resetEverything = useAppStore((s) => s.resetEverything);
  const ensureSeed = useAppStore((s) => s.ensureSeed);
  const personal = useAppStore((s) => s.personal);
  const updatePersonal = useAppStore((s) => s.updatePersonal);

  const [name, setName] = useState(user?.name ?? '');
  const [city, setCity] = useState(user?.homeCity ?? '');
  const [errors, setErrors] = useState<Record<string, string | null>>({});
  const [curOpen, setCurOpen] = useState(false);
  const [key, setKey] = useState('');
  const [hasKey, setHasKey] = useState(false);
  const [keyErr, setKeyErr] = useState<string | null>(null);

  useEffect(() => {
    getApiKey().then((k) => setHasKey(!!k));
  }, []);

  const saveProfile = () => {
    const e = { name: validateName(name), city: maxLen(40, 'City')(city) };
    setErrors(e);
    if (e.name || e.city) return tap('warning');
    updateProfile({ name: name.trim(), homeCity: city.trim() || undefined });
    toast('Profile saved');
  };

  const pickAvatarPhoto = async () => {
    try {
      const f = await pickImage();
      if (f) {
        updateProfile({ avatar: f.uri });
        toast('Profile photo updated');
      }
    } catch {
      toast('Couldn’t open your photos', { tone: 'warn' });
    }
  };

  const saveKey = async () => {
    if (!looksLikeApiKey(key)) return setKeyErr('Anthropic keys start with “sk-ant-”. Check you copied the whole key.');
    await setApiKey(key.trim());
    setKey('');
    setKeyErr(null);
    setHasKey(true);
    update({ aiMode: 'claude' });
    toast('Key saved securely — Claude mode on');
  };
```

```
  const scales: { v: TextScale; l: string }[] = [
    { v: 1, l: 'Default' },
    { v: 1.12, l: 'Large' },
    { v: 1.25, l: 'Extra large' },
  ];

  return (
    <Screen header={<Header title={t('settings.title')} subtitle="Make TogetherWeGo yours" />}>
      {/* Make it yours */}
      <Press onPress={() => router.push('/personalize')} style={styles.yours} accessibilityLabel="Make it yours — change theme, interest
s and home screen">
        <LinearGradient colors={[colors.heroA, colors.heroB, colors.heroC]} start={{ x: 0, y: 0 }} end={{ x: 1, y: 1 }} style={StyleShee
t.absoluteFill} />
        <View style={styles.yoursEmoji}>
          <T style={{ fontSize: 28, lineHeight: 34 }}>{personal.emoji}</T>
        </View>
        <View style={{ flex: 1 }}>
          <T variant="title" weight="bold" color={colors.white}>
            Make it yours 🎨
          </T>
          <T variant="caption" color="rgba(255,255,255,0.9)">
            {themes[appearance.theme].emoji} {themes[appearance.theme].name} theme · {cornerStyles[appearance.corners].name} corners ·
{personal.interests.length} interests
          </T>
        </View>
        <Ionicons name="chevron-forward" size={20} color={colors.white} />
      </Press>
      <Card style={{ marginBottom: 14 }}>
        <ToggleRow icon="sparkles-outline" label="Animations" sub="Flying plane, floating cards and smooth entrances" value={personal.mo
tion} onChange={(v) => updatePersonal({ motion: v })} />
      </Card>

      {/* Profile */}
      <Card style={{ gap: 12 }}>
        <T variant="title">Profile</T>
        <View style={{ flexDirection: 'row', alignItems: 'center', gap: 12 }}>
          <Avatar name={user?.name ?? 'You'} src={user?.avatar} size={60} />
          <View style={{ flex: 1, gap: 6 }}>
            <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 6 }}>
              {AVATARS.map((a) => (
                <Press key={a} onPress={() => updateProfile({ avatar: a })} accessibilityLabel="Choose this avatar" style={[styles.avata
rOpt, user?.avatar === a && { borderColor: colors.primary }]}>
                  <Avatar name="Avatar" src={a} size={36} />
                </Press>
              ))}
            </ScrollView>
            <View style={{ flexDirection: 'row', gap: 8 }}>
              <Chip small icon="image-outline" label="Upload photo" onPress={pickAvatarPhoto} />
              <Chip small icon="text-outline" label="Initials" onPress={() => updateProfile({ avatar: undefined })} />
            </View>
          </View>
        </View>
        <Input label="Name" value={name} onChangeText={setName} error={errors.name} icon="person-outline" />
        <Input label="Home city" value={city} onChangeText={setCity} error={errors.city} icon="home-outline" placeholder="e.g. Middletow
n, DE" />
        <T variant="caption" color={colors.textSecondary}>
          Signed in as {user?.email} · {user?.provider === 'google' ? 'Google' : 'email & password'}
        </T>
        <Button label="Save profile" size="md" onPress={saveProfile} />
      </Card>

      {/* Language */}
      <Card style={{ gap: 10, marginTop: 14 }}>
        <T variant="title">{t('settings.language')}</T>
        <View style={styles.wrap}>
          {LANGUAGES.map((l) => (
            <Chip key={l.code} label={`${l.flag} ${l.native}`} active={settings.language === l.code} onPress={() => update({ language:
l.code })} />
          ))}
        </View>
      </Card>

      {/* Currency & time */}
      <Card style={{ marginTop: 14 }}>
        <ListRow icon="cash-outline" label={t('settings.currency')} sub={`${settings.homeCurrency} · ${currencyByCode(settings.homeCurre
ncy)?.name ?? ''}`} onPress={() => setCurOpen(true)} />
        <ToggleRow icon="globe-outline" label="Show itinerary times in trip time" sub="Off = convert to your phone’s time zone" value={s
ettings.showTripTime} onChange={(v) => update({ showTripTime: v })} />
      </Card>

      {/* Notifications */}
      <Card style={{ marginTop: 14, gap: 6 }}>
        <ToggleRow
          icon="notifications-outline"
          label={t('settings.notifications')}
```

```
          sub="Live alerts before activities with the 🔔 on"
          value={settings.notificationsEnabled}
          onChange={async (v) => {
            update({ notificationsEnabled: v });
            if (v) {
              const ok = await ensurePermission();
              toast(ok ? 'Reminders on' : 'Allow notifications in your phone settings', { tone: ok ? 'ok' : 'warn' });
            }
          }}
        />
        <T variant="small" weight="semibold">
          {t('settings.lead')}
        </T>
        <View style={styles.wrap}>
          {[10, 30, 60, 120].map((m) => (
            <Chip key={m} small label={m < 60 ? `${m} min` : `${m / 60} hr`} active={settings.reminderLeadMin === m} onPress={() => upda
te({ reminderLeadMin: m })} />
          ))}
        </View>
        <Button label="Send a test reminder" variant="soft" size="md" icon="alarm-outline" onPress={() => { sendTestReminder(5); toast
('Arriving in 5 seconds…', { icon: 'alarm-outline' }); }} style={{ marginTop: 6 }} />
      </Card>

      {/* Accessibility */}
      <Card style={{ marginTop: 14, gap: 8 }}>
        <T variant="title">Accessibility</T>
        <T variant="small" weight="semibold">
          {t('settings.textSize')}
        </T>
        <View style={styles.wrap}>
          {scales.map((s) => (
            <Chip key={s.v} label={s.l} active={settings.textScale === s.v} onPress={() => update({ textScale: s.v })} />
          ))}
        </View>
        <ToggleRow icon="phone-portrait-outline" label="Haptic feedback" value={settings.haptics} onChange={(v) => update({ haptics: v
})} />
        <T variant="caption" color={colors.textSecondary}>
          Every button has a screen-reader label, colours meet WCAG AA contrast, and the app follows your phone’s font size too.
        </T>
      </Card>

      {/* AI */}
      <Card style={{ marginTop: 14, gap: 10 }}>
        <View style={{ flexDirection: 'row', alignItems: 'center', gap: 8 }}>
          <Ionicons name="sparkles" size={18} color={colors.primary} />
          <T variant="title">Trip Assistant</T>
        </View>
        <View style={styles.wrap}>
          <Chip icon="cloud-offline-outline" label="Offline (built-in)" active={settings.aiMode === 'offline'} onPress={() => update({ a
iMode: 'offline' })} />
          <Chip
            icon="sparkles-outline"
            label="Claude AI"
            active={settings.aiMode === 'claude'}
            onPress={() => {
              if (!hasKey) return toast('Add your Anthropic API key first', { tone: 'warn' });
              update({ aiMode: 'claude' });
            }}
          />
        </View>
        <T variant="caption" color={colors.textSecondary}>
          Offline mode answers from your trip data with no internet. Claude mode sends questions plus trip details to Anthropic’s {CLAUD
E_MODEL} model using your own key, which is stored in the phone’s secure keychain.
        </T>
        {hasKey ? (
          <View style={styles.keyRow}>
            <Ionicons name="key" size={16} color={colors.primary} />
            <T variant="small" style={{ flex: 1 }}>
              API key saved securely
            </T>
            <Press
              onPress={async () => {
                await setApiKey(null);
                setHasKey(false);
                update({ aiMode: 'offline' });
                toast('Key removed');
              }}
              hitSlop={8}>
              <T variant="small" weight="semibold" color={colors.red}>
                Remove
              </T>
            </Press>
          </View>
        ) : (
          <>
            <Input placeholder="sk-ant-…" value={key} onChangeText={(v) => { setKey(v); setKeyErr(null); }} secureToggle autoCapitalize
```

```
="none" autoCorrect={false} error={keyErr} icon="key-outline" />
            <Button label="Save key" size="md" variant="secondary" onPress={saveKey} disabled={!key} />
          </>
        )}
      </Card>

      {/* Demo */}
      <Card style={{ marginTop: 14, gap: 10 }}>
        <T variant="title">Demo timeline</T>
        <T variant="caption" color={colors.textSecondary}>
          Re-create the sample Amalfi Squad trip as upcoming, in progress or finished to show every stage from planning through completi
on.
        </T>
        <View style={styles.wrap}>
          {(
            [
              ['upcoming', 'Starts in 3 days'],
              ['active', 'Day 2 in progress'],
              ['completed', 'Just finished'],
            ] as const
          ).map(([k, l]) => (
            <Chip
              key={k}
              small
              label={l}
              active={timeline === k}
              onPress={() =>
                confirmAction('Reset the demo trip?', `“Amalfi Squad” will be rebuilt as “${l}”. Your other trips are untouched.`, () =>
{
                  reseed(k);
                  toast('Demo trip updated');
                })
              }
            />
          ))}
        </View>
      </Card>

      <Card style={{ marginTop: 14 }}>
        <ListRow icon="help-circle-outline" label="How to use TogetherWeGo" onPress={() => router.push('/help')} />
        <ListRow icon="information-circle-outline" label="About, credits & sources" onPress={() => router.push('/about')} />
        {trip ? (
          <ListRow
            icon="download-outline"
            label="Export this trip"
            sub="Share a JSON backup of the itinerary, budget and checklist"
            onPress={() => {
              const { messages, photos, docs, ...rest } = trip;
              void messages;
              void photos;
              void docs;
              shareText(JSON.stringify(rest, null, 1), `${trip.name} backup`);
            }}
          />
        ) : null}
        <ListRow
          icon="refresh-circle-outline"
          tint={colors.red}
          label="Reset app data"
          sub="Deletes all trips and accounts on this device"
          onPress={() =>
            confirmAction('Reset everything?', 'All trips, chats and accounts on this device will be erased and the demo data restore
d.', async () => {
              resetEverything();
              await ensureSeed();
              router.replace('/sign-in');
            }, { destructive: true, confirmLabel: 'Reset' })
          }
        />
      </Card>

      <Button
        label={t('settings.signOut')}
        variant="danger"
        icon="log-out-outline"
        style={{ marginTop: 16 }}
        onPress={() =>
          confirmAction('Sign out?', 'Your trips stay saved on this device.', () => {
            signOut();
            router.replace('/sign-in');
          }, { confirmLabel: 'Sign out' })
        }
      />
      <T variant="caption" color={colors.textMuted} center style={{ marginTop: 14 }}>
        TogetherWeGo v1.0 · Made for FBLA Mobile Application Development 2026–27
      </T>
```

```
      <CurrencyPickerSheet visible={curOpen} onClose={() => setCurOpen(false)} value={settings.homeCurrency} onPick={(c) => update({ hom
eCurrency: c })} title="Home currency" />
    </Screen>
  );
}

const styles = StyleSheet.create({
  yours: { flexDirection: 'row', alignItems: 'center', gap: 12, padding: 14, borderRadius: radius.lg, overflow: 'hidden', marginBottom:
14 },
  yoursEmoji: { width: 50, height: 50, borderRadius: 25, backgroundColor: 'rgba(255,255,255,0.25)', alignItems: 'center', justifyConten
t: 'center' },
  wrap: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
  avatarOpt: { borderRadius: 22, borderWidth: 2, borderColor: 'transparent', padding: 1 },
  keyRow: { flexDirection: 'row', alignItems: 'center', gap: 8, padding: 12, borderRadius: radius.md, backgroundColor: colors.primarySof
ter },
});
```

src/app/sign-in.tsx

```
import { Ionicons, MaterialCommunityIcons } from '@expo/vector-icons';
import { Link, router } from 'expo-router';
import { useRef, useState } from 'react';
import { StyleSheet, View, type TextInput } from 'react-native';

import { Logo } from '@/components/Logo';
import { Button } from '@/components/ui/Button';
import { Divider } from '@/components/ui/bits';
import { showDialog, toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { Float } from '@/components/ui/motion';
import { Press, tap } from '@/components/ui/Press';
import { Screen } from '@/components/ui/Screen';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import { useT } from '@/i18n';
import { copy } from '@/services/share';
import { DEMO_EMAIL, DEMO_PASSWORD } from '@/store/seed';
import { useAppStore } from '@/store/useAppStore';
import { colors, radius } from '@/theme';
import { validateEmail, validateName } from '@/utils/validation';

export default function SignIn() {
  const t = useT();
  const signIn = useAppStore((s) => s.signIn);
  const signInWithGoogle = useAppStore((s) => s.signInWithGoogle);
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [remember, setRemember] = useState(true);
  const [errors, setErrors] = useState<{ email?: string | null; password?: string | null; form?: string }>({});
  const [loading, setLoading] = useState(false);
  const [googleOpen, setGoogleOpen] = useState(false);
  const pwRef = useRef<TextInput>(null);

  const submit = async () => {
    const e = validateEmail(email);
    const p = password ? null : 'Enter your password';
    setErrors({ email: e, password: p });
    if (e || p) return tap('warning');
    setLoading(true);
    const res = await signIn(email, password, remember);
    setLoading(false);
    if (!res.ok) {
      setErrors({ form: res.error });
      tap('warning');
      return;
    }
    tap('success');
    router.replace('/(tabs)/explore');
  };

  const fillDemo = () => {
    setEmail(DEMO_EMAIL);
    setPassword(DEMO_PASSWORD);
    setErrors({});
    toast('Demo account filled in — tap Sign In');
  };

  return (
    <Screen contentStyle={{ paddingTop: 28 }}>
      <Float amount={6} duration={2400}>
        <Logo />
      </Float>
      <T variant="display" style={{ marginTop: 28 }} accessibilityRole="header">
        {t('auth.welcome')}
      </T>
      <T variant="bodySm" color={colors.textSecondary} style={{ marginTop: 4, marginBottom: 22 }}>
        {t('auth.welcomeSub')}
      </T>

      <View style={{ gap: 12 }}>
        <Input
          inlineLabel
          label={t('auth.email')}
          icon="mail-outline"
          value={email}
          onChangeText={(v) => {
            setEmail(v);
            if (errors.email || errors.form) setErrors({});
          }}
          autoCapitalize="none"
          autoComplete="email"
          keyboardType="email-address"
          textContentType="emailAddress"
```

```
          returnKeyType="next"
          onSubmitEditing={() => pwRef.current?.focus()}
          error={errors.email}
          testID="email"
        />
        <Input
          ref={pwRef}
          inlineLabel
          label={t('auth.password')}
          icon="lock-closed-outline"
          value={password}
          onChangeText={(v) => {
            setPassword(v);
            if (errors.password || errors.form) setErrors({});
          }}
          secureToggle
          autoComplete="password"
          textContentType="password"
          returnKeyType="go"
          onSubmitEditing={submit}
          error={errors.password}
          testID="password"
        />
      </View>

      <View style={styles.row}>
        <Press onPress={() => setRemember((r) => !r)} style={styles.remember} accessibilityRole="checkbox" accessibilityState={{ checke
d: remember }} accessibilityLabel={t('auth.remember')}>
          <View style={[styles.box, remember && styles.boxOn]}>{remember ? <Ionicons name="checkmark" size={13} color={colors.white} />
: null}</View>
          <T variant="small" color={colors.textSecondary}>
            {t('auth.remember')}
          </T>
        </Press>
        <Link href="/forgot-password" asChild>
          <Press accessibilityRole="link">
            <T variant="small" weight="semibold" color={colors.primary}>
              {t('auth.forgot')}
            </T>
          </Press>
        </Link>
      </View>

      {errors.form ? (
        <View style={styles.formError} accessibilityLiveRegion="assertive">
          <Ionicons name="alert-circle" size={18} color={colors.red} />
          <T variant="small" color={colors.red} style={{ flex: 1 }}>
            {errors.form}
          </T>
        </View>
      ) : null}

      <Button label={t('auth.signIn')} onPress={submit} loading={loading} testID="sign-in" />

      <View style={styles.orRow}>
        <Divider style={{ flex: 1 }} />
        <T variant="caption" color={colors.textMuted}>
          {t('auth.or')}
        </T>
        <Divider style={{ flex: 1 }} />
      </View>

      <Press onPress={() => setGoogleOpen(true)} style={styles.google} accessibilityLabel={t('auth.google')}>
        <MaterialCommunityIcons name="google" size={18} color={colors.text} />
        <T variant="title" weight="medium">
          {t('auth.google')}
        </T>
      </Press>

      <View style={styles.signupRow}>
        <T variant="small" color={colors.textSecondary}>
          {t('auth.noAccount')}{' '}
        </T>
        <Link href="/sign-up" asChild>
          <Press accessibilityRole="link">
            <T variant="small" weight="bold">
              {t('auth.signUp')}
            </T>
          </Press>
        </Link>
      </View>

      <Press onPress={fillDemo} style={styles.demo} scaleTo={0.99} accessibilityLabel="Use the demo account">
        <View style={{ flexDirection: 'row', alignItems: 'center', gap: 6, marginBottom: 10 }}>
          <Ionicons name="information-circle-outline" size={16} color={colors.primary} />
          <T variant="small" weight="semibold" color={colors.primary}>
```

```
            {t('auth.demo')}
          </T>
          <T variant="caption" color={colors.textMuted} style={{ marginLeft: 'auto' }}>
            Tap to fill
          </T>
        </View>
        {[
          ['Email', DEMO_EMAIL],
          ['Password', DEMO_PASSWORD],
        ].map(([k, v]) => (
          <View key={k} style={styles.demoRow}>
            <T variant="caption" color={colors.textSecondary} style={{ width: 62 }}>
              {k}
            </T>
            <T variant="caption" style={{ flex: 1 }} selectable>
              {v}
            </T>
            <Press onPress={() => copy(v, `${k} copied`)} accessibilityLabel={`Copy demo ${k.toLowerCase()}`} style={styles.copyBtn} hit
Slop={6}>
              <Ionicons name="copy-outline" size={13} color={colors.primary} />
            </Press>
          </View>
        ))}
      </Press>

      <T variant="caption" color={colors.textMuted} center style={{ marginTop: 18 }}>
        By continuing, you agree to our{' '}
        <T variant="caption" weight="semibold" color={colors.textSecondary} onPress={() => legal('Terms of Service')}>
          Terms of Service
        </T>{' '}
        and{' '}
        <T variant="caption" weight="semibold" color={colors.textSecondary} onPress={() => legal('Privacy Policy')}>
          Privacy Policy
        </T>
      </T>

      <GoogleSheet
        visible={googleOpen}
        onClose={() => setGoogleOpen(false)}
        onPick={async (mail, name) => {
          setGoogleOpen(false);
          await signInWithGoogle(mail, name);
          tap('success');
          router.replace('/(tabs)/explore');
        }}
      />
    </Screen>
  );
}

function legal(title: string) {
  showDialog(
    title,
    title === 'Privacy Policy'
      ? 'TogetherWeGo stores your trips on this device. Passwords are salted and hashed (SHA-256); API keys go in the secure keychain. W
e never sell data. Location is only used when you tap a nearby/airport feature.'
      : 'TogetherWeGo is a student project built for FBLA Mobile Application Development. Use it to plan trips with friends — be kind in
chats and only upload documents you own.',
    [{ label: 'Got it' }],
    'shield-checkmark-outline',
  );
}

/** Simulated Google account chooser — a production build would use OAuth via expo-auth-session. */
function GoogleSheet({ visible, onClose, onPick }: { visible: boolean; onClose: () => void; onPick: (email: string, name: string) => voi
d }) {
  const [other, setOther] = useState(false);
  const [mail, setMail] = useState('');
  const [name, setName] = useState('');
  const [errs, setErrs] = useState<{ mail?: string | null; name?: string | null }>({});
  return (
    <Sheet visible={visible} onClose={onClose} title="Choose an account" subtitle="to continue to TogetherWeGo">
      <Press onPress={() => onPick(DEMO_EMAIL, 'Maya Chen')} style={styles.gAcct}>
        <View style={[styles.gAvatar, { backgroundColor: colors.primary }]}>
          <T variant="title" color={colors.white}>
            M
          </T>
        </View>
        <View style={{ flex: 1 }}>
          <T variant="bodySm" weight="semibold">
            Maya Chen
          </T>
          <T variant="caption" color={colors.textSecondary}>
            {DEMO_EMAIL}
          </T>
        </View>
```

```
        <Ionicons name="chevron-forward" size={16} color={colors.textMuted} />
      </Press>
      {other ? (
        <View style={{ gap: 12 }}>
          <Input label="Name" value={name} onChangeText={setName} error={errs.name} autoCapitalize="words" icon="person-outline" />
          <Input label="Google email" value={mail} onChangeText={setMail} error={errs.mail} autoCapitalize="none" keyboardType="email-ad
dress" icon="mail-outline" />
          <Button
            label="Continue"
            onPress={() => {
              const e = { mail: validateEmail(mail), name: validateName(name) };
              setErrs(e);
              if (!e.mail && !e.name) onPick(mail, name.trim());
            }}
          />
        </View>
      ) : (
        <Press onPress={() => setOther(true)} style={styles.gAcct}>
          <View style={[styles.gAvatar, { backgroundColor: colors.bgAlt }]}>
            <Ionicons name="person-add-outline" size={18} color={colors.textSecondary} />
          </View>
          <T variant="bodySm" weight="medium">
            Use another account
          </T>
        </Press>
      )}
      <T variant="caption" color={colors.textMuted}>
        Demo sign-in: in a store build this opens Google’s secure OAuth screen.
      </T>
    </Sheet>
  );
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', marginVertical: 16 },
  remember: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  box: { width: 18, height: 18, borderRadius: 5, borderWidth: 1.5, borderColor: colors.borderStrong, alignItems: 'center', justifyConten
t: 'center' },
  boxOn: { backgroundColor: colors.primary, borderColor: colors.primary },
  formError: { flexDirection: 'row', gap: 8, alignItems: 'center', backgroundColor: colors.redSoft, padding: 12, borderRadius: radius.s
m, marginBottom: 12 },
  orRow: { flexDirection: 'row', alignItems: 'center', gap: 10, marginVertical: 18 },
  google: {
    height: 52,
    borderRadius: radius.md,
    borderWidth: 1,
    borderColor: colors.border,
    backgroundColor: colors.card,
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'center',
    gap: 10,
  },
  signupRow: { flexDirection: 'row', justifyContent: 'center', alignItems: 'center', marginTop: 20 },
  demo: { marginTop: 22, padding: 14, borderRadius: radius.md, backgroundColor: colors.card, borderWidth: 1, borderColor: colors.border
},
  demoRow: { flexDirection: 'row', alignItems: 'center', paddingVertical: 4 },
  copyBtn: { width: 26, height: 26, borderRadius: 8, backgroundColor: colors.primarySofter, alignItems: 'center', justifyContent: 'cente
r' },
  gAcct: { flexDirection: 'row', alignItems: 'center', gap: 12, padding: 12, borderRadius: radius.md, borderWidth: 1, borderColor: color
s.border },
  gAvatar: { width: 40, height: 40, borderRadius: 20, alignItems: 'center', justifyContent: 'center' },
});
```

src/app/sign-up.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { router } from 'expo-router';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { Input } from '@/components/ui/Input';
import { Press, tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { useAppStore } from '@/store/useAppStore';
import { colors, radius } from '@/theme';
import { passwordStrength, validateEmail, validateName, validatePassword } from '@/utils/validation';

export default function SignUp() {
  const signUp = useAppStore((s) => s.signUp);
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [confirm, setConfirm] = useState('');
  const [agree, setAgree] = useState(false);
  const [errors, setErrors] = useState<Record<string, string | null | undefined>>({});
  const [loading, setLoading] = useState(false);
  const strength = passwordStrength(password);

  const submit = async () => {
    const e = {
      name: validateName(name),
      email: validateEmail(email),
      password: validatePassword(password),
      confirm: !confirm ? 'Re-enter your password' : confirm !== password ? 'Passwords don’t match' : null,
      agree: agree ? null : 'Please accept the terms to continue',
    };
    setErrors(e);
    if (Object.values(e).some(Boolean)) return tap('warning');
    setLoading(true);
    const res = await signUp(name, email, password);
    setLoading(false);
    if (!res.ok) {
      setErrors({ email: res.error });
      return tap('warning');
    }
    tap('success');
    router.replace('/(tabs)/explore');
    // Brand-new account: offer the personalization survey straight away.
    setTimeout(() => router.push('/personalize'), 350);
  };

  const bars = [colors.red, colors.orange, colors.yellow, '#6BAA75', colors.primary];

  return (
    <Screen header={<Header title="Create account" subtitle="Start planning trips with your crew" />}>
      <View style={{ gap: 14 }}>
        <Input label="Full name" icon="person-outline" value={name} onChangeText={setName} autoCapitalize="words" autoComplete="name" er
ror={errors.name} placeholder="e.g. Alex Rivera" />
        <Input label="Email address" icon="mail-outline" value={email} onChangeText={setEmail} autoCapitalize="none" keyboardType="email
-address" autoComplete="email" error={errors.email} placeholder="you@email.com" />
        <View style={{ gap: 8 }}>
          <Input label="Password" icon="lock-closed-outline" value={password} onChangeText={setPassword} secureToggle error={errors.pass
word} placeholder="At least 8 characters" textContentType="newPassword" />
          {password ? (
            <View style={{ gap: 6 }}>
              <View style={{ flexDirection: 'row', gap: 4 }}>
                {[0, 1, 2, 3].map((i) => (
                  <View key={i} style={[styles.bar, { backgroundColor: i < strength.score ? bars[strength.score] : colors.border }]} />
                ))}
              </View>
              <T variant="caption" color={colors.textSecondary}>
                Strength: {strength.label}
              </T>
            </View>
          ) : null}
        </View>
        <Input label="Confirm password" icon="lock-closed-outline" value={confirm} onChangeText={setConfirm} secureToggle error={errors.
confirm} placeholder="Re-enter password" />
        <Press onPress={() => setAgree((a) => !a)} style={styles.agree} accessibilityRole="checkbox" accessibilityState={{ checked: agre
e }}>
          <View style={[styles.box, agree && styles.boxOn]}>{agree ? <Ionicons name="checkmark" size={13} color={colors.white} /> : nul
l}</View>
          <T variant="small" color={colors.textSecondary} style={{ flex: 1 }}>
            I agree to the Terms of Service and Privacy Policy
          </T>
        </Press>
        {errors.agree ? (
```

```
          <T variant="caption" color={colors.red}>
            {errors.agree}
          </T>
        ) : null}
        <Button label="Create account" onPress={submit} loading={loading} />
        <View style={styles.note}>
          <Ionicons name="shield-checkmark-outline" size={16} color={colors.primary} />
          <T variant="caption" color={colors.textSecondary} style={{ flex: 1 }}>
            Your password is salted and hashed on-device — it’s never stored in plain text.
          </T>
        </View>
      </View>
    </Screen>
  );
}

const styles = StyleSheet.create({
  bar: { flex: 1, height: 5, borderRadius: 3 },
  agree: { flexDirection: 'row', alignItems: 'center', gap: 10 },
  box: { width: 20, height: 20, borderRadius: 6, borderWidth: 1.5, borderColor: colors.borderStrong, alignItems: 'center', justifyConten
t: 'center' },
  boxOn: { backgroundColor: colors.primary, borderColor: colors.primary },
  note: { flexDirection: 'row', gap: 8, alignItems: 'center', padding: 12, borderRadius: radius.sm, backgroundColor: colors.primarySofte
r },
});
```

src/app/survey.tsx

```
import { Image } from 'expo-image';
import { LinearGradient } from 'expo-linear-gradient';
import { router } from 'expo-router';
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { AddToTripSheet } from '@/components/explore';
import { Button } from '@/components/ui/Button';
import { Pill, ProgressBar } from '@/components/ui/bits';
import { Press, tap } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { destinationById, destinations } from '@/data/destinations';
import { imageSource } from '@/data/images';
import { surveyQuestions } from '@/data/reference';
import { useAppStore } from '@/store/useAppStore';
import { colors, radius } from '@/theme';
import { money } from '@/utils/format';

/** Scores every destination against the answers: tag matches + budget fit + rating. */
function recommend(answers: string[][]): { id: string; match: number }[] {
  const tags = answers.flat();
  const budgetTag = tags.find((t) => t.startsWith('$'));
  const budget = budgetTag ? Number(budgetTag.slice(1)) : 2;
  const scored = destinations.map((d) => {
    let s = 0;
    for (const t of tags) if (!t.startsWith('$') && d.tags.includes(t as never)) s += 2;
    s -= Math.abs(d.priceLevel - budget) * 1.5;
    s += (d.rating - 4.5) * 4;
    return { id: d.id, s };
  });
  const max = Math.max(...scored.map((x) => x.s));
  const min = Math.min(...scored.map((x) => x.s));
  return scored
    .sort((a, b) => b.s - a.s)
    .slice(0, 5)
    .map((x) => ({ id: x.id, match: Math.round(70 + ((x.s - min) / Math.max(1, max - min)) * 29) }));
}

export default function Survey() {
  const saved = useAppStore((s) => s.surveyResult);
  const save = useAppStore((s) => s.setSurveyResult);
  const [step, setStep] = useState(saved ? surveyQuestions.length : 0);
  const [answers, setAnswers] = useState<string[][]>([]);
  const [results, setResults] = useState<{ id: string; match: number }[]>(saved ? saved.map((id, i) => ({ id, match: 98 - i * 4 })) :
[]);
  const [adding, setAdding] = useState<string | null>(null);

  const pick = (tags: string[]) => {
    const next = [...answers.slice(0, step), tags];
    setAnswers(next);
    tap();
    if (step + 1 >= surveyQuestions.length) {
      const r = recommend(next);
      setResults(r);
      save(r.map((x) => x.id));
    }
    setStep(step + 1);
  };

  if (step >= surveyQuestions.length) {
    return (
      <Screen header={<Header title="Your matches" subtitle="Based on your travel style" />}>
        <T variant="small" color={colors.textSecondary} style={{ marginBottom: 14 }}>
          We matched your answers against 32 destinations — tap one to explore it, or add it to a trip.
        </T>
        <View style={{ gap: 14 }}>
          {results.map((r, i) => {
            const d = destinationById(r.id)!;
            return (
              <Press key={r.id} onPress={() => router.push({ pathname: '/destination/[id]', params: { id: d.id } })} style={[styles.resu
lt, i === 0 && { height: 300 }]} scaleTo={0.985}>
                <Image source={imageSource(d.image)} style={StyleSheet.absoluteFill} contentFit="cover" transition={250} />
                <LinearGradient colors={['rgba(0,0,0,0)', 'rgba(0,0,0,0.72)']} locations={[0.35, 1]} style={StyleSheet.absoluteFill} />
                <View style={styles.match}>
                  <T variant="micro" weight="bold" color={colors.primary}>
                    {r.match}% match
                  </T>
                </View>
                <View style={styles.resultText}>
                  {i === 0 ? (
                    <T variant="kicker" color="rgba(255,255,255,0.85)">
                      Top pick for you
                    </T>
```

```
                ) : null}
                <T variant={i === 0 ? 'display' : 'h2'} color={colors.white}>
                  {d.city}
                </T>
                <T variant="small" color="rgba(255,255,255,0.9)" numberOfLines={2}>
                  {d.blurb}
                </T>
                <View style={{ flexDirection: 'row', gap: 6, marginTop: 6, alignItems: 'center' }}>
                  <Pill label={`~${money(d.dailyCost)}/day`} tone="yellow" />
                  <Pill label={`★ ${d.rating}`} />
                  <View style={{ flex: 1 }} />
                  <Button label="Add" size="sm" full={false} variant="light" icon="add" onPress={() => setAdding(d.id)} />
                </View>
              </View>
            </Press>
          );
        })}
      </View>
      <Button
        label="Retake survey"
        variant="secondary"
        icon="refresh-outline"
        style={{ marginTop: 16 }}
        onPress={() => {
          setAnswers([]);
          setStep(0);
        }}
      />
      <AddToTripSheet destinationId={adding} onClose={() => setAdding(null)} />
    </Screen>
  );
}

const q = surveyQuestions[step];
return (
  <Screen header={<Header title="Find your next trip" subtitle={`Question ${step + 1} of ${surveyQuestions.length}`} />}>
    <ProgressBar value={step / surveyQuestions.length} height={6} />
    <T variant="h1" style={{ marginTop: 22, marginBottom: 18 }} accessibilityRole="header">
      {q.q}
    </T>
    <View style={styles.grid}>
      {q.options.map((o) => (
        <Press key={o.label} onPress={() => pick(o.tags)} style={styles.option} accessibilityLabel={o.label}>
          <T style={{ fontSize: 40, lineHeight: 48 }}>{o.emoji}</T>
          <T variant="small" weight="semibold" center>
            {o.label}
          </T>
        </Press>
      ))}
    </View>
    {step > 0 ? <Button label="Back" variant="ghost" icon="arrow-back" onPress={() => setStep(step - 1)} style={{ marginTop: 16 }} />
: null}
  </Screen>
);
}

const styles = StyleSheet.create({
  grid: { flexDirection: 'row', flexWrap: 'wrap', gap: 12 },
  option: { width: '47.5%', aspectRatio: 1.05, borderRadius: radius.xl, backgroundColor: colors.card, borderWidth: 1, borderColor: color
s.border, alignItems: 'center', justifyContent: 'center', gap: 8, padding: 12 },
  result: { height: 220, borderRadius: radius.xl, overflow: 'hidden' },
  match: { position: 'absolute', top: 12, right: 12, backgroundColor: colors.white, paddingHorizontal: 10, paddingVertical: 5, borderRad
ius: 999 },
  resultText: { position: 'absolute', left: 16, right: 16, bottom: 14 },
});
```

src/app/trips.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { router } from 'expo-router';
import { StyleSheet, View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { AvatarStack, EmptyState, Pill } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Press } from '@/components/ui/Press';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { imageSource } from '@/data/images';
import { totalSpent, tripStatus, useAppStore, useMyTrips } from '@/store/useAppStore';
import type { Trip, TripStatus } from '@/store/types';
import { colors, radius, shadow } from '@/theme';
import { money } from '@/utils/format';
import { rangeLabel } from '@/utils/time';

const ORDER: { key: TripStatus; title: string }[] = [
  { key: 'active', title: 'In progress' },
  { key: 'planning', title: 'Upcoming' },
  { key: 'completed', title: 'Completed' },
];

export default function Trips() {
  const trips = useMyTrips();
  const activeId = useAppStore((s) => s.activeTripId);
  const setActive = useAppStore((s) => s.setActiveTrip);

  const open = (t: Trip) => {
    setActive(t.id);
    toast(`Switched to ${t.name}`);
    router.navigate('/(tabs)/itinerary');
  };

  return (
    <Screen header={<Header title="My trips" subtitle={`${trips.length} trip${trips.length === 1 ? '' : 's'} · plan, travel, remember`}
/>}>
      <View style={{ flexDirection: 'row', gap: 10, marginBottom: 18 }}>
        <Button label="New trip" icon="add" size="md" onPress={() => router.push('/new-trip')} style={{ flex: 1 }} />
        <Button label="Join with code" icon="people-outline" variant="secondary" size="md" onPress={() => router.push('/join')} style={{
flex: 1 }} />
      </View>
      {trips.length === 0 ? <EmptyState icon="airplane-outline" title="No trips yet" body="Create one or join a friend’s trip with their
invite code." /> : null}
      {ORDER.map(({ key, title }) => {
        const list = trips.filter((t) => tripStatus(t) === key);
        if (!list.length) return null;
        return (
          <View key={key} style={{ marginBottom: 18, gap: 10 }}>
            <T variant="kicker" color={colors.textSecondary}>
              {title}
            </T>
            {list.map((t) => {
              const d = destinationById(t.destinationId);
              const isActive = t.id === activeId;
              return (
                <Press key={t.id} onPress={() => open(t)} style={[styles.card, isActive && styles.cardActive]} accessibilityLabel={`${t.
name}${isActive ? ', current trip' : ''}`}>
                  <Image source={imageSource(t.cover)} style={styles.cover} contentFit="cover" />
                  <View style={{ flex: 1, gap: 4 }}>
                    <View style={{ flexDirection: 'row', alignItems: 'center', gap: 6 }}>
                      <T variant="title" numberOfLines={1} style={{ flex: 1 }}>
                        {t.name}
                      </T>
                      {isActive ? <Ionicons name="checkmark-circle" size={18} color={colors.primary} /> : null}
                    </View>
                    <T variant="caption" color={colors.textSecondary}>
                      {d?.city}, {d?.country}
                    </T>
                    <T variant="caption" color={colors.textSecondary}>
                      {rangeLabel(t.startDate, t.endDate)}
                    </T>
                    <View style={{ flexDirection: 'row', alignItems: 'center', gap: 8, marginTop: 2 }}>
                      <AvatarStack people={t.members} max={4} size={22} />
                      <Pill label={`${money(totalSpent(t))} / ${money(t.budget)}`} tone="gray" />
                    </View>
                  </View>
                </Press>
              );
            })}
          </View>
        );
```

```
      ))}
    </Screen>
  );
}

const styles = StyleSheet.create({
  card: { flexDirection: 'row', gap: 12, backgroundColor: colors.card, borderRadius: radius.lg, padding: 10, borderWidth: 1, borderColo
r: colors.border, ...shadow },
  cardActive: { borderColor: colors.primary, borderWidth: 2 },
  cover: { width: 88, height: 96, borderRadius: radius.md },
});
```

src/app/weather.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useLocalSearchParams } from 'expo-router';
import { useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { Card, Chip, Pill } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Header, Screen } from '@/components/ui/Screen';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { useWeather } from '@/hooks/useTrip';
import { cToF, fetchWeather, weatherInfo } from '@/services/weather';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import { colors, radius } from '@/theme';
import { timeAgo } from '@/utils/format';
import { dayLabel, weekdayShort } from '@/utils/time';

export default function Weather() {
  const params = useLocalSearchParams<{ id?: string }>();
  const trip = useActiveTrip();
  const options = trip ? [trip.destinationId, ...trip.extraDestinationIds] : [];
  const [id, setId] = useState(params.id ?? trip?.destinationId ?? 'tokyo');
  const [unitF, setUnitF] = useState(false);
  const [loading, setLoading] = useState(false);
  const addPacking = useAppStore((s) => s.addPackingItems);
  const w = useWeather(id);
  const d = destinationById(id)!;
  const u = (c: number) => (unitF ? `${cToF(c)}°F` : `${c}°C`);
  const info = w ? weatherInfo(w.current.code) : null;
  const rainy = w?.daily.filter((x) => x.rain >= 50) ?? [];
  const best = w?.daily.slice().sort((a, b) => a.rain - b.rain || b.max - a.max)[0];
  const maxT = Math.max(...(w?.daily.map((x) => x.max) ?? [30]));
  const minT = Math.min(...(w?.daily.map((x) => x.min) ?? [0]));

  const tips: string[] = [];
  if (w) {
    if (rainy.length) tips.push('Compact umbrella or rain jacket');
    if (maxT >= 27) tips.push('Sunscreen, hat & refillable water bottle');
    if (minT <= 12) tips.push('Warm layer for evenings');
    if (minT <= 3) tips.push('Gloves & beanie');
    tips.push('Comfortable walking shoes');
  }

  return (
    <Screen
      header={<Header title={`Weather · ${d.city}`} subtitle={w ? `${w.live ? 'Live forecast' : 'Estimated (offline)'} · updated ${timeA
go(w.fetchedAt)}` : 'Loading…'} />}
      onRefresh={async () => {
        setLoading(true);
        const r = await fetchWeather(id, true);
        setLoading(false);
        toast(r?.live ? 'Forecast updated' : 'Offline — showing the last saved forecast', { tone: r?.live ? 'ok' : 'warn' });
      }}
      refreshing={loading}>
      {options.length > 1 ? (
        <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 6 }} style={{ marginBottom: 12 }}>
          {options.map((o) => (
            <Chip key={o} small label={destinationById(o)?.city ?? o} active={o === id} onPress={() => setId(o)} />
          ))}
        </ScrollView>
      ) : null}

      {w && info ? (
        <>
          <View style={styles.hero}>
            <View style={{ flex: 1 }}>
              <T variant="kicker" color="rgba(255,255,255,0.75)">
                Right now
              </T>
              <T variant="display" color={colors.white} style={{ fontSize: 54, lineHeight: 60 }}>
                {u(w.current.temp)}
              </T>
              <T variant="title" color={colors.white}>
                {info.label}
              </T>
              <View style={{ flexDirection: 'row', gap: 14, marginTop: 8 }}>
                <T variant="caption" color="rgba(255,255,255,0.85)">
                  💧 {w.current.humidity}%
                </T>
                <T variant="caption" color="rgba(255,255,255,0.85)">
                  💨 {w.current.wind} km/h
                </T>
              </View>
```

```
            </View>
            <Ionicons name={info.icon} size={84} color="#FFD27A" />
          </View>

          <View style={styles.unitRow}>
            {!w.live ? <Pill label="Estimate — connect to refresh" tone="orange" icon="cloud-offline-outline" /> : <Pill label="Live · O
pen-Meteo" icon="radio-button-on" />}
            <View style={{ flexDirection: 'row', gap: 6 }}>
              <Chip small label="°C" active={!unitF} onPress={() => setUnitF(false)} />
              <Chip small label="°F" active={unitF} onPress={() => setUnitF(true)} />
            </View>
          </View>

          <Card padded={false} style={{ paddingHorizontal: 14, paddingVertical: 6 }}>
            {w.daily.map((x, i) => {
              const di = weatherInfo(x.code);
              const span = Math.max(1, maxT - minT);
              return (
                <View key={x.date} style={[styles.day, i === w.daily.length - 1 && { borderBottomWidth: 0 }]}>
                  <View style={{ width: 70 }}>
                    <T variant="small" weight="semibold">
                      {i === 0 ? 'Today' : weekdayShort(x.date)}
                    </T>
                    <T variant="micro" color={colors.textMuted}>
                      {dayLabel(x.date)}
                    </T>
                  </View>
                  <Ionicons name={di.icon} size={22} color={x.rain >= 50 ? colors.blue : '#E8A23A'} />
                  <T variant="caption" color={x.rain >= 50 ? colors.blue : colors.textMuted} style={{ width: 38, textAlign: 'right' }}>
                    {x.rain}%
                  </T>
                  <T variant="small" color={colors.textSecondary} style={{ width: 38, textAlign: 'right' }}>
                    {unitF ? cToF(x.min) : x.min}°
                  </T>
                  <View style={styles.bar}>
                    <View style={[styles.barFill, { left: `${((x.min - minT) / span) * 100}%`, right: `${((maxT - x.max) / span) * 100}%
` }]} />
                  </View>
                  <T variant="small" weight="semibold" style={{ width: 34 }}>
                    {unitF ? cToF(x.max) : x.max}°
                  </T>
                </View>
              );
            })}
          </Card>

          {best ? (
            <Card style={{ marginTop: 14, flexDirection: 'row', gap: 12, alignItems: 'center' }}>
              <Ionicons name="sunny" size={26} color="#E8A23A" />
              <View style={{ flex: 1 }}>
                <T variant="small" weight="semibold">
                  Best outdoor day: {weekdayShort(best.date)} {dayLabel(best.date)}
                </T>
                <T variant="caption" color={colors.textSecondary}>
                  {best.rain}% rain, up to {u(best.max)} — plan hikes and day trips here.
                </T>
              </View>
            </Card>
          ) : null}

          <Card style={{ marginTop: 14, gap: 8 }}>
            <T variant="title">What to pack</T>
            {tips.map((t) => (
              <View key={t} style={{ flexDirection: 'row', gap: 8, alignItems: 'center' }}>
                <Ionicons name="checkmark-circle-outline" size={16} color={colors.primary} />
                <T variant="small">{t}</T>
              </View>
            ))}
            {trip ? (
              <Button
                label="Add these to the packing list"
                variant="soft"
                size="md"
                icon="briefcase-outline"
                onPress={() => {
                  addPacking(trip.id, tips.map((text) => ({ text, category: 'Weather' })));
                  toast('Added to the shared packing list');
                }}
              />
            ) : null}
          </Card>
        </>
      ) : (
        <Card>
          <T variant="small" color={colors.textSecondary}>
            Fetching the forecast…
```

```
          </T>
        </Card>
      )}
    </Screen>
  );
}

const styles = StyleSheet.create({
  hero: { flexDirection: 'row', alignItems: 'center', backgroundColor: '#2E6B8F', borderRadius: radius.xl, padding: 20 },
  unitRow: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', marginVertical: 14 },
  day: { flexDirection: 'row', alignItems: 'center', gap: 10, paddingVertical: 10, borderBottomWidth: 1, borderBottomColor: colors.borde
r },
  bar: { flex: 1, height: 6, borderRadius: 3, backgroundColor: colors.bgAlt },
  barFill: { position: 'absolute', top: 0, bottom: 0, borderRadius: 3, backgroundColor: '#F2B33D' },
});
```

src/components/CurrencyConverter.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { useState } from 'react';
import { StyleSheet, TextInput, View } from 'react-native';

import { convert, useRates } from '@/services/currency';
import { colors, fonts, radius } from '@/theme';
import { money } from '@/utils/format';
import { parseAmount } from '@/utils/validation';

import { CurrencyPickerSheet } from './CurrencyPickerSheet';
import { Press } from './ui/Press';
import { T } from './ui/T';

/** Two-way converter used on the Budget tab and the Currency screen. */
export function CurrencyConverter({ from, to, onChange }: { from: string; to: string; onChange: (from: string, to: string) => void }) {
  const rates = useRates();
  const [amount, setAmount] = useState('50');
  const [picking, setPicking] = useState<'from' | 'to' | null>(null);
  const n = parseAmount(amount);
  const out = n === null ? null : convert(n, from, to, rates);
  const unit = convert(1, from, to, rates);

  const code = (which: 'from' | 'to', c: string) => (
    <Press onPress={() => setPicking(which)} style={styles.code} accessibilityLabel={`Change ${which} currency, currently ${c}`}>
      <T variant="small" weight="bold">
        {c}
      </T>
      <Ionicons name="chevron-down" size={16} color={colors.text} />
    </Press>
  );

  return (
    <View style={{ gap: 10 }}>
      <View style={styles.row}>
        <View style={[styles.box, n === null && !!amount && { borderColor: colors.red }]}>
          <TextInput
            value={amount}
            onChangeText={(v) => setAmount(v.replace(/[^0-9.,]/g, ''))}
            keyboardType="decimal-pad"
            style={styles.input}
            accessibilityLabel={`Amount in ${from}`}
            maxLength={12}
            placeholder="0"
            placeholderTextColor={colors.textMuted}
          />
        </View>
        {code('from', from)}
      </View>
      <View style={styles.swapRow}>
        <Press onPress={() => onChange(to, from)} style={styles.swap} accessibilityLabel="Swap currencies" hitSlop={8}>
          <Ionicons name="swap-vertical" size={16} color={colors.text} />
        </Press>
      </View>
      <View style={styles.row}>
        <View style={styles.box}>
          <T variant="h3" weight="bold" numberOfLines={1} adjustsFontSizeToFit style={{ flex: 1 }}>
            {out === null ? '—' : money(out, to, { decimals: true })}
          </T>
        </View>
        {code('to', to)}
      </View>
      {n === null && amount ? (
        <T variant="caption" color={colors.red}>
          Enter a number, e.g. 25 or 25.50
        </T>
      ) : null}
      <View style={styles.rateRow}>
        <View style={[styles.liveDot, { backgroundColor: rates.live ? colors.positive : colors.orange }]} />
        <T variant="caption" color={colors.textSecondary}>
          1 {from} = {unit < 0.01 ? unit.toFixed(5) : unit.toFixed(unit > 100 ? 1 : 4)} {to} · {rates.live ? 'live rate' : 'offline rat
e'}
        </T>
      </View>
      <CurrencyPickerSheet
        visible={!!picking}
        onClose={() => setPicking(null)}
        value={picking === 'from' ? from : to}
        onPick={(c) => (picking === 'from' ? onChange(c, to) : onChange(from, c))}
      />
    </View>
  );
}

const styles = StyleSheet.create({
```

```
  row: { flexDirection: 'row', alignItems: 'center', gap: 10 },
  box: { flex: 1, minWidth: 0, height: 50, borderRadius: radius.md, backgroundColor: colors.bgAlt, borderWidth: 1, borderColor: 'transpa
rent', flexDirection: 'row', alignItems: 'center', paddingHorizontal: 16 },
  input: { flex: 1, minWidth: 0, fontFamily: fonts.semibold, fontSize: 18, color: colors.text, ...({ outlineStyle: 'none' } as object)
},
  code: { width: 108, height: 50, borderRadius: radius.md, backgroundColor: colors.bgAlt, flexDirection: 'row', alignItems: 'center', ju
stifyContent: 'space-between', paddingHorizontal: 14 },
  swapRow: { alignItems: 'center' },
  swap: { width: 34, height: 34, borderRadius: 17, backgroundColor: colors.bgAlt, alignItems: 'center', justifyContent: 'center' },
  rateRow: { flexDirection: 'row', alignItems: 'center', gap: 6 },
  liveDot: { width: 7, height: 7, borderRadius: 4 },
});
```

src/components/CurrencyPickerSheet.tsx

```
import { useState } from 'react';
import { StyleSheet, TextInput, View } from 'react-native';

import { currencies } from '@/data/reference';
import { colors, fonts, radius } from '@/theme';

import { Press } from './ui/Press';
import { Sheet } from './ui/Sheet';
import { T } from './ui/T';

export function CurrencyPickerSheet({ visible, onClose, value, onPick, title = 'Choose currency' }: { visible: boolean; onClose: () => v
oid; value: string; onPick: (code: string) => void; title?: string }) {
  const [q, setQ] = useState('');
  const list = currencies.filter((c) => !q || c.code.toLowerCase().includes(q.toLowerCase()) || c.name.toLowerCase().includes(q.toLowerC
ase()));
  return (
    <Sheet visible={visible} onClose={onClose} title={title}>
      <TextInput value={q} onChangeText={setQ} placeholder="Search currency…" placeholderTextColor={colors.textMuted} style={styles.sear
ch} autoCapitalize="none" />
      <View style={{ gap: 6 }}>
        {list.map((c) => (
          <Press
            key={c.code}
            onPress={() => {
              onPick(c.code);
              onClose();
            }}
            style={[styles.row, c.code === value && styles.rowOn]}
            accessibilityState={{ selected: c.code === value }}>
            <View style={styles.sym}>
              <T variant="small" weight="bold" color={colors.primary}>
                {c.symbol.length > 3 ? c.code.slice(0, 1) : c.symbol}
              </T>
            </View>
            <T variant="bodySm" weight="semibold" style={{ width: 48 }}>
              {c.code}
            </T>
            <T variant="small" color={colors.textSecondary} style={{ flex: 1 }}>
              {c.name}
            </T>
          </Press>
        ))}
      </View>
    </Sheet>
  );
}

const styles = StyleSheet.create({
  search: { height: 44, borderRadius: radius.sm, borderWidth: 1, borderColor: colors.border, paddingHorizontal: 12, fontFamily: fonts.re
gular, color: colors.text, backgroundColor: colors.cardMuted },
  row: { flexDirection: 'row', alignItems: 'center', gap: 10, padding: 10, borderRadius: radius.sm },
  rowOn: { backgroundColor: colors.primarySofter },
  sym: { width: 32, height: 32, borderRadius: 16, backgroundColor: colors.primarySofter, alignItems: 'center', justifyContent: 'center'
},
});
```

```
import { Ionicons } from '@expo/vector-icons';
import { useRef, useState } from 'react';
import { Animated, Modal, ScrollView, StyleSheet, View, useWindowDimensions } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { useAppStore } from '@/store/useAppStore';
import { MAX_WIDTH, colors, radius, shadow } from '@/theme';

import { LogoMark } from './Logo';
import { Button } from './ui/Button';
import type { IconName } from './ui/bits';
import { FadeIn, nativeDriver, useMotion } from './ui/motion';
import { Press } from './ui/Press';
import { T } from './ui/T';

const STEPS: { icon: IconName; tab: string; title: string; body: string; points: string[] }[] = [
  { icon: 'home-outline', tab: 'Home', title: 'Your trip at a glance', body: 'The moment you open the app you see the hero photo, invite
code, weather and your group budget.', points: ['Up next and countdown', 'Weather and local time', 'Budget progress'] },
  { icon: 'calendar-outline', tab: 'Plan', title: 'Plan it together', body: 'Build the itinerary day by day. Everyone votes “I’m in”, th
en the group locks the winners in.', points: ['Itinerary by day', 'Vote on ideas', 'Lock in the plan'] },
  { icon: 'wallet-outline', tab: 'Budget', title: 'Split it fairly', body: 'Log expenses in any currency and see exactly who owes who. S
nap a receipt so nothing gets lost.', points: ['Who owes who', 'Receipts and categories', 'Settle up in fewer payments'] },
  { icon: 'chatbubbles-outline', tab: 'Chat', title: 'Talk it out', body: 'Message the crew in one place, or ask Scout, the built-in AI,
about weather, food and what’s next.', points: ['Group chat', 'Scout AI assistant', 'Works offline'] },
  { icon: 'grid-outline', tab: 'More', title: 'Everything else', body: 'Offline documents, the trip map, a packing list, the photo dump,
a trip quiz and more live here.', points: ['Offline docs', 'Trip map', 'Packing, photos and quiz'] },
];

/** First-run walkthrough: five swipeable cards, one per tab. Marks the tour as seen when it closes. */
export function GuidedTour({ visible, onClose }: { visible: boolean; onClose: () => void }) {
  const setToured = useAppStore((s) => s.setToured);
  const finish = () => {
    setToured(true);
    onClose();
  };
  return (
    <Modal visible={visible} animationType="fade" onRequestClose={finish} statusBarTranslucent>
      {visible ? <TourBody onFinish={finish} /> : null}
    </Modal>
  );
}

function TourBody({ onFinish }: { onFinish: () => void }) {
  const insets = useSafeAreaInsets();
  const { width } = useWindowDimensions();
  const motion = useMotion();
  const pageW = Math.min(width, MAX_WIDTH);
  const scroller = useRef<ScrollView>(null);
  const [index, setIndex] = useState(0);
  const [scrollX] = useState(() => new Animated.Value(0));
  const last = index === STEPS.length - 1;

  const goTo = (i: number) => {
    setIndex(i);
    scroller.current?.scrollTo({ x: i * pageW, animated: true });
  };

  return (
    <View style={[styles.root, { paddingTop: insets.top + 8, paddingBottom: Math.max(insets.bottom, 16) }]}>
      <View style={[styles.column, { width: pageW }]}>
        <View style={styles.top}>
          <View style={styles.brand}>
            <LogoMark size={30} />
            <T variant="small" weight="extrabold">
              TogetherWeGo
            </T>
          </View>
          {last ? null : (
            <Press onPress={onFinish} hitSlop={10} accessibilityLabel="Skip the tour">
              <T variant="small" weight="semibold" color={colors.textSecondary}>
                Skip
              </T>
            </Press>
          )}
        </View>

        <FadeIn style={{ flex: 1 }} duration={420} distance={16}>
          <Animated.ScrollView
            ref={scroller}
            horizontal
            pagingEnabled
            showsHorizontalScrollIndicator={false}
            scrollEventThrottle={16}
            onScroll={Animated.event([{ nativeEvent: { contentOffset: { x: scrollX } } }], {
```

```
              useNativeDriver: nativeDriver,
              listener: (e: { nativeEvent: { contentOffset: { x: number } } }) => setIndex(Math.max(0, Math.min(STEPS.length - 1, Math.r
ound(e.nativeEvent.contentOffset.x / pageW)))),
            })}
            style={{ flex: 1 }}>
            {STEPS.map((s, i) => {
              const range = [(i - 1) * pageW, i * pageW, (i + 1) * pageW];
              const pop = motion ? { opacity: scrollX.interpolate({ inputRange: range, outputRange: [0.45, 1, 0.45], extrapolate: 'clam
p' }), transform: [{ scale: scrollX.interpolate({ inputRange: range, outputRange: [0.92, 1, 0.92], extrapolate: 'clamp' }) }] } : null;
              return (
                <View key={s.tab} style={[styles.page, { width: pageW }]} accessible accessibilityLabel={`Step ${i + 1} of ${STEPS.lengt
h}. ${s.tab}. ${s.title}. ${s.body}`}>
                    <Animated.View style={[styles.card, pop]}>
                      <T variant="display" color={colors.bgAlt} style={styles.number}>
                        {`0${i + 1}`}
                      </T>
                      <View style={styles.icon}>
                        <Ionicons name={s.icon} size={30} color={colors.onAccent} />
                      </View>
                      <T variant="kicker" color={colors.textSecondary} style={{ marginTop: 22 }}>
                        {s.tab}
                      </T>
                      <T variant="display" style={{ fontSize: 34, lineHeight: 38, marginTop: 4 }}>
                        {s.title}
                      </T>
                      <T variant="body" color={colors.textSecondary} style={{ marginTop: 10 }}>
                        {s.body}
                      </T>
                      <View style={{ gap: 10, marginTop: 20 }}>
                        {s.points.map((p) => (
                          <View key={p} style={styles.point}>
                            <View style={styles.tick}>
                              <Ionicons name="checkmark" size={13} color={colors.onAccent} />
                            </View>
                            <T variant="bodySm" weight="semibold">
                              {p}
                            </T>
                          </View>
                        ))}
                      </View>
                    </Animated.View>
                  </View>
              );
            })}
          </Animated.ScrollView>
        </FadeIn>

        <View style={styles.dots} accessibilityRole="tablist">
          {STEPS.map((s, i) => (
            <Press key={s.tab} onPress={() => goTo(i)} hitSlop={8} haptic={false} accessibilityRole="tab" accessibilityState={{ selecte
d: i === index }} accessibilityLabel={`Step ${i + 1} of ${STEPS.length}: ${s.tab}`}>
              <View style={[styles.dot, i === index && styles.dotOn]} />
            </Press>
          ))}
        </View>
        <View style={{ paddingHorizontal: 20 }}>
          <Button label={last ? 'Got it' : 'Next'} iconRight={last ? 'checkmark' : 'arrow-forward'} onPress={() => (last ? onFinish() :
goTo(index + 1))} />
        </View>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.bg, alignItems: 'center' },
  column: { flex: 1 },
  top: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', paddingHorizontal: 20, height: 44 },
  brand: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  page: { paddingHorizontal: 20, justifyContent: 'center' },
  card: { backgroundColor: colors.card, borderRadius: 28, padding: 24, borderWidth: 1, borderColor: colors.border, ...shadow },
  number: { position: 'absolute', top: 14, right: 20, fontSize: 64, lineHeight: 68 },
  icon: { width: 64, height: 64, borderRadius: 32, backgroundColor: colors.accent, alignItems: 'center', justifyContent: 'center' },
  point: { flexDirection: 'row', alignItems: 'center', gap: 10 },
  tick: { width: 22, height: 22, borderRadius: 11, backgroundColor: colors.accent, alignItems: 'center', justifyContent: 'center' },
  dots: { flexDirection: 'row', alignSelf: 'center', alignItems: 'center', gap: 8, paddingVertical: 18 },
  dot: { width: 8, height: 8, borderRadius: radius.pill, backgroundColor: colors.borderStrong },
  dotOn: { width: 24, backgroundColor: colors.primary },
});
```

src/components/Logo.tsx

```
import { MaterialCommunityIcons } from '@expo/vector-icons';
import { View } from 'react-native';

import { colors } from '@/theme';

import { T } from './ui/T';

/** Brand mark: forest-green tile with a take-off plane (matches the app icon). */
export function LogoMark({ size = 40 }: { size?: number }) {
  return (
    <View
      accessibilityLabel="TogetherWeGo logo"
      style={{ width: size, height: size, borderRadius: size * 0.28, backgroundColor: colors.primary, alignItems: 'center', justifyConte
nt: 'center' }}>
      <MaterialCommunityIcons name="airplane-takeoff" size={size * 0.56} color={colors.white} />
    </View>
  );
}

export function Logo({ size = 40 }: { size?: number }) {
  return (
    <View style={{ flexDirection: 'row', alignItems: 'center', gap: 10 }}>
      <LogoMark size={size} />
      <T variant="h3" weight="bold">
        TogetherWeGo
      </T>
    </View>
  );
}
```

src/components/ReminderService.tsx

```
import { useEffect, useMemo } from 'react';
import { AppState } from 'react-native';

import { checkDueReminders, syncReminders } from '@/services/notifications';
import { useAppStore } from '@/store/useAppStore';

/**
 * Invisible component that keeps reminders live:
 * - re-schedules OS notifications whenever reminder-relevant data changes
 * - polls every 20 s while the app is open to surface due reminders in-app
 */
export function ReminderService() {
  const trips = useAppStore((s) => s.trips);
  const lead = useAppStore((s) => s.settings.reminderLeadMin);
  const enabled = useAppStore((s) => s.settings.notificationsEnabled);
  const userId = useAppStore((s) => s.session?.userId);

  const signature = useMemo(
    () =>
      [lead, enabled, userId]
        .concat(trips.flatMap((t) => t.activities.filter((a) => a.reminder).map((a) => `${a.id}|${a.date}|${a.time}`)))
        .join(','),
    [trips, lead, enabled, userId],
  );

  useEffect(() => {
    const timer = setTimeout(() => {
      syncReminders().catch(() => {});
    }, 1200);
    return () => clearTimeout(timer);
  }, [signature]);

  useEffect(() => {
    checkDueReminders();
    const id = setInterval(checkDueReminders, 20000);
    const sub = AppState.addEventListener('change', (st) => st === 'active' && checkDueReminders());
    return () => {
      clearInterval(id);
      sub.remove();
    };
  }, []);

  return null;
}
```

src/components/ShareSheet.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { StyleSheet, View } from 'react-native';

import { copy, shareText, shareTo, socialTargets } from '@/services/share';
import { colors, radius } from '@/theme';

import { Button } from './ui/Button';
import { Press } from './ui/Press';
import { Sheet } from './ui/Sheet';
import { T } from './ui/T';

/** Social share picker: direct deep links into social apps + the OS share sheet. */
export function ShareSheet({ visible, onClose, title, message, preview }: { visible: boolean; onClose: () => void; title: string; messag
e: string; preview?: string }) {
  return (
    <Sheet visible={visible} onClose={onClose} title={title} subtitle="Send it straight to your group’s favourite app">
      {preview ? (
        <View style={styles.preview}>
          <T variant="small" color={colors.textSecondary}>
            {preview}
          </T>
        </View>
      ) : null}
      <View style={styles.grid}>
        {socialTargets.map((s) => (
          <Press
            key={s.key}
            onPress={() => {
              onClose();
              shareTo(s.key, message);
            }}
            style={styles.item}
            accessibilityLabel={`Share via ${s.label}`}>
            <View style={[styles.icon, { backgroundColor: s.color }]}>
              <Ionicons name={s.icon} size={22} color={colors.white} />
            </View>
            <T variant="micro" weight="medium">
              {s.label}
            </T>
          </Press>
        ))}
      </View>
      <View style={{ flexDirection: 'row', gap: 8 }}>
        <Button label="Copy" variant="secondary" icon="copy-outline" size="md" onPress={() => copy(message)} style={{ flex: 1 }} />
        <Button
          label="More apps"
          icon="share-outline"
          size="md"
          onPress={() => {
            onClose();
            shareText(message);
          }}
          style={{ flex: 1 }}
        />
      </View>
    </Sheet>
  );
}

const styles = StyleSheet.create({
  preview: { backgroundColor: colors.cardMuted, borderRadius: radius.md, padding: 12, borderWidth: 1, borderColor: colors.border },
  grid: { flexDirection: 'row', flexWrap: 'wrap', rowGap: 16 },
  item: { width: '25%', alignItems: 'center', gap: 6 },
  icon: { width: 52, height: 52, borderRadius: 26, alignItems: 'center', justifyContent: 'center' },
});
```

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { LinearGradient } from 'expo-linear-gradient';
import { useEffect, useState, type ReactNode } from 'react';
import { Animated, Easing, StyleSheet, View } from 'react-native';

import { nativeDriver, useLoop } from '@/components/ui/motion';
import { Press } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { imageSource } from '@/data/images';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { useNow } from '@/hooks/useTrip';
import { colors } from '@/theme';
import { partsInTz } from '@/utils/time';

export type TimeOfDay = 'day' | 'night';

/** Hours 19:00–05:59 at the destination count as night. */
export function timeOfDayAt(tz: string, now: number): TimeOfDay {
  const h = Number(partsInTz(now, tz).hhmm.slice(0, 2));
  return h >= 19 || h < 6 ? 'night' : 'day';
}

/** Deterministic pseudo-random list so stars sit in the same place every render. */
const STARS = Array.from({ length: 34 }, (_, i) => {
  const a = Math.sin(i * 91.7) * 43758.5453;
  const b = Math.sin(i * 12.9898 + 4.1) * 24634.6345;
  return { x: (a - Math.floor(a)) * 100, y: (b - Math.floor(b)) * 52, size: 1.5 + ((i * 7) % 3), group: i % 3 };
});

/**
 * Photo hero that follows the day/night cycle of the trip’s city: by day a warm sky glare, a sun and drifting
 * clouds; by night the photo is graded to blue hour with twinkling stars and a moon. The sun/moon chip lets you
 * preview the other half of the day. Children are drawn on top (header buttons, headline…).
 */
export function SkylineHero({ image, tz, height, children }: { image: string; tz: string; height: number; children?: ReactNode }) {
  const insets = useSafeAreaInsets();
  const now = useNow(60000);
  const auto = timeOfDayAt(tz, now);
  const [forced, setForced] = useState<TimeOfDay | null>(null);
  const mode = forced ?? auto;
  const [night] = useState(() => new Animated.Value(auto === 'night' ? 1 : 0));

  useEffect(() => {
    const anim = Animated.timing(night, { toValue: mode === 'night' ? 1 : 0, duration: 1100, easing: Easing.inOut(Easing.cubic), useNati
veDriver: nativeDriver });
    anim.start();
    return () => anim.stop();
  }, [mode, night]);

  const zoom = useLoop(18000, { pingPong: true });
  const twinkle = [useLoop(1700, { pingPong: true, rest: 0.6 }), useLoop(2300, { pingPong: true, delay: 400, rest: 0.6 }), useLoop(2900,
{ pingPong: true, delay: 900, rest: 0.6 })];
  const cloudA = useLoop(26000, { rest: 0.2 });
  const cloudB = useLoop(38000, { delay: 3000, rest: 0.7 });

  const dayOpacity = night.interpolate({ inputRange: [0, 1], outputRange: [1, 0] });

  return (
    <View style={{ height, overflow: 'hidden', backgroundColor: '#0B1020' }}>
      <Animated.View style={[StyleSheet.absoluteFill, { transform: [{ scale: zoom.interpolate({ inputRange: [0, 1], outputRange: [1.02,
1.14] }) }] }]}>
        <Image source={imageSource(image)} style={StyleSheet.absoluteFill} contentFit="cover" transition={300} accessibilityIgnoresInver
tColors />
      </Animated.View>

      {/* Day: warm light, sun, clouds */}
      <Animated.View pointerEvents="none" style={[StyleSheet.absoluteFill, { opacity: dayOpacity }]}>
        <LinearGradient colors={['rgba(255,214,120,0.30)', 'rgba(255,255,255,0)']} start={{ x: 1, y: 0 }} end={{ x: 0.2, y: 0.6 }} style
={StyleSheet.absoluteFill} />
        <View style={styles.sunGlow} />
        <View style={styles.sun} />
        <Animated.View style={[styles.cloud, { top: height * 0.16, transform: [{ translateX: cloudA.interpolate({ inputRange: [0, 1], ou
tputRange: [-120, 520] }) }] }]}>
          <Ionicons name="cloud" size={64} color="rgba(255,255,255,0.38)" />
        </Animated.View>
        <Animated.View style={[styles.cloud, { top: height * 0.3, transform: [{ translateX: cloudB.interpolate({ inputRange: [0, 1], out
putRange: [520, -140] }) }] }]}>
          <Ionicons name="cloud" size={44} color="rgba(255,255,255,0.28)" />
        </Animated.View>
      </Animated.View>

      {/* Night: blue-hour grade, stars, moon */}
```

```
      <Animated.View pointerEvents="none" style={[StyleSheet.absoluteFill, { opacity: night }]}>
        <LinearGradient colors={['rgba(5,10,45,0.82)', 'rgba(20,28,85,0.58)', 'rgba(8,10,35,0.72)']} locations={[0, 0.55, 1]} style={Sty
leSheet.absoluteFill} />
        <View style={[StyleSheet.absoluteFill, { backgroundColor: 'rgba(40,60,160,0.16)' }]} />
        {STARS.map((s, i) => (
          <Animated.View key={i} style={[styles.star, { left: `${s.x}%`, top: `${s.y}%`, width: s.size, height: s.size, borderRadius: s.
size, opacity: twinkle[s.group].interpolate({ inputRange: [0, 1], outputRange: [0.25, 1] }) }]} />
        ))}
        <View style={styles.moonGlow} />
        <View style={styles.moon}>
          <Ionicons name="moon" size={26} color="#FFF4C7" />
        </View>
      </Animated.View>

      <LinearGradient pointerEvents="none" colors={['rgba(0,0,0,0.35)', 'rgba(0,0,0,0)', 'rgba(0,0,0,0.72)']} locations={[0, 0.4, 1]} st
yle={StyleSheet.absoluteFill} />

      {children}

      <Press
        onPress={() => setForced(mode === 'night' ? 'day' : 'night')}
        onLongPress={() => setForced(null)}
        style={[styles.toggle, { top: insets.top + 62 }]}
        accessibilityLabel={mode === 'night' ? 'Showing night. Tap for day' : 'Showing day. Tap for night'}
        accessibilityHint="Long press to follow the local time again">
        <Ionicons name={mode === 'night' ? 'moon' : 'sunny'} size={13} color={colors.white} />
        <T variant="micro" weight="bold" color={colors.white}>
          {mode === 'night' ? 'Night' : 'Day'}
          {forced ? '' : ' · live'}
        </T>
      </Press>
    </View>
  );
}

const styles = StyleSheet.create({
  sun: { position: 'absolute', right: 34, top: 54, width: 46, height: 46, borderRadius: 23, backgroundColor: '#FFE08A', opacity: 0.9 },
  sunGlow: { position: 'absolute', right: 8, top: 28, width: 98, height: 98, borderRadius: 49, backgroundColor: '#FFD25E', opacity: 0.28
},
  cloud: { position: 'absolute', left: 0 },
  star: { position: 'absolute', backgroundColor: '#FFFFFF' },
  moon: { position: 'absolute', right: 32, top: 52, width: 44, height: 44, borderRadius: 22, alignItems: 'center', justifyContent: 'cent
er', backgroundColor: 'rgba(255,244,199,0.14)' },
  moonGlow: { position: 'absolute', right: 10, top: 30, width: 88, height: 88, borderRadius: 44, backgroundColor: 'rgba(255,244,199,0.1
4)' },
  toggle: { position: 'absolute', right: 16, flexDirection: 'row', alignItems: 'center', gap: 5, paddingHorizontal: 10, paddingVertical:
6, borderRadius: 999, backgroundColor: 'rgba(0,0,0,0.38)' },
});
```

src/components/TabBar.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Tabs } from 'expo-router/js-tabs';
import { useEffect, useState, type ComponentProps } from 'react';
import { Animated, StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { useT } from '@/i18n';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import { MAX_WIDTH, colors } from '@/theme';

import type { IconName } from './ui/bits';
import { nativeDriver, useMotion } from './ui/motion';
import { Press } from './ui/Press';
import { T } from './ui/T';

type BottomTabBarProps = Parameters<NonNullable<ComponentProps<typeof Tabs>['tabBar']>>[0];

const TABS: Record<string, { label: string; icon: IconName; active: IconName }> = {
  explore: { label: 'nav.explore', icon: 'home-outline', active: 'home' },
  itinerary: { label: 'nav.itinerary', icon: 'calendar-outline', active: 'calendar' },
  budget: { label: 'nav.budget', icon: 'wallet-outline', active: 'wallet' },
  chat: { label: 'nav.chat', icon: 'chatbubble-outline', active: 'chatbubble' },
  profile: { label: 'nav.profile', icon: 'grid-outline', active: 'grid' },
};

/** Bottom navigation matching the reference: soft-green pill behind the active tab. */
export function TabBar({ state, navigation }: BottomTabBarProps) {
  const t = useT();
  const insets = useSafeAreaInsets();
  const trip = useActiveTrip();
  const lastSeen = useAppStore((s) => s.notifications.filter((n) => !n.read && n.kind === 'chat').length);
  const unreadChat = trip ? lastSeen : 0;
  const motion = useMotion();
  const visible = state.routes.filter((r) => TABS[r.name]);
  const activeSlot = Math.max(0, visible.findIndex((r) => r.key === state.routes[state.index]?.key));
  const [innerW, setInnerW] = useState(0);
  const [slide] = useState(() => new Animated.Value(activeSlot));
  const [pop] = useState(() => new Animated.Value(1));

  // The highlight pill glides to the selected tab and the new icon does a little bounce.
  useEffect(() => {
    if (!motion) {
      slide.setValue(activeSlot);
      return;
    }
    Animated.spring(slide, { toValue: activeSlot, useNativeDriver: nativeDriver, friction: 8, tension: 90 }).start();
    pop.setValue(0.7);
    Animated.spring(pop, { toValue: 1, useNativeDriver: nativeDriver, friction: 4, tension: 160 }).start();
  }, [activeSlot, motion, slide, pop]);

  const slotW = innerW > 0 ? (innerW - 16) / Math.max(1, visible.length) : 0;
  const pillW = Math.min(76, slotW - 4);

  return (
    <View style={[styles.bar, { paddingBottom: Math.max(insets.bottom, 10) }]}>
      <View style={styles.inner} onLayout={(e) => setInnerW(e.nativeEvent.layout.width)}>
        {slotW > 0 ? (
          <Animated.View
            pointerEvents="none"
            style={[
              styles.indicator,
              { width: pillW, left: 8 + (slotW - pillW) / 2, transform: [{ translateX: slide.interpolate({ inputRange: [0, Math.max(1, v
isible.length - 1)], outputRange: [0, slotW * Math.max(1, visible.length - 1)] }) }] },
            ]}
          />
        ) : null}
        {state.routes.map((route, index) => {
          const cfg = TABS[route.name];
          if (!cfg) return null;
          const focused = state.index === index;
          const onPress = () => {
            const event = navigation.emit({ type: 'tabPress', target: route.key, canPreventDefault: true });
            if (!focused && !event.defaultPrevented) navigation.navigate(route.name);
          };
          return (
            <Press
              key={route.key}
              onPress={onPress}
              accessibilityRole="tab"
              accessibilityState={{ selected: focused }}
              accessibilityLabel={t(cfg.label)}
              style={styles.item}
              scaleTo={0.92}>
              <View style={[styles.pill, focused && slotW === 0 && styles.pillActive]}>
```

```
                <Animated.View style={focused ? { transform: [{ scale: pop }] } : undefined}>
                  <Ionicons name={focused ? cfg.active : cfg.icon} size={21} color={focused ? colors.primary : colors.textMuted} />
                  {route.name === 'chat' && unreadChat > 0 && !focused ? <View style={styles.dot} /> : null}
                </Animated.View>
                <T variant="micro" weight={focused ? 'semibold' : 'medium'} color={focused ? colors.primary : colors.textMuted} numberOf
Lines={1} style={{ fontSize: 10.5 }}>
                  {t(cfg.label)}
                </T>
              </View>
            </Press>
          );
        })}
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  bar: { backgroundColor: colors.card, borderTopWidth: 1, borderTopColor: colors.border, alignItems: 'center' },
  inner: { flexDirection: 'row', width: '100%', maxWidth: MAX_WIDTH, paddingHorizontal: 8, paddingTop: 8 },
  item: { flex: 1, alignItems: 'center' },
  pill: { alignItems: 'center', justifyContent: 'center', gap: 2, paddingHorizontal: 10, minWidth: 58, height: 50, borderRadius: 25 },
  pillActive: { backgroundColor: colors.navActive },
  indicator: { position: 'absolute', top: 8, height: 50, borderRadius: 25, backgroundColor: colors.navActive },
  dot: { position: 'absolute', top: -1, right: -3, width: 8, height: 8, borderRadius: 4, backgroundColor: colors.red, borderWidth: 1.5,
borderColor: colors.card },
});
```

src/components/explore.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import { LinearGradient } from 'expo-linear-gradient';
import { router } from 'expo-router';
import { StyleSheet, View, type StyleProp, type ViewStyle } from 'react-native';

import { destinationById, type Destination } from '@/data/destinations';
import { imageSource } from '@/data/images';
import { useActiveTrip, useAppStore } from '@/store/useAppStore';
import { colors, radius, shadow } from '@/theme';
import { money } from '@/utils/format';

import { Chip, ListRow } from './ui/bits';
import { toast } from './ui/feedback';
import { Press, tap } from './ui/Press';
import { Sheet } from './ui/Sheet';
import { T } from './ui/T';

export function FavStar({ id, size = 30 }: { id: string; size?: number }) {
  const fav = useAppStore((s) => s.favorites.includes(id));
  const toggle = useAppStore((s) => s.toggleFavorite);
  return (
    <Press
      onPress={() => {
        toggle(id);
        tap();
        toast(fav ? 'Removed from favourites' : 'Saved to favourites ⭐', { icon: fav ? 'star-outline' : 'star' });
      }}
      hitSlop={6}
      accessibilityLabel={fav ? 'Remove favourite' : 'Save as favourite'}
      style={[styles.star, { width: size, height: size, borderRadius: size / 2 }]}>
      <Ionicons name={fav ? 'star' : 'star-outline'} size={size * 0.5} color={fav ? colors.star : colors.white} />
    </Press>
  );
}

export function PlaceCard({ d, style, height = 190 }: { d: Destination; style?: StyleProp<ViewStyle>; height?: number }) {
  return (
    <Press onPress={() => router.push({ pathname: '/destination/[id]', params: { id: d.id } })} style={[styles.place, { height }, styl
e]} accessibilityLabel={`${d.city}, ${d.country}, rated ${d.rating}`}>
      <Image source={imageSource(d.image)} style={StyleSheet.absoluteFill} contentFit="cover" transition={200} />
      <LinearGradient colors={['rgba(0,0,0,0)', 'rgba(0,0,0,0.65)']} locations={[0.45, 1]} style={StyleSheet.absoluteFill} />
      <View style={{ position: 'absolute', top: 10, right: 10 }}>
        <FavStar id={d.id} />
      </View>
      <View style={styles.placeText}>
        <T variant="title" weight="bold" color={colors.white} numberOfLines={1}>
          {d.city}
        </T>
        <View style={styles.row}>
          <Ionicons name="location-sharp" size={10} color="rgba(255,255,255,0.85)" />
          <T variant="micro" color="rgba(255,255,255,0.85)" numberOfLines={1} style={{ flex: 1 }}>
            {d.country}
          </T>
          <Ionicons name="star" size={10} color={colors.star} />
          <T variant="micro" weight="bold" color={colors.white}>
            {d.rating.toFixed(1)}
          </T>
        </View>
      </View>
    </Press>
  );
}

export function AddToTripSheet({ destinationId, onClose }: { destinationId: string | null; onClose: () => void }) {
  const trip = useActiveTrip();
  const updateTrip = useAppStore((s) => s.updateTrip);
  const addBucket = useAppStore((s) => s.addBucket);
  const d = destinationId ? destinationById(destinationId) : undefined;
  if (!d) return <Sheet visible={false} onClose={onClose}>{null}</Sheet>;
  const already = trip && (trip.destinationId === d.id || trip.extraDestinationIds.includes(d.id));
  return (
    <Sheet visible={!!destinationId} onClose={onClose} title={`Add ${d.city}`} subtitle={`${d.country} · ★ ${d.rating} · ~${money(d.dail
yCost)}/day per person`}>
      {trip ? (
        <ListRow
          icon="add-circle-outline"
          label={already ? `Already part of ${trip.name}` : `Add as a stop on “${trip.name}”`}
          sub={already ? 'Its places show up in Nearby and the assistant' : 'Its places appear in Nearby, the map and assistant suggesti
ons'}
          onPress={() => {
            if (!already) {
              updateTrip(trip.id, { extraDestinationIds: [...trip.extraDestinationIds, d.id] });
              toast(`${d.city} added to ${trip.name}`);
```

```
            }
            onClose();
          }}
        />
      ) : null}
      <ListRow
        icon="airplane-outline"
        label="Start a new group trip here"
        sub="Pick dates, set a budget and invite friends"
        onPress={() => {
          onClose();
          setTimeout(() => router.push({ pathname: '/new-trip', params: { destination: d.id } }), 180);
        }}
      />
      <ListRow
        icon="flag-outline"
        label="Save to bucket list"
        onPress={() => {
          addBucket(`Visit ${d.city}, ${d.country}`, d.id);
          toast('Saved to your bucket list');
          onClose();
        }}
      />
      <View style={{ flexDirection: 'row', flexWrap: 'wrap', gap: 6 }}>
        {d.highlights.slice(0, 4).map((h) => (
          <Chip key={h} small tone="soft" label={h} />
        ))}
      </View>
    </Sheet>
  );
}

const styles = StyleSheet.create({
  star: { backgroundColor: 'rgba(0,0,0,0.32)', alignItems: 'center', justifyContent: 'center' },
  place: { borderRadius: radius.lg, overflow: 'hidden', backgroundColor: colors.bgAlt },
  placeText: { position: 'absolute', left: 12, right: 12, bottom: 12, gap: 2 },
  row: { flexDirection: 'row', alignItems: 'center', gap: 3 },
  featured: { backgroundColor: colors.card, borderRadius: radius.lg, overflow: 'hidden', borderWidth: 1, borderColor: colors.border, ...
shadow },
  featBadge: { position: 'absolute', top: 12, left: 12, backgroundColor: colors.primary, paddingHorizontal: 10, paddingVertical: 4, bord
erRadius: 999 },
  tag: { paddingHorizontal: 9, paddingVertical: 4, borderRadius: 999, borderWidth: 1, borderColor: colors.border },
  shareBtn: { width: 44, height: 44, borderRadius: 22, borderWidth: 1, borderColor: colors.border, alignItems: 'center', justifyContent:
'center' },
});
```

src/components/map/OfflineRadar.tsx

```
/**
 * Offline map: plots every stop around the trip centre using real distances
 * and bearings — no map tiles or internet needed. Handy on a plane or when
 * roaming data is off.
 */
import { useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { Press } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { colors } from '@/theme';
import { projectKm } from '@/utils/geo';

import type { MapPoint } from './types';

export function OfflineRadar({ center, points, radiusKm, selectedId, onSelect }: { center: { lat: number; lng: number }; points: MapPoin
t[]; radiusKm: number; selectedId?: string | null; onSelect: (id: string) => void }) {
  const [size, setSize] = useState(300);
  const half = size / 2;
  const scale = half / radiusKm;
  const rings = [0.25, 0.5, 1].map((f) => Math.max(0.5, Math.round(radiusKm * f * 10) / 10));

  return (
    <View style={styles.wrap} onLayout={(e) => setSize(Math.min(e.nativeEvent.layout.width, e.nativeEvent.layout.height))}>
      <View style={{ width: size, height: size }}>
        {rings.map((r) => (
          <View
            key={r}
            style={[styles.ring, { width: r * scale * 2, height: r * scale * 2, borderRadius: r * scale, left: half - r * scale, top: ha
lf - r * scale }]}>
            <T variant="micro" color={colors.textMuted} style={{ position: 'absolute', top: -2, alignSelf: 'center', backgroundColor: co
lors.bgAlt, paddingHorizontal: 3 }}>
              {r} km
            </T>
          </View>
        ))}
        <View style={[styles.axis, { left: half, top: 0, width: 1, height: size }]} />
        <View style={[styles.axis, { top: half, left: 0, height: 1, width: size }]} />
        <T variant="micro" weight="bold" color={colors.textSecondary} style={{ position: 'absolute', top: 4, left: half + 4 }}>
          N
        </T>
        {points.map((p) => {
          const { x, y } = projectKm(center, p);
          if (Math.hypot(x, y) > radiusKm * 1.02) return null;
          const sel = p.id === selectedId;
          return (
            <Press
              key={p.id}
              onPress={() => onSelect(p.id)}
              hitSlop={8}
              accessibilityLabel={`${p.title}, ${p.subtitle}`}
              style={[
                styles.dot,
                { left: half + x * scale - (sel ? 9 : 6), top: half - y * scale - (sel ? 9 : 6), backgroundColor: p.color, width: sel ?
18 : 12, height: sel ? 18 : 12, borderRadius: 9 },
              ]}
            />
          );
        })}
        <View style={[styles.center, { left: half - 7, top: half - 7 }]} />
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  wrap: { ...StyleSheet.absoluteFill, backgroundColor: colors.bgAlt, alignItems: 'center', justifyContent: 'center' },
  ring: { position: 'absolute', borderWidth: 1, borderColor: colors.borderStrong, borderStyle: 'dashed' },
  axis: { position: 'absolute', backgroundColor: colors.border },
  dot: { position: 'absolute', borderWidth: 2, borderColor: colors.white },
  center: { position: 'absolute', width: 14, height: 14, borderRadius: 7, backgroundColor: colors.primary, borderWidth: 3, borderColor:
colors.white },
});
```

src/components/map/TripMap.tsx

```
/** Native map (Apple Maps on iOS, Google Maps on Android) via react-native-maps. */
import { useEffect, useRef } from 'react';
import { StyleSheet } from 'react-native';
import MapView, { Circle, Marker } from 'react-native-maps';

import type { TripMapProps } from './types';

export function TripMap({ center, points, radiusKm, selectedId, onSelect, showsUser }: TripMapProps) {
  const ref = useRef<MapView>(null);
  const delta = Math.max(0.04, (radiusKm / 111) * 2.6);

  useEffect(() => {
    ref.current?.animateToRegion({ latitude: center.lat, longitude: center.lng, latitudeDelta: delta, longitudeDelta: delta }, 400);
  }, [center.lat, center.lng, delta]);

  useEffect(() => {
    const p = points.find((x) => x.id === selectedId);
    if (p) ref.current?.animateToRegion({ latitude: p.lat, longitude: p.lng, latitudeDelta: 0.03, longitudeDelta: 0.03 }, 350);
  }, [selectedId, points]);

  return (
    <MapView
      ref={ref}
      style={StyleSheet.absoluteFill}
      initialRegion={{ latitude: center.lat, longitude: center.lng, latitudeDelta: delta, longitudeDelta: delta }}
      showsUserLocation={showsUser}
      showsCompass
      toolbarEnabled={false}>
      <Circle center={{ latitude: center.lat, longitude: center.lng }} radius={radiusKm * 1000} strokeColor="rgba(31,77,37,0.6)" fillCol
or="rgba(31,77,37,0.08)" strokeWidth={1.5} />
      {points.map((p) => (
        <Marker
          key={p.id}
          coordinate={{ latitude: p.lat, longitude: p.lng }}
          title={p.title}
          description={p.subtitle}
          pinColor={p.color}
          onPress={() => onSelect(p.id)}
          tracksViewChanges={false}
        />
      ))}
    </MapView>
  );
}
```

src/components/map/TripMap.web.tsx

```
/** Web preview: OpenStreetMap embed (react-native-maps is native-only). */
import { createElement } from 'react';
import { StyleSheet, View } from 'react-native';

import type { TripMapProps } from './types';

export function TripMap({ center, points, radiusKm, selectedId }: TripMapProps) {
  const focus = points.find((p) => p.id === selectedId) ?? center;
  const d = Math.max(0.02, (radiusKm / 111) * 1.3);
  const bbox = [focus.lng - d, focus.lat - d * 0.7, focus.lng + d, focus.lat + d * 0.7].join(',');
  const src = `https://www.openstreetmap.org/export/embed.html?bbox=${bbox}&layer=mapnik&marker=${focus.lat},${focus.lng}`;
  return (
    <View style={StyleSheet.absoluteFill}>
      {createElement('iframe', { src, title: 'Trip map', style: { border: 0, width: '100%', height: '100%' }, loading: 'lazy' })}
    </View>
  );
}
```

src/components/map/types.ts

```
export interface MapPoint {
  id: string;
  lat: number;
  lng: number;
  title: string;
  subtitle: string;
  color: string;
  kind: 'activity' | 'place' | 'airport' | 'me';
}

export interface TripMapProps {
  center: { lat: number; lng: number };
  points: MapPoint[];
  radiusKm: number;
  selectedId?: string | null;
  onSelect: (id: string) => void;
  showsUser?: boolean;
}
```

```
import { LinearGradient } from 'expo-linear-gradient';
import { memo, useId, useState, type ReactNode } from 'react';
import { Animated, StyleSheet, View } from 'react-native';
import Svg, { Circle, Defs, G, Path, Pattern, RadialGradient, Rect, Stop, Use } from 'react-native-svg';

import { useLoop } from '@/components/ui/motion';
import type { Landmark, Pano, Roof } from '@/data/panoramas';

import { LOOKS, SHAPES, type Part, type Tone } from './shapes';

/** Layer speeds relative to the far layer. Wider layers scroll faster, which gives the parallax. */
export const K = { far: 1, mid: 1.4, near: 1.9, sky: 0.5 } as const;

export function layout(S: number) {
  const base = 4 * S;
  return { far: base * K.far, mid: base * K.mid, near: base * K.near, sky: base * K.sky };
}

/** Compass heading (0–360°) at the centre of the screen for a scroll position. */
export function headingOf(pos: number, S: number) {
  const W = layout(S).mid;
  return (((((pos * K.mid + S / 2) % W) + W) % W) / W) * 360;
}

/** Scroll position that centres a given compass heading. */
export function posForHeading(deg: number, S: number) {
  return ((deg / 360) * layout(S).mid - S / 2) / K.mid;
}

/** x position of something at compass heading `deg` inside a layer (so it lines up with the mid layer). */
function xFor(layerK: number, W: number, deg: number, S: number) {
  return (((posForHeading(deg, S) * layerK + S / 2) % W) + W) % W;
}

function rng(seed: number) {
  let a = seed >>> 0;
  return () => {
    a = (a + 0x6d2b79f5) >>> 0;
    let t = a;
    t = Math.imul(t ^ (t >>> 15), t | 1);
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}

function shade(hex: string, amt: number) {
  const n = parseInt(hex.slice(1), 16);
  const ch = (v: number) => Math.max(0, Math.min(255, Math.round(v + (amt >= 0 ? (255 - v) * amt : v * amt))));
  const r = ch((n >> 16) & 255);
  const g = ch((n >> 8) & 255);
  const b = ch(n & 255);
  return `#${((1 << 24) | (r << 16) | (g << 8) | b).toString(16).slice(1)}`;
}

interface Building {
  x: number;
  w: number;
  h: number;
  roof: Roof;
  v: number;
  amt: number;
}

function genBuildings(seed: number, W: number, wr: [number, number], hr: [number, number], gap: number, roofs: Roof[], towers: number,
H: number): Building[] {
  const rand = rng(seed);
  const out: Building[] = [];
  let x = 0;
  while (x < W) {
    let w = wr[0] + rand() * (wr[1] - wr[0]);
    if (W - x - w < wr[0]) w = W - x;
    const tall = rand() < towers;
    const h = H * (tall ? hr[1] * (0.6 + rand() * 0.4) : hr[0] + (hr[1] - hr[0]) * Math.pow(rand(), 1.8) * 0.7);
    out.push({ x, w: Math.max(4, w - gap), h, roof: roofs[Math.floor(rand() * roofs.length)], v: Math.floor(rand() * 4), amt: (rand() -
0.5) * 0.22 });
    x += w;
  }
  return out;
}

/** Four window grids with different lit patterns, so skylines don't look tiled. */
function WindowPatterns({ id, night, color, base }: { id: string; night: boolean; color: string; base: string }) {
  return (
    <>
```

```
      {[0, 1, 2, 3].map((v) => {
        const rand = rng(900 + v * 17);
        const cells: ReactNode[] = [];
        for (let i = 0; i < 4; i++)
          for (let j = 0; j < 4; j++) {
            const lit = rand() < 0.55;
            if (night && !lit) continue;
            cells.push(<Rect key={`${i}-${j}`} x={3 + i * 6} y={4 + j * 9} width={3} height={5} fill={night ? color : lit ? '#FFFFFF' :
base} opacity={night ? 0.95 : lit ? 0.4 : 0.55} />);
          }
        return (
          <Pattern key={v} id={`${id}${v}`} x={0} y={0} width={24} height={36} patternUnits="userSpaceOnUse">
            {cells}
          </Pattern>
        );
      })}
    </>
  );
}

function roofEl(b: Building, y: number, fill: string, night: boolean, key: string): ReactNode {
  const { x, w, h } = b;
  switch (b.roof) {
    case 'step':
      return <Rect key={key} x={x + w * 0.25} y={y - h * 0.08} width={w * 0.5} height={h * 0.08} fill={fill} />;
    case 'spire':
      return (
        <G key={key}>
          <Rect x={x + w / 2 - 0.8} y={y - h * 0.2} width={1.6} height={h * 0.2} fill={fill} />
          {night ? <Circle cx={x + w / 2} cy={y - h * 0.2} r={1.8} fill="#FF4B3E" /> : null}
        </G>
      );
    case 'slant':
      return <Path key={key} d={`M${x} ${y}L${x + w * 0.15} ${y - 4}L${x + w} ${y - 12}L${x + w} ${y}Z`} fill={fill} />;
    case 'gable':
      return <Path key={key} d={`M${x - 1} ${y}L${x + w / 2} ${y - w * 0.42}L${x + w + 1} ${y}Z`} fill={night ? shade('#8A4A3A', -0.6) :
'#B5614A'} />;
    case 'dome':
      return <Path key={key} d={`M${x + w * 0.1} ${y}A${w * 0.4} ${w * 0.4} 0 0 1 ${x + w * 0.9} ${y}Z`} fill={shade(fill, night ? 0.1 :
-0.1)} />;
    default:
      return null;
  }
}

function Landmarks({ list, layer, W, base, H, night, S }: { list: Landmark[]; layer: 'mid' | 'far'; W: number; base: number; H: number;
night: boolean; S: number }) {
  return (
    <>
      {list
        .filter((lm) => !!lm.far === (layer === 'far'))
        .map((lm) => {
          const look = LOOKS[lm.kind];
          const idx = night ? 1 : 0;
          const x = xFor(K[layer], W, lm.heading, S);
          const colour = (k: Tone) => look[k][idx];
          return (
            <G key={lm.id} transform={`translate(${x} ${base}) scale(${H * lm.size})`}>
              {night && look.glowR > 0 ? <Circle cx={0} cy={-0.5} r={look.glowR} fill={`url(#glow-${lm.kind})`} /> : null}
              {SHAPES[lm.kind].map((pt: Part, i) => {
                const f = colour(pt.k);
                if (pt.t === 'p') return <Path key={i} d={pt.d} fill={f} />;
                if (pt.t === 'r') return <Rect key={i} x={pt.x} y={pt.y} width={pt.w} height={pt.h} fill={f} />;
                if (pt.t === 'c') return <Circle key={i} cx={pt.x} cy={pt.y} r={pt.r} fill={f} />;
                if (pt.t === 'l') return <Path key={i} d={`M${pt.x1} ${pt.y1}L${pt.x2} ${pt.y2}`} stroke={f} strokeWidth={pt.w} />;
                return <Path key={i} d={pt.d} stroke={f} strokeWidth={pt.w} fill="none" />;
              })}
            </G>
          );
        })}
    </>
  );
}

function GlowDefs({ list }: { list: Landmark[] }) {
  return (
    <>
      {list.map((lm) => (
        <RadialGradient key={lm.id} id={`glow-${lm.kind}`} cx="50%" cy="50%" r="50%">
          <Stop offset="0" stopColor={LOOKS[lm.kind].glow} stopOpacity={0.55} />
          <Stop offset="1" stopColor={LOOKS[lm.kind].glow} stopOpacity={0} />
        </RadialGradient>
      ))}
    </>
  );
}
```

```
/** One scrolling layer: content is drawn once and repeated twice side by side so it can wrap around 360°. */
function Wrap({ W, H, defs, children }: { W: number; H: number; defs: ReactNode; children: ReactNode }) {
  const id = `strip${useId().replace(/\W/g, '')}`;
  return (
    <Svg width={W * 2} height={H} style={{ position: 'absolute', left: 0, top: 0 }}>
      <Defs>{defs}</Defs>
      <G id={id}>{children}</G>
      <Use href={`#${id}`} x={W} />
    </Svg>
  );
}

function skylineLayers(pano: Pano, S: number, H: number, night: boolean) {
  const W = layout(S);
  const i = night ? 1 : 0;
  const rand = rng(pano.seed * 7);
  const midBase = H * 0.58;
  const nearBase = H * 0.67;
  const farBase = H * 0.5;

  // ---- far layer: mountains or distant blocks
  const far: ReactNode[] = [];
  if (pano.mountains) {
    const f = rng(pano.seed + 3);
    const ph = [f() * 6, f() * 6, f() * 6];
    let d = `M0 ${farBase}`;
    for (let x = 0; x <= W.far; x += 12) {
      const t = (x / W.far) * Math.PI * 2;
      const amp =
        pano.mountains === 'hills'
          ? 0.05 + 0.03 * Math.sin(t * 3 + ph[0]) + 0.02 * Math.sin(t * 7 + ph[1])
          : pano.mountains === 'granite'
            ? 0.07 + 0.1 * Math.pow(Math.abs(Math.sin(t * 3 + ph[0])), 1.6) + 0.03 * Math.sin(t * 9 + ph[1])
            : 0.08 + 0.1 * Math.abs(Math.sin(t * 4 + ph[0])) + 0.04 * Math.sin(t * 11 + ph[2]);
      d += `L${x} ${farBase - H * amp}`;
    }
    d += `L${W.far} ${farBase}Z`;
    far.push(<Path key="mount" d={d} fill={pano.tone.far[i]} />);
  } else {
    genBuildings(pano.seed + 5, W.far, [12, 30], [0.03, 0.15], 2, ['flat', 'step', 'flat'], 0.1, H).forEach((b, k) =>
      far.push(<Rect key={`f${k}`} x={b.x} y={farBase - b.h} width={b.w} height={b.h} fill={shade(pano.tone.far[i], b.amt)} />),
    );
  }

  // ---- mid layer: the skyline
  const mid: ReactNode[] = [];
  const midDefs: ReactNode[] = [];
  if (pano.ground === 'slope') {
    const f = rng(pano.seed + 9);
    const ph = [f() * 6, f() * 6];
    const top = (x: number) => midBase - H * (0.13 + 0.06 * Math.sin((x / W.mid) * Math.PI * 2 * 2 + ph[0]) + 0.04 * Math.sin((x / W.mi
d) * Math.PI * 2 * 5 + ph[1]));
    let d = `M0 ${midBase}`;
    for (let x = 0; x <= W.mid; x += 12) d += `L${x} ${top(x)}`;
    d += `L${W.mid} ${midBase}Z`;
    mid.push(<Path key="hill" d={d} fill={pano.tone.mid[i]} />);
    const colours = pano.houses ?? ['#F2B8A2'];
    for (let row = 0; row < 2; row++) {
      let x = row * 11;
      let n = 0;
      while (x < W.mid - 40) {
        const w = 24 + rand() * 16;
        const h = H * (0.035 + rand() * 0.02);
        const y = Math.min(midBase - h, top(x + w / 2) + row * H * 0.035 + rand() * 6);
        const c = colours[Math.floor(rand() * colours.length)];
        const fill = night ? shade(c, -0.62) : c;
        mid.push(<Path key={`h${row}-${n}`} d={`M${x} ${y + h}L${x} ${y + 6}L${x + w / 2} ${y}L${x + w} ${y + 6}L${x + w} ${y + h}Z`} fi
ll={fill} />);
        if (night) mid.push(<Rect key={`w${row}-${n}`} x={x + w * 0.3} y={y + h * 0.45} width={4} height={5} fill={pano.lights} />);
        x += w + 3;
        n++;
      }
    }
  } else {
    midDefs.push(<WindowPatterns key="wp" id={night ? 'wmn' : 'wmd'} night={night} color={pano.lights} base={shade(pano.tone.mid[i], 0.2
5)} />);
    genBuildings(pano.seed, W.mid, pano.mid.w, pano.mid.h, pano.mid.gap, pano.mid.roofs, pano.mid.towers, H).forEach((b, k) => {
      const fill = shade(pano.tone.mid[i], b.amt);
      const y = midBase - b.h;
      mid.push(<Rect key={`m${k}`} x={b.x} y={y} width={b.w} height={b.h} fill={fill} />);
      if (b.w > 8 && b.h > 16) mid.push(<Rect key={`mw${k}`} x={b.x + 1} y={y + 2} width={b.w - 2} height={b.h - 2} fill={`url(#${night
? 'wmn' : 'wmd'}${b.v})`} />);
      mid.push(roofEl(b, y, fill, night, `mr${k}`));
      if (pano.ground === 'water')
        mid.push(<Rect key={`mr${k}f`} x={b.x} y={midBase} width={b.w} height={Math.min(H * 0.1, b.h * 0.45)} fill={night ? pano.lights
```

```
: '#FFFFFF'} opacity={night ? 0.12 : 0.1} />);
    });
  }

  // ---- near layer: low rooftops and street lamps
  const near: ReactNode[] = [];
  const nearDefs: ReactNode[] = [<WindowPatterns key="wp" id={night ? 'wnn' : 'wnd'} night={night} color={pano.lights} base={shade(pano.
tone.near[i], 0.2)} />];
  genBuildings(pano.seed + 1, W.near, [34, 78], [pano.near.h[0], pano.near.h[1]], 6, pano.houses ? ['gable', 'flat'] : ['flat', 'gable',
'step'], 0, H).forEach((b, k) => {
    const c = pano.houses ? pano.houses[k % pano.houses.length] : pano.tone.near[i];
    const fill = pano.houses ? (night ? shade(c, -0.68) : c) : shade(c, b.amt);
    const y = nearBase - b.h;
    near.push(
      <Rect key={`n${k}`} x={b.x} y={y} width={b.w} height={b.h} fill={fill} />,
      <Rect key={`nw${k}`} x={b.x + 2} y={y + 3} width={b.w - 4} height={b.h - 4} fill={`url(#${night ? 'wnn' : 'wnd'}${b.v})`} />,
      roofEl({ ...b, roof: b.roof === 'step' ? 'flat' : b.roof }, y, fill, night, `nr${k}`),
    );
  });
  for (let x = 40; x < W.near; x += 130 + rand() * 40) {
    near.push(<Rect key={`lp${x}`} x={x} y={nearBase - H * 0.075} width={2} height={H * 0.075} fill={night ? '#20243A' : '#4A505C'} />);
    near.push(night ? <Circle key={`lg${x}`} cx={x + 1} cy={nearBase - H * 0.078} r={13} fill={pano.lights} opacity={0.3} /> : null);
    near.push(<Circle key={`lb${x}`} cx={x + 1} cy={nearBase - H * 0.078} r={2.6} fill={night ? '#FFF1C9' : '#9AA3B0'} />);
  }
  // Foreground details: lane markings on streets, shimmer on water.
  if (pano.ground === 'road') {
    for (let x = 0; x < W.near; x += 64) near.push(<Rect key={`ln${x}`} x={x} y={H * 0.79} width={30} height={3} fill={night ? '#C9C3A0'
: '#E8E3C8'} opacity={night ? 0.3 : 0.7} />);
    near.push(<Rect key="edge" x={0} y={H * 0.72} width={W.near} height={2} fill={night ? '#2A2E48' : '#6B7280'} opacity={0.8} />);
  } else {
    for (let k = 0; k < 140; k++) near.push(<Rect key={`sh${k}`} x={rand() * W.near} y={H * (0.69 + rand() * 0.27)} width={14 + rand() *
40} height={1.6} rx={1} fill={night ? pano.lights : '#FFFFFF'} opacity={night ? 0.14 + rand() * 0.14 : 0.18 + rand() * 0.2} />);
  }
  return { far, mid, near, midDefs, nearDefs, W, farBase, midBase, nearBase };
}

function Skyline({ pano, S, H, night, pos }: { pano: Pano; S: number; H: number; night: boolean; pos: Animated.Value }) {
  const [built] = useState(() => skylineLayers(pano, S, H, night));
  const { W } = built;
  const glowDefs = <GlowDefs list={pano.landmarks} />;
  return (
    <>
      <Layer pos={pos} k={K.far} W={W.far} H={H} defs={null}>
        {built.far}
        <Landmarks list={pano.landmarks} layer="far" W={W.far} base={built.farBase} H={H} night={night} S={S} />
      </Layer>
      <Layer pos={pos} k={K.mid} W={W.mid} H={H} defs={[...built.midDefs, glowDefs]}>
        {built.mid}
        <Landmarks list={pano.landmarks} layer="mid" W={W.mid} base={built.midBase} H={H} night={night} S={S} />
      </Layer>
      <Layer pos={pos} k={K.near} W={W.near} H={H} defs={built.nearDefs}>
        {built.near}
      </Layer>
    </>
  );
}

/** A layer wrapper that scrolls with the shared position value. */
function Layer({ pos, k, W, H, defs, children }: { pos: Animated.Value; k: number; W: number; H: number; defs: ReactNode; children: Reac
tNode }) {
  const tx = Animated.multiply(Animated.modulo(Animated.multiply(pos, k), W), -1);
  return (
    <Animated.View pointerEvents="none" style={[StyleSheet.absoluteFill, { transform: [{ translateX: tx }] }]}>
      <Wrap W={W} H={H} defs={defs}>
        {children}
      </Wrap>
    </Animated.View>
  );
}

function SkyDecor({ pano, S, H, night }: { pano: Pano; S: number; H: number; night: boolean }) {
  const Ws = layout(S).sky;
  const [items] = useState(() => {
    const rand = rng(pano.seed * 13);
    const stars = Array.from({ length: 70 }, () => ({ x: rand() * Ws, y: rand() * H * 0.42, r: 0.6 + rand() * 1.4, o: 0.5 + rand() * 0.5
}));
    const clouds = Array.from({ length: 5 }, () => ({ x: rand() * Ws, y: H * (0.1 + rand() * 0.22), w: 70 + rand() * 90 }));
    return { stars, clouds };
  });
  const sunX = xFor(K.sky, Ws, 205, S);
  const moonX = xFor(K.sky, Ws, 150, S);
  return (
    <Wrap
      W={Ws}
      H={H}
      defs={
```

```
        <>
          <RadialGradient id="sun" cx="50%" cy="50%" r="50%">
            <Stop offset="0" stopColor="#FFE9A8" stopOpacity={0.9} />
            <Stop offset="1" stopColor="#FFE9A8" stopOpacity={0} />
          </RadialGradient>
          <RadialGradient id="moon" cx="50%" cy="50%" r="50%">
            <Stop offset="0" stopColor="#FFF4C7" stopOpacity={0.5} />
            <Stop offset="1" stopColor="#FFF4C7" stopOpacity={0} />
          </RadialGradient>
        </>
      }>
      {night ? (
        <>
          {items.stars.map((st, i) => (
            <Circle key={i} cx={st.x} cy={st.y} r={st.r} fill="#FFFFFF" opacity={st.o} />
          ))}
          <Circle cx={moonX} cy={H * 0.17} r={70} fill="url(#moon)" />
          <Circle cx={moonX} cy={H * 0.17} r={20} fill="#FFF4C7" />
          <Circle cx={moonX + 8} cy={H * 0.17 - 4} r={18} fill={pano.sky.nightTop} opacity={0.9} />
        </>
      ) : (
        <>
          <Circle cx={sunX} cy={H * 0.2} r={150} fill="url(#sun)" />
          <Circle cx={sunX} cy={H * 0.2} r={26} fill="#FFF3C4" />
          {items.clouds.map((c, i) => (
            <G key={i} opacity={0.75}>
              <Rect x={c.x} y={c.y} width={c.w} height={14} rx={7} fill="#FFFFFF" />
              <Rect x={c.x + c.w * 0.2} y={c.y - 9} width={c.w * 0.5} height={16} rx={8} fill="#FFFFFF" />
            </G>
          ))}
        </>
      )}
    </Wrap>
  );
}

function CarLights({ H, S, color }: { H: number; S: number; color: string }) {
  const a = useLoop(9000, { rest: 0 });
  const b = useLoop(12000, { rest: 0 });
  const dots = Array.from({ length: 18 }, (_, i) => i);
  const row = (key: string, v: Animated.Value, y: number, c: string, dir: 1 | -1) => (
    <Animated.View
      key={key}
      pointerEvents="none"
      style={{ position: 'absolute', top: y, left: 0, width: S * 2, height: 6, transform: [{ translateX: v.interpolate({ inputRange: [0,
1], outputRange: dir === 1 ? [-S, 0] : [0, -S] }) }] }}>
      {dots.map((i) => (
        <View key={i} style={{ position: 'absolute', left: (i * S * 2) / 18 + ((i * 37) % 23), width: 11, height: 4, borderRadius: 2, ba
ckgroundColor: c, opacity: 0.9 }} />
      ))}
    </Animated.View>
  );
  return (
    <>
      {row('w', a, H * 0.755, '#FFF6D8', -1)}
      {row('r', b, H * 0.8, '#FF5A4D', 1)}
      <View pointerEvents="none" style={{ position: 'absolute', left: 0, right: 0, top: H * 0.745, height: H * 0.08, backgroundColor: co
lor, opacity: 0.05 }} />
    </>
  );
}

function PanoramaSceneImpl({ pano, night, pos, S, H }: { pano: Pano; night: Animated.Value; pos: Animated.Value; S: number; H: number })
{
  const flicker = useLoop(2600, { pingPong: true, rest: 0.5 });
  const nightOp = Animated.multiply(night, flicker.interpolate({ inputRange: [0, 1], outputRange: [0.93, 1] }));
  const dayOp = night.interpolate({ inputRange: [0, 1], outputRange: [1, 0] });
  const water = pano.ground !== 'road';
  const ground = water ? { day: ['#5B97C9', '#2F6A9E'], night: ['#0B1235', '#04061A'] } : { day: ['#454C59', '#2B3039'], night: ['#0E112
2', '#05060F'] };
  const skyTx = Animated.multiply(Animated.modulo(Animated.multiply(pos, K.sky), layout(S).sky), -1);
  const sparkle = useLoop(3200, { pingPong: true, rest: 0.5 });
  return (
    <View style={StyleSheet.absoluteFill} pointerEvents="none">
      {/* Sky */}
      <Animated.View style={[StyleSheet.absoluteFill, { opacity: dayOp }]}>
        <LinearGradient colors={[pano.sky.dayTop, pano.sky.dayHorizon]} locations={[0.05, 0.55]} style={StyleSheet.absoluteFill} />
      </Animated.View>
      <Animated.View style={[StyleSheet.absoluteFill, { opacity: night }]}>
        <LinearGradient colors={[pano.sky.nightTop, pano.sky.nightHorizon]} locations={[0.05, 0.58]} style={StyleSheet.absoluteFill} />
      </Animated.View>
      <Animated.View style={[StyleSheet.absoluteFill, { opacity: dayOp, transform: [{ translateX: skyTx }] }]}>
        <SkyDecor pano={pano} S={S} H={H} night={false} />
      </Animated.View>
      <Animated.View style={[StyleSheet.absoluteFill, { opacity: Animated.multiply(night, sparkle.interpolate({ inputRange: [0, 1], outp
utRange: [0.8, 1] })), transform: [{ translateX: skyTx }] }]}>
```

```
        <SkyDecor pano={pano} S={S} H={H} night />
      </Animated.View>

      {/* Ground / water */}
      <Animated.View style={[styles.ground, { top: H * 0.5, opacity: dayOp }]}>
        <LinearGradient colors={ground.day as [string, string]} style={StyleSheet.absoluteFill} />
      </Animated.View>
      <Animated.View style={[styles.ground, { top: H * 0.5, opacity: night }]}>
        <LinearGradient colors={ground.night as [string, string]} style={StyleSheet.absoluteFill} />
      </Animated.View>

      {/* Skyline: day set and night set cross-fade */}
      <Animated.View style={[StyleSheet.absoluteFill, { opacity: dayOp }]}>
        <Skyline pano={pano} S={S} H={H} night={false} pos={pos} />
      </Animated.View>
      <Animated.View style={[StyleSheet.absoluteFill, { opacity: nightOp }]}>
        <Skyline pano={pano} S={S} H={H} night pos={pos} />
      </Animated.View>

      {/* Warm city glow above the skyline at night */}
      <Animated.View style={[styles.glow, { top: H * 0.3, height: H * 0.3, opacity: Animated.multiply(night, 0.5) }]}>
        <LinearGradient colors={['rgba(255,150,70,0)', pano.lights]} style={StyleSheet.absoluteFill} />
      </Animated.View>
      <Animated.View style={[styles.glow, { top: H * 0.3, height: H * 0.3, opacity: Animated.multiply(dayOp, 0.28) }]}>
        <LinearGradient colors={['rgba(255,255,255,0)', '#FFFFFF']} style={StyleSheet.absoluteFill} />
      </Animated.View>

      {!water ? (
        <Animated.View style={[StyleSheet.absoluteFill, { opacity: night }]}>
          <CarLights S={S} H={H} color={pano.lights} />
        </Animated.View>
      ) : null}
    </View>
  );
}

export const PanoramaScene = memo(PanoramaSceneImpl);

const styles = StyleSheet.create({
  ground: { position: 'absolute', left: 0, right: 0, bottom: 0 },
  glow: { position: 'absolute', left: 0, right: 0 },
});
```

src/components/panorama/shapes.ts

```
import type { LandmarkKind } from '@/data/panoramas';

/**
 * Landmark silhouettes in unit coordinates: x is centred, y is negative upwards, 1 = full height.
 * Tones: m = main body, a = accent, l = light (windows / lamps — glows at night).
 */
export type Tone = 'm' | 'a' | 'l';
export type Part =
  | { t: 'p'; d: string; k: Tone }
  | { t: 'r'; x: number; y: number; w: number; h: number; k: Tone }
  | { t: 'c'; x: number; y: number; r: number; k: Tone }
  | { t: 'l'; x1: number; y1: number; x2: number; y2: number; w: number; k: Tone }
  | { t: 's'; d: string; w: number; k: Tone };

const p = (d: string, k: Tone): Part => ({ t: 'p', d, k });
const r = (x: number, y: number, w: number, h: number, k: Tone): Part => ({ t: 'r', x, y, w, h, k });
const c = (x: number, y: number, rad: number, k: Tone): Part => ({ t: 'c', x, y, r: rad, k });
const l = (x1: number, y1: number, x2: number, y2: number, w: number, k: Tone): Part => ({ t: 'l', x1, y1, x2, y2, w, k });
const s = (d: string, w: number, k: Tone): Part => ({ t: 's', d, w, k });

export interface Look {
  /** [day, night] colour for each tone. */
  m: [string, string];
  a: [string, string];
  l: [string, string];
  /** Night glow colour. */
  glow: string;
  /** Glow radius in units. */
  glowR: number;
}

const shell = (x: number, w: number, hh: number) =>
  `M${x} -.06C${x} ${-0.06 - hh * 0.4} ${x + w * 0.4} ${-0.06 - hh} ${x + w * 0.8} ${-0.06 - hh}C${x + w * 0.9} ${-0.06 - hh * 0.6} ${x
+ w} ${-0.06 - hh * 0.2} ${x + w} -.06Z`;

function eye(): Part[] {
  const out: Part[] = [s('M-.5 -.5A.5 .5 0 1 0 .5 -.5A.5 .5 0 1 0 -.5 -.5', 0.012, 'a'), l(0, -0.5, -0.2, 0, 0.016, 'm'), l(0, -0.5, 0.
2, 0, 0.016, 'm')];
  for (let i = 0; i < 16; i++) {
    const a = (i / 16) * Math.PI * 2;
    const x = Math.cos(a) * 0.5;
    const y = -0.5 + Math.sin(a) * 0.5;
    out.push(l(0, -0.5, x, y, 0.003, 'm'), c(x, y, 0.017, 'l'));
  }
  return out;
}

function harbour(): Part[] {
  const out: Part[] = [s('M-1.1 0Q0 -1.5 1.1 0', 0.04, 'm'), r(-1.3, -0.12, 2.6, 0.03, 'm'), r(-1.3, -0.12, 2.6, 0.01, 'l'), r(-1.16, -
0.45, 0.12, 0.45, 'a'), r(1.04, -0.45, 0.12, 0.45, 'a')];
  for (let i = 1; i < 20; i++) {
    const t = i / 20;
    out.push(l(-1.1 + 2.2 * t, -3 * t * (1 - t), -1.1 + 2.2 * t, -0.12, 0.004, 'm'));
  }
  return out;
}

function operaParts(): Part[] {
  const shells: [number, number, number][] = [
    [-0.66, 0.3, 0.58],
    [-0.4, 0.28, 0.48],
    [-0.16, 0.26, 0.38],
    [0.08, 0.34, 0.64],
    [0.36, 0.32, 0.52],
    [0.62, 0.2, 0.3],
  ];
  return [r(-0.75, -0.06, 1.55, 0.06, 'a'), ...shells.map(([x, w, hh]) => p(shell(x, w, hh), 'm'))];
}

function colosseum(): Part[] {
  const out: Part[] = [p('M-.8 0L-.8-.4L-.62-.5L.5-.5L.8-.4L.8 0Z', 'm')];
  for (let row = 0; row < 3; row++) for (let i = 0; i < 15; i++) out.push(r(-0.76 + i * 0.105, -0.13 - row * 0.12, 0.05, 0.08, 'a'));
  return out;
}

function supertrees(): Part[] {
  const out: Part[] = [];
  for (const [cx, hh] of [
    [-0.5, 0.7],
    [0, 1],
    [0.5, 0.8],
  ] as const) {
    out.push(
      p(`M${cx - 0.03} 0L${cx - 0.008} ${-hh}L${cx + 0.008} ${-hh}L${cx + 0.03} 0Z`, 'm'),
```

```
      p(`M${cx - 0.24} ${-hh}Q${cx} ${-hh - 0.3} ${cx + 0.24} ${-hh}Z`, 'a'),
      c(cx, -hh - 0.07, 0.018, 'l'),
    );
  }
  return out;
}

export const SHAPES: Record<LandmarkKind, Part[]> = {
  eiffel: [
    p('M-.19 0L-.015-1L.015-1L.19 0L.13 0Q0 -.34 -.13 0Z', 'm'),
    r(-0.13, -0.405, 0.26, 0.016, 'm'),
    r(-0.075, -0.7, 0.15, 0.014, 'm'),
    r(-0.004, -1.07, 0.008, 0.07, 'm'),
    c(0, -0.41, 0.012, 'l'),
    c(0, -0.71, 0.01, 'l'),
    r(-0.004, -0.98, 0.008, 0.2, 'l'),
  ],
  tokyotower: [
    p('M-.14 0L-.03-.66L-.016-.78L-.006-1L.006-1L.016-.78L.03-.66L.14 0L.1 0Q0 -.22 -.1 0Z', 'm'),
    r(-0.065, -0.34, 0.13, 0.03, 'a'),
    r(-0.04, -0.62, 0.08, 0.025, 'a'),
    r(-0.12, -0.15, 0.24, 0.02, 'a'),
    c(0, -0.33, 0.012, 'l'),
  ],
  skytree: [p('M-.055 0L-.014-.6L.014-.6L.055 0Z', 'm'), r(-0.05, -0.64, 0.1, 0.03, 'a'), r(-0.032, -0.8, 0.064, 0.025, 'a'), r(-0.004,
-1, 0.008, 0.2, 'm'), r(-0.003, -0.6, 0.006, 0.4, 'l')],
  fuji: [p('M-1 0L-.12-.5L-.06-.52L0-.55L.06-.52L.12-.5L1 0Z', 'm'), p('M-.16-.43L-.12-.5L-.06-.52L0-.55L.06-.52L.12-.5L.16-.43L.09-.46
L.04-.4L-.02-.45L-.08-.39Z', 'a')],
  esb: [
    r(-0.15, -0.4, 0.3, 0.4, 'm'),
    r(-0.11, -0.62, 0.22, 0.22, 'm'),
    r(-0.075, -0.78, 0.15, 0.16, 'm'),
    r(-0.045, -0.88, 0.09, 0.1, 'a'),
    r(-0.022, -0.93, 0.044, 0.05, 'a'),
    r(-0.004, -1, 0.008, 0.07, 'a'),
    r(-0.03, -0.86, 0.06, 0.02, 'l'),
  ],
  wtc: [p('M-.075 0L-.075-.7L-.003-.86L.003-.86L.075-.7L.075 0Z', 'm'), r(-0.003, -1, 0.006, 0.15, 'a'), r(-0.003, -0.86, 0.006, 0.86,
'l')],
  chrysler: [r(-0.07, -0.66, 0.14, 0.66, 'm'), p('M-.07-.66Q-.07-.9 0-.95Q.07-.9 .07-.66Z', 'a'), r(-0.004, -1, 0.008, 0.06, 'a'), c(0,
-0.82, 0.014, 'l')],
  bigben: [
    r(-0.05, -0.78, 0.1, 0.78, 'm'),
    p('M-.065-.78L0-1L.065-.78Z', 'a'),
    r(-0.065, -0.8, 0.13, 0.03, 'a'),
    c(0, -0.62, 0.032, 'l'),
    r(0.05, -0.2, 0.7, 0.2, 'm'),
    r(0.2, -0.3, 0.05, 0.1, 'm'),
    r(0.45, -0.28, 0.04, 0.08, 'm'),
    p('M.2-.3L.225-.38L.25-.3Z', 'a'),
    r(0.1, -0.14, 0.6, 0.03, 'l'),
  ],
  eye: eye(),
  shard: [p('M-.07 0L0-1L.07 0Z', 'm'), l(0, -1, -0.03, 0, 0.004, 'a'), l(0, -1, 0.03, 0, 0.004, 'a'), l(0, -1, 0, 0, 0.006, 'l')],
  burj: [
    p('M-.07 0L-.05-.35L-.035-.5L-.03-.62L-.02-.78L-.012-.9L-.003-1L.003-1L.012-.9L.02-.78L.03-.62L.035-.5L.05-.35L.07 0Z', 'm'),
    l(0, 0, 0, -0.95, 0.005, 'l'),
    l(-0.03, -0.2, -0.015, -0.6, 0.003, 'a'),
    l(0.03, -0.2, 0.015, -0.6, 0.003, 'a'),
  ],
  burjalarab: [p('M-.2 0L-.2-.75Q-.02-1 .02-1Q.2-.45 .2 0Z', 'm'), r(-0.45, -0.03, 0.9, 0.03, 'a'), r(-0.012, -1.1, 0.012, 0.12, 'a'), l
(-0.2, -0.75, 0.02, -1, 0.012, 'l')],
  opera: operaParts(),
  harbour: harbour(),
  sydtower: [r(-0.012, -0.7, 0.024, 0.7, 'm'), p('M-.06-.7L.06-.7L.045-.8L-.045-.8Z', 'a'), r(-0.05, -0.83, 0.1, 0.05, 'a'), r(-0.004, -
1, 0.008, 0.17, 'm'), r(-0.045, -0.8, 0.09, 0.012, 'l')],
  mbs: [
    p('M-.52 0L-.34 0L-.22-1L-.4-1Z', 'm'),
    p('M-.09 0L.09 0L.09-1L-.09-1Z', 'm'),
    p('M.34 0L.52 0L.4-1L.22-1Z', 'm'),
    r(-0.66, -1.05, 1.32, 0.05, 'a'),
    p('M.66-1.05L.84-1.03L.66-1Z', 'a'),
    r(-0.66, -1, 1.32, 0.012, 'l'),
  ],
  supertrees: supertrees(),
  christ: [
    p('M-1 0Q-.4 -.15 -.2 -.6Q-.1 -.95 0 -.95Q.1 -.95 .2 -.6Q.5 -.15 1 0Z', 'm'),
    r(-0.016, -1, 0.032, 0.03, 'a'),
    r(-0.007, -1.12, 0.014, 0.12, 'a'),
    r(-0.06, -1.1, 0.12, 0.014, 'a'),
    c(0, -1.135, 0.009, 'a'),
  ],
  sugarloaf: [p('M-.5 0Q-.52 -.6 -.15 -.78Q.2 -.9 .4 -.5Q.52 -.2 .55 0Z', 'm'), p('M.45 0Q.6 -.35 .9 -.35Q1.3 -.3 1.5 0Z', 'm'), l(0.28,
-0.7, 1.0, -0.4, 0.004, 'a'), r(0.6, -0.6, 0.05, 0.04, 'l')],
  colosseum: colosseum(),
  stpeters: [
```

```
    r(-0.4, -0.28, 0.8, 0.28, 'm'),
    r(-0.16, -0.6, 0.32, 0.32, 'm'),
    p('M-.16-.6Q-.16-1 0-1Q.16-1 .16-.6Z', 'a'),
    r(-0.03, -1.12, 0.06, 0.12, 'm'),
    r(-0.004, -1.25, 0.008, 0.13, 'a'),
    r(-0.3, -0.2, 0.6, 0.02, 'l'),
  ],
  duomo: [
    r(-0.3, -0.5, 0.6, 0.5, 'm'),
    p('M-.22-.5Q-.22-.95 0-.95Q.22-.95 .22-.5Z', 'a'),
    l(-0.1, -0.5, -0.06, -0.93, 0.02, 'l'),
    l(0.1, -0.5, 0.06, -0.93, 0.02, 'l'),
    r(0.32, -0.78, 0.12, 0.78, 'm'),
    p('M.3-.78L.38-.92L.46-.78Z', 'a'),
    r(-0.24, -0.28, 0.12, 0.14, 'l'),
    r(0.12, -0.28, 0.12, 0.14, 'l'),
  ],
  arc: [p('M-.45 0L-.45-1L.45-1L.45 0L.18 0L.18-.55Q0 -.8 -.18-.55L-.18 0Z', 'm'), r(-0.45, -0.9, 0.9, 0.02, 'a'), r(-0.45, -1, 0.9, 0.0
5, 'a')],
  sacre: [
    r(-0.4, -0.35, 0.8, 0.35, 'm'),
    p('M-.15-.35Q-.15-.85 0-.9Q.15-.85 .15-.35Z', 'm'),
    p('M-.38-.35Q-.38-.6 -.28-.62Q-.18-.6 -.18-.35Z', 'm'),
    p('M.18-.35Q.18-.6 .28-.62Q.38-.6 .38-.35Z', 'm'),
    r(-0.012, -1, 0.024, 0.1, 'a'),
    r(-0.3, -0.2, 0.6, 0.02, 'l'),
  ],
};

const warm = (day: string, night: string): [string, string] => [day, night];

export const LOOKS: Record<LandmarkKind, Look> = {
  eiffel: { m: warm('#5B4B40', '#C98F2E'), a: warm('#6F5E50', '#E5B04C'), l: warm('#8C7A6A', '#FFE08A'), glow: '#FFC14D', glowR: 0.6 },
  tokyotower: { m: warm('#D9432B', '#FF7A3A'), a: warm('#F3F3F3', '#FFF1C9'), l: warm('#F3F3F3', '#FFF1C9'), glow: '#FF8A4A', glowR: 0.5
},
  skytree: { m: warm('#C9D2DC', '#6FB6FF'), a: warm('#EEF3F8', '#BFE4FF'), l: warm('#E8EEF5', '#CFEAFF'), glow: '#6FB6FF', glowR: 0.4 },
  fuji: { m: warm('#8C9DB8', '#1B2150'), a: warm('#FFFFFF', '#8FA4E0'), l: warm('#FFFFFF', '#8FA4E0'), glow: '#000000', glowR: 0 },
  esb: { m: warm('#B9B2A6', '#EAD9A8'), a: warm('#9C9488', '#FFD36B'), l: warm('#D7D1C6', '#FFF0B8'), glow: '#FFD36B', glowR: 0.5 },
  wtc: { m: warm('#8FA6C0', '#BFD7F5'), a: warm('#C9D5E2', '#FFFFFF'), l: warm('#DCE6F0', '#FFFFFF'), glow: '#BFD7F5', glowR: 0.45 },
  chrysler: { m: warm('#A9A39C', '#E6DFC8'), a: warm('#D5DAE0', '#F4F8FF'), l: warm('#F5F5F5', '#FFFFFF'), glow: '#F4F8FF', glowR: 0.45
},
  bigben: { m: warm('#B3A58E', '#D9B76E'), a: warm('#7D8B6E', '#F0D8A0'), l: warm('#E8E2D0', '#FFF6C9'), glow: '#FFD98A', glowR: 0.45 },
  eye: { m: warm('#8FA3B8', '#5EB6FF'), a: warm('#B5C7DA', '#8AD0FF'), l: warm('#FFFFFF', '#CFEFFF'), glow: '#5EB6FF', glowR: 0.7 },
  shard: { m: warm('#9DB6CE', '#CFE3FF'), a: warm('#C5D6E6', '#FFFFFF'), l: warm('#E6EEF6', '#FFFFFF'), glow: '#CFE3FF', glowR: 0.4 },
  burj: { m: warm('#B9C6D4', '#DDEBFF'), a: warm('#8EA2B8', '#7DB8FF'), l: warm('#E8F0F8', '#FFFFFF'), glow: '#9CC8FF', glowR: 0.4 },
  burjalarab: { m: warm('#F4F1EA', '#5A9BFF'), a: warm('#D9D5CB', '#B58BFF'), l: warm('#FFFFFF', '#E1D0FF'), glow: '#7B8CFF', glowR: 0.6
},
  opera: { m: warm('#F2EFE8', '#FFD9A0'), a: warm('#C9C3B6', '#C97F4A'), l: warm('#FFFFFF', '#FFE9C0'), glow: '#FFC878', glowR: 0.7 },
  harbour: { m: warm('#59616B', '#8EB8E8'), a: warm('#B59A7B', '#F4C27A'), l: warm('#FFFFFF', '#FFE08A'), glow: '#8EB8E8', glowR: 0.5 },
  sydtower: { m: warm('#9AA3AD', '#F4C95D'), a: warm('#C9A24B', '#FFE08A'), l: warm('#FFFFFF', '#FFF0B0'), glow: '#F4C95D', glowR: 0.4
},
  mbs: { m: warm('#B5BFC9', '#E6EEF8'), a: warm('#8FA0B2', '#FFB36B'), l: warm('#E8F2FF', '#FFE7A8'), glow: '#8FC8FF', glowR: 0.5 },
  supertrees: { m: warm('#4B6B52', '#2E4A3A'), a: warm('#6FA37A', '#C25BFF'), l: warm('#D0F0D8', '#FF8CF0'), glow: '#C25BFF', glowR: 0.7
},
  christ: { m: warm('#4D6B5A', '#10202A'), a: warm('#F2F2F2', '#FFF3C9'), l: warm('#FFFFFF', '#FFF3C9'), glow: '#FFF3C9', glowR: 0.1 },
  sugarloaf: { m: warm('#5E7467', '#16222B'), a: warm('#C9D2CC', '#FFD07A'), l: warm('#FFFFFF', '#FFD07A'), glow: '#FFD07A', glowR: 0 },
  colosseum: { m: warm('#C7A57C', '#6D4B33'), a: warm('#8E6E4A', '#FFC873'), l: warm('#E8D2B0', '#FFD89A'), glow: '#FFB45E', glowR: 0.5
},
  stpeters: { m: warm('#D9CDB6', '#F0CF8F'), a: warm('#B8A98B', '#FFE3A8'), l: warm('#F2EAD8', '#FFF0C8'), glow: '#FFD98A', glowR: 0.5
},
  duomo: { m: warm('#F2D5B0', '#FFCF8A'), a: warm('#E0A93C', '#FFE08A'), l: warm('#4E9E6B', '#7DFFB0'), glow: '#FFC878', glowR: 0.5 },
  arc: { m: warm('#C9BFA9', '#EBCB86'), a: warm('#A89E88', '#FFE2A0'), l: warm('#FFFFFF', '#FFF0C8'), glow: '#FFD98A', glowR: 0.5 },
  sacre: { m: warm('#F4EFE6', '#FFE9B8'), a: warm('#D9D2C4', '#FFF6D9'), l: warm('#FFFFFF', '#FFF6D9'), glow: '#FFE9B8', glowR: 0.5 },
};
```

src/components/pickers.tsx

```
/** Cross-platform time & date pickers (identical on iOS, Android and web). */
import { Ionicons } from '@expo/vector-icons';
import { useMemo, useState } from 'react';
import { StyleSheet, View } from 'react-native';

import { colors, radius } from '@/theme';
import { addDays, daysBetween, parseYMD, time12, ymdFromUtcDate } from '@/utils/time';

import { Button } from './ui/Button';
import { Chip } from './ui/bits';
import { Press } from './ui/Press';
import { Sheet } from './ui/Sheet';
import { T } from './ui/T';

function Stepper({ label, value, onUp, onDown }: { label: string; value: string; onUp: () => void; onDown: () => void }) {
  return (
    <View style={styles.stepper}>
      <Press onPress={onUp} accessibilityLabel={`Increase ${label}`} style={styles.stepBtn}>
        <Ionicons name="chevron-up" size={22} color={colors.primary} />
      </Press>
      <T variant="display" style={{ fontSize: 40, lineHeight: 48 }} accessibilityLabel={`${label} ${value}`}>
        {value}
      </T>
      <Press onPress={onDown} accessibilityLabel={`Decrease ${label}`} style={styles.stepBtn}>
        <Ionicons name="chevron-down" size={22} color={colors.primary} />
      </Press>
      <T variant="caption" color={colors.textMuted}>
        {label}
      </T>
    </View>
  );
}

export function TimePickerSheet({ visible, value, onClose, onChange }: { visible: boolean; value: string; onClose: () => void; onChange:
(v: string) => void }) {
  const [h, setH] = useState(10);
  const [m, setM] = useState(0);
  // Re-sync with the field each time the sheet opens (derived during render, no effect needed).
  const [openedWith, setOpenedWith] = useState<string | null>(null);
  const openKey = visible ? value : null;
  if (openKey !== openedWith) {
    setOpenedWith(openKey);
    if (visible) {
      const [hh, mm] = value.split(':').map(Number);
      setH(hh || 0);
      setM(mm || 0);
    }
  }
  const hhmm = `${String(h).padStart(2, '0')}:${String(m).padStart(2, '0')}`;
  const pm = h >= 12;
  const presets = ['07:00', '09:00', '12:30', '15:00', '18:00', '19:30', '21:00'];

  return (
    <Sheet
      visible={visible}
      onClose={onClose}
      title="Pick a time"
      subtitle={`Trip local time · ${time12(hhmm)}`}
      footer={
        <Button
          label="Set time"
          onPress={() => {
            onChange(hhmm);
            onClose();
          }}
        />
      }>
      <View style={styles.timeRow}>
        <Stepper label="hour" value={String(h % 12 === 0 ? 12 : h % 12)} onUp={() => setH((x) => (x + 1) % 24)} onDown={() => setH((x) =
> (x + 23) % 24)} />
        <T variant="display" style={{ fontSize: 40, lineHeight: 48, marginTop: -18 }}>
          :
        </T>
        <Stepper label="minute" value={String(m).padStart(2, '0')} onUp={() => setM((x) => (x + 5) % 60)} onDown={() => setM((x) => (x +
55) % 60)} />
        <View style={{ gap: 8, marginLeft: 6 }}>
          <Chip label="AM" active={!pm} onPress={() => pm && setH((x) => x - 12)} small />
          <Chip label="PM" active={pm} onPress={() => !pm && setH((x) => x + 12)} small />
        </View>
      </View>
      <View style={styles.wrap}>
        {presets.map((p) => (
          <Chip
            key={p}
```

```
            small
            label={time12(p)}
            active={p === hhmm}
            onPress={() => {
              const [a, b] = p.split(':').map(Number);
              setH(a);
              setM(b);
            }}
          />
        ))}
      </View>
    </Sheet>
  );
}

const WD = ['S', 'M', 'T', 'W', 'T', 'F', 'S'];
const MONTHS = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'Decembe
r'];

export function DatePickerSheet({
  visible,
  value,
  onClose,
  onChange,
  min,
  max,
  title = 'Pick a date',
}: {
  visible: boolean;
  value: string;
  onClose: () => void;
  onChange: (v: string) => void;
  min?: string;
  max?: string;
  title?: string;
}) {
  const [cursor, setCursor] = useState(value);
  const [openedWith, setOpenedWith] = useState<string | null>(null);
  const openKey = visible ? value : null;
  if (openKey !== openedWith) {
    setOpenedWith(openKey);
    if (visible) setCursor(value);
  }
  const { y, m } = parseYMD(cursor);
  const cells = useMemo(() => {
    const first = new Date(Date.UTC(y, m - 1, 1));
    const offset = first.getUTCDay();
    const daysInMonth = new Date(Date.UTC(y, m, 0)).getUTCDate();
    const out: (string | null)[] = Array(offset).fill(null);
    for (let d = 1; d <= daysInMonth; d++) out.push(ymdFromUtcDate(new Date(Date.UTC(y, m - 1, d))));
    while (out.length % 7) out.push(null);
    return out;
  }, [y, m]);

  const shiftMonth = (n: number) => setCursor(ymdFromUtcDate(new Date(Date.UTC(y, m - 1 + n, 1))));
  const disabled = (d: string) => (min && daysBetween(min, d) < 0) || (max && daysBetween(d, max) < 0);

  return (
    <Sheet visible={visible} onClose={onClose} title={title}>
      <View style={styles.monthRow}>
        <Press onPress={() => shiftMonth(-1)} accessibilityLabel="Previous month" style={styles.monthBtn}>
          <Ionicons name="chevron-back" size={18} color={colors.text} />
        </Press>
        <T variant="title">
          {MONTHS[m - 1]} {y}
        </T>
        <Press onPress={() => shiftMonth(1)} accessibilityLabel="Next month" style={styles.monthBtn}>
          <Ionicons name="chevron-forward" size={18} color={colors.text} />
        </Press>
      </View>
      <View style={styles.grid}>
        {WD.map((w, i) => (
          <View key={`w${i}`} style={styles.cell}>
            <T variant="caption" color={colors.textMuted} weight="semibold">
              {w}
            </T>
          </View>
        ))}
        {cells.map((d, i) =>
          d ? (
            <Press
              key={d}
              disabled={!!disabled(d)}
              onPress={() => {
                onChange(d);
                onClose();
```

```
              }}
              accessibilityLabel={d}
              accessibilityState={{ selected: d === value, disabled: !!disabled(d) }}
              style={[styles.cell, styles.day, d === value && styles.daySel, disabled(d) && { opacity: 0.3 }]}>
              <T variant="bodySm" weight={d === value ? 'bold' : 'regular'} color={d === value ? colors.white : colors.text}>
                {parseYMD(d).d}
              </T>
            </Press>
          ) : (
            <View key={`e${i}`} style={styles.cell} />
          ),
        )}
      </View>
      <View style={styles.wrap}>
        <Chip small label="Today" onPress={() => setCursor(ymdFromUtcDate(new Date()))} />
        <Chip small label="+1 week" onPress={() => setCursor(addDays(cursor, 7))} />
        <Chip small label="+1 month" onPress={() => shiftMonth(1)} />
      </View>
    </Sheet>
  );
}

const styles = StyleSheet.create({
  timeRow: { flexDirection: 'row', alignItems: 'center', justifyContent: 'center', gap: 6, paddingVertical: 8 },
  stepper: { alignItems: 'center', width: 76 },
  stepBtn: { width: 44, height: 36, borderRadius: radius.sm, backgroundColor: colors.primarySofter, alignItems: 'center', justifyConten
t: 'center' },
  wrap: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
  monthRow: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between' },
  monthBtn: { width: 36, height: 36, borderRadius: 18, borderWidth: 1, borderColor: colors.border, alignItems: 'center', justifyContent:
'center' },
  grid: { flexDirection: 'row', flexWrap: 'wrap' },
  cell: { width: `${100 / 7}%`, aspectRatio: 1, alignItems: 'center', justifyContent: 'center' },
  day: { borderRadius: 999 },
  daySel: { backgroundColor: colors.primary },
});
```

src/components/trip/ActivityCard.tsx

```
import { Ionicons } from '@expo/vector-icons';
import * as Linking from 'expo-linking';
import { useState } from 'react';
import { LayoutAnimation, Platform, StyleSheet, View } from 'react-native';

import { AvatarStack, type IconName } from '@/components/ui/bits';
import { confirmAction, toast } from '@/components/ui/feedback';
import { Press, tap } from '@/components/ui/Press';
import { T } from '@/components/ui/T';
import { destinationById } from '@/data/destinations';
import { useT } from '@/i18n';
import { convert, useRates } from '@/services/currency';
import { ensurePermission } from '@/services/notifications';
import { useAppStore } from '@/store/useAppStore';
import type { Activity, Trip } from '@/store/types';
import { colors, radius, shadow } from '@/theme';
import { lengthTag, money } from '@/utils/format';
import { dayLabel, weekdayShort } from '@/utils/time';

import { typeEmoji } from './meta';

interface Props {
  trip: Trip;
  activity: Activity;
  onEdit: (a: Activity) => void;
  onAssign: (a: Activity) => void;
  /** 'vote' adds the progress bar + “Lock in” button used on the Vote tab. */
  mode?: 'plan' | 'vote';
}

/** One activity: emoji badge, when/where, tags, and a yellow vote pill with the people who said “I’m in”. */
export function ActivityCard({ trip, activity: a, onEdit, onAssign, mode = 'plan' }: Props) {
  const t = useT();
  const rates = useRates();
  const [open, setOpen] = useState(false);
  const userId = useAppStore((s) => s.session?.userId ?? '');
  const vote = useAppStore((s) => s.voteActivity);
  const update = useAppStore((s) => s.updateActivity);
  const remove = useAppStore((s) => s.deleteActivity);
  const currency = destinationById(trip.destinationId)?.currency ?? 'USD';
  const ups = Object.entries(a.votes).filter(([, v]) => v === 1).map(([id]) => id);
  const myVote = a.votes[userId];
  const total = trip.members.length;
  const needed = Math.floor(total / 2) + 1;
  const yesPeople = ups.map((id) => trip.members.find((m) => m.id === id)).filter(Boolean) as Trip['members'];
  const cost = a.costPerPerson > 0 ? `${money(convert(a.costPerPerson, 'USD', currency, rates), currency)} pp` : 'Free';

  const toggle = () => {
    if (Platform.OS !== 'web') LayoutAnimation.configureNext(LayoutAnimation.Presets.easeInEaseOut);
    setOpen((o) => !o);
  };

  const directions = () => {
    const q = a.lat && a.lng ? `${a.lat},${a.lng}` : encodeURIComponent(`${a.title} ${a.location}`);
    Linking.openURL(`https://www.google.com/maps/search/?api=1&query=${q}`).catch(() => toast('Couldn’t open maps', { tone: 'warn' }));
  };

  const toggleReminder = async () => {
    const next = !a.reminder;
    if (next) {
      const ok = await ensurePermission();
      if (!ok && Platform.OS !== 'web') toast('Enable notifications in Settings to get alerts', { tone: 'warn', icon: 'notifications-off
-outline' });
    }
    update(trip.id, a.id, { reminder: next });
    tap();
    toast(next ? `Reminder on — we’ll ping the group before ${a.title}` : 'Reminder turned off', { icon: next ? 'notifications' : 'notif
ications-off-outline' });
  };

  const lockIn = () => {
    if (trip.ownerId !== userId) {
      const boss = trip.members.find((m) => m.id === trip.ownerId)?.name.split(' ')[0] ?? 'the organizer';
      return toast(`Only ${boss} (the organizer) can lock this in`, { tone: 'warn', icon: 'lock-closed-outline' });
    }
    update(trip.id, a.id, { status: 'confirmed', reminder: true });
    tap('success');
    toast(`Locked in: ${a.title} 🔒`, { icon: 'lock-closed' });
  };

  return (
    <View style={[styles.card, a.done && { opacity: 0.6 }]}>
      <Press onPress={toggle} scaleTo={0.99} accessibilityLabel={`${a.title}, ${weekdayShort(a.date)} ${a.time}, ${ups.length} of ${tota
l} in`} accessibilityHint="Shows details and actions">
```

```
        <View style={styles.top}>
          <View style={styles.badge}>
            <T style={{ fontSize: 22, lineHeight: 28 }}>{typeEmoji[a.type]}</T>
          </View>
          <View style={{ flex: 1, gap: 5 }}>
            <T variant="title" weight="bold" numberOfLines={open ? undefined : 2} style={[{ fontSize: 17, lineHeight: 22 }, a.done && {
textDecorationLine: 'line-through' }]}>
              {a.title}
            </T>
            <View style={styles.meta}>
              <View style={styles.inline}>
                <Ionicons name="time-outline" size={13} color={colors.textMuted} />
                <T variant="caption" color={colors.textSecondary}>
                  {weekdayShort(a.date)}, {dayLabel(a.date)} · {a.time}
                </T>
              </View>
              <View style={[styles.inline, { flexShrink: 1 }]}>
                <Ionicons name="location-outline" size={13} color={colors.textMuted} />
                <T variant="caption" color={colors.textSecondary} numberOfLines={1} style={{ flexShrink: 1 }}>
                  {a.location}
                </T>
              </View>
            </View>
            <View style={styles.tags}>
              <Tag label={lengthTag(a.durationMin)} />
              <Tag label={cost} />
              {a.status !== 'confirmed' && mode === 'plan' ? <Tag label={t(a.status === 'voting' ? 'itin.voting' : 'itin.pending')} tone
="warn" /> : null}
            </View>
          </View>
          <Press
            onPress={() =>
              confirmAction('Delete activity?', `“${a.title}” will be removed for everyone.`, () => {
                remove(trip.id, a.id);
                toast('Activity deleted');
              }, { confirmLabel: 'Delete', destructive: true })
            }
            hitSlop={10}
            accessibilityLabel={`Delete ${a.title}`}>
            <Ionicons name="trash-outline" size={17} color={colors.textMuted} />
          </Press>
        </View>
      </Press>

      <View style={styles.bottom}>
        <Press
          onPress={() => {
            vote(trip.id, a.id, 1);
            tap();
          }}
          style={[styles.votePill, myVote === 1 ? styles.votePillOn : styles.votePillOff]}
          accessibilityRole="checkbox"
          accessibilityState={{ checked: myVote === 1 }}
          accessibilityLabel={`I’m in. ${ups.length} of ${total} are in`}>
          <Ionicons name={myVote === 1 ? 'thumbs-up' : 'thumbs-up-outline'} size={14} color={myVote === 1 ? colors.onAccent : colors.tex
tSecondary} />
          <T variant="small" weight="bold" color={myVote === 1 ? colors.onAccent : colors.textSecondary}>
            {ups.length}/{total}
          </T>
        </Press>
        {yesPeople.length ? <AvatarStack people={yesPeople} max={4} size={24} /> : null}
        {mode === 'vote' ? (
          <>
            <View style={styles.track} accessibilityRole="progressbar" accessibilityLabel={`${ups.length} of ${needed} votes needed`}>
              <View style={[styles.fill, { width: `${Math.min(1, ups.length / needed) * 100}%` }]} />
            </View>
            <Press onPress={lockIn} style={styles.lock} accessibilityLabel={`Lock in ${a.title}`}>
              <Ionicons name="checkmark" size={14} color={colors.onPrimary} />
              <T variant="caption" weight="bold" color={colors.onPrimary}>
                Lock in
              </T>
            </Press>
          </>
        ) : (
          <>
            <View style={{ flex: 1 }} />
            <Press onPress={toggleReminder} hitSlop={8} accessibilityLabel={a.reminder ? 'Turn reminder off' : 'Turn reminder on'} style
={[styles.bell, a.reminder && styles.bellOn]}>
              <Ionicons name={a.reminder ? 'notifications' : 'notifications-outline'} size={15} color={a.reminder ? colors.onPrimary : c
olors.textSecondary} />
            </Press>
          </>
        )}
      </View>

      {open ? (
```

```
        <View style={styles.details}>
          {a.notes ? (
            <View style={styles.notes}>
              <Ionicons name="document-text-outline" size={14} color={colors.textSecondary} />
              <T variant="small" color={colors.textSecondary} style={{ flex: 1 }}>
                {a.notes}
              </T>
            </View>
          ) : null}
          {a.status === 'voting' ? (
            <T variant="caption" color={colors.textMuted}>
              Confirms automatically when more than half the crew votes “I’m in”, or the organizer can lock it in from the Vote tab.
            </T>
          ) : null}
          <View style={styles.actions}>
            <Action icon={myVote === -1 ? 'thumbs-down' : 'thumbs-down-outline'} label={myVote === -1 ? 'Skipping' : 'Skip'} onPress={()
=> vote(trip.id, a.id, -1)} />
            <Action icon="people-outline" label="Assign" onPress={() => onAssign(a)} />
            <Action icon="navigate-outline" label="Directions" onPress={directions} />
            <Action icon={a.done ? 'refresh-outline' : 'checkmark-done-outline'} label={a.done ? 'Undo' : 'Done'} onPress={() => update
(trip.id, a.id, { done: !a.done })} />
            <Action
              icon="swap-horizontal"
              label="Status"
              onPress={() => {
                const next = a.status === 'confirmed' ? 'voting' : a.status === 'voting' ? 'pending' : 'confirmed';
                update(trip.id, a.id, { status: next });
                toast(`Marked as ${next}`);
              }}
            />
            <Action icon="create-outline" label="Edit" onPress={() => onEdit(a)} />
          </View>
        </View>
      ) : null}
    </View>
  );
}

function Tag({ label, tone }: { label: string; tone?: 'warn' }) {
  return (
    <View style={[styles.tag, tone === 'warn' && { backgroundColor: colors.accentSoft }]}>
      <T variant="micro" weight="semibold" color={tone === 'warn' ? colors.onAccent : colors.textSecondary}>
        {label}
      </T>
    </View>
  );
}

function Action({ icon, label, onPress }: { icon: IconName; label: string; onPress: () => void }) {
  return (
    <Press onPress={onPress} style={styles.action} accessibilityLabel={label}>
      <View style={styles.actionIcon}>
        <Ionicons name={icon} size={16} color={colors.text} />
      </View>
      <T variant="micro" color={colors.textSecondary}>
        {label}
      </T>
    </Press>
  );
}

const styles = StyleSheet.create({
  card: { backgroundColor: colors.card, borderRadius: radius.xl, padding: 16, marginBottom: 12, ...shadow },
  top: { flexDirection: 'row', alignItems: 'flex-start', gap: 12 },
  badge: { width: 46, height: 46, borderRadius: 23, backgroundColor: colors.bgAlt, alignItems: 'center', justifyContent: 'center' },
  meta: { flexDirection: 'row', alignItems: 'center', flexWrap: 'wrap', columnGap: 10, rowGap: 2 },
  inline: { flexDirection: 'row', alignItems: 'center', gap: 4 },
  tags: { flexDirection: 'row', flexWrap: 'wrap', gap: 6, marginTop: 2 },
  tag: { backgroundColor: colors.bgAlt, borderRadius: 999, paddingHorizontal: 9, paddingVertical: 4 },
  bottom: { flexDirection: 'row', alignItems: 'center', gap: 8, marginTop: 14 },
  votePill: { flexDirection: 'row', alignItems: 'center', gap: 6, height: 34, paddingHorizontal: 12, borderRadius: 999 },
  votePillOn: { backgroundColor: colors.accent },
  votePillOff: { backgroundColor: colors.bgAlt },
  track: { flex: 1, height: 6, borderRadius: 3, backgroundColor: colors.bgAlt, overflow: 'hidden' },
  fill: { height: 6, borderRadius: 3, backgroundColor: colors.primary },
  lock: { flexDirection: 'row', alignItems: 'center', gap: 4, height: 34, paddingHorizontal: 14, borderRadius: 999, backgroundColor: col
ors.primary },
  bell: { width: 34, height: 34, borderRadius: 17, alignItems: 'center', justifyContent: 'center', backgroundColor: colors.bgAlt },
  bellOn: { backgroundColor: colors.primary },
  details: { marginTop: 14, paddingTop: 14, borderTopWidth: 1, borderTopColor: colors.border, gap: 12 },
  notes: { flexDirection: 'row', gap: 8, backgroundColor: colors.cardMuted, padding: 10, borderRadius: radius.sm },
  actions: { flexDirection: 'row', justifyContent: 'space-between' },
  action: { alignItems: 'center', gap: 4, minWidth: 44 },
  actionIcon: { width: 36, height: 36, borderRadius: 18, backgroundColor: colors.bgAlt, alignItems: 'center', justifyContent: 'center'
},
});
```

```
import { Ionicons } from '@expo/vector-icons';
import { useMemo, useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';

import { TimePickerSheet } from '@/components/pickers';
import { Button } from '@/components/ui/Button';
import { Chip, ToggleRow } from '@/components/ui/bits';
import { toast } from '@/components/ui/feedback';
import { Input } from '@/components/ui/Input';
import { tap } from '@/components/ui/Press';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import { useT } from '@/i18n';
import { ensurePermission } from '@/services/notifications';
import { useAppStore } from '@/store/useAppStore';
import type { Activity, ActivityType, Trip } from '@/store/types';
import { colors } from '@/theme';
import { durationLabel } from '@/utils/format';
import { dayLabel, time12, tripDays, weekdayShort } from '@/utils/time';
import { maxLen, parseAmount, required, validateAmount, firstError } from '@/utils/validation';

import { activityTypes } from './meta';

export interface ActivityDraft {
  title?: string;
  location?: string;
  type?: ActivityType;
  durationMin?: number;
  costPerPerson?: number;
  lat?: number;
  lng?: number;
  placeId?: string;
  notes?: string;
}

const DURATIONS = [30, 60, 90, 120, 180, 300, 480];

interface Props {
  visible: boolean;
  onClose: () => void;
  trip: Trip;
  date: string;
  editing?: Activity | null;
  draft?: ActivityDraft | null;
}

export function AddActivitySheet({ visible, onClose, trip, date, editing, draft }: Props) {
  const t = useT();
  const add = useAppStore((s) => s.addActivity);
  const update = useAppStore((s) => s.updateActivity);
  const userId = useAppStore((s) => s.session?.userId ?? '');
  const [type, setType] = useState<ActivityType>('sightseeing');
  const [day, setDay] = useState(date);
  const [time, setTime] = useState('10:00');
  const [title, setTitle] = useState('');
  const [location, setLocation] = useState('');
  const [duration, setDuration] = useState(90);
  const [cost, setCost] = useState('');
  const [notes, setNotes] = useState('');
  const [vote, setVote] = useState(false);
  const [remind, setRemind] = useState(true);
  const [timeOpen, setTimeOpen] = useState(false);
  const [errors, setErrors] = useState<Record<string, string | null>>({});

  // Reset the form whenever the sheet opens for a new activity / edit / draft.
  const [openedFor, setOpenedFor] = useState<string | object | null>(null);
  const openKey = visible ? (editing?.id ?? draft ?? 'new') : null;
  if (openKey !== openedFor) {
    setOpenedFor(openKey);
    if (visible) resetForm();
  }

  function resetForm() {
    const src = editing ?? draft ?? {};
    setType(src.type ?? 'sightseeing');
    setDay(editing?.date ?? date);
    setTime(editing?.time ?? '10:00');
    setTitle(src.title ?? '');
    setLocation(src.location ?? '');
    setDuration(src.durationMin ?? 90);
    setCost(src.costPerPerson ? String(src.costPerPerson) : '');
    setNotes(src.notes ?? '');
    setVote(editing ? editing.status === 'voting' : false);
    setRemind(editing ? editing.reminder : true);
```

```
    setErrors({});
  }

  const days = tripDays(trip.startDate, trip.endDate);

  // Semantic check: warn about overlapping plans on the same day.
  const clash = useMemo(() => {
    const [h, m] = time.split(':').map(Number);
    const start = h * 60 + m;
    const end = start + duration;
    return trip.activities.find((a) => {
      if (a.date !== day || a.id === editing?.id) return false;
      const [ah, am] = a.time.split(':').map(Number);
      const s = ah * 60 + am;
      return start < s + a.durationMin && s < end;
    });
  }, [trip.activities, day, time, duration, editing?.id]);

  const submit = async () => {
    const e = {
      title: firstError(title, required('an activity name'), maxLen(60, 'Activity name')),
      location: firstError(location, required('a location'), maxLen(60, 'Location')),
      cost: cost.trim() ? validateAmount(cost, { min: 0, max: 10000 }) : null,
      notes: maxLen(280, 'Notes')(notes),
    };
    setErrors(e);
    if (Object.values(e).some(Boolean)) return tap('warning');
    const payload = {
      date: day,
      time,
      title: title.trim(),
      location: location.trim(),
      type,
      durationMin: duration,
      costPerPerson: cost.trim() ? parseAmount(cost) ?? 0 : 0,
      notes: notes.trim() || undefined,
      status: vote ? ('voting' as const) : editing && editing.status !== 'voting' ? editing.status : ('confirmed' as const),
      reminder: remind,
    };
    if (remind) ensurePermission().catch(() => {});
    if (editing) {
      update(trip.id, editing.id, payload);
      toast('Activity updated');
    } else {
      add(trip.id, { ...payload, assigned: [userId], lat: draft?.lat, lng: draft?.lng, placeId: draft?.placeId });
      toast(`Added to ${dayLabel(day)}${vote ? ' — group vote started' : ''}`);
    }
    tap('success');
    onClose();
  };

  return (
    <Sheet
      visible={visible}
      onClose={onClose}
      title={editing ? 'Edit Activity' : t('add.title')}
      subtitle={t('add.subtitle')}
      footer={<Button label={editing ? t('add.save') : t('add.submit')} onPress={submit} testID="add-activity-submit" />}>
      <View style={{ gap: 8 }}>
        <T variant="small" weight="semibold">
          {t('add.type')}
        </T>
        <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 8 }}>
          {activityTypes.map((a) => (
            <Chip key={a.key} label={a.label} icon={a.icon} active={type === a.key} onPress={() => setType(a.key)} />
          ))}
        </ScrollView>
      </View>

      <View style={{ gap: 8 }}>
        <T variant="small" weight="semibold">
          Day
        </T>
        <ScrollView horizontal showsHorizontalScrollIndicator={false} contentContainerStyle={{ gap: 8 }}>
          {days.map((d, i) => (
            <Chip key={d} small label={`Day ${i + 1} · ${weekdayShort(d)} ${dayLabel(d)}`} active={day === d} onPress={() => setDay(d)}
/>
          ))}
        </ScrollView>
      </View>

      <Input
        label={t('add.time')}
        icon="time-outline"
        value={time12(time)}
        onPressField={() => setTimeOpen(true)}
```

```
        right={<Ionicons name="pencil" size={15} color={colors.textMuted} />}
      />

      <Input label={t('add.name')} placeholder={t('add.namePh')} value={title} onChangeText={setTitle} error={errors.title} maxLength={70} testID="activity-name" />
      <Input label={t('add.location')} placeholder={t('add.locationPh')} icon="location-sharp" value={location} onChangeText={setLocation} error={errors.location} maxLength={70} testID="activity-location" />

      <View style={{ gap: 8 }}>
        <T variant="small" weight="semibold">
          {t('add.duration')}
        </T>
        <View style={styles.wrap}>
          {DURATIONS.map((d) => (
            <Chip key={d} small label={d === 300 ? 'Half day' : d === 480 ? 'Full day' : durationLabel(d)} active={duration === d} onPress={() => setDuration(d)} />
          ))}
        </View>
      </View>

      {clash ? (
        <View style={styles.warn} accessibilityLiveRegion="polite">
          <Ionicons name="warning-outline" size={16} color="#B45A1C" />
          <T variant="caption" color="#B45A1C" style={{ flex: 1 }}>
            Overlaps with “{clash.title}” at {time12(clash.time)}. You can still add it.
          </T>
        </View>
      ) : null}

      <Input label={t('add.cost')} placeholder="0 = free" icon="cash-outline" value={cost} onChangeText={setCost} keyboardType="decimal-pad" error={errors.cost} />
      <Input label={t('add.notes')} placeholder={t('add.notesPh')} value={notes} onChangeText={setNotes} multiline error={errors.notes} maxLength={300} />

      <View>
        <ToggleRow icon="people-outline" label={t('add.vote')} sub="Everyone gets a vote; majority confirms it" value={vote} onChange={setVote} />
        <ToggleRow icon="notifications-outline" label={t('add.remind')} value={remind} onChange={setRemind} />
      </View>

      <TimePickerSheet visible={timeOpen} value={time} onClose={() => setTimeOpen(false)} onChange={setTime} />
    </Sheet>
  );
}

const styles = StyleSheet.create({
  wrap: { flexDirection: 'row', flexWrap: 'wrap', gap: 8 },
  warn: { flexDirection: 'row', gap: 8, alignItems: 'center', backgroundColor: colors.orangeSoft, padding: 10, borderRadius: 10 },
});
```

src/components/trip/MemberPickerSheet.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { StyleSheet, View } from 'react-native';

import { Avatar } from '@/components/ui/bits';
import { Press } from '@/components/ui/Press';
import { Sheet } from '@/components/ui/Sheet';
import { T } from '@/components/ui/T';
import type { Member } from '@/store/types';
import { colors, radius } from '@/theme';

/** Multi- or single-select list of trip members. */
export function MemberPickerSheet({
  visible,
  onClose,
  members,
  selected,
  onToggle,
  title = 'Assign people',
  single,
}: {
  visible: boolean;
  onClose: () => void;
  members: Member[];
  selected: string[];
  onToggle: (id: string) => void;
  title?: string;
  single?: boolean;
}) {
  return (
    <Sheet visible={visible} onClose={onClose} title={title} subtitle={single ? 'Choose one person' : 'Tap to add or remove'}>
      {members.map((m) => {
        const on = selected.includes(m.id);
        return (
          <Press
            key={m.id}
            onPress={() => {
              onToggle(m.id);
              if (single) onClose();
            }}
            style={[styles.row, on && styles.rowOn]}
            accessibilityRole="checkbox"
            accessibilityState={{ checked: on }}>
            <Avatar name={m.name} src={m.avatar} size={36} />
            <View style={{ flex: 1 }}>
              <T variant="bodySm" weight="semibold">
                {m.name}
              </T>
              <T variant="caption" color={colors.textSecondary}>
                {m.role}
              </T>
            </View>
            <Ionicons name={on ? 'checkmark-circle' : 'ellipse-outline'} size={22} color={on ? colors.primary : colors.borderStrong} />
          </Press>
        );
      })}
    </Sheet>
  );
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', alignItems: 'center', gap: 12, padding: 10, borderRadius: radius.md, borderWidth: 1, borderColor: colors.
border },
  rowOn: { borderColor: colors.primaryLine, backgroundColor: colors.primarySofter },
});
```

src/components/trip/NoTrip.tsx

```
import { router } from 'expo-router';
import { View } from 'react-native';

import { Button } from '@/components/ui/Button';
import { EmptyState } from '@/components/ui/bits';
import { Screen } from '@/components/ui/Screen';

/** Shown on trip tabs when the signed-in user isn't part of any trip yet. */
export function NoTrip({ title = 'No trip selected' }: { title?: string }) {
  return (
    <Screen contentStyle={{ flexGrow: 1, justifyContent: 'center' }}>
      <EmptyState icon="airplane-outline" title={title} body="Create a trip and invite your group, or join a friend’s trip with their in
vite code.">
        <View style={{ gap: 10, alignSelf: 'stretch', marginTop: 12 }}>
          <Button label="Create a trip" icon="add" onPress={() => router.push('/new-trip')} />
          <Button label="Join with a code" icon="people-outline" variant="secondary" onPress={() => router.push('/join')} />
        </View>
      </EmptyState>
    </Screen>
  );
}
```

src/components/trip/TripMenuSheet.tsx

```
import { router } from 'expo-router';

import { ListRow } from '@/components/ui/bits';
import { confirmAction, toast } from '@/components/ui/feedback';
import { Sheet } from '@/components/ui/Sheet';
import { printPlanner, sharePlannerPdf } from '@/services/print';
import { inviteMessage, shareText } from '@/services/share';
import { tripStatus, useAppStore } from '@/store/useAppStore';
import type { Trip } from '@/store/types';
import { colors } from '@/theme';

export function TripMenuSheet({ visible, onClose, trip }: { visible: boolean; onClose: () => void; trip: Trip }) {
  const userId = useAppStore((s) => s.session?.userId);
  const updateTrip = useAppStore((s) => s.updateTrip);
  const leaveTrip = useAppStore((s) => s.leaveTrip);
  const deleteTrip = useAppStore((s) => s.deleteTrip);
  const owner = trip.ownerId === userId;
  const status = tripStatus(trip);
  const go = (href: Parameters<typeof router.push>[0]) => {
    onClose();
    setTimeout(() => router.push(href), 180);
  };

  return (
    <Sheet visible={visible} onClose={onClose} title={trip.name} subtitle="Trip options">
      <ListRow icon="swap-horizontal" label="Switch or create trip" sub="All your group trips in one place" onPress={() => go('/trips')}
/>
      <ListRow icon="create-outline" label="Edit trip details" sub="Name, dates and group budget" onPress={() => go({ pathname: '/new-tr
ip', params: { edit: trip.id } })} />
      <ListRow icon="map-outline" label="Trip map" onPress={() => go('/map')} />
      <ListRow icon="airplane-outline" label="Flights & arrivals" onPress={() => go('/flights')} />
      <ListRow
        icon="print-outline"
        label="Print planner"
        sub="A clean day-by-day printout with checkboxes"
        onPress={async () => {
          onClose();
          try {
            await printPlanner(trip);
          } catch {
            toast('Allow pop-ups to print the planner', { tone: 'warn' });
          }
        }}
      />
      <ListRow
        icon="document-text-outline"
        label="Save planner as PDF"
        sub="Share it to Files, email or the group chat"
        onPress={async () => {
          onClose();
          await sharePlannerPdf(trip).catch(() => toast('Couldn’t create the PDF', { tone: 'warn' }));
        }}
      />
      <ListRow icon="share-social-outline" label="Share invite" sub={`Code ${trip.inviteCode}`} onPress={() => shareText(inviteMessage(t
rip.name, trip.inviteCode))} />
      <ListRow icon="trophy-outline" label="Trip recap" sub="Stats, highlights and memories" onPress={() => go('/recap')} />
      <ListRow
        icon={status === 'completed' ? 'refresh-outline' : 'flag-outline'}
        label={status === 'completed' ? 'Reopen trip' : 'Mark trip as complete'}
        sub={status === 'completed' ? 'Move it back to planning' : 'Wrap up, settle balances and see the recap'}
        onPress={() => {
          const done = status !== 'completed';
          updateTrip(trip.id, { completed: done });
          onClose();
          toast(done ? 'Trip completed 🎉 — here’s your recap' : 'Trip reopened');
          if (done) setTimeout(() => router.push('/recap'), 250);
        }}
      />
      {owner ? (
        <ListRow
          icon="trash-outline"
          tint={colors.red}
          label="Delete trip"
          sub="Removes it for everyone on this device"
          onPress={() => {
            onClose();
            confirmAction('Delete this trip?', `“${trip.name}” and all its plans, expenses and chats will be deleted. This can’t be undo
ne.`, () => {
              deleteTrip(trip.id);
              toast('Trip deleted');
              router.navigate('/(tabs)/explore');
            }, { confirmLabel: 'Delete trip', destructive: true });
          }}
        />
```

```
      ) : (
        <ListRow
          icon="exit-outline"
          tint={colors.red}
          label="Leave trip"
          onPress={() => {
            onClose();
            confirmAction('Leave this trip?', 'You can rejoin later with the invite code.', () => {
              leaveTrip(trip.id);
              toast('You left the trip');
              router.navigate('/(tabs)/explore');
            }, { confirmLabel: 'Leave', destructive: true });
          }}
        />
      )}
    </Sheet>
  );
}
```

src/components/trip/meta.ts

```
import type { IconName } from '@/components/ui/bits';
import type { PlaceCategory } from '@/data/places';
import type { ActivityStatus, ActivityType, ExpenseCategory } from '@/store/types';
import { colors } from '@/theme';

export const activityTypes: { key: ActivityType; label: string; icon: IconName; color: string; bg: string }[] = [
  { key: 'transport', label: 'Transport', icon: 'train-outline', color: '#2B5FB0', bg: colors.blueSoft },
  { key: 'stay', label: 'Stay', icon: 'bed-outline', color: colors.purple, bg: colors.purpleSoft },
  { key: 'food', label: 'Food', icon: 'restaurant-outline', color: '#C2410C', bg: colors.orangeSoft },
  { key: 'sightseeing', label: 'Sightseeing', icon: 'camera-outline', color: colors.teal, bg: colors.tealSoft },
  { key: 'culture', label: 'Culture', icon: 'business-outline', color: '#8B5E0F', bg: colors.yellowSoft },
  { key: 'adventure', label: 'Adventure', icon: 'bonfire-outline', color: colors.primary, bg: colors.primarySoft },
  { key: 'nightlife', label: 'Nightlife', icon: 'musical-notes-outline', color: '#B4235A', bg: '#FCE4EE' },
  { key: 'shopping', label: 'Shopping', icon: 'bag-handle-outline', color: '#5B4AB8', bg: colors.purpleSoft },
];

export const typeMeta = (t: ActivityType) => activityTypes.find((x) => x.key === t) ?? activityTypes[3];

export const statusMeta: Record<ActivityStatus, { tone: 'green' | 'orange' | 'blue'; key: string; dot: string }> = {
  confirmed: { tone: 'green', key: 'itin.confirmed', dot: colors.primary },
  voting: { tone: 'orange', key: 'itin.voting', dot: colors.orange },
  pending: { tone: 'blue', key: 'itin.pending', dot: colors.blue },
};

/** Map place categories (Nearby) onto itinerary activity types. */
export const placeToActivityType: Record<PlaceCategory, ActivityType> = {
  food: 'food',
  sightseeing: 'sightseeing',
  adventure: 'adventure',
  culture: 'culture',
  nightlife: 'nightlife',
  shopping: 'shopping',
  relax: 'sightseeing',
};

export const placeCategoryIcon: Record<PlaceCategory, IconName> = {
  food: 'restaurant-outline',
  sightseeing: 'camera-outline',
  adventure: 'bonfire-outline',
  culture: 'business-outline',
  nightlife: 'musical-notes-outline',
  shopping: 'bag-handle-outline',
  relax: 'leaf-outline',
};

export const expenseCategories: { key: ExpenseCategory; icon: IconName }[] = [
  { key: 'Food', icon: 'restaurant-outline' },
  { key: 'Activities', icon: 'ticket-outline' },
  { key: 'Transport', icon: 'train-outline' },
  { key: 'Stays', icon: 'bed-outline' },
  { key: 'Flights', icon: 'airplane-outline' },
  { key: 'Shopping', icon: 'bag-handle-outline' },
  { key: 'Other', icon: 'ellipsis-horizontal' },
];

/** Filter chips for itinerary/nearby (the brief's "adventures, sightseeing, etc."). */
export const activityFilters: { key: 'all' | ActivityType; labelKey: string }[] = [
  { key: 'all', labelKey: 'cat.all' },
  { key: 'adventure', labelKey: 'cat.adventure' },
  { key: 'sightseeing', labelKey: 'cat.sightseeing' },
  { key: 'food', labelKey: 'cat.food' },
  { key: 'culture', labelKey: 'cat.culture' },
  { key: 'nightlife', labelKey: 'cat.nightlife' },
  { key: 'shopping', labelKey: 'cat.shopping' },
];

/** Emoji shown in the round icon of an activity card and in the filter chips. */
export const typeEmoji: Record<ActivityType, string> = {
  transport: '🚐',
  stay: '🏡',
  food: '🍝',
  sightseeing: '📷',
  culture: '🏛️',
  adventure: '🥾',
  nightlife: '🍸',
  shopping: '🛍️',
};
```

src/components/ui/Button.tsx

```
import { Ionicons } from '@expo/vector-icons';
import type { ComponentProps } from 'react';
import { LinearGradient } from 'expo-linear-gradient';
import { ActivityIndicator, StyleSheet, View, type StyleProp, type ViewStyle } from 'react-native';

import { colors, radius, shadow } from '@/theme';

import { Press } from './Press';
import { T } from './T';

type IconName = ComponentProps<typeof Ionicons>['name'];

export interface ButtonProps {
  label: string;
  onPress?: () => void;
  variant?: 'primary' | 'secondary' | 'soft' | 'ghost' | 'danger' | 'light';
  size?: 'lg' | 'md' | 'sm';
  icon?: IconName;
  iconRight?: IconName;
  loading?: boolean;
  disabled?: boolean;
  full?: boolean;
  style?: StyleProp<ViewStyle>;
  accessibilityHint?: string;
  testID?: string;
}

const palette = {
  primary: { bg: colors.primary, fg: colors.onPrimary, border: colors.primary },
  secondary: { bg: colors.card, fg: colors.text, border: colors.border },
  soft: { bg: colors.primarySoft, fg: colors.primary, border: colors.primarySoft },
  ghost: { bg: 'transparent', fg: colors.primary, border: 'transparent' },
  danger: { bg: colors.redSoft, fg: colors.red, border: colors.redSoft },
  light: { bg: 'rgba(255,255,255,0.18)', fg: colors.white, border: 'rgba(255,255,255,0.35)' },
};

export function Button({
  label,
  onPress,
  variant = 'primary',
  size = 'lg',
  icon,
  iconRight,
  loading,
  disabled,
  full = true,
  style,
  accessibilityHint,
  testID,
}: ButtonProps) {
  const p = palette[variant];
  const h = size === 'lg' ? 52 : size === 'md' ? 44 : 36;
  return (
    <Press
      testID={testID}
      accessibilityLabel={label}
      accessibilityHint={accessibilityHint}
      accessibilityState={{ disabled: !!disabled || !!loading, busy: !!loading }}
      disabled={disabled || loading}
      onPress={onPress}
      style={[
        styles.base,
        { height: h, backgroundColor: p.bg, borderColor: p.border, paddingHorizontal: size === 'sm' ? 14 : 20 },
        full && { alignSelf: 'stretch' },
        variant === 'primary' && [shadow, { overflow: 'hidden' as const }],
        (disabled || loading) && { opacity: 0.55 },
        style,
      ]}>
      {variant === 'primary' ? <LinearGradient colors={[colors.primary, colors.heroB]} start={{ x: 0, y: 0 }} end={{ x: 1, y: 1 }} style
={StyleSheet.absoluteFill} /> : null}
      {loading ? (
        <ActivityIndicator color={p.fg} />
      ) : (
        <View style={styles.row}>
          {icon && <Ionicons name={icon} size={size === 'sm' ? 15 : 18} color={p.fg} />}
          <T variant={size === 'sm' ? 'small' : 'title'} weight="semibold" color={p.fg}>
            {label}
          </T>
          {iconRight && <Ionicons name={iconRight} size={size === 'sm' ? 15 : 18} color={p.fg} />}
        </View>
      )}
    </Press>
  );
}
```

```
const styles = StyleSheet.create({
  base: {
    borderRadius: radius.pill,
    borderWidth: 1,
    alignItems: 'center',
    justifyContent: 'center',
  },
  row: { flexDirection: 'row', alignItems: 'center', gap: 8 },
});
```

src/components/ui/Donut.tsx

```
import { StyleSheet, View } from 'react-native';
import Svg, { Circle } from 'react-native-svg';

import { colors } from '@/theme';

import { T } from './T';

/** Ring chart of spending by category with the remaining budget in the middle. */
export function Donut({ parts, size = 132, stroke = 20, centerLabel, centerValue }: { parts: { value: number; color: string }[]; size?:
number; stroke?: number; centerLabel: string; centerValue: string }) {
  const r = (size - stroke) / 2;
  const c = 2 * Math.PI * r;
  const total = parts.reduce((s, p) => s + p.value, 0);
  let offset = 0;
  return (
    <View style={{ width: size, height: size }} accessibilityRole="image" accessibilityLabel={`${centerLabel} ${centerValue}`}>
      <Svg width={size} height={size} viewBox={`0 0 ${size} ${size}`} style={{ transform: [{ rotate: '-90deg' }] }}>
        <Circle cx={size / 2} cy={size / 2} r={r} stroke={colors.bgAlt} strokeWidth={stroke} fill="none" />
        {total > 0
          ? parts.map((p, i) => {
              const len = (p.value / total) * c;
              const el = <Circle key={i} cx={size / 2} cy={size / 2} r={r} stroke={p.color} strokeWidth={stroke} fill="none" strokeDasha
rray={`${Math.max(0, len - 2)} ${c}`} strokeDashoffset={-offset} />;
              offset += len;
              return el;
            })
          : null}
      </Svg>
      <View style={[StyleSheet.absoluteFill, { alignItems: 'center', justifyContent: 'center' }]}>
        <T variant="micro" color={colors.textSecondary} style={{ letterSpacing: 1 }}>
          {centerLabel}
        </T>
        <T variant="h3" weight="extrabold">
          {centerValue}
        </T>
      </View>
    </View>
  );
}
```

src/components/ui/Input.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { forwardRef, useState, type ComponentProps, type ReactNode } from 'react';
import { Platform, StyleSheet, TextInput, View, type StyleProp, type TextInputProps, type ViewStyle } from 'react-native';

import { useAppStore } from '@/store/useAppStore';
import { colors, fonts, radius } from '@/theme';

import { Press } from './Press';
import { T } from './T';

type IconName = ComponentProps<typeof Ionicons>['name'];

export interface InputProps extends TextInputProps {
  label?: string;
  /** Show the label inside the field (floating style from the sign-in screen). */
  inlineLabel?: boolean;
  icon?: IconName;
  error?: string | null;
  hint?: string;
  right?: ReactNode;
  secureToggle?: boolean;
  containerStyle?: StyleProp<ViewStyle>;
  onPressField?: () => void;
}

export const Input = forwardRef<TextInput, InputProps>(function Input(
  { label, inlineLabel, icon, error, hint, right, secureToggle, containerStyle, multiline, onPressField, style, secureTextEntry, onFocu
s, onBlur, ...rest },
  ref,
) {
  const [focused, setFocused] = useState(false);
  const [hidden, setHidden] = useState(true);
  const scale = useAppStore((s) => s.settings.textScale);
  const hasValue = !!rest.value;
  const borderColor = error ? colors.red : focused ? colors.primary : colors.border;

  const field = (
    <View
      style={[
        styles.field,
        multiline && styles.multiline,
        { borderColor },
        focused && !error && styles.focused,
        error && styles.errored,
      ]}>
      {icon && <Ionicons name={icon} size={18} color={error ? colors.red : colors.textMuted} style={{ marginRight: 10 }} />}
      <View style={{ flex: 1 }}>
        {inlineLabel && label && (hasValue || focused) ? (
          <T variant="micro" color={colors.textSecondary} style={{ marginBottom: 1 }}>
            {label}
          </T>
        ) : null}
        <TextInput
          ref={ref}
          placeholderTextColor={colors.textMuted}
          multiline={multiline}
          editable={!onPressField}
          pointerEvents={onPressField ? 'none' : 'auto'}
          secureTextEntry={secureToggle ? hidden : secureTextEntry}
          accessibilityLabel={label}
          {...rest}
          placeholder={inlineLabel && !hasValue && !focused ? label : rest.placeholder}
          onFocus={(e) => {
            setFocused(true);
            onFocus?.(e);
          }}
          onBlur={(e) => {
            setFocused(false);
            onBlur?.(e);
          }}
          style={[
            styles.input,
            { fontSize: 15 * scale },
            multiline && { minHeight: 64, textAlignVertical: 'top' },
            Platform.OS === 'web' && ({ outlineStyle: 'none' } as object),
            style,
          ]}
        />
      </View>
      {secureToggle && (
        <Press accessibilityLabel={hidden ? 'Show password' : 'Hide password'} onPress={() => setHidden((h) => !h)} hitSlop={10}>
          <Ionicons name={hidden ? 'eye-outline' : 'eye-off-outline'} size={19} color={colors.textMuted} />
        </Press>
      )}
```

```
      {right}
    </View>
  );

  return (
    <View style={[{ gap: 6 }, containerStyle]}>
      {label && !inlineLabel ? (
        <T variant="small" weight="semibold">
          {label}
        </T>
      ) : null}
      {onPressField ? (
        <Press onPress={onPressField} scaleTo={0.99} accessibilityLabel={label}>
          {field}
        </Press>
      ) : (
        field
      )}
      {error ? (
        <T variant="caption" color={colors.red} accessibilityLiveRegion="polite">
          {error}
        </T>
      ) : hint ? (
        <T variant="caption" color={colors.textMuted}>
          {hint}
        </T>
      ) : null}
    </View>
  );
});

const styles = StyleSheet.create({
  field: {
    minHeight: 50,
    borderRadius: radius.md,
    borderWidth: 1.2,
    backgroundColor: colors.cardMuted,
    paddingHorizontal: 14,
    flexDirection: 'row',
    alignItems: 'center',
  },
  multiline: { alignItems: 'flex-start', paddingVertical: 12 },
  focused: { backgroundColor: colors.white },
  errored: { backgroundColor: '#FFF8F8' },
  input: {
    fontFamily: fonts.regular,
    color: colors.text,
    paddingVertical: Platform.OS === 'ios' ? 12 : 8,
  },
});
```

src/components/ui/Press.tsx

```
import * as Haptics from 'expo-haptics';
import { Platform, Pressable, type PressableProps, type StyleProp, type ViewStyle } from 'react-native';

import { useAppStore } from '@/store/useAppStore';

export function tap(kind: 'light' | 'medium' | 'success' | 'warning' = 'light') {
  if (Platform.OS === 'web' || !useAppStore.getState().settings.haptics) return;
  if (kind === 'success') Haptics.notificationAsync(Haptics.NotificationFeedbackType.Success).catch(() => {});
  else if (kind === 'warning') Haptics.notificationAsync(Haptics.NotificationFeedbackType.Warning).catch(() => {});
  else Haptics.impactAsync(kind === 'medium' ? Haptics.ImpactFeedbackStyle.Medium : Haptics.ImpactFeedbackStyle.Light).catch(() => {});
}

export interface PressProps extends Omit<PressableProps, 'style'> {
  style?: StyleProp<ViewStyle>;
  /** Scale applied while pressed. */
  scaleTo?: number;
  haptic?: boolean;
}

/** Pressable with a subtle press-in scale + optional haptic tick. */
export function Press({ style, scaleTo = 0.97, haptic = true, onPress, accessibilityRole = 'button', ...rest }: PressProps) {
  const state = rest.accessibilityState;
  return (
    <Pressable
      accessibilityRole={accessibilityRole}
      // Mirror the state as ARIA props so web screen readers announce selection too.
      aria-selected={state?.selected}
      aria-checked={state?.checked}
      {...rest}
      onPress={(e) => {
        if (haptic) tap();
        onPress?.(e);
      }}
      style={({ pressed }) => [style, pressed && !rest.disabled && { transform: [{ scale: scaleTo }], opacity: 0.92 }]}
    />
  );
}
```

src/components/ui/Screen.tsx

```
import { Ionicons } from '@expo/vector-icons';
import { LinearGradient } from 'expo-linear-gradient';
import { router } from 'expo-router';
import type { ReactNode } from 'react';
import { RefreshControl, ScrollView, StyleSheet, View, type StyleProp, type ViewStyle } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { MAX_WIDTH, appearance, colors } from '@/theme';

import { Press } from './Press';
import { FadeIn } from './motion';
import { T } from './T';

/** Page container: cream background, safe-area padding and a phone-width column on big screens. */
export function Screen({
  children,
  scroll = true,
  padded = true,
  contentStyle,
  header,
  footer,
  bottomInset = 24,
  onRefresh,
  refreshing,
}: {
  children: ReactNode;
  scroll?: boolean;
  padded?: boolean;
  contentStyle?: StyleProp<ViewStyle>;
  header?: ReactNode;
  footer?: ReactNode;
  bottomInset?: number;
  onRefresh?: () => void;
  refreshing?: boolean;
}) {
  const insets = useSafeAreaInsets();
  const inner = [padded && { paddingHorizontal: 20 }, { paddingBottom: bottomInset + (footer ? 0 : insets.bottom) }, contentStyle];
  return (
    <View style={[styles.root, { paddingTop: insets.top }]}>
      {/* Soft wash of the theme colour behind the top of every page (the flat Amalfi look skips it). */}
      {appearance.theme !== 'amalfi' ? (
        <>
          <LinearGradient pointerEvents="none" colors={[colors.primarySoft, colors.primarySofter, colors.bg]} locations={[0, 0.45, 1]} s
tyle={styles.wash} />
          <View pointerEvents="none" style={styles.glow} />
        </>
      ) : null}
      <View style={styles.column}>
        {header}
        {scroll ? (
          <ScrollView
            style={{ flex: 1 }}
            contentContainerStyle={inner}
            keyboardShouldPersistTaps="handled"
            showsVerticalScrollIndicator={false}
            refreshControl={onRefresh ? <RefreshControl refreshing={!!refreshing} onRefresh={onRefresh} tintColor={colors.primary} /> :
undefined}>
            <FadeIn duration={360} distance={12}>
              {children}
            </FadeIn>
          </ScrollView>
        ) : (
          <View style={[{ flex: 1 }, inner]}>{children}</View>
        )}
        {footer}
      </View>
    </View>
  );
}

/** Stack-screen header: blue back link, big condensed title + subtitle, optional right actions. */
export function Header({ title, subtitle, kicker, right, onBack, back = true, backLabel = 'Back' }: { title: string; subtitle?: string;
kicker?: string; right?: ReactNode; onBack?: () => void; back?: boolean; backLabel?: string }) {
  return (
    <View style={styles.header}>
      {back ? (
        <Press
          hitSlop={10}
          accessibilityLabel={`Go back to ${backLabel}`}
          style={styles.back}
          onPress={() => {
            if (onBack) onBack();
            else if (router.canGoBack()) router.back();
            else router.replace('/(tabs)/explore');
```

```
          }}>
          <Ionicons name="chevron-back" size={18} color={colors.blue} />
          <T variant="small" weight="semibold" color={colors.blue}>
            {backLabel}
          </T>
        </Press>
      ) : null}
      <View style={{ flexDirection: 'row', alignItems: 'flex-end', gap: 12 }}>
        <View style={{ flex: 1 }}>
          {kicker ? (
            <T variant="kicker" color={colors.textSecondary}>
              {kicker}
            </T>
          ) : null}
          <T variant="display" style={{ fontSize: 36, lineHeight: 40 }} numberOfLines={2} accessibilityRole="header">
            {title}
          </T>
          {subtitle ? (
            <T variant="small" color={colors.textSecondary} numberOfLines={2} style={{ marginTop: 2 }}>
              {subtitle}
            </T>
          ) : null}
        </View>
        {right}
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.bg, alignItems: 'center', overflow: 'hidden' },
  wash: { position: 'absolute', top: 0, left: 0, right: 0, height: 300 },
  glow: { position: 'absolute', top: -90, right: -70, width: 240, height: 240, borderRadius: 120, backgroundColor: colors.accent, opacit
y: 0.12 },
  column: { flex: 1, width: '100%', maxWidth: MAX_WIDTH },
  header: { paddingHorizontal: 20, paddingTop: 8, paddingBottom: 14, gap: 4 },
  back: { flexDirection: 'row', alignItems: 'center', alignSelf: 'flex-start', marginLeft: -4 },
});
```

src/components/ui/Segmented.tsx

```
import { StyleSheet, View } from 'react-native';

import { colors } from '@/theme';

import { Press } from './Press';
import { T } from './T';

/** Pill-shaped tab switcher: grey track, white selected pill (Itinerary | Vote, Crew | Scout AI…). */
export function Segmented<K extends string>({ options, value, onChange }: { options: { key: K; label: string }[]; value: K; onChange:
(k: K) => void }) {
  return (
    <View style={styles.track} accessibilityRole="tablist">
      {options.map((o) => {
        const on = o.key === value;
        return (
          <Press key={o.key} onPress={() => onChange(o.key)} style={[styles.item, on && styles.on]} accessibilityRole="tab" accessibilit
yState={{ selected: on }} accessibilityLabel={o.label} scaleTo={0.98}>
            <T variant="small" weight={on ? 'bold' : 'semibold'} color={on ? colors.text : colors.textSecondary}>
              {o.label}
            </T>
          </Press>
        );
      })}
    </View>
  );
}

const styles = StyleSheet.create({
  track: { flexDirection: 'row', backgroundColor: colors.bgAlt, borderRadius: 999, padding: 4 },
  item: { flex: 1, height: 38, alignItems: 'center', justifyContent: 'center', borderRadius: 999, flexDirection: 'row', gap: 6 },
  on: { backgroundColor: colors.card, boxShadow: '0px 1px 4px rgba(0,0,0,0.12)' } as object,
});
```

src/components/ui/Sheet.tsx

```
import { useEffect, useState, type ReactNode } from 'react';
import {
  Animated,
  Easing,
  KeyboardAvoidingView,
  Modal,
  Platform,
  Pressable,
  ScrollView,
  StyleSheet,
  View,
  useWindowDimensions,
} from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { MAX_WIDTH, colors, radius } from '@/theme';

import { T } from './T';

export interface SheetProps {
  visible: boolean;
  onClose: () => void;
  title?: string;
  subtitle?: string;
  children: ReactNode;
  footer?: ReactNode;
  /** Fraction of screen height the sheet may use. */
  maxHeight?: number;
  scroll?: boolean;
}

/** Bottom sheet with a drag-handle look, slide-up animation and a tappable backdrop. */
export function Sheet({ visible, onClose, title, subtitle, children, footer, maxHeight = 0.9, scroll = true }: SheetProps) {
  const { height } = useWindowDimensions();
  const insets = useSafeAreaInsets();
  const [mounted, setMounted] = useState(visible);
  const [anim] = useState(() => new Animated.Value(0));
  // Mount as soon as we're asked to show; unmount only after the close animation.
  if (visible && !mounted) setMounted(true);

  useEffect(() => {
    if (visible) {
      Animated.timing(anim, { toValue: 1, duration: 260, easing: Easing.out(Easing.cubic), useNativeDriver: Platform.OS !== 'web' }).sta
rt();
    } else if (mounted) {
      Animated.timing(anim, { toValue: 0, duration: 200, easing: Easing.in(Easing.cubic), useNativeDriver: Platform.OS !== 'web' }).star
t(() =>
        setMounted(false),
      );
    }
  }, [visible, anim, mounted]);

  if (!mounted) return null;

  const translateY = anim.interpolate({ inputRange: [0, 1], outputRange: [height * 0.6, 0] });

  return (
    <Modal transparent visible={mounted} onRequestClose={onClose} statusBarTranslucent animationType="none">
      <KeyboardAvoidingView behavior={Platform.OS === 'ios' ? 'padding' : undefined} style={{ flex: 1 }}>
        <Animated.View style={[StyleSheet.absoluteFill, { backgroundColor: colors.overlay, opacity: anim }]}>
          <Pressable style={{ flex: 1 }} onPress={onClose} accessibilityLabel="Close" accessibilityRole="button" />
        </Animated.View>
        <View style={styles.wrap} pointerEvents="box-none">
          <Animated.View
            accessibilityViewIsModal
            style={[styles.sheet, { maxHeight: height * maxHeight, paddingBottom: Math.max(insets.bottom, 16), transform: [{ translateY
}] }]}>
            <View style={styles.handle} />
            {title ? (
              <View style={{ paddingHorizontal: 22, paddingTop: 6, paddingBottom: 10 }}>
                <T variant="h2" accessibilityRole="header">
                  {title}
                </T>
                {subtitle ? (
                  <T variant="small" color={colors.textSecondary} style={{ marginTop: 2 }}>
                    {subtitle}
                  </T>
                ) : null}
              </View>
            ) : null}
            {scroll ? (
              <ScrollView
                keyboardShouldPersistTaps="handled"
                contentContainerStyle={{ paddingHorizontal: 22, paddingBottom: 12, gap: 16 }}
```

```
              showsVerticalScrollIndicator={false}>
              {children}
            </ScrollView>
          ) : (
            <View style={{ paddingHorizontal: 22, flexShrink: 1 }}>{children}</View>
          )}
          {footer ? <View style={{ paddingHorizontal: 22, paddingTop: 8 }}>{footer}</View> : null}
        </Animated.View>
      </View>
    </KeyboardAvoidingView>
  </Modal>
 );
}

const styles = StyleSheet.create({
 wrap: { flex: 1, justifyContent: 'flex-end', alignItems: 'center' },
 sheet: {
  width: '100%',
  maxWidth: MAX_WIDTH,
  backgroundColor: colors.card,
  borderTopLeftRadius: radius.xl + 4,
  borderTopRightRadius: radius.xl + 4,
 },
 handle: {
  alignSelf: 'center',
  width: 42,
  height: 5,
  borderRadius: 3,
  backgroundColor: colors.borderStrong,
  marginTop: 10,
  marginBottom: 10,
 },
});
```

src/components/ui/Slider.tsx

```
import { useState } from 'react';
import { StyleSheet, View, type GestureResponderEvent } from 'react-native';

import { colors } from '@/theme';

export interface SliderProps {
  value: number;
  min: number;
  max: number;
  step?: number;
  onChange: (v: number) => void;
  label: string;
  formatValue?: (v: number) => string;
}

/** Lightweight cross-platform slider (track + thumb) styled like the reference radius control. */
export function Slider({ value, min, max, step = 1, onChange, label, formatValue }: SliderProps) {
  const [width, setWidth] = useState(1);

  // Children ignore touches, so locationX is always relative to the track container.
  const update = (e: GestureResponderEvent) => {
    const ratio = Math.max(0, Math.min(1, e.nativeEvent.locationX / width));
    const next = Math.round((min + ratio * (max - min)) / step) * step;
    if (next !== value) onChange(next);
  };

  const pct = (value - min) / (max - min);
  return (
    <View
      accessible
      accessibilityRole="adjustable"
      accessibilityLabel={label}
      accessibilityValue={{ min, max, now: value, text: formatValue?.(value) }}
      accessibilityActions={[{ name: 'increment' }, { name: 'decrement' }]}
      onAccessibilityAction={(e) => {
        if (e.nativeEvent.actionName === 'increment') onChange(Math.min(max, value + step));
        if (e.nativeEvent.actionName === 'decrement') onChange(Math.max(min, value - step));
      }}
      style={styles.hit}
      onLayout={(e) => setWidth(e.nativeEvent.layout.width)}
      onStartShouldSetResponder={() => true}
      onMoveShouldSetResponder={() => true}
      onResponderTerminationRequest={() => false}
      onResponderGrant={update}
      onResponderMove={update}>
      <View pointerEvents="none" style={styles.track}>
        <View style={[styles.fill, { width: `${pct * 100}%` }]} />
      </View>
      <View pointerEvents="none" style={[styles.thumb, { left: pct * width - 11 }]} />
    </View>
  );
}

const styles = StyleSheet.create({
  hit: { height: 32, justifyContent: 'center' },
  track: { height: 6, borderRadius: 3, backgroundColor: colors.border, overflow: 'hidden' },
  fill: { height: '100%', backgroundColor: colors.primary, borderRadius: 3 },
  thumb: {
    position: 'absolute',
    width: 22,
    height: 22,
    borderRadius: 11,
    backgroundColor: colors.primary,
    borderWidth: 4,
    borderColor: colors.white,
    top: 5,
    shadowColor: '#000',
    shadowOpacity: 0.18,
    shadowRadius: 4,
    shadowOffset: { width: 0, height: 2 },
    elevation: 3,
  },
});
```

src/components/ui/T.tsx

```
import { StyleSheet, Text, type TextProps, type TextStyle } from 'react-native';

import { useAppStore } from '@/store/useAppStore';
import { colors, fonts } from '@/theme';

type Variant = 'display' | 'headline' | 'h1' | 'h2' | 'h3' | 'title' | 'body' | 'bodySm' | 'small' | 'caption' | 'kicker' | 'micro';
type Weight = 'regular' | 'medium' | 'semibold' | 'bold' | 'extrabold';

const variants: Record<Variant, TextStyle> = {
  display: { fontSize: 42, lineHeight: 46, fontFamily: fonts.display, letterSpacing: 0.3, textTransform: 'uppercase' },
  headline: { fontSize: 24, lineHeight: 28, fontFamily: fonts.display, letterSpacing: 0.3, textTransform: 'uppercase' },
  h1: { fontSize: 26, lineHeight: 32, fontFamily: fonts.bold, letterSpacing: -0.4 },
  h2: { fontSize: 22, lineHeight: 28, fontFamily: fonts.bold, letterSpacing: -0.3 },
  h3: { fontSize: 18, lineHeight: 24, fontFamily: fonts.semibold, letterSpacing: -0.2 },
  title: { fontSize: 15.5, lineHeight: 21, fontFamily: fonts.semibold },
  body: { fontSize: 15, lineHeight: 21, fontFamily: fonts.regular },
  bodySm: { fontSize: 14, lineHeight: 19, fontFamily: fonts.regular },
  small: { fontSize: 13, lineHeight: 18, fontFamily: fonts.regular },
  caption: { fontSize: 12, lineHeight: 16, fontFamily: fonts.regular },
  kicker: { fontSize: 11, lineHeight: 14, fontFamily: fonts.semibold, letterSpacing: 1.6, textTransform: 'uppercase' },
  micro: { fontSize: 10.5, lineHeight: 13, fontFamily: fonts.medium },
};

export interface TProps extends TextProps {
  variant?: Variant;
  weight?: Weight;
  color?: string;
  center?: boolean;
}

/** App text: applies the brand font, type scale and the user's text-size setting. */
export function T({ variant = 'body', weight, color, center, style, ...rest }: TProps) {
  const scale = useAppStore((s) => s.settings.textScale);
  const v = variants[variant];
  const scaled: TextStyle =
    scale === 1
      ? {}
      : { fontSize: (v.fontSize ?? 15) * scale, lineHeight: v.lineHeight ? v.lineHeight * scale : undefined };
  return (
    <Text
      maxFontSizeMultiplier={1.4}
      {...rest}
      style={[
        styles.base,
        v,
        scaled,
        weight && { fontFamily: fonts[weight] },
        color ? { color } : null,
        center && { textAlign: 'center' },
        style,
      ]}
    />
  );
}

const styles = StyleSheet.create({
  base: { color: colors.text },
});
```

```
/** Small presentational building blocks shared by every screen. */
import { Ionicons } from '@expo/vector-icons';
import { Image } from 'expo-image';
import type { ComponentProps, ReactNode } from 'react';
import { StyleSheet, Switch, View, type StyleProp, type ViewStyle } from 'react-native';

import { imageSource } from '@/data/images';
import { colors, isDark, radius, shadow } from '@/theme';
import { initials } from '@/utils/format';

import { Press } from './Press';
import { T } from './T';

export type IconName = ComponentProps<typeof Ionicons>['name'];

export function Card({ children, style, padded = true, onPress, accessibilityLabel }: { children: ReactNode; style?: StyleProp<ViewStyle
>; padded?: boolean; onPress?: () => void; accessibilityLabel?: string }) {
  const s = [styles.card, padded && { padding: 16 }, style];
  if (onPress)
    return (
      <Press onPress={onPress} style={s} scaleTo={0.985} accessibilityLabel={accessibilityLabel}>
        {children}
      </Press>
    );
  return <View style={s}>{children}</View>;
}

export function Chip({
  label,
  active,
  onPress,
  icon,
  image,
  small,
  tone,
  style,
}: {
  label: string;
  active?: boolean;
  onPress?: () => void;
  icon?: IconName;
  image?: string;
  small?: boolean;
  tone?: 'soft';
  style?: StyleProp<ViewStyle>;
}) {
  const bg = active ? colors.primary : tone === 'soft' ? colors.primarySofter : colors.card;
  const fg = active ? colors.white : colors.text;
  return (
    <Press
      onPress={onPress}
      accessibilityRole="button"
      accessibilityState={{ selected: !!active }}
      accessibilityLabel={label}
      style={[
        styles.chip,
        small && styles.chipSmall,
        { backgroundColor: bg, borderColor: active ? colors.primary : colors.border },
        !!image && { paddingLeft: 5 },
        style,
      ]}>
      {image ? <Image source={imageSource(image)} style={styles.chipImg} contentFit="cover" /> : null}
      {icon ? <Ionicons name={icon} size={small ? 13 : 15} color={fg} /> : null}
      <T variant={small ? 'caption' : 'small'} weight={active ? 'semibold' : 'medium'} color={fg} numberOfLines={1}>
        {label}
      </T>
    </Press>
  );
}

export function Avatar({ name, src, size = 34, ring }: { name: string; src?: string; size?: number; ring?: string }) {
  const palette = ['#DCEFD3', '#FDEBDD', '#E3ECFB', '#EEE8FB', '#FDF3DC', '#DDF3F0'];
  const bg = palette[(name.charCodeAt(0) + name.length) % palette.length];
  return (
    <View
      accessibilityLabel={name}
      style={{
        width: size,
        height: size,
        borderRadius: size / 2,
        backgroundColor: bg,
        alignItems: 'center',
        justifyContent: 'center',
        overflow: 'hidden',
```

```
        borderWidth: ring ? 2 : 0,
        borderColor: ring,
      }}>
      {src ? (
        <Image source={imageSource(src)} style={{ width: '100%', height: '100%' }} contentFit="cover" />
      ) : (
        <T variant={size > 40 ? 'title' : 'micro'} weight="bold" color={isDark ? '#1F4D25' : colors.primary} style={size <= 28 ? { fontS
ize: 9.5 } : null}>
          {initials(name)}
        </T>
      )}
    </View>
  );
}

export function AvatarStack({ people, max = 3, size = 28 }: { people: { name: string; avatar?: string }[]; max?: number; size?: number
}) {
  const shown = people.slice(0, max);
  const extra = people.length - shown.length;
  return (
    <View style={{ flexDirection: 'row', alignItems: 'center' }}>
      {shown.map((p, i) => (
        <View key={`${p.name}${i}`} style={{ marginLeft: i === 0 ? 0 : -size * 0.32 }}>
          <Avatar name={p.name} src={p.avatar} size={size} ring={colors.white} />
        </View>
      ))}
      {extra > 0 && (
        <View
          style={{
            marginLeft: -size * 0.32,
            width: size,
            height: size,
            borderRadius: size / 2,
            backgroundColor: colors.primarySoft,
            borderWidth: 2,
            borderColor: colors.white,
            alignItems: 'center',
            justifyContent: 'center',
          }}>
          <T variant="micro" weight="bold" color={colors.primary}>
            +{extra}
          </T>
        </View>
      )}
    </View>
  );
}

export function ProgressBar({ value, color = colors.primary, track = colors.border, height = 6 }: { value: number; color?: string; trac
k?: string; height?: number }) {
  const pct = Math.max(0, Math.min(1, value));
  return (
    <View
      accessibilityRole="progressbar"
      accessibilityValue={{ min: 0, max: 100, now: Math.round(pct * 100) }}
      style={{ height, borderRadius: height, backgroundColor: track, overflow: 'hidden' }}>
      <View style={{ width: `${pct * 100}%`, height: '100%', backgroundColor: color, borderRadius: height }} />
    </View>
  );
}

export function IconButton({
  icon,
  onPress,
  label,
  size = 40,
  bg = colors.card,
  color = colors.text,
  badge,
  bordered = true,
}: {
  icon: IconName;
  onPress?: () => void;
  label: string;
  size?: number;
  bg?: string;
  color?: string;
  badge?: number | boolean;
  bordered?: boolean;
}) {
  return (
    <Press
      onPress={onPress}
      accessibilityLabel={label}
      hitSlop={6}
      style={{
```

```
        width: size,
        height: size,
        borderRadius: size / 2,
        backgroundColor: bg,
        alignItems: 'center',
        justifyContent: 'center',
        borderWidth: bordered ? 1 : 0,
        borderColor: colors.border,
      }}>
      <Ionicons name={icon} size={size * 0.46} color={color} />
      {badge ? (
        <View style={styles.badge}>
          {typeof badge === 'number' ? (
            <T variant="micro" weight="bold" color={colors.white} style={{ fontSize: 9 }}>
              {badge > 9 ? '9+' : badge}
            </T>
          ) : null}
        </View>
      ) : null}
    </Press>
  );
}

export function SectionHeader({ title, action, onAction, style }: { title: string; action?: string; onAction?: () => void; style?: StyleProp<ViewStyle> }) {
  return (
    <View style={[styles.sectionHeader, style]}>
      <T variant="headline" accessibilityRole="header" style={{ flexShrink: 1 }}>
        {title}
      </T>
      {action ? (
        <Press onPress={onAction} hitSlop={8} accessibilityLabel={action}>
          <T variant="small" weight="semibold" color={colors.textSecondary}>
            {action}
          </T>
        </Press>
      ) : null}
    </View>
  );
}

export function Pill({ label, tone = 'green', icon }: { label: string; tone?: 'green' | 'orange' | 'blue' | 'red' | 'gray' | 'yellow' | 'purple'; icon?: IconName }) {
  const map = {
    green: [colors.primarySoft, isDark ? '#6FD39A' : colors.primary],
    orange: [colors.orangeSoft, isDark ? colors.orange : '#B45A1C'],
    blue: [colors.blueSoft, isDark ? '#7FB0F0' : '#2B5FB0'],
    red: [colors.redSoft, colors.red],
    gray: [colors.bgAlt, colors.textSecondary],
    yellow: [colors.yellowSoft, isDark ? colors.yellow : '#94660A'],
    purple: [colors.purpleSoft, colors.purple],
  } as const;
  const [bg, fg] = map[tone];
  return (
    <View style={[styles.pill, { backgroundColor: bg }]}>
      {icon ? <Ionicons name={icon} size={11} color={fg} /> : null}
      <T variant="micro" weight="semibold" color={fg}>
        {label}
      </T>
    </View>
  );
}

export function EmptyState({ icon, title, body, children }: { icon: IconName; title: string; body?: string; children?: ReactNode }) {
  return (
    <View style={styles.empty}>
      <View style={styles.emptyIcon}>
        <Ionicons name={icon} size={26} color={colors.primary} />
      </View>
      <T variant="title" center>
        {title}
      </T>
      {body ? (
        <T variant="small" color={colors.textSecondary} center>
          {body}
        </T>
      ) : null}
      {children}
    </View>
  );
}

export function ToggleRow({ label, sub, value, onChange, icon }: { label: string; sub?: string; value: boolean; onChange: (v: boolean) => void; icon?: IconName }) {
  return (
    <View style={styles.toggleRow}>
```

```
      {icon ? (
        <View style={styles.rowIcon}>
          <Ionicons name={icon} size={17} color={colors.primary} />
        </View>
      ) : null}
      <View style={{ flex: 1 }}>
        <T variant="bodySm" weight="medium">
          {label}
        </T>
        {sub ? (
          <T variant="caption" color={colors.textSecondary}>
            {sub}
          </T>
        ) : null}
      </View>
      <Switch
        accessibilityLabel={label}
        value={value}
        onValueChange={onChange}
        trackColor={{ false: colors.borderStrong, true: colors.primary }}
        thumbColor={colors.white}
        {...({ activeThumbColor: colors.white } as object)}
      />
    </View>
  );
}

export function ListRow({ icon, label, sub, onPress, right, tint = colors.primary }: { icon: IconName; label: string; sub?: string; onPr
ess?: () => void; right?: ReactNode; tint?: string }) {
  return (
    <Press onPress={onPress} style={styles.listRow} scaleTo={0.99} accessibilityLabel={label}>
      <View style={[styles.rowIcon, { backgroundColor: tint === colors.primary ? colors.primarySofter : `${tint}1F` }]}>
        <Ionicons name={icon} size={17} color={tint} />
      </View>
      <View style={{ flex: 1 }}>
        <T variant="bodySm" weight="medium">
          {label}
        </T>
        {sub ? (
          <T variant="caption" color={colors.textSecondary} numberOfLines={2}>
            {sub}
          </T>
        ) : null}
      </View>
      {right ?? <Ionicons name="chevron-forward" size={16} color={colors.textMuted} />}
    </Press>
  );
}

export function Divider({ style }: { style?: StyleProp<ViewStyle> }) {
  return <View style={[{ height: 1, backgroundColor: colors.border }, style]} />;
}

const styles = StyleSheet.create({
  card: {
    backgroundColor: colors.card,
    borderRadius: radius.lg,
    borderWidth: 1,
    borderColor: colors.border,
    ...shadow,
  },
  chip: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 6,
    height: 38,
    paddingHorizontal: 14,
    borderRadius: radius.pill,
    borderWidth: 1,
  },
  chipSmall: { height: 30, paddingHorizontal: 11, gap: 4 },
  chipImg: { width: 28, height: 28, borderRadius: 14 },
  badge: {
    position: 'absolute',
    top: 5,
    right: 5,
    minWidth: 15,
    height: 15,
    paddingHorizontal: 3,
    borderRadius: 8,
    backgroundColor: colors.red,
    alignItems: 'center',
    justifyContent: 'center',
    borderWidth: 1.5,
    borderColor: colors.white,
  },
```

```
  sectionHeader: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', marginBottom: 12 },
  pill: { flexDirection: 'row', alignItems: 'center', gap: 3, paddingHorizontal: 8, paddingVertical: 3, borderRadius: radius.pill, align
Self: 'flex-start' },
  empty: { alignItems: 'center', gap: 8, paddingVertical: 36, paddingHorizontal: 24 },
  emptyIcon: {
    width: 60,
    height: 60,
    borderRadius: 30,
    backgroundColor: colors.primarySofter,
    alignItems: 'center',
    justifyContent: 'center',
    marginBottom: 6,
  },
  toggleRow: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingVertical: 10 },
  rowIcon: { width: 34, height: 34, borderRadius: 10, backgroundColor: colors.primarySofter, alignItems: 'center', justifyContent: 'cent
er' },
  listRow: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingVertical: 11 },
});
```

```
/**
 * Global toast + dialog system. Works the same on iOS, Android and web
 * (React Native's Alert is a no-op on web, so we render our own).
 */
import { Ionicons } from '@expo/vector-icons';
import { useEffect, useState } from 'react';
import { Animated, Modal, Platform, Pressable, StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';
import { create } from 'zustand';

import { colors, radius, shadowStrong } from '@/theme';

import { Button } from './Button';
import type { IconName } from './bits';
import { tap } from './Press';
import { T } from './T';

interface ToastState {
  toast: { id: number; text: string; icon: IconName; tone: 'ok' | 'info' | 'warn' } | null;
  show: (text: string, opts?: { icon?: IconName; tone?: 'ok' | 'info' | 'warn' }) => void;
  hide: () => void;
}

export const useToast = create<ToastState>((set) => ({
  toast: null,
  show: (text, opts) => set({ toast: { id: Date.now(), text, icon: opts?.icon ?? 'checkmark-circle', tone: opts?.tone ?? 'ok' } }),
  hide: () => set({ toast: null }),
}));

export const toast = (text: string, opts?: { icon?: IconName; tone?: 'ok' | 'info' | 'warn' }) => {
  if (opts?.tone === 'warn') tap('warning');
  useToast.getState().show(text, opts);
};

export function ToastHost() {
  const t = useToast((s) => s.toast);
  const hide = useToast((s) => s.hide);
  const insets = useSafeAreaInsets();
  const [anim] = useState(() => new Animated.Value(0));

  useEffect(() => {
    if (!t) return;
    anim.setValue(0);
    Animated.spring(anim, { toValue: 1, useNativeDriver: Platform.OS !== 'web', friction: 8 }).start();
    const timer = setTimeout(() => {
      Animated.timing(anim, { toValue: 0, duration: 180, useNativeDriver: Platform.OS !== 'web' }).start(() => hide());
    }, 2600);
    return () => clearTimeout(timer);
  }, [t, anim, hide]);

  if (!t) return null;
  const bg = t.tone === 'warn' ? '#3A1F1F' : colors.primaryDark;
  return (
    <Animated.View
      pointerEvents="box-none"
      style={[
        styles.toastWrap,
        { top: insets.top + 10, opacity: anim, transform: [{ translateY: anim.interpolate({ inputRange: [0, 1], outputRange: [-30, 0] })
}] },
      ]}>
      <Pressable onPress={hide} style={[styles.toast, { backgroundColor: bg }]} accessibilityLiveRegion="polite" accessibilityRole="aler
t">
        <Ionicons name={t.icon} size={18} color={t.tone === 'warn' ? '#FFB4B4' : '#BFE5AE'} />
        <T variant="small" weight="medium" color={colors.white} style={{ flex: 1 }}>
          {t.text}
        </T>
      </Pressable>
    </Animated.View>
  );
}

export interface DialogAction {
  label: string;
  style?: 'default' | 'cancel' | 'destructive';
  onPress?: () => void;
}

interface DialogState {
  dialog: { title: string; message?: string; actions: DialogAction[]; icon?: IconName } | null;
  open: (d: DialogState['dialog']) => void;
  close: () => void;
}

export const useDialog = create<DialogState>((set) => ({
```

```
  dialog: null,
  open: (d) => set({ dialog: d }),
  close: () => set({ dialog: null }),
}));

export function showDialog(title: string, message?: string, actions: DialogAction[] = [{ label: 'OK' }], icon?: IconName) {
  useDialog.getState().open({ title, message, actions, icon });
}

export function confirmAction(
  title: string,
  message: string,
  onConfirm: () => void,
  opts: { confirmLabel?: string; destructive?: boolean } = {},
) {
  showDialog(
    title,
    message,
    [
      { label: 'Cancel', style: 'cancel' },
      { label: opts.confirmLabel ?? 'Confirm', style: opts.destructive ? 'destructive' : 'default', onPress: onConfirm },
    ],
    opts.destructive ? 'warning-outline' : 'help-circle-outline',
  );
}

export function DialogHost() {
  const d = useDialog((s) => s.dialog);
  const close = useDialog((s) => s.close);
  if (!d) return null;
  return (
    <Modal transparent visible animationType="fade" onRequestClose={close} statusBarTranslucent>
      <View style={styles.dialogBackdrop}>
        <View style={styles.dialog} accessibilityViewIsModal accessibilityRole="alert">
          {d.icon ? (
            <View style={styles.dialogIcon}>
              <Ionicons name={d.icon} size={24} color={colors.primary} />
            </View>
          ) : null}
          <T variant="h3" center>
            {d.title}
          </T>
          {d.message ? (
            <T variant="bodySm" color={colors.textSecondary} center style={{ marginTop: 6 }}>
              {d.message}
            </T>
          ) : null}
          <View style={{ gap: 8, marginTop: 18, alignSelf: 'stretch' }}>
            {d.actions.map((a) => (
              <Button
                key={a.label}
                label={a.label}
                size="md"
                variant={a.style === 'cancel' ? 'secondary' : a.style === 'destructive' ? 'danger' : 'primary'}
                onPress={() => {
                  close();
                  a.onPress?.();
                }}
              />
            ))}
          </View>
        </View>
      </View>
    </Modal>
  );
}

const styles = StyleSheet.create({
  toastWrap: { position: 'absolute', left: 16, right: 16, alignItems: 'center', zIndex: 1000 },
  toast: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 10,
    paddingHorizontal: 16,
    paddingVertical: 13,
    borderRadius: radius.md,
    maxWidth: 480,
    width: '100%',
    ...shadowStrong,
  },
  dialogBackdrop: { flex: 1, backgroundColor: colors.overlay, alignItems: 'center', justifyContent: 'center', padding: 28 },
  dialog: {
    width: '100%',
    maxWidth: 360,
    backgroundColor: colors.card,
    borderRadius: radius.xl,
```

```
    padding: 22,
    alignItems: 'center',
    ...shadowStrong,
  },
  dialogIcon: {
    width: 48,
    height: 48,
    borderRadius: 24,
    backgroundColor: colors.primarySofter,
    alignItems: 'center',
    justifyContent: 'center',
    marginBottom: 10,
  },
});
```

src/components/ui/motion.tsx

```
/**
 * Small motion kit built on React Native's Animated API.
 * Every animation respects the "Animations" switch in Make it yours / Settings
 * (people who get motion-sick, or just want a calmer app, can turn it off).
 */
import { useEffect, useState, type ReactNode } from 'react';
import { Animated, Easing, Platform, type StyleProp, type ViewStyle } from 'react-native';

import { useAppStore } from '@/store/useAppStore';

export const nativeDriver = Platform.OS !== 'web';

export function useMotion() {
  return useAppStore((s) => s.personal.motion);
}

/** Fades + slides its children in once, after `delay` ms. Use for staggered entrances. */
export function FadeIn({
  children,
  delay = 0,
  from = 'up',
  distance = 18,
  duration = 480,
  style,
}: {
  children: ReactNode;
  delay?: number;
  from?: 'up' | 'down' | 'left' | 'right' | 'scale';
  distance?: number;
  duration?: number;
  style?: StyleProp<ViewStyle>;
}) {
  const motion = useMotion();
  const [v] = useState(() => new Animated.Value(motion ? 0 : 1));
  useEffect(() => {
    if (!motion) return;
    const anim = Animated.timing(v, {
      toValue: 1,
      duration,
      delay,
      easing: Easing.out(Easing.cubic),
      useNativeDriver: nativeDriver,
    });
    anim.start();
    return () => anim.stop();
  }, [v, delay, duration, motion]);

  const offset = v.interpolate({
    inputRange: [0, 1],
    outputRange: [from === 'down' || from === 'right' ? -distance : distance, 0],
  });
  const transform =
    from === 'scale'
      ? [
          {
            scale: v.interpolate({
              inputRange: [0, 1],
              outputRange: [0.86, 1],
            }),
          },
        ]
      : from === 'left' || from === 'right'
        ? [{ translateX: offset }]
        : [{ translateY: offset }];
  return <Animated.View style={[style, { opacity: v, transform }]}>{children}</Animated.View>;
}

/** A 0 → 1 value that loops forever (or sits at `rest` when motion is off). */
export function useLoop(duration: number, { delay = 0, rest = 0.5, pingPong = false }: { delay?: number; rest?: number; pingPong?: boole
an } = {}) {
  const motion = useMotion();
  const [v] = useState(() => new Animated.Value(motion ? 0 : rest));
  useEffect(() => {
    if (!motion) {
      v.setValue(rest);
      return;
    }
    v.setValue(0);
    const one = Animated.timing(v, {
      toValue: 1,
      duration,
      easing: pingPong ? Easing.inOut(Easing.sin) : Easing.linear,
      useNativeDriver: nativeDriver,
    });
```

```
    const seq = pingPong
      ? Animated.sequence([
          one,
          Animated.timing(v, {
            toValue: 0,
            duration,
            easing: Easing.inOut(Easing.sin),
            useNativeDriver: nativeDriver,
          }),
        ])
      : Animated.sequence([
          one,
          Animated.timing(v, {
            toValue: 0,
            duration: 0,
            useNativeDriver: nativeDriver,
          }),
        ]);
    const loop = Animated.loop(seq);
    const timer = setTimeout(() => loop.start(), delay);
    return () => {
      clearTimeout(timer);
      loop.stop();
    };
  }, [v, duration, delay, rest, pingPong, motion]);
  return v;
}

/** Gently bobs its children up and down. */
export function Float({ children, amount = 6, duration = 2200, delay = 0, style }: { children: ReactNode; amount?: number; duration?: nu
mber; delay?: number; style?: StyleProp<ViewStyle> }) {
  const v = useLoop(duration, { delay, pingPong: true });
  const translateY = v.interpolate({
    inputRange: [0, 1],
    outputRange: [-amount / 2, amount / 2],
  });
  return <Animated.View style={[style, { transform: [{ translateY }] }]}>{children}</Animated.View>;
}

/** Soft pulsing glow (scale + opacity) — great behind icons and "live" dots. */
export function Pulse({ children, scale = 1.12, duration = 1400, style }: { children?: ReactNode; scale?: number; duration?: number; sty
le?: StyleProp<ViewStyle> }) {
  const v = useLoop(duration, { pingPong: true, rest: 0 });
  return (
    <Animated.View
      style={[
        style,
        {
          opacity: v.interpolate({
            inputRange: [0, 1],
            outputRange: [1, 0.75],
          }),
          transform: [
            {
              scale: v.interpolate({
                inputRange: [0, 1],
                outputRange: [1, scale],
              }),
            },
          ],
        },
      ]}>
      {children}
    </Animated.View>
  );
}
```

src/data/airports.ts

```
/** Major international airports (IATA, coordinates, IANA time zone). Works fully offline. */
export interface Airport {
  iata: string;
  name: string;
  city: string;
  country: string;
  lat: number;
  lng: number;
  tz: string;
}

const A = (iata: string, name: string, city: string, country: string, lat: number, lng: number, tz: string): Airport => ({
  iata, name, city, country, lat, lng, tz,
});

export const airports: Airport[] = [
  // Japan & East Asia
  A('HND', 'Haneda Airport', 'Tokyo', 'Japan', 35.5494, 139.7798, 'Asia/Tokyo'),
  A('NRT', 'Narita International', 'Tokyo', 'Japan', 35.772, 140.3929, 'Asia/Tokyo'),
  A('KIX', 'Kansai International', 'Osaka', 'Japan', 34.4347, 135.244, 'Asia/Tokyo'),
  A('ITM', 'Osaka Itami', 'Osaka', 'Japan', 34.7855, 135.4382, 'Asia/Tokyo'),
  A('NGO', 'Chubu Centrair', 'Nagoya', 'Japan', 34.8584, 136.8054, 'Asia/Tokyo'),
  A('CTS', 'New Chitose', 'Sapporo', 'Japan', 42.7752, 141.6923, 'Asia/Tokyo'),
  A('FUK', 'Fukuoka Airport', 'Fukuoka', 'Japan', 33.5859, 130.4511, 'Asia/Tokyo'),
  A('OKA', 'Naha Airport', 'Okinawa', 'Japan', 26.1958, 127.6459, 'Asia/Tokyo'),
  A('ICN', 'Incheon International', 'Seoul', 'South Korea', 37.4602, 126.4407, 'Asia/Seoul'),
  A('GMP', 'Gimpo International', 'Seoul', 'South Korea', 37.5583, 126.7906, 'Asia/Seoul'),
  A('PEK', 'Beijing Capital', 'Beijing', 'China', 40.0799, 116.6031, 'Asia/Shanghai'),
  A('PVG', 'Shanghai Pudong', 'Shanghai', 'China', 31.1443, 121.8083, 'Asia/Shanghai'),
  A('HKG', 'Hong Kong International', 'Hong Kong', 'China', 22.308, 113.9185, 'Asia/Hong_Kong'),
  A('TPE', 'Taoyuan International', 'Taipei', 'Taiwan', 25.0797, 121.2342, 'Asia/Taipei'),
  // South-East Asia & India
  A('SIN', 'Changi Airport', 'Singapore', 'Singapore', 1.3644, 103.9915, 'Asia/Singapore'),
  A('BKK', 'Suvarnabhumi', 'Bangkok', 'Thailand', 13.69, 100.7501, 'Asia/Bangkok'),
  A('DMK', 'Don Mueang', 'Bangkok', 'Thailand', 13.9126, 100.6067, 'Asia/Bangkok'),
  A('DPS', 'Ngurah Rai International', 'Bali', 'Indonesia', -8.7482, 115.1675, 'Asia/Makassar'),
  A('CGK', 'Soekarno–Hatta', 'Jakarta', 'Indonesia', -6.1256, 106.6559, 'Asia/Jakarta'),
  A('KUL', 'Kuala Lumpur International', 'Kuala Lumpur', 'Malaysia', 2.7456, 101.7099, 'Asia/Kuala_Lumpur'),
  A('MNL', 'Ninoy Aquino International', 'Manila', 'Philippines', 14.5086, 121.0198, 'Asia/Manila'),
  A('HAN', 'Noi Bai International', 'Hanoi', 'Vietnam', 21.2187, 105.8042, 'Asia/Ho_Chi_Minh'),
  A('SGN', 'Tan Son Nhat', 'Ho Chi Minh City', 'Vietnam', 10.8188, 106.6519, 'Asia/Ho_Chi_Minh'),
  A('DEL', 'Indira Gandhi International', 'New Delhi', 'India', 28.5562, 77.1, 'Asia/Kolkata'),
  A('BOM', 'Chhatrapati Shivaji Maharaj', 'Mumbai', 'India', 19.0896, 72.8656, 'Asia/Kolkata'),
  A('BLR', 'Kempegowda International', 'Bengaluru', 'India', 13.1986, 77.7066, 'Asia/Kolkata'),
  A('MLE', 'Velana International', 'Malé', 'Maldives', 4.1918, 73.5291, 'Indian/Maldives'),
  // Middle East & Africa
  A('DXB', 'Dubai International', 'Dubai', 'UAE', 25.2532, 55.3657, 'Asia/Dubai'),
  A('DOH', 'Hamad International', 'Doha', 'Qatar', 25.2731, 51.6081, 'Asia/Qatar'),
  A('IST', 'Istanbul Airport', 'Istanbul', 'Türkiye', 41.2753, 28.7519, 'Europe/Istanbul'),
  A('CAI', 'Cairo International', 'Cairo', 'Egypt', 30.1219, 31.4056, 'Africa/Cairo'),
  A('RAK', 'Marrakesh Menara', 'Marrakech', 'Morocco', 31.6069, -8.0363, 'Africa/Casablanca'),
  A('CPT', 'Cape Town International', 'Cape Town', 'South Africa', -33.9715, 18.6021, 'Africa/Johannesburg'),
  A('JNB', 'O. R. Tambo International', 'Johannesburg', 'South Africa', -26.1367, 28.2411, 'Africa/Johannesburg'),
  A('NBO', 'Jomo Kenyatta International', 'Nairobi', 'Kenya', -1.3192, 36.9278, 'Africa/Nairobi'),
  // Europe
  A('LHR', 'Heathrow', 'London', 'United Kingdom', 51.47, -0.4543, 'Europe/London'),
  A('LGW', 'Gatwick', 'London', 'United Kingdom', 51.1537, -0.1821, 'Europe/London'),
  A('CDG', 'Charles de Gaulle', 'Paris', 'France', 49.0097, 2.5479, 'Europe/Paris'),
  A('ORY', 'Orly', 'Paris', 'France', 48.7262, 2.3652, 'Europe/Paris'),
  A('AMS', 'Schiphol', 'Amsterdam', 'Netherlands', 52.3105, 4.7683, 'Europe/Amsterdam'),
  A('FRA', 'Frankfurt Airport', 'Frankfurt', 'Germany', 50.0379, 8.5622, 'Europe/Berlin'),
  A('MUC', 'Munich Airport', 'Munich', 'Germany', 48.3537, 11.775, 'Europe/Berlin'),
  A('BER', 'Berlin Brandenburg', 'Berlin', 'Germany', 52.3667, 13.5033, 'Europe/Berlin'),
  A('ZRH', 'Zurich Airport', 'Zurich', 'Switzerland', 47.4582, 8.5555, 'Europe/Zurich'),
  A('GVA', 'Geneva Airport', 'Geneva', 'Switzerland', 46.2381, 6.109, 'Europe/Zurich'),
  A('VIE', 'Vienna International', 'Vienna', 'Austria', 48.1103, 16.5697, 'Europe/Vienna'),
  A('PRG', 'Václav Havel Airport', 'Prague', 'Czechia', 50.1008, 14.26, 'Europe/Prague'),
  A('FCO', 'Leonardo da Vinci–Fiumicino', 'Rome', 'Italy', 41.8003, 12.2389, 'Europe/Rome'),
  A('MXP', 'Milan Malpensa', 'Milan', 'Italy', 45.6306, 8.7281, 'Europe/Rome'),
  A('VCE', 'Venice Marco Polo', 'Venice', 'Italy', 45.5053, 12.3519, 'Europe/Rome'),
  A('PSA', 'Pisa International', 'Pisa', 'Italy', 43.6839, 10.3927, 'Europe/Rome'),
  A('NAP', 'Naples International', 'Naples', 'Italy', 40.886, 14.2908, 'Europe/Rome'),
  A('MAD', 'Adolfo Suárez Madrid–Barajas', 'Madrid', 'Spain', 40.4983, -3.5676, 'Europe/Madrid'),
  A('BCN', 'Josep Tarradellas Barcelona–El Prat', 'Barcelona', 'Spain', 41.2974, 2.0833, 'Europe/Madrid'),
  A('LIS', 'Humberto Delgado', 'Lisbon', 'Portugal', 38.7742, -9.1342, 'Europe/Lisbon'),
  A('OPO', 'Francisco Sá Carneiro', 'Porto', 'Portugal', 41.2481, -8.6814, 'Europe/Lisbon'),
  A('DUB', 'Dublin Airport', 'Dublin', 'Ireland', 53.4264, -6.2499, 'Europe/Dublin'),
  A('CPH', 'Copenhagen Airport', 'Copenhagen', 'Denmark', 55.618, 12.656, 'Europe/Copenhagen'),
  A('ARN', 'Stockholm Arlanda', 'Stockholm', 'Sweden', 59.6498, 17.9238, 'Europe/Stockholm'),
  A('OSL', 'Oslo Gardermoen', 'Oslo', 'Norway', 60.1976, 11.1004, 'Europe/Oslo'),
  A('KEF', 'Keflavík International', 'Reykjavík', 'Iceland', 63.985, -22.6056, 'Atlantic/Reykjavik'),
  A('ATH', 'Athens International', 'Athens', 'Greece', 37.9364, 23.9445, 'Europe/Athens'),
  A('JTR', 'Santorini (Thira)', 'Santorini', 'Greece', 36.3992, 25.4793, 'Europe/Athens'),
  // Oceania
```

```
  A('SYD', 'Sydney Kingsford Smith', 'Sydney', 'Australia', -33.9399, 151.1753, 'Australia/Sydney'),
  A('MEL', 'Melbourne Airport', 'Melbourne', 'Australia', -37.669, 144.841, 'Australia/Melbourne'),
  A('AKL', 'Auckland Airport', 'Auckland', 'New Zealand', -37.0082, 174.785, 'Pacific/Auckland'),
  // Americas
  A('JFK', 'John F. Kennedy International', 'New York', 'United States', 40.6413, -73.7781, 'America/New_York'),
  A('EWR', 'Newark Liberty', 'Newark', 'United States', 40.6895, -74.1745, 'America/New_York'),
  A('LGA', 'LaGuardia', 'New York', 'United States', 40.7769, -73.874, 'America/New_York'),
  A('PHL', 'Philadelphia International', 'Philadelphia', 'United States', 39.8744, -75.2424, 'America/New_York'),
  A('ILG', 'Wilmington Airport', 'Wilmington', 'United States', 39.6787, -75.6065, 'America/New_York'),
  A('BWI', 'Baltimore/Washington', 'Baltimore', 'United States', 39.1774, -76.6684, 'America/New_York'),
  A('IAD', 'Washington Dulles', 'Washington, D.C.', 'United States', 38.9531, -77.4565, 'America/New_York'),
  A('DCA', 'Reagan National', 'Washington, D.C.', 'United States', 38.8512, -77.0402, 'America/New_York'),
  A('BOS', 'Logan International', 'Boston', 'United States', 42.3656, -71.0096, 'America/New_York'),
  A('ATL', 'Hartsfield–Jackson', 'Atlanta', 'United States', 33.6407, -84.4277, 'America/New_York'),
  A('MCO', 'Orlando International', 'Orlando', 'United States', 28.4312, -81.3081, 'America/New_York'),
  A('MIA', 'Miami International', 'Miami', 'United States', 25.7959, -80.287, 'America/New_York'),
  A('CLT', 'Charlotte Douglas', 'Charlotte', 'United States', 35.214, -80.9431, 'America/New_York'),
  A('DTW', 'Detroit Metropolitan', 'Detroit', 'United States', 42.2162, -83.3554, 'America/Detroit'),
  A('ORD', "O'Hare International", 'Chicago', 'United States', 41.9742, -87.9073, 'America/Chicago'),
  A('DFW', 'Dallas/Fort Worth', 'Dallas', 'United States', 32.8998, -97.0403, 'America/Chicago'),
  A('IAH', 'George Bush Intercontinental', 'Houston', 'United States', 29.9902, -95.3368, 'America/Chicago'),
  A('MSP', 'Minneapolis–Saint Paul', 'Minneapolis', 'United States', 44.8848, -93.2223, 'America/Chicago'),
  A('DEN', 'Denver International', 'Denver', 'United States', 39.8561, -104.6737, 'America/Denver'),
  A('PHX', 'Phoenix Sky Harbor', 'Phoenix', 'United States', 33.4352, -112.0101, 'America/Phoenix'),
  A('LAS', 'Harry Reid International', 'Las Vegas', 'United States', 36.084, -115.1537, 'America/Los_Angeles'),
  A('LAX', 'Los Angeles International', 'Los Angeles', 'United States', 33.9416, -118.4085, 'America/Los_Angeles'),
  A('SFO', 'San Francisco International', 'San Francisco', 'United States', 37.6213, -122.379, 'America/Los_Angeles'),
  A('SEA', 'Seattle–Tacoma', 'Seattle', 'United States', 47.4502, -122.3088, 'America/Los_Angeles'),
  A('HNL', 'Daniel K. Inouye International', 'Honolulu', 'United States', 21.3245, -157.9251, 'Pacific/Honolulu'),
  A('YYZ', 'Toronto Pearson', 'Toronto', 'Canada', 43.6777, -79.6248, 'America/Toronto'),
  A('YVR', 'Vancouver International', 'Vancouver', 'Canada', 49.1967, -123.1815, 'America/Vancouver'),
  A('YYC', 'Calgary International', 'Calgary', 'Canada', 51.1215, -114.0076, 'America/Edmonton'),
  A('YUL', 'Montréal–Trudeau', 'Montreal', 'Canada', 45.4706, -73.7408, 'America/Toronto'),
  A('MEX', 'Benito Juárez International', 'Mexico City', 'Mexico', 19.4361, -99.0719, 'America/Mexico_City'),
  A('CUN', 'Cancún International', 'Cancún', 'Mexico', 21.0365, -86.877, 'America/Cancun'),
  A('GIG', 'Rio de Janeiro–Galeão', 'Rio de Janeiro', 'Brazil', -22.81, -43.2506, 'America/Sao_Paulo'),
  A('GRU', 'São Paulo–Guarulhos', 'São Paulo', 'Brazil', -23.4356, -46.4731, 'America/Sao_Paulo'),
  A('LIM', 'Jorge Chávez International', 'Lima', 'Peru', -12.0219, -77.1143, 'America/Lima'),
  A('CUZ', 'Alejandro Velasco Astete', 'Cusco', 'Peru', -13.5357, -71.9388, 'America/Lima'),
  A('BOG', 'El Dorado International', 'Bogotá', 'Colombia', 4.7016, -74.1469, 'America/Bogota'),
  A('EZE', 'Ministro Pistarini', 'Buenos Aires', 'Argentina', -34.8222, -58.5358, 'America/Argentina/Buenos_Aires'),
  A('SCL', 'Arturo Merino Benítez', 'Santiago', 'Chile', -33.393, -70.7858, 'America/Santiago'),
];

export const airportByIata = (code: string) =>
  airports.find((a) => a.iata === code.trim().toUpperCase());
```

src/data/destinations.ts

```
import type { PhotoKey } from './images';

export type DestinationTag =
  | 'best'
  | 'adventure'
  | 'food'
  | 'culture'
  | 'beach'
  | 'nature'
  | 'city'
  | 'nightlife'
  | 'romantic'
  | 'family';

export interface Destination {
  id: string;
  city: string;
  country: string;
  countryCode: string;
  image: PhotoKey;
  rating: number;
  reviews: number;
  lat: number;
  lng: number;
  tz: string;
  currency: string;
  airport: string;
  language: string;
  tags: DestinationTag[];
  /** Typical daily spend per traveller in USD (mid-range: food, transit, one paid sight). */
  dailyCost: number;
  priceLevel: 1 | 2 | 3 | 4;
  bestMonths: string;
  blurb: string;
  highlights: string[];
}

export const destinations: Destination[] = [
  {
    id: 'positano', city: 'Positano', country: 'Italy', countryCode: 'IT', image: 'amalfi', rating: 4.9, reviews: 9400,
    lat: 40.628, lng: 14.485, tz: 'Europe/Rome', currency: 'EUR', airport: 'NAP', language: 'Italian',
    tags: ['best', 'beach', 'romantic', 'food', 'adventure'], dailyCost: 230, priceLevel: 4, bestMonths: 'May–Jun, Sep–Oct',
    blurb: 'Pastel cliffside villages, lemon groves and boat trips on the Amalfi Coast.',
    highlights: ['Path of the Gods hike', 'Boat day to Capri', 'Fornillo Beach', 'Limoncello tasting', 'Sunset in Ravello'],
  },
  {
    id: 'tokyo', city: 'Tokyo', country: 'Japan', countryCode: 'JP', image: 'tokyo', rating: 4.9, reviews: 18420,
    lat: 35.6717, lng: 139.765, tz: 'Asia/Tokyo', currency: 'JPY', airport: 'HND', language: 'Japanese',
    tags: ['best', 'food', 'culture', 'city', 'nightlife', 'family'], dailyCost: 150, priceLevel: 3, bestMonths: 'Mar–May, Oct–Nov',
    blurb: 'Neon skylines, centuries-old shrines and the best food scene on earth — Tokyo rewards every kind of traveller.',
    highlights: ['Senso-ji Temple', 'Shibuya Crossing', 'Tsukiji Outer Market', 'teamLab Planets', 'Meiji Jingu'],
  },
  {
    id: 'kyoto', city: 'Kyoto', country: 'Japan', countryCode: 'JP', image: 'kyoto', rating: 4.8, reviews: 12210,
    lat: 35.0116, lng: 135.7681, tz: 'Asia/Tokyo', currency: 'JPY', airport: 'KIX', language: 'Japanese',
    tags: ['best', 'culture', 'nature', 'romantic'], dailyCost: 130, priceLevel: 3, bestMonths: 'Mar–Apr, Nov',
    blurb: 'Lantern-lit lanes, 1,600 temples and bamboo forests — Japan’s ancient capital moves at a gentler pace.',
    highlights: ['Fushimi Inari Taisha', 'Arashiyama Bamboo Grove', 'Kiyomizu-dera', 'Gion district', 'Nishiki Market'],
  },
  {
    id: 'paris', city: 'Paris', country: 'France', countryCode: 'FR', image: 'paris', rating: 4.8, reviews: 25110,
    lat: 48.8566, lng: 2.3522, tz: 'Europe/Paris', currency: 'EUR', airport: 'CDG', language: 'French',
    tags: ['best', 'food', 'culture', 'city', 'romantic'], dailyCost: 190, priceLevel: 4, bestMonths: 'Apr–Jun, Sep–Oct',
    blurb: 'Café terraces, world-class museums and the Seine at golden hour — Paris never gets old.',
    highlights: ['Eiffel Tower', 'Louvre Museum', 'Montmartre', 'Seine river cruise', 'Le Marais food walk'],
  },
  {
    id: 'bali', city: 'Bali', country: 'Indonesia', countryCode: 'ID', image: 'bali', rating: 4.7, reviews: 15330,
    lat: -8.3405, lng: 115.092, tz: 'Asia/Makassar', currency: 'IDR', airport: 'DPS', language: 'Indonesian',
    tags: ['beach', 'nature', 'adventure', 'culture', 'romantic'], dailyCost: 70, priceLevel: 1, bestMonths: 'Apr–Oct',
    blurb: 'Rice terraces, surf breaks and lake temples — Bali balances adventure and calm.',
    highlights: ['Ulun Danu Beratan Temple', 'Tegallalang Rice Terraces', 'Mount Batur sunrise trek', 'Uluwatu', 'Ubud Monkey Forest'],
  },
  {
    id: 'bangkok', city: 'Bangkok', country: 'Thailand', countryCode: 'TH', image: 'bangkok', rating: 4.7, reviews: 14200,
    lat: 13.7563, lng: 100.5018, tz: 'Asia/Bangkok', currency: 'THB', airport: 'BKK', language: 'Thai',
    tags: ['food', 'culture', 'city', 'nightlife'], dailyCost: 65, priceLevel: 1, bestMonths: 'Nov–Feb',
    blurb: 'Golden temples, floating markets and legendary street food at every corner.',
    highlights: ['Grand Palace', 'Wat Arun', 'Chatuchak Weekend Market', 'Chinatown street food', 'Chao Phraya boat'],
  },
  {
    id: 'singapore', city: 'Singapore', country: 'Singapore', countryCode: 'SG', image: 'singapore', rating: 4.9, reviews: 13020,
    lat: 1.3521, lng: 103.8198, tz: 'Asia/Singapore', currency: 'SGD', airport: 'SIN', language: 'English',
    tags: ['best', 'food', 'city', 'family'], dailyCost: 160, priceLevel: 3, bestMonths: 'Feb–Apr',
```

```
  blurb: 'A garden city of hawker centres, futuristic skylines and spotless streets.',
  highlights: ['Gardens by the Bay', 'Marina Bay Sands', 'Hawker centres', 'Sentosa Island', 'Little India'],
},
{
  id: 'cape-town', city: 'Cape Town', country: 'South Africa', countryCode: 'ZA', image: 'cape-town', rating: 4.8, reviews: 8800,
  lat: -33.9249, lng: 18.4241, tz: 'Africa/Johannesburg', currency: 'ZAR', airport: 'CPT', language: 'English',
  tags: ['adventure', 'nature', 'beach', 'food'], dailyCost: 90, priceLevel: 2, bestMonths: 'Nov–Mar',
  blurb: 'Table Mountain, penguins on the beach and winelands an hour away.',
  highlights: ['Table Mountain', 'Boulders Beach', 'Cape Point', 'V&A Waterfront', 'Stellenbosch'],
},
{
  id: 'new-york', city: 'New York', country: 'United States', countryCode: 'US', image: 'new-york', rating: 4.7, reviews: 30420,
  lat: 40.7128, lng: -74.006, tz: 'America/New_York', currency: 'USD', airport: 'JFK', language: 'English',
  tags: ['city', 'food', 'culture', 'nightlife', 'family'], dailyCost: 230, priceLevel: 4, bestMonths: 'Apr–Jun, Sep–Nov',
  blurb: 'Broadway, bagels and a skyline you've seen in a thousand films.',
  highlights: ['Central Park', 'Top of the Rock', 'Brooklyn Bridge', 'The Met', 'Broadway show'],
},
{
  id: 'santorini', city: 'Santorini', country: 'Greece', countryCode: 'GR', image: 'santorini', rating: 4.8, reviews: 9900,
  lat: 36.3932, lng: 25.4615, tz: 'Europe/Athens', currency: 'EUR', airport: 'JTR', language: 'Greek',
  tags: ['best', 'beach', 'romantic'], dailyCost: 180, priceLevel: 4, bestMonths: 'May–Jun, Sep',
  blurb: 'Whitewashed villages on a volcanic caldera with the Aegean's best sunsets.',
  highlights: ['Oia sunset', 'Red Beach', 'Caldera boat tour', 'Akrotiri ruins', 'Fira to Oia hike'],
},
{
  id: 'rome', city: 'Rome', country: 'Italy', countryCode: 'IT', image: 'rome', rating: 4.8, reviews: 22340,
  lat: 41.9028, lng: 12.4964, tz: 'Europe/Rome', currency: 'EUR', airport: 'FCO', language: 'Italian',
  tags: ['best', 'culture', 'food', 'city', 'romantic'], dailyCost: 160, priceLevel: 3, bestMonths: 'Apr–Jun, Sep–Oct',
  blurb: '2,700 years of history, perfect carbonara and a fountain on every piazza.',
  highlights: ['Colosseum', 'Vatican Museums', 'Trevi Fountain', 'Trastevere dinner', 'Pantheon'],
},
{
  id: 'london', city: 'London', country: 'United Kingdom', countryCode: 'GB', image: 'london', rating: 4.7, reviews: 26800,
  lat: 51.5072, lng: -0.1276, tz: 'Europe/London', currency: 'GBP', airport: 'LHR', language: 'English',
  tags: ['city', 'culture', 'family', 'nightlife'], dailyCost: 210, priceLevel: 4, bestMonths: 'May–Sep',
  blurb: 'Free museums, royal palaces and the Thames winding through it all.',
  highlights: ['Tower of London', 'British Museum', 'Borough Market', 'West End show', 'Camden'],
},
{
  id: 'iceland', city: 'Reykjavík', country: 'Iceland', countryCode: 'IS', image: 'iceland', rating: 4.9, reviews: 7300,
  lat: 64.1466, lng: -21.9426, tz: 'Atlantic/Reykjavik', currency: 'ISK', airport: 'KEF', language: 'Icelandic',
  tags: ['adventure', 'nature'], dailyCost: 240, priceLevel: 4, bestMonths: 'Jun–Aug (midnight sun), Sep–Mar (auroras)',
  blurb: 'Canyons, glaciers and northern lights — a road-trip country made for groups.',
  highlights: ['Golden Circle', 'Fjaðrárgljúfur canyon', 'Blue Lagoon', 'Glacier hike', 'Northern lights hunt'],
},
{
  id: 'swiss-alps', city: 'Lauterbrunnen', country: 'Switzerland', countryCode: 'CH', image: 'swiss-alps', rating: 4.9, reviews: 6100,
  lat: 46.5935, lng: 7.9091, tz: 'Europe/Zurich', currency: 'CHF', airport: 'ZRH', language: 'German',
  tags: ['best', 'adventure', 'nature', 'family'], dailyCost: 260, priceLevel: 4, bestMonths: 'Jun–Sep, Dec–Mar',
  blurb: 'A valley of 72 waterfalls under the Jungfrau — hiking and rail journeys at their best.',
  highlights: ['Staubbach Falls', 'Jungfraujoch', 'Mürren cliff walk', 'Paragliding Interlaken', 'Trümmelbach Falls'],
},
{
  id: 'maldives', city: 'Malé Atoll', country: 'Maldives', countryCode: 'MV', image: 'maldives', rating: 4.9, reviews: 5400,
  lat: 4.1755, lng: 73.5093, tz: 'Indian/Maldives', currency: 'MVR', airport: 'MLE', language: 'Dhivehi',
  tags: ['beach', 'romantic', 'nature'], dailyCost: 320, priceLevel: 4, bestMonths: 'Nov–Apr',
  blurb: 'Overwater villas and reef snorkelling straight from your deck.',
  highlights: ['Reef snorkelling', 'Sandbank picnic', 'Sunset dolphin cruise', 'Local island visit', 'Bioluminescent beach'],
},
{
  id: 'machu-picchu', city: 'Cusco', country: 'Peru', countryCode: 'PE', image: 'machu-picchu', rating: 4.9, reviews: 9800,
  lat: -13.1631, lng: -72.545, tz: 'America/Lima', currency: 'PEN', airport: 'CUZ', language: 'Spanish',
  tags: ['best', 'adventure', 'culture', 'nature'], dailyCost: 95, priceLevel: 2, bestMonths: 'May–Sep',
  blurb: 'The Inca citadel in the clouds, reached by train or a four-day trek.',
  highlights: ['Machu Picchu', 'Sacred Valley', 'Rainbow Mountain', 'Inca Trail', 'San Pedro Market'],
},
{
  id: 'ha-long-bay', city: 'Ha Long Bay', country: 'Vietnam', countryCode: 'VN', image: 'ha-long-bay', rating: 4.8, reviews: 8700,
  lat: 20.9101, lng: 107.1839, tz: 'Asia/Ho_Chi_Minh', currency: 'VND', airport: 'HAN', language: 'Vietnamese',
  tags: ['nature', 'adventure', 'romantic'], dailyCost: 75, priceLevel: 1, bestMonths: 'Mar–May, Sep–Nov',
  blurb: 'Emerald water and 1,600 limestone islands — best seen from a kayak.',
  highlights: ['Overnight junk cruise', 'Kayak to lagoons', 'Sung Sot Cave', 'Ti Top Island', 'Floating villages'],
},
{
  id: 'dubai', city: 'Dubai', country: 'United Arab Emirates', countryCode: 'AE', image: 'dubai', rating: 4.6, reviews: 16900,
  lat: 25.2048, lng: 55.2708, tz: 'Asia/Dubai', currency: 'AED', airport: 'DXB', language: 'Arabic',
  tags: ['city', 'adventure', 'family'], dailyCost: 200, priceLevel: 4, bestMonths: 'Nov–Mar',
  blurb: 'Record-breaking towers, desert dunes and souks of gold and spice.',
  highlights: ['Burj Khalifa', 'Desert safari', 'Dubai Marina', 'Gold Souk', 'Museum of the Future'],
},
{
  id: 'barcelona', city: 'Barcelona', country: 'Spain', countryCode: 'ES', image: 'barcelona', rating: 4.8, reviews: 21100,
  lat: 41.3874, lng: 2.1686, tz: 'Europe/Madrid', currency: 'EUR', airport: 'BCN', language: 'Spanish',
  tags: ['best', 'beach', 'food', 'culture', 'city', 'nightlife'], dailyCost: 150, priceLevel: 3, bestMonths: 'May–Jun, Sep',
  blurb: 'Gaudí architecture, tapas bars and a city beach in the same afternoon.',
  highlights: ['Sagrada Família', 'Park Güell', 'La Boqueria', 'Gothic Quarter', 'Barceloneta beach'],
```

```
  },
  {
    id: 'sydney', city: 'Sydney', country: 'Australia', countryCode: 'AU', image: 'sydney', rating: 4.8, reviews: 14100,
    lat: -33.8688, lng: 151.2093, tz: 'Australia/Sydney', currency: 'AUD', airport: 'SYD', language: 'English',
    tags: ['beach', 'city', 'adventure', 'family'], dailyCost: 190, priceLevel: 4, bestMonths: 'Sep–Nov, Mar–May',
    blurb: 'Harbour ferries, coastal walks and surf beaches minutes from the city.',
    highlights: ['Opera House', 'Bondi to Coogee walk', 'Harbour Bridge climb', 'Manly ferry', 'Blue Mountains'],
  },
  {
    id: 'seoul', city: 'Seoul', country: 'South Korea', countryCode: 'KR', image: 'seoul', rating: 4.8, reviews: 11800,
    lat: 37.5665, lng: 126.978, tz: 'Asia/Seoul', currency: 'KRW', airport: 'ICN', language: 'Korean',
    tags: ['food', 'city', 'culture', 'nightlife'], dailyCost: 120, priceLevel: 2, bestMonths: 'Apr–Jun, Sep–Nov',
    blurb: 'Palaces, K-BBQ, night markets and the coolest cafés in Asia.',
    highlights: ['Gyeongbokgung Palace', 'Bukchon Hanok Village', 'Myeongdong street food', 'N Seoul Tower', 'Hongdae nightlife'],
  },
  {
    id: 'istanbul', city: 'Istanbul', country: 'Türkiye', countryCode: 'TR', image: 'istanbul', rating: 4.7, reviews: 13500,
    lat: 41.0082, lng: 28.9784, tz: 'Europe/Istanbul', currency: 'TRY', airport: 'IST', language: 'Turkish',
    tags: ['culture', 'food', 'city'], dailyCost: 85, priceLevel: 2, bestMonths: 'Apr–May, Sep–Nov',
    blurb: 'Where Europe meets Asia: mosques, bazaars and Bosphorus ferries.',
    highlights: ['Blue Mosque', 'Hagia Sophia', 'Grand Bazaar', 'Bosphorus cruise', 'Galata Tower'],
  },
  {
    id: 'amsterdam', city: 'Amsterdam', country: 'Netherlands', countryCode: 'NL', image: 'amsterdam', rating: 4.7, reviews: 15200,
    lat: 52.3676, lng: 4.9041, tz: 'Europe/Amsterdam', currency: 'EUR', airport: 'AMS', language: 'Dutch',
    tags: ['city', 'culture', 'romantic'], dailyCost: 180, priceLevel: 3, bestMonths: 'Apr–May, Sep',
    blurb: 'Canal-side bikes, golden-age art and the coziest cafés in Europe.',
    highlights: ['Rijksmuseum', 'Anne Frank House', 'Canal cruise', 'Vondelpark', 'Jordaan'],
  },
  {
    id: 'prague', city: 'Prague', country: 'Czechia', countryCode: 'CZ', image: 'prague', rating: 4.8, reviews: 12400,
    lat: 50.0755, lng: 14.4378, tz: 'Europe/Prague', currency: 'CZK', airport: 'PRG', language: 'Czech',
    tags: ['culture', 'city', 'romantic', 'nightlife'], dailyCost: 95, priceLevel: 2, bestMonths: 'Apr–Jun, Sep–Oct',
    blurb: 'A fairytale old town of spires and the Charles Bridge at dawn.',
    highlights: ['Charles Bridge', 'Prague Castle', 'Old Town Square', 'Astronomical Clock', 'Letná beer garden'],
  },
  {
    id: 'lisbon', city: 'Lisbon', country: 'Portugal', countryCode: 'PT', image: 'lisbon', rating: 4.8, reviews: 13900,
    lat: 38.7223, lng: -9.1393, tz: 'Europe/Lisbon', currency: 'EUR', airport: 'LIS', language: 'Portuguese',
    tags: ['food', 'culture', 'city', 'beach'], dailyCost: 120, priceLevel: 2, bestMonths: 'Mar–Jun, Sep–Oct',
    blurb: 'Yellow trams, pastel de nata and miradouros over terracotta rooftops.',
    highlights: ['Tram 28', 'Belém Tower', 'Alfama fado night', 'Sintra day trip', 'Time Out Market'],
  },
  {
    id: 'banff', city: 'Banff', country: 'Canada', countryCode: 'CA', image: 'banff', rating: 4.9, reviews: 8200,
    lat: 51.1784, lng: -115.5708, tz: 'America/Edmonton', currency: 'CAD', airport: 'YYC', language: 'English',
    tags: ['adventure', 'nature', 'family'], dailyCost: 170, priceLevel: 3, bestMonths: 'Jun–Sep',
    blurb: 'Turquoise glacier lakes framed by the Canadian Rockies.',
    highlights: ['Moraine Lake', 'Lake Louise', 'Icefields Parkway', 'Johnston Canyon', 'Banff Gondola'],
  },
  {
    id: 'marrakech', city: 'Marrakech', country: 'Morocco', countryCode: 'MA', image: 'marrakech', rating: 4.6, reviews: 9100,
    lat: 31.6295, lng: -7.9811, tz: 'Africa/Casablanca', currency: 'MAD', airport: 'RAK', language: 'Arabic',
    tags: ['culture', 'food', 'adventure'], dailyCost: 70, priceLevel: 1, bestMonths: 'Mar–May, Sep–Nov',
    blurb: 'Spice-scented souks, riads with hidden courtyards and Atlas mountain day trips.',
    highlights: ['Jemaa el-Fnaa', 'Jardin Majorelle', 'Medina souks', 'Atlas Mountains', 'Agafay desert dinner'],
  },
  {
    id: 'rio', city: 'Rio de Janeiro', country: 'Brazil', countryCode: 'BR', image: 'rio', rating: 4.7, reviews: 10600,
    lat: -22.9068, lng: -43.1729, tz: 'America/Sao_Paulo', currency: 'BRL', airport: 'GIG', language: 'Portuguese',
    tags: ['beach', 'adventure', 'nightlife', 'nature'], dailyCost: 90, priceLevel: 2, bestMonths: 'Dec–Mar',
    blurb: 'Sugarloaf sunsets, samba and the world’s most famous beaches.',
    highlights: ['Sugarloaf Mountain', 'Christ the Redeemer', 'Copacabana', 'Selarón Steps', 'Tijuca Forest hike'],
  },
  {
    id: 'mexico-city', city: 'Mexico City', country: 'Mexico', countryCode: 'MX', image: 'mexico-city', rating: 4.7, reviews: 11300,
    lat: 19.4326, lng: -99.1332, tz: 'America/Mexico_City', currency: 'MXN', airport: 'MEX', language: 'Spanish',
    tags: ['food', 'culture', 'city', 'nightlife'], dailyCost: 80, priceLevel: 2, bestMonths: 'Mar–May, Oct–Nov',
    blurb: 'Tacos al pastor, Aztec ruins and colourful neighbourhoods like Coyoacán.',
    highlights: ['Teotihuacán pyramids', 'Frida Kahlo Museum', 'Xochimilco boats', 'Zócalo', 'Street-taco crawl'],
  },
  {
    id: 'grand-canyon', city: 'Grand Canyon', country: 'United States', countryCode: 'US', image: 'grand-canyon', rating: 4.9, reviews:
9600,
    lat: 36.0544, lng: -112.1401, tz: 'America/Phoenix', currency: 'USD', airport: 'PHX', language: 'English',
    tags: ['adventure', 'nature', 'family'], dailyCost: 140, priceLevel: 2, bestMonths: 'Mar–May, Sep–Nov',
    blurb: 'A mile-deep canyon carved over six million years — sunrise here is unforgettable.',
    highlights: ['South Rim sunrise', 'Bright Angel Trail', 'Desert View Watchtower', 'Horseshoe Bend', 'Skywalk'],
  },
  {
    id: 'cinque-terre', city: 'Cinque Terre', country: 'Italy', countryCode: 'IT', image: 'cinque-terre', rating: 4.8, reviews: 7800,
    lat: 44.1461, lng: 9.6439, tz: 'Europe/Rome', currency: 'EUR', airport: 'PSA', language: 'Italian',
    tags: ['beach', 'nature', 'romantic', 'food'], dailyCost: 150, priceLevel: 3, bestMonths: 'May–Jun, Sep',
    blurb: 'Five cliffside villages linked by hiking trails and pesto-scented harbours.',
    highlights: ['Sentiero Azzurro hike', 'Manarola at dusk', 'Vernazza harbour', 'Boat between villages', 'Pesto class'],
  },
```

```
  {
    id: 'venice', city: 'Venice', country: 'Italy', countryCode: 'IT', image: 'venice', rating: 4.7, reviews: 16300,
    lat: 45.4408, lng: 12.3155, tz: 'Europe/Rome', currency: 'EUR', airport: 'VCE', language: 'Italian',
    tags: ['romantic', 'culture', 'city'], dailyCost: 190, priceLevel: 4, bestMonths: 'Apr–Jun, Sep–Oct',
    blurb: 'Gondolas, glass-blowing islands and a city floating on 118 islets.',
    highlights: ['Grand Canal gondola', 'St Mark’s Basilica', 'Burano island', 'Rialto Market', 'Doge’s Palace'],
  },
  {
    id: 'mount-fuji', city: 'Fujiyoshida', country: 'Japan', countryCode: 'JP', image: 'mount-fuji', rating: 4.9, reviews: 6900,
    lat: 35.4875, lng: 138.7976, tz: 'Asia/Tokyo', currency: 'JPY', airport: 'HND', language: 'Japanese',
    tags: ['best', 'nature', 'adventure', 'culture'], dailyCost: 120, priceLevel: 2, bestMonths: 'Apr, Jul–Aug (climb), Nov',
    blurb: 'Chureito Pagoda framing Japan’s sacred peak — an easy day trip from Tokyo.',
    highlights: ['Chureito Pagoda', 'Lake Kawaguchiko', 'Oshino Hakkai', 'Fuji Five Lakes cycling', 'Summit climb (Jul–Aug)'],
  },
];

export const destinationById = (id: string) => destinations.find((d) => d.id === id);

export const exploreCategories: { key: 'all' | DestinationTag; labelKey: string; image: PhotoKey }[] = [
  { key: 'all', labelKey: 'cat.all', image: 'tokyo-skyline' },
  { key: 'best', labelKey: 'cat.best', image: 'santorini' },
  { key: 'adventure', labelKey: 'cat.adventure', image: 'banff' },
  { key: 'food', labelKey: 'cat.food', image: 'ramen' },
  { key: 'culture', labelKey: 'cat.culture', image: 'kyoto' },
  { key: 'beach', labelKey: 'cat.beach', image: 'maldives' },
  { key: 'nature', labelKey: 'cat.nature', image: 'iceland' },
  { key: 'nightlife', labelKey: 'cat.nightlife', image: 'shibuya' },
];
```

src/data/images.ts

```
/**
 * Bundled photography (Unsplash License — see ATTRIBUTIONS.md).
 * Images ship inside the app so Explore, onboarding and the photo dump
 * look complete even with no internet connection.
 */
import type { ImageSourcePropType } from 'react-native';

export const photos = {
  tokyo: require('@/assets/images/photos/tokyo.jpg'),
  kyoto: require('@/assets/images/photos/kyoto.jpg'),
  paris: require('@/assets/images/photos/paris.jpg'),
  bali: require('@/assets/images/photos/bali.jpg'),
  bangkok: require('@/assets/images/photos/bangkok.jpg'),
  singapore: require('@/assets/images/photos/singapore.jpg'),
  'cape-town': require('@/assets/images/photos/cape-town.jpg'),
  'new-york': require('@/assets/images/photos/new-york.jpg'),
  santorini: require('@/assets/images/photos/santorini.jpg'),
  amalfi: require('@/assets/images/photos/amalfi.jpg'),
  rome: require('@/assets/images/photos/rome.jpg'),
  london: require('@/assets/images/photos/london.jpg'),
  iceland: require('@/assets/images/photos/iceland.jpg'),
  'swiss-alps': require('@/assets/images/photos/swiss-alps.jpg'),
  maldives: require('@/assets/images/photos/maldives.jpg'),
  'machu-picchu': require('@/assets/images/photos/machu-picchu.jpg'),
  'ha-long-bay': require('@/assets/images/photos/ha-long-bay.jpg'),
  dubai: require('@/assets/images/photos/dubai.jpg'),
  barcelona: require('@/assets/images/photos/barcelona.jpg'),
  sydney: require('@/assets/images/photos/sydney.jpg'),
  seoul: require('@/assets/images/photos/seoul.jpg'),
  istanbul: require('@/assets/images/photos/istanbul.jpg'),
  amsterdam: require('@/assets/images/photos/amsterdam.jpg'),
  prague: require('@/assets/images/photos/prague.jpg'),
  lisbon: require('@/assets/images/photos/lisbon.jpg'),
  banff: require('@/assets/images/photos/banff.jpg'),
  marrakech: require('@/assets/images/photos/marrakech.jpg'),
  rio: require('@/assets/images/photos/rio.jpg'),
  'mexico-city': require('@/assets/images/photos/mexico-city.jpg'),
  'grand-canyon': require('@/assets/images/photos/grand-canyon.jpg'),
  'cinque-terre': require('@/assets/images/photos/cinque-terre.jpg'),
  venice: require('@/assets/images/photos/venice.jpg'),
  'mount-fuji': require('@/assets/images/photos/mount-fuji.jpg'),
  'onboard-1': require('@/assets/images/photos/onboard-1.jpg'),
  'onboard-2': require('@/assets/images/photos/onboard-2.jpg'),
  'onboard-3': require('@/assets/images/photos/onboard-3.jpg'),
  lagoon: require('@/assets/images/photos/lagoon.jpg'),
  shibuya: require('@/assets/images/photos/shibuya.jpg'),
  akihabara: require('@/assets/images/photos/akihabara.jpg'),
  'fushimi-inari': require('@/assets/images/photos/fushimi-inari.jpg'),
  'omoide-yokocho': require('@/assets/images/photos/omoide-yokocho.jpg'),
  'lantern-alley': require('@/assets/images/photos/lantern-alley.jpg'),
  'fuji-blossom': require('@/assets/images/photos/fuji-blossom.jpg'),
  'tokyo-skyline': require('@/assets/images/photos/tokyo-skyline.jpg'),
  'kyoto-skyline': require('@/assets/images/photos/kyoto-skyline.jpg'),
  sushi: require('@/assets/images/photos/sushi.jpg'),
  ramen: require('@/assets/images/photos/ramen.jpg'),
  planning: require('@/assets/images/photos/planning.jpg'),
  'avatar-maya': require('@/assets/images/photos/avatar-maya.jpg'),
  'avatar-diego': require('@/assets/images/photos/avatar-diego.jpg'),
  'avatar-priya': require('@/assets/images/photos/avatar-priya.jpg'),
  'avatar-sam': require('@/assets/images/photos/avatar-sam.jpg'),
  'avatar-ana': require('@/assets/images/photos/avatar-ana.jpg'),
  'avatar-noah': require('@/assets/images/photos/avatar-noah.jpg'),
} as const satisfies Record<string, ImageSourcePropType>;

export type PhotoKey = keyof typeof photos;

/** Accepts either a bundled photo key or a file/remote URI. */
export function imageSource(ref: string | undefined | null): ImageSourcePropType | undefined {
  if (!ref) return undefined;
  if (ref in photos) return photos[ref as PhotoKey];
  return { uri: ref };
}
```

src/data/panoramas.ts

```
/**
 * Recipes for the illustrated 360° city views (src/app/panorama.tsx).
 * Everything is drawn in code from these numbers, so it works offline and needs no image licences.
 */
export type Roof = 'flat' | 'step' | 'spire' | 'slant' | 'gable' | 'dome';

export type LandmarkKind =
  | 'eiffel'
  | 'arc'
  | 'sacre'
  | 'tokyotower'
  | 'skytree'
  | 'fuji'
  | 'esb'
  | 'wtc'
  | 'chrysler'
  | 'bigben'
  | 'eye'
  | 'shard'
  | 'burj'
  | 'burjalarab'
  | 'opera'
  | 'harbour'
  | 'sydtower'
  | 'mbs'
  | 'supertrees'
  | 'christ'
  | 'sugarloaf'
  | 'colosseum'
  | 'stpeters'
  | 'duomo';

export interface Landmark {
  id: string;
  name: string;
  fact: string;
  kind: LandmarkKind;
  /** Compass heading (0–360°) at which it appears in the view. */
  heading: number;
  /** Height as a fraction of the screen height. */
  size: number;
  /** Far landmarks (mountains) sit behind the skyline. */
  far?: boolean;
}

export interface Pano {
  id: string;
  /** Matching destination id, so trips can open their own city. */
  destId: string;
  name: string;
  country: string;
  emoji: string;
  tz: string;
  seed: number;
  /** 'road' = street with cars, 'water' = harbour/sea in front, 'slope' = houses climbing a hillside. */
  ground: 'road' | 'water' | 'slope';
  mountains?: 'hills' | 'granite' | 'cliffs';
  /** Mid-layer skyline: width range in px, height range as a fraction of screen height, gap in px, roof styles. */
  mid: { w: [number, number]; h: [number, number]; gap: number; roofs: Roof[]; towers: number };
  /** Near-layer low buildings (height fraction). */
  near: { h: [number, number] };
  sky: { dayTop: string; dayHorizon: string; nightTop: string; nightHorizon: string };
  /** [day, night] fill for the far, mid and near layers. */
  tone: { far: [string, string]; mid: [string, string]; near: [string, string] };
  /** Colour of lit windows at night. */
  lights: string;
  /** Optional pastel house colours (Positano). */
  houses?: string[];
  landmarks: Landmark[];
}

export const PANORAMAS: Pano[] = [
  {
    id: 'positano',
    destId: 'positano',
    name: 'Positano',
    country: 'Italy',
    emoji: '🍋',
    tz: 'Europe/Rome',
    seed: 11,
    ground: 'slope',
    mountains: 'cliffs',
    mid: { w: [18, 34], h: [0.05, 0.12], gap: 2, roofs: ['gable', 'flat', 'dome'], towers: 0 },
    near: { h: [0.03, 0.07] },
```

```
    sky: { dayTop: '#4FA8EE', dayHorizon: '#EAF6FF', nightTop: '#070B28', nightHorizon: '#5A3F6E' },
    tone: { far: ['#8FA39A', '#1A2340'], mid: ['#7A9B7F', '#142030'], near: ['#C9916F', '#1B1426'] },
    lights: '#FFC878',
    houses: ['#F2B8A2', '#F7D49A', '#E9967A', '#F4E3B1', '#E7A27C', '#F6C6D0', '#FBE7B0'],
    landmarks: [
      {
        id: 'duomo',
        name: 'Santa Maria Assunta',
        fact: 'Positano’s landmark church with a colourful majolica-tiled dome; its Black Madonna icon is said to date from the 13th cen
tury.',
        kind: 'duomo',
        heading: 70,
        size: 0.2,
      },
    ],
  },
  {
    id: 'tokyo',
    destId: 'tokyo',
    name: 'Tokyo',
    country: 'Japan',
    emoji: '🗼',
    tz: 'Asia/Tokyo',
    seed: 21,
    ground: 'road',
    mountains: 'hills',
    mid: { w: [16, 40], h: [0.05, 0.26], gap: 3, roofs: ['flat', 'step', 'slant', 'spire'], towers: 0.18 },
    near: { h: [0.05, 0.14] },
    sky: { dayTop: '#6BAFE8', dayHorizon: '#D6EAF8', nightTop: '#0A0F2C', nightHorizon: '#6B3A72' },
    tone: { far: ['#B3C2D4', '#1A2147'], mid: ['#8797AD', '#121832'], near: ['#5B6A80', '#0B0F22'] },
    lights: '#FFD27A',
    landmarks: [
      {
        id: 'tokyotower',
        name: 'Tokyo Tower',
        fact: '333 m tall and inspired by the Eiffel Tower. It is painted orange and white so aircraft can see it.',
        kind: 'tokyotower',
        heading: 40,
        size: 0.34,
      },
      { id: 'skytree', name: 'Tokyo Skytree', fact: '634 m — the tallest tower in the world, opened in 2012.', kind: 'skytree', heading:
150, size: 0.4 },
      { id: 'fuji', name: 'Mt Fuji', fact: '3,776 m, Japan’s highest peak. On clear days you can see it from Tokyo.', kind: 'fuji', head
ing: 255, size: 0.2, far: true },
    ],
  },
  {
    id: 'paris',
    destId: 'paris',
    name: 'Paris',
    country: 'France',
    emoji: '🥐',
    tz: 'Europe/Paris',
    seed: 31,
    ground: 'road',
    mid: { w: [20, 44], h: [0.04, 0.1], gap: 1, roofs: ['gable', 'slant', 'flat', 'dome'], towers: 0.02 },
    near: { h: [0.05, 0.11] },
    sky: { dayTop: '#8DB8E8', dayHorizon: '#F4E8D6', nightTop: '#0E1230', nightHorizon: '#6D3F5C' },
    tone: { far: ['#C9CFD8', '#1D2348'], mid: ['#B5AA9A', '#171B38'], near: ['#8E8372', '#0E1124'] },
    lights: '#FFCB7A',
    landmarks: [
      { id: 'eiffel', name: 'Eiffel Tower', fact: '330 m tall and finished in 1889. After dark it sparkles for five minutes every hou
r.', kind: 'eiffel', heading: 60, size: 0.36 },
      { id: 'sacre', name: 'Sacré-Cœur', fact: 'A white basilica on Montmartre hill, finished in 1914 — the best free view in Paris.', k
ind: 'sacre', heading: 170, size: 0.2 },
      { id: 'arc', name: 'Arc de Triomphe', fact: 'Commissioned by Napoleon in 1806. Twelve avenues meet around it.', kind: 'arc', headi
ng: 270, size: 0.16 },
    ],
  },
  {
    id: 'new-york',
    destId: 'new-york',
    name: 'New York',
    country: 'USA',
    emoji: '🗽',
    tz: 'America/New_York',
    seed: 41,
    ground: 'water',
    mid: { w: [16, 38], h: [0.08, 0.34], gap: 2, roofs: ['flat', 'step', 'spire', 'slant'], towers: 0.3 },
    near: { h: [0.05, 0.13] },
    sky: { dayTop: '#5FA2E6', dayHorizon: '#D7E8F8', nightTop: '#070B22', nightHorizon: '#473366' },
    tone: { far: ['#A6B7CC', '#18204A'], mid: ['#7C8DA6', '#10162F'], near: ['#55637A', '#0A0E20'] },
    lights: '#FFD684',
    landmarks: [
      { id: 'esb', name: 'Empire State Building', fact: '443 m with its antenna, opened in 1931. The top lights change colour for holida
ys and events.', kind: 'esb', heading: 30, size: 0.4 },
```

```
      { id: 'chrysler', name: 'Chrysler Building', fact: 'An Art Deco gem from 1930 with a stainless-steel crown that glows at night.',
kind: 'chrysler', heading: 75, size: 0.36 },
      { id: 'wtc', name: 'One World Trade Center', fact: '541 m (1,776 feet) — the tallest building in the Western Hemisphere.', kind:
'wtc', heading: 160, size: 0.44 },
    ],
  },
  {
    id: 'london',
    destId: 'london',
    name: 'London',
    country: 'United Kingdom',
    emoji: '🎡',
    tz: 'Europe/London',
    seed: 51,
    ground: 'water',
    mid: { w: [18, 40], h: [0.05, 0.14], gap: 2, roofs: ['flat', 'gable', 'step'], towers: 0.08 },
    near: { h: [0.05, 0.12] },
    sky: { dayTop: '#8FA9C4', dayHorizon: '#DDE2E7', nightTop: '#0C1128', nightHorizon: '#52405A' },
    tone: { far: ['#B8C1CC', '#1C2347'], mid: ['#8D98A6', '#141A36'], near: ['#667180', '#0C1022'] },
    lights: '#FFD07A',
    landmarks: [
      { id: 'bigben', name: 'Big Ben', fact: 'Technically the bell inside the Elizabeth Tower, which is 96 m tall.', kind: 'bigben', hea
ding: 20, size: 0.3 },
      { id: 'eye', name: 'London Eye', fact: '135 m tall — one full turn takes about 30 minutes.', kind: 'eye', heading: 85, size: 0.28
},
      { id: 'shard', name: 'The Shard', fact: '310 m, the tallest building in the UK, opened in 2012.', kind: 'shard', heading: 205, siz
e: 0.4 },
    ],
  },
  {
    id: 'dubai',
    destId: 'dubai',
    name: 'Dubai',
    country: 'UAE',
    emoji: '🏙️',
    tz: 'Asia/Dubai',
    seed: 61,
    ground: 'water',
    mid: { w: [14, 34], h: [0.07, 0.3], gap: 4, roofs: ['flat', 'slant', 'spire', 'step'], towers: 0.3 },
    near: { h: [0.04, 0.1] },
    sky: { dayTop: '#E4B67A', dayHorizon: '#F8E6C6', nightTop: '#0B1026', nightHorizon: '#8A4A2E' },
    tone: { far: ['#D9C4A2', '#221E3E'], mid: ['#B79F7E', '#171A36'], near: ['#8C7757', '#0C0F22'] },
    lights: '#FFD58A',
    landmarks: [
      { id: 'burj', name: 'Burj Khalifa', fact: '828 m — the tallest building in the world since 2010.', kind: 'burj', heading: 90, siz
e: 0.5 },
      { id: 'burjalarab', name: 'Burj Al Arab', fact: 'A sail-shaped hotel on its own island, 321 m tall.', kind: 'burjalarab', heading:
225, size: 0.34 },
    ],
  },
  {
    id: 'sydney',
    destId: 'sydney',
    name: 'Sydney',
    country: 'Australia',
    emoji: '🦘',
    tz: 'Australia/Sydney',
    seed: 71,
    ground: 'water',
    mid: { w: [16, 36], h: [0.05, 0.2], gap: 3, roofs: ['flat', 'step', 'slant'], towers: 0.1 },
    near: { h: [0.05, 0.11] },
    sky: { dayTop: '#4FA9EE', dayHorizon: '#DDF1FF', nightTop: '#080D2C', nightHorizon: '#4C3B72' },
    tone: { far: ['#A8C0D4', '#1A2248'], mid: ['#7E96AE', '#121936'], near: ['#586C84', '#0A0F24'] },
    lights: '#FFD27A',
    landmarks: [
      { id: 'opera', name: 'Sydney Opera House', fact: 'Opened in 1973. Its roof is covered by more than a million tiles.', kind: 'oper
a', heading: 60, size: 0.16 },
      { id: 'harbour', name: 'Sydney Harbour Bridge', fact: 'Opened in 1932 — locals call it “the Coathanger”.', kind: 'harbour', headin
g: 135, size: 0.22 },
      { id: 'sydtower', name: 'Sydney Tower', fact: '309 m — the tallest structure in Sydney.', kind: 'sydtower', heading: 255, size: 0.
4 },
    ],
  },
  {
    id: 'singapore',
    destId: 'singapore',
    name: 'Singapore',
    country: 'Singapore',
    emoji: '🦁',
    tz: 'Asia/Singapore',
    seed: 81,
    ground: 'water',
    mid: { w: [16, 36], h: [0.07, 0.26], gap: 3, roofs: ['flat', 'step', 'slant', 'spire'], towers: 0.22 },
    near: { h: [0.04, 0.1] },
    sky: { dayTop: '#53B7EA', dayHorizon: '#E5F6FF', nightTop: '#080C2A', nightHorizon: '#2D4E7A' },
    tone: { far: ['#A9C8D6', '#162448'], mid: ['#7FA0B0', '#101A38'], near: ['#587888', '#0A1122'] },
```

```
    lights: '#FFE08A',
    landmarks: [
      { id: 'mbs', name: 'Marina Bay Sands', fact: 'Three towers topped by a 340 m-long SkyPark with the world’s largest rooftop infinit
y pool.', kind: 'mbs', heading: 80, size: 0.2 },
      { id: 'supertrees', name: 'Supertree Grove', fact: 'Giant solar-powered “trees” at Gardens by the Bay, up to 50 m tall and lit up
every night.', kind: 'supertrees', heading: 200, size: 0.2 },
    ],
  },
  {
    id: 'rio',
    destId: 'rio',
    name: 'Rio de Janeiro',
    country: 'Brazil',
    emoji: '⛰️',
    tz: 'America/Sao_Paulo',
    seed: 91,
    ground: 'water',
    mountains: 'granite',
    mid: { w: [16, 36], h: [0.05, 0.18], gap: 3, roofs: ['flat', 'step', 'slant'], towers: 0.08 },
    near: { h: [0.04, 0.1] },
    sky: { dayTop: '#4DAFE6', dayHorizon: '#E9F6FF', nightTop: '#0A1030', nightHorizon: '#6B3C5E' },
    tone: { far: ['#6F9A86', '#13213A'], mid: ['#8FA2B4', '#131B38'], near: ['#667A8E', '#0B1124'] },
    lights: '#FFD27A',
    landmarks: [
      { id: 'christ', name: 'Christ the Redeemer', fact: 'A 30 m statue on top of 700 m-high Corcovado, finished in 1931.', kind: 'chris
t', heading: 60, size: 0.28, far: true },
      { id: 'sugarloaf', name: 'Sugarloaf Mountain', fact: 'A 396 m granite peak reached by a two-stage cable car since 1912.', kind: 's
ugarloaf', heading: 190, size: 0.3, far: true },
    ],
  },
  {
    id: 'rome',
    destId: 'rome',
    name: 'Rome',
    country: 'Italy',
    emoji: '🏛️',
    tz: 'Europe/Rome',
    seed: 101,
    ground: 'road',
    mountains: 'hills',
    mid: { w: [20, 44], h: [0.04, 0.11], gap: 1, roofs: ['gable', 'flat', 'dome', 'slant'], towers: 0.02 },
    near: { h: [0.05, 0.11] },
    sky: { dayTop: '#74B2E6', dayHorizon: '#F6E8CC', nightTop: '#0D1130', nightHorizon: '#7A4A3A' },
    tone: { far: ['#B9B79C', '#1E2444'], mid: ['#C2A17E', '#1A1D3A'], near: ['#9A7A58', '#0F1226'] },
    lights: '#FFC878',
    landmarks: [
      { id: 'colosseum', name: 'Colosseum', fact: 'Opened in AD 80 and could seat roughly 50,000 spectators.', kind: 'colosseum', headin
g: 50, size: 0.2 },
      { id: 'stpeters', name: 'St Peter’s Basilica', fact: 'Michelangelo’s dome rises about 136 m — one of the tallest domes in the worl
d.', kind: 'stpeters', heading: 200, size: 0.3 },
    ],
  },
];

export const panoById = (id: string | undefined) => PANORAMAS.find((p) => p.id === id || p.destId === id);
```

src/data/personal.ts

```
import { photos } from '@/data/images';

import type { DestinationTag } from './destinations';

/** Travel interests offered in "Make it yours" — each maps to a destination tag. */
export const INTERESTS: {
  key: DestinationTag;
  label: string;
  emoji: string;
  image: keyof typeof photos;
}[] = [
  { key: 'food', label: 'Food & street eats', emoji: '🍜', image: 'ramen' },
  { key: 'adventure', label: 'Adventure', emoji: '🧗', image: 'banff' },
  { key: 'beach', label: 'Beaches', emoji: '🏖️', image: 'maldives' },
  { key: 'culture', label: 'Culture & history', emoji: '🏯', image: 'kyoto' },
  { key: 'nature', label: 'Nature escapes', emoji: '🌿', image: 'iceland' },
  { key: 'nightlife', label: 'Nightlife', emoji: '🌃', image: 'shibuya' },
  { key: 'best', label: 'Iconic sights', emoji: '📸', image: 'santorini' },
];
```

src/data/places.ts

```
import { destinationById } from './destinations';

export type PlaceCategory =
  | 'food'
  | 'sightseeing'
  | 'adventure'
  | 'culture'
  | 'nightlife'
  | 'shopping'
  | 'relax';

export interface Place {
  id: string;
  destinationId: string;
  name: string;
  area: string;
  category: PlaceCategory;
  /** Typical visit length in minutes. */
  durationMin: number;
  /** Typical cost per person in USD (0 = free). */
  cost: number;
  rating: number;
  lat: number;
  lng: number;
  blurb: string;
}

const P = (
  destinationId: string,
  id: string,
  name: string,
  area: string,
  category: PlaceCategory,
  durationMin: number,
  cost: number,
  rating: number,
  lat: number,
  lng: number,
  blurb: string,
): Place => ({ destinationId, id, name, area, category, durationMin, cost, rating, lat, lng, blurb });

const curated: Place[] = [
  // Positano & the Amalfi Coast
  P('positano', 'da-vincenzo', 'Da Vincenzo', 'Viale Pasitea', 'food', 120, 64, 4.7, 40.6295, 14.488, 'Family-run trattoria: fresh pasta
with a view over the rooftops.'),
  P('positano', 'path-of-gods', 'Path of the Gods', 'Bomerano', 'adventure', 300, 0, 4.9, 40.619, 14.5176, 'A 7 km cliff trail above the
sea, ending in Nocelle.'),
  P('positano', 'capri-boat', 'Boat tour to Capri', 'Positano Marina', 'adventure', 540, 140, 4.8, 40.6272, 14.4868, 'Full day on the wa
ter: Blue Grotto, swim stops and limoncello.'),
  P('positano', 'fornillo', 'Fornillo Beach', 'Fornillo', 'relax', 180, 35, 4.6, 40.6255, 14.4795, 'Quieter pebble beach — rent loungers
and eat at the beach club.'),
  P('positano', 'spiaggia-grande', 'Spiaggia Grande', 'Positano', 'relax', 120, 0, 4.4, 40.6275, 14.487, 'The main beach with the church
dome behind it.'),
  P('positano', 'santa-maria', 'Santa Maria Assunta', 'Positano', 'culture', 30, 0, 4.6, 40.6286, 14.489, 'Majolica-tiled dome and a Byz
antine Black Madonna.'),
  P('positano', 'sunset-bar', 'Sunset aperitivo at Le Sirenuse', 'Positano', 'nightlife', 90, 40, 4.8, 40.6293, 14.4868, 'Spritz on the
terrace as the lights come on.'),
  P('positano', 'limoncello', 'Limoncello tasting', 'Amalfi', 'food', 60, 23, 4.7, 40.634, 14.6027, 'Taste lemon-grove liqueurs and lear
n how they’re made.'),
  P('positano', 'duomo-amalfi', 'Amalfi Cathedral', 'Amalfi', 'culture', 60, 4, 4.7, 40.6348, 14.603, 'Striped façade and the Cloister o
f Paradise.'),
  P('positano', 'paper-museum', 'Museum of Amalfi Paper', 'Amalfi', 'culture', 45, 5, 4.5, 40.6355, 14.6013, 'Medieval paper mill still
making sheets by hand.'),
  P('positano', 'villa-rufolo', 'Villa Rufolo gardens', 'Ravello', 'sightseeing', 90, 8, 4.8, 40.649, 14.6118, 'Terraced gardens with th
e postcard view of the coast.'),
  P('positano', 'furore', 'Fiordo di Furore', 'Furore', 'adventure', 90, 0, 4.6, 40.6139, 14.5517, 'A tiny fjord beach under a stone bri
dge — great for jumping in.'),
  P('positano', 'emerald-grotto', 'Emerald Grotto', 'Conca dei Marini', 'sightseeing', 60, 8, 4.5, 40.6195, 14.5666, 'Sea cave glowing g
reen; row-boat tours.'),
  P('positano', 'trattoria-chitarrella', 'Trattoria da Adolfo', 'Laurito Beach', 'food', 150, 55, 4.6, 40.6224, 14.4731, 'Grilled fish a
nd mozzarella-on-lemon-leaf, reached by boat.'),
  P('positano', 'lemon-gelato', 'Gelato on Via dei Mulini', 'Positano', 'food', 30, 6, 4.5, 40.6291, 14.4851, 'Lemon sorbet served in a
real lemon.'),
  // Tokyo
  P('tokyo', 'tsukiji', 'Tsukiji Outer Market', 'Tsukiji', 'food', 90, 25, 4.7, 35.6655, 139.7707, 'Tamagoyaki, fresh tuna and grilled s
callops from 400+ stalls.'),
  P('tokyo', 'sensoji', 'Senso-ji Temple', 'Asakusa', 'culture', 90, 0, 4.8, 35.7148, 139.7967, 'Tokyo’s oldest temple with the iconic K
aminarimon gate.'),
  P('tokyo', 'shibuya-crossing', 'Shibuya Crossing', 'Shibuya', 'sightseeing', 45, 0, 4.6, 35.6595, 139.7005, 'The world’s busiest scram
ble — best seen from Shibuya Sky.'),
  P('tokyo', 'meiji', 'Meiji Jingu Shrine', 'Harajuku', 'culture', 60, 0, 4.7, 35.6764, 139.6993, 'A forest shrine in the middle of the
city.'),
  P('tokyo', 'teamlab', 'teamLab Planets', 'Toyosu', 'culture', 150, 30, 4.8, 35.6492, 139.7898, 'Walk barefoot through immersive digita
l-art rooms.'),
```

```
  P('tokyo', 'hamarikyu', 'Hama-rikyu Gardens', 'Shiodome', 'relax', 50, 2, 4.5, 35.66, 139.7633, 'Edo-era tidal garden with a teahouse on the pond.'),
  P('tokyo', 'skytree', 'Tokyo Skytree', 'Oshiage', 'sightseeing', 90, 25, 4.6, 35.7101, 139.8107, '634 m tower — on clear days you can see Mt Fuji.'),
  P('tokyo', 'gyoen', 'Shinjuku Gyoen', 'Shinjuku', 'relax', 90, 4, 4.7, 35.6852, 139.71, 'Huge garden for a picnic break between neighbourhoods.'),
  P('tokyo', 'omoide', 'Omoide Yokocho', 'Shinjuku', 'food', 60, 20, 4.5, 35.6929, 139.6996, 'Smoky lantern-lit alley of tiny yakitori bars.'),
  P('tokyo', 'akihabara', 'Akihabara Electric Town', 'Akihabara', 'shopping', 120, 0, 4.4, 35.6984, 139.7731, 'Arcades, retro games and anime shops.'),
  P('tokyo', 'odaiba', 'Odaiba Seaside Park', 'Odaiba', 'adventure', 240, 0, 4.4, 35.6298, 139.7745, 'Waterfront boardwalk, Rainbow Bridge views and the Gundam statue.'),
  P('tokyo', 'ichiran', 'Ichiran Ramen Shibuya', 'Shibuya', 'food', 45, 12, 4.5, 35.6612, 139.7016, 'Solo-booth tonkotsu ramen, open 24 hours.'),
  P('tokyo', 'sushi-dai', 'Sushi Dai Toyosu', 'Toyosu', 'food', 60, 45, 4.8, 35.6457, 139.7847, 'Legendary omakase breakfast at the fish market.'),
  P('tokyo', 'tokyo-tower', 'Tokyo Tower', 'Minato', 'sightseeing', 60, 10, 4.5, 35.6586, 139.7454, 'Retro orange landmark with a glass-floor deck.'),
  P('tokyo', 'ueno', 'Ueno Park & Museums', 'Ueno', 'culture', 180, 10, 4.5, 35.7156, 139.7745, 'National Museum, zoo and street-food stalls.'),
  P('tokyo', 'takeshita', 'Takeshita Street', 'Harajuku', 'shopping', 90, 15, 4.3, 35.6716, 139.7036, 'Crêpes, kawaii fashion and people-watching.'),
  P('tokyo', 'golden-gai', 'Golden Gai', 'Shinjuku', 'nightlife', 120, 30, 4.5, 35.6938, 139.7046, '200 micro-bars squeezed into six alleys.'),
  P('tokyo', 'takao', 'Mount Takao Hike', 'Hachioji', 'adventure', 360, 10, 4.7, 35.6251, 139.2436, 'Forest trails, a monkey park and a summit view of Fuji.'),
  P('tokyo', 'imperial', 'Imperial Palace East Gardens', 'Chiyoda', 'sightseeing', 60, 0, 4.4, 35.6852, 139.7528, 'Castle ruins and seasonal blooms — free entry.'),
  P('tokyo', 'gokart', 'Street Go-Kart Tour', 'Shinagawa', 'adventure', 120, 90, 4.6, 35.6284, 139.7387, 'Drive a go-kart past Tokyo Tower in costume.'),
  P('tokyo', 'sumida', 'Sumida River Cruise', 'Asakusa', 'sightseeing', 60, 12, 4.4, 35.7107, 139.7983, 'Water bus from Asakusa to Hama-rikyu.'),
  P('tokyo', 'karaoke', 'Karaoke Kan Shibuya', 'Shibuya', 'nightlife', 120, 20, 4.3, 35.66, 139.6985, 'Private karaoke rooms — perfect for groups.'),
  P('tokyo', 'gonpachi', 'Gonpachi Nishi-Azabu', 'Roppongi', 'food', 90, 35, 4.4, 35.6563, 139.724, 'Izakaya that inspired the Kill Bill fight scene.'),
  P('tokyo', 'afuri', 'Afuri Ramen Ebisu', 'Ebisu', 'food', 45, 13, 4.5, 35.6467, 139.7101, 'Light yuzu-shio ramen, a local favourite.'),
  P('tokyo', 'yanaka', 'Yanaka Ginza', 'Yanaka', 'food', 90, 10, 4.4, 35.7277, 139.766, 'Old-Tokyo shopping street with croquettes and cats.'),
  // Kyoto
  P('kyoto', 'inari', 'Fushimi Inari Taisha', 'Fushimi', 'culture', 150, 0, 4.9, 34.9671, 135.7727, 'Thousands of vermilion torii gates up Mount Inari.'),
  P('kyoto', 'bamboo', 'Arashiyama Bamboo Grove', 'Arashiyama', 'sightseeing', 60, 0, 4.6, 35.017, 135.6713, 'Towering bamboo — go before 8 am to beat crowds.'),
  P('kyoto', 'kiyomizu', 'Kiyomizu-dera', 'Higashiyama', 'culture', 90, 3, 4.8, 34.9949, 135.785, 'Wooden stage temple overlooking the city.'),
  P('kyoto', 'nishiki', 'Nishiki Market', 'Nakagyo', 'food', 90, 20, 4.5, 35.005, 135.7649, '“Kyoto’s kitchen” — skewers, pickles and matcha treats.'),
  P('kyoto', 'kinkakuji', 'Kinkaku-ji (Golden Pavilion)', 'Kita', 'sightseeing', 60, 3, 4.7, 35.0394, 135.7292, 'Gold-leaf pavilion reflected in a mirror pond.'),
  P('kyoto', 'gion', 'Gion Evening Walk', 'Gion', 'culture', 90, 0, 4.6, 35.0037, 135.7788, 'Teahouse lanes where geiko still walk at dusk.'),
  P('kyoto', 'philosopher', 'Philosopher’s Path', 'Sakyo', 'relax', 60, 0, 4.5, 35.0268, 135.7945, 'Canal-side stroll lined with cherry trees.'),
  P('kyoto', 'tea', 'Tea Ceremony Experience', 'Higashiyama', 'culture', 60, 40, 4.8, 34.9986, 135.7809, 'Learn to whisk matcha in a traditional machiya.'),
  P('kyoto', 'pontocho', 'Pontocho Alley Dinner', 'Nakagyo', 'food', 90, 40, 4.5, 35.0058, 135.7708, 'Narrow lane of riverside restaurants.'),
  P('kyoto', 'monkey', 'Iwatayama Monkey Park', 'Arashiyama', 'adventure', 90, 6, 4.5, 35.0108, 135.677, 'Short hike to wild macaques and city views.'),
  P('kyoto', 'hozugawa', 'Hozugawa River Boat Ride', 'Kameoka', 'adventure', 120, 35, 4.6, 35.0128, 135.6744, 'Two-hour wooden boat ride through a gorge.'),
];

const highlightCategories: PlaceCategory[] = ['sightseeing', 'culture', 'food', 'adventure', 'relax'];

/**
 * Curated nearby places for a destination. Destinations without curated data
 * get their highlights placed at deterministic offsets around the centre so
 * the radius filter still behaves sensibly.
 */
export function placesFor(destinationId: string): Place[] {
  const list = curated.filter((p) => p.destinationId === destinationId);
  if (list.length) return list;
  const d = destinationById(destinationId);
  if (!d) return [];
  return d.highlights.map((name, i) => {
    const angle = (i * 137.5 * Math.PI) / 180;
    const km = 1.2 + i * 1.7;
    return P(
      d.id, `${d.id}-h${i}`, name, d.city, highlightCategories[i % highlightCategories.length],
      [90, 120, 60, 240, 90][i % 5], [0, 15, 20, 45, 10][i % 5], 4.5 + ((i * 7) % 5) / 10,
      d.lat + (km / 111) * Math.cos(angle), d.lng + (km / (111 * Math.cos((d.lat * Math.PI) / 180))) * Math.sin(angle),
      `One of ${d.city}’s must-do experiences.`,
    );
```

```
  });
}

export const allCuratedPlaces = curated;
```

```
/**
 * Offline reference data: currencies, emergency numbers, phrasebook, quiz,
 * survey and packing suggestions. Everything here works with zero connectivity.
 */

export interface Currency {
  code: string;
  name: string;
  symbol: string;
  /** Fallback rate per 1 USD, used when live rates can't be fetched. */
  perUsd: number;
}

export const currencies: Currency[] = [
  { code: 'USD', name: 'US Dollar', symbol: '$', perUsd: 1 },
  { code: 'EUR', name: 'Euro', symbol: '€', perUsd: 0.86 },
  { code: 'GBP', name: 'British Pound', symbol: '£', perUsd: 0.75 },
  { code: 'JPY', name: 'Japanese Yen', symbol: '¥', perUsd: 152.4 },
  { code: 'CAD', name: 'Canadian Dollar', symbol: 'C$', perUsd: 1.38 },
  { code: 'AUD', name: 'Australian Dollar', symbol: 'A$', perUsd: 1.52 },
  { code: 'INR', name: 'Indian Rupee', symbol: '₹', perUsd: 88.2 },
  { code: 'CNY', name: 'Chinese Yuan', symbol: '¥', perUsd: 7.12 },
  { code: 'KRW', name: 'South Korean Won', symbol: '₩', perUsd: 1390 },
  { code: 'MXN', name: 'Mexican Peso', symbol: 'MX$', perUsd: 18.4 },
  { code: 'CHF', name: 'Swiss Franc', symbol: 'CHF', perUsd: 0.8 },
  { code: 'SGD', name: 'Singapore Dollar', symbol: 'S$', perUsd: 1.29 },
  { code: 'THB', name: 'Thai Baht', symbol: '฿', perUsd: 32.4 },
  { code: 'IDR', name: 'Indonesian Rupiah', symbol: 'Rp', perUsd: 16400 },
  { code: 'VND', name: 'Vietnamese Dong', symbol: '₫', perUsd: 26300 },
  { code: 'AED', name: 'UAE Dirham', symbol: 'AED', perUsd: 3.67 },
  { code: 'TRY', name: 'Turkish Lira', symbol: '₺', perUsd: 41.5 },
  { code: 'ZAR', name: 'South African Rand', symbol: 'R', perUsd: 17.4 },
  { code: 'BRL', name: 'Brazilian Real', symbol: 'R$', perUsd: 5.35 },
  { code: 'PEN', name: 'Peruvian Sol', symbol: 'S/', perUsd: 3.5 },
  { code: 'CZK', name: 'Czech Koruna', symbol: 'Kč', perUsd: 20.9 },
  { code: 'ISK', name: 'Icelandic Króna', symbol: 'kr', perUsd: 122 },
  { code: 'MAD', name: 'Moroccan Dirham', symbol: 'MAD', perUsd: 9.1 },
  { code: 'MVR', name: 'Maldivian Rufiyaa', symbol: 'Rf', perUsd: 15.4 },
  { code: 'NZD', name: 'New Zealand Dollar', symbol: 'NZ$', perUsd: 1.7 },
];

export const currencyByCode = (code: string) => currencies.find((c) => c.code === code);

export interface EmergencyInfo {
  police: string;
  ambulance: string;
  fire: string;
  tips: string[];
}

export const emergencyByCountry: Record<string, EmergencyInfo> = {
  JP: { police: '110', ambulance: '119', fire: '119', tips: ['Japan Visitor Hotline (24h, English): 050-3816-2787', 'Koban (police boxe
s) are on most corners — staff help with directions too.', 'Carry your passport: foreigners must have ID at all times.'] },
  FR: { police: '17', ambulance: '15', fire: '18', tips: ['112 works from any phone in the EU.', 'Pharmacies (green cross) give free adv
ice for minor issues.'] },
  IT: { police: '112', ambulance: '118', fire: '115', tips: ['112 is the single EU emergency number.'] },
  ES: { police: '112', ambulance: '112', fire: '112', tips: ['Tourist police offices help with lost passports.'] },
  GB: { police: '999', ambulance: '999', fire: '999', tips: ['Call 111 for non-emergency NHS medical advice.'] },
  US: { police: '911', ambulance: '911', fire: '911', tips: ['Text 911 is available in many counties.'] },
  TH: { police: '191', ambulance: '1669', fire: '199', tips: ['Tourist Police (English): 1155.'] },
  ID: { police: '110', ambulance: '118', fire: '113', tips: ['112 also works from mobiles.'] },
  SG: { police: '999', ambulance: '995', fire: '995', tips: ['Non-emergency ambulance: 1777.'] },
  AU: { police: '000', ambulance: '000', fire: '000', tips: ['112 also works on mobiles.'] },
  KR: { police: '112', ambulance: '119', fire: '119', tips: ['1330 — tourist helpline in English, Japanese and Chinese.'] },
  DEFAULT: { police: '112', ambulance: '112', fire: '112', tips: ['112 works on most mobile networks worldwide.', 'Save your country's e
mbassy number before you fly.'] },
};

export interface Phrase {
  en: string;
  local: string;
  roman?: string;
}

export const phrasebook: Record<string, { lang: string; speechCode: string; phrases: Phrase[] }> = {
  Japanese: {
    lang: 'Japanese', speechCode: 'ja-JP', phrases: [
      { en: 'Hello', local: 'こんにちは', roman: 'Konnichiwa' },
      { en: 'Thank you', local: 'ありがとうございます', roman: 'Arigatou gozaimasu' },
      { en: 'Excuse me / Sorry', local: 'すみません', roman: 'Sumimasen' },
      { en: 'How much is this?', local: 'これはいくらですか？', roman: 'Kore wa ikura desu ka?' },
      { en: 'Where is the station?', local: '駅はどこですか？', roman: 'Eki wa doko desu ka?' },
      { en: 'Table for six, please', local: '六人です', roman: 'Roku-nin desu' },
      { en: 'The bill, please', local: 'お会計お願いします', roman: 'Okaikei onegaishimasu' },
```

```
      { en: 'I have an allergy', local: 'アレルギーがあります', roman: 'Arerugī ga arimasu' },
      { en: 'Help!', local: '助けて！', roman: 'Tasukete!' },
      { en: 'Delicious!', local: 'おいしい！', roman: 'Oishii!' },
    ],
  },
  French: {
    lang: 'French', speechCode: 'fr-FR', phrases: [
      { en: 'Hello', local: 'Bonjour' }, { en: 'Thank you', local: 'Merci beaucoup' },
      { en: 'Excuse me', local: 'Excusez-moi' }, { en: 'How much is this?', local: 'Combien ça coûte ?' },
      { en: 'Where is the metro?', local: 'Où est le métro ?' }, { en: 'The bill, please', local: 'L’addition, s’il vous plaît' },
      { en: 'Help!', local: 'Au secours !' },
    ],
  },
  Spanish: {
    lang: 'Spanish', speechCode: 'es-ES', phrases: [
      { en: 'Hello', local: 'Hola' }, { en: 'Thank you', local: 'Muchas gracias' },
      { en: 'Excuse me', local: 'Perdón' }, { en: 'How much is this?', local: '¿Cuánto cuesta?' },
      { en: 'Where is the bathroom?', local: '¿Dónde está el baño?' }, { en: 'The bill, please', local: 'La cuenta, por favor' },
      { en: 'Help!', local: '¡Ayuda!' },
    ],
  },
  Italian: {
    lang: 'Italian', speechCode: 'it-IT', phrases: [
      { en: 'Hello', local: 'Ciao / Buongiorno' }, { en: 'Thank you', local: 'Grazie mille' },
      { en: 'How much is this?', local: 'Quanto costa?' }, { en: 'The bill, please', local: 'Il conto, per favore' },
      { en: 'Help!', local: 'Aiuto!' },
    ],
  },
  Korean: {
    lang: 'Korean', speechCode: 'ko-KR', phrases: [
      { en: 'Hello', local: '안녕하세요', roman: 'Annyeonghaseyo' }, { en: 'Thank you', local: '감사합니다', roman: 'Gamsahamnida' },
      { en: 'How much is this?', local: '이거 얼마예요?', roman: 'Igeo eolmayeyo?' }, { en: 'Help!', local: '도와주세요!', roman: 'Dowajusey
o!' },
    ],
  },
  Thai: {
    lang: 'Thai', speechCode: 'th-TH', phrases: [
      { en: 'Hello', local: 'สวัสดี', roman: 'Sawasdee' }, { en: 'Thank you', local: 'ขอบคุณ', roman: 'Khob khun' },
      { en: 'How much?', local: 'เท่าไหร่', roman: 'Tao rai?' }, { en: 'Not spicy', local: 'ไม่เผ็ด', roman: 'Mai phet' },
    ],
  },
  Portuguese: {
    lang: 'Portuguese', speechCode: 'pt-PT', phrases: [
      { en: 'Hello', local: 'Olá' }, { en: 'Thank you', local: 'Obrigado / Obrigada' },
      { en: 'How much is this?', local: 'Quanto custa?' }, { en: 'Help!', local: 'Socorro!' },
    ],
  },
};

export interface QuizQuestion {
  q: string;
  options: string[];
  answer: number;
  fact: string;
}

export const quizQuestions: QuizQuestion[] = [
  { q: 'Which city has the busiest pedestrian scramble crossing in the world?', options: ['Osaka', 'Tokyo', 'Seoul', 'Hong Kong'], answe
r: 1, fact: 'Up to 3,000 people cross Shibuya Crossing every time the lights change.' },
  { q: 'How many torii gates line the trails of Fushimi Inari in Kyoto (approx.)?', options: ['500', '1,000', '10,000', '50,000'], answe
r: 2, fact: 'Around 10,000 gates — each donated by a business or individual.' },
  { q: 'What does “itadakimasu” express before a meal in Japan?', options: ['Cheers', 'Gratitude for the food', 'The bill please', 'Good
morning'], answer: 1, fact: 'It thanks everyone (and everything) that made the meal possible.' },
  { q: 'Which currency is used in Switzerland?', options: ['Euro', 'Swiss franc', 'Krone', 'Pound'], answer: 1, fact: 'Switzerland kept
the franc (CHF) and is not in the eurozone.' },
  { q: 'The Eiffel Tower was originally built for which event?', options: ['1889 World’s Fair', 'The 1900 Olympics', 'A royal wedding',
'WWI victory'], answer: 0, fact: 'It was meant to stand for only 20 years!' },
  { q: 'Machu Picchu was built by which civilisation?', options: ['Maya', 'Aztec', 'Inca', 'Olmec'], answer: 2, fact: 'Built around 1450
for the Inca emperor Pachacuti.' },
  { q: 'Which country has the most time zones (including territories)?', options: ['Russia', 'USA', 'France', 'China'], answer: 2, fact:
'France spans 12 time zones thanks to its overseas territories.' },
  { q: 'In which city would you ride a vaporetto?', options: ['Amsterdam', 'Venice', 'Lisbon', 'Istanbul'], answer: 1, fact: 'Vaporetti
are Venice’s water buses.' },
  { q: 'What is the tallest building in the world (2026)?', options: ['Shanghai Tower', 'Burj Khalifa', 'Merdeka 118', 'Tokyo Skytree'],
answer: 1, fact: 'Dubai’s Burj Khalifa stands 828 m tall.' },
  { q: 'Tipping at restaurants in Japan is…', options: ['Expected (15%)', 'Expected (10%)', 'Not customary', 'Only in cash'], answer: 2,
fact: 'Great service is included — a sincere thank-you is enough.' },
  { q: 'Which airport code belongs to Tokyo Haneda?', options: ['TYO', 'NRT', 'HND', 'HAN'], answer: 2, fact: 'HAN is Hanoi — a classic
mix-up when booking!' },
  { q: 'Which of these is a UNESCO World Heritage site in Vietnam?', options: ['Ha Long Bay', 'Phi Phi Islands', 'Komodo', 'Boracay'], a
nswer: 0, fact: 'Ha Long Bay has ~1,600 limestone islands and islets.' },
];

export interface SurveyQuestion {
  id: string;
  q: string;
```

```
  options: { label: string; emoji: string; tags: string[] }[];
}

export const surveyQuestions: SurveyQuestion[] = [
  { id: 'vibe', q: 'Pick your ideal trip vibe', options: [
    { label: 'City lights', emoji: '🌃', tags: ['city', 'nightlife'] },
    { label: 'Beach & sun', emoji: '🏖️', tags: ['beach', 'romantic'] },
    { label: 'Mountains & trails', emoji: '🏔️', tags: ['adventure', 'nature'] },
    { label: 'History & culture', emoji: '🏛️', tags: ['culture'] },
  ] },
  { id: 'food', q: 'How important is food on this trip?', options: [
    { label: 'It’s the whole point', emoji: '🍜', tags: ['food', 'food'] },
    { label: 'Love a good meal', emoji: '🍝', tags: ['food'] },
    { label: 'Fuel for adventures', emoji: '🥪', tags: ['adventure'] },
    { label: 'Not fussed', emoji: '🤷', tags: [] },
  ] },
  { id: 'budget', q: 'Daily budget per person?', options: [
    { label: 'Under $90', emoji: '🪙', tags: ['$1'] },
    { label: '$90 – $160', emoji: '💵', tags: ['$2'] },
    { label: '$160 – $230', emoji: '💳', tags: ['$3'] },
    { label: 'Treat ourselves', emoji: '💎', tags: ['$4'] },
  ] },
  { id: 'pace', q: 'What pace does your group like?', options: [
    { label: 'Packed schedule', emoji: '⚡', tags: ['city', 'adventure'] },
    { label: 'Balanced', emoji: '⚖️', tags: ['culture', 'food'] },
    { label: 'Slow & relaxed', emoji: '🧘', tags: ['beach', 'romantic', 'nature'] },
    { label: 'Spontaneous', emoji: '🎲', tags: ['nightlife'] },
  ] },
  { id: 'who', q: 'Who’s coming?', options: [
    { label: 'Friends', emoji: '👯', tags: ['nightlife', 'adventure'] },
    { label: 'Family with kids', emoji: '👨‍👩‍👧', tags: ['family'] },
    { label: 'Partner', emoji: '💞', tags: ['romantic'] },
    { label: 'Classmates / club', emoji: '🎒', tags: ['culture', 'city'] },
  ] },
];

export const packingBase: { category: string; items: string[] }[] = [
  { category: 'Documents', items: ['Passport', 'Visa / entry form', 'Travel insurance card', 'Printed itinerary', 'Driver’s licence'] },
  { category: 'Tech', items: ['Phone charger', 'Universal adapter', 'Power bank', 'Headphones', 'eSIM / SIM card'] },
  { category: 'Clothing', items: ['Comfortable walking shoes', 'Light jacket', 'Socks & underwear', 'Sleepwear', 'Swimsuit'] },
  { category: 'Toiletries', items: ['Toothbrush & paste', 'Sunscreen', 'Medication', 'Deodorant', 'Hand sanitiser'] },
  { category: 'Extras', items: ['Reusable water bottle', 'Day backpack', 'Snacks for the flight', 'Coin purse (cash-heavy countries)']
},
];

export const ROLE_OPTIONS = [
  'Organizer',
  'Treasurer',
  'Navigator',
  'Food Lead',
  'Photographer',
  'Planner',
  'Activities Lead',
  'Packing Lead',
  'DJ',
  'Member',
] as const;
export type Role = (typeof ROLE_OPTIONS)[number];

export const roleDescriptions: Record<Role, string> = {
  Organizer: 'Owns the trip, invites people and has the final say.',
  Treasurer: 'Tracks the budget and settles up who owes what.',
  Navigator: 'Gets everyone from A to B — maps, trains and transfers.',
  'Food Lead': 'Finds the best places to eat and books tables.',
  Photographer: 'Captures memories and curates the photo dump.',
  Planner: 'Builds the itinerary and keeps the schedule on track.',
  'Activities Lead': 'Researches activities and runs the group votes.',
  'Packing Lead': 'Owns the checklist and shared packing list.',
  DJ: 'In charge of road-trip playlists and karaoke picks.',
  Member: 'Along for the ride — votes, chats and has fun.',
};
```

src/hooks/useTrip.ts

```
import { useEffect, useState } from 'react';

import { destinationById } from '@/data/destinations';
import { fetchRates } from '@/services/currency';
import { fetchWeather } from '@/services/weather';
import { useAppStore } from '@/store/useAppStore';
import type { Trip, WeatherSnapshot } from '@/store/types';

/** Cached weather for a destination; refreshes in the background when online. */
export function useWeather(destinationId: string | undefined): WeatherSnapshot | undefined {
  const snap = useAppStore((s) => (destinationId ? s.weather[destinationId] : undefined));
  useEffect(() => {
    if (destinationId) fetchWeather(destinationId).catch(() => {});
  }, [destinationId]);
  return snap;
}

export function useTripDestination(trip: Trip | undefined) {
  return trip ? destinationById(trip.destinationId) : undefined;
}

/** Re-renders every `ms` so countdowns and "time ago" labels stay live. */
export function useNow(ms = 30000) {
  const [now, setNow] = useState(() => Date.now());
  useEffect(() => {
    const id = setInterval(() => setNow(Date.now()), ms);
    return () => clearInterval(id);
  }, [ms]);
  return now;
}

export function useRatesLoader() {
  useEffect(() => {
    fetchRates().catch(() => {});
  }, []);
}
```

src/i18n/index.ts

```
import { useCallback } from 'react';

import { useAppStore } from '@/store/useAppStore';

import { dictionaries, english, type LangCode, type StringKey } from './strings';

export { LANGUAGES } from './strings';
export type { LangCode, StringKey } from './strings';

export function translate(lang: LangCode, key: StringKey | string, vars?: Record<string, string | number>): string {
  const dict = dictionaries[lang] ?? {};
  let s: string = (dict as Record<string, string>)[key] ?? (english as Record<string, string>)[key] ?? key;
  if (vars) for (const [k, v] of Object.entries(vars)) s = s.replace(new RegExp(`\\{${k}\\}`, 'g'), String(v));
  return s;
}

/** `const t = useT(); t('nav.explore')` — re-renders when the language changes. */
export function useT() {
  const lang = useAppStore((s) => s.settings.language);
  return useCallback((key: StringKey | string, vars?: Record<string, string | number>) => translate(lang, key, vars), [lang]);
}
```

src/i18n/strings.ts

```
/* UI strings for the six supported languages. English is the source of truth;
   missing keys in other languages fall back to English. */

export const LANGUAGES = [
  { code: 'en', label: 'English', native: 'English', flag: '🇺🇸' },
  { code: 'es', label: 'Spanish', native: 'Español', flag: '🇪🇸' },
  { code: 'fr', label: 'French', native: 'Français', flag: '🇫🇷' },
  { code: 'ja', label: 'Japanese', native: '日本語', flag: '🇯🇵' },
  { code: 'hi', label: 'Hindi', native: 'हिन्दी', flag: '🇮🇳' },
  { code: 'zh', label: 'Chinese', native: '中文', flag: '🇨🇳' },
] as const;

export type LangCode = (typeof LANGUAGES)[number]['code'];

const en = {
  'nav.explore': 'Home',
  'nav.itinerary': 'Plan',
  'nav.budget': 'Budget',
  'nav.chat': 'Chat',
  'nav.profile': 'More',

  'greet.morning': 'Good morning',
  'greet.afternoon': 'Good afternoon',
  'greet.evening': 'Good evening',
  'greet.night': 'Hello night owl',

  'explore.title': 'Find Your Perfect\nDestination',
  'explore.subtitle': 'Plan, explore, and travel together.',
  'explore.search': 'Search a city or destination…',
  'explore.popular': 'Popular Places',
  'explore.featured': 'Featured Destination',
  'explore.seeAll': 'See all',
  'explore.matches': '{n} matches',
  'explore.nearby': 'Nearby within radius',
  'explore.activeTrip': 'Active trip',
  'explore.upcomingTrip': 'Upcoming trip',
  'explore.pastTrip': 'Completed trip',
  'explore.left': '{amount} left',
  'explore.members': '{n} members',
  'explore.reminder': 'Reminder',
  'explore.reminderSub': 'We'll ping everyone {n} min before.',
  'explore.nearestAirport': 'Nearest airport',
  'explore.addToTrip': 'Add to Trip',
  'explore.noResults': 'No destinations match "{q}"',
  'explore.tools': 'Trip tools',

  'cat.all': 'All',
  'cat.best': 'Best place',
  'cat.adventure': 'Adventures',
  'cat.food': 'Food',
  'cat.culture': 'Culture',
  'cat.beach': 'Beaches',
  'cat.nature': 'Nature',
  'cat.nightlife': 'Nightlife',
  'cat.sightseeing': 'Sightseeing',
  'cat.shopping': 'Shopping',
  'cat.relax': 'Relax',

  'len.any': 'Any length',
  'len.u1': 'Under 1 hour',
  'len.1-2': '1–2 hours',
  'len.2-4': '2–4 hours',
  'len.half': 'Half day',
  'len.full': 'Full day',

  'itin.members': '{n} members',
  'itin.noActivities': 'No activities yet',
  'itin.noActivitiesSub': 'Tap the + button to add your\nfirst activity for this day.',
  'itin.noMatch': 'No activities match these filters',
  'itin.addActivity': 'Add Activity',
  'itin.flights': 'Flights & arrivals',
  'itin.flightsSub': 'Times shown in {zone}. Tap to switch time zone.',
  'itin.addFlight': 'Add my flight',
  'itin.assigned': '{n} assigned',
  'itin.confirmed': 'Confirmed',
  'itin.voting': 'Voting',
  'itin.pending': 'Pending',
  'itin.perPerson': '/person',

  'add.title': 'Add Activity',
  'add.subtitle': 'Fill in the details for your new activity.',
  'add.type': 'Activity Type',
  'add.time': 'Time',
  'add.name': 'Activity Name',
```

```
  'add.namePh': 'e.g. Fushimi Inari Shrine visit',
  'add.location': 'Location',
  'add.locationPh': 'e.g. Fushimi, Kyoto',
  'add.duration': 'Duration',
  'add.cost': 'Cost per person (USD)',
  'add.notes': 'Notes (optional)',
  'add.notesPh': 'Any notes for the group…',
  'add.vote': 'Put it to a group vote',
  'add.remind': 'Remind everyone before it starts',
  'add.submit': 'Add to Itinerary',
  'add.save': 'Save changes',

  'budget.kicker': 'Group spending',
  'budget.title': 'Budget',
  'budget.spent': 'Spent so far',
  'budget.of': 'of {total} · {per} per person',
  'budget.used': '{pct}% of the group budget used',
  'budget.breakdown': 'Spending breakdown',
  'budget.currency': 'Currency',
  'budget.expenses': 'Expenses',
  'budget.cap': 'Budget cap',
  'budget.addExpense': 'Add an expense',
  'budget.whatFor': 'What was it for?',
  'budget.add': 'Add expense',
  'budget.settle': 'Settle up',
  'budget.planner': 'Daily spending planner',
  'budget.paidBy': 'paid by {name}',

  'chat.kicker': '{n} travellers',
  'chat.title': 'Trip chat',
  'chat.placeholder': 'Message your trip pals…',
  'chat.askAi': 'Ask the Trip Assistant…',
  'chat.assistant': 'Trip Assistant',
  'chat.typing': '{name} is typing…',
  'chat.q.budget': 'How’s the budget?',
  'chat.q.food': 'Best food nearby?',
  'chat.q.weather': 'What’s the weather?',
  'chat.q.next': 'What’s next?',

  'hub.kicker': 'Trip hub',
  'hub.title': 'Your group',
  'hub.invite': 'Invite code',
  'hub.share': 'Share',
  'hub.members': 'Members & roles',
  'hub.vote': 'Vote on activities',
  'hub.checklist': 'Checklist',
  'hub.done': '{a}/{b} done',
  'hub.addTask': 'Add a task',
  'hub.documents': 'Documents',
  'hub.documentsSub': 'Marked items stay readable without wifi or data.',
  'hub.offline': 'Offline',
  'hub.tools': 'More for this trip',

  'auth.welcome': 'Welcome back!',
  'auth.welcomeSub': 'Sign in to continue planning your adventures.',
  'auth.email': 'Email Address',
  'auth.password': 'Password',
  'auth.remember': 'Remember me',
  'auth.forgot': 'Forgot password?',
  'auth.signIn': 'Sign In',
  'auth.or': 'or continue with',
  'auth.google': 'Continue with Google',
  'auth.noAccount': 'Don’t have an account?',
  'auth.signUp': 'Sign Up',
  'auth.demo': 'Demo Account',
  'auth.terms': 'By continuing, you agree to our Terms of Service and Privacy Policy',

  'common.cancel': 'Cancel',
  'common.save': 'Save',
  'common.delete': 'Delete',
  'common.done': 'Done',
  'common.next': 'Next',
  'common.getStarted': 'Get Started',
  'common.skip': 'Skip',
  'common.back': 'Back',
  'common.edit': 'Edit',
  'common.close': 'Close',

  'settings.title': 'Settings',
  'settings.language': 'Language',
  'settings.currency': 'Home currency',
  'settings.notifications': 'Activity reminders',
  'settings.lead': 'Remind me before',
  'settings.textSize': 'Text size',
  'settings.signOut': 'Sign out',
};
```

```
export type StringKey = keyof typeof en;
type Dict = Partial<Record<StringKey, string>>;

const es: Dict = {
  'nav.explore': 'Inicio', 'nav.itinerary': 'Plan', 'nav.budget': 'Presupuesto', 'nav.chat': 'Chat', 'nav.profile': 'Más',
  'greet.morning': 'Buenos días', 'greet.afternoon': 'Buenas tardes', 'greet.evening': 'Buenas noches', 'greet.night': 'Hola, noctámbul
o',
  'explore.title': 'Encuentra tu\ndestino perfecto', 'explore.subtitle': 'Planea, explora y viaja en grupo.',
  'explore.search': 'Busca una ciudad o destino…', 'explore.popular': 'Lugares populares', 'explore.featured': 'Destino destacado',
  'explore.seeAll': 'Ver todo', 'explore.matches': '{n} resultados', 'explore.nearby': 'Cerca dentro del radio',
  'explore.activeTrip': 'Viaje activo', 'explore.upcomingTrip': 'Próximo viaje', 'explore.pastTrip': 'Viaje completado',
  'explore.left': 'Quedan {amount}', 'explore.members': '{n} miembros', 'explore.reminder': 'Recordatorio',
  'explore.reminderSub': 'Avisaremos a todos {n} min antes.', 'explore.nearestAirport': 'Aeropuerto más cercano',
  'explore.addToTrip': 'Añadir al viaje', 'explore.noResults': 'Ningún destino coincide con “{q}”', 'explore.tools': 'Herramientas',
  'cat.all': 'Todo', 'cat.best': 'Lo mejor', 'cat.adventure': 'Aventuras', 'cat.food': 'Comida', 'cat.culture': 'Cultura',
  'cat.beach': 'Playas', 'cat.nature': 'Naturaleza', 'cat.nightlife': 'Vida nocturna', 'cat.sightseeing': 'Turismo', 'cat.shopping': 'Co
mpras', 'cat.relax': 'Relax',
  'len.any': 'Cualquier duración', 'len.u1': 'Menos de 1 hora', 'len.1-2': '1–2 horas', 'len.2-4': '2–4 horas', 'len.half': 'Medio día',
'len.full': 'Día completo',
  'itin.members': '{n} miembros', 'itin.noActivities': 'Aún no hay actividades', 'itin.noActivitiesSub': 'Toca + para añadir la primera
\nactividad del día.',
  'itin.noMatch': 'Ninguna actividad coincide', 'itin.addActivity': 'Añadir actividad', 'itin.flights': 'Vuelos y llegadas',
  'itin.flightsSub': 'Horas en {zone}. Toca para cambiar la zona horaria.', 'itin.addFlight': 'Añadir mi vuelo', 'itin.assigned': '{n} a
signados',
  'itin.confirmed': 'Confirmado', 'itin.voting': 'Votación', 'itin.pending': 'Pendiente', 'itin.perPerson': '/persona',
  'add.title': 'Añadir actividad', 'add.subtitle': 'Completa los detalles de tu actividad.', 'add.type': 'Tipo de actividad', 'add.tim
e': 'Hora',
  'add.name': 'Nombre de la actividad', 'add.location': 'Ubicación', 'add.duration': 'Duración', 'add.cost': 'Coste por persona (USD)',
  'add.notes': 'Notas (opcional)', 'add.notesPh': 'Notas para el grupo…', 'add.vote': 'Someter a votación', 'add.remind': 'Recordar a to
dos antes',
  'add.submit': 'Añadir al itinerario', 'add.save': 'Guardar cambios',
  'budget.kicker': 'Gastos del grupo', 'budget.title': 'Presupuesto', 'budget.spent': 'Gastado hasta ahora', 'budget.of': 'de {total} ·
{per} por persona',
  'budget.used': '{pct}% del presupuesto usado', 'budget.breakdown': 'Desglose de gastos', 'budget.currency': 'Moneda', 'budget.expense
s': 'Gastos',
  'budget.cap': 'Límite', 'budget.addExpense': 'Añadir un gasto', 'budget.whatFor': '¿Para qué fue?', 'budget.add': 'Añadir gasto',
  'budget.settle': 'Saldar cuentas', 'budget.planner': 'Planificador diario', 'budget.paidBy': 'pagado por {name}',
  'chat.kicker': '{n} viajeros', 'chat.title': 'Chat del viaje', 'chat.placeholder': 'Escribe a tu grupo…', 'chat.askAi': 'Pregunta al a
sistente…',
  'chat.assistant': 'Asistente de viaje', 'chat.typing': '{name} está escribiendo…', 'chat.q.budget': '¿Cómo va el presupuesto?',
  'chat.q.food': '¿Buena comida cerca?', 'chat.q.weather': '¿Qué tiempo hace?', 'chat.q.next': '¿Qué sigue?',
  'hub.kicker': 'Centro del viaje', 'hub.title': 'Tu grupo', 'hub.invite': 'Código de invitación', 'hub.share': 'Compartir',
  'hub.members': 'Miembros y roles', 'hub.vote': 'Vota actividades', 'hub.checklist': 'Lista de tareas', 'hub.done': '{a}/{b} hechas',
  'hub.addTask': 'Añadir tarea', 'hub.documents': 'Documentos', 'hub.documentsSub': 'Disponibles sin wifi ni datos.', 'hub.offline': 'Si
n conexión', 'hub.tools': 'Más para este viaje',
  'auth.welcome': '¡Bienvenido de nuevo!', 'auth.welcomeSub': 'Inicia sesión para seguir planeando.', 'auth.email': 'Correo electrónic
o',
  'auth.password': 'Contraseña', 'auth.remember': 'Recordarme', 'auth.forgot': '¿Olvidaste tu contraseña?', 'auth.signIn': 'Iniciar sesi
ón',
  'auth.or': 'o continúa con', 'auth.google': 'Continuar con Google', 'auth.noAccount': '¿No tienes cuenta?', 'auth.signUp': 'Regístrat
e',
  'auth.demo': 'Cuenta de demostración', 'auth.terms': 'Al continuar, aceptas los Términos y la Política de privacidad',
  'common.cancel': 'Cancelar', 'common.save': 'Guardar', 'common.delete': 'Eliminar', 'common.done': 'Listo', 'common.next': 'Siguient
e',
  'common.getStarted': 'Empezar', 'common.skip': 'Omitir', 'common.back': 'Atrás', 'common.edit': 'Editar', 'common.close': 'Cerrar',
  'settings.title': 'Ajustes', 'settings.language': 'Idioma', 'settings.currency': 'Moneda local', 'settings.notifications': 'Recordator
ios',
  'settings.lead': 'Avisarme antes', 'settings.textSize': 'Tamaño del texto', 'settings.signOut': 'Cerrar sesión',
};

const fr: Dict = {
  'nav.explore': 'Accueil', 'nav.itinerary': 'Plan', 'nav.budget': 'Budget', 'nav.chat': 'Discussion', 'nav.profile': 'Plus',
  'greet.morning': 'Bonjour', 'greet.afternoon': 'Bon après-midi', 'greet.evening': 'Bonsoir', 'greet.night': 'Bonsoir, oiseau de nuit',
  'explore.title': 'Trouvez votre\ndestination idéale', 'explore.subtitle': 'Planifiez, explorez et voyagez ensemble.',
  'explore.search': 'Rechercher une ville ou destination…', 'explore.popular': 'Lieux populaires', 'explore.featured': 'Destination à la
une',
  'explore.seeAll': 'Tout voir', 'explore.matches': '{n} résultats', 'explore.nearby': 'À proximité dans le rayon',
  'explore.activeTrip': 'Voyage en cours', 'explore.upcomingTrip': 'Voyage à venir', 'explore.pastTrip': 'Voyage terminé',
  'explore.left': '{amount} restants', 'explore.members': '{n} membres', 'explore.reminder': 'Rappel', 'explore.reminderSub': 'On prévie
nt tout le monde {n} min avant.',
  'explore.nearestAirport': 'Aéroport le plus proche', 'explore.addToTrip': 'Ajouter au voyage', 'explore.noResults': 'Aucune destinatio
n pour « {q} »', 'explore.tools': 'Outils du voyage',
  'cat.all': 'Tout', 'cat.best': 'Incontournables', 'cat.adventure': 'Aventures', 'cat.food': 'Gastronomie', 'cat.culture': 'Culture',
  'cat.beach': 'Plages', 'cat.nature': 'Nature', 'cat.nightlife': 'Vie nocturne', 'cat.sightseeing': 'Visites', 'cat.shopping': 'Shoppin
g', 'cat.relax': 'Détente',
  'len.any': 'Toute durée', 'len.u1': 'Moins d’1 heure', 'len.1-2': '1–2 heures', 'len.2-4': '2–4 heures', 'len.half': 'Demi-journée',
'len.full': 'Journée entière',
  'itin.members': '{n} membres', 'itin.noActivities': 'Aucune activité', 'itin.noActivitiesSub': 'Touchez + pour ajouter la première\nac
tivité de la journée.',
  'itin.noMatch': 'Aucune activité ne correspond', 'itin.addActivity': 'Ajouter', 'itin.flights': 'Vols et arrivées',
  'itin.flightsSub': 'Heures affichées en {zone}. Touchez pour changer.', 'itin.addFlight': 'Ajouter mon vol', 'itin.assigned': '{n} ass
ignés',
  'itin.confirmed': 'Confirmé', 'itin.voting': 'Vote', 'itin.pending': 'En attente', 'itin.perPerson': '/pers.',
  'add.title': 'Ajouter une activité', 'add.subtitle': 'Renseignez les détails de l’activité.', 'add.type': 'Type d’activité', 'add.tim
e': 'Heure',
```

```
  'add.name': 'Nom de l’activité', 'add.location': 'Lieu', 'add.duration': 'Durée', 'add.cost': 'Coût par personne (USD)', 'add.notes': 'Notes (facultatif)',
  'add.notesPh': 'Des notes pour le groupe…', 'add.vote': 'Soumettre au vote', 'add.remind': 'Rappeler à tous avant le début', 'add.submit': 'Ajouter à l’itinéraire', 'add.save': 'Enregistrer',
  'budget.kicker': 'Dépenses du groupe', 'budget.title': 'Budget', 'budget.spent': 'Dépensé', 'budget.of': 'sur {total} · {per} par personne',
  'budget.used': '{pct} % du budget utilisé', 'budget.breakdown': 'Répartition', 'budget.currency': 'Devise', 'budget.expenses': 'Dépenses', 'budget.cap': 'Plafond',
  'budget.addExpense': 'Ajouter une dépense', 'budget.whatFor': 'Pour quoi ?', 'budget.add': 'Ajouter', 'budget.settle': 'Régler les comptes',
  'budget.planner': 'Planificateur quotidien', 'budget.paidBy': 'payé par {name}',
  'chat.kicker': '{n} voyageurs', 'chat.title': 'Discussion', 'chat.placeholder': 'Écrivez au groupe…', 'chat.askAi': 'Demandez à l’assistant…',
  'chat.assistant': 'Assistant voyage', 'chat.typing': '{name} écrit…', 'chat.q.budget': 'Où en est le budget ?', 'chat.q.food': 'Bonne cuisine près d’ici ?',
  'chat.q.weather': 'Quel temps fait-il ?', 'chat.q.next': 'Et ensuite ?',
  'hub.kicker': 'Espace voyage', 'hub.title': 'Votre groupe', 'hub.invite': 'Code d’invitation', 'hub.share': 'Partager', 'hub.members': 'Membres et rôles',
  'hub.vote': 'Votez pour les activités', 'hub.checklist': 'Liste de tâches', 'hub.done': '{a}/{b} faites', 'hub.addTask': 'Ajouter une tâche',
  'hub.documents': 'Documents', 'hub.documentsSub': 'Lisibles sans wifi ni données.', 'hub.offline': 'Hors ligne', 'hub.tools': 'Plus pour ce voyage',
  'auth.welcome': 'Bon retour !', 'auth.welcomeSub': 'Connectez-vous pour continuer à planifier.', 'auth.email': 'Adresse e-mail', 'auth.password': 'Mot de passe',
  'auth.remember': 'Se souvenir de moi', 'auth.forgot': 'Mot de passe oublié ?', 'auth.signIn': 'Se connecter', 'auth.or': 'ou continuer avec',
  'auth.google': 'Continuer avec Google', 'auth.noAccount': 'Pas de compte ?', 'auth.signUp': 'S’inscrire', 'auth.demo': 'Compte de démo',
  'auth.terms': 'En continuant, vous acceptez nos Conditions et notre Politique de confidentialité',
  'common.cancel': 'Annuler', 'common.save': 'Enregistrer', 'common.delete': 'Supprimer', 'common.done': 'Terminé', 'common.next': 'Suivant',
  'common.getStarted': 'Commencer', 'common.skip': 'Passer', 'common.back': 'Retour', 'common.edit': 'Modifier', 'common.close': 'Fermer',
  'settings.title': 'Réglages', 'settings.language': 'Langue', 'settings.currency': 'Devise principale', 'settings.notifications': 'Rappels d’activité',
  'settings.lead': 'Me rappeler avant', 'settings.textSize': 'Taille du texte', 'settings.signOut': 'Se déconnecter',
};

const ja: Dict = {
  'nav.explore': 'ホーム', 'nav.itinerary': '計画', 'nav.budget': '予算', 'nav.chat': 'チャット', 'nav.profile': 'その他',
  'greet.morning': 'おはようございます', 'greet.afternoon': 'こんにちは', 'greet.evening': 'こんばんは', 'greet.night': 'こんばんは',
  'explore.title': 'ぴったりの\n旅先を見つけよう', 'explore.subtitle': 'みんなで計画して、探して、旅しよう。', 'explore.search': '都市や旅先を検索…',
  'explore.popular': '人気の場所', 'explore.featured': '注目の旅先', 'explore.seeAll': 'すべて見る', 'explore.matches': '{n}件',
  'explore.nearby': '半径内のスポット', 'explore.activeTrip': '旅行中', 'explore.upcomingTrip': '次の旅行', 'explore.pastTrip': '完了した旅行',
  'explore.left': '残り{amount}', 'explore.members': '{n}人', 'explore.reminder': 'リマインダー', 'explore.reminderSub': '{n}分前に全員に通知します。',
  'explore.nearestAirport': '最寄りの空港', 'explore.addToTrip': '旅行に追加', 'explore.noResults': '「{q}」に一致する旅先はありません', 'explore.tools': '旅のツール',
  'cat.all': 'すべて', 'cat.best': 'おすすめ', 'cat.adventure': 'アドベンチャー', 'cat.food': 'グルメ', 'cat.culture': '文化', 'cat.beach': 'ビーチ',
  'cat.nature': '自然', 'cat.nightlife': 'ナイトライフ', 'cat.sightseeing': '観光', 'cat.shopping': 'ショッピング', 'cat.relax': 'リラックス',
  'len.any': 'すべての長さ', 'len.u1': '1時間未満', 'len.1-2': '1〜2時間', 'len.2-4': '2〜4時間', 'len.half': '半日', 'len.full': '1日',
  'itin.members': '{n}人', 'itin.noActivities': 'まだ予定はありません', 'itin.noActivitiesSub': '＋ボタンでこの日の\n最初の予定を追加しましょう。',
  'itin.noMatch': '条件に合う予定はありません', 'itin.addActivity': '予定を追加', 'itin.flights': 'フライトと到着', 'itin.flightsSub': '{zone}で表示中。タップで切替。',
  'itin.addFlight': 'フライトを追加', 'itin.assigned': '{n}人担当', 'itin.confirmed': '確定', 'itin.voting': '投票中', 'itin.pending': '保留', 'itin.perPerson': '/人',
  'add.title': '予定を追加', 'add.subtitle': '新しい予定の詳細を入力してください。', 'add.type': '種類', 'add.time': '時間', 'add.name': '予定の名前',
  'add.location': '場所', 'add.duration': '所要時間', 'add.cost': '1人あたりの費用 (USD)', 'add.notes': 'メモ（任意）', 'add.notesPh': 'グループへのメモ…',
  'add.vote': 'グループ投票にする', 'add.remind': '開始前に全員に通知', 'add.submit': '旅程に追加', 'add.save': '変更を保存',
  'budget.kicker': 'グループの支出', 'budget.title': '予算', 'budget.spent': 'これまでの支出', 'budget.of': '{total}中 · 1人あたり{per}',
  'budget.used': 'グループ予算の{pct}%を使用', 'budget.breakdown': '支出の内訳', 'budget.currency': '通貨', 'budget.expenses': '支出', 'budget.cap': '予算上限',
  'budget.addExpense': '支出を追加', 'budget.whatFor': '何の支出？', 'budget.add': '支出を追加', 'budget.settle': '精算', 'budget.planner': '1日の予算プランナー', 'budget.paidBy': '{name}が支払い',
  'chat.kicker': '旅行者{n}人', 'chat.title': '旅のチャット', 'chat.placeholder': '旅の仲間にメッセージ…', 'chat.askAi': 'アシスタントに質問…',
  'chat.assistant': '旅のアシスタント', 'chat.typing': '{name}が入力中…', 'chat.q.budget': '予算はどう？', 'chat.q.food': '近くのおすすめグルメは？',
  'chat.q.weather': '天気は？', 'chat.q.next': '次の予定は？',
  'hub.kicker': '旅のハブ', 'hub.title': 'あなたのグループ', 'hub.invite': '招待コード', 'hub.share': '共有', 'hub.members': 'メンバーと役割',
  'hub.vote': 'アクティビティに投票', 'hub.checklist': 'チェックリスト', 'hub.done': '{a}/{b} 完了', 'hub.addTask': 'タスクを追加',
  'hub.documents': '書類', 'hub.documentsSub': 'Wi-Fiや通信なしでも閲覧できます。', 'hub.offline': 'オフライン', 'hub.tools': 'この旅のツール',
  'auth.welcome': 'おかえりなさい！', 'auth.welcomeSub': 'ログインして旅の計画を続けましょう。', 'auth.email': 'メールアドレス', 'auth.password': 'パスワード',
  'auth.remember': 'ログイン状態を保持', 'auth.forgot': 'パスワードをお忘れですか？', 'auth.signIn': 'ログイン', 'auth.or': 'または',
  'auth.google': 'Googleで続行', 'auth.noAccount': 'アカウントをお持ちでないですか？', 'auth.signUp': '新規登録', 'auth.demo': 'デモアカウント',
  'auth.terms': '続行すると、利用規約とプライバシーポリシーに同意したことになります',
  'common.cancel': 'キャンセル', 'common.save': '保存', 'common.delete': '削除', 'common.done': '完了', 'common.next': '次へ', 'common.getStarted': 'はじめる',
  'common.skip': 'スキップ', 'common.back': '戻る', 'common.edit': '編集', 'common.close': '閉じる',
  'settings.title': '設定', 'settings.language': '言語', 'settings.currency': '自国通貨', 'settings.notifications': '予定のリマインダー',
  'settings.lead': '通知のタイミング', 'settings.textSize': '文字サイズ', 'settings.signOut': 'ログアウト',
};
```

```
const hi: Dict = {
  'nav.explore': 'होम', 'nav.itinerary': 'योजना', 'nav.budget': 'बजट', 'nav.chat': 'चैट', 'nav.profile': 'और',
  'greet.morning': 'सुप्रभात', 'greet.afternoon': 'नमस्ते', 'greet.evening': 'शुभ संध्या', 'greet.night': 'नमस्ते',
  'explore.title': 'अपनी परफ़ेक्ट\nमंज़िल खोजें', 'explore.subtitle': 'साथ मिलकर योजना बनाएँ, घूमें और यात्रा करें।', 'explore.search': 'शहर या जगह खोजें…',
  'explore.popular': 'लोकप्रिय जगहें', 'explore.featured': 'विशेष मंज़िल', 'explore.seeAll': 'सभी देखें', 'explore.matches': '{n} परिणाम',
  'explore.nearby': 'दायरे में आस-पास', 'explore.activeTrip': 'चालू यात्रा', 'explore.upcomingTrip': 'आगामी यात्रा', 'explore.pastTrip': 'पूरी हुई यात्रा',
  'explore.left': '{amount} बाकी', 'explore.members': '{n} सदस्य', 'explore.reminder': 'रिमाइंडर', 'explore.reminderSub': 'हम सबको {n} मिनट पहले याद दिलाएँगे।',
  'explore.nearestAirport': 'नज़दीकी हवाई अड्डा', 'explore.addToTrip': 'यात्रा में जोड़ें', 'explore.noResults': '“{q}” से कोई मंज़िल नहीं मिली', 'explore.tools': 'यात्रा टूल',
  'cat.all': 'सभी', 'cat.best': 'सर्वश्रेष्ठ', 'cat.adventure': 'रोमांच', 'cat.food': 'खाना', 'cat.culture': 'संस्कृति', 'cat.beach': 'समुद्र तट',
  'cat.nature': 'प्रकृति', 'cat.nightlife': 'नाइटलाइफ़', 'cat.sightseeing': 'दर्शनीय स्थल', 'cat.shopping': 'खरीदारी', 'cat.relax': 'आराम',
  'len.any': 'कोई भी अवधि', 'len.u1': '1 घंटे से कम', 'len.1-2': '1–2 घंटे', 'len.2-4': '2–4 घंटे', 'len.half': 'आधा दिन', 'len.full': 'पूरा दिन',
  'itin.members': '{n} सदस्य', 'itin.noActivities': 'अभी कोई गतिविधि नहीं', 'itin.noActivitiesSub': 'इस दिन की पहली गतिविधि जोड़ने के लिए\n+ बटन दबाएँ।',
  'itin.noMatch': 'इन फ़िल्टर से कोई गतिविधि नहीं मिली', 'itin.addActivity': 'गतिविधि जोड़ें', 'itin.flights': 'उड़ानें और आगमन',
  'itin.flightsSub': 'समय {zone} में दिखाया गया है। बदलने के लिए टैप करें।', 'itin.addFlight': 'मेरी उड़ान जोड़ें', 'itin.assigned': '{n} ज़िम्मेदार',
  'itin.confirmed': 'पक्का', 'itin.voting': 'वोटिंग', 'itin.pending': 'बाकी', 'itin.perPerson': '/व्यक्ति',
  'add.title': 'गतिविधि जोड़ें', 'add.subtitle': 'नई गतिविधि का विवरण भरें।', 'add.type': 'गतिविधि का प्रकार', 'add.time': 'समय', 'add.name': 'गतिविधि का नाम',
  'add.location': 'स्थान', 'add.duration': 'अवधि', 'add.cost': 'प्रति व्यक्ति खर्च (USD)', 'add.notes': 'नोट्स (वैकल्पिक)', 'add.notesPh': 'समूह के लिए नोट्स…',
  'add.vote': 'समूह वोट के लिए रखें', 'add.remind': 'शुरू होने से पहले सबको याद दिलाएँ', 'add.submit': 'यात्रा योजना में जोड़ें', 'add.save': 'बदलाव सहेजें',
  'budget.kicker': 'समूह खर्च', 'budget.title': 'बजट', 'budget.spent': 'अब तक खर्च', 'budget.of': '{total} में से · {per} प्रति व्यक्ति',
  'budget.used': 'समूह बजट का {pct}% इस्तेमाल', 'budget.breakdown': 'खर्च का ब्यौरा', 'budget.currency': 'मुद्रा', 'budget.expenses': 'खर्च', 'budget.cap': 'बजट सीमा',
  'budget.addExpense': 'खर्च जोड़ें', 'budget.whatFor': 'किस लिए?', 'budget.add': 'खर्च जोड़ें', 'budget.settle': 'हिसाब बराबर करें', 'budget.planner': 'दैनिक खर्च योजना', 'budget.paidBy': '{name} ने भुगतान किया',
  'chat.kicker': '{n} यात्री', 'chat.title': 'यात्रा चैट', 'chat.placeholder': 'अपने साथियों को संदेश भेजें…', 'chat.askAi': 'ट्रिप असिस्टेंट से पूछें…',
  'chat.assistant': 'ट्रिप असिस्टेंट', 'chat.typing': '{name} लिख रहे हैं…', 'chat.q.budget': 'बजट कैसा है?', 'chat.q.food': 'आस-पास अच्छा खाना?',
  'chat.q.weather': 'मौसम कैसा है?', 'chat.q.next': 'आगे क्या है?',
  'hub.kicker': 'ट्रिप हब', 'hub.title': 'आपका समूह', 'hub.invite': 'आमंत्रण कोड', 'hub.share': 'शेयर', 'hub.members': 'सदस्य और भूमिकाएँ',
  'hub.vote': 'गतिविधियों पर वोट करें', 'hub.checklist': 'चेकलिस्ट', 'hub.done': '{a}/{b} पूरे', 'hub.addTask': 'काम जोड़ें', 'hub.documents': 'दस्तावेज़',
  'hub.documentsSub': 'बिना वाई-फ़ाई या डेटा के भी पढ़ सकते हैं।', 'hub.offline': 'ऑफ़लाइन', 'hub.tools': 'इस यात्रा के लिए और',
  'auth.welcome': 'फिर से स्वागत है!', 'auth.welcomeSub': 'अपनी यात्रा की योजना जारी रखने के लिए साइन इन करें।', 'auth.email': 'ईमेल पता', 'auth.password': 'पासवर्ड',
  'auth.remember': 'मुझे याद रखें', 'auth.forgot': 'पासवर्ड भूल गए?', 'auth.signIn': 'साइन इन', 'auth.or': 'या इसके साथ जारी रखें',
  'auth.google': 'Google के साथ जारी रखें', 'auth.noAccount': 'खाता नहीं है?', 'auth.signUp': 'साइन अप', 'auth.demo': 'डेमो खाता',
  'auth.terms': 'जारी रखकर आप हमारी सेवा की शर्तों और गोपनीयता नीति से सहमत होते हैं',
  'common.cancel': 'रद्द करें', 'common.save': 'सहेजें', 'common.delete': 'हटाएँ', 'common.done': 'हो गया', 'common.next': 'आगे', 'common.getStarted': 'शुरू करें',
  'common.skip': 'छोड़ें', 'common.back': 'वापस', 'common.edit': 'बदलें', 'common.close': 'बंद करें',
  'settings.title': 'सेटिंग्स', 'settings.language': 'भाषा', 'settings.currency': 'घरेलू मुद्रा', 'settings.notifications': 'गतिविधि रिमाइंडर',
  'settings.lead': 'पहले याद दिलाएँ', 'settings.textSize': 'टेक्स्ट का आकार', 'settings.signOut': 'साइन आउट',
};

const zh: Dict = {
  'nav.explore': '首页', 'nav.itinerary': '计划', 'nav.budget': '预算', 'nav.chat': '聊天', 'nav.profile': '更多',
  'greet.morning': '早上好', 'greet.afternoon': '下午好', 'greet.evening': '晚上好', 'greet.night': '夜深了',
  'explore.title': '发现你的\n理想目的地', 'explore.subtitle': '一起计划、探索、出发。', 'explore.search': '搜索城市或目的地…',
  'explore.popular': '热门地点', 'explore.featured': '精选目的地', 'explore.seeAll': '查看全部', 'explore.matches': '{n} 个结果',
  'explore.nearby': '半径范围内的地点', 'explore.activeTrip': '进行中的旅行', 'explore.upcomingTrip': '即将出发', 'explore.pastTrip': '已完成的旅行',
  'explore.left': '剩余 {amount}', 'explore.members': '{n} 位成员', 'explore.reminder': '提醒', 'explore.reminderSub': '我们会提前 {n} 分钟通知大家。',
  'explore.nearestAirport': '最近的机场', 'explore.addToTrip': '加入旅行', 'explore.noResults': '没有与“{q}”匹配的目的地', 'explore.tools': '旅行工具',
  'cat.all': '全部', 'cat.best': '最佳', 'cat.adventure': '探险', 'cat.food': '美食', 'cat.culture': '文化', 'cat.beach': '海滩', 'cat.nature': '自然',
  'cat.nightlife': '夜生活', 'cat.sightseeing': '观光', 'cat.shopping': '购物', 'cat.relax': '放松',
  'len.any': '不限时长', 'len.u1': '1 小时以内', 'len.1-2': '1–2 小时', 'len.2-4': '2–4 小时', 'len.half': '半天', 'len.full': '全天',
  'itin.members': '{n} 位成员', 'itin.noActivities': '还没有活动', 'itin.noActivitiesSub': '点击 + 按钮添加\n当天的第一个活动。', 'itin.noMatch': '没有符合筛选的活动',
  'itin.addActivity': '添加活动', 'itin.flights': '航班与抵达', 'itin.flightsSub': '时间以 {zone} 显示，点击切换时区。', 'itin.addFlight': '添加我的航班',
  'itin.assigned': '{n} 人负责', 'itin.confirmed': '已确认', 'itin.voting': '投票中', 'itin.pending': '待定', 'itin.perPerson': '/人',
  'add.title': '添加活动', 'add.subtitle': '填写新活动的详细信息。', 'add.type': '活动类型', 'add.time': '时间', 'add.name': '活动名称', 'add.location': '地点',
  'add.duration': '时长', 'add.cost': '人均费用 (USD)', 'add.notes': '备注（可选）', 'add.notesPh': '给大家的备注…', 'add.vote': '发起小组投票',
  'add.remind': '开始前提醒所有人', 'add.submit': '加入行程', 'add.save': '保存修改',
  'budget.kicker': '小组开销', 'budget.title': '预算', 'budget.spent': '目前已花费', 'budget.of': '共 {total} · 人均 {per}', 'budget.used': '已使用小组预算的 {pct}%',
  'budget.breakdown': '支出明细', 'budget.currency': '货币', 'budget.expenses': '支出', 'budget.cap': '预算上限', 'budget.addExpense': '添加支出',
  'budget.whatFor': '用途是什么？', 'budget.add': '添加支出', 'budget.settle': '结算', 'budget.planner': '每日花费规划', 'budget.paidBy': '{name} 支付',
  'chat.kicker': '{n} 位旅伴', 'chat.title': '旅行群聊', 'chat.placeholder': '给旅伴发消息…', 'chat.askAi': '问问旅行助手…', 'chat.assistant': '旅行助手',
  'chat.typing': '{name} 正在输入…', 'chat.q.budget': '预算怎么样？', 'chat.q.food': '附近有什么好吃的？', 'chat.q.weather': '天气如何？', 'chat.q.next': '接下来做什么？',
  'hub.kicker': '旅行中心', 'hub.title': '你的小组', 'hub.invite': '邀请码', 'hub.share': '分享', 'hub.members': '成员与分工', 'hub.vote': '为活动投票',
  'hub.checklist': '清单', 'hub.done': '已完成 {a}/{b}', 'hub.addTask': '添加任务', 'hub.documents': '文件', 'hub.documentsSub': '无网络时也能查看。',
  'hub.offline': '离线可用', 'hub.tools': '更多旅行工具',
  'auth.welcome': '欢迎回来！', 'auth.welcomeSub': '登录以继续规划你的旅程。', 'auth.email': '电子邮箱', 'auth.password': '密码', 'auth.remember': '记住我',
```

```
  'auth.forgot': '忘记密码？', 'auth.signIn': '登录', 'auth.or': '或使用以下方式', 'auth.google': '使用 Google 继续', 'auth.noAccount': '还没有账号？',
  'auth.signUp': '注册', 'auth.demo': '演示账号', 'auth.terms': '继续即表示你同意我们的服务条款和隐私政策',
  'common.cancel': '取消', 'common.save': '保存', 'common.delete': '删除', 'common.done': '完成', 'common.next': '下一步', 'common.getStarted': '开始使用',
  'common.skip': '跳过', 'common.back': '返回', 'common.edit': '编辑', 'common.close': '关闭',
  'settings.title': '设置', 'settings.language': '语言', 'settings.currency': '本国货币', 'settings.notifications': '活动提醒', 'settings.lead': '提前提醒',
  'settings.textSize': '文字大小', 'settings.signOut': '退出登录',
};

export const dictionaries: Record<LangCode, Dict> = { en, es, fr, ja, hi, zh };
export const english = en;
```

```
/**
 * Trip Assistant.
 * 1. Offline engine (default): intent matching over the live trip data, so it
 *    answers accurately with no internet — important at venues with bad wifi.
 * 2. Claude mode (optional): when a user adds their own Anthropic API key in
 *    Settings, questions go to Claude with the trip as context. Any failure
 *    falls back to the offline engine.
 */
import Anthropic from '@anthropic-ai/sdk';

import { destinationById } from '@/data/destinations';
import { placesFor } from '@/data/places';
import { currencies, emergencyByCountry, phrasebook } from '@/data/reference';
import { balances, settlePlan, totalSpent } from '@/store/useAppStore';
import type { ChatMessage, RatesSnapshot, Trip, WeatherSnapshot } from '@/store/types';
import { durationLabel, money } from '@/utils/format';
import { distanceKm, kmLabel, nearestAirports } from '@/utils/geo';
import { addDays, dayLabel, deviceTimeZone, formatInTz, hoursAhead, partsInTz, time12, todayInTz, tzAbbrev } from '@/utils/time';

import { convert } from './currency';
import { cToF, weatherInfo } from './weather';

export const CLAUDE_MODEL = 'claude-opus-5-5';

export interface AssistantContext {
  trip: Trip;
  userId: string;
  weather?: WeatherSnapshot;
  rates: RatesSnapshot | null;
  homeCurrency: string;
}

const first = (t: Trip, id: string) => t.members.find((m) => m.id === id)?.name.split(' ')[0] ?? 'Someone';

const CURRENCY_WORDS: Record<string, string> = {
  dollar: 'USD', dollars: 'USD', usd: 'USD', $: 'USD', yen: 'JPY', jpy: 'JPY', '¥': 'JPY', euro: 'EUR', euros: 'EUR', eur: 'EUR', '€':
'EUR',
  pound: 'GBP', pounds: 'GBP', gbp: 'GBP', '£': 'GBP', won: 'KRW', krw: 'KRW', baht: 'THB', thb: 'THB', rupee: 'INR', rupees: 'INR', in
r: 'INR',
  peso: 'MXN', pesos: 'MXN', mxn: 'MXN', franc: 'CHF', francs: 'CHF', chf: 'CHF', cad: 'CAD', aud: 'AUD', sgd: 'SGD', yuan: 'CNY', cny:
'CNY',
};

function currencyFrom(word: string | undefined): string | undefined {
  if (!word) return undefined;
  const w = word.toLowerCase();
  if (CURRENCY_WORDS[w]) return CURRENCY_WORDS[w];
  const up = w.toUpperCase();
  return currencies.some((c) => c.code === up) ? up : undefined;
}

export function answerLocally(question: string, ctx: AssistantContext): string {
  const q = question.toLowerCase().trim();
  const { trip } = ctx;
  const dest = destinationById(trip.destinationId);
  const tz = dest?.tz ?? 'UTC';
  const spent = totalSpent(trip);
  const per = spent / Math.max(1, trip.members.length);
  const has = (re: RegExp) => re.test(q);

  // Currency conversion: "convert 50 usd to jpy", "how much is 3000 yen"
  const conv = q.match(/([\d][\d,.]*)\s*([a-z$€£¥]{1,8})?\s*(?:to|in|into)?\s*([a-z$€£¥]{3,8})?/);
  if (has(/convert|exchange|how much is|in (yen|dollars|euros|usd|jpy|eur)|→/) && conv) {
    const amount = Number(conv[1].replace(/,/g, ''));
    const from = currencyFrom(conv[2]) ?? (has(/yen|jpy/) && !has(/to (yen|jpy)/) ? 'JPY' : 'USD');
    let to = currencyFrom(conv[3]) ?? (from === (dest?.currency ?? 'JPY') ? ctx.homeCurrency : dest?.currency ?? 'JPY');
    if (to === from) to = from === 'USD' ? dest?.currency ?? 'EUR' : 'USD';
    const out = convert(amount, from, to, ctx.rates);
    return `${money(amount, from, { decimals: true })} ≈ ${money(out, to, { decimals: true })} ${ctx.rates?.live ? '(live rate)' : '(off
line reference rate)'}. Tip: the Budget tab has a full converter with home/local view.`;
  }

  if (has(/owe|settle|split|pay (back|me)|who pays|balance/)) {
    const plan = settlePlan(trip);
    if (!plan.length) return 'Everyone is square right now — no one owes anything. 🎉';
    const lines = plan.slice(0, 5).map((p) => `• ${first(trip, p.from)} → ${first(trip, p.to)}: ${money(p.amount, 'USD', { decimals: tru
e })}`);
    const mine = balances(trip)[ctx.userId] ?? 0;
    return `Here's the simplest way to settle up (${plan.length} transfer${plan.length > 1 ? 's' : ''}):\n${lines.join('\n')}\n\nYou are
${mine >= 0 ? `owed ${money(mine)}` : `owing ${money(-mine)}`}. Tap "Settle up" in Budget to record payments.`;
  }

  if (has(/budget|spent|spend|money|afford|left|expens|cost so far/)) {
    const pct = Math.round((spent / Math.max(1, trip.budget)) * 100);
```

```
    const byCat: Record<string, number> = {};
    trip.expenses.filter((e) => !e.settlement).forEach((e) => (byCat[e.category] = (byCat[e.category] ?? 0) + e.amount));
    const top = Object.entries(byCat).sort((a, b) => b[1] - a[1])[0];
    const remaining = trip.budget - spent;
    const days = Math.max(1, Math.round((Date.parse(trip.endDate) - Date.parse(trip.startDate)) / 86400000) + 1);
    return `You've committed ${money(spent)} of ${money(trip.budget)} — about ${pct}% of the group budget (${money(per)} per person).
${top ? `Biggest category: ${top[0]} (${money(top[1])}). ` : ''}${remaining >= 0 ? `${money(remaining)} left ≈ ${money(remaining / days
/ trip.members.length)} per person per day.` : `You're ${money(-remaining)} over — Food is usually the easiest place to trim.`}`;
  }

  if (has(/weather|rain|temperature|forecast|umbrella|cold|hot|sunny|snow|degrees/)) {
    const w = ctx.weather;
    if (!w) return `I don't have a forecast for ${dest?.city ?? 'the destination'} yet — open the Weather screen while online and I'll r
emember it for offline use.`;
    const now = weatherInfo(w.current.code);
    const tomorrow = w.daily[1];
    const rainy = w.daily.slice(0, 3).some((d) => d.rain >= 50);
    return `${dest?.city}: ${w.current.temp}°C (${cToF(w.current.temp)}°F), ${now.label.toLowerCase()}. ${tomorrow ? `Tomorrow ${tomorro
w.min}–${tomorrow.max}°C, ${tomorrow.rain}% chance of rain. ` : ''}${rainy ? 'Pack an umbrella ☔ — rain is likely in the next few days.'
: 'Looks good for walking days. 👟'}${w.live ? '' : ' (estimated — connect to refresh)'}`;
  }

  if (has(/food|eat|restaurant|dinner|lunch|breakfast|ramen|sushi|hungry|snack|cafe|coffee/)) {
    const cheap = has(/cheap|budget|afford|under/);
    const center = dest ? { lat: dest.lat, lng: dest.lng } : null;
    const list = placesFor(trip.destinationId)
      .concat(trip.extraDestinationIds.flatMap((id) => placesFor(id)))
      .filter((p) => p.category === 'food' && (!cheap || p.cost <= 20))
      .map((p) => ({ p, km: center ? distanceKm(center, p) : 0 }))
      .sort((a, b) => b.p.rating - a.p.rating)
      .slice(0, 3);
    if (!list.length) return 'I couldn't find food spots for this destination yet — try the Nearby screen to widen the radius.';
    return `Top-rated food${cheap ? ' on a budget' : ''} near ${dest?.city}:\n${list
      .map(({ p, km }) => `• ${p.name} (${p.area}) — ★${p.rating}, ~${p.cost ? money(p.cost) : 'free'}/person, ${kmLabel(km)} from centr
e`)
      .join('\n')}\nWant one added to the itinerary? Open Nearby and tap "Add".`;
  }

  if (has(/flight|land|arriv|depart|airport|plane/)) {
    const arrivals = trip.flights.filter((f) => f.direction === 'arrival').sort((a, b) => a.arriveUtc - b.arriveUtc);
    const missing = trip.members.filter((m) => !trip.flights.some((f) => f.memberId === m.id)).map((m) => m.name.split(' ')[0]);
    const air = dest ? nearestAirports(dest, 1)[0] : undefined;
    const lines = arrivals.map((f) => `• ${first(trip, f.memberId)} — ${f.flightNo}, lands ${formatInTz(f.arriveUtc, tz)} ${tzAbbrev(t
z)}`);
    return `${lines.length ? `Arrivals:\n${lines.join('\n')}` : 'No flights added yet.'}${missing.length ? `\n${missing.join(', ')} stil
l need to add flights.` : ''}${air ? `\nNearest airport to ${dest?.city} centre: ${air.iata} (${air.name}).` : ''}`;
  }

  if (has(/time zone|timezone|what time|jet ?lag|time difference/)) {
    const home = deviceTimeZone();
    const diff = hoursAhead(tz, home);
    const p = partsInTz(Date.now(), tz);
    return `It's ${time12(p.hhmm)} in ${dest?.city} (${tzAbbrev(tz)}). That's ${diff === 0 ? 'the same as' : `${Math.abs(diff)}h ${diff
> 0 ? 'ahead of' : 'behind'}`} your phone's time zone. Itinerary times are always shown in local trip time.${Math.abs(diff) >= 6 ? ' Jet
-lag tip: get daylight on arrival day and avoid naps over 30 min.' : ''}`;
  }

  if (has(/how (do|to) (i |you |we )?say|translate|phrase|in japanese|in french|in spanish/)) {
    const book = dest ? phrasebook[dest.language] : undefined;
    if (!book) return 'Open the Phrasebook from the menu for common phrases with audio.';
    const hit = book.phrases.find((p) => q.includes(p.en.toLowerCase().replace(/[?!]/g, '').split(' / ')[0]));
    if (hit) return `"${hit.en}" in ${book.lang}: ${hit.local}${hit.roman ? ` (${hit.roman})` : ''}. Tap the speaker in the Phrasebook t
o hear it.`;
    return `Handy ${book.lang} phrases:\n${book.phrases.slice(0, 4).map((p) => `• ${p.en} — ${p.local}${p.roman ? ` (${p.roman})` : ''}
`).join('\n')}`;
  }

  if (has(/emergency|police|hospital|ambulance|lost passport|help me|doctor|stolen/)) {
    const em = emergencyByCountry[dest?.countryCode ?? ''] ?? emergencyByCountry.DEFAULT;
    return `In ${dest?.country ?? 'this country'}: Police ${em.police} · Ambulance ${em.ambulance} · Fire ${em.fire}.\n${em.tips[0] ??
''}\nThe Emergency screen works offline and can share your location by text.`;
  }

  if (has(/pack|bring|luggage|suitcase/)) {
    const left = trip.packing.filter((p) => !p.packed);
    const rainy = ctx.weather?.daily.slice(0, 5).some((d) => d.rain >= 50);
    return `${left.length ? `Still to pack (${left.length}): ${left.slice(0, 5).map((p) => p.text).join(', ')}${left.length > 5 ? '…' :
''}.` : 'Everything on the packing list is packed! 🎒'}${rainy ? ' Rain is forecast — add an umbrella.' : ''} ${dest?.countryCode === 'J
P' ? 'Japan uses type A plugs (100V) and is still cash-friendly — bring a coin purse.' : ''}`.trim();
  }

  if (has(/vote|poll|decide|undecided/)) {
    const open = trip.polls.filter((p) => !p.closed);
    const voting = trip.activities.filter((a) => a.status === 'voting');
    const pollLines = open.map((p) => {
      const leader = p.options.slice().sort((a, b) => b.votes.length - a.votes.length)[0];
```

```
      const voted = p.options.some((o) => o.votes.includes(ctx.userId));
      return `• “${p.question}” — leading: ${leader?.text ?? '—'}${voted ? '' : ' (you haven’t voted)'}`;
    });
    return `${pollLines.length ? `Open polls:\n${pollLines.join('\n')}` : 'No open polls.'}${voting.length ? `\nActivities waiting on votes: ${voting.map((a) => a.title).join(', ')}.` : ''}`;
  }

  if (has(/task|checklist|to-?do|remaining|left to do/)) {
    const open = trip.tasks.filter((k) => !k.done);
    if (!open.length) return 'The checklist is all done ✅';
    return `${open.length} task${open.length > 1 ? 's' : ''} left:\n${open.map((k) => `• ${k.text}${k.assignee ? ` — ${first(trip, k.assignee)}` : ''}`).join('\n')}`;
  }

  if (has(/who('s| is)|role|treasurer|navigator|photographer|members?/)) {
    return `Roles for ${trip.name}:\n${trip.members.map((m) => `• ${m.name.split(' ')[0]} — ${m.role}`).join('\n')}\nChange roles in Profile → Members & roles.`;
  }

  const dayMatch = q.match(/day\s*(\d{1,2})/);
  if (has(/next|today|tomorrow|schedule|itinerary|plan for|what('s| is) on/) || dayMatch) {
    const today = todayInTz(tz);
    let target = today;
    if (has(/tomorrow/)) target = addDays(today, 1);
    if (dayMatch) target = addDays(trip.startDate, Number(dayMatch[1]) - 1);
    if (has(/next/) && !dayMatch) {
      const nowP = partsInTz(Date.now(), tz);
      const upcoming = trip.activities
        .filter((a) => a.date > nowP.ymd || (a.date === nowP.ymd && a.time >= nowP.hhmm))
        .sort((a, b) => (a.date + a.time).localeCompare(b.date + b.time))[0];
      if (!upcoming) return 'Nothing else is scheduled — time to plan something fun! Try “suggest something to do”.';
      return `Next up: ${upcoming.title} — ${dayLabel(upcoming.date)} at ${time12(upcoming.time)}, ${upcoming.location} (${durationLabel(upcoming.durationMin)}). ${upcoming.reminder ? 'Reminder is on 🔔' : 'Turn on the bell to get a reminder.'}`;
    }
    if (target < trip.startDate) target = trip.startDate;
    const acts = trip.activities.filter((a) => a.date === target).sort((a, b) => a.time.localeCompare(b.time));
    if (!acts.length) return `${dayLabel(target)} is a free day so far. Want ideas? Ask “what should we do?”`;
    return `${dayLabel(target)}:\n${acts.map((a) => `• ${time12(a.time)} ${a.title}${a.status !== 'confirmed' ? ` (${a.status})` : ''}`).join('\n')}`;
  }

  if (has(/what (should|can|could) we do|idea|suggest|recommend|bored|activity|things to do|adventure|museum|culture|nightlife|shopping/)) {
    const planned = new Set(trip.activities.map((a) => a.placeId));
    const cat = has(/adventure|hike|outdoor/) ? 'adventure' : has(/museum|culture|temple|shrine/) ? 'culture' : has(/night|bar|karaoke/) ? 'nightlife' : has(/shop/) ? 'shopping' : undefined;
    const ideas = placesFor(trip.destinationId)
      .concat(trip.extraDestinationIds.flatMap((id) => placesFor(id)))
      .filter((p) => !planned.has(p.id) && (!cat || p.category === cat))
      .sort((a, b) => b.rating - a.rating)
      .slice(0, 3);
    if (!ideas.length) return 'You’ve already planned the top spots! Try the Recommendations survey for new destinations.';
    return `Ideas not on your itinerary yet:\n${ideas.map((p) => `• ${p.name} — ${p.category}, ${durationLabel(p.durationMin)}, ${p.cost ? `${money(p.cost)}/person` : 'free'} (★${p.rating})`).join('\n')}\nAdd one from the Nearby screen or put it to a vote.`;
  }

  if (has(/^(hi|hey|hello|yo|sup|good (morning|evening|afternoon))\b/)) {
    return `Hey! 👋 I’m your Trip Assistant for ${trip.name}. Ask me about the budget, weather, food nearby, flights, what’s next, or “convert 50 USD to ${dest?.currency ?? 'EUR'}”.`;
  }

  return `I can help with: budget & who owes who, weather, food nearby, today’s/tomorrow’s plan, flights & arrivals, currency (“convert 20 USD”), time zones, phrases, packing and emergencies. Try one of the quick questions below!`;
}

function systemPrompt(ctx: AssistantContext): string {
  const { trip } = ctx;
  const dest = destinationById(trip.destinationId);
  const tz = dest?.tz ?? 'UTC';
  const name = (id: string) => trip.members.find((m) => m.id === id)?.name ?? id;
  const acts = trip.activities
    .slice()
    .sort((a, b) => (a.date + a.time).localeCompare(b.date + b.time))
    .map((a) => `${a.date} ${a.time} | ${a.title} | ${a.location} | ${a.status} | $${a.costPerPerson}/pp | ${durationLabel(a.durationMin)}`)
    .join('\n');
  const flights = trip.flights.map((f) => `${name(f.memberId)} ${f.direction} ${f.flightNo} ${f.from}->${f.to} ${formatInTz(f.direction === 'arrival' ? f.arriveUtc : f.departUtc, tz)}`).join('\n');
  const w = ctx.weather;
  return [
    'You are the Trip Assistant inside TogetherWeGo, a group trip-planning app, replying in the group chat.',
    'Be friendly and concise: 1–4 short sentences or a short bullet list. Use the trip data below; do not invent bookings.',
    'You cannot edit the trip yourself — when someone wants a change, suggest it and point them to the right screen (Itinerary → Add Activity, Budget → Add expense, Profile → polls/checklist).',
    '',
    `Trip: ${trip.name} — ${dest?.city}, ${dest?.country}; ${trip.startDate} to ${trip.endDate}; local time zone ${tz}; today (local)
```

```
${todayInTz(tz)}.`,
    `Members: ${trip.members.map((m) => `${m.name} (${m.role})`).join(', ')}.`,
    `Budget: spent $${Math.round(totalSpent(trip))} of $${trip.budget} cap. Expenses: ${trip.expenses.map((e) => `${e.title} $${e.amoun
t} (${e.category}, paid by ${name(e.paidBy)})`).join('; ')}.`,
    w ? `Weather now: ${w.current.temp}°C ${weatherInfo(w.current.code).label}; next days: ${w.daily.slice(0, 4).map((d) => `${d.date}
${d.min}-${d.max}°C rain ${d.rain}%`).join(', ')}.` : '',
    `Itinerary (local times):\n${acts || 'none yet'}`,
    flights ? `Flights:\n${flights}` : '',
    `Open tasks: ${trip.tasks.filter((k) => !k.done).map((k) => k.text).join(', ') || 'none'}.`,
  ]
    .filter(Boolean)
    .join('\n');
}

export async function askClaude(apiKey: string, question: string, history: ChatMessage[], ctx: AssistantContext): Promise<string> {
  const client = new Anthropic({ apiKey, dangerouslyAllowBrowser: true, maxRetries: 1, timeout: 45_000 });
  const recent = history.filter((m) => m.kind !== 'system').slice(-12);
  const messages: Anthropic.Beta.BetaMessageParam[] = [];
  for (const m of recent) {
    if (m.ai) messages.push({ role: 'assistant', content: m.text });
    else messages.push({ role: 'user', content: `${ctx.trip.members.find((x) => x.id === m.authorId)?.name ?? 'Member'}: ${m.text}` });
  }
  while (messages.length && messages[0].role !== 'user') messages.shift();
  messages.push({ role: 'user', content: question });

  const resp = await client.beta.messages.create({
    model: CLAUDE_MODEL,
    max_tokens: 2048,
    betas: ['server-side-fallback-2026-07-01'],
    fallbacks: 'default',
    output_config: { effort: 'low' },
    system: systemPrompt(ctx),
    messages,
  });
  if (resp.stop_reason === 'refusal') throw new Error('refused');
  const text = resp.content
    .map((b) => (b.type === 'text' ? b.text : ''))
    .join('\n')
    .trim();
  if (!text) throw new Error('empty');
  return text;
}
```

src/services/currency.ts

```
/**
 * Exchange rates from open.er-api.com (free, no key), cached for 6 hours and
 * persisted for offline use. Falls back to bundled reference rates.
 */
import { currencies } from '@/data/reference';
import { useAppStore } from '@/store/useAppStore';
import type { RatesSnapshot } from '@/store/types';

const CACHE_MS = 6 * 60 * 60 * 1000;

export function fallbackRates(): RatesSnapshot {
  const rates: Record<string, number> = {};
  currencies.forEach((c) => (rates[c.code] = c.perUsd));
  return { base: 'USD', rates, fetchedAt: 0, live: false };
}

export async function fetchRates(force = false): Promise<RatesSnapshot> {
  const cached = useAppStore.getState().rates;
  if (!force && cached?.live && Date.now() - cached.fetchedAt < CACHE_MS) return cached;
  try {
    const ctrl = new AbortController();
    const timer = setTimeout(() => ctrl.abort(), 8000);
    const res = await fetch('https://open.er-api.com/v6/latest/USD', { signal: ctrl.signal });
    clearTimeout(timer);
    const j = await res.json();
    if (j.result !== 'success' || !j.rates) throw new Error('bad response');
    const snap: RatesSnapshot = { base: 'USD', rates: j.rates, fetchedAt: Date.now(), live: true };
    useAppStore.getState().setRates(snap);
    return snap;
  } catch {
    return cached ?? fallbackRates();
  }
}

export function convert(amount: number, from: string, to: string, rates: RatesSnapshot | null): number {
  const r = rates?.rates ?? fallbackRates().rates;
  const f = r[from] ?? fallbackRates().rates[from] ?? 1;
  const t = r[to] ?? fallbackRates().rates[to] ?? 1;
  return (amount / f) * t;
}

export function useRates(): RatesSnapshot {
  return useAppStore((s) => s.rates) ?? fallbackRates();
}
```

```
/**
 * Offline document vault. Picked files are copied into the app's private
 * document directory (native) so they stay readable with no connection.
 * On web, small files are kept as data URLs in local storage.
 */
import * as DocumentPicker from 'expo-document-picker';
import { Directory, File, Paths } from 'expo-file-system';
import * as ImagePicker from 'expo-image-picker';
import { Platform } from 'react-native';

export const MAX_WEB_BYTES = 2 * 1024 * 1024;
export const MAX_NATIVE_BYTES = 25 * 1024 * 1024;

export interface SavedFile {
  uri: string;
  name: string;
  size?: number;
  kind: 'pdf' | 'image' | 'file';
}

function kindOf(name: string, mime?: string | null): SavedFile['kind'] {
  if (mime?.startsWith('image/') || /\.(png|jpe?g|heic|webp|gif)$/i.test(name)) return 'image';
  if (mime === 'application/pdf' || /\.pdf$/i.test(name)) return 'pdf';
  return 'file';
}

async function toDataUrl(file: Blob): Promise<string> {
  return new Promise((resolve, reject) => {
    const r = new FileReader();
    r.onload = () => resolve(String(r.result));
    r.onerror = () => reject(r.error);
    r.readAsDataURL(file);
  });
}

async function persist(uri: string, name: string, webFile?: Blob | null): Promise<string> {
  if (Platform.OS === 'web') {
    if (webFile) return toDataUrl(webFile);
    return uri;
  }
  const dir = new Directory(Paths.document, 'vault');
  if (!dir.exists) dir.create({ intermediates: true, idempotent: true });
  const safe = `${Date.now()}-${name.replace(/[^\w.-]+/g, '_')}`;
  const dest = new File(dir, safe);
  await new File(uri).copy(dest);
  return dest.uri;
}

export class FileTooLargeError extends Error {}

/** Lets the user pick a PDF/image/any file and saves an offline copy. */
export async function pickDocument(): Promise<SavedFile | null> {
  const res = await DocumentPicker.getDocumentAsync({ copyToCacheDirectory: true, multiple: false, type: ['application/pdf', 'image/*',
'*/*'] });
  if (res.canceled || !res.assets?.[0]) return null;
  const a = res.assets[0];
  const limit = Platform.OS === 'web' ? MAX_WEB_BYTES : MAX_NATIVE_BYTES;
  if (a.size && a.size > limit) throw new FileTooLargeError(`Files must be under ${Math.round(limit / 1024 / 1024)} MB`);
  const uri = await persist(a.uri, a.name, (a as { file?: Blob }).file ?? null);
  return { uri, name: a.name, size: a.size, kind: kindOf(a.name, a.mimeType) };
}

/** Picks a photo from the library (or camera) and keeps an offline copy. */
export async function pickImage(fromCamera = false): Promise<SavedFile | null> {
  if (fromCamera) {
    const perm = await ImagePicker.requestCameraPermissionsAsync();
    if (!perm.granted) throw new Error('Camera permission is needed to take photos.');
  }
  const opts: ImagePicker.ImagePickerOptions = { mediaTypes: ['images'], quality: 0.7, allowsEditing: false };
  const res = fromCamera ? await ImagePicker.launchCameraAsync(opts) : await ImagePicker.launchImageLibraryAsync(opts);
  if (res.canceled || !res.assets?.[0]) return null;
  const a = res.assets[0];
  const name = a.fileName ?? `photo-${Date.now()}.jpg`;
  if (Platform.OS === 'web') {
    if (a.fileSize && a.fileSize > MAX_WEB_BYTES) throw new FileTooLargeError('Photos must be under 2 MB in the web preview');
    return { uri: a.uri, name, size: a.fileSize, kind: 'image' };
  }
  const uri = await persist(a.uri, name);
  return { uri, name, size: a.fileSize, kind: 'image' };
}

export function deleteLocalFile(uri?: string) {
  if (!uri || Platform.OS === 'web' || !uri.startsWith('file:')) return;
  try {
```

```
    const f = new File(uri);
    if (f.exists) f.delete();
  } catch {
    // already gone
  }
}

export function sizeLabel(bytes?: number) {
  if (!bytes) return '';
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${Math.round(bytes / 1024)} KB`;
  return `${(bytes / 1024 / 1024).toFixed(1)} MB`;
}
```

src/services/location.ts

```
/** Device location with graceful fallbacks (permission denied, web, timeouts). */
import * as Location from 'expo-location';

import type { LatLng } from '@/utils/geo';

export interface LocationResult extends LatLng {
  source: 'device' | 'fallback';
}

export async function getLocation(fallback: LatLng): Promise<LocationResult> {
  try {
    const perm = await Location.requestForegroundPermissionsAsync();
    if (perm.status !== 'granted') return { ...fallback, source: 'fallback' };
    const last = await Location.getLastKnownPositionAsync().catch(() => null);
    const pos =
      last ??
      (await Promise.race([
        Location.getCurrentPositionAsync({ accuracy: Location.Accuracy.Balanced }),
        new Promise<null>((r) => setTimeout(() => r(null), 8000)),
      ]));
    if (!pos) return { ...fallback, source: 'fallback' };
    return { lat: pos.coords.latitude, lng: pos.coords.longitude, source: 'device' };
  } catch {
    return { ...fallback, source: 'fallback' };
  }
}
```

src/services/notifications.ts

```
/**
 * Live activity reminders.
 * - iOS / Android: real local notifications scheduled with expo-notifications
 *   (they fire even when the app is closed).
 * - Every platform: an in-app watcher adds due reminders to the notification
 *   centre and, on web, shows a toast + browser notification.
 */
import * as Notifications from 'expo-notifications';
import { router } from 'expo-router';
import { Platform } from 'react-native';

import { toast } from '@/components/ui/feedback';
import { destinationById } from '@/data/destinations';
import { tripStatus, useAppStore } from '@/store/useAppStore';
import type { Activity, Trip } from '@/store/types';
import { time12, zonedToUtc } from '@/utils/time';

const isNative = Platform.OS !== 'web';
let initialised = false;

export function initNotifications() {
  if (initialised || !isNative) return;
  initialised = true;
  Notifications.setNotificationHandler({
    handleNotification: async () => ({ shouldShowBanner: true, shouldShowList: true, shouldPlaySound: true, shouldSetBadge: false }),
  });
  if (Platform.OS === 'android') {
    Notifications.setNotificationChannelAsync('reminders', {
      name: 'Activity reminders',
      importance: Notifications.AndroidImportance.HIGH,
      vibrationPattern: [0, 200, 120, 200],
      lightColor: '#FBC91B',
    }).catch(() => {});
  }
  Notifications.addNotificationResponseReceivedListener((resp) => {
    const route = resp.notification.request.content.data?.route;
    if (typeof route === 'string') setTimeout(() => router.push(route as never), 300);
  });
}

export async function ensurePermission(): Promise<boolean> {
  if (!isNative) {
    // Browsers may leave the prompt unanswered forever, so never block on it:
    // ask in the background and rely on in-app toasts until it's granted.
    if (typeof window !== 'undefined' && 'Notification' in window) {
      if (Notification.permission === 'granted') return true;
      if (Notification.permission === 'default') Notification.requestPermission().catch(() => {});
    }
    return false;
  }
  try {
    const cur = await Notifications.getPermissionsAsync();
    if (cur.granted) return true;
    const req = await Notifications.requestPermissionsAsync();
    return req.granted;
  } catch {
    return false;
  }
}

export function activityStartUtc(trip: Trip, a: Activity): number {
  const tz = destinationById(trip.destinationId)?.tz ?? 'UTC';
  return zonedToUtc(a.date, a.time, tz);
}

export function reminderBody(trip: Trip, a: Activity, lead: number) {
  return {
    title: `⏰ ${a.title} in ${lead} min`,
    body: `${time12(a.time)} · ${a.location} — ${trip.name}`,
  };
}

/** Reschedules every reminder for the signed-in user's upcoming activities. */
export async function syncReminders(): Promise<number> {
  const s = useAppStore.getState();
  if (!isNative) return 0;
  try {
    await Notifications.cancelAllScheduledNotificationsAsync();
  } catch {
    return 0;
  }
  if (!s.settings.notificationsEnabled || !s.session) return 0;
  const granted = (await Notifications.getPermissionsAsync().catch(() => null))?.granted;
  if (!granted) return 0;
```

```
  const lead = s.settings.reminderLeadMin;
  const now = Date.now();
  const due: { at: number; trip: Trip; a: Activity }[] = [];
  for (const trip of s.trips) {
    if (!trip.members.some((m) => m.id === s.session?.userId) || tripStatus(trip) === 'completed') continue;
    for (const a of trip.activities) {
      if (!a.reminder) continue;
      const at = activityStartUtc(trip, a) - lead * 60000;
      if (at > now + 5000 && at < now + 45 * 86400000) due.push({ at, trip, a });
    }
  }
  due.sort((x, y) => x.at - y.at);
  for (const { at, trip, a } of due.slice(0, 48)) {
    const { title, body } = reminderBody(trip, a, lead);
    await Notifications.scheduleNotificationAsync({
      content: { title, body, data: { route: '/(tabs)/itinerary', tripId: trip.id, activityId: a.id } },
      trigger: { type: Notifications.SchedulableTriggerInputTypes.DATE, date: new Date(at), channelId: 'reminders' },
    }).catch(() => {});
  }
  return Math.min(due.length, 48);
}

function browserNotify(title: string, body: string) {
  try {
    if (!isNative && typeof window !== 'undefined' && 'Notification' in window && Notification.permission === 'granted') {
      new Notification(title, { body, icon: '/favicon.ico' });
    }
  } catch {
    // ignore
  }
}

/** Sends a sample reminder in a few seconds so the feature can be demoed live. */
export async function sendTestReminder(seconds = 5) {
  const s = useAppStore.getState();
  const trip = s.trips.find((x) => x.id === s.activeTripId);
  const a = trip?.activities.find((x) => x.reminder) ?? trip?.activities[0];
  const title = a ? `⏰ ${a.title} starts soon` : '⏰ Test reminder';
  const body = a && trip ? `${time12(a.time)} · ${a.location} — ${trip.name}` : 'Reminders are working!';
  await ensurePermission();
  if (isNative) {
    await Notifications.scheduleNotificationAsync({
      content: { title, body, data: { route: '/(tabs)/itinerary' } },
      trigger: { type: Notifications.SchedulableTriggerInputTypes.TIME_INTERVAL, seconds, channelId: 'reminders' },
    }).catch(() => {});
  }
  setTimeout(() => {
    useAppStore.getState().pushNotification({ kind: 'reminder', title, body, tripId: trip?.id, route: '/(tabs)/itinerary' });
    if (!isNative) {
      toast(`${title} — ${body}`, { icon: 'alarm-outline', tone: 'info' });
      browserNotify(title, body);
    }
  }, seconds * 1000);
}

/** Called on an interval while the app is open: surfaces reminders that just came due. */
export function checkDueReminders() {
  const s = useAppStore.getState();
  if (!s.session || !s.settings.notificationsEnabled) return;
  const lead = s.settings.reminderLeadMin;
  const now = Date.now();
  for (const trip of s.trips) {
    if (!trip.members.some((m) => m.id === s.session?.userId)) continue;
    for (const a of trip.activities) {
      if (!a.reminder) continue;
      const start = activityStartUtc(trip, a);
      const fireAt = start - lead * 60000;
      const key = `${a.id}@${a.date}T${a.time}-${lead}`;
      if (now >= fireAt && now < start && !s.firedReminders[key]) {
        s.markReminderFired(key);
        const { title, body } = reminderBody(trip, a, Math.max(1, Math.round((start - now) / 60000)));
        s.pushNotification({ kind: 'reminder', title, body, tripId: trip.id, route: '/(tabs)/itinerary' });
        if (!isNative) {
          toast(`${title}`, { icon: 'alarm-outline', tone: 'info' });
          browserNotify(title, body);
        }
      }
    }
  }
}

/** The next upcoming reminder-enabled activity across the active trip (for the "live" banner). */
export function nextUpcoming(trip: Trip | undefined): { a: Activity; startUtc: number } | null {
  if (!trip) return null;
  const now = Date.now();
  let best: { a: Activity; startUtc: number } | null = null;
```

```
  for (const a of trip.activities) {
    const st = activityStartUtc(trip, a);
    if (st > now - 30 * 60000 && (!best || st < best.startUtc)) best = { a, startUtc: st };
  }
  return best;
}
```

src/services/print.ts

```
/** Printable trip planner: builds a styled HTML document and prints / saves it as PDF. */
import * as Print from 'expo-print';
import * as Sharing from 'expo-sharing';
import { Platform } from 'react-native';

import { destinationById } from '@/data/destinations';
import { emergencyByCountry } from '@/data/reference';
import { totalSpent } from '@/store/useAppStore';
import type { Trip } from '@/store/types';
import { durationLabel, money } from '@/utils/format';
import { dayLabel, formatInTz, rangeLabel, time12, tripDays, tzAbbrev, weekdayShort } from '@/utils/time';

const esc = (s: string) => s.replace(/[&<>"']/g, (c) => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' })[c]!);

export function plannerHtml(trip: Trip): string {
  const dest = destinationById(trip.destinationId);
  const tz = dest?.tz ?? 'UTC';
  const name = (id: string) => trip.members.find((m) => m.id === id)?.name ?? 'Member';
  const days = tripDays(trip.startDate, trip.endDate)
    .map((d, i) => {
      const acts = trip.activities.filter((a) => a.date === d).sort((a, b) => a.time.localeCompare(b.time));
      const rows = acts.length
        ? acts
            .map(
              (a) => `<tr><td class="time">${time12(a.time)}</td><td><b>${esc(a.title)}</b><div class="muted">${esc(a.location)} · ${dur
ationLabel(a.durationMin)}${a.costPerPerson ? ` · ${money(a.costPerPerson)}/person` : ''}</div>${a.notes ? `<div class="note">${esc(a.no
tes)}</div>` : ''}</td><td class="who">${a.assigned.map((x) => esc(name(x).split(' ')[0])).join(', ')}</td><td class="box">☐</td></tr>`,
            )
            .join('')
        : '<tr><td colspan="4" class="muted">Free day — explore at your own pace ✨</td></tr>';
      return `<section class="day"><h3>Day ${i + 1} · ${weekdayShort(d)} ${dayLabel(d)}</h3><table>${rows}</table></section>`;
    })
    .join('');
  const flights = trip.flights
    .slice()
    .sort((a, b) => a.arriveUtc - b.arriveUtc)
    .map((f) => `<tr><td>${esc(name(f.memberId))}</td><td>${esc(f.flightNo)} · ${f.from}→${f.to}</td><td>${f.direction === 'arrival' ?
'Arrives' : 'Departs'} ${formatInTz(f.direction === 'arrival' ? f.arriveUtc : f.departUtc, tz)}</td></tr>`)
    .join('');
  const em = emergencyByCountry[dest?.countryCode ?? ''] ?? emergencyByCountry.DEFAULT;
  const spent = totalSpent(trip);
  return `<!doctype html><html><head><meta charset="utf-8"><title>${esc(trip.name)} — Planner</title>
<style>
  *{box-sizing:border-box} body{font-family:-apple-system,Segoe UI,Roboto,Helvetica,Arial,sans-serif;color:#1A1C17;margin:28px;font-siz
e:12.5px}
  header{display:flex;justify-content:space-between;align-items:flex-end;border-bottom:3px solid #1F4D25;padding-bottom:10px;margin-bott
om:16px}
  h1{margin:0;font-size:24px;color:#1F4D25} h2{font-size:15px;margin:18px 0 6px;color:#1F4D25} h3{font-size:13px;margin:14px 0 6px;backg
round:#EAF5E4;padding:6px 10px;border-radius:6px}
  table{width:100%;border-collapse:collapse} td{padding:6px 8px;border-bottom:1px solid #E7E9DF;vertical-align:top}
  .time{width:70px;font-weight:600;white-space:nowrap} .who{width:110px;color:#5F645A} .box{width:20px;font-size:15px}
  .muted{color:#7A7F73;font-size:11.5px} .note{margin-top:3px;font-style:italic;color:#5F645A}
  .grid{display:grid;grid-template-columns:repeat(3,1fr);gap:8px} .stat{background:#F4F4EE;border-radius:8px;padding:8px 10px}
  .stat b{display:block;font-size:16px;color:#1F4D25} .day{page-break-inside:avoid} footer{margin-top:18px;color:#959A8E;font-size:10.5p
x;text-align:center}
</style></head><body>
<header><div><h1>${esc(trip.name)}</h1><div class="muted">${esc(dest ? `${dest.city}, ${dest.country}` : '')} · ${rangeLabel(trip.startD
ate, trip.endDate)} · times in ${tzAbbrev(tz)}</div></div><div class="muted">Invite code <b>${trip.inviteCode}</b></div></header>
<div class="grid">
  <div class="stat">Travellers<b>${trip.members.length}</b>${trip.members.map((m) => esc(m.name.split(' ')[0]) + ' (' + esc(m.role) +
')').join(', ')}</div>
  <div class="stat">Budget<b>${money(spent)} / ${money(trip.budget)}</b>${money(trip.budget / Math.max(1, trip.members.length))} per per
son</div>
  <div class="stat">Emergency<b>Police ${em.police} · Ambulance ${em.ambulance}</b>${esc(em.tips[0] ?? '')}</div>
</div>
<h2>Day-by-day itinerary</h2>${days}
${flights ? `<h2>Flights & arrivals</h2><table>${flights}</table>` : ''}
<h2>Checklist</h2><table>${trip.tasks.map((k) => `<tr><td class="box">${k.done ? '☑' : '☐'}</td><td>${esc(k.text)}</td><td class="who">
${k.assignee ? esc(name(k.assignee).split(' ')[0]) : 'Everyone'}</td></tr>`).join('')}</table>
<footer>Printed from TogetherWeGo · Plan together, travel better</footer>
</body></html>`;
}

export async function printPlanner(trip: Trip) {
  const html = plannerHtml(trip);
  if (Platform.OS === 'web') {
    const w = window.open('', '_blank');
    if (!w) throw new Error('Pop-up blocked');
    w.document.write(html);
    w.document.close();
    w.focus();
    setTimeout(() => w.print(), 350);
    return;
  }
```

```
  await Print.printAsync({ html });
}

export async function sharePlannerPdf(trip: Trip) {
  if (Platform.OS === 'web') return printPlanner(trip);
  const { uri } = await Print.printToFileAsync({ html: plannerHtml(trip) });
  if (await Sharing.isAvailableAsync()) {
    await Sharing.shareAsync(uri, { mimeType: 'application/pdf', dialogTitle: `${trip.name} planner`, UTI: 'com.adobe.pdf' });
  }
}
```

src/services/secrets.ts

```
/**
 * Sensitive values (the optional Anthropic API key) live in the OS keychain /
 * keystore via expo-secure-store — never in the regular app database.
 * Web has no secure enclave, so the preview keeps it in session storage only.
 */
import * as SecureStore from 'expo-secure-store';
import { Platform } from 'react-native';

const KEY = 'twg_anthropic_key';

export async function getApiKey(): Promise<string | null> {
  try {
    if (Platform.OS === 'web') return typeof sessionStorage !== 'undefined' ? sessionStorage.getItem(KEY) : null;
    return await SecureStore.getItemAsync(KEY);
  } catch {
    return null;
  }
}

export async function setApiKey(value: string | null): Promise<void> {
  if (Platform.OS === 'web') {
    if (typeof sessionStorage === 'undefined') return;
    if (value) sessionStorage.setItem(KEY, value);
    else sessionStorage.removeItem(KEY);
    return;
  }
  if (value) await SecureStore.setItemAsync(KEY, value);
  else await SecureStore.deleteItemAsync(KEY);
}

export const looksLikeApiKey = (v: string) => /^sk-ant-[A-Za-z0-9_-]{20,}$/.test(v.trim());
```

src/services/share.ts

```
/**
 * Social media integration: native share sheet plus direct deep links into
 * WhatsApp, X, Facebook, Messenger, Telegram, Instagram, SMS and email.
 */
import { Asset } from 'expo-asset';
import * as Clipboard from 'expo-clipboard';
import * as Linking from 'expo-linking';
import * as Sharing from 'expo-sharing';
import { Platform, Share } from 'react-native';

import { toast } from '@/components/ui/feedback';
import type { IconName } from '@/components/ui/bits';
import { photos, type PhotoKey } from '@/data/images';

export type SocialTarget = 'whatsapp' | 'x' | 'facebook' | 'messenger' | 'telegram' | 'instagram' | 'sms' | 'email';

export const socialTargets: { key: SocialTarget; label: string; icon: IconName; color: string }[] = [
  { key: 'whatsapp', label: 'WhatsApp', icon: 'logo-whatsapp', color: '#25D366' },
  { key: 'instagram', label: 'Instagram', icon: 'logo-instagram', color: '#E1306C' },
  { key: 'x', label: 'X', icon: 'logo-twitter', color: '#111111' },
  { key: 'facebook', label: 'Facebook', icon: 'logo-facebook', color: '#1877F2' },
  { key: 'messenger', label: 'Messenger', icon: 'chatbubble-ellipses-outline', color: '#0084FF' },
  { key: 'telegram', label: 'Telegram', icon: 'paper-plane', color: '#229ED9' },
  { key: 'sms', label: 'Messages', icon: 'chatbox-ellipses-outline', color: '#34C759' },
  { key: 'email', label: 'Email', icon: 'mail-outline', color: '#5F645A' },
];

export function inviteLink(code: string): string {
  return Linking.createURL('/join', { queryParams: { code } });
}

export function inviteMessage(tripName: string, code: string): string {
  return `Join my trip “${tripName}” on TogetherWeGo ✈️\nInvite code: ${code}\nOpen: ${inviteLink(code)}`;
}

export async function copy(text: string, label = 'Copied to clipboard') {
  await Clipboard.setStringAsync(text);
  toast(label, { icon: 'copy-outline' });
}

/** Opens the OS share sheet (Instagram, Snapchat, iMessage, Slack… whatever is installed). */
export async function shareText(message: string, title = 'TogetherWeGo') {
  try {
    if (Platform.OS === 'web') {
      const nav = typeof navigator !== 'undefined' ? (navigator as Navigator & { share?: (d: object) => Promise<void> }) : undefined;
      if (nav?.share) {
        await nav.share({ title, text: message });
        return;
      }
      await copy(message, 'Share text copied — paste it anywhere');
      return;
    }
    await Share.share({ message, title });
  } catch {
    // user cancelled
  }
}

async function openFirst(urls: string[]) {
  for (const url of urls) {
    try {
      if (Platform.OS === 'web') {
        if (url.startsWith('http') || url.startsWith('mailto') || url.startsWith('sms')) {
          window.open(url, '_blank');
          return true;
        }
        continue;
      }
      if (url.startsWith('http') || (await Linking.canOpenURL(url))) {
        await Linking.openURL(url);
        return true;
      }
    } catch {
      // try next
    }
  }
  return false;
}

/** Shares straight into a specific social app, falling back to its web intent. */
export async function shareTo(target: SocialTarget, text: string, url?: string) {
  const t = encodeURIComponent(text);
  const u = encodeURIComponent(url ?? '');
  const map: Record<SocialTarget, string[]> = {
```

```
    whatsapp: [`whatsapp://send?text=${t}`, `https://wa.me/?text=${t}`],
    x: [`twitter://post?message=${t}`, `https://x.com/intent/tweet?text=${t}`],
    facebook: [`https://www.facebook.com/sharer/sharer.php?u=${u || encodeURIComponent('https://expo.dev')}&quote=${t}`],
    messenger: [`fb-messenger://share?link=${u}`, `https://www.messenger.com/`],
    telegram: [`tg://msg?text=${t}`, `https://t.me/share/url?url=${u}&text=${t}`],
    instagram: ['instagram://story-camera', 'instagram://app', 'https://www.instagram.com/'],
    sms: [Platform.OS === 'ios' ? `sms:&body=${t}` : `sms:?body=${t}`],
    email: [`mailto:?subject=${encodeURIComponent('Join our trip on TogetherWeGo')}&body=${t}`],
  };
  if (target === 'instagram') {
    // Instagram has no text-share URL: copy the caption first, then open the app.
    await Clipboard.setStringAsync(text);
    toast('Caption copied — paste it in Instagram', { icon: 'logo-instagram' });
  }
  const ok = await openFirst(map[target]);
  if (!ok) await shareText(text);
}

/** Shares a photo file (bundled demo photo or one from the camera roll). */
export async function sharePhoto(uri: string, caption: string) {
  try {
    let fileUri = uri;
    if (uri in photos) {
      const asset = Asset.fromModule(photos[uri as PhotoKey]);
      await asset.downloadAsync();
      fileUri = asset.localUri ?? asset.uri;
    }
    if (Platform.OS !== 'web' && (await Sharing.isAvailableAsync())) {
      await Sharing.shareAsync(fileUri, { dialogTitle: caption, mimeType: 'image/jpeg' });
      return;
    }
    await shareText(`${caption} 📸 — shared from TogetherWeGo`);
  } catch {
    await shareText(`${caption} 📸 — shared from TogetherWeGo`);
  }
}
```

src/services/weather.ts

```
/**
 * Weather via Open-Meteo (free, no API key). Results are cached in the store
 * so the last forecast stays visible offline; if the network is down on first
 * load we fall back to a typical-climate estimate and label it as such.
 */
import { destinationById, type Destination } from '@/data/destinations';
import type { IconName } from '@/components/ui/bits';
import { useAppStore } from '@/store/useAppStore';
import type { WeatherSnapshot } from '@/store/types';
import { addDays, todayInTz } from '@/utils/time';

const CACHE_MS = 30 * 60 * 1000;

export function weatherInfo(code: number): { label: string; icon: IconName } {
  if (code === 0) return { label: 'Clear sky', icon: 'sunny' };
  if (code <= 2) return { label: 'Partly cloudy', icon: 'partly-sunny' };
  if (code === 3) return { label: 'Overcast', icon: 'cloud-outline' };
  if (code <= 48) return { label: 'Foggy', icon: 'cloud-outline' };
  if (code <= 57) return { label: 'Drizzle', icon: 'rainy-outline' };
  if (code <= 67) return { label: 'Rain', icon: 'rainy-outline' };
  if (code <= 77) return { label: 'Snow', icon: 'snow-outline' };
  if (code <= 82) return { label: 'Rain showers', icon: 'rainy-outline' };
  if (code <= 86) return { label: 'Snow showers', icon: 'snow-outline' };
  return { label: 'Thunderstorms', icon: 'thunderstorm-outline' };
}

function estimate(d: Destination): WeatherSnapshot {
  // Rough climate model: warmer near the equator, seasonal swing by hemisphere.
  const month = new Date().getUTCMonth();
  const north = d.lat >= 0;
  const season = Math.cos(((month - (north ? 6.5 : 0.5)) / 12) * 2 * Math.PI);
  const base = 27 - Math.abs(d.lat) * 0.42 + season * Math.min(12, Math.abs(d.lat) * 0.3);
  const today = todayInTz(d.tz);
  const codes = [1, 2, 3, 61, 2, 0, 80];
  return {
    destinationId: d.id,
    fetchedAt: Date.now(),
    live: false,
    current: { temp: Math.round(base), code: 2, wind: 11, humidity: 64 },
    daily: codes.map((code, i) => ({
      date: addDays(today, i),
      max: Math.round(base + 3 + ((i * 7) % 3)),
      min: Math.round(base - 5 + ((i * 5) % 3)),
      code,
      rain: code >= 61 ? 60 : code === 3 ? 25 : 10,
    })),
  };
}

export async function fetchWeather(destinationId: string, force = false): Promise<WeatherSnapshot | undefined> {
  const d = destinationById(destinationId);
  if (!d) return undefined;
  const cached = useAppStore.getState().weather[destinationId];
  if (!force && cached?.live && Date.now() - cached.fetchedAt < CACHE_MS) return cached;
  try {
    const url =
      `https://api.open-meteo.com/v1/forecast?latitude=${d.lat}&longitude=${d.lng}` +
      '&current=temperature_2m,relative_humidity_2m,weather_code,wind_speed_10m' +
      '&daily=weather_code,temperature_2m_max,temperature_2m_min,precipitation_probability_max&timezone=auto&forecast_days=7';
    const ctrl = new AbortController();
    const timer = setTimeout(() => ctrl.abort(), 8000);
    const res = await fetch(url, { signal: ctrl.signal });
    clearTimeout(timer);
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const j = await res.json();
    const snap: WeatherSnapshot = {
      destinationId,
      fetchedAt: Date.now(),
      live: true,
      current: {
        temp: Math.round(j.current.temperature_2m),
        code: j.current.weather_code,
        wind: Math.round(j.current.wind_speed_10m),
        humidity: Math.round(j.current.relative_humidity_2m),
      },
      daily: (j.daily.time as string[]).map((date, i) => ({
        date,
        max: Math.round(j.daily.temperature_2m_max[i]),
        min: Math.round(j.daily.temperature_2m_min[i]),
        code: j.daily.weather_code[i],
        rain: j.daily.precipitation_probability_max?.[i] ?? 0,
      })),
    };
    useAppStore.getState().setWeather(snap);
```

```
    return snap;
  } catch {
    if (cached) return cached;
    const est = estimate(d);
    useAppStore.getState().setWeather(est);
    return est;
  }
}

export function weatherSummary(w: WeatherSnapshot | undefined): string {
  if (!w) return 'Loading forecast…';
  const today = w.daily[0];
  const later = today && today.rain >= 50 ? ' · rain likely later' : today && today.rain >= 30 ? ' · chance of showers' : '';
  return `${weatherInfo(w.current.code).label.toLowerCase()}${later}`;
}

export const cToF = (c: number) => Math.round((c * 9) / 5 + 32);
```

```
/**
 * Demo content. Dates are generated relative to "today" so the sample trip is
 * always fresh — whether the app is opened today or at the competition months
 * later. Settings → Demo timeline can re-seed it as upcoming, in-progress or
 * completed to show the whole trip lifecycle.
 */
import { packingBase } from '@/data/reference';
import { uid } from '@/utils/format';
import { addDays, todayInTz, zonedToUtc } from '@/utils/time';

import type { Activity, Expense, Flight, Member, PackingItem, Photo, Trip } from './types';

export const DEMO_EMAIL = 'maya.chen@togetherwego.app';
export const DEMO_PASSWORD = 'TravelTogether2026!';
export const DEMO_USER_ID = 'u-maya';

export type DemoTimeline = 'upcoming' | 'active' | 'completed';

const TOKYO_TZ = 'Asia/Tokyo';
const AMALFI_TZ = 'Europe/Rome';

/** The 6 members of the Tokyo & Kyoto trip. */
export const demoMembers: Member[] = [
  { id: DEMO_USER_ID, name: 'Maya Chen', email: DEMO_EMAIL, avatar: 'avatar-maya', role: 'Navigator', homeCity: 'New York' },
  { id: 'm-diego', name: 'Diego Alvarez', avatar: 'avatar-diego', role: 'Food Lead', homeCity: 'New York' },
  { id: 'm-priya', name: 'Priya Patel', avatar: 'avatar-priya', role: 'Treasurer', homeCity: 'San Francisco' },
  { id: 'm-sam', name: 'Sam Okafor', avatar: 'avatar-sam', role: 'Photographer', homeCity: 'New York' },
  { id: 'm-ana', name: 'Ana Silva', avatar: 'avatar-ana', role: 'Planner', homeCity: 'Boston' },
  { id: 'm-noah', name: 'Noah Kim', avatar: 'avatar-noah', role: 'Activities Lead', homeCity: 'Chicago' },
];

const ALL = demoMembers.map((m) => m.id);
const votesFrom = (ids: string[], down: string[] = []): Record<string, 1 | -1> => {
  const v: Record<string, 1 | -1> = {};
  ids.forEach((id) => (v[id] = 1));
  down.forEach((id) => (v[id] = -1));
  return v;
};

function act(start: string, day: number, a: Omit<Activity, 'id' | 'date' | 'createdBy' | 'reminder'> & { reminder?: boolean }): Activity
{
  return { id: uid('a-'), date: addDays(start, day), createdBy: DEMO_USER_ID, reminder: a.reminder ?? a.status === 'confirmed', ...a };
}

function demoActivities(start: string): Activity[] {
  return [
    // Day 1
    act(start, 0, { time: '14:30', title: 'Arrive at Haneda Airport (HND)', location: 'Ōta, Tokyo', type: 'transport', durationMin: 150,
costPerPerson: 0, status: 'confirmed', assigned: [DEMO_USER_ID, 'm-sam'], votes: votesFrom(ALL), lat: 35.5494, lng: 139.7798, notes: 'Me
et at the Keikyu Line ticket gates, Terminal 3 arrivals.' }),
    act(start, 0, { time: '17:30', title: 'Check-in — Shinjuku Grand Hotel', location: 'Shinjuku, Tokyo', type: 'stay', durationMin: 30,
costPerPerson: 180, status: 'confirmed', assigned: [DEMO_USER_ID], votes: votesFrom(ALL), lat: 35.6938, lng: 139.7034 }),
    act(start, 0, { time: '19:30', title: 'Welcome Dinner — Omoide Yokocho', location: 'Shinjuku, Tokyo', type: 'food', durationMin: 12
0, costPerPerson: 30, status: 'confirmed', assigned: ['m-diego'], votes: votesFrom(ALL.slice(0, 5)), lat: 35.6929, lng: 139.6996, placeI
d: 'omoide' }),
    // Day 2
    act(start, 1, { time: '08:00', title: 'Tsukiji Outer Market Breakfast', location: 'Tsukiji, Tokyo', type: 'food', durationMin: 90, c
ostPerPerson: 25, status: 'confirmed', assigned: ['m-diego'], votes: votesFrom(ALL.slice(0, 4)), lat: 35.6655, lng: 139.7707, placeId:
'tsukiji' }),
    act(start, 1, { time: '10:30', title: 'Senso-ji Temple & Asakusa', location: 'Asakusa, Tokyo', type: 'culture', durationMin: 120, co
stPerPerson: 0, status: 'voting', assigned: ['m-ana', 'm-noah'], votes: votesFrom(['m-ana', 'm-noah', 'm-priya'], ['m-sam']), lat: 35.71
48, lng: 139.7967, placeId: 'sensoji' }),
    act(start, 1, { time: '14:00', title: 'teamLab Planets Digital Art Museum', location: 'Toyosu, Tokyo', type: 'culture', durationMin:
150, costPerPerson: 30, status: 'pending', assigned: ['m-ana', 'm-priya', 'm-noah'], votes: votesFrom(ALL.slice(0, 5)), lat: 35.6492, ln
g: 139.7898, placeId: 'teamlab', notes: 'Wear shorts — you walk through knee-deep water!' }),
    act(start, 1, { time: '20:00', title: 'Shibuya Crossing & Rooftop Bar', location: 'Shibuya, Tokyo', type: 'nightlife', durationMin:
180, costPerPerson: 40, status: 'confirmed', assigned: ['m-sam'], votes: votesFrom(ALL), lat: 35.6595, lng: 139.7005, placeId: 'shibuya-
crossing' }),
    // Day 3
    act(start, 2, { time: '09:00', title: 'Meiji Jingu & Harajuku stroll', location: 'Harajuku, Tokyo', type: 'culture', durationMin: 15
0, costPerPerson: 0, status: 'confirmed', assigned: ['m-ana'], votes: votesFrom(ALL), lat: 35.6764, lng: 139.6993, placeId: 'meiji' }),
    act(start, 2, { time: '13:30', title: 'Akihabara arcades & retro games', location: 'Akihabara, Tokyo', type: 'shopping', durationMi
n: 120, costPerPerson: 15, status: 'confirmed', assigned: ['m-noah'], votes: votesFrom(ALL.slice(1)), lat: 35.6984, lng: 139.7731, place
Id: 'akihabara' }),
    act(start, 2, { time: '19:00', title: 'Karaoke night in Shibuya', location: 'Shibuya, Tokyo', type: 'nightlife', durationMin: 120, c
ostPerPerson: 20, status: 'voting', assigned: ['m-noah'], votes: votesFrom(['m-noah', 'm-diego', 'm-sam']), lat: 35.66, lng: 139.6985, p
laceId: 'karaoke' }),
    // Day 4
    act(start, 3, { time: '07:30', title: 'Day trip: Mt Fuji & Chureito Pagoda', location: 'Fujiyoshida, Yamanashi', type: 'adventure',
durationMin: 540, costPerPerson: 45, status: 'confirmed', assigned: [DEMO_USER_ID, 'm-sam'], votes: votesFrom(ALL), lat: 35.5016, lng: 1
38.8013, notes: 'Bus from Shinjuku Expressway Bus Terminal, gate B.' }),
    // Day 5
    act(start, 4, { time: '08:30', title: 'Shinkansen to Kyoto', location: 'Tokyo Station', type: 'transport', durationMin: 135, costPer
Person: 0, status: 'confirmed', assigned: [DEMO_USER_ID], votes: votesFrom(ALL), lat: 35.6812, lng: 139.7671, notes: 'Covered by JR Pas
```

```
s. Sit on the right (seat E) for Fuji views.' }),
    act(start, 4, { time: '13:00', title: 'Check-in — Gion ryokan', location: 'Gion, Kyoto', type: 'stay', durationMin: 30, costPerPerso
n: 140, status: 'confirmed', assigned: ['m-priya'], votes: votesFrom(ALL), lat: 35.0037, lng: 135.7788 }),
    act(start, 4, { time: '17:00', title: 'Gion evening walk', location: 'Gion, Kyoto', type: 'culture', durationMin: 90, costPerPerson:
0, status: 'confirmed', assigned: ['m-ana'], votes: votesFrom(ALL.slice(0, 5)), lat: 35.0037, lng: 135.7788, placeId: 'gion' }),
    // Day 6
    act(start, 5, { time: '06:30', title: 'Fushimi Inari sunrise hike', location: 'Fushimi, Kyoto', type: 'culture', durationMin: 150, c
ostPerPerson: 0, status: 'confirmed', assigned: ['m-sam'], votes: votesFrom(ALL.slice(0, 4)), lat: 34.9671, lng: 135.7727, placeId: 'ina
ri' }),
    act(start, 5, { time: '11:00', title: 'Nishiki Market lunch crawl', location: 'Nakagyo, Kyoto', type: 'food', durationMin: 90, costP
erPerson: 20, status: 'confirmed', assigned: ['m-diego'], votes: votesFrom(ALL), lat: 35.005, lng: 135.7649, placeId: 'nishiki' }),
    act(start, 5, { time: '15:00', title: 'Tea ceremony experience', location: 'Higashiyama, Kyoto', type: 'culture', durationMin: 60, c
ostPerPerson: 40, status: 'voting', assigned: ['m-priya'], votes: votesFrom(['m-priya', 'm-ana'], ['m-noah']), lat: 34.9986, lng: 135.78
09, placeId: 'tea' }),
    // Day 7
    act(start, 6, { time: '07:30', title: 'Arashiyama Bamboo Grove', location: 'Arashiyama, Kyoto', type: 'sightseeing', durationMin: 6
0, costPerPerson: 0, status: 'confirmed', assigned: ['m-sam'], votes: votesFrom(ALL), lat: 35.017, lng: 135.6713, placeId: 'bamboo' }),
    act(start, 6, { time: '13:00', title: 'Hozugawa river boat ride', location: 'Kameoka, Kyoto', type: 'adventure', durationMin: 120, c
ostPerPerson: 35, status: 'pending', assigned: ['m-noah'], votes: votesFrom(['m-noah', 'm-diego', 'm-sam', DEMO_USER_ID]), lat: 35.0128,
lng: 135.6744, placeId: 'hozugawa' }),
    // Day 8 left empty on purpose (free day)
    // Day 9
    act(start, 8, { time: '10:00', title: 'Shinkansen back to Tokyo', location: 'Kyoto Station', type: 'transport', durationMin: 135, co
stPerPerson: 0, status: 'confirmed', assigned: [DEMO_USER_ID], votes: votesFrom(ALL), lat: 34.9858, lng: 135.7588 }),
    act(start, 8, { time: '19:00', title: 'Farewell dinner — Gonpachi', location: 'Nishi-Azabu, Tokyo', type: 'food', durationMin: 120,
costPerPerson: 35, status: 'voting', assigned: ['m-diego'], votes: votesFrom(['m-diego', 'm-ana', 'm-priya']), lat: 35.6563, lng: 139.72
4, placeId: 'gonpachi' }),
    // Day 10
    act(start, 9, { time: '06:30', title: 'Head to Haneda for departures', location: 'Haneda Airport, Tokyo', type: 'transport', duratio
nMin: 60, costPerPerson: 6, status: 'confirmed', assigned: [DEMO_USER_ID], votes: votesFrom(ALL), lat: 35.5494, lng: 139.7798 }),
  ];
}

interface Airfield {
  tz: string;
  code: string;
}
const HND: Airfield = { tz: TOKYO_TZ, code: 'HND' };
const NAP: Airfield = { tz: AMALFI_TZ, code: 'NAP' };

function flightPair(at: Airfield, memberId: string, airline: string, outNo: string, inNo: string, from: string, start: string, end: stri
ng, arr: string, dep: string, hoursOut: number, hoursBack: number, arrDayOffset = 0): Flight[] {
  const arriveUtc = zonedToUtc(addDays(start, arrDayOffset), arr, at.tz);
  const departUtc = zonedToUtc(end, dep, at.tz);
  const confirmation = 'TWG' + memberId.slice(-3).toUpperCase();
  return [
    { id: uid('f-'), memberId, airline, flightNo: outNo, from, to: at.code, departUtc: arriveUtc - hoursOut * 3600000, arriveUtc, direct
ion: 'arrival', seat: '32A', confirmation },
    { id: uid('f-'), memberId, airline, flightNo: inNo, from: at.code, to: from, departUtc, arriveUtc: departUtc + hoursBack * 3600000,
direction: 'departure', seat: '41C', confirmation },
  ];
}

function demoExpenses(start: string): Expense[] {
  const before = addDays(start, -20);
  return [
    { id: uid('e-'), title: 'Round-trip flights', amount: 1840, category: 'Flights', paidBy: 'm-priya', splitBetween: ALL, date: before
},
    { id: uid('e-'), title: 'Shinjuku apartment', amount: 960, category: 'Stays', paidBy: DEMO_USER_ID, splitBetween: ALL, date: addDays
(before, 2) },
    { id: uid('e-'), title: 'JR Rail passes', amount: 420, category: 'Transport', paidBy: 'm-sam', splitBetween: ALL, date: addDays(befo
re, 5) },
    { id: uid('e-'), title: 'Group dinners', amount: 265, category: 'Food', paidBy: 'm-diego', splitBetween: ALL, date: addDays(before,
8), original: { amount: 40386, currency: 'JPY' } },
    { id: uid('e-'), title: 'teamLab tickets', amount: 132, category: 'Activities', paidBy: 'm-ana', splitBetween: ALL, date: addDays(be
fore, 9) },
  ];
}

function demoPacking(): PackingItem[] {
  const done = new Set(['Passport', 'Phone charger', 'Universal adapter', 'Comfortable walking shoes']);
  return packingBase.flatMap((g) => g.items.map((text) => ({ id: uid('p-'), text, category: g.category, packed: done.has(text) })));
}

function demoPhotos(start: string, now: number): Photo[] {
  const list: [string, string, string, number][] = [
    ['shibuya', 'Shibuya after dark 🌃', 'm-sam', 1],
    ['ramen', 'Diego's 2am ramen find 🍜', 'm-diego', 0],
    ['fushimi-inari', '10,000 gates and counting ⛩️', 'm-sam', 5],
    ['lantern-alley', 'Lantern alley vibes', 'm-ana', 2],
    ['sushi', 'Omakase at the market', 'm-priya', 1],
    ['tokyo-skyline', 'View from the tower', DEMO_USER_ID, 2],
    ['fuji-blossom', 'Fuji finally showed up 🗻', 'm-noah', 3],
    ['omoide-yokocho', 'Yakitori alley', 'm-diego', 0],
    ['akihabara', 'Akiba arcade crew', 'm-noah', 2],
    ['kyoto-skyline', 'Kyoto from Kiyomizu', 'm-ana', 5],
```

```
  ];
  return list.map(([uri, caption, by, day], i) => ({
    id: uid('ph-'), uri, caption, by, day: addDays(start, day), ts: now - i * 3600000,
    likes: ALL.filter((_, j) => (i + j) % 3 !== 0),
  }));
}

export function inviteCodeFor(name: string): string {
  const letters = name.replace(/[^A-Za-z]/g, '').toUpperCase().padEnd(3, 'X').slice(0, 3);
  return `${letters}-${Math.floor(1000 + Math.random() * 9000)}`;
}

/** The big 10-day Tokyo & Kyoto trip — kept as a second trip so multi-trip management has something to show. */
export function buildTokyoTrip(now = Date.now()): Trip {
  const today = todayInTz(TOKYO_TZ);
  const start = addDays(today, 45);
  const end = addDays(start, 9);
  const flights = [
    ...flightPair(HND, DEMO_USER_ID, 'ANA', 'NH 175', 'NH 176', 'JFK', start, end, '14:20', '09:05', 14, 13),
    ...flightPair(HND, 'm-diego', 'Japan Airlines', 'JL 006', 'JL 005', 'JFK', start, end, '16:40', '11:30', 14, 13),
    ...flightPair(HND, 'm-priya', 'United', 'UA 837', 'UA 838', 'SFO', start, end, '07:15', '09:05', 11, 10, 1),
    ...flightPair(HND, 'm-sam', 'ANA', 'NH 175', 'NH 176', 'JFK', start, addDays(end, -1), '14:20', '18:00', 14, 13),
  ];
  const t = (mins: number) => now - mins * 60000;
  return {
    id: 'trip-tokyo',
    name: 'Tokyo & Kyoto Explorer',
    destinationId: 'tokyo',
    extraDestinationIds: ['kyoto', 'mount-fuji'],
    startDate: start,
    endDate: end,
    cover: 'tokyo',
    budget: 4200,
    inviteCode: 'TKY-2481',
    ownerId: DEMO_USER_ID,
    members: demoMembers.map((m) => ({ ...m })),
    activities: demoActivities(start),
    expenses: demoExpenses(start),
    flights,
    polls: [
      {
        id: uid('poll-'), question: 'Day 4 evening — what are we doing?', createdBy: 'm-noah', createdAt: t(600),
        options: [
          { id: 'o1', text: 'Robot show in Shinjuku', votes: ['m-noah'] },
          { id: 'o2', text: 'Sumida river cruise', votes: ['m-priya'] },
          { id: 'o3', text: 'Karaoke night', votes: ['m-diego', 'm-ana', 'm-sam'] },
        ],
      },
      {
        id: uid('poll-'), question: 'Farewell dinner: where should we go?', createdBy: 'm-diego', createdAt: t(300),
        options: [
          { id: 'o1', text: 'Gonpachi izakaya (Kill Bill vibes)', votes: ['m-diego', 'm-ana'] },
          { id: 'o2', text: 'Sushi omakase splurge', votes: ['m-priya'] },
          { id: 'o3', text: 'Ramen crawl in Ebisu', votes: ['m-noah'] },
        ],
      },
    ],
    tasks: [
      { id: uid('t-'), text: 'Passports valid 6+ months', done: true },
      { id: uid('t-'), text: 'Book airport transfer', done: false, assignee: DEMO_USER_ID },
      { id: uid('t-'), text: 'Travel insurance', done: false, assignee: 'm-priya' },
      { id: uid('t-'), text: 'Download offline maps', done: true, assignee: 'm-sam' },
      { id: uid('t-'), text: 'Pocket wifi pickup code', done: false, assignee: 'm-diego' },
    ],
    docs: [
      {
        id: uid('d-'), name: 'Flight tickets (all members)', kind: 'note', addedBy: 'm-priya', addedAt: t(4000), offline: true,
        note: 'Maya — ANA NH 175 · JFK → HND · Conf. TWGAYA\nSam — ANA NH 175 · JFK → HND · Conf. TWGSAM\nDiego — JAL JL 006 · JFK → HND
· Conf. TWGEGO\nPriya — United UA 837 · SFO → HND · Conf. TWGIYA\n\nCheck-in opens 24h before departure. Bring the passport used for boo
king.',
      },
      {
        id: uid('d-'), name: 'Apartment booking', kind: 'note', addedBy: DEMO_USER_ID, addedAt: t(3800), offline: true,
        note: 'Shinjuku Grand Hotel — 3 rooms, 4 nights\nCheck-in from 15:00 · Check-out 11:00\nBooking ref: SGH-55120\n\nShow the taxi
driver:\n新宿駅までお願いします (To Shinjuku Station, please)',
      },
      {
        id: uid('d-'), name: 'Passport scans', kind: 'note', addedBy: DEMO_USER_ID, addedAt: t(3600), offline: true,
        note: 'Encrypted copies stored on each member’s phone.\nIf a passport is lost: report at the nearest koban (police box), then co
ntact your embassy.\nUS Embassy Tokyo: +81 3-3224-5000',
      },
      {
        id: uid('d-'), name: 'Emergency contacts', kind: 'note', addedBy: 'm-ana', addedAt: t(3000), offline: true,
        note: 'Police 110 · Ambulance/Fire 119\nJapan Visitor Hotline (24h, English): 050-3816-2787\nTravel insurance: TravelSafe +1 800
555 0199, policy TS-88213',
      },
```

```
    ],
    messages: [
      { id: uid('c-'), authorId: 'm-priya', text: 'JR passes are booked 🎉 picking them up at Haneda', ts: t(320) },
      { id: uid('c-'), authorId: 'm-ana', text: 'Can we lock in teamLab for Day 2 afternoon? Tickets sell out', ts: t(260) },
      { id: uid('c-'), authorId: 'm-noah', text: 'Voted! Also started a poll for Day 4 evening 👀', ts: t(200) },
      { id: uid('c-'), authorId: 'm-diego', text: 'Found a ramen spot 4 min from the apartment, open till 2am 🍜', ts: t(42) },
      { id: uid('c-'), authorId: DEMO_USER_ID, text: 'Adding it to Day 1 after Shibuya!', ts: t(40) },
      { id: uid('c-'), authorId: 'assistant', ai: true, aiSource: 'local', text: 'Heads up: Priya lands at 07:15 on Day 2 — the morning after everyone else. Want me to move Tsukiji breakfast to 09:30 so she can join?', ts: t(38) },
    ],
    photos: demoPhotos(start, now),
    packing: demoPacking(),
    completed: false,
    createdAt: t(60 * 24 * 30),
  };
}


// ---------------------------------------------------------------------------
// Amalfi Squad — the main demo trip (4 friends, 6 days, Positano)
// ---------------------------------------------------------------------------
const squad: Member[] = [
  { id: DEMO_USER_ID, name: 'Maya Chen', email: DEMO_EMAIL, avatar: 'avatar-maya', role: 'Navigator', homeCity: 'New York' },
  { id: 'm-diego', name: 'Diego Alvarez', avatar: 'avatar-diego', role: 'Food Lead', homeCity: 'New York' },
  { id: 'm-priya', name: 'Priya Patel', avatar: 'avatar-priya', role: 'Treasurer', homeCity: 'San Francisco' },
  { id: 'm-sam', name: 'Sam Okafor', avatar: 'avatar-sam', role: 'Photographer', homeCity: 'New York' },
];
const SQUAD = squad.map((m) => m.id);

function amalfiActivities(start: string): Activity[] {
  return [
    act(start, 0, { time: '15:40', title: 'Arrive at Naples Airport (NAP)', location: 'Naples Airport', type: 'transport', durationMin:
90, costPerPerson: 0, status: 'confirmed', assigned: [DEMO_USER_ID], votes: votesFrom(SQUAD), lat: 40.886, lng: 14.2908, notes: 'Private
transfer to Positano, about 1h 30m. Driver holds a TogetherWeGo sign.' }),
    act(start, 0, { time: '17:45', title: 'Check-in — Villa Margherita', location: 'Positano', type: 'stay', durationMin: 30, costPerPer
son: 410, status: 'confirmed', assigned: ['m-priya'], votes: votesFrom(SQUAD), lat: 40.6283, lng: 14.4853, notes: 'Keys in the lockbox,
code in the Docs folder.' }),
    act(start, 0, { time: '19:00', title: 'Dinner at Da Vincenzo', location: 'Viale Pasitea', type: 'food', durationMin: 120, costPerPer
son: 64, status: 'confirmed', assigned: ['m-diego'], votes: votesFrom(SQUAD), lat: 40.6295, lng: 14.488, placeId: 'da-vincenzo' }),
    act(start, 1, { time: '08:00', title: 'Path of the Gods hike', location: 'Bomerano', type: 'adventure', durationMin: 300, costPerPer
son: 0, status: 'confirmed', assigned: [DEMO_USER_ID, 'm-sam'], votes: votesFrom([DEMO_USER_ID, 'm-diego', 'm-sam']), lat: 40.619, lng:
14.5176, placeId: 'path-of-gods', notes: 'Bring water, sun hat and sturdy shoes. Bus from Amalfi to Bomerano at 07:15.' }),
    act(start, 1, { time: '20:00', title: 'Sunset aperitivo at Le Sirenuse', location: 'Positano', type: 'nightlife', durationMin: 90, c
ostPerPerson: 40, status: 'confirmed', assigned: ['m-sam'], votes: votesFrom(SQUAD.slice(0, 3)), lat: 40.6293, lng: 14.4868, placeId: 's
unset-bar' }),
    act(start, 2, { time: '10:00', title: 'Boat tour to Capri', location: 'Positano Marina', type: 'adventure', durationMin: 540, costPe
rPerson: 140, status: 'confirmed', assigned: [DEMO_USER_ID, 'm-priya'], votes: votesFrom([DEMO_USER_ID, 'm-priya']), lat: 40.6272, lng:
14.4868, placeId: 'capri-boat' }),
    act(start, 3, { time: '11:00', title: 'Beach day at Fornillo Beach', location: 'Fornillo Beach', type: 'sightseeing', durationMin: 1
80, costPerPerson: 35, status: 'voting', assigned: [], votes: {}, reminder: false, lat: 40.6255, lng: 14.4795, placeId: 'fornillo' }),
    act(start, 3, { time: '16:00', title: 'Limoncello tasting', location: 'Amalfi', type: 'food', durationMin: 45, costPerPerson: 23, st
atus: 'voting', assigned: ['m-priya'], votes: votesFrom(['m-priya']), reminder: false, lat: 40.634, lng: 14.6027, placeId: 'limoncello'
}),
    act(start, 4, { time: '10:00', title: 'Villa Rufolo gardens in Ravello', location: 'Ravello', type: 'sightseeing', durationMin: 150,
costPerPerson: 8, status: 'confirmed', assigned: ['m-sam'], votes: votesFrom(SQUAD), lat: 40.649, lng: 14.6118, placeId: 'villa-rufolo'
}),
    act(start, 4, { time: '19:30', title: 'Farewell dinner — Trattoria da Adolfo', location: 'Laurito Beach', type: 'food', durationMin:
150, costPerPerson: 55, status: 'confirmed', assigned: ['m-diego'], votes: votesFrom(SQUAD.slice(0, 3)), lat: 40.6224, lng: 14.4731, pla
ceId: 'trattoria-chitarrella' }),
    act(start, 5, { time: '14:30', title: 'Transfer to Naples Airport', location: 'Positano', type: 'transport', durationMin: 100, costP
erPerson: 35, status: 'confirmed', assigned: [DEMO_USER_ID], votes: votesFrom(SQUAD), lat: 40.628, lng: 14.485 }),
  ];
}

export function buildDemoTrip(timeline: DemoTimeline = 'upcoming', now = Date.now()): Trip {
  const today = todayInTz(AMALFI_TZ);
  const offset = timeline === 'upcoming' ? 3 : timeline === 'active' ? -1 : -14;
  const start = addDays(today, offset);
  const end = addDays(start, 5);
  const t = (mins: number) => now - mins * 60000;
  const before = addDays(start, -20);
  return {
    id: 'trip-amalfi',
    name: 'Amalfi Squad',
    destinationId: 'positano',
    extraDestinationIds: [],
    startDate: start,
    endDate: end,
    cover: 'amalfi',
    budget: 4000,
    inviteCode: 'AML-2026',
    ownerId: DEMO_USER_ID,
    members: squad.map((m) => ({ ...m })),
    activities: amalfiActivities(start),
    expenses: [
      { id: uid('e-'), title: 'Villa Margherita (6 nights)', amount: 1650, category: 'Stays', paidBy: 'm-priya', splitBetween: SQUAD, da
```

```
te: before },
      { id: uid('e-'), title: 'Airport transfers', amount: 140, category: 'Transport', paidBy: DEMO_USER_ID, splitBetween: SQUAD, date:
addDays(before, 4) },
      { id: uid('e-'), title: 'Scooters & ferry tickets', amount: 210, category: 'Transport', paidBy: 'm-sam', splitBetween: SQUAD, dat
e: addDays(before, 6) },
      { id: uid('e-'), title: 'Dinner deposits & groceries', amount: 86, category: 'Food', paidBy: 'm-diego', splitBetween: SQUAD, date:
addDays(before, 8), original: { amount: 74, currency: 'EUR' } },
    ],
    flights: [
      ...flightPair(NAP, DEMO_USER_ID, 'Delta', 'DL 184', 'DL 185', 'JFK', start, end, '15:40', '19:15', 9.5, 10),
      ...flightPair(NAP, 'm-priya', 'ITA Airways', 'AZ 609', 'AZ 610', 'JFK', start, end, '17:05', '21:30', 9.5, 10),
    ],
    polls: [
      {
        id: uid('poll-'), question: 'Beach day — which beach?', createdBy: 'm-sam', createdAt: t(600),
        options: [
          { id: 'o1', text: 'Fornillo (quiet, pebbles)', votes: ['m-sam', 'm-diego'] },
          { id: 'o2', text: 'Spiaggia Grande (lively)', votes: ['m-priya'] },
          { id: 'o3', text: 'Fiordo di Furore (jump in!)', votes: [DEMO_USER_ID] },
        ],
      },
    ],
    tasks: [
      { id: uid('t-'), text: 'Passports valid 6+ months', done: true },
      { id: uid('t-'), text: 'Book airport transfers', done: true, assignee: DEMO_USER_ID },
      { id: uid('t-'), text: 'Travel insurance', done: false, assignee: 'm-priya' },
      { id: uid('t-'), text: 'Rent scooters (licences!)', done: false, assignee: 'm-sam' },
      { id: uid('t-'), text: 'Reserve Da Vincenzo for 4', done: true, assignee: 'm-diego' },
      { id: uid('t-'), text: 'Download offline maps', done: false, assignee: DEMO_USER_ID },
    ],
    docs: [
      {
        id: uid('d-'), name: 'Flight tickets', kind: 'note', addedBy: 'm-priya', addedAt: t(4000), offline: true,
        note: 'Maya — Delta DL 184 · JFK → NAP · Conf. TWGAYA\nPriya — ITA Airways AZ 609 · JFK → NAP · Conf. TWGIYA\n\nCheck-in opens 2
4h before departure. Bring the passport used for booking.',
      },
      {
        id: uid('d-'), name: 'Villa Margherita booking', kind: 'note', addedBy: 'm-priya', addedAt: t(3800), offline: true,
        note: 'Villa Margherita, Via Chiesa Nuova, Positano\nCheck-in from 15:00 · Check-out 10:00\nBooking ref: VMG-48213\nDoor code ar
rives by message on the day.\n\nTaxi phrase:\nVilla Margherita, per favore (Villa Margherita, please)',
      },
      {
        id: uid('d-'), name: 'Passport scans', kind: 'note', addedBy: DEMO_USER_ID, addedAt: t(3600), offline: true,
        note: 'Copies are stored on each member’s phone.\nIf a passport is lost: report it to the Carabinieri, then contact your embass
y.\nUS Consulate Naples: +39 081 583 8111',
      },
      {
        id: uid('d-'), name: 'Emergency contacts', kind: 'note', addedBy: 'm-diego', addedAt: t(3000), offline: true,
        note: 'EU emergency 112 · Ambulance 118 · Fire 115\nTravel insurance: TravelSafe +1 800 555 0199, policy TS-88213\nNearest hospi
tal: Ospedale Costa d’Amalfi, Castiglione di Ravello',
      },
    ],
    messages: [
      { id: uid('c-'), authorId: 'm-priya', text: 'Villa is booked! 🏡 Paid the deposit.', ts: t(320) },
      { id: uid('c-'), authorId: 'm-sam', text: 'I’ll handle maps & the scooter rental.', ts: t(260) },
      { id: uid('c-'), authorId: 'm-diego', text: 'Dinner tonight at 7, I reserved for 4 🍝', ts: t(200) },
    ],
    photos: [
      { id: uid('ph-'), uri: 'amalfi', caption: 'First look at the coast 😍', by: 'm-sam', day: start, ts: t(120), likes: [DEMO_USER_ID,
'm-priya'] },
      { id: uid('ph-'), uri: 'cinque-terre', caption: 'Cinque Terre next time?', by: 'm-priya', day: start, ts: t(90), likes: ['m-dieg
o'] },
    ],
    packing: demoPacking(),
    completed: timeline === 'completed',
    createdAt: t(60 * 24 * 30),
  };
}

export function buildSecondaryTrips(now = Date.now()): Trip[] {
  const today = todayInTz('UTC');
  const baliStart = addDays(today, 120);
  const bcnStart = addDays(today, -160);
  const four = demoMembers.slice(0, 4).map((m) => ({ ...m }));
  return [
    buildTokyoTrip(now),
    {
      id: 'trip-bali', name: 'Bali Surf & Chill', destinationId: 'bali', extraDestinationIds: [], startDate: baliStart, endDate: addDays
(baliStart, 6),
      cover: 'bali', budget: 3000, inviteCode: 'BAL-7316', ownerId: 'm-priya', members: four,
      activities: [
        { id: uid('a-'), date: addDays(baliStart, 1), time: '04:00', title: 'Mount Batur sunrise trek', location: 'Kintamani, Bali', typ
e: 'adventure', durationMin: 360, costPerPerson: 45, status: 'voting', assigned: ['m-priya'], votes: votesFrom(['m-priya', DEMO_USER_I
D]), reminder: false, createdBy: 'm-priya' },
        { id: uid('a-'), date: addDays(baliStart, 2), time: '09:00', title: 'Surf lesson at Kuta', location: 'Kuta, Bali', type: 'advent
ure', durationMin: 150, costPerPerson: 30, status: 'pending', assigned: ['m-sam'], votes: votesFrom(['m-sam']), reminder: false, created
```

```
By: 'm-sam' },
      ],
      expenses: [{ id: uid('e-'), title: 'Villa deposit', amount: 600, category: 'Stays', paidBy: 'm-priya', splitBetween: four.map((m)
=> m.id), date: today }],
      flights: [], polls: [], tasks: [{ id: uid('t-'), text: 'Decide villa vs. hostel', done: false, assignee: 'm-priya' }], docs: [],
      messages: [{ id: uid('c-'), authorId: 'm-priya', text: 'Started planning Bali for next year 🌴 who's in?', ts: now - 86400000 }],
      photos: [], packing: [], createdAt: now - 86400000 * 2,
    },
    {
      id: 'trip-bcn', name: 'Barcelona Spring Break', destinationId: 'barcelona', extraDestinationIds: [], startDate: bcnStart, endDate:
addDays(bcnStart, 5),
      cover: 'barcelona', budget: 2400, inviteCode: 'BCN-4402', ownerId: DEMO_USER_ID, members: four, completed: true,
      activities: [
        { id: uid('a-'), date: addDays(bcnStart, 1), time: '10:00', title: 'Sagrada Família tour', location: 'Eixample', type: 'cultur
e', durationMin: 120, costPerPerson: 34, status: 'confirmed', assigned: [DEMO_USER_ID], votes: votesFrom(four.map((m) => m.id)), reminde
r: false, createdBy: DEMO_USER_ID, done: true },
        { id: uid('a-'), date: addDays(bcnStart, 2), time: '13:00', title: 'Tapas crawl in El Born', location: 'El Born', type: 'food',
durationMin: 180, costPerPerson: 40, status: 'confirmed', assigned: ['m-diego'], votes: votesFrom(four.map((m) => m.id)), reminder: fals
e, createdBy: 'm-diego', done: true },
        { id: uid('a-'), date: addDays(bcnStart, 3), time: '11:00', title: 'Barceloneta beach day', location: 'Barceloneta', type: 'sigh
tseeing', durationMin: 300, costPerPerson: 0, status: 'confirmed', assigned: ['m-sam'], votes: votesFrom(four.map((m) => m.id)), reminde
r: false, createdBy: 'm-sam', done: true },
      ],
      expenses: [
        { id: uid('e-'), title: 'Flights', amount: 1320, category: 'Flights', paidBy: DEMO_USER_ID, splitBetween: four.map((m) => m.id),
date: bcnStart },
        { id: uid('e-'), title: 'Airbnb Gothic Quarter', amount: 640, category: 'Stays', paidBy: 'm-priya', splitBetween: four.map((m) =
> m.id), date: bcnStart },
        { id: uid('e-'), title: 'Tapas & dinners', amount: 310, category: 'Food', paidBy: 'm-diego', splitBetween: four.map((m) => m.i
d), date: addDays(bcnStart, 2) },
      ],
      flights: [], polls: [], tasks: [], docs: [], messages: [], packing: [],
      photos: [
        { id: uid('ph-'), uri: 'barcelona', caption: 'Rooftop views', by: 'm-sam', likes: [DEMO_USER_ID], ts: now - 86400000 * 150, day:
addDays(bcnStart, 1) },
      ],
      createdAt: now - 86400000 * 200,
    },
  ];
}

/** Trips that exist "on the server" and can be joined with an invite code. */
export function directoryTrips(now = Date.now()): Trip[] {
  const start = addDays(todayInTz('UTC'), 45);
  const jordan: Member = { id: 'm-jordan', name: 'Jordan Lee', role: 'Organizer', homeCity: 'Philadelphia' };
  const kai: Member = { id: 'm-kai', name: 'Kai Thompson', role: 'Food Lead', homeCity: 'Philadelphia' };
  return [
    {
      id: 'trip-seoul', name: 'Seoul Food Crawl', destinationId: 'seoul', extraDestinationIds: [], startDate: start, endDate: addDays(st
art, 5),
      cover: 'seoul', budget: 2600, inviteCode: 'SEO-5521', ownerId: jordan.id, members: [jordan, kai],
      activities: [
        { id: uid('a-'), date: addDays(start, 1), time: '18:00', title: 'Gwangjang Market night food tour', location: 'Jongno, Seoul', t
ype: 'food', durationMin: 150, costPerPerson: 35, status: 'confirmed', assigned: [kai.id], votes: votesFrom([jordan.id, kai.id]), remind
er: true, createdBy: kai.id },
      ],
      expenses: [], flights: [], polls: [], tasks: [], docs: [], photos: [], packing: [],
      messages: [{ id: uid('c-'), authorId: jordan.id, text: 'Welcome to the Seoul crew! Drop your flight times when you book ✈️', ts: n
ow - 3600000 }],
      createdAt: now - 86400000,
    },
  ];
}

export { AMALFI_TZ as DEMO_TZ };
```

src/store/types.ts

```
import type { Role } from '@/data/reference';
import type { LangCode } from '@/i18n/strings';

export type ActivityType = 'transport' | 'stay' | 'food' | 'sightseeing' | 'adventure' | 'culture' | 'nightlife' | 'shopping';
export type ActivityStatus = 'confirmed' | 'voting' | 'pending';

export interface Member {
  id: string;
  name: string;
  email?: string;
  avatar?: string;
  role: Role;
  homeCity?: string;
  /** Members added by name only (no account yet). */
  invited?: boolean;
}

export interface Activity {
  id: string;
  date: string;
  time: string;
  title: string;
  location: string;
  type: ActivityType;
  durationMin: number;
  costPerPerson: number;
  status: ActivityStatus;
  assigned: string[];
  /** memberId → +1 (up) / -1 (down) */
  votes: Record<string, 1 | -1>;
  notes?: string;
  reminder: boolean;
  lat?: number;
  lng?: number;
  placeId?: string;
  createdBy: string;
  done?: boolean;
}

export type ExpenseCategory = 'Flights' | 'Stays' | 'Food' | 'Activities' | 'Transport' | 'Shopping' | 'Other';

export interface Expense {
  id: string;
  title: string;
  /** Always stored in the trip's base currency (USD by default). */
  amount: number;
  category: ExpenseCategory;
  paidBy: string;
  splitBetween: string[];
  date: string;
  /** Original amount when entered in a foreign currency. */
  original?: { amount: number; currency: string };
  settlement?: boolean;
  /** Photo uri of the receipt. */
  receipt?: string;
}

export interface Flight {
  id: string;
  memberId: string;
  airline: string;
  flightNo: string;
  from: string;
  to: string;
  departUtc: number;
  arriveUtc: number;
  direction: 'arrival' | 'departure';
  seat?: string;
  confirmation?: string;
  ticketUri?: string;
  ticketName?: string;
}

export interface PollOption {
  id: string;
  text: string;
  votes: string[];
}

export interface Poll {
  id: string;
  question: string;
  options: PollOption[];
  createdBy: string;
```

```
  closed?: boolean;
  createdAt: number;
}

export interface Task {
  id: string;
  text: string;
  done: boolean;
  assignee?: string;
}

export interface TripDoc {
  id: string;
  name: string;
  kind: 'pdf' | 'image' | 'note' | 'file';
  uri?: string;
  note?: string;
  size?: number;
  addedBy: string;
  addedAt: number;
  offline: boolean;
}

export interface ChatMessage {
  id: string;
  authorId: string;
  text: string;
  ts: number;
  kind?: 'text' | 'system';
  /** True when the message came from the AI trip assistant. */
  ai?: boolean;
  aiSource?: 'local' | 'claude';
  /** Which thread the message belongs to: the human crew chat (default) or the Scout AI thread. */
  channel?: 'crew' | 'ai';
}

export interface Photo {
  id: string;
  uri: string;
  caption: string;
  by: string;
  likes: string[];
  ts: number;
  day?: string;
}

export interface PackingItem {
  id: string;
  text: string;
  category: string;
  packed: boolean;
  /** Member id of whoever is bringing it. */
  broughtBy?: string;
}

export type TripStatus = 'planning' | 'active' | 'completed';

export interface Trip {
  id: string;
  name: string;
  destinationId: string;
  extraDestinationIds: string[];
  startDate: string;
  endDate: string;
  cover: string;
  /** Group budget cap in USD. */
  budget: number;
  inviteCode: string;
  ownerId: string;
  members: Member[];
  activities: Activity[];
  expenses: Expense[];
  flights: Flight[];
  polls: Poll[];
  tasks: Task[];
  docs: TripDoc[];
  messages: ChatMessage[];
  photos: Photo[];
  packing: PackingItem[];
  /** Manual override: user marked the trip as finished. */
  completed?: boolean;
  createdAt: number;
}

export interface Account {
  id: string;
```

```
  name: string;
  email: string;
  passwordHash: string;
  salt: string;
  avatar?: string;
  homeCity?: string;
  provider: 'password' | 'google';
}

export interface BucketItem {
  id: string;
  text: string;
  destinationId?: string;
  done: boolean;
  addedAt: number;
}

export interface AppNotification {
  id: string;
  title: string;
  body: string;
  ts: number;
  read: boolean;
  kind: 'reminder' | 'chat' | 'vote' | 'expense' | 'system' | 'member';
  tripId?: string;
  route?: string;
}

export type TextScale = 1 | 1.12 | 1.25;

export interface Settings {
  language: LangCode;
  homeCurrency: string;
  notificationsEnabled: boolean;
  reminderLeadMin: number;
  textScale: TextScale;
  showTripTime: boolean;
  aiMode: 'offline' | 'claude';
  haptics: boolean;
}

/** Home-screen blocks the user can show or hide in "Make it yours". */
export type HomeSection = 'glance' | 'budget' | 'upnext' | 'quick' | 'forYou' | 'nearby' | 'discover';

/** Answers from the "Make it yours" personalization survey. */
export interface Personal {
  done: boolean;
  nickname: string;
  emoji: string;
  interests: string[];
  travelStyle: 'chill' | 'balanced' | 'packed';
  budgetStyle: 'saver' | 'mid' | 'treat';
  sections: Record<HomeSection, boolean>;
  motion: boolean;
  /** True once the first-run guided tour has been seen. */
  toured?: boolean;
}

export interface WeatherSnapshot {
  destinationId: string;
  fetchedAt: number;
  current: { temp: number; code: number; wind: number; humidity: number };
  daily: { date: string; max: number; min: number; code: number; rain: number }[];
  live: boolean;
}

export interface RatesSnapshot {
  base: 'USD';
  rates: Record<string, number>;
  fetchedAt: number;
  live: boolean;
}
```

src/store/useAppStore.ts

```
/**
 * Single source of truth for the app (MVVM "model" layer).
 * - zustand for state + actions, immer for safe nested updates
 * - persisted to AsyncStorage so everything works offline
 * - passwords are salted + SHA-256 hashed, never stored in plain text
 */
import AsyncStorage from '@react-native-async-storage/async-storage';
import * as Crypto from 'expo-crypto';
import { create } from 'zustand';
import { createJSONStorage, persist } from 'zustand/middleware';
import { immer } from 'zustand/middleware/immer';

import { destinationById } from '@/data/destinations';
import type { Role } from '@/data/reference';
import { uid } from '@/utils/format';
import { daysBetween, todayInTz } from '@/utils/time';
import { EMAIL_RE } from '@/utils/validation';

import {
  DEMO_EMAIL,
  DEMO_PASSWORD,
  DEMO_USER_ID,
  buildDemoTrip,
  buildSecondaryTrips,
  directoryTrips,
  inviteCodeFor,
  type DemoTimeline,
} from './seed';
import type {
  Personal,
  Account,
  Activity,
  AppNotification,
  BucketItem,
  ChatMessage,
  Expense,
  Flight,
  Member,
  PackingItem,
  Photo,
  Poll,
  RatesSnapshot,
  Settings,
  Trip,
  TripDoc,
  TripStatus,
  WeatherSnapshot,
} from './types';

export async function hashPassword(password: string, salt: string): Promise<string> {
  return Crypto.digestStringAsync(Crypto.CryptoDigestAlgorithm.SHA256, `${salt}:${password}`);
}

type Result = { ok: true } | { ok: false; error: string };

interface Session {
  userId: string;
  remember: boolean;
}

interface State {
  hydrated: boolean;
  hasOnboarded: boolean;
  session: Session | null;
  accounts: Account[];
  trips: Trip[];
  activeTripId: string | null;
  bucket: BucketItem[];
  favorites: string[];
  notifications: AppNotification[];
  settings: Settings;
  weather: Record<string, WeatherSnapshot>;
  rates: RatesSnapshot | null;
  quizBest: number;
  surveyResult: string[] | null;
  personal: Personal;
  /** Set just before the app reloads to apply a new look: keeps the session for that one restart. */
  restyled: boolean;
  firedReminders: Record<string, number>;
  demoTimeline: DemoTimeline;
  seeded: boolean;
}

interface Actions {
```

```
  setHydrated: () => void;
  ensureSeed: () => Promise<void>;
  completeOnboarding: () => void;

  signIn: (email: string, password: string, remember: boolean) => Promise<Result>;
  signUp: (name: string, email: string, password: string) => Promise<Result>;
  signInWithGoogle: (email: string, name: string) => Promise<Result>;
  signOut: () => void;
  accountExists: (email: string) => boolean;
  resetPassword: (email: string, newPassword: string) => Promise<Result>;
  updateProfile: (patch: Partial<Pick<Account, 'name' | 'homeCity' | 'avatar'>>) => void;

  setActiveTrip: (id: string) => void;
  createTrip: (t: { name: string; destinationId: string; startDate: string; endDate: string; budget: number }) => string;
  joinTrip: (code: string) => { ok: true; tripId: string } | { ok: false; error: string };
  leaveTrip: (tripId: string) => void;
  deleteTrip: (tripId: string) => void;
  updateTrip: (tripId: string, patch: Partial<Pick<Trip, 'name' | 'startDate' | 'endDate' | 'budget' | 'completed' | 'cover' | 'destinat
ionId' | 'extraDestinationIds'>>) => void;
  regenerateInvite: (tripId: string) => void;
  reseedDemo: (timeline: DemoTimeline) => void;

  addMember: (tripId: string, name: string, email?: string) => void;
  removeMember: (tripId: string, memberId: string) => void;
  setRole: (tripId: string, memberId: string, role: Role) => void;

  addActivity: (tripId: string, a: Omit<Activity, 'id' | 'createdBy' | 'votes'> & { votes?: Activity['votes'] }) => string;
  updateActivity: (tripId: string, id: string, patch: Partial<Activity>) => void;
  deleteActivity: (tripId: string, id: string) => void;
  voteActivity: (tripId: string, id: string, value: 1 | -1) => void;
  toggleAssign: (tripId: string, id: string, memberId: string) => void;

  addExpense: (tripId: string, e: Omit<Expense, 'id'>) => void;
  deleteExpense: (tripId: string, id: string) => void;
  settleUp: (tripId: string, from: string, to: string, amount: number) => void;

  addFlight: (tripId: string, f: Omit<Flight, 'id'>) => void;
  deleteFlight: (tripId: string, id: string) => void;

  createPoll: (tripId: string, question: string, options: string[]) => void;
  votePoll: (tripId: string, pollId: string, optionId: string) => void;
  closePoll: (tripId: string, pollId: string) => void;
  deletePoll: (tripId: string, pollId: string) => void;

  addTask: (tripId: string, text: string, assignee?: string) => void;
  toggleTask: (tripId: string, id: string) => void;
  deleteTask: (tripId: string, id: string) => void;
  assignTask: (tripId: string, id: string, memberId?: string) => void;

  addDoc: (tripId: string, d: Omit<TripDoc, 'id' | 'addedAt' | 'addedBy'>) => void;
  deleteDoc: (tripId: string, id: string) => void;

  sendMessage: (tripId: string, msg: Omit<ChatMessage, 'id' | 'ts'> & { ts?: number }) => void;
  clearChat: (tripId: string) => void;

  addPhoto: (tripId: string, p: Omit<Photo, 'id' | 'ts' | 'likes' | 'by'>) => void;
  toggleLikePhoto: (tripId: string, id: string) => void;
  deletePhoto: (tripId: string, id: string) => void;

  togglePacked: (tripId: string, id: string) => void;
  assignPacking: (tripId: string, id: string, memberId?: string) => void;
  addPackingItems: (tripId: string, items: Omit<PackingItem, 'id' | 'packed'>[]) => void;
  deletePackingItem: (tripId: string, id: string) => void;

  addBucket: (text: string, destinationId?: string) => void;
  toggleBucket: (id: string) => void;
  deleteBucket: (id: string) => void;
  toggleFavorite: (destinationId: string) => void;

  pushNotification: (n: Omit<AppNotification, 'id' | 'ts' | 'read'>) => void;
  markAllRead: () => void;
  markKindRead: (kind: AppNotification['kind']) => void;
  clearNotifications: () => void;

  updateSettings: (patch: Partial<Settings>) => void;
  setWeather: (w: WeatherSnapshot) => void;
  setRates: (r: RatesSnapshot) => void;
  setQuizBest: (n: number) => void;
  setSurveyResult: (ids: string[] | null) => void;
  updatePersonal: (patch: Partial<Personal>) => void;
  setRestyled: (v: boolean) => void;
  setToured: (v: boolean) => void;
  markReminderFired: (key: string) => void;
  resetEverything: () => void;
}
```

```
export type AppStore = State & Actions;

const defaultSettings: Settings = {
  language: 'en',
  homeCurrency: 'USD',
  notificationsEnabled: true,
  reminderLeadMin: 30,
  textScale: 1,
  showTripTime: true,
  aiMode: 'offline',
  haptics: true,
};

export const defaultPersonal: Personal = {
  done: false,
  nickname: '',
  emoji: '✈️',
  interests: [],
  travelStyle: 'balanced',
  budgetStyle: 'mid',
  sections: { glance: true, budget: true, upnext: true, quick: true, forYou: true, nearby: true, discover: true },
  motion: true,
};

const initialState: State = {
  hydrated: false,
  hasOnboarded: false,
  session: null,
  accounts: [],
  trips: [],
  activeTripId: null,
  bucket: [],
  favorites: ['tokyo', 'kyoto'],
  notifications: [],
  settings: defaultSettings,
  weather: {},
  rates: null,
  quizBest: 0,
  surveyResult: null,
  personal: defaultPersonal,
  restyled: false,
  firedReminders: {},
  demoTimeline: 'upcoming',
  seeded: false,
};

function seedBucket(): BucketItem[] {
  const now = Date.now();
  return [
    { id: uid('b-'), text: 'See the northern lights in Iceland', destinationId: 'iceland', done: false, addedAt: now - 9e7 },
    { id: uid('b-'), text: 'Sunrise at Machu Picchu', destinationId: 'machu-picchu', done: false, addedAt: now - 8e7 },
    { id: uid('b-'), text: 'Eat street tacos in Mexico City', destinationId: 'mexico-city', done: false, addedAt: now - 7e7 },
    { id: uid('b-'), text: 'Watch the sunset in Oia, Santorini', destinationId: 'santorini', done: false, addedAt: now - 6e7 },
    { id: uid('b-'), text: 'Tapas crawl in Barcelona', destinationId: 'barcelona', done: true, addedAt: now - 5e7 },
  ];
}

function seedNotifications(): AppNotification[] {
  const now = Date.now();
  return [
    { id: uid('n-'), kind: 'vote', title: 'New poll from Sam', body: 'Beach day — which beach? Cast your vote.', ts: now - 600 * 60000,
read: false, tripId: 'trip-amalfi', route: '/crew' },
    { id: uid('n-'), kind: 'expense', title: 'Diego added an expense', body: 'Dinner deposits & groceries · $86 split 4 ways', ts: now -
420 * 60000, read: false, tripId: 'trip-amalfi', route: '/(tabs)/budget' },
    { id: uid('n-'), kind: 'chat', title: 'Diego in Amalfi Squad', body: 'Dinner tonight at 7, I reserved for 4 🍝', ts: now - 42 * 6000
0, read: false, tripId: 'trip-amalfi', route: '/(tabs)/chat' },
  ];
}

export const useAppStore = create<AppStore>()(
  persist(
    immer((set, get) => {
      /** Mutates a trip in place (immer draft) if it exists. */
      const withTrip = (tripId: string, fn: (t: Trip, s: State) => void) =>
        set((s) => {
          const t = s.trips.find((x) => x.id === tripId);
          if (t) fn(t, s);
        });
      const me = () => get().session?.userId ?? DEMO_USER_ID;

      return {
        ...initialState,

        setHydrated: () => set({ hydrated: true }),

        ensureSeed: async () => {
```

```
          if (get().seeded) return;
          const salt = Crypto.randomUUID();
          const passwordHash = await hashPassword(DEMO_PASSWORD, salt);
          const demo: Account = { id: DEMO_USER_ID, name: 'Maya Chen', email: DEMO_EMAIL, passwordHash, salt, avatar: 'avatar-maya', hom
eCity: 'New York', provider: 'password' };
          set((s) => {
            if (s.seeded) return;
            s.accounts = [demo, ...s.accounts.filter((a) => a.email !== DEMO_EMAIL)];
            s.trips = [buildDemoTrip('upcoming'), ...buildSecondaryTrips(), ...s.trips];
            s.activeTripId = 'trip-amalfi';
            s.bucket = seedBucket();
            s.notifications = seedNotifications();
            s.seeded = true;
          });
        },

        completeOnboarding: () => set({ hasOnboarded: true }),

        signIn: async (email, password, remember) => {
          const acct = get().accounts.find((a) => a.email.toLowerCase() === email.trim().toLowerCase());
          if (!acct) return { ok: false, error: 'No account found with that email. Try Sign Up instead.' };
          if (acct.provider === 'google' && !acct.passwordHash) return { ok: false, error: 'This account uses Google sign-in.' };
          const hash = await hashPassword(password, acct.salt);
          if (hash !== acct.passwordHash) return { ok: false, error: 'Incorrect password. Please try again.' };
          set((s) => {
            s.session = { userId: acct.id, remember };
            const mine = s.trips.find((t) => t.members.some((m) => m.id === acct.id));
            if (!s.trips.find((t) => t.id === s.activeTripId && t.members.some((m) => m.id === acct.id))) {
              s.activeTripId = mine?.id ?? null;
            }
          });
          return { ok: true };
        },

        signUp: async (name, email, password) => {
          const e = email.trim().toLowerCase();
          if (!EMAIL_RE.test(e)) return { ok: false, error: 'Enter a valid email address.' };
          if (get().accounts.some((a) => a.email.toLowerCase() === e)) {
            return { ok: false, error: 'An account with this email already exists. Sign in instead.' };
          }
          const salt = Crypto.randomUUID();
          const passwordHash = await hashPassword(password, salt);
          const id = uid('u-');
          set((s) => {
            s.accounts.push({ id, name: name.trim(), email: e, passwordHash, salt, provider: 'password' });
            s.session = { userId: id, remember: true };
            s.activeTripId = null;
          });
          return { ok: true };
        },

        signInWithGoogle: async (email, name) => {
          const e = email.trim().toLowerCase();
          let acct = get().accounts.find((a) => a.email.toLowerCase() === e);
          if (!acct) {
            acct = { id: uid('u-'), name, email: e, passwordHash: '', salt: '', provider: 'google' };
            const created = acct;
            set((s) => {
              s.accounts.push(created);
            });
          }
          const id = acct.id;
          set((s) => {
            s.session = { userId: id, remember: true };
            const mine = s.trips.find((t) => t.members.some((m) => m.id === id));
            s.activeTripId = mine?.id ?? null;
          });
          return { ok: true };
        },

        signOut: () => set({ session: null }),

        accountExists: (email) => get().accounts.some((a) => a.email.toLowerCase() === email.trim().toLowerCase()),

        resetPassword: async (email, newPassword) => {
          const e = email.trim().toLowerCase();
          if (!get().accounts.some((a) => a.email.toLowerCase() === e)) return { ok: false, error: 'No account found with that email.'
};
          const salt = Crypto.randomUUID();
          const passwordHash = await hashPassword(newPassword, salt);
          set((s) => {
            const a = s.accounts.find((x) => x.email.toLowerCase() === e);
            if (a) {
              a.salt = salt;
              a.passwordHash = passwordHash;
              a.provider = 'password';
```

```
          }
        });
        return { ok: true };
      },

      updateProfile: (patch) =>
        set((s) => {
          const a = s.accounts.find((x) => x.id === s.session?.userId);
          if (!a) return;
          Object.assign(a, patch);
          for (const t of s.trips) {
            const m = t.members.find((x) => x.id === a.id);
            if (m) {
              if (patch.name) m.name = patch.name;
              if (patch.avatar !== undefined) m.avatar = patch.avatar;
              if (patch.homeCity !== undefined) m.homeCity = patch.homeCity;
            }
          }
        }),

      setActiveTrip: (id) => set({ activeTripId: id }),

      createTrip: ({ name, destinationId, startDate, endDate, budget }) => {
        const id = uid('trip-');
        const acct = get().accounts.find((a) => a.id === me());
        const dest = destinationById(destinationId);
        set((s) => {
          s.trips.unshift({
            id, name: name.trim(), destinationId, extraDestinationIds: [], startDate, endDate,
            cover: dest?.image ?? 'planning', budget, inviteCode: inviteCodeFor(dest?.city ?? name), ownerId: me(),
            members: [{ id: me(), name: acct?.name ?? 'You', email: acct?.email, avatar: acct?.avatar, role: 'Organizer', homeCity: ac
ct?.homeCity }],
            activities: [], expenses: [], flights: [], polls: [], tasks: [
              { id: uid('t-'), text: 'Invite your travel group', done: false, assignee: me() },
              { id: uid('t-'), text: 'Agree on a budget', done: false },
              { id: uid('t-'), text: 'Add everyone’s flights', done: false },
            ], docs: [], photos: [], packing: [],
            messages: [{ id: uid('c-'), authorId: 'assistant', ai: true, aiSource: 'local', ts: Date.now(), text: `Welcome to ${name.t
rim()}! Share invite code to bring your group in, then ask me anything — budget, weather, food nearby or what to do next.` }],
            createdAt: Date.now(),
          });
          s.activeTripId = id;
        });
        return id;
      },

      joinTrip: (code) => {
        const c = code.trim().toUpperCase();
        const user = me();
        const acct = get().accounts.find((a) => a.id === user);
        const local = get().trips.find((t) => t.inviteCode === c);
        if (local) {
          if (local.members.some((m) => m.id === user)) {
            set({ activeTripId: local.id });
            return { ok: false, error: `You’re already in “${local.name}”.` };
          }
          set((s) => {
            const t = s.trips.find((x) => x.id === local.id)!;
            t.members.push({ id: user, name: acct?.name ?? 'New member', email: acct?.email, avatar: acct?.avatar, role: 'Member', hom
eCity: acct?.homeCity });
            t.messages.push({ id: uid('c-'), authorId: 'system', kind: 'system', text: `${acct?.name ?? 'Someone'} joined the trip 👋
`, ts: Date.now() });
            s.activeTripId = t.id;
          });
          return { ok: true, tripId: local.id };
        }
        const remote = directoryTrips().find((t) => t.inviteCode === c);
        if (!remote) return { ok: false, error: 'No trip found with that code. Check with your friend and try again.' };
        set((s) => {
          remote.members.push({ id: user, name: acct?.name ?? 'New member', email: acct?.email, avatar: acct?.avatar, role: 'Member',
homeCity: acct?.homeCity });
          remote.messages.push({ id: uid('c-'), authorId: 'system', kind: 'system', text: `${acct?.name ?? 'Someone'} joined the trip
👋`, ts: Date.now() });
          s.trips.unshift(remote);
          s.activeTripId = remote.id;
        });
        return { ok: true, tripId: remote.id };
      },

      leaveTrip: (tripId) =>
        set((s) => {
          const t = s.trips.find((x) => x.id === tripId);
          if (!t) return;
          t.members = t.members.filter((m) => m.id !== s.session?.userId);
          if (s.activeTripId === tripId) {
            s.activeTripId = s.trips.find((x) => x.id !== tripId && x.members.some((m) => m.id === s.session?.userId))?.id ?? null;
```

```
      }
    }),

  deleteTrip: (tripId) =>
    set((s) => {
      s.trips = s.trips.filter((t) => t.id !== tripId);
      if (s.activeTripId === tripId) {
        s.activeTripId = s.trips.find((x) => x.members.some((m) => m.id === s.session?.userId))?.id ?? null;
      }
    }),

  updateTrip: (tripId, patch) => withTrip(tripId, (t) => Object.assign(t, patch)),

  regenerateInvite: (tripId) =>
    withTrip(tripId, (t) => {
      t.inviteCode = inviteCodeFor(destinationById(t.destinationId)?.city ?? t.name);
    }),

  reseedDemo: (timeline) =>
    set((s) => {
      const fresh = buildDemoTrip(timeline);
      s.trips = [fresh, ...s.trips.filter((t) => t.id !== fresh.id)];
      s.activeTripId = fresh.id;
      s.demoTimeline = timeline;
      s.firedReminders = {};
    }),

  addMember: (tripId, name, email) =>
    withTrip(tripId, (t) => {
      t.members.push({ id: uid('m-'), name: name.trim(), email: email?.trim() || undefined, role: 'Member', invited: true });
      t.messages.push({ id: uid('c-'), authorId: 'system', kind: 'system', text: `${name.trim()} was added to the trip`, ts: Date.
now() });
    }),

  removeMember: (tripId, memberId) =>
    withTrip(tripId, (t) => {
      t.members = t.members.filter((m) => m.id !== memberId);
      t.activities.forEach((a) => {
        a.assigned = a.assigned.filter((x) => x !== memberId);
        delete a.votes[memberId];
      });
      t.tasks.forEach((k) => {
        if (k.assignee === memberId) k.assignee = undefined;
      });
    }),

  setRole: (tripId, memberId, role) =>
    withTrip(tripId, (t) => {
      const m = t.members.find((x) => x.id === memberId);
      if (m) m.role = role;
    }),

  addActivity: (tripId, a) => {
    const id = uid('a-');
    withTrip(tripId, (t) => {
      t.activities.push({ ...a, votes: a.votes ?? { [me()]: 1 }, id, createdBy: me() });
    });
    return id;
  },

  updateActivity: (tripId, id, patch) =>
    withTrip(tripId, (t) => {
      const a = t.activities.find((x) => x.id === id);
      if (a) Object.assign(a, patch);
    }),

  deleteActivity: (tripId, id) =>
    withTrip(tripId, (t) => {
      t.activities = t.activities.filter((x) => x.id !== id);
    }),

  voteActivity: (tripId, id, value) =>
    withTrip(tripId, (t) => {
      const a = t.activities.find((x) => x.id === id);
      if (!a) return;
      const user = me();
      if (a.votes[user] === value) delete a.votes[user];
      else a.votes[user] = value;
      // Majority rule: a voting item is confirmed once more than half the group upvotes it.
      const ups = Object.values(a.votes).filter((v) => v === 1).length;
      if (a.status === 'voting' && ups > t.members.length / 2) a.status = 'confirmed';
    }),

  toggleAssign: (tripId, id, memberId) =>
    withTrip(tripId, (t) => {
      const a = t.activities.find((x) => x.id === id);
```

```
        if (!a) return;
        a.assigned = a.assigned.includes(memberId) ? a.assigned.filter((x) => x !== memberId) : [...a.assigned, memberId];
      }),

    addExpense: (tripId, e) =>
      withTrip(tripId, (t) => {
        t.expenses.unshift({ ...e, id: uid('e-') });
      }),

    deleteExpense: (tripId, id) =>
      withTrip(tripId, (t) => {
        t.expenses = t.expenses.filter((x) => x.id !== id);
      }),

    settleUp: (tripId, from, to, amount) =>
      withTrip(tripId, (t) => {
        const toName = t.members.find((m) => m.id === to)?.name.split(' ')[0] ?? 'member';
        t.expenses.unshift({
          id: uid('e-'), title: `Settle-up to ${toName}`, amount: Math.round(amount * 100) / 100, category: 'Other',
          paidBy: from, splitBetween: [to], date: todayInTz('UTC'), settlement: true,
        });
      }),

    addFlight: (tripId, f) =>
      withTrip(tripId, (t) => {
        t.flights.push({ ...f, id: uid('f-') });
      }),

    deleteFlight: (tripId, id) =>
      withTrip(tripId, (t) => {
        t.flights = t.flights.filter((x) => x.id !== id);
      }),

    createPoll: (tripId, question, options) =>
      withTrip(tripId, (t) => {
        t.polls.unshift({
          id: uid('poll-'), question: question.trim(), createdBy: me(), createdAt: Date.now(),
          options: options.map((text, i) => ({ id: `o${i + 1}`, text: text.trim(), votes: [] })),
        } satisfies Poll);
        t.messages.push({ id: uid('c-'), authorId: 'system', kind: 'system', text: `New poll: ${question.trim()}`, ts: Date.now()
});
      }),

    votePoll: (tripId, pollId, optionId) =>
      withTrip(tripId, (t) => {
        const p = t.polls.find((x) => x.id === pollId);
        if (!p || p.closed) return;
        const user = me();
        const already = p.options.find((o) => o.id === optionId)?.votes.includes(user);
        p.options.forEach((o) => (o.votes = o.votes.filter((v) => v !== user)));
        if (!already) p.options.find((o) => o.id === optionId)?.votes.push(user);
      }),

    closePoll: (tripId, pollId) =>
      withTrip(tripId, (t) => {
        const p = t.polls.find((x) => x.id === pollId);
        if (p) p.closed = !p.closed;
      }),

    deletePoll: (tripId, pollId) =>
      withTrip(tripId, (t) => {
        t.polls = t.polls.filter((x) => x.id !== pollId);
      }),

    addTask: (tripId, text, assignee) =>
      withTrip(tripId, (t) => {
        t.tasks.push({ id: uid('t-'), text: text.trim(), done: false, assignee });
      }),

    toggleTask: (tripId, id) =>
      withTrip(tripId, (t) => {
        const k = t.tasks.find((x) => x.id === id);
        if (k) k.done = !k.done;
      }),

    deleteTask: (tripId, id) =>
      withTrip(tripId, (t) => {
        t.tasks = t.tasks.filter((x) => x.id !== id);
      }),

    assignTask: (tripId, id, memberId) =>
      withTrip(tripId, (t) => {
        const k = t.tasks.find((x) => x.id === id);
        if (k) k.assignee = memberId;
      }),
```

```
  addDoc: (tripId, d) =>
    withTrip(tripId, (t) => {
      t.docs.unshift({ ...d, id: uid('d-'), addedAt: Date.now(), addedBy: me() });
    }),

  deleteDoc: (tripId, id) =>
    withTrip(tripId, (t) => {
      t.docs = t.docs.filter((x) => x.id !== id);
    }),

  sendMessage: (tripId, msg) =>
    withTrip(tripId, (t) => {
      t.messages.push({ ...msg, id: uid('c-'), ts: msg.ts ?? Date.now() });
      if (t.messages.length > 400) t.messages.splice(0, t.messages.length - 400);
    }),

  clearChat: (tripId) =>
    withTrip(tripId, (t) => {
      t.messages = [];
    }),

  addPhoto: (tripId, p) =>
    withTrip(tripId, (t) => {
      t.photos.unshift({ ...p, id: uid('ph-'), ts: Date.now(), likes: [], by: me() });
    }),

  toggleLikePhoto: (tripId, id) =>
    withTrip(tripId, (t) => {
      const p = t.photos.find((x) => x.id === id);
      if (!p) return;
      const user = me();
      p.likes = p.likes.includes(user) ? p.likes.filter((x) => x !== user) : [...p.likes, user];
    }),

  deletePhoto: (tripId, id) =>
    withTrip(tripId, (t) => {
      t.photos = t.photos.filter((x) => x.id !== id);
    }),

  togglePacked: (tripId, id) =>
    withTrip(tripId, (t) => {
      const p = t.packing.find((x) => x.id === id);
      if (p) p.packed = !p.packed;
    }),

  assignPacking: (tripId, id, memberId) =>
    withTrip(tripId, (t) => {
      const p = t.packing.find((x) => x.id === id);
      if (p) p.broughtBy = memberId;
    }),

  addPackingItems: (tripId, items) =>
    withTrip(tripId, (t) => {
      const have = new Set(t.packing.map((p) => p.text.toLowerCase()));
      items.forEach((i) => {
        if (!have.has(i.text.toLowerCase())) t.packing.push({ ...i, id: uid('p-'), packed: false });
      });
    }),

  deletePackingItem: (tripId, id) =>
    withTrip(tripId, (t) => {
      t.packing = t.packing.filter((x) => x.id !== id);
    }),

  addBucket: (text, destinationId) =>
    set((s) => {
      s.bucket.unshift({ id: uid('b-'), text: text.trim(), destinationId, done: false, addedAt: Date.now() });
    }),

  toggleBucket: (id) =>
    set((s) => {
      const b = s.bucket.find((x) => x.id === id);
      if (b) b.done = !b.done;
    }),

  deleteBucket: (id) =>
    set((s) => {
      s.bucket = s.bucket.filter((x) => x.id !== id);
    }),

  toggleFavorite: (destinationId) =>
    set((s) => {
      s.favorites = s.favorites.includes(destinationId)
        ? s.favorites.filter((x) => x !== destinationId)
        : [...s.favorites, destinationId];
    }),
```

```
        pushNotification: (n) =>
          set((s) => {
            s.notifications.unshift({ ...n, id: uid('n-'), ts: Date.now(), read: false });
            if (s.notifications.length > 60) s.notifications.length = 60;
          }),

        markAllRead: () =>
          set((s) => {
            s.notifications.forEach((n) => (n.read = true));
          }),

        markKindRead: (kind) =>
          set((s) => {
            s.notifications.forEach((n) => {
              if (n.kind === kind) n.read = true;
            });
          }),

        clearNotifications: () => set({ notifications: [] }),

        updateSettings: (patch) =>
          set((s) => {
            Object.assign(s.settings, patch);
          }),

        setWeather: (w) =>
          set((s) => {
            s.weather[w.destinationId] = w;
          }),

        setRates: (r) => set({ rates: r }),
        setQuizBest: (n) => set((s) => ({ quizBest: Math.max(s.quizBest, n) })),
        setSurveyResult: (ids) => set({ surveyResult: ids }),
        setRestyled: (v) => set({ restyled: v }),
        setToured: (v) =>
          set((s) => {
            s.personal.toured = v;
          }),
        updatePersonal: (patch) =>
          set((s) => {
            Object.assign(s.personal, patch);
          }),

        markReminderFired: (key) =>
          set((s) => {
            s.firedReminders[key] = Date.now();
          }),

        resetEverything: () =>
          set(() => ({ ...initialState, hydrated: true, hasOnboarded: true })),
      };
    }),
    {
      name: 'togetherwego-v1',
      version: 2,
      // v2: "Amalfi Squad" replaced the Tokyo trip as the main demo — re-seed the demo trips once, keep the user's own.
      migrate: (persisted, version) => {
        const old = persisted as Partial<State>;
        if (version >= 2) return old as AppStore;
        const demoIds = ['trip-tokyo', 'trip-bali', 'trip-bcn', 'trip-amalfi'];
        return { ...old, seeded: false, activeTripId: null, trips: (old.trips ?? []).filter((t) => !demoIds.includes(t.id)) } as AppStor
e;
      },
      // New settings added in later versions fall back to their defaults instead of vanishing.
      merge: (persisted, current) => {
        const p = (persisted ?? {}) as Partial<State>;
        return { ...current, ...p, personal: { ...current.personal, ...p.personal, sections: { ...current.personal.sections, ...p.person
al?.sections } } } as AppStore;
      },
      storage: createJSONStorage(() => AsyncStorage),
      partialize: (s) => {
        // Don't persist the session unless "Remember me" was ticked.
        const { hydrated, ...rest } = s;
        void hydrated;
        return { ...rest, session: s.session?.remember || s.restyled ? s.session : null };
      },
      onRehydrateStorage: () => (state) => {
        state?.setHydrated();
      },
    },
  ),
);

// ---------- Selectors & derived data (the "view-model" helpers) ----------
```

```
export function useCurrentUser(): Account | undefined {
  return useAppStore((s) => s.accounts.find((a) => a.id === s.session?.userId));
}

export function useActiveTrip(): Trip | undefined {
  return useAppStore((s) => {
    const uidNow = s.session?.userId;
    const t = s.trips.find((x) => x.id === s.activeTripId);
    if (t && (!uidNow || t.members.some((m) => m.id === uidNow))) return t;
    return undefined;
  });
}

export function useMyTrips(): Trip[] {
  const userId = useAppStore((s) => s.session?.userId);
  const trips = useAppStore((s) => s.trips);
  return trips.filter((t) => t.members.some((m) => m.id === userId));
}

export function tripStatus(t: Trip): TripStatus {
  if (t.completed) return 'completed';
  const dest = destinationById(t.destinationId);
  const today = todayInTz(dest?.tz ?? 'UTC');
  if (daysBetween(today, t.startDate) > 0) return 'planning';
  if (daysBetween(t.endDate, today) > 0) return 'completed';
  return 'active';
}

export function memberById(t: Trip | undefined, id: string): Member | undefined {
  return t?.members.find((m) => m.id === id);
}

export function totalSpent(t: Trip): number {
  return t.expenses.filter((e) => !e.settlement).reduce((sum, e) => sum + e.amount, 0);
}

/** Net balance per member: positive = is owed money, negative = owes. */
export function balances(t: Trip): Record<string, number> {
  const bal: Record<string, number> = {};
  t.members.forEach((m) => (bal[m.id] = 0));
  for (const e of t.expenses) {
    const split = e.splitBetween.filter((id) => id in bal);
    if (!split.length) continue;
    bal[e.paidBy] = (bal[e.paidBy] ?? 0) + e.amount;
    const share = e.amount / split.length;
    split.forEach((id) => (bal[id] -= share));
  }
  return bal;
}

/** Greedy minimal-transfer settle-up plan. */
export function settlePlan(t: Trip): { from: string; to: string; amount: number }[] {
  const bal = balances(t);
  const debtors = Object.entries(bal).filter(([, v]) => v < -0.5).map(([id, v]) => ({ id, v: -v })).sort((a, b) => b.v - a.v);
  const creditors = Object.entries(bal).filter(([, v]) => v > 0.5).map(([id, v]) => ({ id, v })).sort((a, b) => b.v - a.v);
  const plan: { from: string; to: string; amount: number }[] = [];
  let i = 0;
  let j = 0;
  while (i < debtors.length && j < creditors.length) {
    const amt = Math.min(debtors[i].v, creditors[j].v);
    plan.push({ from: debtors[i].id, to: creditors[j].id, amount: Math.round(amt * 100) / 100 });
    debtors[i].v -= amt;
    creditors[j].v -= amt;
    if (debtors[i].v < 0.5) i++;
    if (creditors[j].v < 0.5) j++;
  }
  return plan;
}
```

src/theme/index.ts

```
/**
 * TogetherWeGo design tokens.
 * The default "Forest" palette is sampled from the reference design: warm cream
 * canvas, deep forest green for primary actions, soft sage for highlights.
 *
 * Users can pick another palette and corner style in "Make it yours"
 * (src/app/personalize.tsx). Styles are created once when each module loads,
 * so the saved choice is read synchronously here — before any screen module
 * evaluates — and applied by mutating these token objects in place. Changing
 * the theme then reloads the app (see applyAppearance).
 */
import { File, Paths } from 'expo-file-system';
import { DevSettings, Platform } from 'react-native';

export const colors = {
  bg: '#F4F4EE',
  bgAlt: '#EEF0E7',
  card: '#FFFFFF',
  cardMuted: '#F7F8F3',
  border: '#E7E9DF',
  borderStrong: '#D5D9CC',

  primary: '#1F4D25',
  primaryDark: '#163A1B',
  primaryPressed: '#173E1C',
  primarySoft: '#DCEFD3',
  primarySofter: '#EAF5E4',
  primaryLine: '#C3DFB6',
  navActive: '#CFE6C4',

  /** Pop colour for highlights, badges and the animated hero. */
  accent: '#F2B33D',
  accentSoft: '#FDF3DC',
  heroA: '#1F4D25',
  heroB: '#2F7A3A',
  heroC: '#8CC97A',

  text: '#1A1C17',
  textSecondary: '#5F645A',
  textMuted: '#959A8E',
  onPrimary: '#FFFFFF',
  /** Text on the yellow accent. */
  onAccent: '#111111',
  /** Money coming back / good news. */
  positive: '#1E9E57',

  orange: '#E8833A',
  orangeSoft: '#FDEBDD',
  blue: '#3D7DD8',
  blueSoft: '#E3ECFB',
  yellow: '#F2B33D',
  yellowSoft: '#FDF3DC',
  red: '#D64545',
  redSoft: '#FBE4E4',
  purple: '#7C5CD6',
  purpleSoft: '#EEE8FB',
  teal: '#2A9D8F',
  tealSoft: '#DDF3F0',
  star: '#F5B301',

  overlay: 'rgba(16, 24, 14, 0.45)',
  white: '#FFFFFF',
  black: '#000000',
};

type Palette = Partial<typeof colors>;
export type ThemeKey = 'amalfi' | 'forest' | 'ocean' | 'sunset' | 'lavender' | 'sakura' | 'midnight';
export type CornerStyle = 'rounded' | 'bubbly' | 'sharp';

/** Every palette the user can choose from in "Make it yours". */
export const themes: Record<
  ThemeKey,
  {
    name: string;
    emoji: string;
    vibe: string;
    dark?: boolean;
    palette: Palette;
  }
> = {
  amalfi: {
    name: 'Amalfi',
    emoji: '🍋',
    vibe: 'Bold black, sunny yellow',
```

```
    palette: {
      bg: '#F1F2F4', bgAlt: '#E8E9ED', cardMuted: '#F5F6F8', border: '#E6E7EC', borderStrong: '#D2D4DB',
      primary: '#0B0B0D', primaryDark: '#000000', primaryPressed: '#1B1B1F', primarySoft: '#E9EAEE', primarySofter: '#F3F4F6', primaryLi
ne: '#D5D7DE', navActive: 'rgba(0,0,0,0)',
      accent: '#FBC91B', accentSoft: '#FEF3C7', heroA: '#0B0B0D', heroB: '#0B0B0D', heroC: '#FBC91B',
      text: '#0D0D10', textSecondary: '#666973', textMuted: '#9A9DA8',
      blue: '#1E73E8', blueSoft: '#E1EDFD',
    },
  },
  forest: {
    name: 'Forest',
    emoji: '🌲',
    vibe: 'Calm, earthy, classic',
    palette: {},
  },
  ocean: {
    name: 'Ocean',
    emoji: '🌊',
    vibe: 'Fresh blues & sea breeze',
    palette: {
      bg: '#F1F6FA',
      bgAlt: '#E6EFF6',
      cardMuted: '#F5F9FC',
      border: '#DFE8EF',
      borderStrong: '#C9D7E3',
      primary: '#0F4C81',
      primaryDark: '#0A3A63',
      primaryPressed: '#0C406D',
      primarySoft: '#D5E6F5',
      primarySofter: '#E8F1FA',
      primaryLine: '#B3D0EA',
      navActive: '#CFE2F4',
      accent: '#FF8A5B',
      accentSoft: '#FFE9DF',
      heroA: '#0A3A63',
      heroB: '#1F78B4',
      heroC: '#5CC8D6',
      text: '#14202B',
      textSecondary: '#566573',
      textMuted: '#8F9CA8',
    },
  },
  sunset: {
    name: 'Sunset',
    emoji: '🌅',
    vibe: 'Warm oranges & golden hour',
    palette: {
      bg: '#FFF6EE',
      bgAlt: '#FCEBDD',
      cardMuted: '#FFF9F4',
      border: '#F3E2D3',
      borderStrong: '#E8CDB6',
      primary: '#C2410C',
      primaryDark: '#9A3412',
      primaryPressed: '#A8390B',
      primarySoft: '#FDDCC6',
      primarySofter: '#FFEDE1',
      primaryLine: '#F8C3A0',
      navActive: '#FCD6BD',
      accent: '#7C3AED',
      accentSoft: '#EDE4FD',
      heroA: '#9A3412',
      heroB: '#EA580C',
      heroC: '#FBBF24',
      text: '#2A1A12',
      textSecondary: '#6E5A4E',
      textMuted: '#A8968A',
    },
  },
  lavender: {
    name: 'Lavender',
    emoji: '💜',
    vibe: 'Dreamy purples & soft pastels',
    palette: {
      bg: '#F7F4FC',
      bgAlt: '#EEE9F8',
      cardMuted: '#FAF8FD',
      border: '#E7E1F2',
      borderStrong: '#D3C9E8',
      primary: '#5B3FA8',
      primaryDark: '#462F85',
      primaryPressed: '#4E3693',
      primarySoft: '#E4DBF7',
      primarySofter: '#F0EBFB',
      primaryLine: '#CDBDF0',
      navActive: '#DED2F6',
```

```
      accent: '#EC4899',
      accentSoft: '#FCE4F1',
      heroA: '#462F85',
      heroB: '#7C5CD6',
      heroC: '#F0A6CA',
      text: '#1E1A2B',
      textSecondary: '#625B73',
      textMuted: '#9A94A8',
    },
  },
  sakura: {
    name: 'Sakura',
    emoji: '🌸',
    vibe: 'Cherry-blossom pink & cream',
    palette: {
      bg: '#FFF5F7',
      bgAlt: '#FCE9EE',
      cardMuted: '#FFF9FA',
      border: '#F5DFE5',
      borderStrong: '#EBC8D2',
      primary: '#B83268',
      primaryDark: '#912451',
      primaryPressed: '#A12A5B',
      primarySoft: '#FAD7E4',
      primarySofter: '#FDEAF1',
      primaryLine: '#F2B6CC',
      navActive: '#F8D0DF',
      accent: '#14B8A6',
      accentSoft: '#D9F6F2',
      heroA: '#912451',
      heroB: '#D94F86',
      heroC: '#FFB4C8',
      text: '#2B1820',
      textSecondary: '#735A64',
      textMuted: '#A8939B',
    },
  },
  midnight: {
    name: 'Midnight',
    emoji: '🌙',
    vibe: 'Dark mode with neon green',
    dark: true,
    palette: {
      bg: '#0E1513',
      bgAlt: '#16201D',
      card: '#18221F',
      cardMuted: '#1C2724',
      border: '#26332F',
      borderStrong: '#34443F',
      primary: '#2FAE66',
      primaryDark: '#238A50',
      primaryPressed: '#28995A',
      primarySoft: '#1E3A2C',
      primarySofter: '#1A2E25',
      primaryLine: '#2D5240',
      navActive: '#21402F',
      accent: '#F5C451',
      accentSoft: '#3A3220',
      heroA: '#0B2E1E',
      heroB: '#16603C',
      heroC: '#2FAE66',
      text: '#ECF2EE',
      textSecondary: '#A9B8B1',
      textMuted: '#74857E',
      orangeSoft: '#3B2A1C',
      blueSoft: '#1C2A3D',
      yellowSoft: '#3A3220',
      redSoft: '#3D1F21',
      purpleSoft: '#2A2340',
      tealSoft: '#173431',
      overlay: 'rgba(0, 0, 0, 0.6)',
      positive: '#4ADE80', onAccent: '#111111',
    },
  },
};

export const cornerStyles: Record<CornerStyle, { name: string; scale: number }> = {
  rounded: { name: 'Rounded', scale: 1 },
  bubbly: { name: 'Bubbly', scale: 1.4 },
  sharp: { name: 'Crisp', scale: 0.45 },
};

export interface Appearance {
  theme: ThemeKey;
  corners: CornerStyle;
}
```

```
const PREFS_KEY = 'togetherwego-appearance';
const DEFAULT_APPEARANCE: Appearance = { theme: 'amalfi', corners: 'rounded' };

function prefsFile() {
  return new File(Paths.document, 'appearance.json');
}

function readAppearance(): Appearance {
  try {
    let raw: string | null = null;
    if (Platform.OS === 'web') raw = typeof localStorage !== 'undefined' ? localStorage.getItem(PREFS_KEY) : null;
    else {
      const f = prefsFile();
      raw = f.exists ? f.textSync() : null;
    }
    const parsed = raw ? (JSON.parse(raw) as Partial<Appearance>) : {};
    return {
      theme: parsed.theme && parsed.theme in themes ? parsed.theme : DEFAULT_APPEARANCE.theme,
      corners: parsed.corners && parsed.corners in cornerStyles ? parsed.corners : DEFAULT_APPEARANCE.corners,
    };
  } catch {
    return DEFAULT_APPEARANCE;
  }
}

function writeAppearance(a: Appearance) {
  try {
    const raw = JSON.stringify(a);
    if (Platform.OS === 'web') localStorage.setItem(PREFS_KEY, raw);
    else prefsFile().write(raw);
  } catch {
    // Storage unavailable (private browsing etc.) — the choice lasts for this session only.
  }
}

/** The appearance the app is currently drawn with (fixed until the next reload). */
export const appearance: Appearance = readAppearance();
export const isDark = !!themes[appearance.theme].dark;
/** The untouched Forest tokens (for previews of other palettes). */
export const baseColors = { ...colors };
Object.assign(colors, themes[appearance.theme].palette);

/** Saves a new look and restarts the UI so every screen picks it up. */
export function applyAppearance(next: Appearance): boolean {
  writeAppearance(next);
  if (next.theme === appearance.theme && next.corners === appearance.corners) return true;
  try {
    if (Platform.OS === 'web') {
      const home = (globalThis as { __TWG_HOME?: string }).__TWG_HOME;
      // Restart from the app's front door (not the survey page we're on).
      window.location.replace(home ?? `${window.location.origin}/`);
    } else DevSettings.reload('Applying your new look');
    return true;
  } catch {
    return false;
  }
}

/** Category colours used by the spending breakdown and expense dots. */
export const categoryColors: Record<string, string> = {
  Flights: '#6C4CE0',
  Stays: colors.text,
  Food: '#FBC91B',
  Activities: '#E8833A',
  Transport: '#1E73E8',
  Shopping: '#D94F86',
  Other: '#9A9DA8',
};

const cornerScale = cornerStyles[appearance.corners].scale;
const r = (n: number) => Math.round(n * cornerScale);
export const radius = {
  xs: r(8),
  sm: r(12),
  md: r(16),
  lg: r(20),
  xl: r(24),
  pill: 999,
};

export const space = {
  xxs: 2,
  xs: 4,
  sm: 8,
  md: 12,
```

```
  lg: 16,
  xl: 20,
  xxl: 24,
  xxxl: 32,
} as const;

export const fonts = {
  regular: 'Manrope_400Regular',
  medium: 'Manrope_500Medium',
  semibold: 'Manrope_600SemiBold',
  bold: 'Manrope_700Bold',
  extrabold: 'Manrope_800ExtraBold',
  /** Tall condensed headline face (THE PLAN, BUDGET…). */
  display: 'Anton_400Regular',
} as const;

export const shadow = Platform.select({
  web: { boxShadow: '0px 6px 18px rgba(30, 50, 20, 0.07)' },
  default: {
    shadowColor: '#1E3214',
    shadowOpacity: 0.07,
    shadowRadius: 12,
    shadowOffset: { width: 0, height: 6 },
    elevation: 2,
  },
}) as object;

export const shadowStrong = Platform.select({
  web: { boxShadow: '0px 10px 28px rgba(20, 45, 15, 0.18)' },
  default: {
    shadowColor: '#14280F',
    shadowOpacity: 0.18,
    shadowRadius: 18,
    shadowOffset: { width: 0, height: 10 },
    elevation: 6,
  },
}) as object;

/** Width of the app column on tablets / web so layouts stay phone-shaped. */
export const MAX_WIDTH = 520;
```

src/utils/format.ts

```
import { currencyByCode } from '@/data/reference';

export function uid(prefix = ''): string {
  return `${prefix}${Date.now().toString(36)}${Math.random().toString(36).slice(2, 8)}`;
}

/** Formats money with the right symbol and sensible decimals for the currency. */
export function money(amount: number, code = 'USD', opts: { decimals?: boolean } = {}): string {
  const cur = currencyByCode(code);
  const bigUnit = ['JPY', 'KRW', 'IDR', 'VND', 'ISK'].includes(code);
  const decimals = opts.decimals && !bigUnit ? 2 : 0;
  const n = Math.abs(amount).toLocaleString('en-US', {
    minimumFractionDigits: decimals,
    maximumFractionDigits: decimals,
  });
  const sign = amount < 0 ? '−' : '';
  const sym = cur?.symbol ?? `${code} `;
  return sym.length > 2 ? `${sign}${n} ${code}` : `${sign}${sym}${n}`;
}

export function initials(name: string): string {
  const parts = name.trim().split(/\s+/).filter(Boolean);
  if (!parts.length) return '?';
  if (parts.length === 1) return parts[0].slice(0, 2).toUpperCase();
  return (parts[0][0] + parts[parts.length - 1][0]).toUpperCase();
}

export function pluralize(n: number, word: string, plural = `${word}s`) {
  return `${n} ${n === 1 ? word : plural}`;
}

export function durationLabel(min: number): string {
  if (min < 60) return `${min}m`;
  const h = Math.floor(min / 60);
  const m = min % 60;
  return m ? `${h}h ${m}m` : `${h}h`;
}

export type LengthFilter = 'any' | 'u1' | '1-2' | '2-4' | 'half' | 'full';

export const LENGTH_FILTERS: { key: LengthFilter; labelKey: string }[] = [
  { key: 'any', labelKey: 'len.any' },
  { key: 'u1', labelKey: 'len.u1' },
  { key: '1-2', labelKey: 'len.1-2' },
  { key: '2-4', labelKey: 'len.2-4' },
  { key: 'half', labelKey: 'len.half' },
  { key: 'full', labelKey: 'len.full' },
];

/** Buckets match the brief: under 1h, 1–2h, 2–4h, half day (4–6h), full day (6h+). */
export function matchesLength(min: number, f: LengthFilter): boolean {
  switch (f) {
    case 'any':
      return true;
    case 'u1':
      return min < 60;
    case '1-2':
      return min >= 60 && min <= 120;
    case '2-4':
      return min > 120 && min <= 240;
    case 'half':
      return min > 240 && min <= 360;
    case 'full':
      return min > 360;
  }
}

export function lengthBucketLabel(min: number): string {
  if (min < 60) return 'Under 1 hour';
  if (min <= 120) return '1–2 hours';
  if (min <= 240) return '2–4 hours';
  if (min <= 360) return 'Half day';
  return 'Full day';
}

/** Short tag for activity cards: <1h, 1-2h, 2-4h, Half day, Full day. */
export function lengthTag(min: number): string {
  if (min < 60) return '<1h';
  if (min <= 120) return '1-2h';
  if (min <= 240) return '2-4h';
  if (min <= 360) return 'Half day';
  return 'Full day';
}
```

```
export function priceTier(cost: number): string {
  if (cost === 0) return 'Free';
  if (cost < 15) return '$';
  if (cost < 40) return '$$';
  return '$$$';
}

export function timeAgo(ts: number, now = Date.now()): string {
  const s = Math.max(0, Math.round((now - ts) / 1000));
  if (s < 45) return 'just now';
  const m = Math.round(s / 60);
  if (m < 60) return `${m}m ago`;
  const h = Math.round(m / 60);
  if (h < 24) return `${h}h ago`;
  const d = Math.round(h / 24);
  return `${d}d ago`;
}

export function countdownLabel(ms: number): string {
  if (ms <= 0) return 'now';
  const totalMin = Math.round(ms / 60000);
  const d = Math.floor(totalMin / 1440);
  const h = Math.floor((totalMin % 1440) / 60);
  const m = totalMin % 60;
  if (d > 0) return `${d}d ${h}h`;
  if (h > 0) return `${h}h ${m}m`;
  return `${m}m`;
}
```

src/utils/geo.ts

```
import { airports, type Airport } from '@/data/airports';

export interface LatLng {
  lat: number;
  lng: number;
}

const R = 6371;
const rad = (d: number) => (d * Math.PI) / 180;

export function distanceKm(a: LatLng, b: LatLng): number {
  const dLat = rad(b.lat - a.lat);
  const dLng = rad(b.lng - a.lng);
  const h = Math.sin(dLat / 2) ** 2 + Math.cos(rad(a.lat)) * Math.cos(rad(b.lat)) * Math.sin(dLng / 2) ** 2;
  return 2 * R * Math.asin(Math.sqrt(h));
}

export function bearingDeg(a: LatLng, b: LatLng): number {
  const y = Math.sin(rad(b.lng - a.lng)) * Math.cos(rad(b.lat));
  const x = Math.cos(rad(a.lat)) * Math.sin(rad(b.lat)) - Math.sin(rad(a.lat)) * Math.cos(rad(b.lat)) * Math.cos(rad(b.lng - a.lng));
  return ((Math.atan2(y, x) * 180) / Math.PI + 360) % 360;
}

export function compassPoint(deg: number): string {
  const pts = ['N', 'NE', 'E', 'SE', 'S', 'SW', 'W', 'NW'];
  return pts[Math.round(deg / 45) % 8];
}

export function kmLabel(km: number): string {
  if (km < 1) return `${Math.round(km * 1000)} m`;
  if (km < 10) return `${km.toFixed(1)} km`;
  return `${Math.round(km)} km`;
}

/** Rough door-to-door estimate for city driving / airport transfer. */
export function driveMinutes(km: number): number {
  return Math.max(5, Math.round((km / 38) * 60 + 8));
}

export function nearestAirports(from: LatLng, count = 5): (Airport & { km: number })[] {
  return airports
    .map((a) => ({ ...a, km: distanceKm(from, a) }))
    .sort((x, y) => x.km - y.km)
    .slice(0, count);
}

/** Projects a point into x/y kilometres east/north of `center` (equirectangular, fine for < 50 km). */
export function projectKm(center: LatLng, p: LatLng) {
  return {
    x: (p.lng - center.lng) * 111.32 * Math.cos(rad(center.lat)),
    y: (p.lat - center.lat) * 110.57,
  };
}
```

src/utils/time.ts

```
/**
 * Date & time-zone helpers. Trip dates are stored as plain 'YYYY-MM-DD' strings
 * in the destination's local time, flights as absolute UTC timestamps. Uses Intl
 * when available and falls back to a fixed-offset table.
 */

const FALLBACK_OFFSETS: Record<string, number> = {
  'Asia/Tokyo': 540, 'Asia/Seoul': 540, 'Asia/Shanghai': 480, 'Asia/Hong_Kong': 480, 'Asia/Taipei': 480,
  'Asia/Singapore': 480, 'Asia/Makassar': 480, 'Asia/Kuala_Lumpur': 480, 'Asia/Manila': 480,
  'Asia/Bangkok': 420, 'Asia/Jakarta': 420, 'Asia/Ho_Chi_Minh': 420, 'Asia/Kolkata': 330,
  'Indian/Maldives': 300, 'Asia/Dubai': 240, 'Asia/Qatar': 180, 'Europe/Istanbul': 180,
  'Europe/London': 0, 'Europe/Dublin': 0, 'Europe/Lisbon': 0, 'Atlantic/Reykjavik': 0,
  'Europe/Paris': 60, 'Europe/Rome': 60, 'Europe/Madrid': 60, 'Europe/Berlin': 60, 'Europe/Amsterdam': 60,
  'Europe/Zurich': 60, 'Europe/Prague': 60, 'Europe/Vienna': 60, 'Africa/Casablanca': 60,
  'Europe/Athens': 120, 'Africa/Johannesburg': 120, 'Africa/Cairo': 120, 'Africa/Nairobi': 180,
  'Australia/Sydney': 600, 'Australia/Melbourne': 600, 'Pacific/Auckland': 720,
  'America/New_York': -300, 'America/Toronto': -300, 'America/Detroit': -300, 'America/Chicago': -360,
  'America/Denver': -420, 'America/Edmonton': -420, 'America/Phoenix': -420,
  'America/Los_Angeles': -480, 'America/Vancouver': -480, 'Pacific/Honolulu': -600,
  'America/Mexico_City': -360, 'America/Cancun': -300, 'America/Lima': -300, 'America/Bogota': -300,
  'America/Sao_Paulo': -180, 'America/Argentina/Buenos_Aires': -180, 'America/Santiago': -240, UTC: 0,
};

const MONTHS = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
const WEEKDAYS = ['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat'];

export const YMD_RE = /^\d{4}-\d{2}-\d{2}$/;
export const HHMM_RE = /^([01]\d|2[0-3]):([0-5]\d)$/;

export function parseYMD(ymd: string) {
  const [y, m, d] = ymd.split('-').map(Number);
  return { y, m, d };
}

export function ymdFromUtcDate(date: Date): string {
  return `${date.getUTCFullYear()}-${String(date.getUTCMonth() + 1).padStart(2, '0')}-${String(date.getUTCDate()).padStart(2, '0')}`;
}

export function isValidYMD(ymd: string): boolean {
  if (!YMD_RE.test(ymd)) return false;
  const { y, m, d } = parseYMD(ymd);
  const dt = new Date(Date.UTC(y, m - 1, d));
  return dt.getUTCFullYear() === y && dt.getUTCMonth() === m - 1 && dt.getUTCDate() === d;
}

export function addDays(ymd: string, n: number): string {
  const { y, m, d } = parseYMD(ymd);
  return ymdFromUtcDate(new Date(Date.UTC(y, m - 1, d + n)));
}

export function daysBetween(a: string, b: string): number {
  const pa = parseYMD(a);
  const pb = parseYMD(b);
  return Math.round((Date.UTC(pb.y, pb.m - 1, pb.d) - Date.UTC(pa.y, pa.m - 1, pa.d)) / 86400000);
}

export function tripDays(start: string, end: string): string[] {
  const n = Math.max(0, daysBetween(start, end));
  return Array.from({ length: n + 1 }, (_, i) => addDays(start, i));
}

export function weekdayShort(ymd: string): string {
  const { y, m, d } = parseYMD(ymd);
  return WEEKDAYS[new Date(Date.UTC(y, m - 1, d)).getUTCDay()];
}

export function dayLabel(ymd: string): string {
  const { m, d } = parseYMD(ymd);
  return `${MONTHS[m - 1]} ${d}`;
}

export function rangeLabel(start: string, end: string): string {
  const a = parseYMD(start);
  const b = parseYMD(end);
  if (a.y === b.y) return `${MONTHS[a.m - 1]} ${a.d} – ${MONTHS[b.m - 1]} ${b.d}, ${b.y}`;
  return `${MONTHS[a.m - 1]} ${a.d}, ${a.y} – ${MONTHS[b.m - 1]} ${b.d}, ${b.y}`;
}

export function time12(hhmm: string): string {
  const [h, m] = hhmm.split(':').map(Number);
  const suffix = h >= 12 ? 'PM' : 'AM';
  const hr = h % 12 === 0 ? 12 : h % 12;
  return `${hr}:${String(m).padStart(2, '0')} ${suffix}`;
}
```

```
export function deviceTimeZone(): string {
  try {
    return Intl.DateTimeFormat().resolvedOptions().timeZone || 'UTC';
  } catch {
    return 'UTC';
  }
}

/** Minutes east of UTC for `tz` at the given instant. */
export function tzOffsetMinutes(tz: string, utcMs: number): number {
  try {
    const dtf = new Intl.DateTimeFormat('en-US', {
      timeZone: tz,
      hourCycle: 'h23',
      year: 'numeric',
      month: '2-digit',
      day: '2-digit',
      hour: '2-digit',
      minute: '2-digit',
      second: '2-digit',
    });
    const parts = dtf.formatToParts(new Date(utcMs));
    const get = (t: string) => Number(parts.find((p) => p.type === t)?.value);
    const asUtc = Date.UTC(get('year'), get('month') - 1, get('day'), get('hour') % 24, get('minute'), get('second'));
    const off = Math.round((asUtc - utcMs) / 60000);
    if (Number.isFinite(off)) return off;
  } catch {
    // fall through to table
  }
  if (tz === deviceTimeZone()) return -new Date(utcMs).getTimezoneOffset();
  return FALLBACK_OFFSETS[tz] ?? 0;
}

/** Converts a wall-clock time in `tz` to a UTC epoch (ms). */
export function zonedToUtc(ymd: string, hhmm: string, tz: string): number {
  const { y, m, d } = parseYMD(ymd);
  const [h, mi] = hhmm.split(':').map(Number);
  const guess = Date.UTC(y, m - 1, d, h, mi);
  const off1 = tzOffsetMinutes(tz, guess);
  let result = guess - off1 * 60000;
  const off2 = tzOffsetMinutes(tz, result);
  if (off2 !== off1) result = guess - off2 * 60000;
  return result;
}

/** Wall-clock parts of a UTC instant in a given zone. */
export function partsInTz(utcMs: number, tz: string) {
  const local = new Date(utcMs + tzOffsetMinutes(tz, utcMs) * 60000);
  return {
    ymd: ymdFromUtcDate(local),
    hhmm: `${String(local.getUTCHours()).padStart(2, '0')}:${String(local.getUTCMinutes()).padStart(2, '0')}`,
  };
}

export function formatInTz(utcMs: number, tz: string): string {
  const p = partsInTz(utcMs, tz);
  return `${dayLabel(p.ymd)} · ${p.hhmm}`;
}

export function utcOffsetLabel(tz: string, at = Date.now()): string {
  const off = tzOffsetMinutes(tz, at);
  const sign = off >= 0 ? '+' : '−';
  const h = Math.floor(Math.abs(off) / 60);
  const m = Math.abs(off) % 60;
  return `UTC${sign}${h}${m ? `:${String(m).padStart(2, '0')}` : ''}`;
}

const ABBREV: Record<string, string> = {
  'Asia/Tokyo': 'JST', 'Asia/Seoul': 'KST', 'Europe/London': 'GMT', 'Europe/Paris': 'CET',
  'America/New_York': 'ET', 'America/Chicago': 'CT', 'America/Denver': 'MT', 'America/Los_Angeles': 'PT',
  'Europe/Rome': 'CET', 'Europe/Madrid': 'CET', 'Europe/Berlin': 'CET', 'Europe/Amsterdam': 'CET', 'Europe/Prague': 'CET', 'Europe/Zuric
h': 'CET', 'Europe/Lisbon': 'WET',
  'Asia/Singapore': 'SGT', 'Asia/Bangkok': 'ICT', 'Australia/Sydney': 'AET', 'Asia/Kolkata': 'IST',
};

export function tzAbbrev(tz: string): string {
  const a = ABBREV[tz];
  if (!a) return tz.split('/').pop()?.replace(/_/g, ' ') ?? tz;
  // Summer time: Central European CET → CEST, UK GMT → BST, Western European WET → WEST.
  const summer = tzOffsetMinutes(tz, Date.now()) - tzOffsetMinutes('UTC', Date.now()) > ({ CET: 60, GMT: 0, WET: 0 }[a] ?? Infinity);
  return summer ? ({ CET: 'CEST', GMT: 'BST', WET: 'WEST' }[a] ?? a) : a;
}

/** Signed hour difference: positive when `tzA` is ahead of `tzB`. */
export function hoursAhead(tzA: string, tzB: string, at = Date.now()): number {
```

```
  return (tzOffsetMinutes(tzA, at) - tzOffsetMinutes(tzB, at)) / 60;
}

export function todayInTz(tz: string): string {
  return partsInTz(Date.now(), tz).ymd;
}

export function greetingFor(hour: number): string {
  if (hour < 5) return 'greet.night';
  if (hour < 12) return 'greet.morning';
  if (hour < 18) return 'greet.afternoon';
  return 'greet.evening';
}
```

```
/**
 * Input validation used across forms. Each validator returns an error message
 * (string) or null when the value is valid. Checks are both syntactic
 * (format) and semantic (does the value make sense for a trip?).
 */
import { airportByIata } from '@/data/airports';
import { HHMM_RE, daysBetween, isValidYMD } from './time';

export type Validator = (value: string) => string | null;

export const required =
  (label: string): Validator =>
  (v) =>
    v.trim() ? null : `Enter ${label}`;

export const maxLen =
  (n: number, label = 'This'): Validator =>
  (v) =>
    v.trim().length > n ? `${label} must be ${n} characters or fewer` : null;

export const EMAIL_RE = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;

export function validateEmail(v: string): string | null {
  if (!v.trim()) return 'Enter your email address';
  if (!EMAIL_RE.test(v.trim())) return 'That email doesn’t look right (e.g. name@mail.com)';
  return null;
}

export function validatePassword(v: string): string | null {
  if (!v) return 'Enter a password';
  if (v.length < 8) return 'Use at least 8 characters';
  if (!/[A-Za-z]/.test(v) || !/\d/.test(v)) return 'Include at least one letter and one number';
  return null;
}

export function passwordStrength(v: string): { score: 0 | 1 | 2 | 3 | 4; label: string } {
  let score = 0;
  if (v.length >= 8) score++;
  if (v.length >= 12) score++;
  if (/[A-Z]/.test(v) && /[a-z]/.test(v)) score++;
  if (/\d/.test(v) && /[^A-Za-z0-9]/.test(v)) score++;
  const labels = ['Too weak', 'Weak', 'Okay', 'Strong', 'Very strong'];
  return { score: score as 0 | 1 | 2 | 3 | 4, label: labels[score] };
}

export function validateName(v: string): string | null {
  const t = v.trim();
  if (!t) return 'Enter a name';
  if (t.length < 2) return 'Name is too short';
  if (t.length > 40) return 'Keep names under 40 characters';
  if (!/^[\p{L}\p{M}' .-]+$/u.test(t)) return 'Names can only contain letters, spaces, hyphens and apostrophes';
  return null;
}

/**
 * Parses a money amount. Accepts "12", "12.50", "1,200" — rejects negatives,
 * letters and absurd values.
 */
export function parseAmount(v: string): number | null {
  const cleaned = v.replace(/,/g, '').trim();
  if (!/^\d+(\.\d{1,2})?$/.test(cleaned)) return null;
  const n = Number(cleaned);
  return Number.isFinite(n) ? n : null;
}

export function validateAmount(v: string, { min = 0.01, max = 1_000_000, label = 'an amount' } = {}): string | null {
  if (!v.trim()) return `Enter ${label}`;
  const n = parseAmount(v);
  if (n === null) return 'Use numbers only, e.g. 25 or 25.50';
  if (n < min) return min > 0 ? `Amount must be more than 0` : 'Amount can’t be negative';
  if (n > max) return `That’s more than ${max.toLocaleString()} — double-check the amount`;
  return null;
}

export function validateTime(v: string): string | null {
  return HHMM_RE.test(v) ? null : 'Use 24-hour time, e.g. 09:30';
}

export function validateTripDates(start: string, end: string): { start?: string; end?: string } {
  const errs: { start?: string; end?: string } = {};
  if (!isValidYMD(start)) errs.start = 'Pick a valid start date';
  if (!isValidYMD(end)) errs.end = 'Pick a valid end date';
  if (!errs.start && !errs.end) {
    const len = daysBetween(start, end);
```

```
    if (len < 0) errs.end = 'End date must be on or after the start date';
    else if (len > 60) errs.end = 'Trips can be at most 60 days long';
  }
  return errs;
}

export const INVITE_RE = /^[A-Z]{3}-\d{4}$/;

export function normalizeInvite(v: string): string {
  const raw = v.toUpperCase().replace(/[^A-Z0-9]/g, '');
  if (raw.length <= 3) return raw;
  return `${raw.slice(0, 3)}-${raw.slice(3, 7)}`;
}

export function validateInvite(v: string): string | null {
  if (!v.trim()) return 'Enter the code your friend shared';
  if (!INVITE_RE.test(v)) return 'Codes look like ABC-1234';
  return null;
}

export const FLIGHT_RE = /^[A-Z0-9]{2}\s?\d{1,4}[A-Z]?$/;

export function validateFlightNumber(v: string): string | null {
  const t = v.trim().toUpperCase();
  if (!t) return 'Enter the flight number';
  if (!FLIGHT_RE.test(t)) return 'Flight numbers look like NH 175 or UA837';
  return null;
}

export function validateIata(v: string, label: string): string | null {
  const t = v.trim().toUpperCase();
  if (!t) return `Enter the ${label} airport code`;
  if (!/^[A-Z]{3}$/.test(t)) return 'Airport codes are 3 letters, e.g. JFK';
  if (!airportByIata(t)) return `We don’t recognise “${t}” — try a major airport code`;
  return null;
}

/** Runs validators in order and returns the first error. */
export function firstError(value: string, ...validators: Validator[]): string | null {
  for (const v of validators) {
    const e = v(value);
    if (e) return e;
  }
  return null;
}
```

docs/DEMO_SCRIPT.md

```
# 7-minute presentation script

Built around the rating sheet so every scored item gets shown. Times are cumulative. Split the speaking parts between teammates (all mem
bers must present).

**Before the 3-minute setup ends:** phone mirrored to the laptop, app open on the sign-in screen, *Settings → Demo timeline* set to **“S
tarts in 3 days”**, notifications allowed, volume up (for the phrasebook audio).

| Time | Show | Say (key points) | Rating-sheet item |
|---|---|---|---|
| 0:00 | Title / app icon | The problem: group trips scattered across chats, spreadsheets and screenshots. TogetherWeGo puts plan, mone
y, people and paperwork in one place that works offline. | Program addresses the prompt |
| 0:25 | Planning doc (personas, journey, MoSCoW) | How we planned: personas → user journey → feature priorities → navigation map → data
model. | Planning process |
| 0:50 | Sign-in → tap Demo Account → Sign In; then show a validation error on sign-up | Built with Expo SDK 57 / React Native for iOS *
*and** Android. Passwords are salted + hashed. Every form validates format *and* meaning. | Platform · Input validation · Data handling
|
| 1:10 | Home: hero photo, tap the ☀️/🌙 chip, countdown, code pill, weather + local time, budget, Up next | The hero follows the local
time in Positano (sun by day, stars by night). Everything you need before a trip is on one screen. | UX design · Functionality |
| 1:35 | Home → 360° chip → spin around Positano, switch to Paris, ☀️/🌙 toggle (lights come on), tap a landmark chip | Any city can be
explored in 360°; at night the windows, landmarks and cars light up. Drawn in code, so it works offline. | Innovation & creativity · UX
|
| 1:50 | More → Crew & roles → Share code → WhatsApp; change a role; vote in a poll; tick the checklist | Invite codes, roles (Treasure
r, Navigator, Food Lead…), polls and a shared checklist. One tap shares into social apps. | **Social media integration** · Collaboration
|
| 2:20 | Plan: filters (Food, Under 1 hour), expand a card → 🔔; Vote tab → **Lock in** | Time + category filters. Votes show who’s in;
the organizer locks in the winner and the reminder turns on. | Scheduling · Collaboration |
| 3:00 | ➕ Add Activity → submit empty (errors) → fill → overlap warning | Syntactic + semantic validation (it even warns about schedul
e clashes). | Input validation |
| 3:20 | More → Flights & arrivals → tap the time-zone line | Flights are stored as exact instants, so arrivals convert between Rome tim
e and your time. | Scheduling / time zones |
| 3:45 | Budget: donut, Local · EUR, ➕ add a € expense + **receipt photo**, Who owes who → Settle up | Multi-currency expenses, fair sp
lits, fewest payments to settle, receipts attached. | Budgeting |
| 4:30 | Chat → Crew, then **Scout AI** → “How’s the budget?” / “Best food nearby?” | The group chat stays human; Scout answers from our
real trip data — **offline**. Optional Claude mode with a key in the secure keychain. | Innovation & creativity |
| 5:00 | Airplane mode → More → Offline Docs, Trip Map → Offline radar, Emergency | Built for bad wifi: documents, map, phrasebook and e
mergency info all work offline. | Data handling · UX |
| 5:20 | More → Packing list → Split the rest fairly | Everyone sees who is bringing what. | Organization |
| 5:40 | Settings → Language → 日本語; text size; then back to English | 6 languages, adjustable text size, screen-reader labels everywhe
re. | Accessibility |
| 5:55 | Home → “Customize this page” → Make it yours → Sunset/Midnight + interests → Apply | Every user makes the app theirs: themes in
cl. dark mode, interests drive “Picked for you”, choose home sections; animations can be switched off. | Innovation · UX |
| 6:15 | Settings → Demo timeline “In progress” → Home “Today” card; then “Just finished” → Recap story card | The full lifecycle: plann
ing → travelling → recap with stats, balances and a shareable story card. | Addresses all parts of prompt |
| 6:40 | More → How it works; README / ATTRIBUTIONS | Layered MVVM architecture, reusable components, strict TypeScript, 0 lint errors;
every library, API and photo documented. | Code quality · Architecture · Documentation |

**Protocol reminders** (full checklist in `docs/RATING_SHEET.md`): max 3 devices, no external speakers, no power, never ask judges to sc
an a QR code or click a link, leave nothing behind, stay silent during set-up.

## Likely questions — short answers
- **How do friends’ phones stay in sync?** In this version data is stored on each device (offline-first). The store/service split means
adding a sync backend (Firebase/Supabase) only changes the service layer — the screens stay the same.
- **Why Expo / React Native?** One TypeScript codebase for iOS and Android, native modules for notifications, maps, camera, files and se
cure storage.
- **How is data protected?** Salted SHA-256 password hashes, optional session persistence, API key in Keychain/Keystore, files in app-pr
ivate storage.
- **What makes it accessible?** Labels and roles on every control, adjustable text size, colour contrast, haptics toggle, 6 languages, c
lear inline errors.
- **What happens with no internet?** Everything keeps working from local storage; weather and exchange rates show the last saved data (o
r an estimate) and say so.
- **How does the AI work offline?** It’s our own intent-matching engine over the trip data (budget math, settle-up algorithm, weather ca
che, place database). Claude is optional for open-ended questions.
```

docs/PLANNING.md

```
# TogetherWeGo — Planning documents

These are the planning artifacts behind the app (rating sheet: *Planning Process — tangible planning documents*).

## 1. Problem statement
Group trips fall apart in group chats: plans live in five apps, nobody knows who paid for what, flights land at different times in diffe
rent time zones, and the one person with the booking PDF has no signal. **TogetherWeGo puts the whole trip — plan, money, people and pap
erwork — in one shared place that still works offline.**

## 2. Personas
| Persona | Needs | Features that serve them |
|---|---|---|
| **Maya, 17 — the organizer** | Get six friends to agree, keep everyone on schedule | Invite codes, roles, polls & activity voting, liv
e reminders, printable planner |
| **Priya, 17 — the treasurer** | Track a shared budget fairly, settle up without awkwardness | Budget cap, multi-currency expenses, spl
its, fewest-payment settle-up |
| **Diego, 16 — the foodie** | Find great food nearby within budget | Nearby radius + price filter, budget finds, Trip Assistant “best f
ood nearby?” |
| **Mr. Kim — a parent chaperone** | Know where everyone is, keep documents & emergency info handy | Flights & arrivals board, offline d
ocument vault, emergency numbers, location sharing |

## 3. User journey
```mermaid
flowchart LR
  A[Onboarding] --> B[Sign in / Sign up]
  B --> C{Has a trip?}
  C -- no --> D[Create trip<br/>or Join with code]
  C -- yes --> E[Home]
  D --> F[Invite group<br/>share code to socials]
  F --> G[Assign roles]
  G --> H[Build itinerary<br/>vote & confirm]
  H --> I[Add flights<br/>time-zone board]
  I --> J[Budget & expenses]
  J --> K[Travel: reminders,<br/>chat + assistant, map,<br/>photo dump]
  K --> L[Mark complete →<br/>Recap & settle up]
```

4. Navigation map
```mermaid
flowchart TB
  Root[Root Stack<br/>Stack.Protected] --> Auth[Onboarding · Sign in · Sign up · Reset password]
  Root --> Tabs[Bottom tabs]
  Tabs --> T1[Home]
  Tabs --> T2[Plan + Vote]
  Tabs --> T3[Budget]
  Tabs --> T4[Chat]
  Tabs --> T5[More menu]
  Root --> More[Stack screens]
  More --> M1[Destination · All destinations · Nearby · Map]
  More --> M2[Flights · Weather · Currency · Nearest airport]
  More --> M3[Photo dump · Documents · Packing · Bucket list]
  More --> M4[Quiz · Survey · Recap · Emergency · Phrasebook]
  More --> M5[My trips · New/Edit trip · Join · Notifications · Settings · Help · About]
  More --> M6[Make it yours — personalization survey]
  More --> M7[Crew & roles · How it works]
  More --> M8[360° City Views — day / night panoramas]
```

5. Architecture
```mermaid
flowchart LR
  subgraph View[Views — src/app + src/components]
    S[Screens] --> UI[UI kit: Button, Sheet, Chip, Input, Slider…]
  end
  subgraph VM[View-model — selector hooks]
    H[useActiveTrip · useCurrentUser · tripStatus · balances · settlePlan]
  end
  subgraph Model[Model — src/store]
    Z[(Zustand + Immer store)] --> P[(AsyncStorage<br/>offline persistence)]
  end
  subgraph Svc[Services — src/services]
    W[weather] & C[currency] & N[notifications] & AI[assistant] & F[files] & PR[print] & SH[share] & L[location] & SE[secrets]
  end
  S --> H --> Z
  S -- actions --> Z
  S --> Svc
  W --> OM[(Open-Meteo)]
  C --> ER[(ExchangeRate API)]
  AI -. optional .-> CL[(Claude API)]
  SE --> KC[(Keychain / Keystore)]
```

6. Data model
```mermaid
```
```

```
erDiagram
  ACCOUNT ||--o{ MEMBER : "is"
  TRIP ||--|{ MEMBER : has
  TRIP ||--o{ ACTIVITY : schedules
  TRIP ||--o{ EXPENSE : tracks
  TRIP ||--o{ FLIGHT : lists
  TRIP ||--o{ POLL : runs
  TRIP ||--o{ TASK : checklist
  TRIP ||--o{ DOC : vault
  TRIP ||--o{ MESSAGE : chat
  TRIP ||--o{ PHOTO : "photo dump"
  TRIP ||--o{ PACKING_ITEM : packs
  ACTIVITY }o--o{ MEMBER : "assigned / votes"
  EXPENSE }o--|| MEMBER : "paid by"
  EXPENSE }o--|{ MEMBER : "split between"
  FLIGHT }o--|| MEMBER : "belongs to"
```

Key decisions:
- **Dates vs. instants.** Itinerary times are wall-clock times in the destination’s zone (`2026-10-06` + `14:30`), because that is how people plan. Flights are stored as UTC instants so they convert correctly to any time zone.
- **Money in one base currency.** Expenses are stored in USD (with the original amount kept) so totals and splits are always comparable; views convert to home or local currency.
- **Majority voting.** A “voting” activity becomes “confirmed” when more than half of members vote yes.

7. Feature priorities (MoSCoW)

Must	Should	Could	Won’t (this version)
Trips + invite codes, roles, itinerary, voting, budget & splits, chat, flights & time zones, reminders, offline storage, validation	AI assistant, currency converter, weather, documents vault, map, nearby radius & filters, printable planner, social sharing	Quiz, survey recommendations, phrasebook TTS, recap, packing suggestions, emergency screen, 6 languages, personalization (themes, interests, home layout), animations	Cloud sync server, real Google OAuth, payments

8. Accessibility & UX rationale
- Reference-matched visual language: cream canvas, forest-green primary, soft-green highlights, pill navigation — calm and readable outdoors.
- Every icon-only control has an `accessibilityLabel`; toggles expose checked/selected state; sliders are adjustable with screen readers.
- Text size setting (Default / Large / Extra large) on top of the phone’s own font scaling.
- Personalization: 6 colour themes including a dark mode, adjustable card shapes and an *Animations* switch for people sensitive to motion.
- Errors appear inline under the field in red with a plain-language fix (“Codes look like ABC-1234”).
- Destructive actions always confirm; toasts confirm every successful action.

9. Test log (manual + automated browser run)

Area	Checks
Auth	Empty-field errors, wrong password, sign-up validation & strength meter, reset-code flow, old password rejected after reset
Plan	Add-activity validation, overlap warning, vote → auto-confirm, **Lock in** (organizer only), reminder toggle, edit, delete, category + time filters
Budget	Receipt attach, expense validation, who-owes-who, settle up, home/local currency view
360° views	Drag/arrows/auto-rotate wrap past 0°, day/night toggle, landmark jump + facts, city picker, entry points (Home, destination, More)
Extras	Day/night hero toggle, Scout AI tab separate from the crew chat, packing assignment + fair split, first-run tour, story card
Budget	Amount validation, ¥ expense converted to USD, settle-up records payment, budget-cap validation
Collaboration	Join with empty/unknown/valid code, add-member validation (name, email), polls (vote, close), checklist add + empty-task error
Flights	Missing-field errors, unknown IATA rejected, flight saved, time-zone toggle
Personalization	Survey requires ≥1 interest, nickname length check, theme/corner choice survives restart, sign-in kept across the theme reload, Explore sections hide/show, “Picked for you” ranking
Platform	`tsc --noEmit` (strict) ✔, `expo lint` 0 problems ✔, `expo-doctor` 21/21 ✔, iOS + Android bundles compile ✔

docs/RATING_SHEET.md

Rating sheet map: where each point is earned

The FBLA 2026-27 *Mobile Application Development* rating sheet has 16 scored lines (110 points). This page lists every line, what we sho
w the judges, and where to find it. It also lists the honest weak spots.

Design and code quality (25)

Line	Pts	Evidence	Where
Planning process	10	Personas, user journey, navigation map, MoSCoW priorities, data model, manual test log, all in one planning do
cument. Terms used: persona, user journey, MoSCoW, MVP, MVVM, offline-first, one-way data flow.	`docs/PLANNING.md`
Classes, modules, components	5	UI kit of reusable components (`Screen`, `Card`, `Button`, `Press`, `Segmented`, `Donut`), trip wid
gets, one component per file, services for every outside system.	`src/components/ui/`, `src/components/trip/`, `src/services/`
Architectural patterns	5	Layered MVVM: screens (views) read selector hooks and call store actions; services wrap APIs and device f
eatures; one-way data flow; Expo Router file-based navigation with `Stack.Protected` guards; platform-specific files (`TripMap.tsx` / `T
ripMap.web.tsx`).	`src/store/`, `src/app/`, *More → How it works*
Innovation and creativity	5	360° city views with day/night and night lights, skyline hero that follows the city's local time, offl
ine Scout AI answering from trip data, "Lock in" voting, 7-step personalization survey with live themes, offline radar map.	`src/app/p
anorama.tsx`, `src/services/assistant.ts`, `src/app/personalize.tsx`

User experience (20)

Line	Pts	Evidence	Where
UX design: user journey, rationale, accessibility	5	Journey map and rationale in the planning doc. Accessibility: labels and roles
on every control, adjustable text size, high-contrast palette, haptics toggle, "Animations" off switch, 6 languages.	`docs/PLANNING.md
`, *Settings*
Intuitive UI or clear instructions	5	First-run guided tour, a 9-step *How to use* page with an FAQ, inline hints, empty states tha
t say what to do next.	`src/components/GuidedTour.tsx`, `src/app/help.tsx`
Icons and graphics consistent	5	Custom app icon (a take-off plane on a forest-green tile) reused as the in-app logo on sign-in, on
boarding, tour and recap; one icon set and one palette throughout.	`assets/images/icon.png`, `src/components/Logo.tsx`
Input validation (syntax **and** meaning)	5	Syntax: email, invite-code format (`AML-2026`), time `HH:MM`, flight number, IATA cod
e, amount parsing. Meaning: trip end after start, amount between limits, activity time-clash warning, duplicate email is refused, joinin
g a trip you are already in is refused.	`src/utils/validation.ts`

Application functionality (25)

Line	Pts	Evidence	Where
Addresses all parts of the prompt	10	The topic names collaboration, communication, scheduling, budgeting, organization and multi-u
ser trip management. Each is a feature set, and the mapping is explained inside the app.	README section 2, *More → How it works → Rubr
ic map*
Social media integration	5	Direct deep links into WhatsApp, X, Facebook, Messenger, Telegram, Instagram, SMS and email, plus the n
ative share sheet. Photos share as images. Invite codes and the trip recap share straight from the app.	`src/services/share.ts`, *Crew
& roles → Share*, *Recap*
Data handling and storage	5	Zustand store persisted to local storage with versioned migrations (integrity); salted password hashe
s, session off when "Remember me" is off, API key in the Keychain/Keystore (protection); export a trip and reset all data (accessibility
and control); whole app works offline.	`src/store/useAppStore.ts`, `src/services/secrets.ts`, *Settings*
Documentation and copyright	5	README, planning doc, attributions: every library with its licence, every photo with its Unsplash so
urce URL, fonts under OFL, no logos or trademarks.	`README.md`, `ATTRIBUTIONS.md`

Presentation delivery (30)

Line	Pts	Prepared	Where
Organized, clearly stated	10	A timed 7-minute script built around this very sheet, with a speaker split.	`docs/DEMO_SCRIPT.md`
Confidence, body language, eye contact, voice	10	Rehearse with the table below. Judges mark this by watching you, not the app.
n/a
Answering questions	10	Short answers to the likely questions (sync, security, offline, why React Native, accessibility).	`docs/D
EMO_SCRIPT.md`

Presentation protocols (10): the checklist

- [ ] At most **3 devices** (e.g. phone + laptop for slides + a tablet). Screens laptop-sized or smaller.
- [ ] **No external speakers**: audio (phrasebook voice) comes from the phone itself.
- [ ] **No power outlet**: charge everything beforehand.
- [ ] **Links and QR codes**: you may show them, but judges must never scan or click them. Do not make the demo depend on a QR code or l
ink.
- [ ] Nothing left with the judges afterwards. Pack up printouts.
- [ ] No food or animals. Topic stays "Together We Go: Group Trip Planner".
- [ ] Do not talk to the judges during the 3-minute set-up.
- [ ] Every teammate presents a part.
- [ ] Venue wifi may fail: use *Settings → Demo timeline*, airplane-mode offline demo, and the screenshots in `docs/screenshots/` as a b
ackup.

Honest weak spots (so you are not surprised in Q&A)

1. **Standalone run.** The rules say the solution must run standalone. Expo Go needs your laptop's dev server, so build an installed cop
y before the event: `npx eas-cli@latest build -p android --profile preview` gives an `.apk` (needs a free Expo account). iPhone standalo
ne needs an Apple Developer account.
2. **Phones don't sync with each other.** Data lives on each device. The service layer is where a Firebase/Supabase backend would plug i
n without touching the screens. Say this out loud rather than letting a judge find it.
3. **Passwords** use one round of salted SHA-256. A real server should use bcrypt or Argon2, and sign-in should run on the server.
4. **Live weather and exchange rates** need internet. Offline, the app shows the last saved data and labels it.